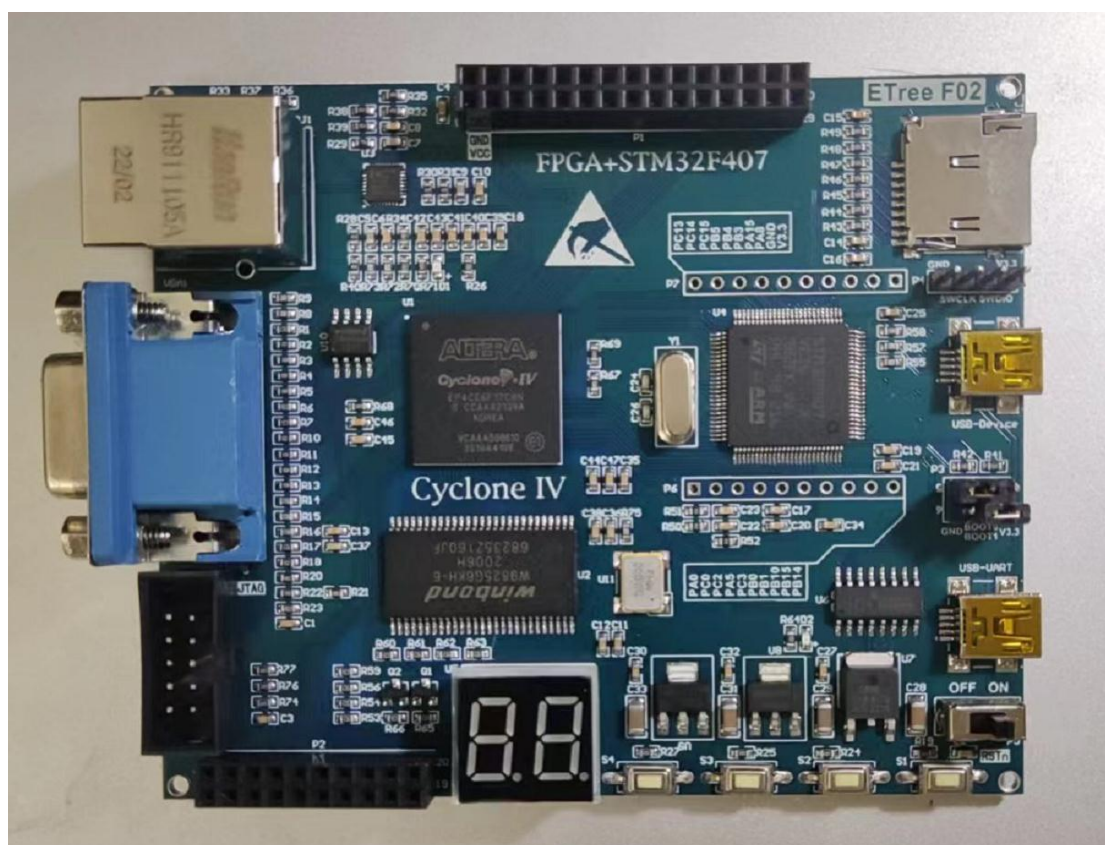


FPGA 实例教程



作者: ETree
日期: 2024-4-6
版本: v2.7

Copyright (C) 2018-20xx ETree Corporation
版权归<益树科技工作室>所有, 转载请注明出处!

版本	内容
V1.0	初始版本, 2018-2-19
V2.0	硬件改版, 增加《STM32F4 联合设计篇》, 2020-10-5
V2.1	更新《STM32F4 联合设计篇》VGA 字符显示章节, 2020-11-8
V2.2	更新《STM32F4 联合设计篇》ADC 章节, 2020-11-23 更新《STM32F4 联合设计篇》内部温度传感器章节, 2020-11-23 更新《STM32F4 联合设计篇》DMA 章节, 2020-11-29
V2.3	更新《STM32F4 联合设计篇》液晶屏显示(2.8 寸)章节, 2021-9-5
V2.4	更新《STM32F4 联合设计篇》SDRAM 读写章节, 2021-9-21 更新《STM32F4 联合设计篇》内存管理章节, 2021-9-21 更新《STM32F4 联合设计篇》FATFS 文件系统章节, 2021-9-21
V2.5	更新《STM32F4 联合设计篇》触摸屏章节, 2021-10-30
V2.6	更新《STM32F4 联合设计篇》TF 卡图片显示章节, 2021-11-20 更新《STM32F4 联合设计篇》摄像头液晶显示章节, 2021-11-21
V2.7	增加《Verilog 数字系统篇》“HDMI 彩条显示”章节, 2024.4.6

注意:《STM32F4 联合设计篇》软件设计源码部分来自于网络, 仅用于个人学习, 请勿用于其它商业用途!

目录

FPGA 实例教程	1
Verilog HDL 数字系统设计篇	12
第一章 板卡介绍.....	13
1.1. FPGA 简介	13
1.2. 板卡资源.....	14
1.3. 板卡供电.....	15
1.4. 下载调试.....	15
第二章 软件安装.....	16
2.1. Quartus II&Modelsim-Altera	16
2.2. USB-Blaster 驱动安装.....	22
第三章 组合逻辑门建模.....	25
3.1. 组合逻辑电路的门级建模.....	25
3.2. 多输入门.....	25
3.3. 多输出门.....	25
3.4. 三态门.....	26
第四章 2 线--4 线译码器	27
4.1. Verilog 代码结构	27
4.2. 门级建模.....	27
4.3. 数据流建模.....	28
第五章 D 锁存器/触发器与时序逻辑电路.....	29
5.1. D 锁存器/触发器.....	29
5.2. 时序逻辑电路(10 进制计数器)	29
第六章 LED 闪烁.....	31
6.1. 硬件介绍.....	31
6.2. 新建工程.....	31
6.3. HDL 设计	34
6.4. 设置 Dual-Purpose Pins	36
6.5. 编译工程与管脚约束.....	36
6.6. 下载 sof 文件.....	37
6.7. jic 文件生成	38
6.8. 下载 jic 文件	39
第七章 关联 Modelsim SE 仿真.....	41
7.1. 关联 Modelsim SE 软件	41
7.2. 测试脚本组成.....	42
7.3. 生成测试脚本.....	43
7.4. 开始仿真.....	45
第八章 按键检测.....	46
8.1. 硬件介绍.....	46
8.2. HDL 设计	46
8.3. 管脚分配.....	48
8.4. 实验操作.....	48
第九章 数码管秒表.....	49

9.1.	硬件介绍.....	49
9.2.	HDL 设计	50
➤	系统框架设计.....	50
➤	秒表进位计数.....	51
9.3.	管脚分配.....	52
9.4.	实验操作.....	52
第十章	UART 串口通信	53
10.1.	硬件介绍.....	53
10.2.	UART 通信协议.....	54
10.3.	HDL 设计	54
➤	系统框架.....	54
➤	串口接收模块设计.....	55
➤	串口发送模块设计.....	58
10.4.	管脚约束.....	60
10.5.	SignalTap 使用	60
➤	打开 SignalTap II Logic Analyzer.....	60
➤	选择 JTAG Chain Configuration	61
➤	设置采样时钟和采样深度.....	61
➤	添加被观察信号.....	62
➤	保存 SinalTap II 文件	63
➤	选择信号的触发条件.....	64
➤	编译工程.....	65
➤	运行 SignalTap II.....	65
10.6.	实验操作.....	65
➤	安装 USB 转串口驱动	65
➤	数据发送与接收.....	66
第十一章	MegaWizard	69
11.1.	PLL 设置	69
11.2.	PLL 例化	73
11.3.	FIFO 设置	73
11.4.	FIFO 例化	78
11.5.	测试激励.....	79
11.6.	仿真结果.....	81
第十二章	以太网环回测试.....	85
12.1.	硬件介绍.....	85
12.2.	RMII	86
12.3.	TCP/IP 协议栈简介	87
12.4.	UDP 协议.....	88
12.5.	IP 协议.....	88
12.6.	PLL 时钟生成	90
12.7.	HDL 设计	93
➤	工程目录.....	93
➤	UDP 发送与接受模块设计.....	94
➤	CRC 发送模块设计	94

12.8.	管脚约束.....	97
12.9.	实验操作.....	97
第十三章	MT9V011 摄像头以太网显示	100
11.1.	硬件介绍.....	100
11.2.	CMOS 工作原理.....	101
11.3.	CMOS 配置接口.....	103
11.4.	HDL 设计	103
➤	系统框架.....	103
➤	I2C 配置模块.....	103
➤	UDP 发送模块.....	105
➤	FIFO 模块	105
11.5.	管脚分配.....	106
11.6.	实验操作.....	107
第十四章	数字频率计.....	110
14.1.	硬件介绍.....	110
14.2.	HDL 设计	110
14.3.	管脚分配.....	114
14.4.	实验操作.....	115
第十五章	上位机调试 CMOS 摄像头参数.....	117
15.1.	硬件介绍.....	117
15.2.	HDL 设计	117
➤	系统框架.....	117
➤	串口接收模块设计.....	117
15.3.	管脚分配.....	121
15.4.	实验操作.....	122
第十六章	MT9M001 摄像头以太网显示	129
16.1.	硬件介绍.....	129
16.2.	HDL 设计	130
➤	系统框架.....	130
➤	摄像头芯片配置(I2C)	130
16.3.	管脚分配.....	132
16.4.	实验操作.....	132
第十七章	VGA 彩条显示.....	135
17.1.	硬件介绍.....	135
17.2.	VGA 时序.....	136
17.3.	HDL 设计	136
➤	工程目录.....	136
➤	VGA 模块.....	137
17.4.	管脚分配.....	140
17.5.	实验操作.....	140
第十八章	MT9V011 摄像头 VGA 显示	142
18.1.	硬件介绍.....	142
18.2.	SDRAM 结构	143
18.3.	SDRAM 操作	144

➤	SDRAM 初始化.....	144
➤	SDRAM 写操作.....	145
➤	SDRAM 读操作.....	146
➤	SDRAM 自刷新操作.....	147
18.4.	VGA 时序.....	148
18.5.	HDL 设计	148
➤	系统框架.....	148
➤	VGA 驱动.....	149
➤	SDRAM 驱动	153
18.6.	管脚分配.....	153
18.7.	实验操作.....	154
第十九章	液晶屏彩条显示(2.8 寸)	157
19.1.	硬件介绍.....	157
19.2.	TFT-LCD 简介	158
19.3.	HDL 设计	160
19.4.	管脚分配.....	163
19.5.	实验操作.....	164
第二十章	MT9V011 液晶屏显示[2.8 寸].....	166
20.1.	硬件介绍.....	166
20.2.	MCU 并口.....	167
20.3.	HDL 设计	168
➤	系统框架.....	168
➤	顶层模块.....	168
20.4.	管脚定义.....	174
20.5.	实验操作.....	174
第二十一章	HDMI 彩条显示	176
21.1.	硬件设计	176
21.2.	HDL 设计	178
21.3.	管脚分配.....	180
21.4.	实验操作.....	180
Verilog 图像处理篇		183
第一章	单色通道提取.....	184
1.1.	Bayer 阵列介绍.....	184
1.2.	算法原理.....	184
1.3.	算法实现.....	185
1.4.	实验效果.....	186
第二章	亮度增强.....	189
2.1.	算法原理.....	189
2.2.	算法实现.....	189
2.3.	实验效果.....	189
第三章	对比度增强.....	192
3.1.	算法原理.....	192
3.2.	算法实现.....	192
3.3.	实验效果.....	192

第四章	图像反转.....	195
4.1.	算法原理.....	195
4.2.	算法实现.....	195
4.3.	实验效果.....	195
第五章	均值滤波.....	197
5.1.	算法原理.....	197
5.2.	算法实现.....	197
5.3.	实验效果.....	200
第六章	高斯滤波.....	203
6.1.	算法原理.....	203
6.2.	算法实现.....	203
6.3.	实验效果.....	204
第七章	图像锐化.....	207
7.1.	算法原理.....	207
7.2.	算法实现.....	207
7.3.	实验效果.....	208
第八章	中值滤波.....	210
8.1.	算法原理.....	210
8.2.	算法实现.....	210
8.3.	实验效果.....	211
第九章	阈值分割.....	214
9.1.	算法原理.....	214
9.2.	算法实现.....	214
9.3.	实验效果.....	214
第十章	边缘检测(Sobel)	217
10.1.	算法原理.....	217
10.2.	算法实现.....	217
10.3.	实验效果.....	219
第十一章	图像膨胀.....	221
11.1.	算法原理.....	221
11.2.	算法实现.....	221
11.3.	实验效果.....	222
第十二章	图像腐蚀.....	224
12.1.	算法原理.....	224
12.2.	算法实现.....	224
12.3.	实验效果.....	225
第十三章	开运算.....	227
13.1.	算法原理.....	227
13.2.	算法实现.....	227
13.3.	实验效果.....	228
第十四章	闭运算.....	230
14.1.	算法原理.....	230
14.2.	算法实现.....	230
14.3.	实验效果.....	231

第十五章	Tophat 形态学滤波.....	233
15.1.	算法原理.....	233
15.2.	算法实现.....	233
15.3.	实验效果.....	234
第十六章	RAW8 转 RGB888.....	237
16.1.	算法原理.....	237
16.2.	算法实现.....	238
16.3.	实验效果.....	240
第十七章	Alpha 背景叠加	241
17.1.	算法原理.....	241
17.2.	算法实现.....	241
17.3.	实现效果.....	241
第十八章	直方图灰度拉伸算法.....	243
18.1.	算法原理.....	243
18.2.	算法实现.....	243
18.3.	实验效果.....	245
第十九章	自动白平衡算法.....	247
19.1.	算法原理.....	247
19.2.	算法实现.....	247
19.3.	实验效果.....	249
STM32F4 设计篇		252
第一章	STM32F4 硬件平台介绍.....	253
1.1.	硬件原理框图.....	253
1.2.	STM32F4 启动模式.....	253
第二章	MDK5 软件	254
2.1.	STM32F4 官方固件库.....	254
2.2.	MDK5 简介	254
2.3.	程序下载.....	255
2.4.	定位函数/变量被定义的地方	258
2.5.	快速注释与取消注释.....	258
第三章	STM32F4 基础知识介绍.....	260
3.1.	STM32F4 总线架构.....	260
3.2.	STM32F4 时钟系统.....	261
第四章	LED 闪烁.....	262
4.1.	硬件设计	262
4.2.	HDL 设计	262
4.3.	STM32F4 IO 简介	263
4.4.	软件设计	263
4.5.	实验操作.....	265
第五章	按键检测.....	266
5.1.	硬件设计	266
5.2.	HDL 设计	266
5.3.	软件设计	267
5.4.	实验操作.....	268

第六章	外部中断.....	269
6.1.	硬件设计.....	269
6.2.	STM32F4 外部中断简介.....	269
6.3.	HDL 设计.....	269
6.4.	软件设计.....	270
6.5.	实验操作.....	271
第七章	串口通信.....	272
7.1.	硬件设计.....	272
7.2.	STM32 串口简介.....	272
7.3.	HDL 设计.....	273
7.4.	软件设计.....	274
7.5.	实验操作.....	276
	➤ 安装 USB 转串口驱动.....	276
	➤ 数据发送与接收.....	277
第八章	定时器.....	278
8.1.	硬件设计.....	278
8.2.	STM32F4 通用定时器.....	278
8.3.	HDL 设计.....	278
8.4.	软件设计.....	279
8.5.	实验操作.....	280
第九章	FSMC 通信.....	281
9.1.	硬件设计.....	281
9.2.	FSMC.....	282
9.3.	HDL 设计.....	285
9.4.	软件设计.....	288
9.5.	实验操作.....	291
第十章	VGA 字符显示.....	293
10.1.	硬件设计.....	293
10.2.	HDL 设计.....	294
10.3.	软件设计.....	294
10.4.	实验操作.....	295
第十一章	ADC.....	297
11.1.	硬件设计.....	297
11.2.	STM32F4 ADC.....	297
11.3.	HDL 设计.....	299
11.4.	软件设计.....	299
11.5.	实验操作.....	300
第十二章	内部温度传感器.....	301
12.1.	硬件设计.....	301
12.2.	STM32F4 内部温度传感器.....	301
12.3.	HDL 设计.....	301
12.4.	软件设计.....	301
12.5.	实验操作.....	302
第十三章	DMA.....	303

13.1.	硬件设计	303
13.2.	STM32F4 DMA.....	303
13.3.	HDL 设计	304
13.4.	软件设计	305
13.5.	实验操作	306
第十四章	液晶屏显示(2.8 寸)	308
14.1.	硬件设计	308
14.2.	MCU 并口.....	309
14.3.	HDL 设计	310
14.4.	软件设计	310
14.5.	实验操作	310
第十五章	SDRAM 读写	312
15.1.	硬件设计	312
15.2.	Avalon MM 总线.....	312
15.3.	Qsys 搭建	312
15.4.	HDL 设计	316
15.5.	软件设计	319
15.6.	实验操作	321
第十六章	内存管理.....	324
15.1.	硬件设计	324
15.2.	内存管理简介	324
15.3.	HDL 设计	325
15.4.	软件设计	325
15.5.	实验操作	326
第十七章	TF-SD 卡	328
16.1.	硬件设计	328
16.2.	HDL 设计	329
16.3.	软件设计	330
16.4.	实验操作	332
第十八章	FATFS 文件系统.....	334
18.1.	硬件设计	334
18.2.	FATFS 简介	334
18.3.	HDL 设计	337
18.4.	软件设计	337
18.5.	实验操作	339
第十九章	TF 卡图片显示	341
19.1.	图片格式简介	341
19.2.	硬件设计	341
19.3.	HDL 设计	341
19.4.	软件设计	348
19.5.	实验操作	352
第二十章	摄像头液晶显示	354
20.1.	DCMI 简介	354
20.2.	硬件设计	355

20.3.	HDL 设计	356
20.4.	软件设计	356
20.5.	实验操作	356
第二十一章	触控屏	358
21.1.	电阻屏简介	358
21.2.	硬件设计	358
21.3.	HDL 设计	359
21.4.	软件设计	359
21.5.	实验操作	360
第二十二章	USB 读卡器	362
19.1.	硬件设计	362
19.2.	USB	362
19.3.	HDL 设计	363
19.4.	软件设计	363
19.5.	实验操作	363
第二十三章	LWIP 移植	365
23.1.	STM32F4 以太网简介	365
23.2.	硬件设计	365
23.3.	HDL 设计	365
23.4.	软件设计	366
23.5.	实验操作	366
第二十四章	UDP 通信	369
24.1.	硬件设计	369
24.2.	HDL 设计	369
24.3.	软件设计	369
24.4.	实验操作	369
END		369

Verilog HDL 数字系统 设计篇

第一章 板卡介绍

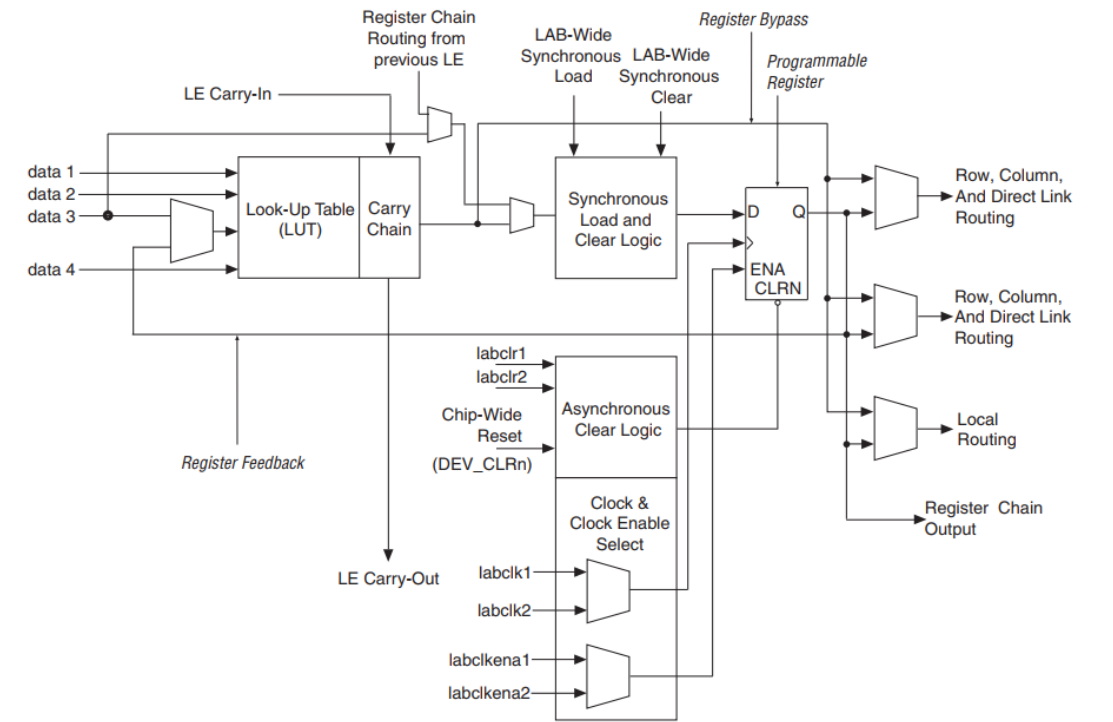
1.1. FPGA 简介

FPGA 全称: Field Programmable Gate Array, 译名为现场可编辑逻辑门阵列。简化的 FPGA 由输入/输出单元、基本可编程逻辑单元、嵌入式块 RAM、布线资源、DSP 乘法器和专用硬核等组成。下表为 Intel FPGA 的 Cyclone IV E 系列资源统计:

Resources	EP4CE6	EP4CE10	EP4CE15	EP4CE22	EP4CE30	EP4CE40	EP4CE55	EP4CE75	EP4CE115
Logic elements (LEs)	6,272	10,320	15,408	22,320	28,848	39,600	55,856	75,408	114,480
Embedded memory (Kbits)	270	414	504	594	594	1,134	2,340	2,745	3,888
Embedded 18 × 18 multipliers	15	23	56	66	66	116	154	200	266
General-purpose PLLs	2	2	4	4	4	4	4	4	4
Global Clock Networks	10	10	20	20	20	20	20	20	20
User I/O Banks	8	8	8	8	8	8	8	8	8
Maximum user I/O (f)	179	179	343	153	532	532	374	426	528

从上表可以看到 Cyclone IV E 系列 FPGA 主要通过 Logic elements(LEs)的数量来区分不同的芯片名。下面我们重点介绍 Logic elements 的结构。

下图为 Logic elements 的结构图, 主要包含四输入的 LUT 查找表、CarryChain 进位链和 D 触发器。LUT 查找表用于实现组合逻辑电路, 进位链用于加法进位运算, D 触发器用于实现时序逻辑电路。关于组合逻辑电路和时序逻辑电路的具体实现, 会在下面章节介绍。

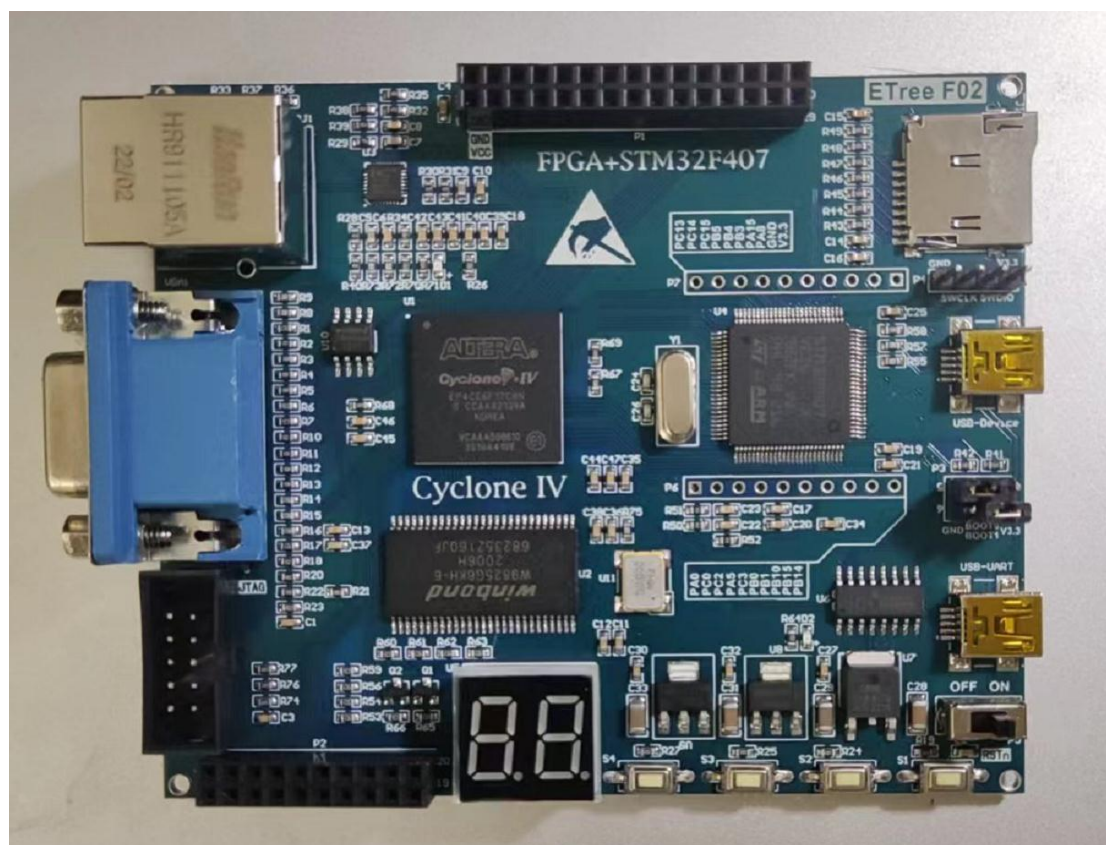


1.2. 板卡资源

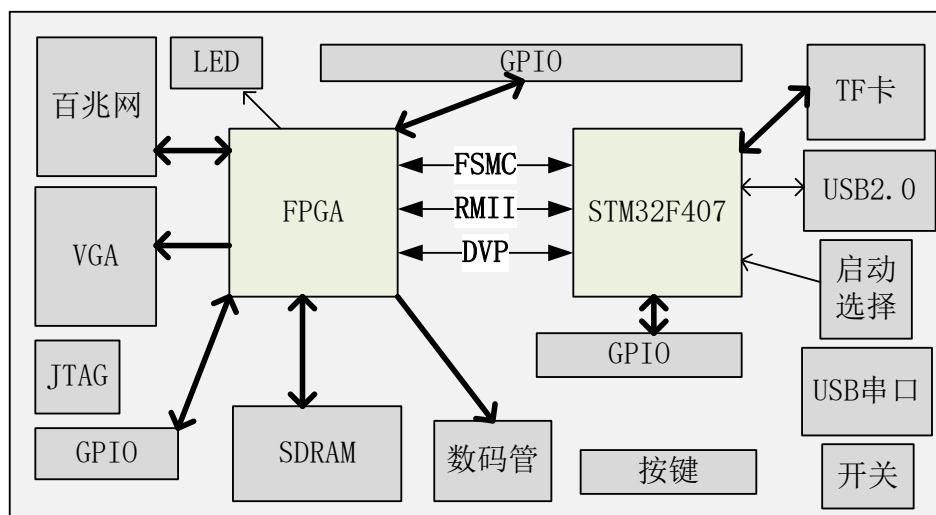
FPGA 开发板的硬件资源资源，具体如所示：

- ✧ FPGA: EP4CE6F17C8N, 6K 逻辑资源;
- ✧ SDRAM: 内存大小为 $4 \times 8M \times 8\text{Bit}$, FPGA 侧控制;
- ✧ VGA: RGB565, 电阻分压, 一共可以显示 65536 种颜色, FPGA 侧控制;
- ✧ 百兆网卡: IP101GR, RMII 接口, FPGA 侧控制;
- ✧ 数码管: 2 位共阳极数码管, FPGA 侧控制;
- ✧ ARM: STM32F407VET6, Cortex M4 内核, 主频 168MHz;
- ✧ USB2.0: STM32F407 侧 USB2.0 全速接口, 可给板卡供电;
- ✧ USB 串口: CH340C, FPGA 侧与 ARM 侧同时控制(只能有一个主控), 可给板卡供电;
- ✧ 独立按键: 4 个独立按键, 一个用于 FPGA 和 ARM 的系统复位, 另外三个 FPGA 侧控制;
- ✧ LED: 一盏 LED 发光二极管, FPGA 侧控制;
- ✧ TF 卡槽: ARM 侧控制;
- ✧ GPIO: 可外接摄像头、液晶屏等外部模块;
- ✧ JTAG: 可以用 USB-Blaster 下载器调试和下载 FPGA;
- ✧ SWD: 可使用 jlink ob 调试和下载 ARM。

下图为板卡实物图:



下图为板卡结构示意图:



FPGA 板卡可以在 <https://shop397545458.taobao.com> 购买，期待朋友们的加入。

1.3. 板卡供电

FPGA 板卡可以通过 USB 连接 PC 的 USB 口来供电。

(注意 USB-Blaster 的 JTAG 下载口不能给板卡供电)

1.4. 下载调试

FPGA 可以通过 USB Blaster 下载器下载并调试，如下图所示：



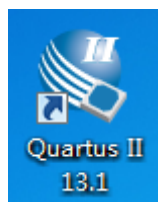
第二章 软件安装

2.1. Quartus II&Modelsim-Altera

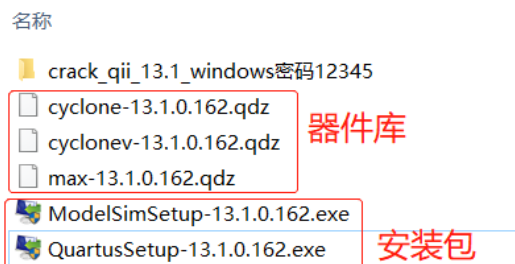
Quartus II 是 Intel 公司的一种可编程的设计环境，由于其功能强大和直观易用，越来越受到数字系统设计者的欢迎。

Modelsim-Altera 是 Intel 公司提供的 EDA 仿真软件，支持 VHDL 和 Verilog 混合仿真。

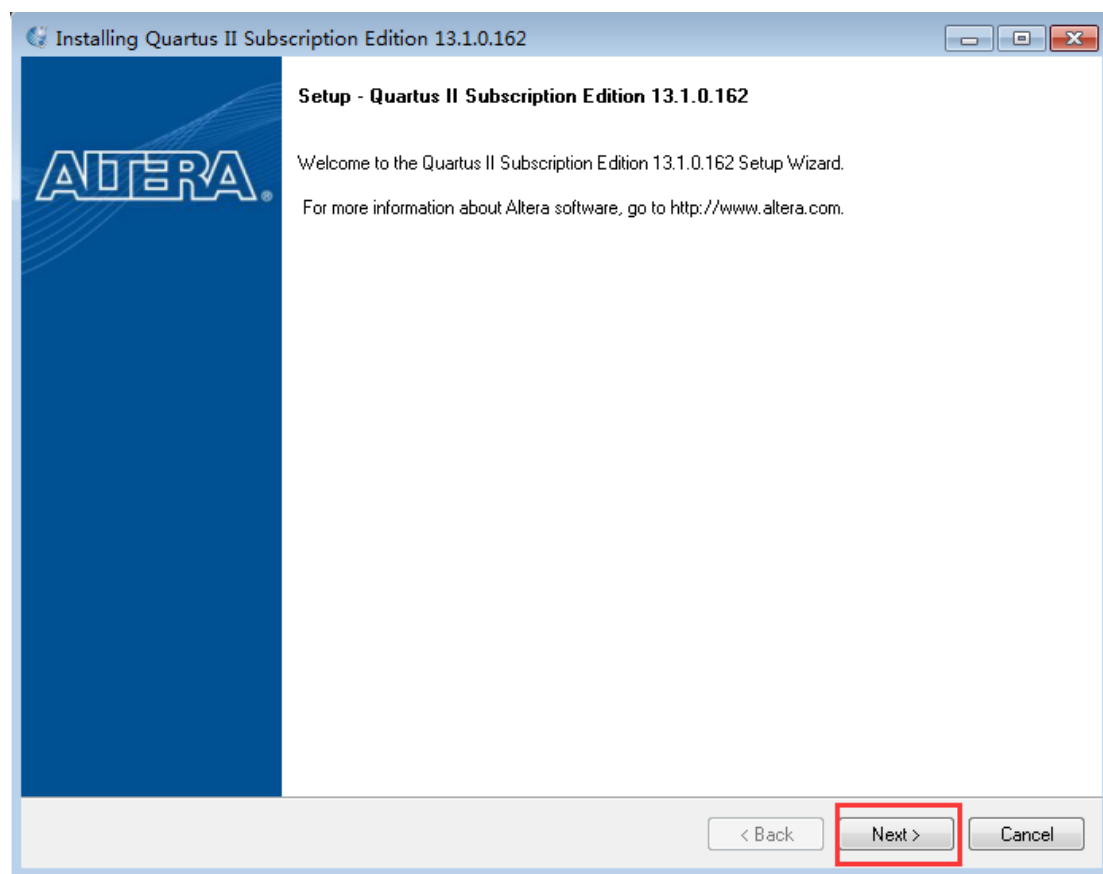
这里我们使用的 Quartus II 版本是 13.1.0.162，使用的 Modelsim-Altera 版本也是 13.1.0.162，具体安装流程可以参考我们录制的视频演示。



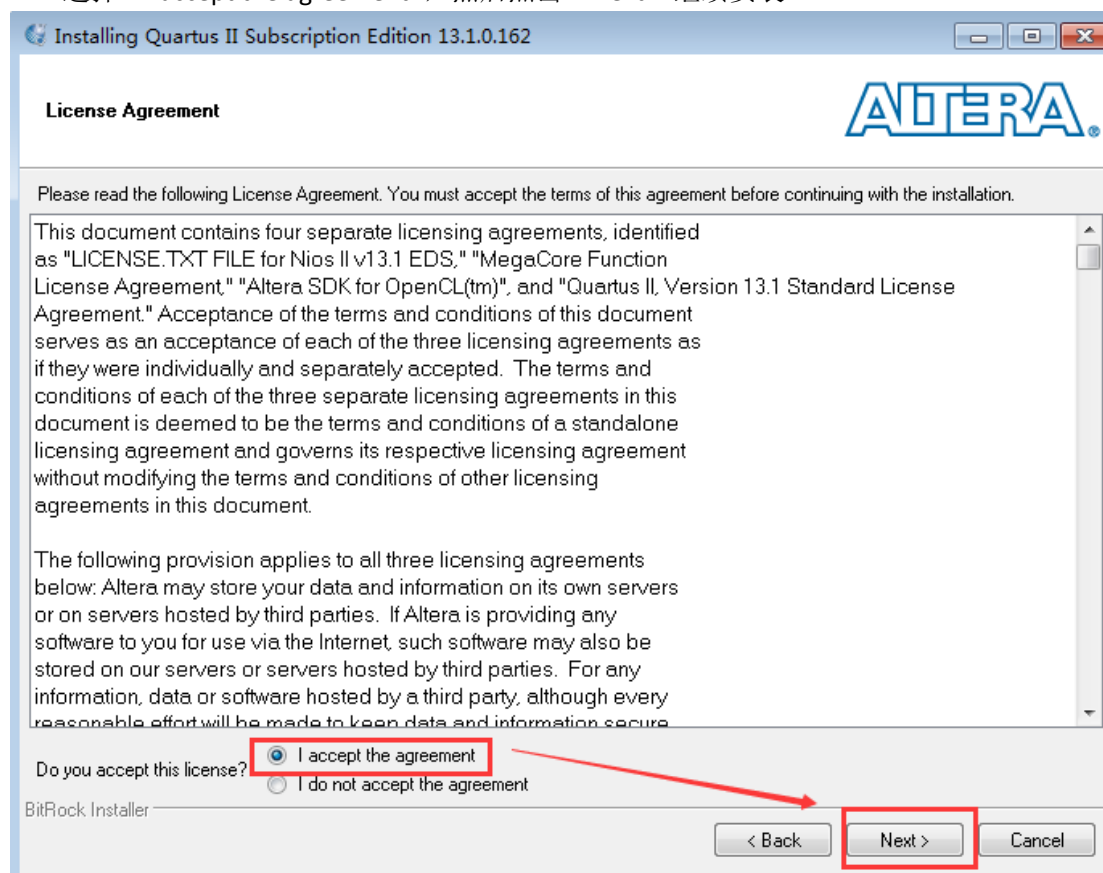
打开“开发软件”目录下的“QuartusSetup-13.1.0.162.exe”安装包文件。



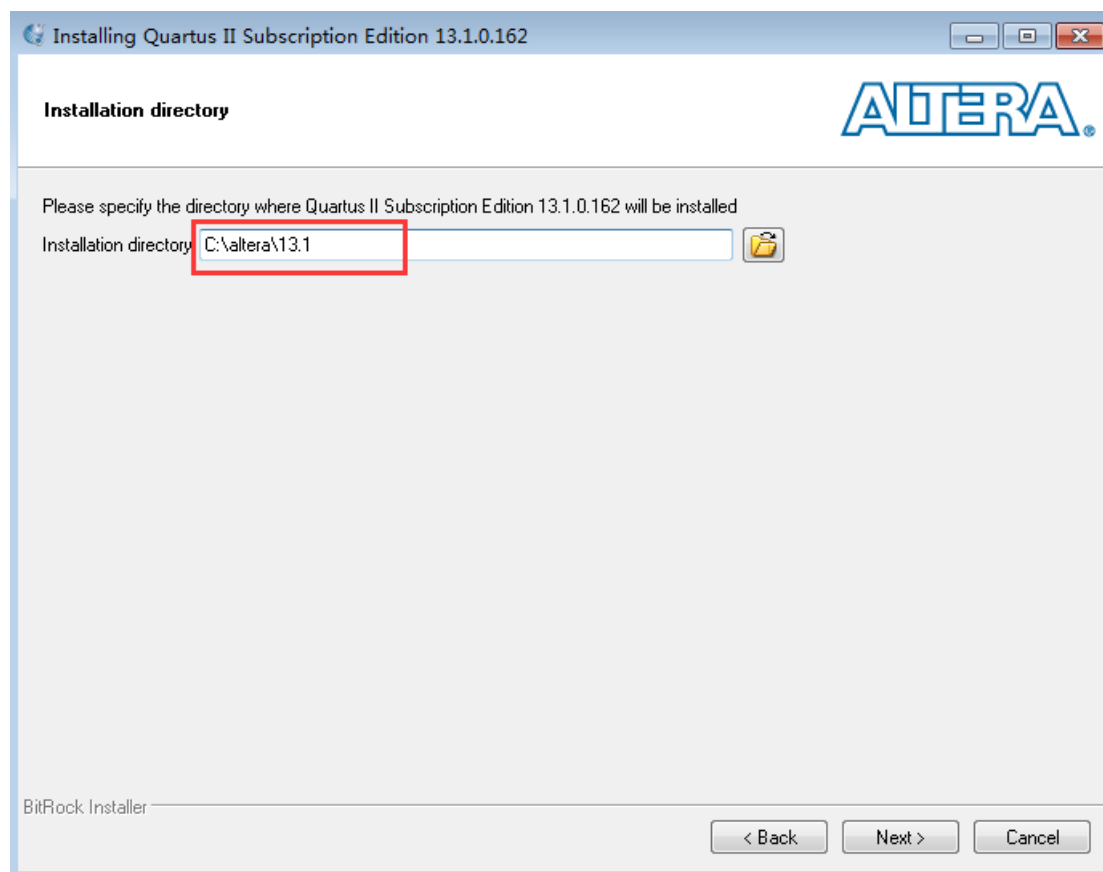
弹出安装对话框，点击“Next”继续安装。



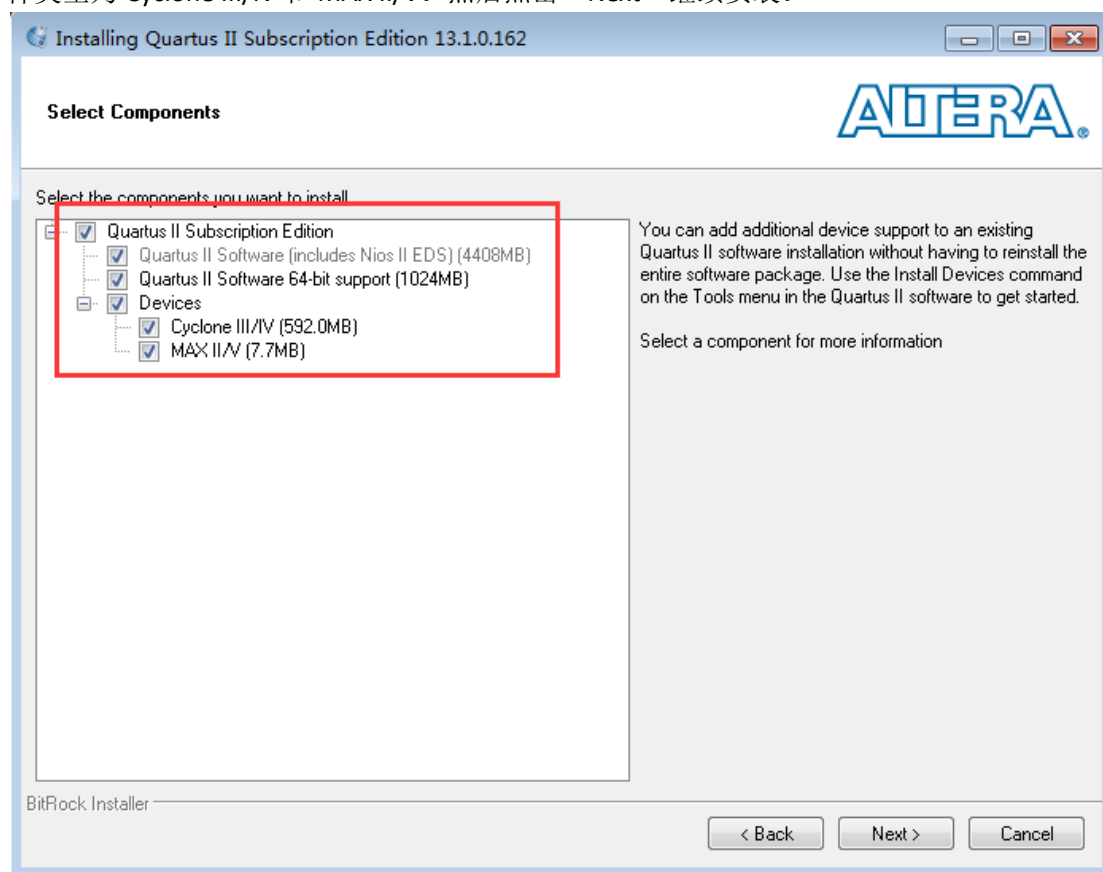
选择 “I accept the agreement”，然后点击 “Next” 继续安装。



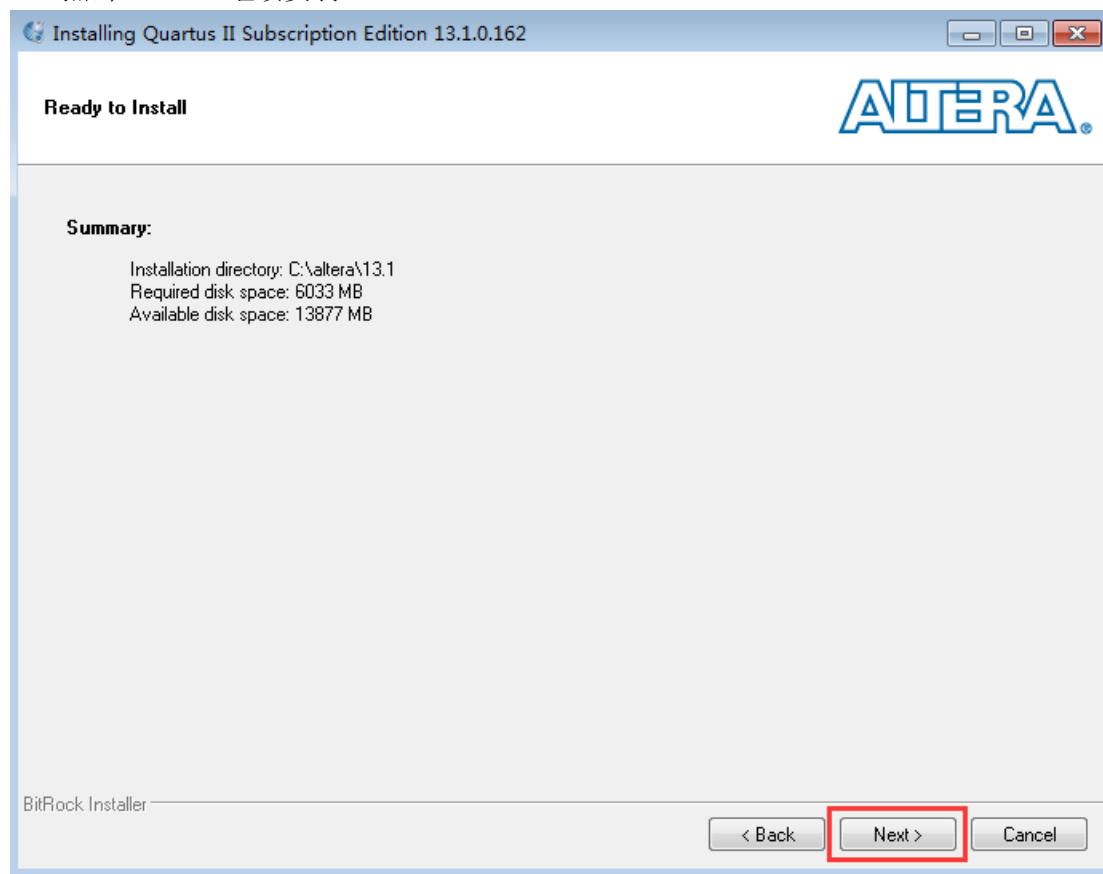
选择安装目录，默认安装在 C 盘根目录下，然后点击 “Next” 继续安装。



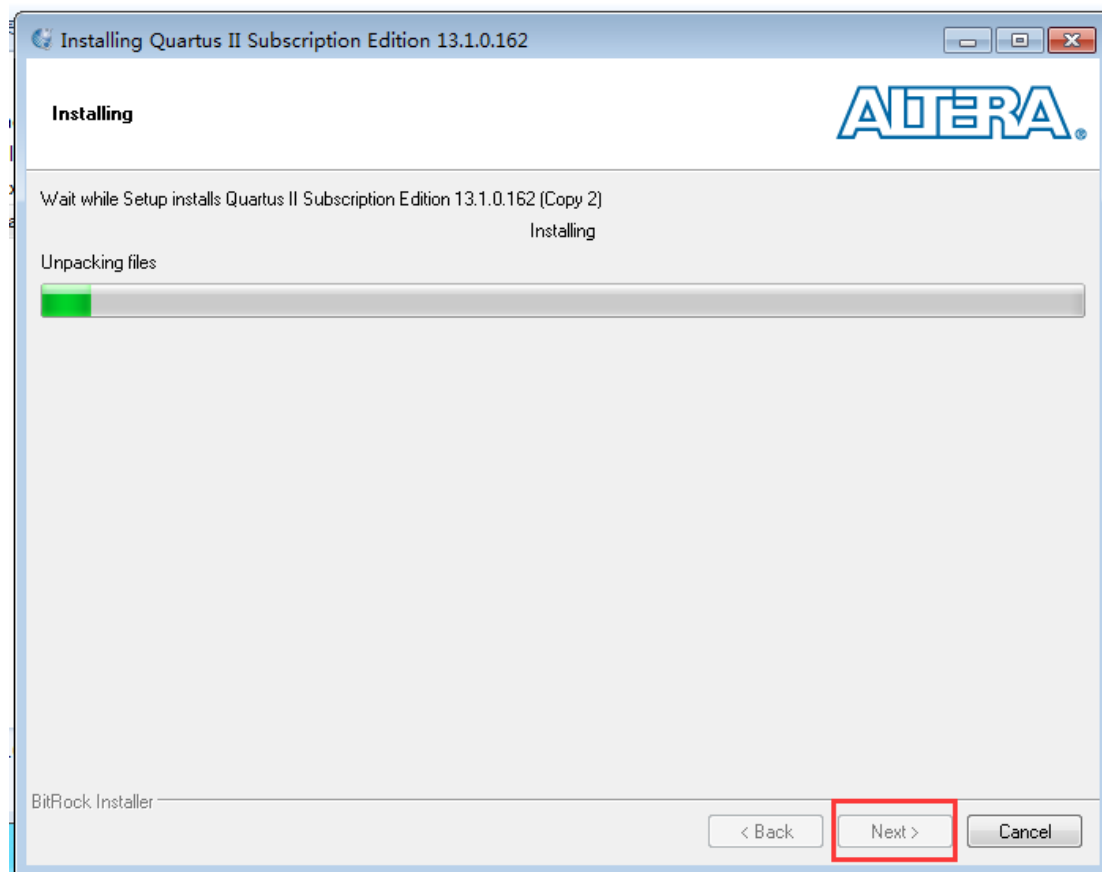
选择安装 Quartus II 类型，默认 32 位必须安装，我们再勾选 64 位版本。选择支持的器件类型为 Cyclone III/IV 和 MAX II/V。然后点击“Next”继续安装。



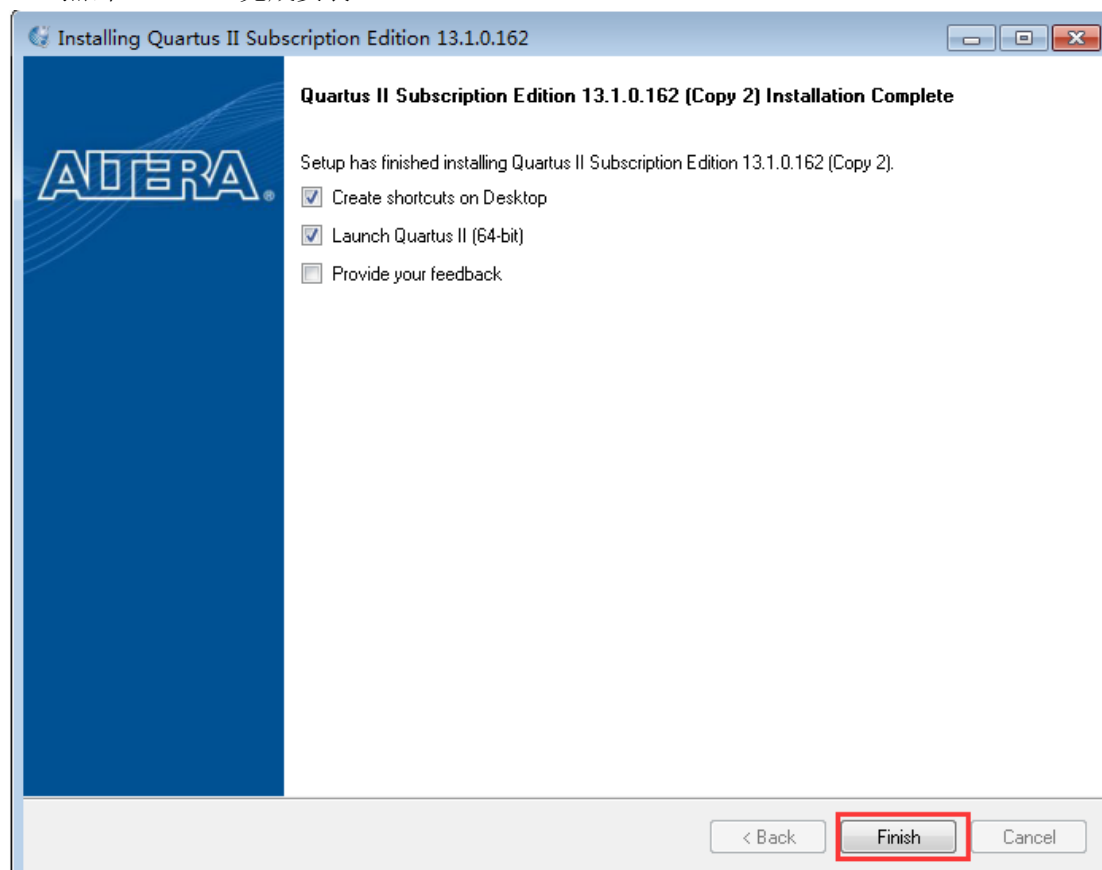
点击“Next”继续安装。



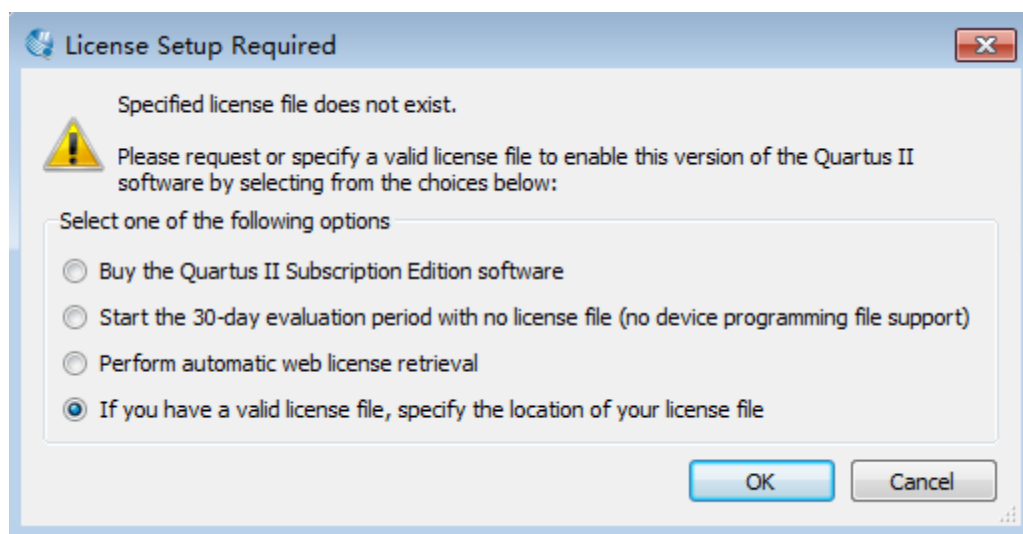
下图表示正在安装，时间会稍微长一点，当绿色进度为 100%时，点击“Next”继续安装。



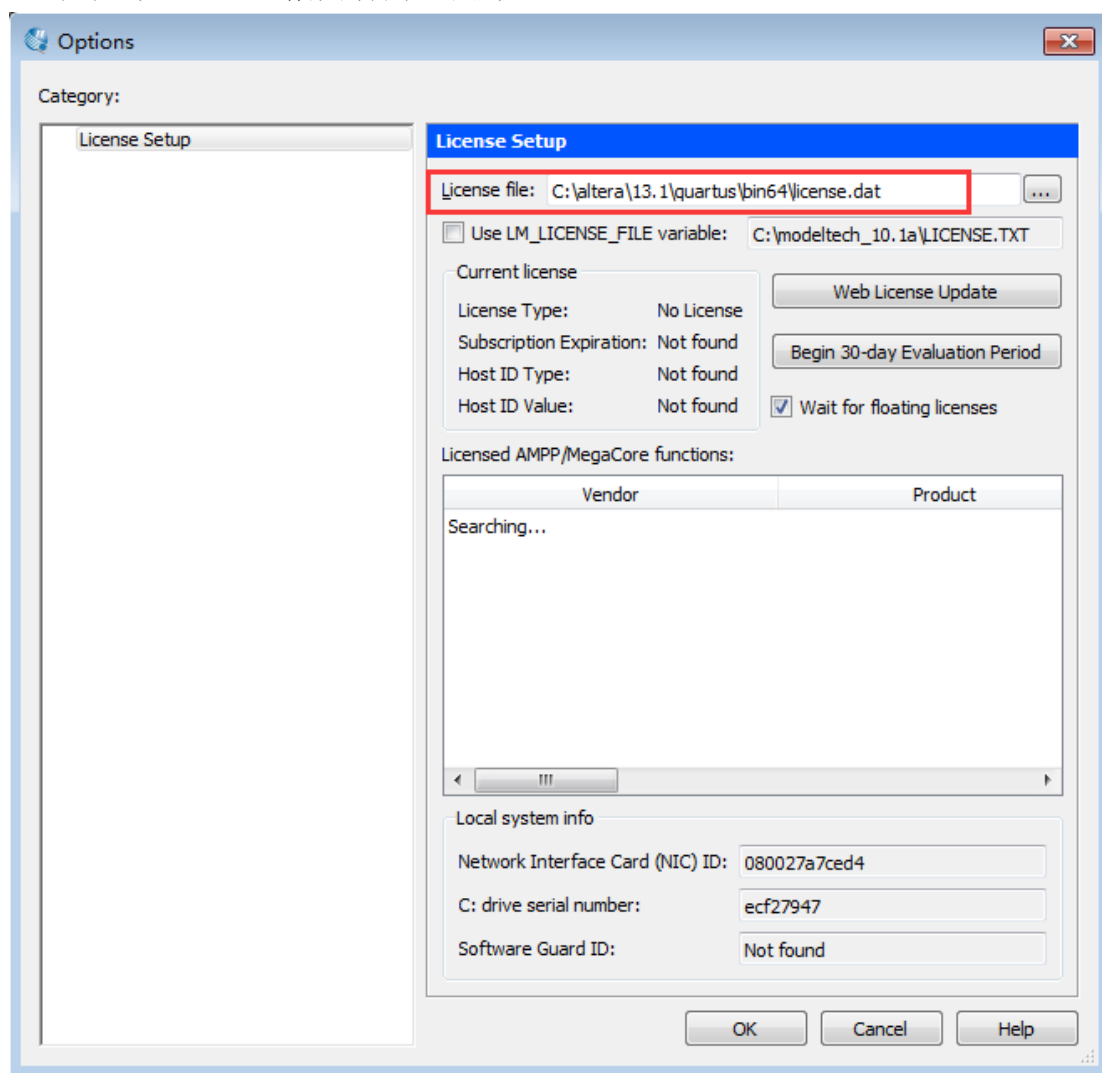
点击“Finish”完成安装。



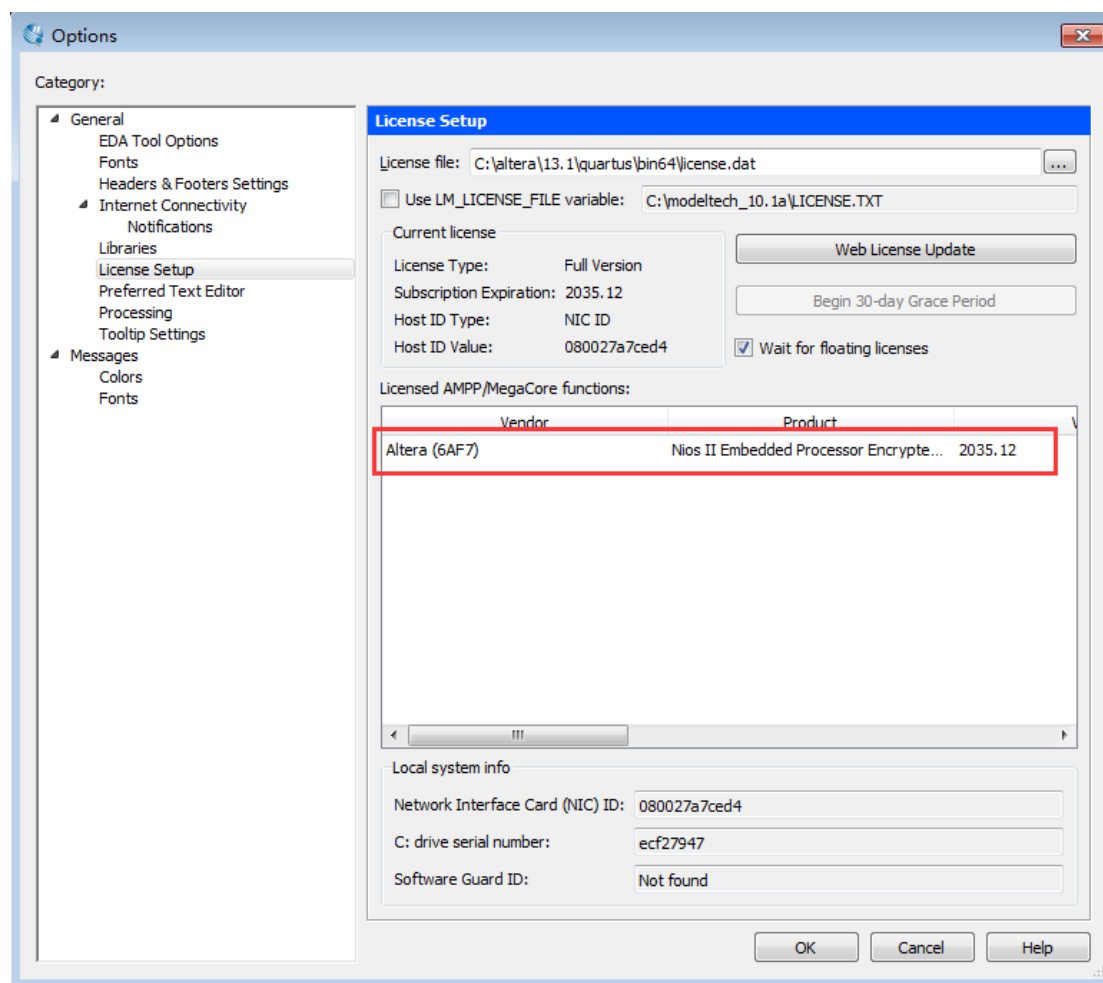
在弹出的对话框中选择 “If you have a valid license file, specify the location of your license file”，点击 OK。



定位到 license.dat 存放的目录，点击 OK。



在 Options | General | license Setup 中显示破解成功到 2035 年 12 月，如下图所示。



注意：具体的破解步骤可以参考破解文件中的“读我.txt”，或者参看视频演示。

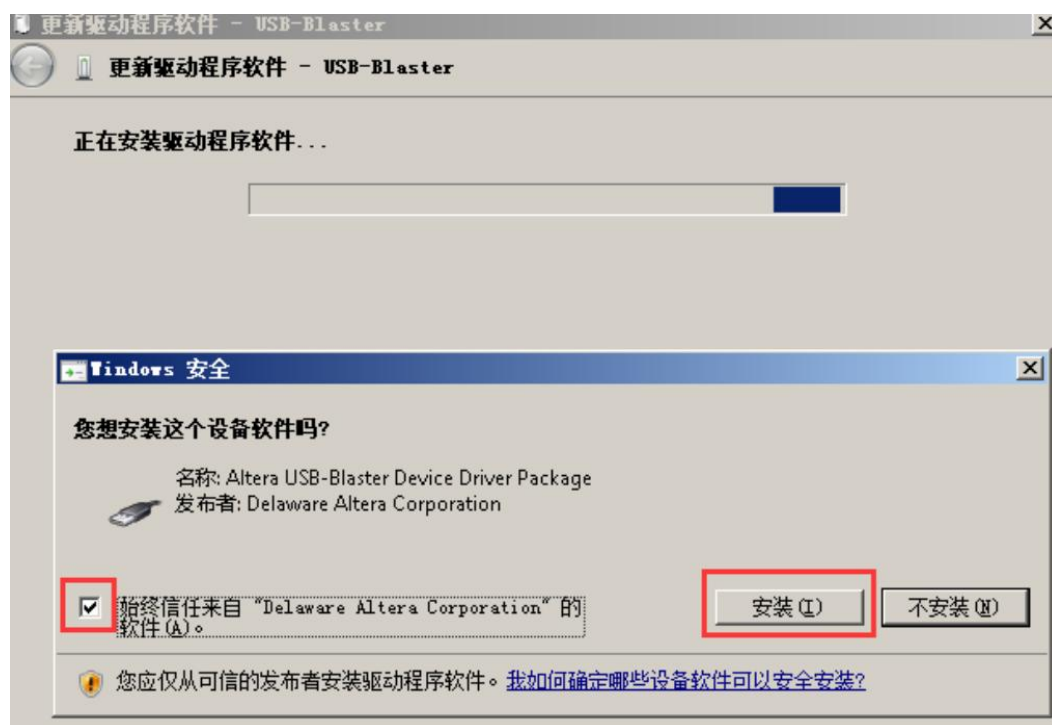
名称

Quartus_II_13.1_x64破解器.exe
读我.txt

2.2. USB-Blaster 驱动安装

使用 Quartus II 软件对开发板下载和调试前，需要安装 USB Blaster 驱动程序。驱动程序在资料盘“Tools\usb-blaster”，**注意不要再往下选择**。

在设备管理器中找到对应的位置，选择“下一步”，在弹出的安全对话框中选择“始终信任”，再点击“安装”按钮。



驱动安装完成，点击"关闭"。



在设备管理器中显示安装好的 USB-Blaster 设备。



第三章 组合逻辑门建模

本章介绍 verilog 的组合逻辑门建模基础知识。

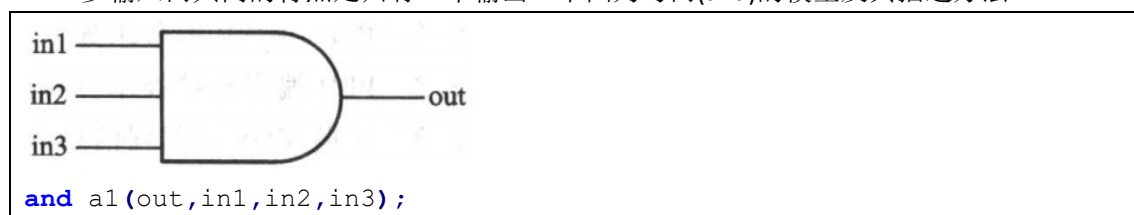
3.1. 组合逻辑电路的门级建模

门级建模就是将逻辑电路图用 verilog HDL 规定的语言描述出来。Verilog 语言中内置 12 个基本门级模型，如下表所示。

元件符号	功能
and	多输入端与门
or	多输入端或门
xor	多输入端异或门
buf	多输出端缓冲器
nand	多输入端与非门
nor	多输入端或非门
xnor	多输入端异或非门
not	多输出端反相器
bufif1	控制信号高电平有效三态缓冲器
bufif0	控制信号低电平有效三态缓冲器
notif1	控制信号高电平有效三态反相器
notif0	控制信号低电平有效三态反相器

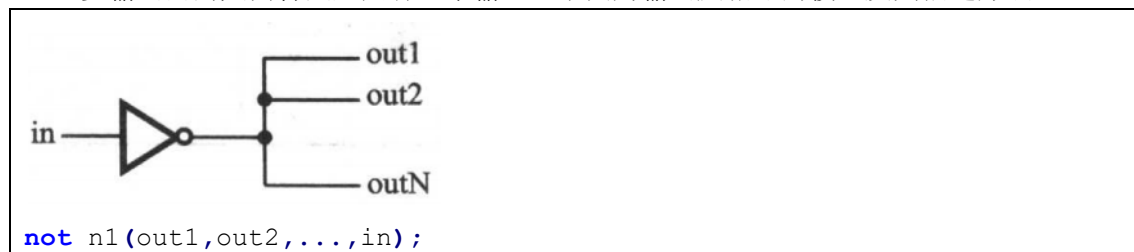
3.2. 多输入门

多输入门共同的特点是只有一个输出。下图为与门(and)的模型及其描述方法。



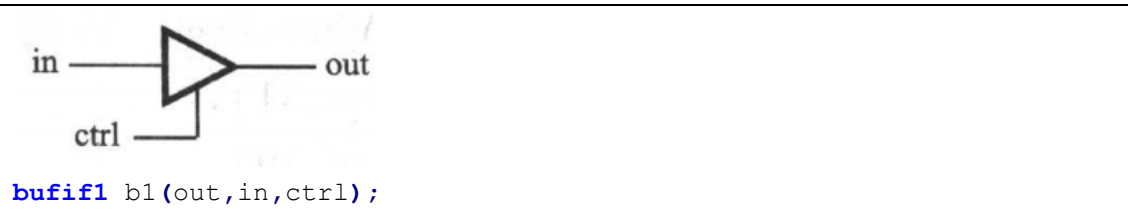
3.3. 多输出门

多输出门的共同特点是只有一个输入。下图为输出反相器的模型及其描述方法。



3.4. 三态门

三态门有一个输入、一个输出和一个控制输入端，如果输入控制信号无效，则三态门输出为高阻态 z 。下图为 `bufif1` 的模型和描述方法。



第四章 2 线--4 线译码器

本章基于 verilog 逻辑门基础设计一款 2 线--4 线译码器电路。

4.1. Verilog 代码结构

在设计译码器之前先介绍一下 verilog 逻辑代码的结构。

```
//File name: top_fpga.v
//Fuction : 顶层文件
//Log :
module top_fpga( //顶层模块名
    input clk, //模块端口
    input rst_n, //模块端口
    output led //模块端口
);
//信号定义
wire a0,a1;
wire [3:0] y;
//模块例化
_2to4decoder inst0(
    .A1(a1), //信号映射
    .A0(a0), //信号映射
    .Y(y) //信号映射
);
//逻辑代码
assign led = ~Y;
endmodule//模块结束
```

Verilog 逻辑之间的关系通过一个顶层 module 模块来组织关系，顶层模块把需要的 module 模块例化(按照模块名+端口名)到自己的模块内，上面的例子就是顶层模块 top_fpga 把“5.2 节”的“_2to4decoder”模块例化到了自己的模块内。同样，其他模块也可以利用同样的方法把需要的模块例化到自己的模块内。

4.2. 门级建模

假设 2 线输入为 A 和 B，四线为 Y[3:0]，则译码的真值表为：

{A1,A0}	Y
{0,0}	{0,0,0,1}
{0,1}	{0,0,1,0}
{1,0}	{0,1,0,0}
{1,1}	{1,0,0,0}

根据真值表得到下面的设计代码：

```
module _2to4decoder(
    input A1, //输入信号
```



```
    input A0,          //输入信号
    output [3:0] Y      //输出信号
);

wire A1_n,A0_n; //电路内部节点信号

not n1(A1_n,A1);
not n2(A0_n,A0);

and n4(Y[0],A1_n,A0_n);
and n5(Y[1],A1_n,A0);
and n6(Y[2],A1,A0_n);
and n7(Y[3],A1,A0);

endmodule
```

4.3. 数据流建模

上一节使用的是门级建模，这章我们使用数据流建模方式实现 2 线--4 线译码器电路。

设计的输入输出接口延用上一章，这里我们可以直接用 `assign` 语句实现这个组合逻辑功能，具体代码如下。

```
module _2to4decoder(
    input A1,          //输入信号
    input A0,          //输入信号
    output [3:0] Y      //输出信号
);

assign Y[0] = ~A1 & ~A0;
assign Y[1] = ~A1 & A0;
assign Y[2] = A1 & ~A0;
assign Y[3] = A1 & A0;

endmodule
```

上面的代码看起来是不是比上一章的代码简单很多，这也是 `verilog` 设计中使用最多的设计方法。

第五章 D 锁存器/触发器与时序逻辑电路

5.1. D 锁存器/触发器

行为级建模就是描述逻辑电路的功能和算法，一般使用 `always` 结构实现。本章实现 D 锁存器和 D 触发器的行为级建模。

下面为 D 锁存器的建模代码。

```
module D_latch(  
    input D,  
    input E,  
    output reg Q  
);  
  
always @(E or D)  
    if(E==1)  
        Q <= D;  
  
endmodule
```

如果输入信号 E 或 D 发生变化，就会执行一次 if 语句。

下面为 D 触发器的建模代码。

```
module DFF(  
    input D,           //输入数据信号  
    input clk,         //输入时钟信号  
    output reg Q       //输出数据信号  
);  
  
always @(posedge clk)  
    Q <= D;  
  
endmodule
```

当时钟 clk 的上升沿到来时，D 的值被传送到 Q。

5.2. 时序逻辑电路(10 进制计数器)

时序逻辑电路和组合逻辑电路的最大区别就是时序逻辑电路是以时钟方式驱动的，以时钟为节拍处理数据信号，本章基于时序逻辑电路的行为级建模方式设计一个 10 进制计数器。

假设输入时钟信号为 clk，输出数据为 Q[3:0](因为 10 进制中最大的位数为 9，而 9 的二进制表示为 1001，这样需要 4 位表示)，具体代码如下。

```
module DFF(  
    input clk,         //输入时钟信号  
    output reg [3:0] Q //输出数据信号  
);
```

```
always @(posedge clk)
    if(Q>=4'b1001)
        Q <= 4'b0000;
    else
        Q <= Q + 1'B1;

endmodule
```

当 clk 的上升沿到来是, always 结构里的语句被执行一次;当 Q 的值还没有到达 9(4'b1001) 时, Q 执行累加 1 操作;当 Q 的值为 9 时, Q 被清零;这样就实现了一个 10 进制的计数器。

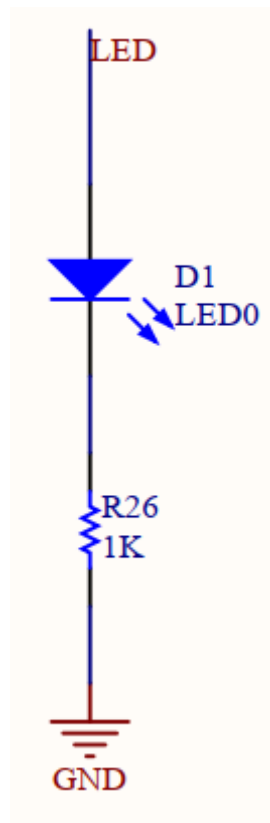
第六章 LED 闪烁

本章是第一个 FPGA 项目，将详细介绍创建新工程的流程，如果用户已经熟悉 Quartus II 开发流程可以略过或作参考。

本设计目的是让 1 盏 LED 每隔 1 秒闪烁一次。

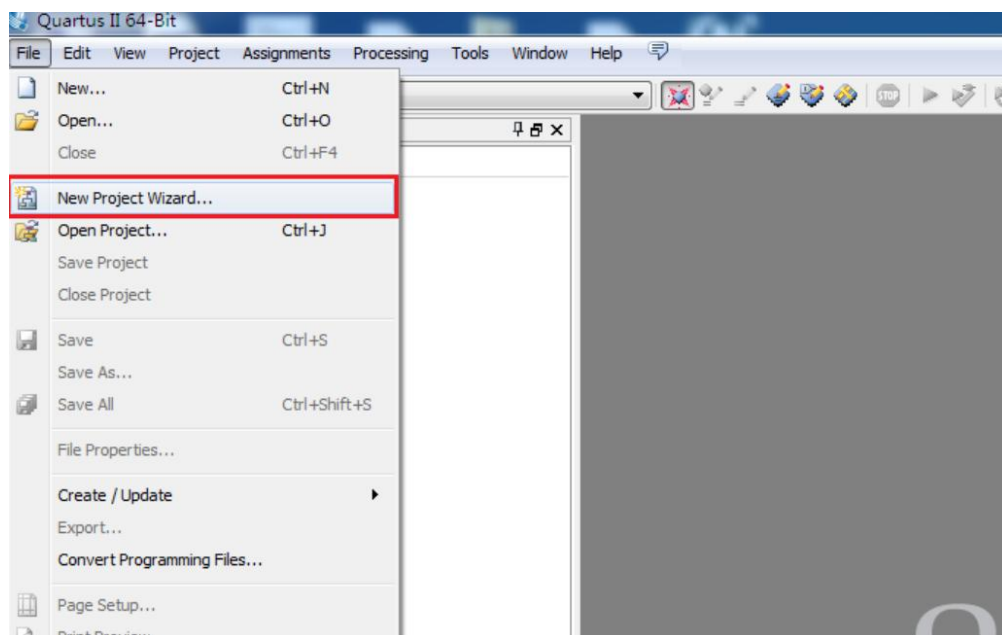
6.1. 硬件介绍

开发板上有 1 盏 LED 灯，1 盏为电源指示灯，1 盏为 FPGA 可控制的 LED 灯(高电平点亮)。

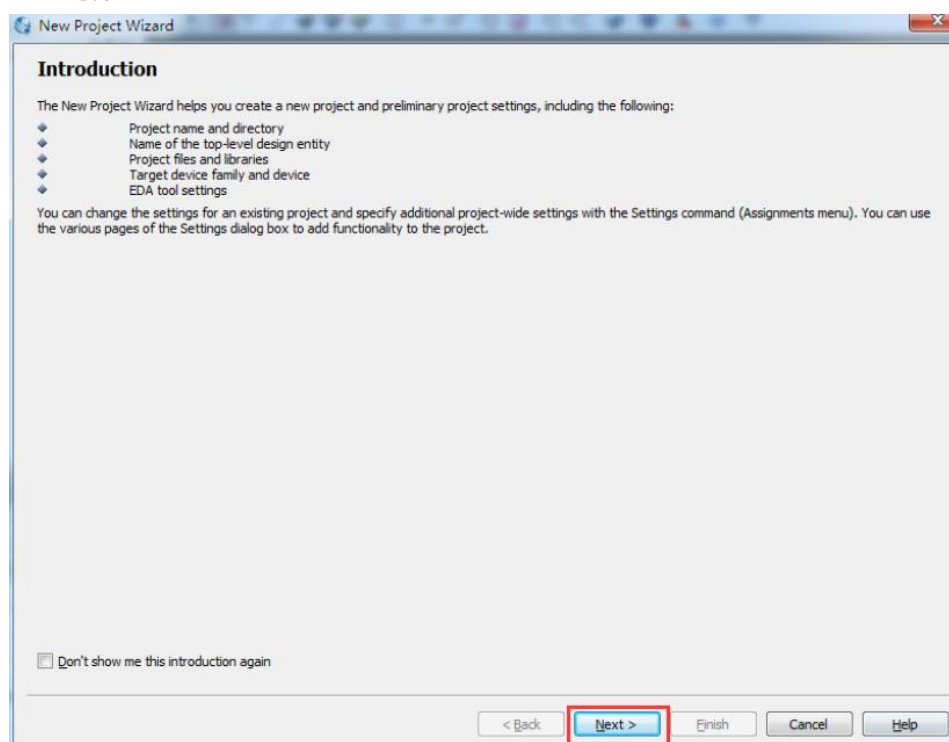


6.2. 新建工程

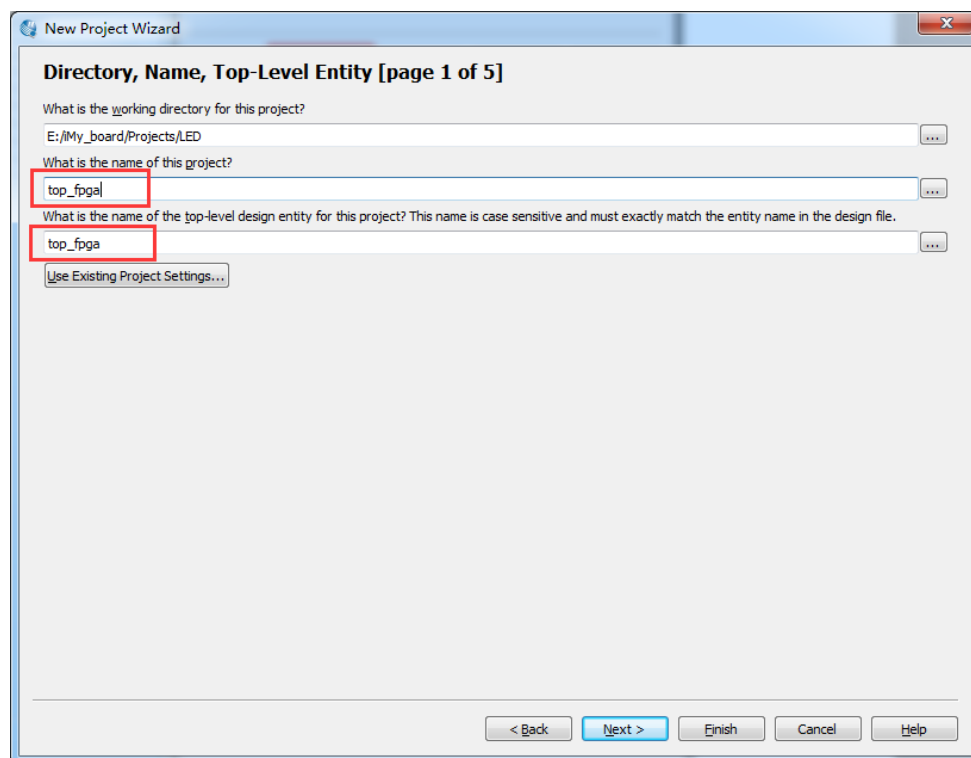
启动 Quartus II，选择菜单 File -> New Project Wizard。



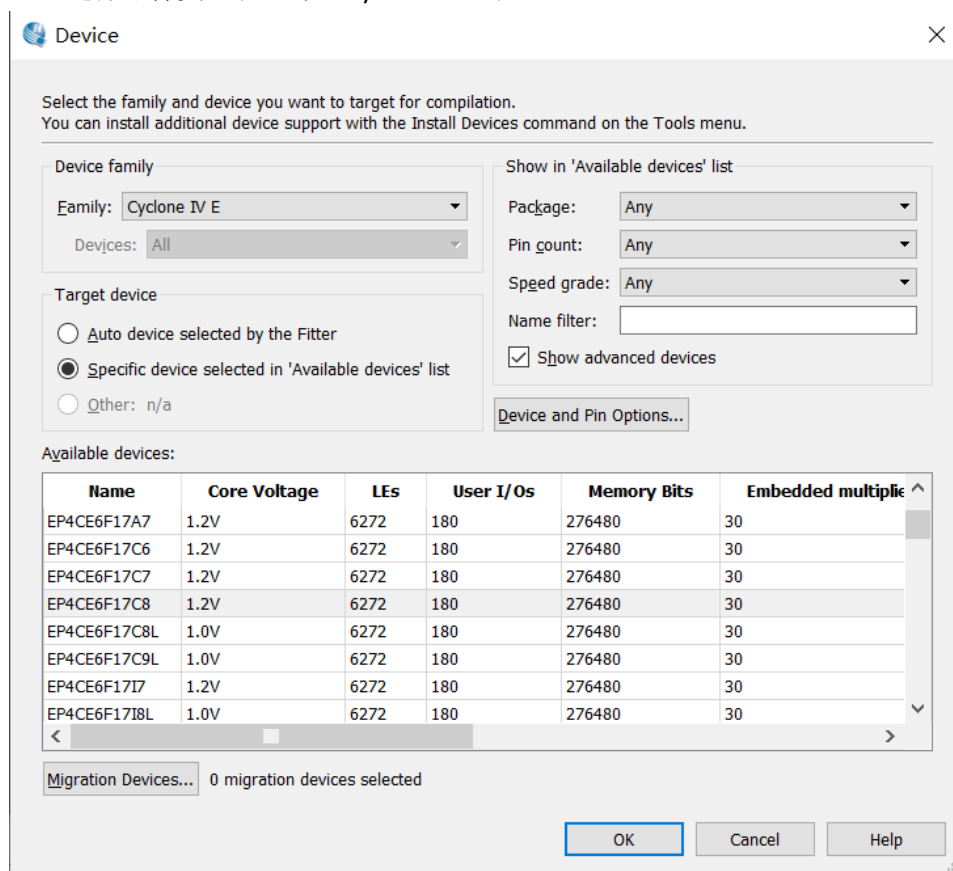
选择"Next".



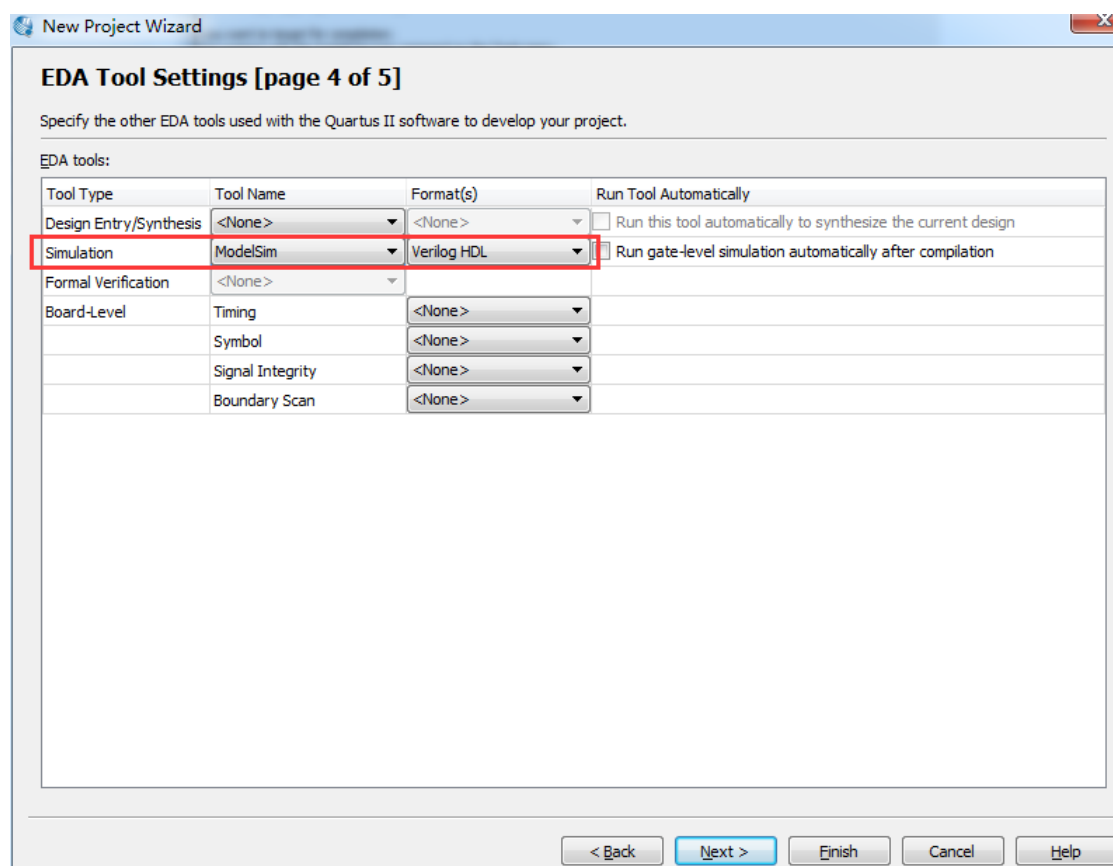
在工程目录下输入工程名(top_fpga)和顶层设计名(top_fpga)，这两个名字一样。



选择器件类型和型号：Cyclone IV E 和 EP4CE6F17C8。



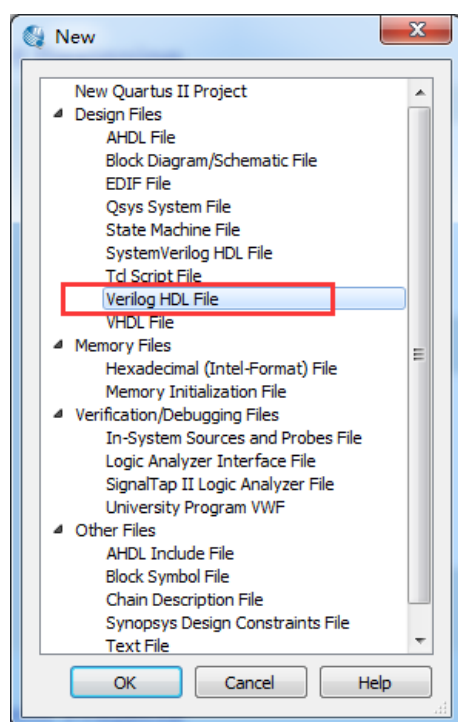
选择仿真软件和仿真语言，这里选择 Modelsim/verilog。



点击"Finish"完成工程创建。

6.3. HDL 设计

新建 top_fpga.v 文件，点击菜单 File -> New，选择 Verilog HDL File。



top_fpga.v 代码设计如下:

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.9.9
//function:        top file of project
//log:             led light

module top_fpga(
    //global signal
    input clk, //50MHz
    input rst_n, //low active

    //led
    output led //high active
);

//寄存器定义
reg [31:0] timer=0;
reg led_r=0;

assign led = led_r;

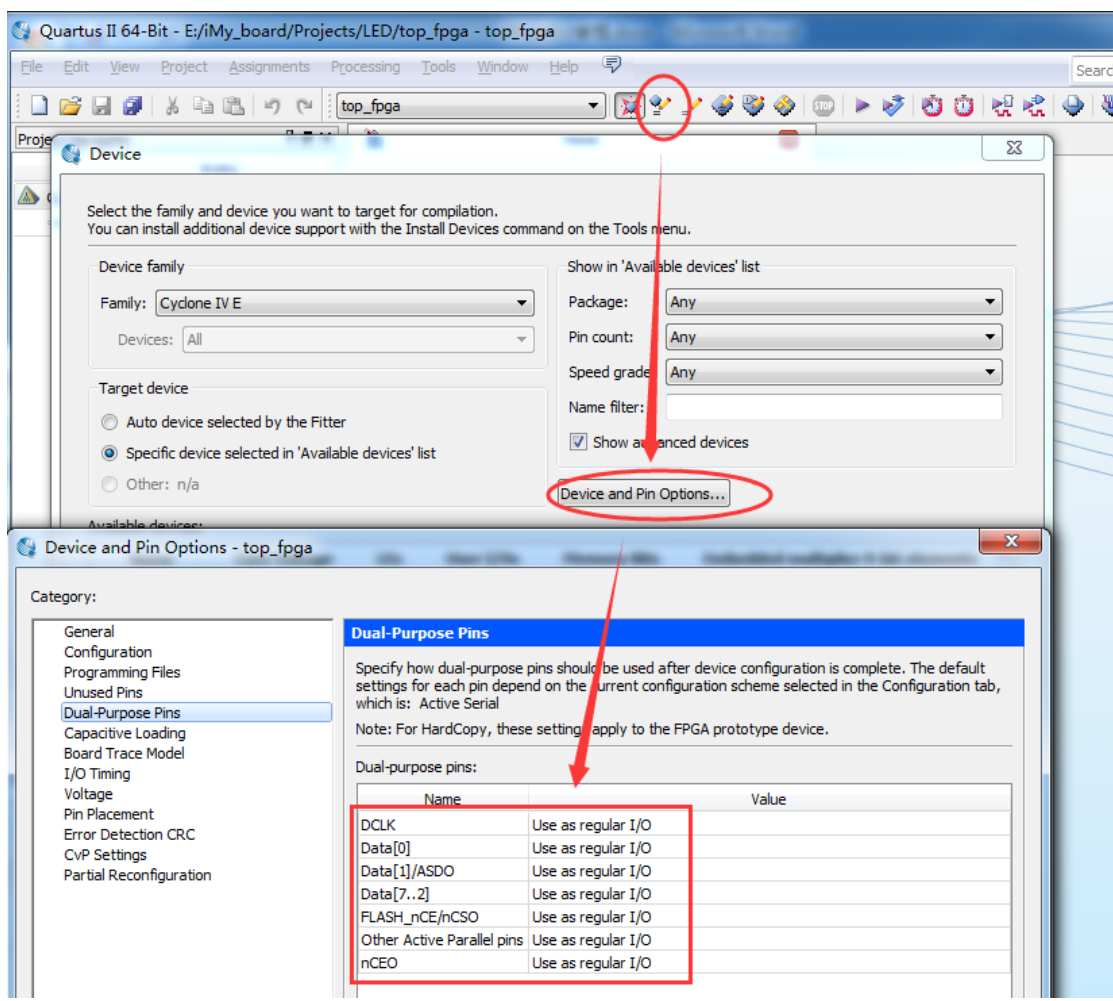
always @(posedge clk or negedge rst_n)
    if (~rst_n)
        timer <= 0; //计数器清零
    else if (timer == 32'd49_999_999) //0.5 秒计数(50M-1=49_999_999)
        timer <= 0; //计数器清零
    else
        timer <= timer + 1'b1; //计数器加 1

//LED 控制
always @(posedge clk or negedge rst_n)
    if (~rst_n)
        led_r <= 1'b0; //LED 灯灭
    else if (timer == 32'd24_999_999) //计数器计到 0.5 秒,
        led_r <= 1'b1; //LED 点亮
    else if (timer == 32'd49_999_999) //计数器计到 1 秒,
        led_r <= 1'b0; //LED 灯灭
    else
        led_r <= led_r;

endmodule
```

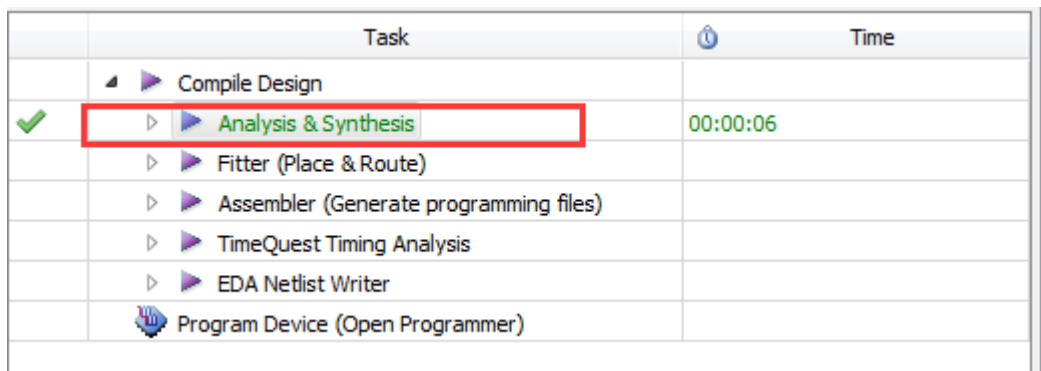
编写好代码后, 保存为 top_fpga.v。

6.4. 设置 Dual-Purpose Pins



6.5. 编译工程与管脚约束

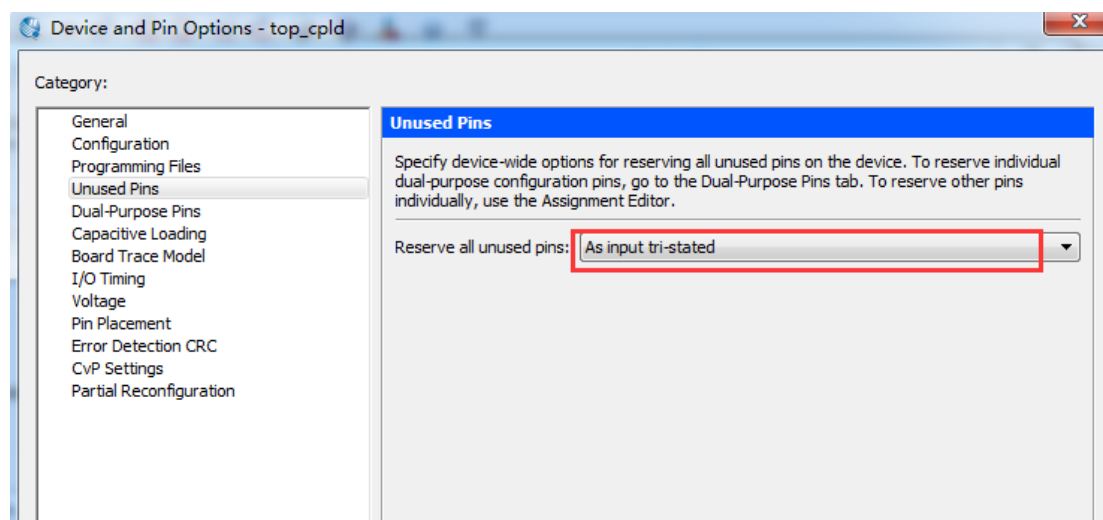
点击 Analysis & Synthesis 按钮，开始综合代码。



选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
in clk	Input	PIN_M16	5
out led	Output	PIN_F6	8
in rst_n	Input	PIN_L10	4

设置 Unused Pins 为输入三态。



点击 Compile Design 按钮，全编译工程。

✓	◀ ▶	Compile Design	00:00:21
✓	▶	Analysis & Synthesis	00:00:03
✓	▶	Fitter (Place & Route)	00:00:09
✓	▶	Assembler (Generate programming files)	00:00:03
✓	▶	TimeQuest Timing Analysis	00:00:04
✓	▶	EDA Netlist Writer	00:00:02
		Program Device (Open Programmer)	

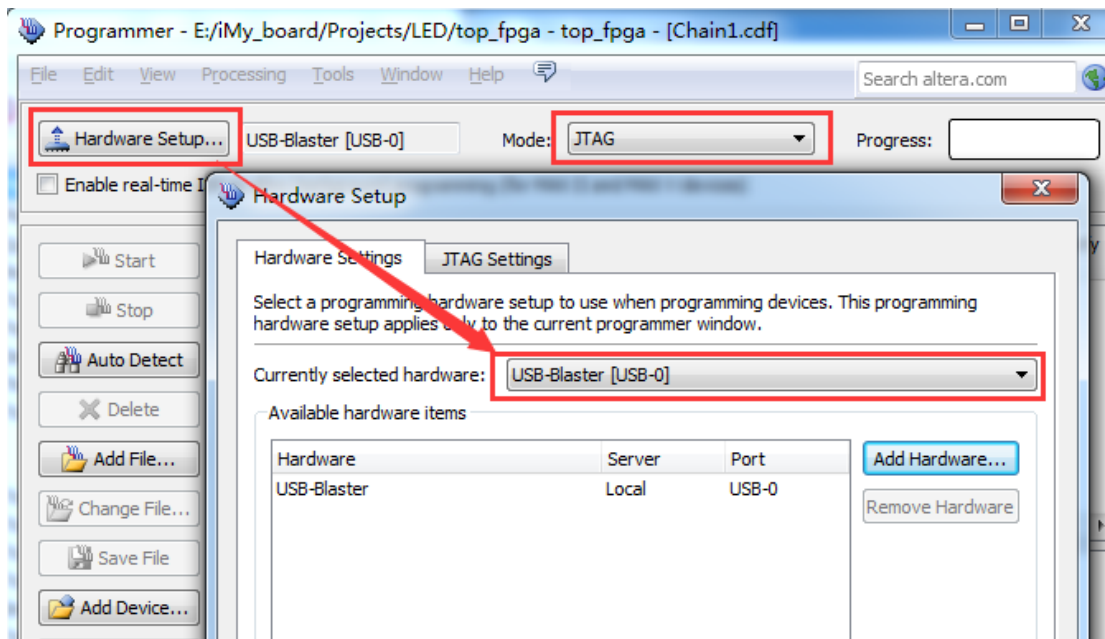
6.6. 下载 sof 文件

一般我们用 .sof 文件来验证我们的设计是否正确，.sof 文件用来在线调试 FPGA，因为 FPGA 断电后电路就恢复初始状态了，所以断电后 .sof 文件也就没有了。

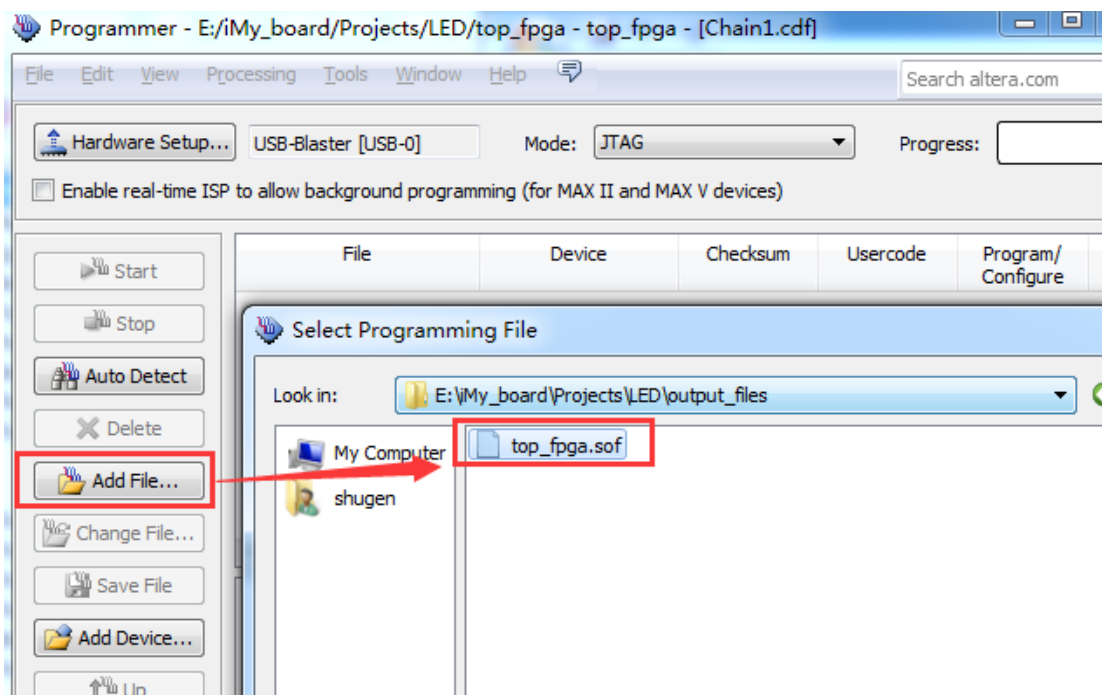
单击 Program Device(Open Programmer)。

Compile Design	00:00:21
Analysis & Synthesis	00:00:03
Fitter (Place & Route)	00:00:09
Assembler (Generate programming files)	00:00:03
TimeQuest Timing Analysis	00:00:04
EDA Netlist Writer	00:00:02
Program Device (Open Programmer)	

点击 Hardware Setup 选择 USB-Blaster[USB-0]，选择 JTAG Mode。



选择下载文件 `top_fpga.sof`，在目录 `output_files` 下。

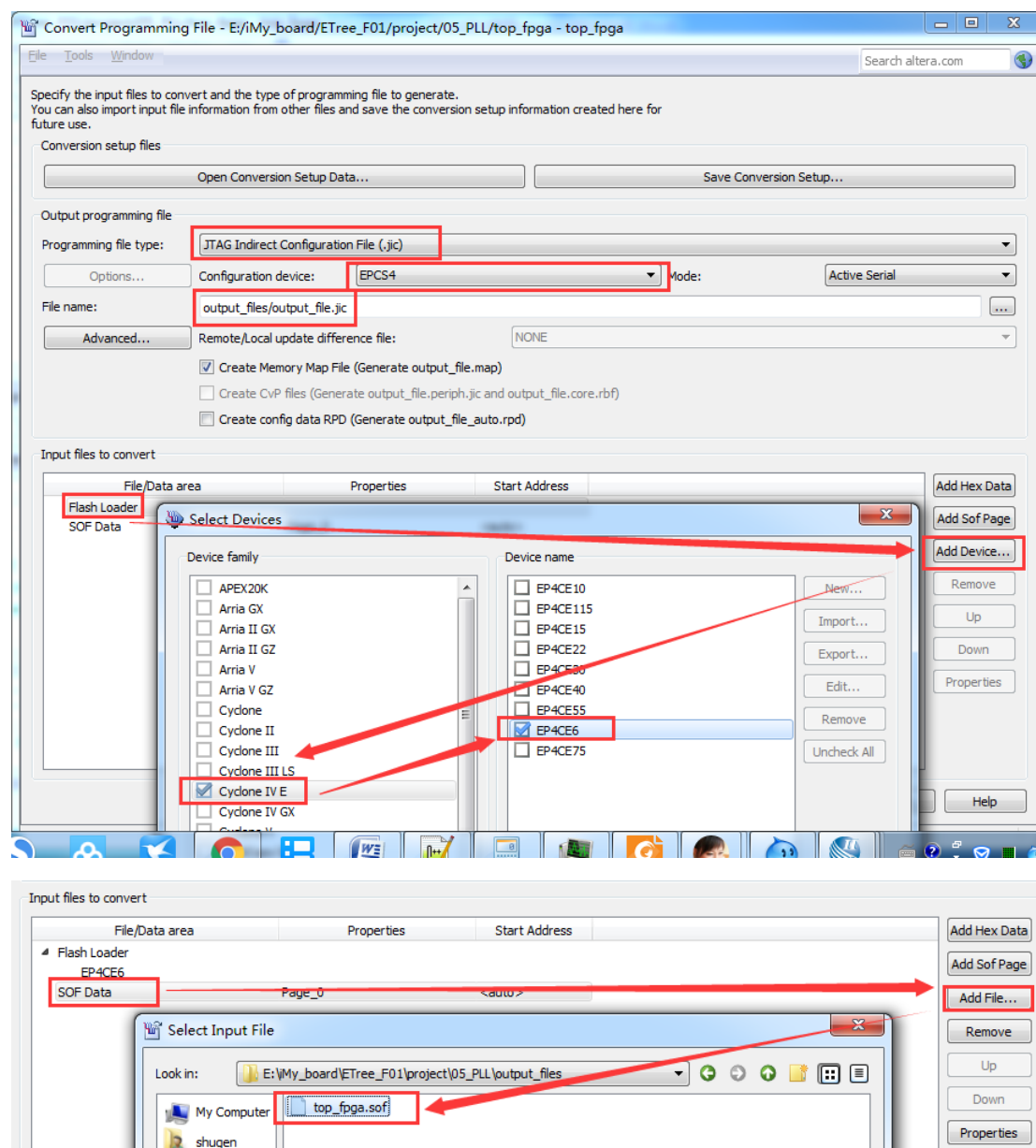


点击"Start"按钮，下载成功后可以看到 100%的绿色精度条，LED 开始流水。

6.7. jic 文件生成

在代码设计验证完成后，我们需要将编译后的配置文件固化到外部 flash，这样每次上电后 flash 会自动配置 FPGA。JIC 文件用于固化配置芯片 EPCS4(SPI Flash)，保存 FPGA 的配置文件，这样在断电重启后不需要再下载代码。

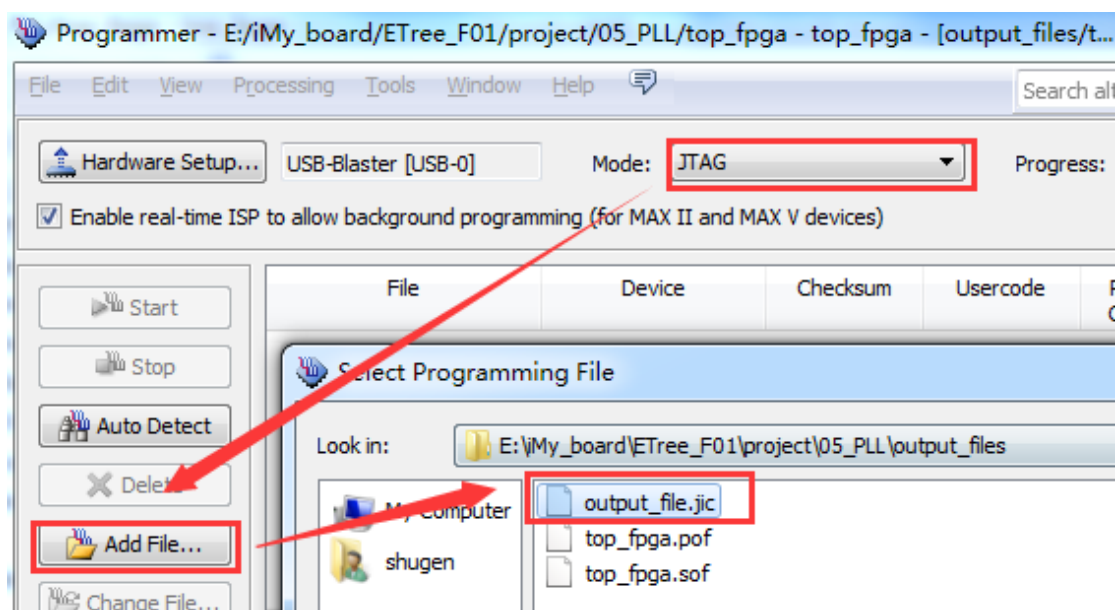
点击 File | Convert Programming File 弹出下面对话框，按照图示设置，Device Family 为 Cyclone IV，Device name 为 **EP4CE6**。



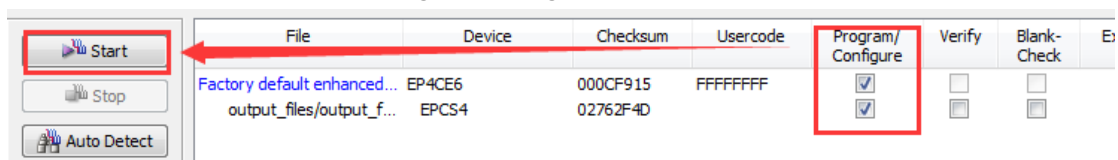
最后点击 Generate 生成 output.jic 文件。

6.8. 下载 jic 文件

这里依然选择 JTAG 模式，然后点击 Add File 添加 output_file.jic 文件，如下图所示。



添加好文件后，需要勾选"Program/Configure"，点击 Start 开始下载。



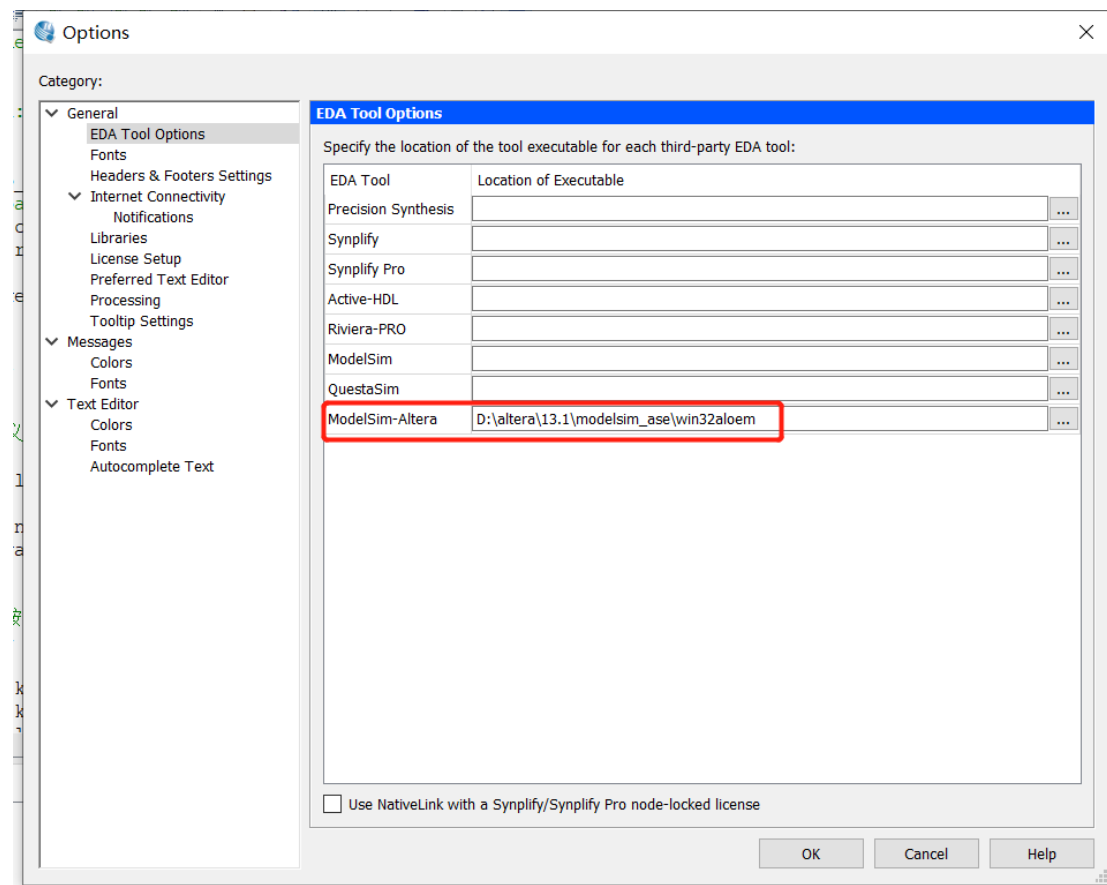
下载成功后可以看到 100%的绿色精度条，断电重启后 LED 不停流水闪烁。

第七章 关联 Modelsim SE 仿真

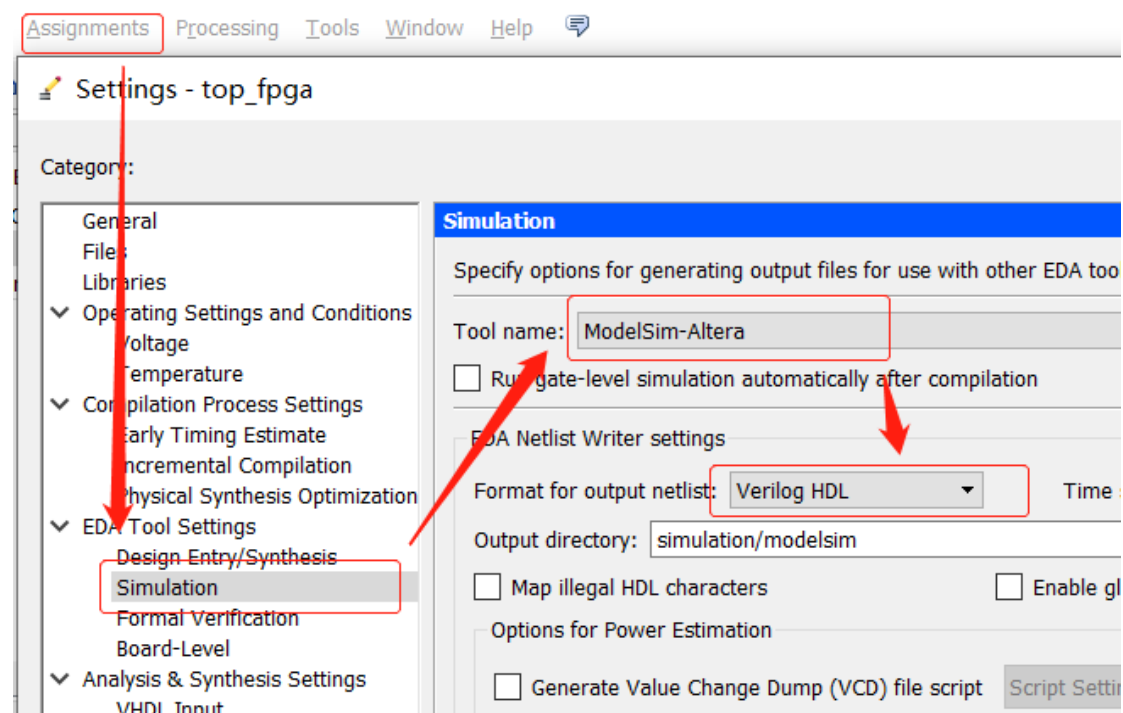
本章基于 LED 闪烁工程，实现 Modelsim-Altera 的仿真工作。

7.1. 关联 Modelsim SE 软件

打开 Quartus II 软件，点击 Tools | Options | General | EDA Tool Options | Modelsim-Altera，添加 Modelsim 的 win32 安装目录。

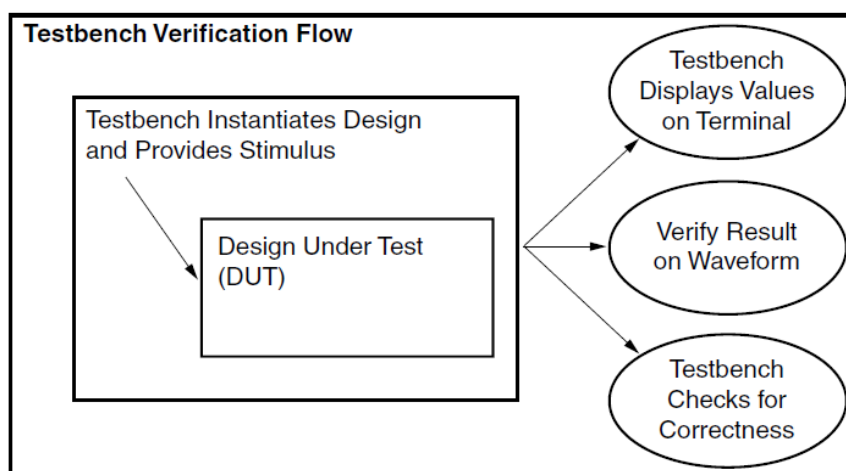


点击 Assignments | EDA Tool Settings | Simulation，仿真工具选择 Modelsim，输出仿真语言选择 Verilog HDL。



7.2. 测试脚本组成

测试脚本的仿真流程如下图所示：



测试脚本的组成如下：

```

//时间单位与精度
`timescale 1ns/1ps
module module_name_tst ();
//test vector input registers
reg clk;
//other input signals
//wires
wire clk_out;
//other output signals
  
```

```
//the inst of tested module
module_nameinst_name(
    .clk(clk),
    .clk_out(clk_out),
    //other port
);
//测试激励代码
endmodule
```

7.3. 生成测试脚本

点击Processing | start | start test-bench template writer, 生成测试脚本模型, 在工程目录LED\simulation\modelsim下。打开top_fpga.vt文件编写测试脚本, 编写好的测试脚本如下:

```
// Copyright (C) 1991-2013 Altera Corporation
// Your use of Altera Corporation's design tools, logic functions
// and other software and tools, and its AMPP partner logic
// functions, and any output files from any of the foregoing
// (including device programming or simulation files), and any
// associated documentation or information are expressly subject
// to the terms and conditions of the Altera Program License
// Subscription Agreement, Altera MegaCore Function License
// Agreement, or other applicable license agreement, including,
// without limitation, that your use is for the sole purpose of
// programming logic devices manufactured by Altera and sold by
// Altera or its authorized distributors. Please refer to the
// applicable agreement for further details.
*****
// This file contains a Verilog test bench template that is freely editable
// to
// suit user's needs .Comments are provided in each section to help the
// user
// fill out necessary details.
//
*****
// Generated on "09/10/2017 08:53:09"
// Verilog Test Bench template for design : top_fpga
// Simulation tool : ModelSim (Verilog)

`timescale1ns/1ps
moduletop_fpga_vlg_tst();
// constants
// general purpose registers
regclk;
regrst_n;
```



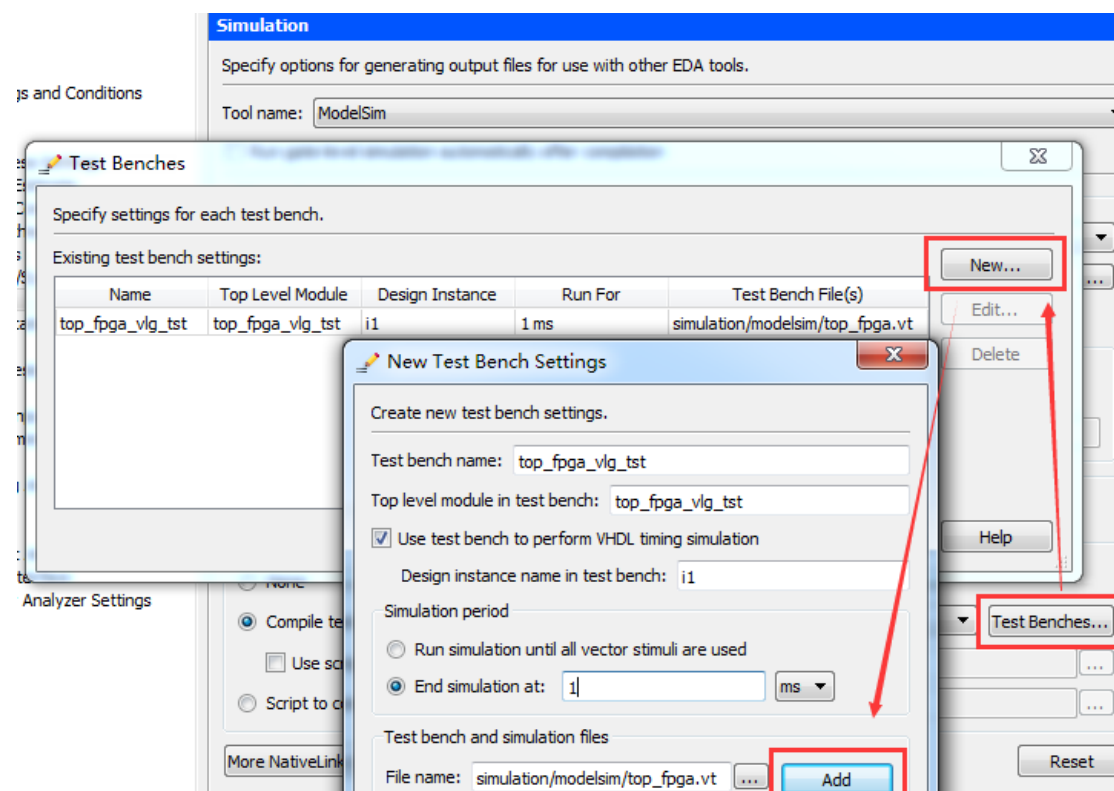
```
// wires
wire led;

// assign statements (if any)
top_fpga il (
// port map - connection between master ports and signals/registers
.clk(clk),
.led(led),
.rst_n(rst_n)
);

//时钟生成
initial
begin
clk=0;
forever clk=#(5)~clk;
$display("Running testbench");
end

//复位生成
initial
begin
rst_n=0;
#50;
rst_n=1;
#1000;
rst_n=1;
end
endmodule
```

点击Assignments | EDA Tool Settings | Simulation | Compile test bench | Test Benchs | New, 在New Test Bench Settings中输入测试脚本名和顶层测试脚本名(一般为同名)、被测试模块例化名、结束仿真时间、和测试脚本文件。

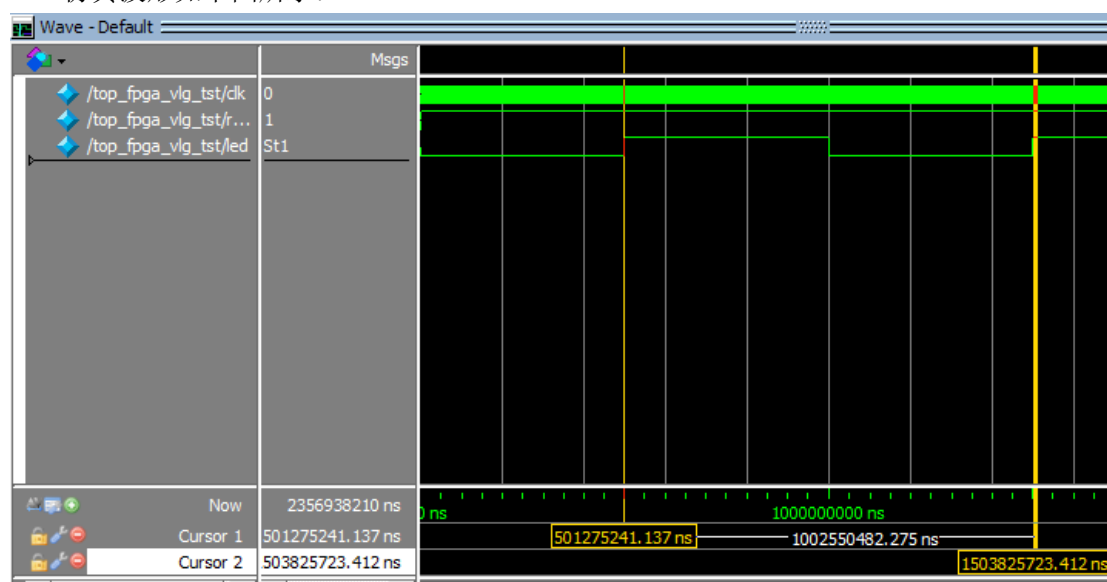


注意，Design instance name in test bench 为 i1，需要和测试脚本里的一样。

7.4. 开始仿真

在仿真前，需要重新编译整个工程，编好后点击 Tools | Run Simulation Tool | RTL Simulation。

仿真波形如下图所示：



通过仿真波形可以看出 LED 信号实现了循环取反操作，设计符合要求。

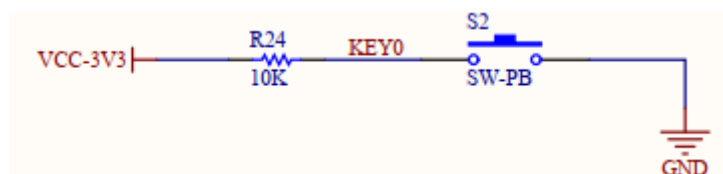
注意：关于 Modelsim 软件操作不包含在该文档中，用户可以参考相关资料。

第八章 按键检测

本章在上一章的基础上，增加按键暂停/开始功能，即按键按下 LED 开始闪烁，再按下 LED 暂停闪烁。

8.1. 硬件介绍

硬件电路如下图所示，这里我们用到按键“S2”，低电平有效(按键按下时，KEY0 被接到 GND)。

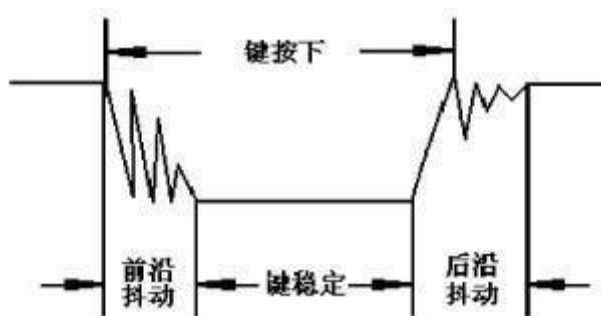


8.2. HDL 设计

由于硬件和操作原因，一般按键按下或释放时，有大约 20ms 的抖动时间，这个时间段电平的状态是不停变化的，FPGA 无法正确捕捉。这里我们要将这 20ms 时间内的抖动信号去除掉。

简单的方法，就是当按键按下后，不停地检测对应 PIN 脚的状态，若能延续超过 20ms 以上为低电平，则认为按键按下一次有效。

按键的抖动示意图如下所示。



下面为按键去抖模块代码：

```
//file name:      key_jitter.v
//author:         ETree
//date:           2017.7.23
//function:       按键消抖
//log:

module key_jitter(
    input  clk,           //50MHz
    input  key_n,         //低电平有效
    output reg key_valid //高电平有效
```

```
);

reg [19:0] cnt0;
always @(posedge clk)
begin
    cnt0 <= cnt0 + 1'b1;
end

reg [3:0] key_down_check;
always @(posedge clk)
begin
    if(cnt0 == 20'hffff)
        key_down_check <= {key_down_check[2:0],~key_n};
    else
        key_down_check <= key_down_check;
end

reg key_valid_r0;
always @(posedge clk)
begin
    key_valid_r0 <= (key_down_check==4'b0011)?1'b1:1'b0;
end

reg key_valid_r1;
reg key_valid_r2;
always @(posedge clk)
begin
    key_valid_r1 <= key_valid_r0;
    key_valid_r2 <= key_valid_r1;
end

wire key_valid_pos;

assign key_valid_pos = ({key_valid_r2,key_valid_r1}==2'b01)?1'b1:1'b0;

always @(posedge clk)
begin
    key_valid <= (key_valid_pos==1'b1)?1'b1:1'b0;
end

endmodule
```

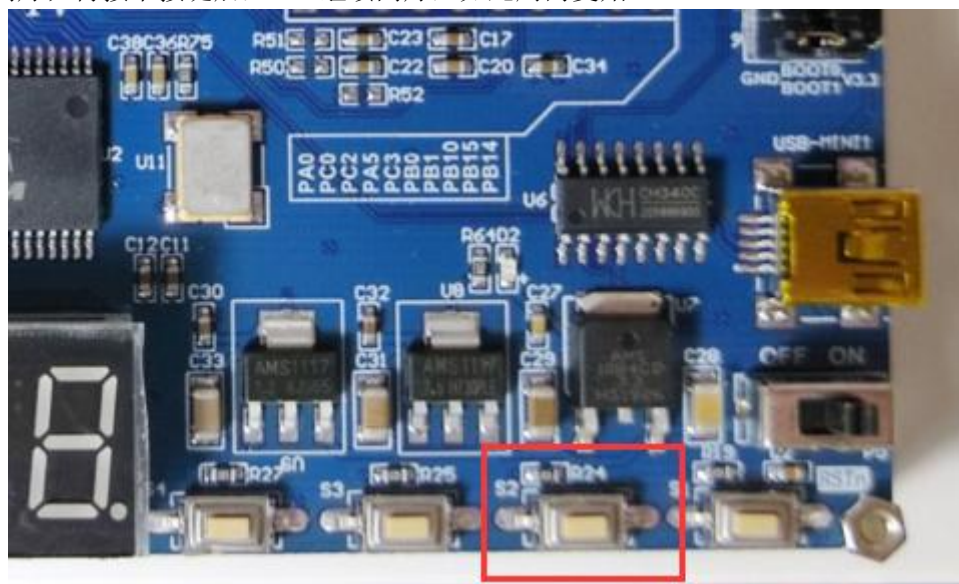
8.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
clk	Input	PIN_M16	5
key0	Input	PIN_K10	4
key1	Input	PIN_L9	4
key2	Input	PIN_K9	4
led	Output	PIN_F6	8
rst_n	Input	PIN_L10	4

8.4. 实验操作

将 FPGA 配置文件下载到板卡后，默认状态 LED 闪烁，按下板卡下方的 S2 按键后，LED 暂停闪烁；再按下按键后，LED 继续闪烁，如此周而复始。



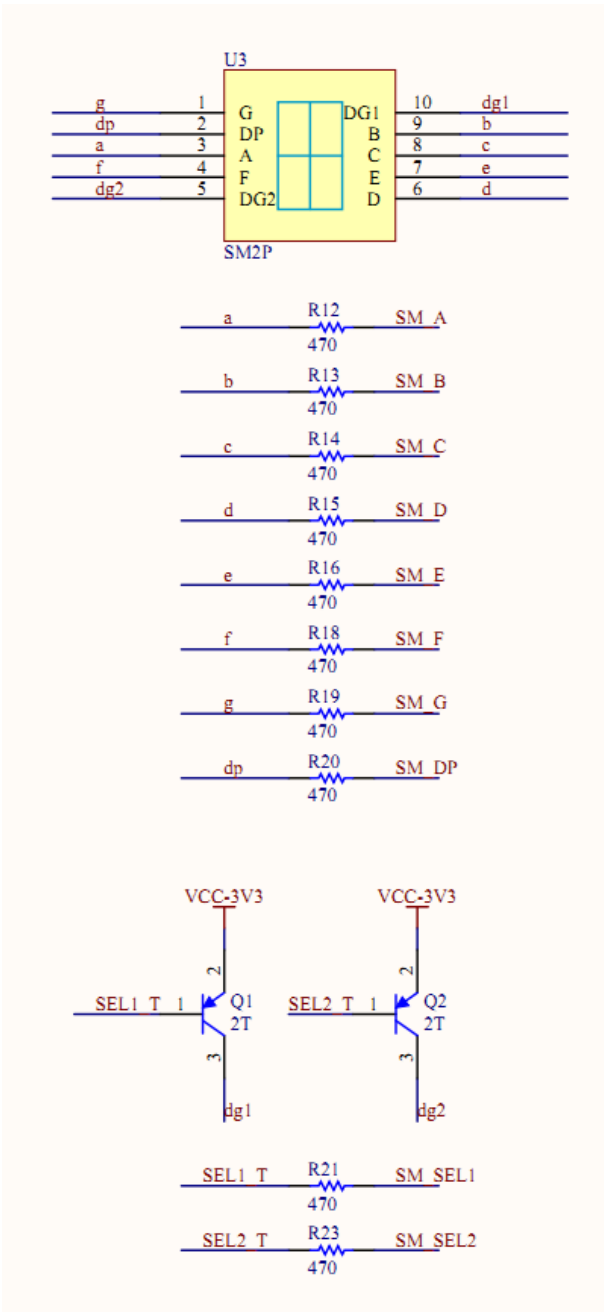
第九章 数码管秒表

本章基于 2 位数码管实现一款秒表。

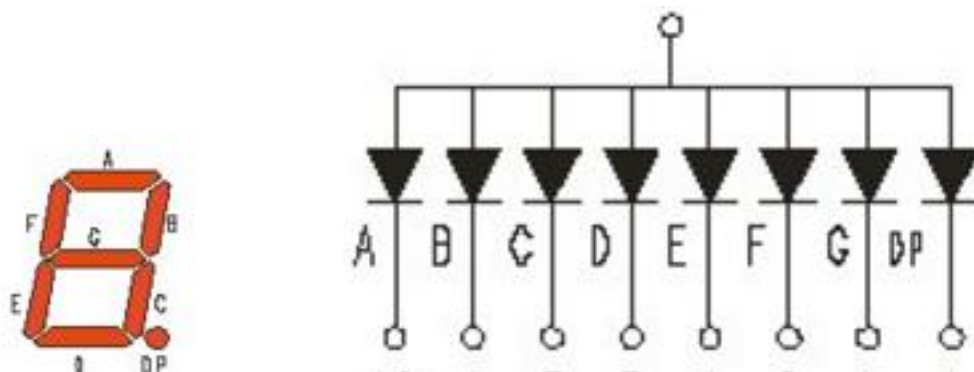
9.1. 硬件介绍

我们知道发光二极管常常用于各种电子产品中作为指示灯，除发光二极管外，常见的用于显示的器件还有数码管，比如电子时钟的显示时间就是用的数码管，万用表中的显示屏也是用的数码管。其实数码管的本质就是发光二极管的组合使用，最常见的就是七段和八段数码管，七段数码管是由 7 个长条形的发光二极管组成，八段数码管比七段数码管多了一个点(通常是右下角的一个点)。

八段数码管硬件电路如下：



八段数码管引脚图如下：数码管为公阳极，即八条 LED 的阳极公用一个端点，这样在 LED 的阴极低电平时 LED 就会发光，LED 阴极为高电平时，LED 就会熄灭，通常我们简称“低电平有效”。



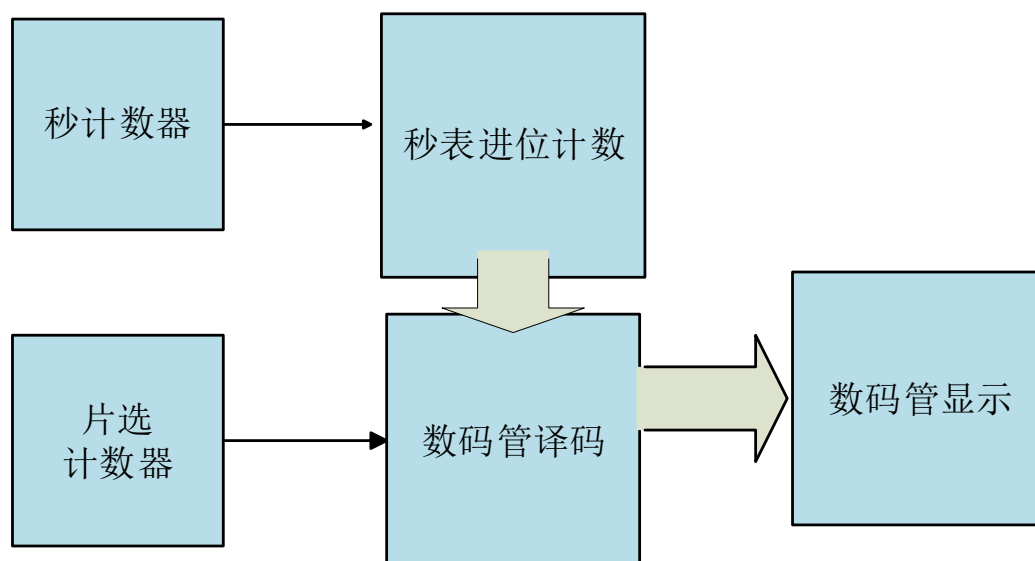
根据八段数码管的引脚图，定义 parameter 常量来对应数码显示，代码如下：

```
parameter ZERO    = 8'b0000_0011, //8'b1100_0000,
ONE              = 8'b1001_1111, //8'b1111_1001,
TWO             = 8'b0010_0101, //8'b1010_0100,
THREE          = 8'b0000_1101, //8'b1011_0000,
FOUR           = 8'b1001_1001, //8'b1001_1001,
FIVE           = 8'b0100_1001, //8'b1001_0010,
SIX            = 8'b0100_0001, //8'b1000_0010,
SEVEN          = 8'b0001_1111, //8'b1111_1000,
EIGHT          = 8'b0000_0001, //8'b1000_0000,
NINE           = 8'b0000_1001, //8'b1001_0000;
AA             = 8'b0001_0001,
BB             = 8'b1100_0001,
CC             = 8'b0110_0011,
DD             = 8'b1000_0101,
EE             = 8'b0110_0001,
FF             = 8'b0111_0001;
```

其中参数编码的高位到低位分别表示：{a, b, c, d, e, f, g, dp}。

9.2. HDL 设计

➤ 系统框架设计



➤ 秒表进位计数

```

//-----秒表进位计数器-----
always @(posedge clk or negedge rst_n)
    if(~rst_n)
        second_0 <= 0;
    else if(timer==0)
        if(second_0<9)
            second_0 <= second_0 + 1'b1;
        else
            second_0 <= 0;
    else
        second_0 <= second_0;

always @(posedge clk or negedge rst_n)
    if(~rst_n)
        second_1 <= 0;
    else if(timer==0)
        if(second_0==9)
            if(second_1<5)
                second_1 <= second_1 + 1'b1;
            else
                second_1 <= 0;
        else
            second_1 <= second_1;
    else
        second_1 <= second_1;
  
```

当 second_0 计数到 9 后，产生进位信号；当 second_1 计数到 5 后，等到各位计数到 9 后，清零。

9.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
in clk	Input	PIN_M16	5
out led	Output	PIN_F6	8
in rst_n	Input	PIN_L10	4
out sm_dat[7]	Output	PIN_R4	3
out sm_dat[6]	Output	PIN_T3	3
out sm_dat[5]	Output	PIN_R3	3
out sm_dat[4]	Output	PIN_T2	3
out sm_dat[3]	Output	PIN_L7	3
out sm_dat[2]	Output	PIN_N3	3
out sm_dat[1]	Output	PIN_P3	3
out sm_dat[0]	Output	PIN_M6	3
out sm_sel[1]	Output	PIN_L4	2
out sm_sel[0]	Output	PIN_L3	2

9.4. 实验操作

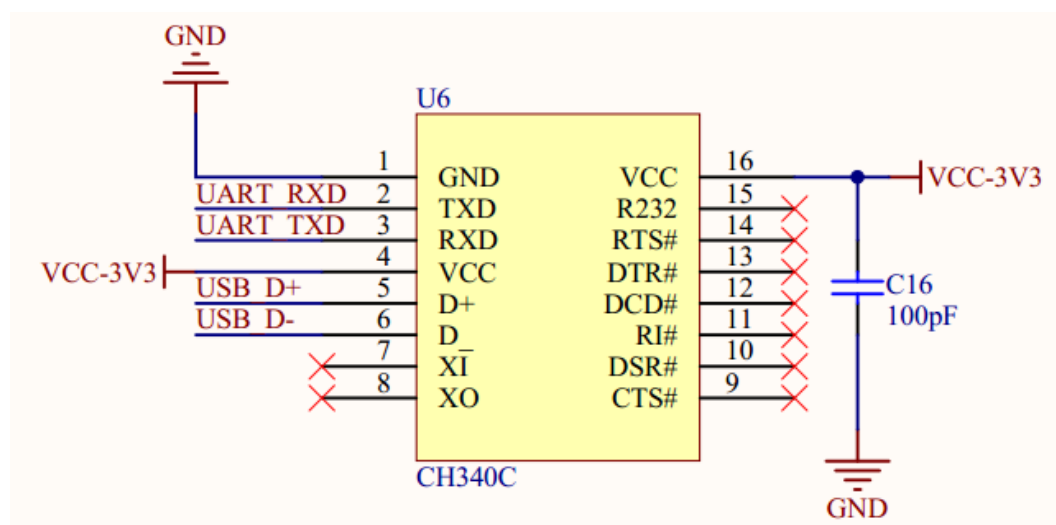
插入 Mini-USB 线缆，打开电源按钮，下载 FPGA 配置文件，数码管便开始秒表计数。

第十章 UART 串口通信

本章基于 USB 转串口芯片完成板卡与 PC 间的通信。每当按键按下一次，FPGA 板卡向 PC 发送一帧数据，通过串口调试助手观察结果。PC 向 FPGA 板卡发送 2 个字节的十六进制数据，FPGA 板卡合并成 16 位的数据，然后通过 SignalTap II 观察结果。

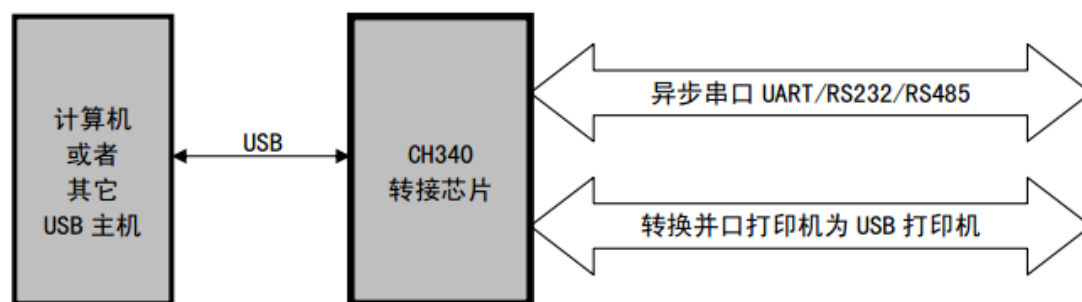
10.1. 硬件介绍

本设计采用 CH340C 芯片来完成 USB 转串口的工作，CH340C 具有高度集成的优点，无需外挂晶振，可节省电路板面积。



CH340 是一个 USB 总线的转接芯片，实现 USB 转串口或 USB 转打印口。

在串口方式下，CH340 提供常用的 MODEM 联络信号，用于为计算机拓展异步串口，或者将普通的串口设备直接升级到 USB 总线。下图为 CH340 芯片的转接示意图。



CH340 具有如下特点：

- ✧ 全速 USB 设备接口，兼容 USB V2.0。
- ✧ 仿真标准串口，用于升级原串口外围设备，或者通过 USB 增加额外串口。
- ✧ 计算机端 Windows 操作系统下的串口应用程序完全兼容，无需修改。
- ✧ 硬件全双工串口，内置收发缓冲区，支持通讯波特率 50bps~2Mbps。
- ✧ 支持常用的 MODEM 联络信号 RTS、DTR、DCD、RI、DSR、CTS。
- ✧ 通过外加电平转换器件，提供 RS232、RS485、RS422 等接口。
- ✧ 软件兼容 CH341，可以直接使用 CH341 的驱动程序。
- ✧ 支持 5V 电源电压和 3.3V 电源电压甚至 3V 电源电压。

✧ 提供 SOP-16 和 SSOP-20 无铅封装，兼容 RoHS。

10.2. UART 通信协议

串口通信的概念非常简单，串口按位(bit)发送和接收字节，物理层只有一根通信线。

串口通信最重要的参数是比特率、数据位、停止位和奇偶校验。对于两个进行通信的端口，这些参数必须匹配。

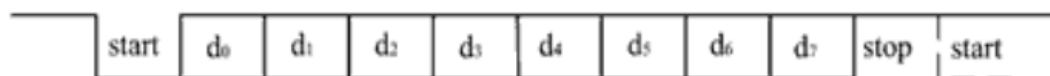
a, 波特率：这是一个衡量通信速度的参数。它表示每秒钟传送的 bit 的个数。例如 300 波特表示每秒钟发送 300 个 bit。

b, 数据位：这是衡量通信中实际数据位的参数。典型的值为 8 位。

c, 停止位：用于表示单个包的最后一位。典型的值为 1, 1.5 和 2 位。由于数据是在传输线上定时的，并且每一个设备有其自己的时钟，很可能在通信中两台设备间出现了小小的不同步。因此停止位不仅仅是表示传输的结束，并且提供计算机校正时钟同步的机会。适用于停止位的位数越多，不同时钟同步的容忍程度越大，但是数据传输率同时也越慢。

d, 奇偶校验位：在串口通信中一种简单的检错方式。对于偶和奇校验的情况，串口会设置校验位(数据位后面的一位)，用一个值确保传输的数据有偶个或者奇个逻辑高位。例如，如果数据是 011，那么对于偶校验，校验位为 0，保证逻辑高的位数是偶数个。如果是奇校验，校验位为 1，这样就有 3 个逻辑高位。高位和低位不真正的检查数据，简单置位逻辑高或者逻辑低校验。这样使得接收设备能够知道一个位的状态，有机会判断是否有噪声干扰了通信或者是否传输和接收数据是否不同步。

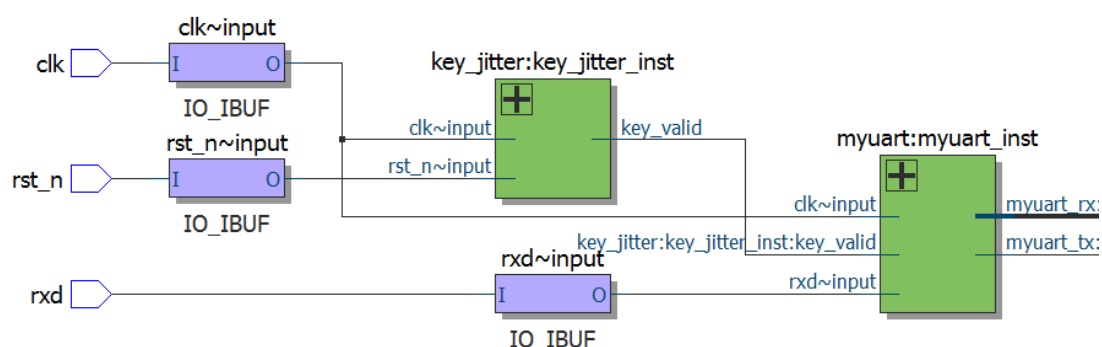
本设计采用的数据帧格式为 10 位，即 1 位起始位+8 位数据位+1 位停止位，波特率为 9600bps。注意在传输数据位时，是从低位向高位传输。



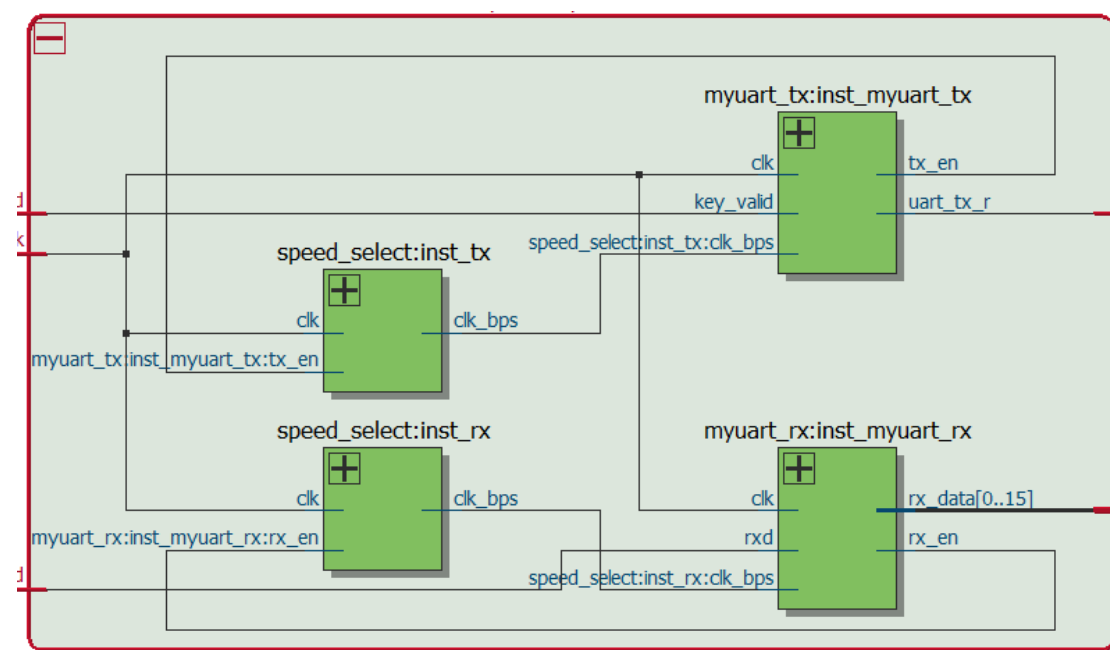
10.3. HDL 设计

➤ 系统框架

下图为系统模块框图，可在实例工程中点击：Tools -> Netlist Viewers -> Technology Map Viewer 查看。



下图为串口接收与发送模块框图，可在打开的系统模块框图中，双击 `myuart` 模块查看。



➤ 串口接收模块设计

```
//file name:      myuart_rx.v
//author:         ETree
//date:           2017.9.9
//function:        数据接收
//log:

module myuart_rx (
    input clk,
    input rst_n,
    output reg bps_start,
    input clk_bps,
    input rxd,
    output reg [15:0] rx_data
);

//串口接收 buffer
reg rxd0;
reg rxd1;
reg rxd2;
reg rxd3;

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
        begin
            rxd0 <= 1'b1;
```

```
        rxd1 <= 1'b1;
        rxd2 <= 1'b1;
        rxd3 <= 1'b1;
    end
    else
        begin
            rxd0 <= rxd;
            rxd1 <= rxd0;
            rxd2 <= rxd1;
            rxd3 <= rxd2;
        end
    end
end

wire rxd_pos;    //串口接收数据的上升沿
wire rxd_neg;    //串口接收数据的下降沿

assign rxd_pos = ~rxd3 & ~rxd2 & rxd1 & rxd0;
assign rxd_neg = rxd3 & ~rxd2 & ~rxd1 & ~rxd0;

//-----接收使能信号-----
reg rx_en;       //串口接收使能信号

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        bps_start <= 1'bz;
        rx_en <= 1'b0;
    end
    else
        if(rxd_neg)
        begin
            bps_start <= 1'b1;
            rx_en <= 1'b1;
        end
        else
            if(num==4'd9)
            begin
                bps_start <= 1'b0;
                rx_en <= 1'b0;
            end
            else
            begin
                bps_start <= bps_start;
```

```
        rx_en <= rx_en;
    end

end

//-----串转并-----
reg [3:0] num;           //串口接收 Bit 计数器
reg [7:0] rx_data_r;     //串转并数据 buffer

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        num <= 0;
        rx_data_r <= 0;
    end
    else
    begin
        if(rx_en)
        begin
            if(clk_bps)
            begin
                num <= num + 1'b1;
                case(num)
                    4'd1: rx_data_r[0] <= rxd;
                    4'd2: rx_data_r[1] <= rxd;
                    4'd3: rx_data_r[2] <= rxd;
                    4'd4: rx_data_r[3] <= rxd;
                    4'd5: rx_data_r[4] <= rxd;
                    4'd6: rx_data_r[5] <= rxd;
                    4'd7: rx_data_r[6] <= rxd;
                    4'd8: rx_data_r[7] <= rxd;
                //      4'd9: rx_data_r[8] <= rxd;
                default: rx_data_r <= rx_data_r;
                endcase
            end
        end
        else
        begin
            if(num == 4'd9)
            begin
                num <= 4'd0;
                rx_data_r <= rx_data_r;
            end
            else
            begin
                num <= num;
                rx_data_r <= rx_data_r;
            end
        end
    end
end
```

```

        end
        else
            begin
                num <= num;
                rx_data_r <= rx_data_r;
            end
        end
    end

    //-----高低字节拼接-----
    always @ (posedge clk or negedge rst_n)
    begin
        if(!rst_n)
            rx_data <= 0;
        else if(num == 4'd9)
            rx_data <= {rx_data[7:0],rx_data_r};
        else
            rx_data <= rx_data;
    end

endmodule

```

➤ 串口发送模块设计

```

//file name:      myuart_tx.v
//author:         ETree
//date:           2017.9.9
//function:       数据发送
//log:

module myuart_tx(
    input clk,
    input rst_n,
    output reg bps_start,
    input clk_bps,
    input key_valid,
    input [7:0] data_in,
    output txd
);

    reg tx_en;           //发送使能信号
    reg [3:0] num;        //并转串 bit 计数器
    reg [7:0] tx_data_r;  //发送字节数据 buffer
    reg uart_tx_r;        //发送接口 buffer
    //-----使能信号生成-----
    always @ (posedge clk or negedge rst_n)

```

```

begin
    if(!rst_n)
        begin
            bps_start <= 1'bz;
            tx_en <= 1'b0;
            tx_data_r <= 8'd0;
        end
    else if(key_valid)
        begin
            bps_start <= 1'b1;
            tx_data_r <= data_in;
            tx_en <= 1'b1;
        end
    else if(num==4'd10)
        begin
            bps_start <= 1'b0;
            tx_en <= 1'b0;
            tx_data_r <=8'h0;
        end
    else
        begin
            bps_start <= bps_start;
            tx_data_r <= tx_data_r;
            tx_en <= tx_en;
        end
    end

    //-----并转串-----
    always @ (posedge clk or negedge rst_n)
    begin
        if(!rst_n)
            begin
                num <= 4'd0;
                uart_tx_r <= 1'b1;
            end
        else if(tx_en)
            begin
                if(clk_bps)
                    begin
                        num <= num + 1'b1;
                        case(num)
                            4'd0: uart_tx_r <= 1'b0;
                            4'd1: uart_tx_r <= tx_data_r[0];    //bit0
                            4'd2: uart_tx_r <= tx_data_r[1];    //bit1
                            4'd3: uart_tx_r <= tx_data_r[2];    //bit2

```



```

4'd4: uart_tx_r <= tx_data_r[3];    //bit3
4'd5: uart_tx_r <= tx_data_r[4];    //bit4
4'd6: uart_tx_r <= tx_data_r[5];    //bit5
4'd7: uart_tx_r <= tx_data_r[6];    //bit6
4'd8: uart_tx_r <= tx_data_r[7];    //bit7
4'd9: uart_tx_r <= 1'b1;
default: uart_tx_r <= 1'b1;
endcase
end
else if(num==4'd10)
    num <= 4'd0;
else
    num <= num;
end
else
    uart_tx_r <= 1'b1;
end

assign txd = uart_tx_r;

endmodule

```

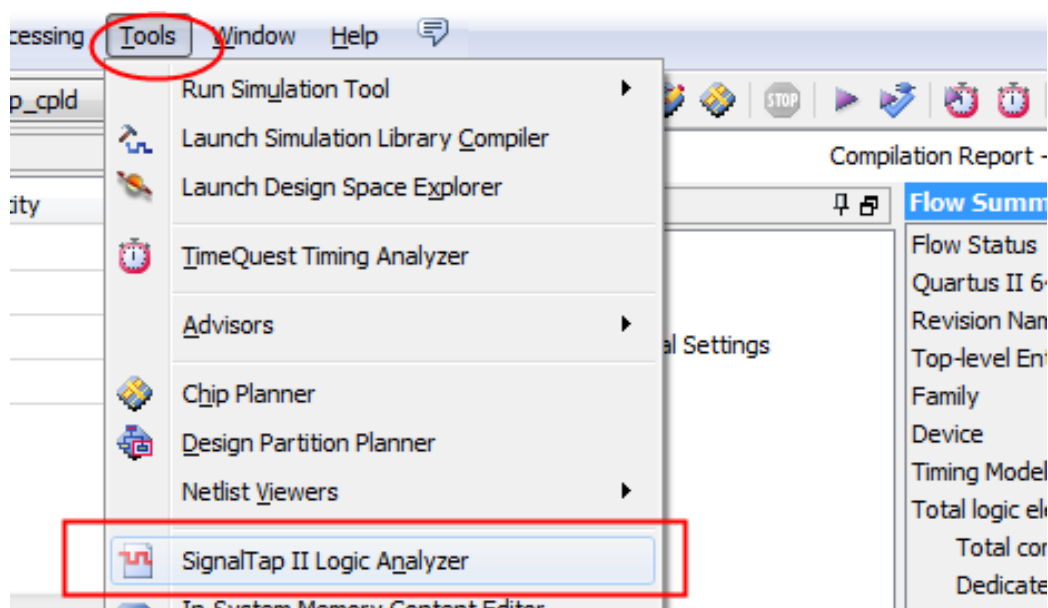
10.4. 管脚约束

选择菜单 Assignment -> Pin Planner，添加管脚约束。

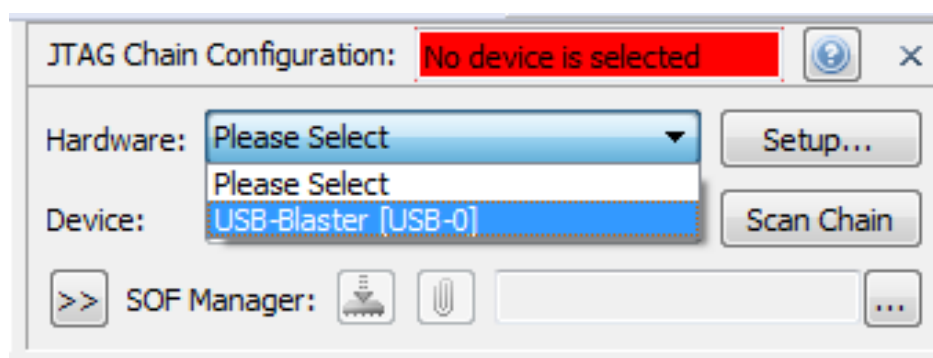
in	key0	Input	PIN_K10	4
in	key1	Input	PIN_L9	4
in	key2	Input	PIN_K9	4
out	led	Output	PIN_F6	8
in	rst_n	Input	PIN_L10	4
in	uart_rx	Input	PIN_T13	4
out	uart_tx	Output	PIN_R13	4

10.5. SignalTap 使用

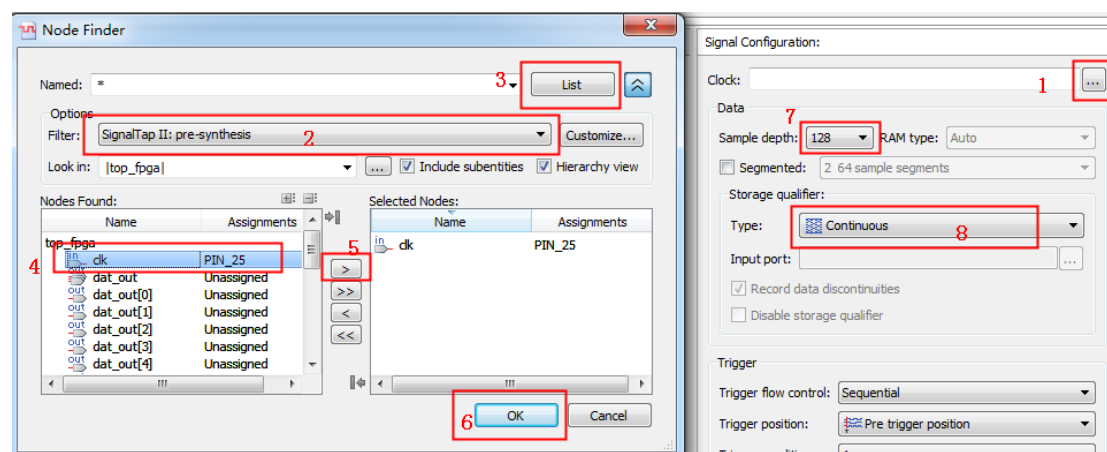
- 打开 SignalTap II Logic Analyzer



- 选择 JTAG Chain Configuration



- 设置采样时钟和采样深度



- 第一步：选择采样时钟；
 第二步：选择过滤类型，这里选 pre-synthesis；
 第三步：将所有符合的信号列出来；
 第四步：选择信号，这里选择 clk；
 第五步：添加信号；

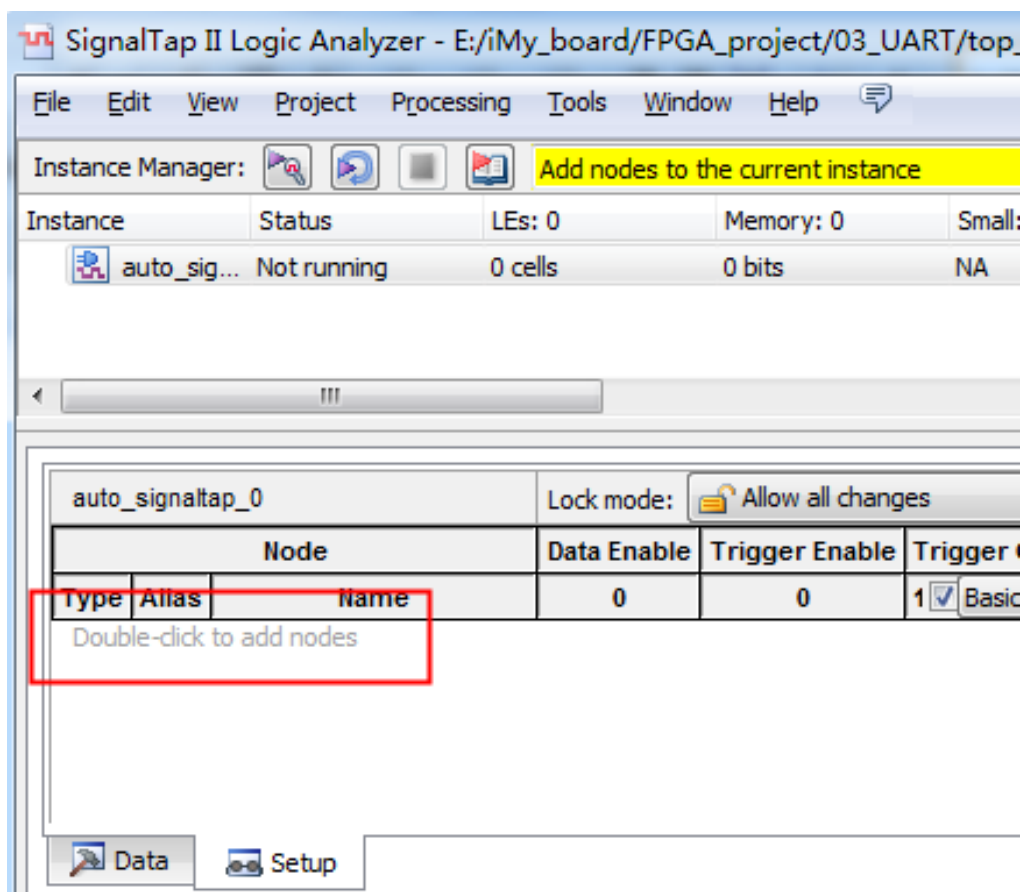
第六步：选择采样时钟完成；

第七步：选择采样深度，这里选择 128；

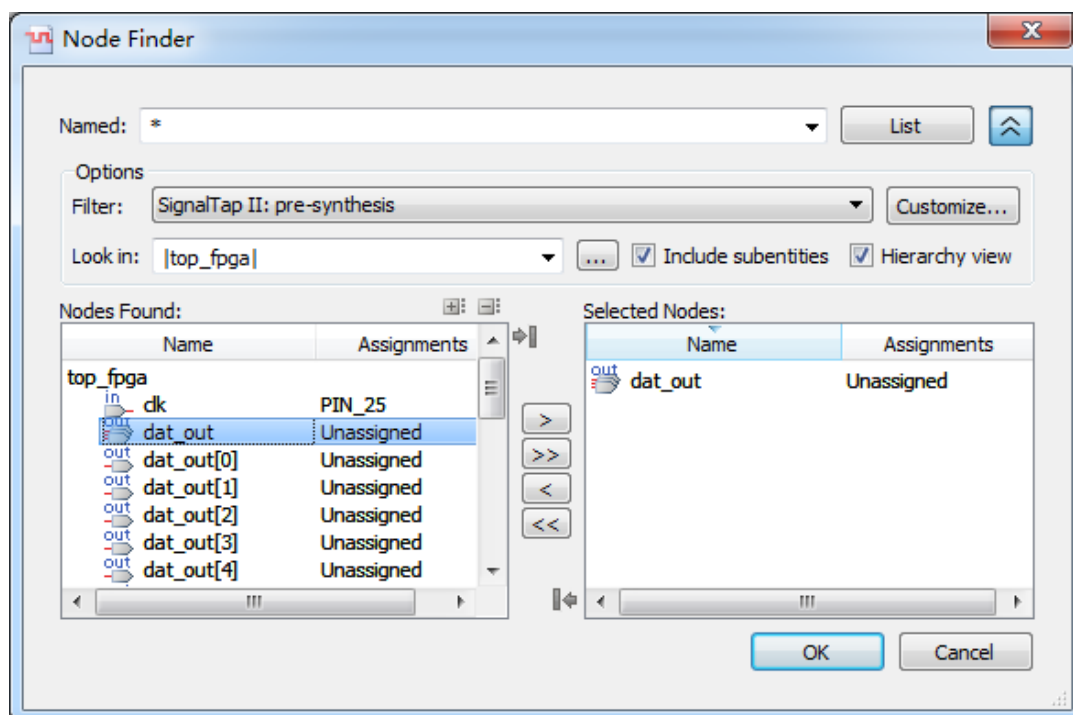
第八步：选择触发方式，这里选择连续触发方式(Continuous)。

➤ 添加被观察信号

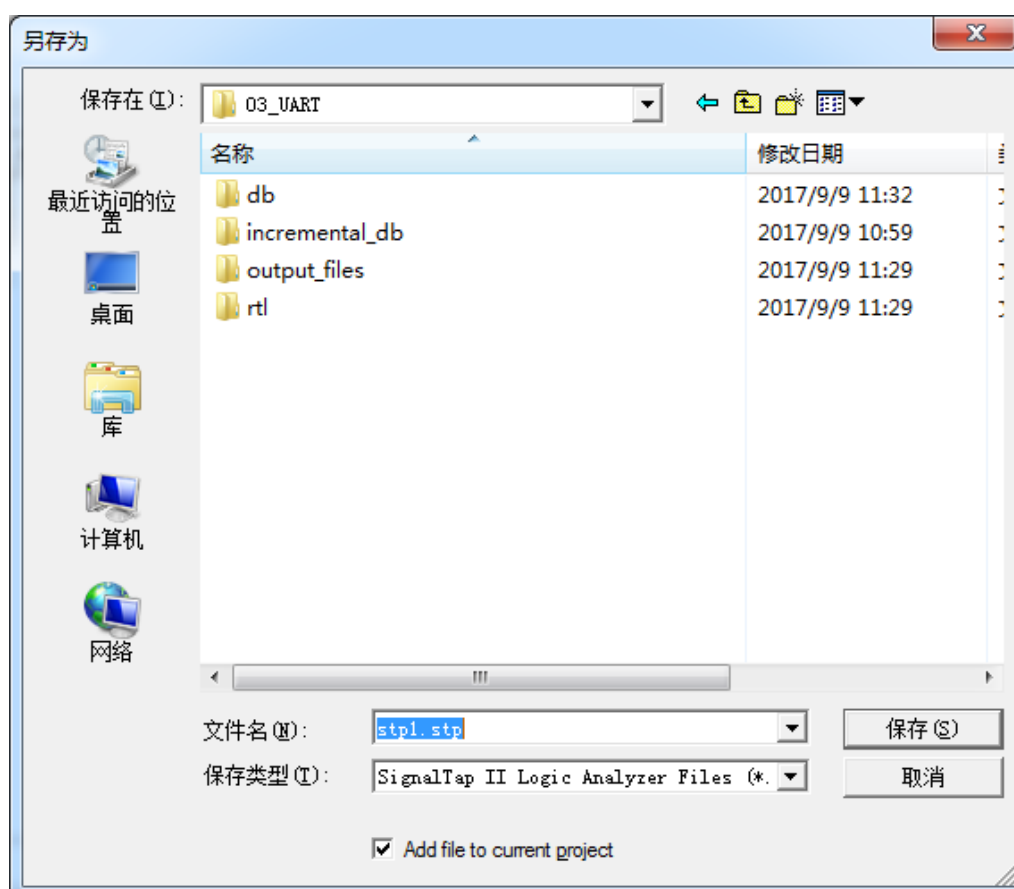
在 Setup 空白区域双击，弹出信号选择窗口；



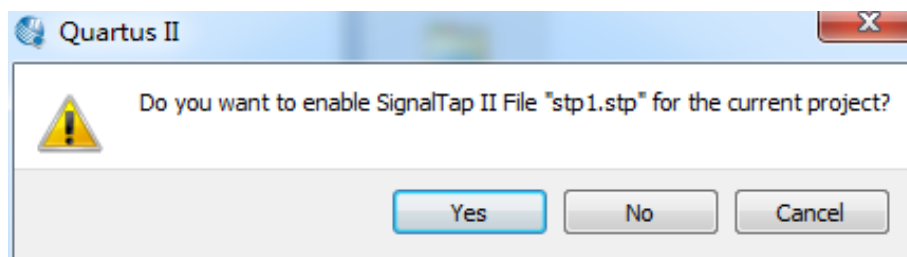
选择需要观察的信号，这里选择 dat_out。



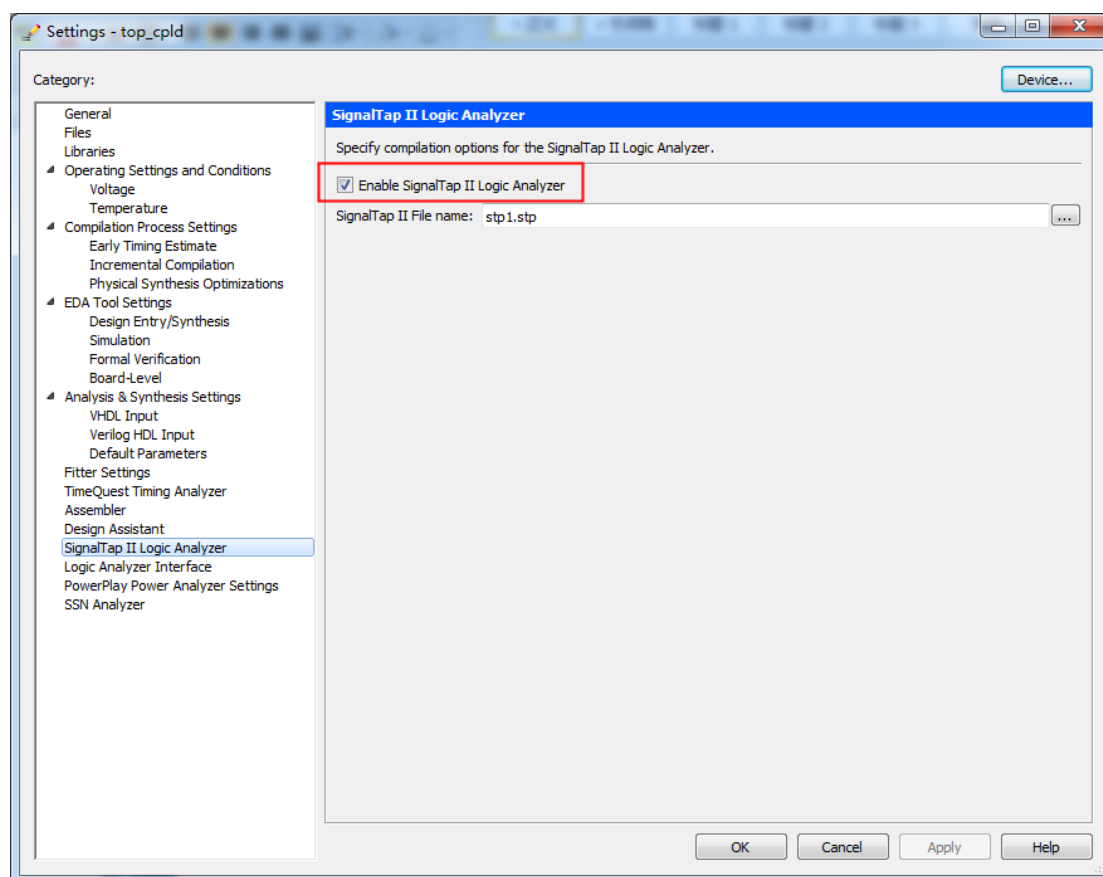
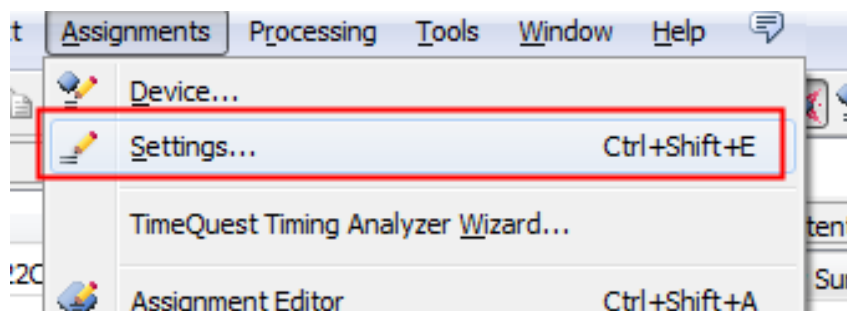
➤ 保存 SignalTap II 文件



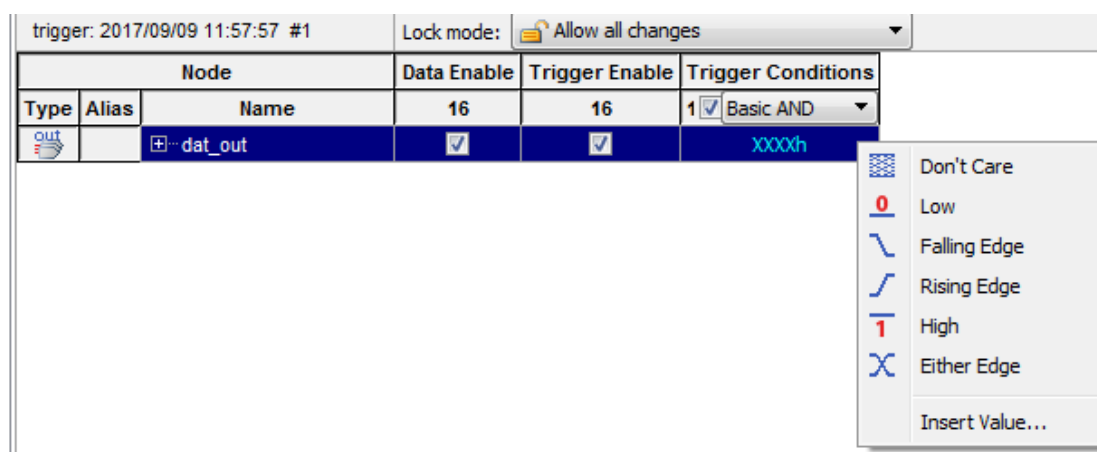
保存好 SignalTap II 文件后，Quartus II 会自动弹出“是否打开 SignalTap II 功能”对话框，这里点击‘Yes’。



如果要关闭 SignalTap II 功能，可在 Settings 中关闭。



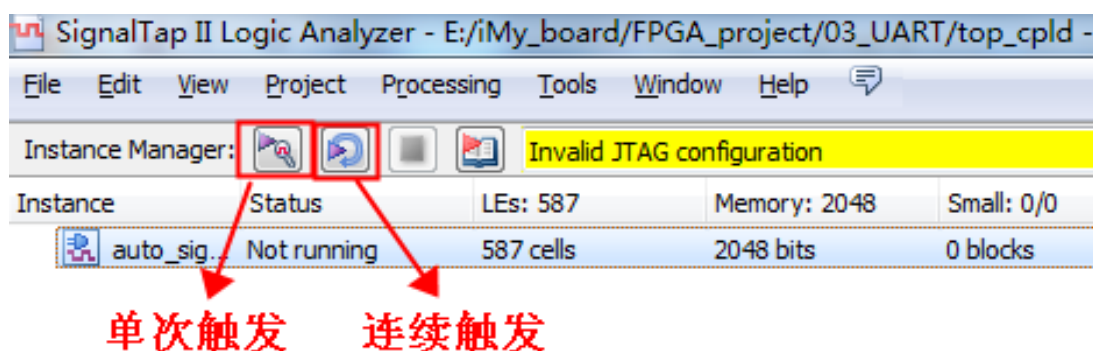
➤ 选择信号的触发条件



➤ 编译工程

添加完 SignalTap II 文件或修改了 SignalTap II 文件的内容，都需要重新编译工程。

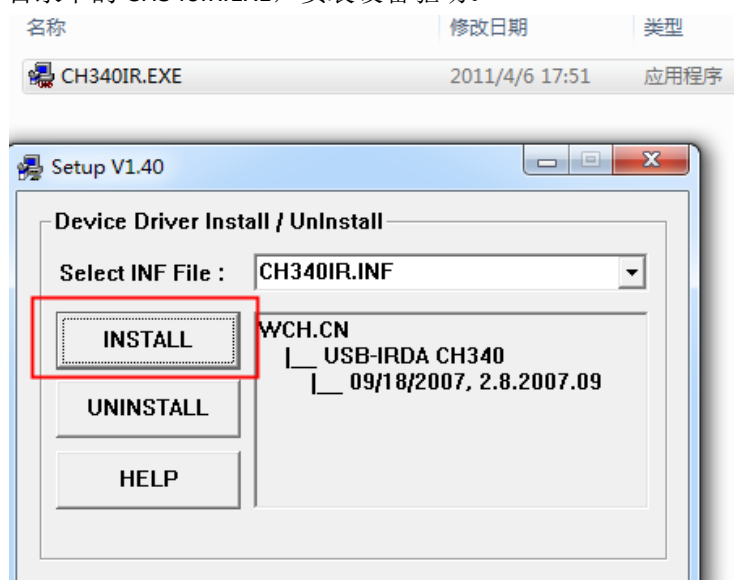
➤ 运行 SignalTap II



10.6. 实验操作

➤ 安装 USB 转串口驱动

双击 Tools 目录下的 CH340IR.EXE，安装设备驱动。



安装好驱动后，再插上 USB 接口，在设备管理器中可以查看 USB-SERIAL 设备。

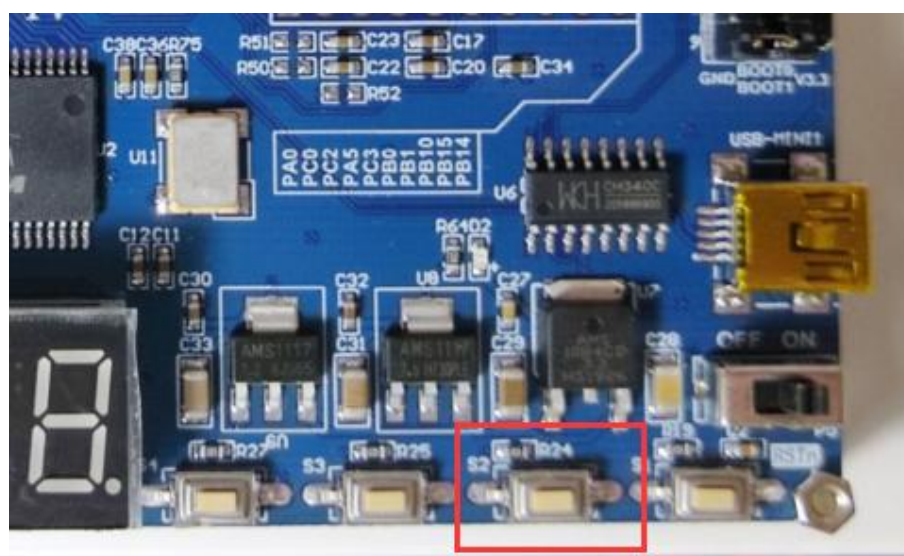


➤ 数据发送与接收

打开 Tools 目录下的‘串口调试助手’，选择正确的‘端口’ (端口号可在设备管理器的端口中找到 USB-BERIAL CH340 (COMX))、波特率(这里设置成 9600)、校验位(None)、数据位(8)、停止位(1)。

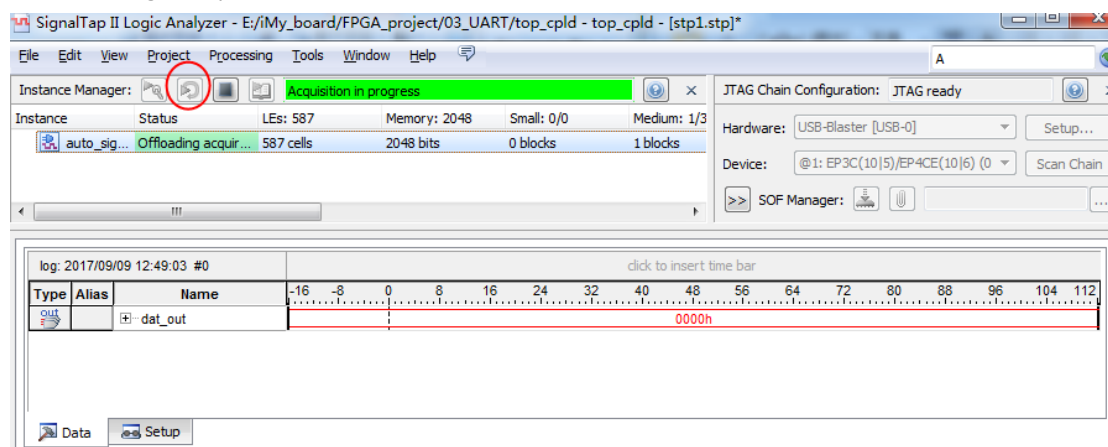
点击‘打开串口’，旁边的黑点会变红。

在接收区勾选‘十六进制显示’，按下开发板的按键 S2，则在接受显示区域显示接收的字节数据，这里的现象是，每按一次按键，显示一个固定字节 0x4A(因 USB 串口同时被 ARM 使用，所以需要将 STM32 下载为 LED 闪烁的例子，不然 FPGA 无法使用)。





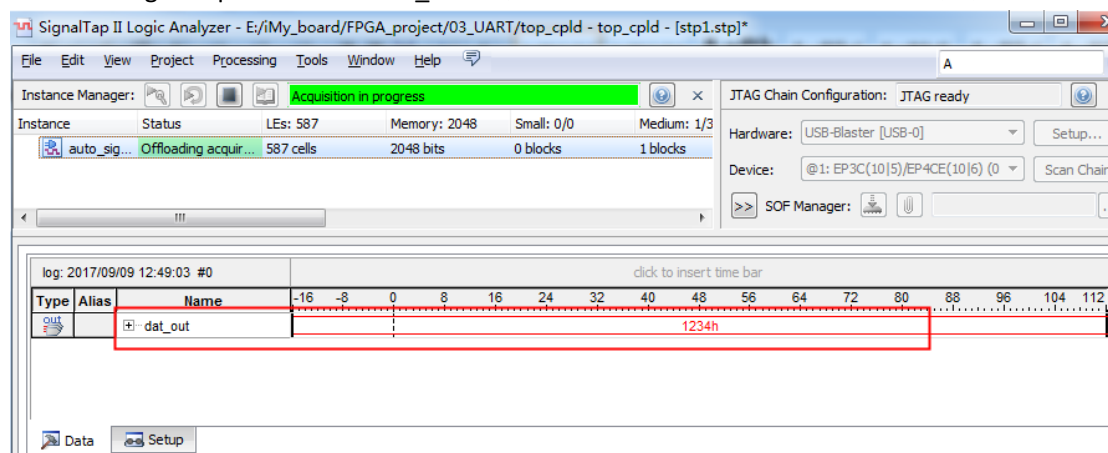
打开 SignalTap II 界面，点击‘连续触发’按钮。



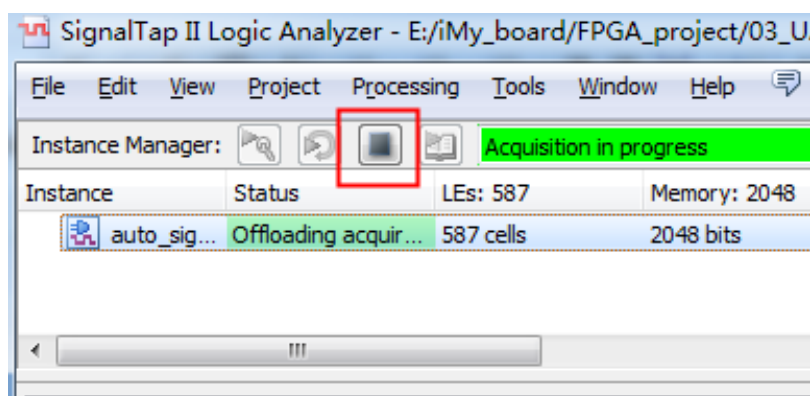
在串口调试助手的发送区勾选‘十六进制发送’，输入发送数据 1234，点击‘手动发送’。



则在 SignalTap II 界面显示 dat_out 的值为 1234，和串口调试助手发送的数据一致。



停止 SignalTap II 抓取数据，点击停止按钮即可。



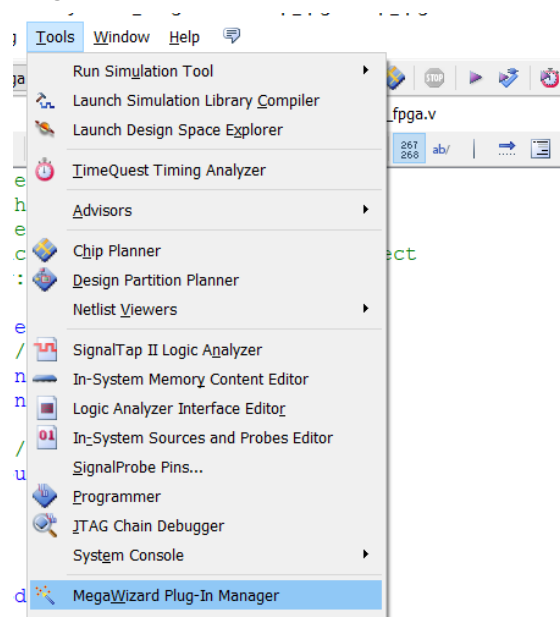
第十一章 MegaWizard

前面章节都没有使用官方的 IP 核,本章开始学习使用官方调用 IP 核的工具 MegaWizard,并学习如何使用 PLL 和 FIFO。

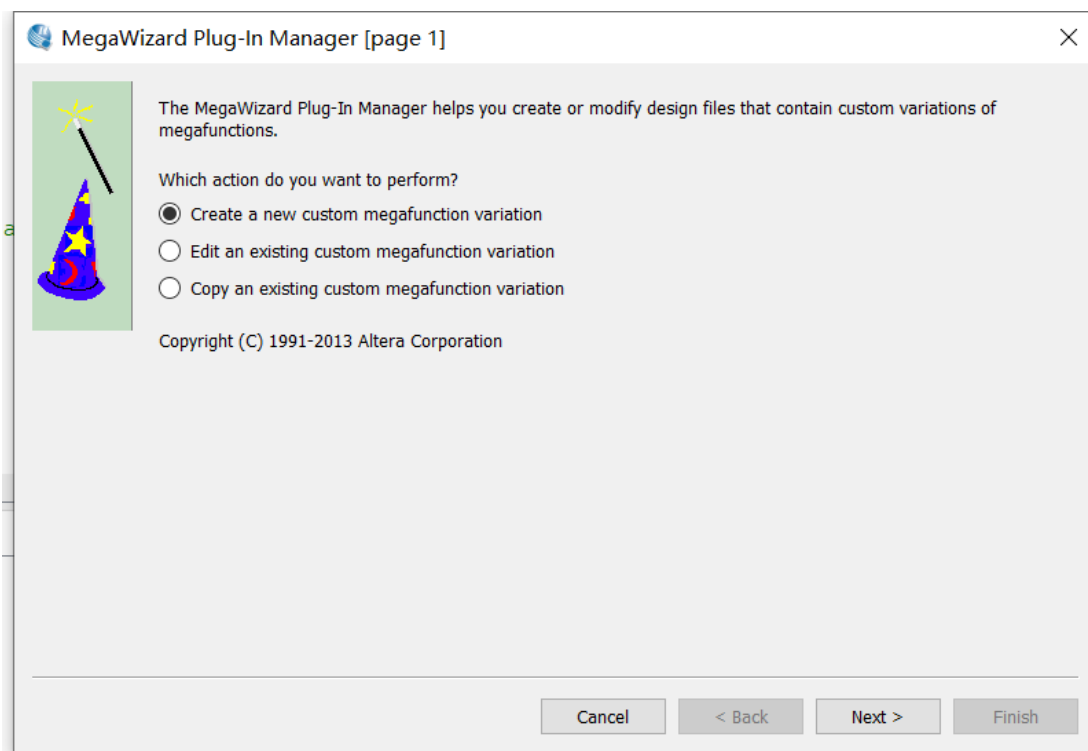
11.1. PLL 设置

PLL 全称为 Phase Locked Loop,即锁相环。PLL 可以对时钟进行倍频、分频、相位偏移和占空比控制等功能。

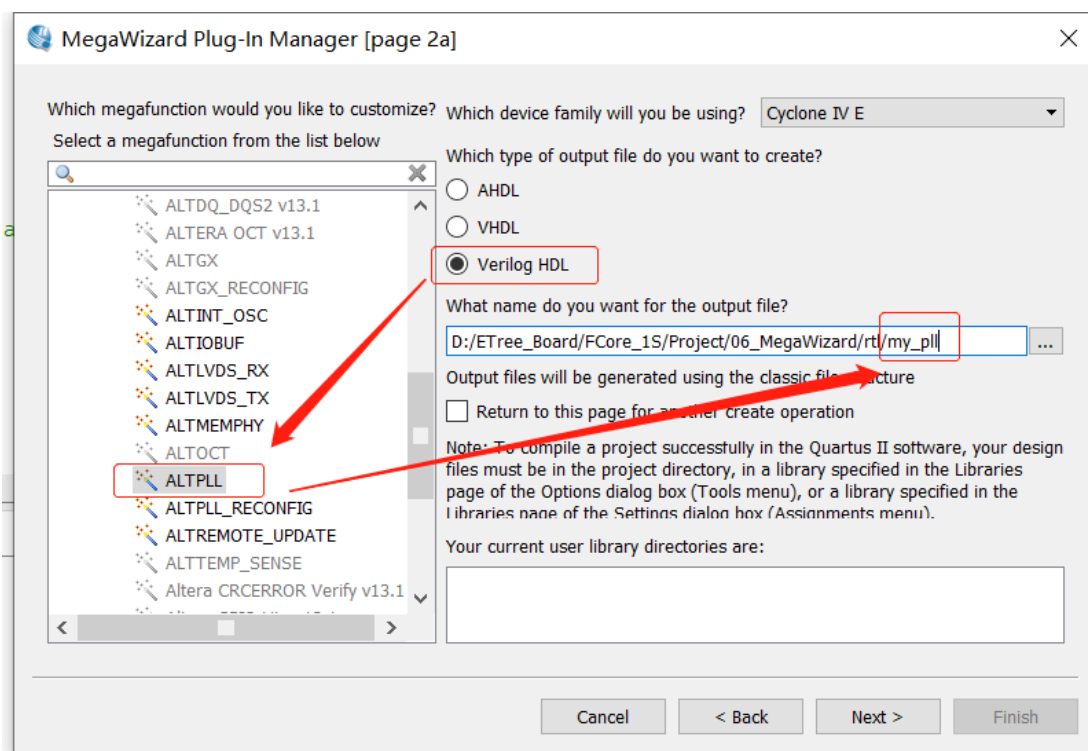
新建一个 quartus 工程,定义时钟输入,点击 Tools|MeagWizard Plug-In Manager 打开 MegaWizard,如下图所示:



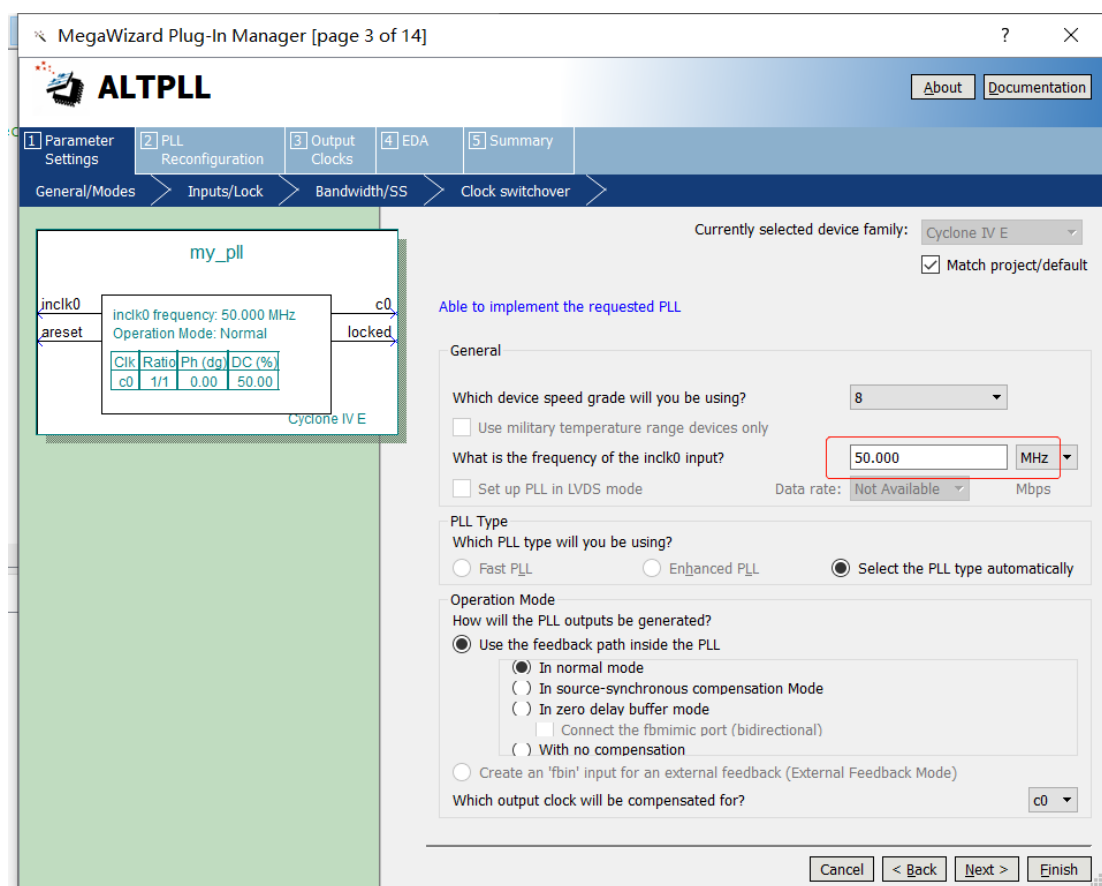
弹出下面对话框,选择“Create a new custom megafunction variation”,点击“Next”,如下图所示:



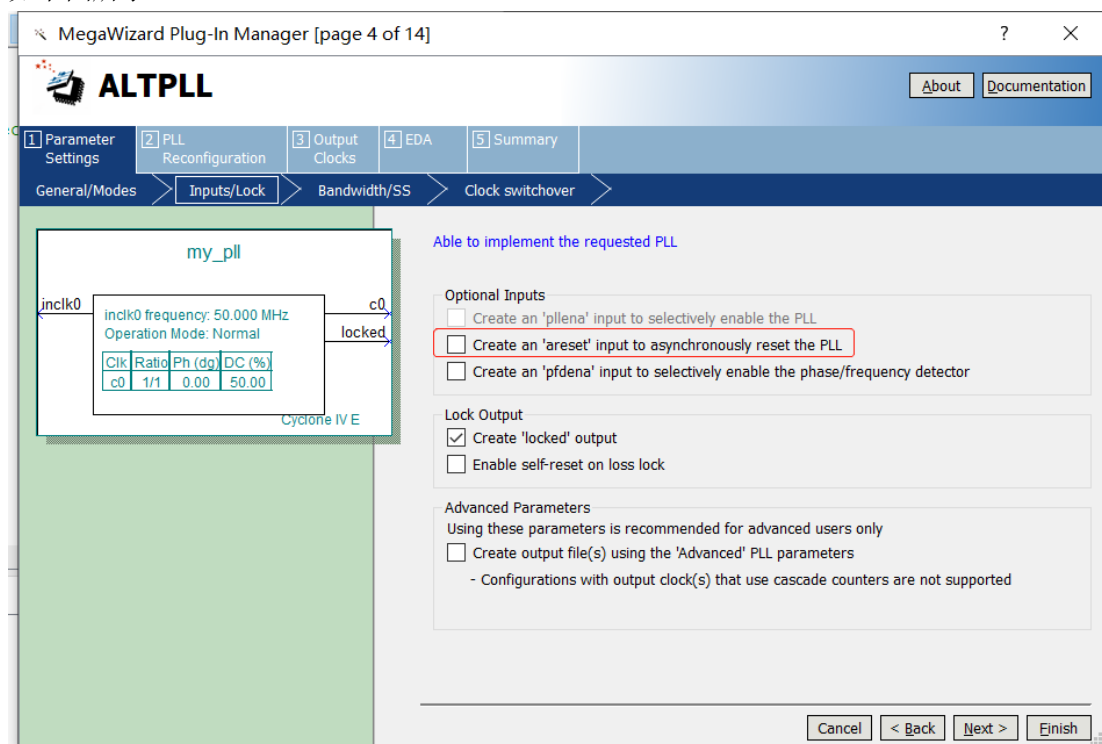
选择开发语言“Verilog HDL”→ALTPLL，输入模块名称为“my_pll”，点击“Next”，如下图所示：



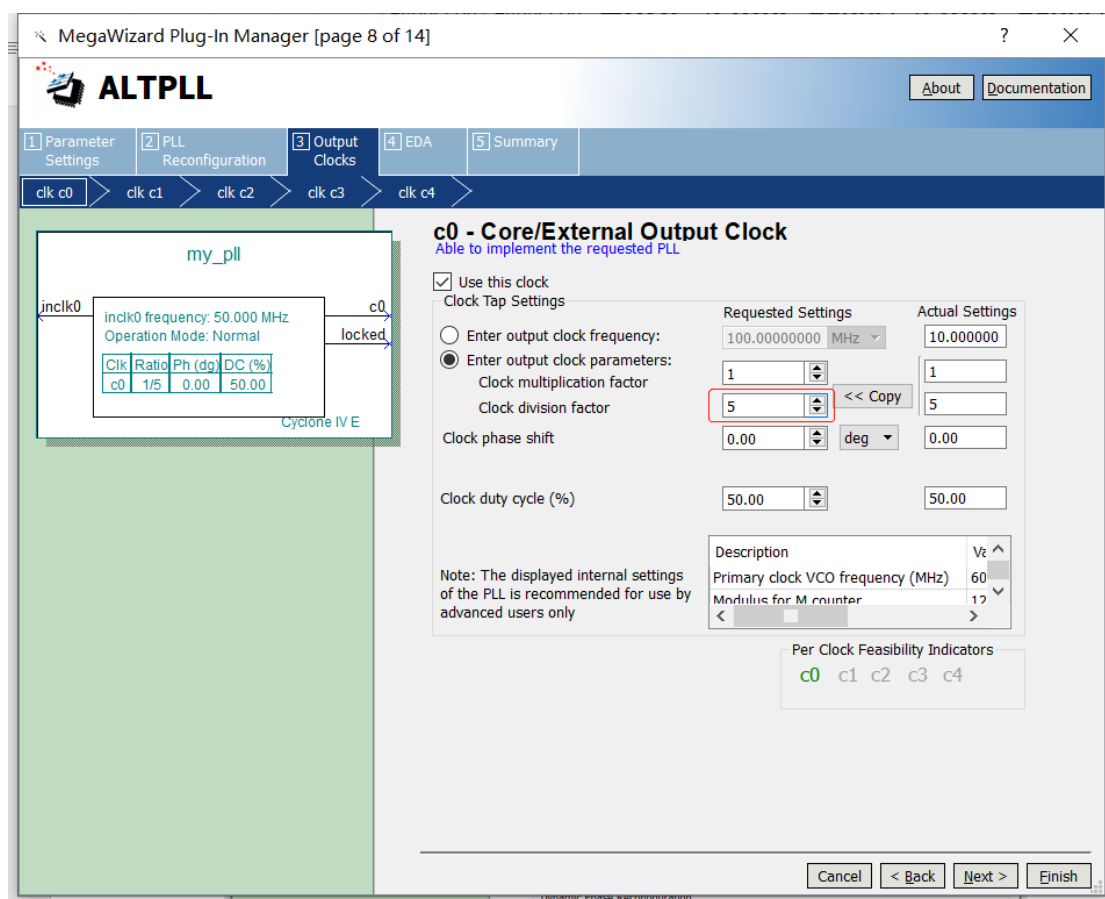
修改输入时钟为 50MHz，点击“Next”，如下图所示：



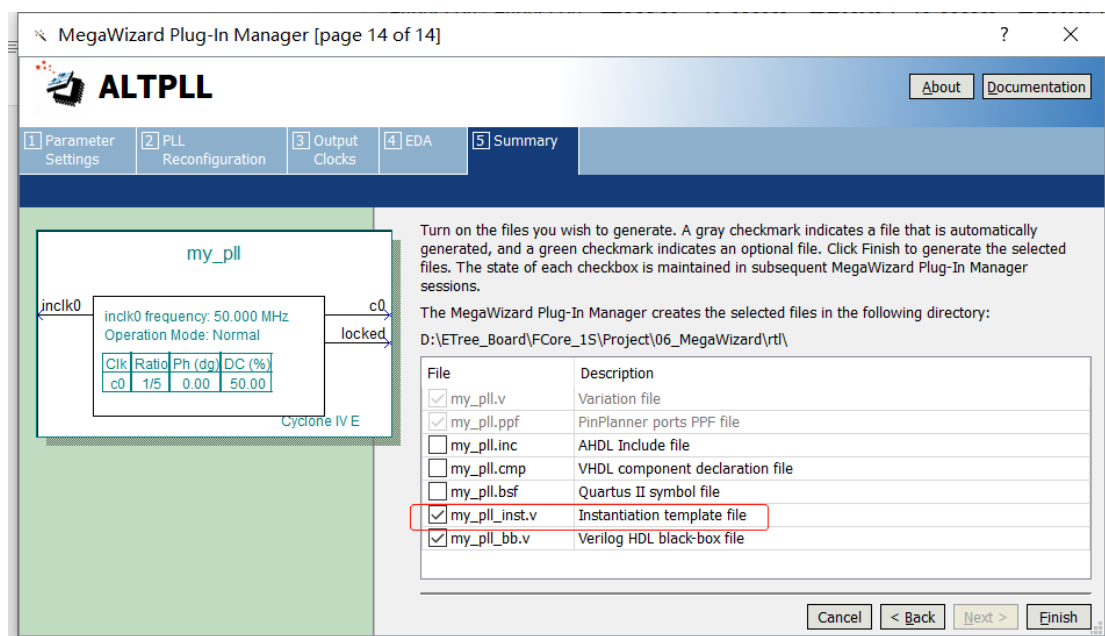
取消勾选“Create an areset input to asynchronously reset the PLL”选项，点击“Next”，如下图所示：



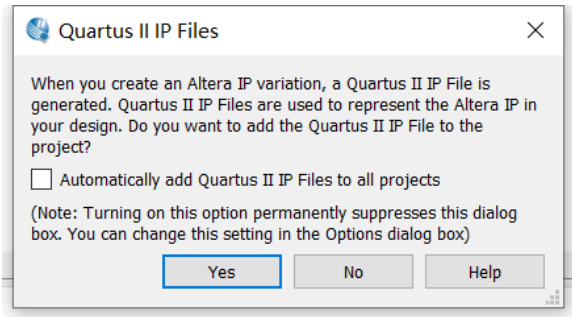
在 Output Clocks 栏下，改变“Clock division factor”为 5，实现 5 分频，点击“Next”，如下图所示：



在 Summary 栏下，勾选 “Instantiation template file”，生产例化文件，点击 “Finish” 按钮，如下图所示：



点击 “Yes” 将 pll 添加到工程中，如下图所示：



11.2. PLL 例化

在设置 PLL 生成目录下，找到刚才生产的 PLL 的例化文件，如下图所示：

此电脑 > Data (D:) > ETree_Board > FCore_1S > Project > 06_MegaWizard > rtl				
名称	修改日期	类型	大小	
my_pll.ppf	2022/4/4 14:25	PPF 文件	1 KB	
my_pll.qip	2022/4/4 14:25	QIP 文件	1 KB	
my_pll.v	2022/4/4 14:25	Verilog File	15 KB	
my_pll_bb.v	2022/4/4 14:25	Verilog File	11 KB	
my_pll_inst.v	2022/4/4 14:25	Verilog File	1 KB	
top_fpga.v	2022/4/4 14:09	Verilog File	1 KB	
top_fpga.v.bak	2017/5/4 20:20	BAK 文件	1 KB	

具体文件内容如下：

```
my_pll  my_pll_inst (
.inclk0 ( inclk0_sig ),
.c0 ( c0_sig ),
.locked ( locked_sig )
);
```

在顶层文件 top_fpga.v 中例化设置好的 PLL，具体代码如下：

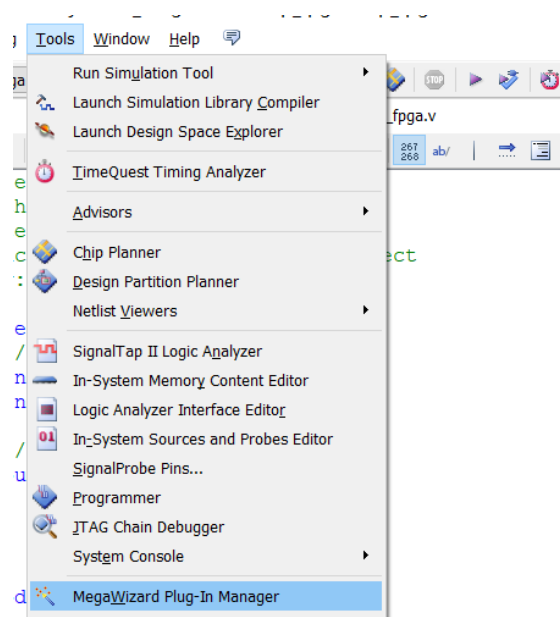
```
wire clk_10m;
wire locked;

my_pll1 my_pll_inst (
.inclk0 ( clk ),
.c0 ( clk_10m ),
.locked ( locked )
);
```

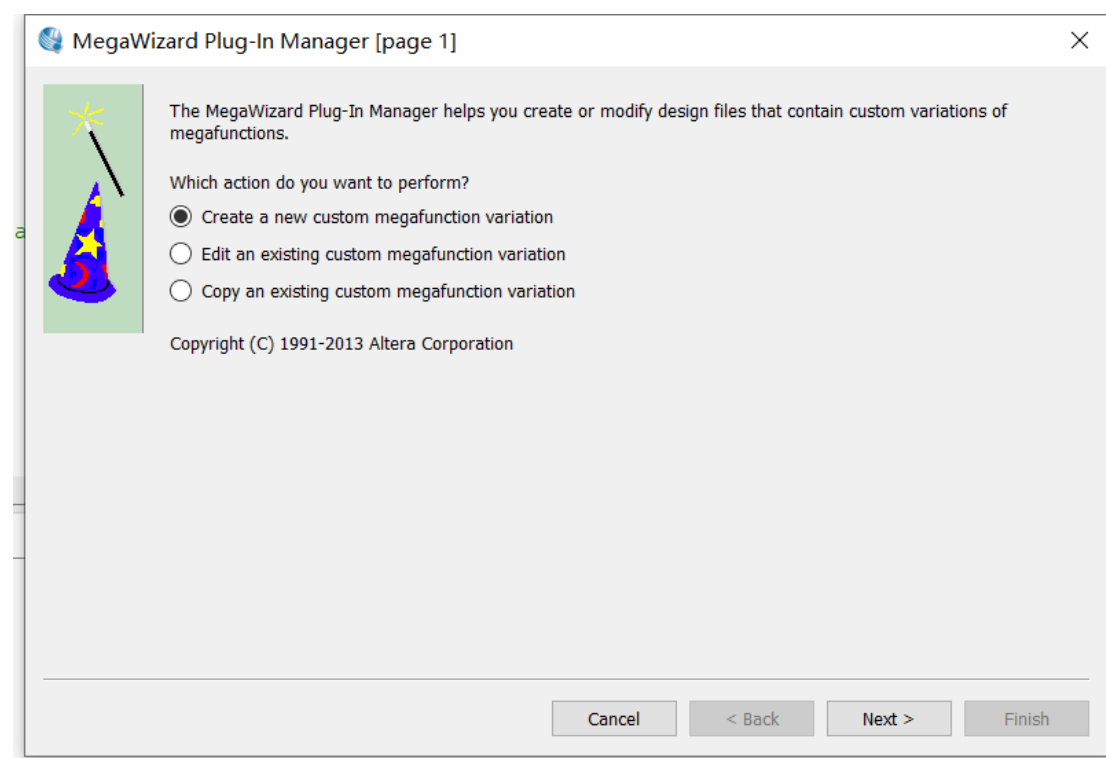
11.3. FIFO 设置

FIFO 全称为 First In First Out，即先进先出，经常被用于数据的缓存或者不同时钟域之间的数据交换。

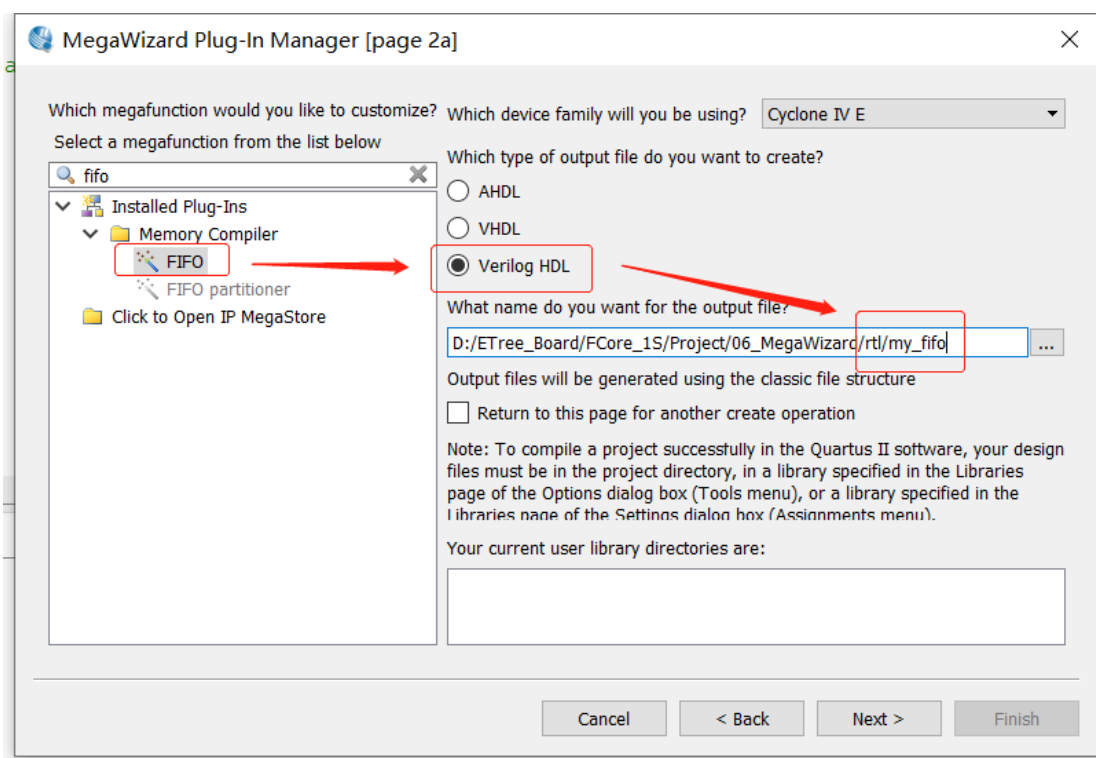
点击 Tools|MeagWizard Plug-In Manager 打开 MegaWizard，如下图所示：



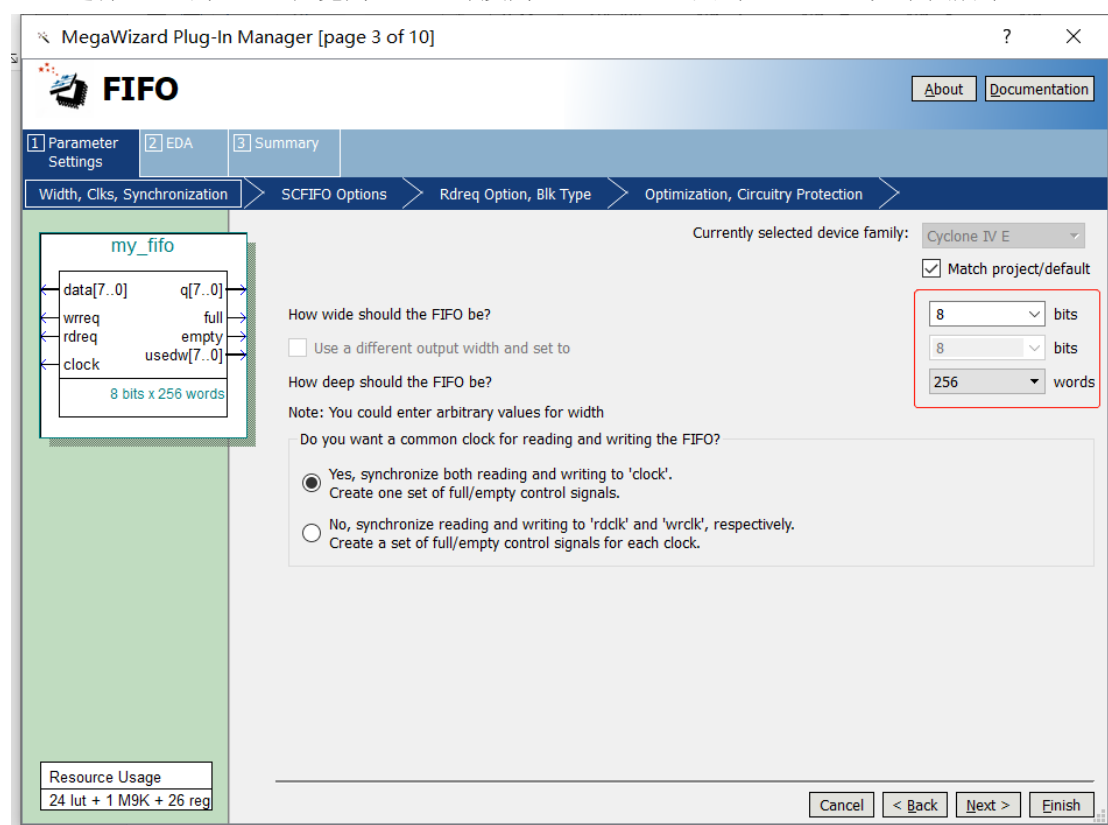
弹出下面对话框，选择“Create a new custom megafunction variation”，点击“Next”，如下图所示：



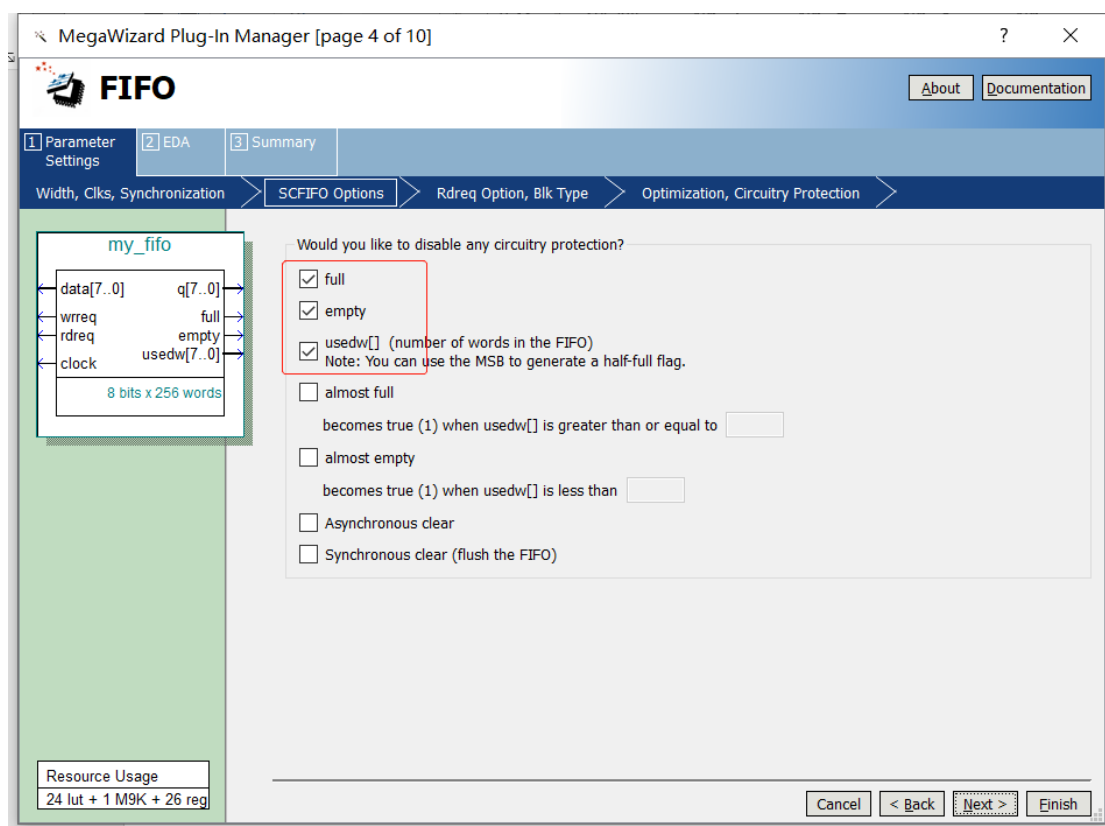
选择开发语言“Verilog HDL”→FIFO，输入模块名称为“my_fifo”，点击“Next”，如下图所示：



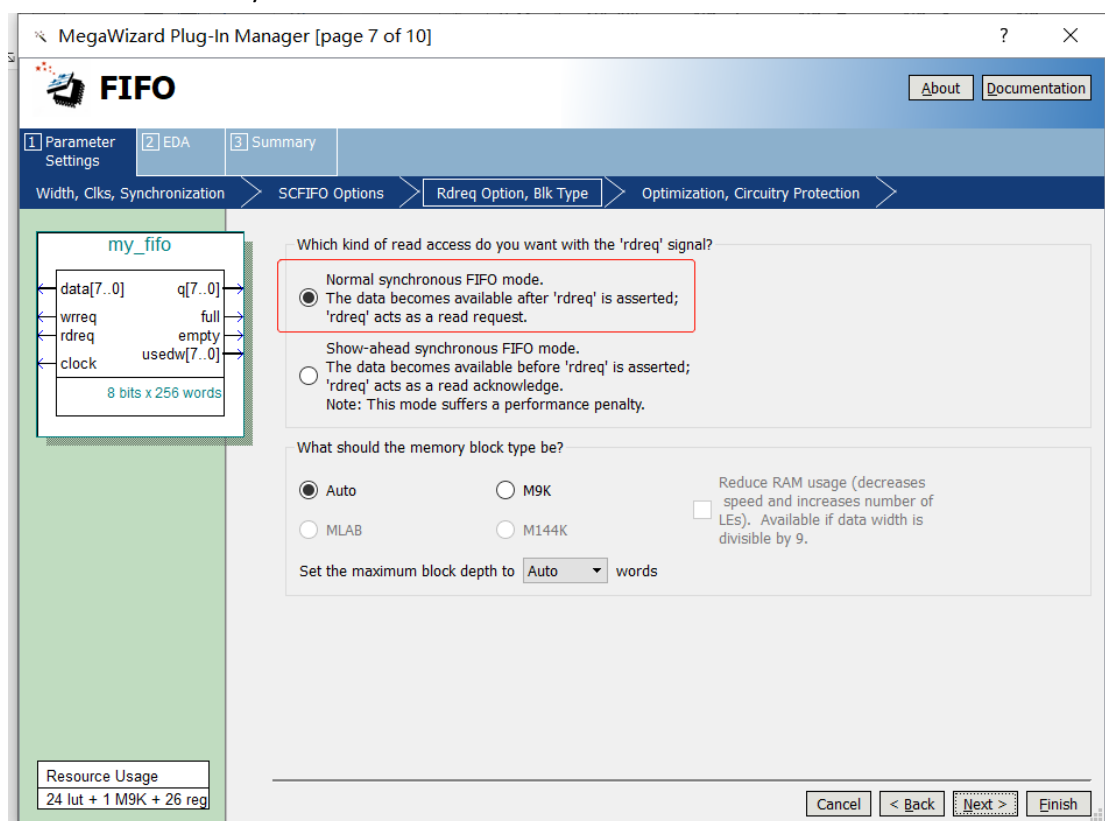
选择 FIFO 的 word 位宽为 8bit，深度为 256 words，点击“Next”，如下图所示：



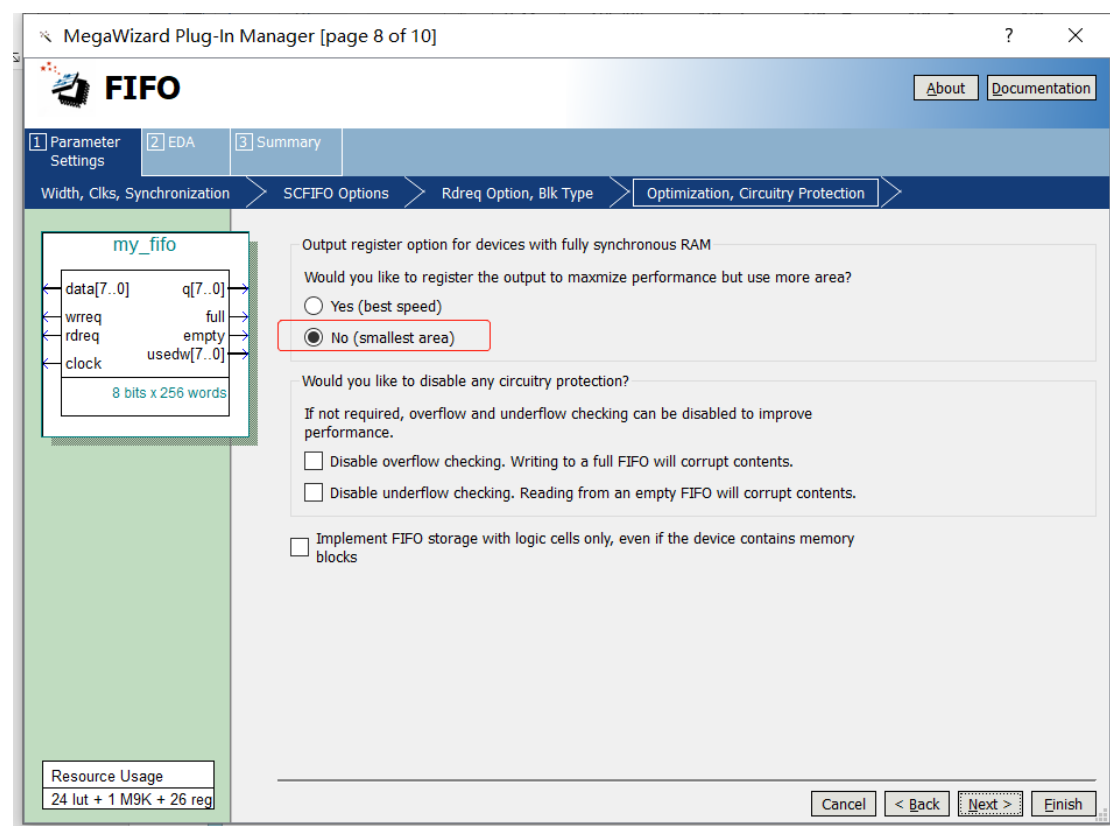
默认勾选“full”、“empty”和“usedw[]”，点击“Next”，如下图所示：



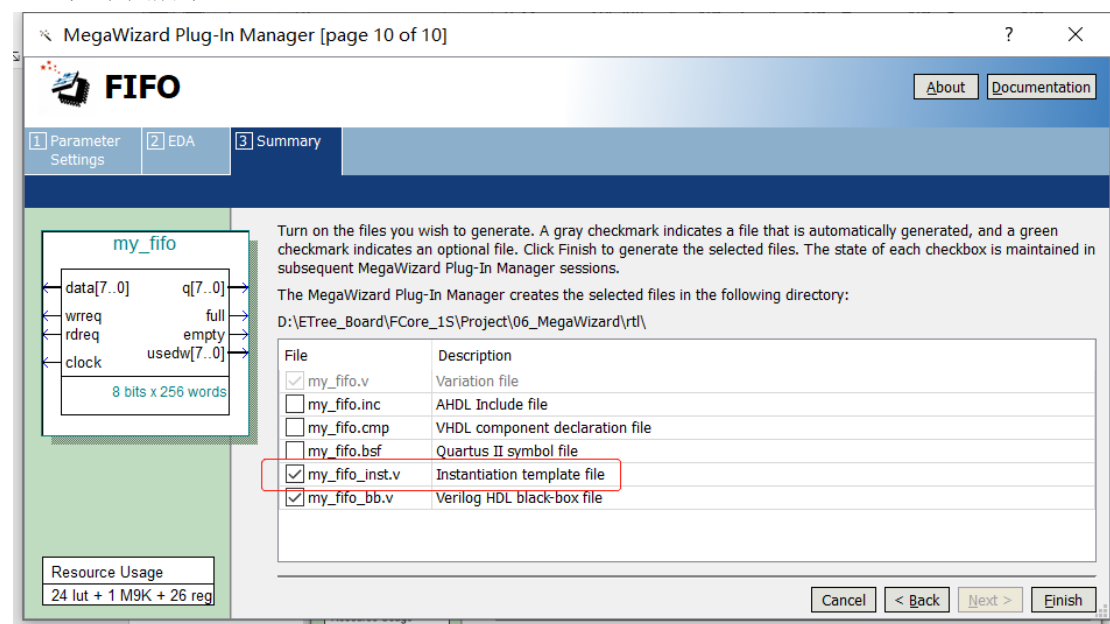
勾选“Normal synchronous FIFO mode”，点击“Next”，如下图所示：



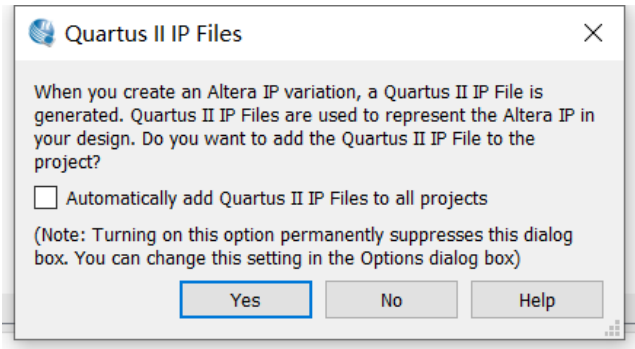
勾选“smallest area”，点击“Next”如下图所示：



在 Summary 栏下，勾选 “Instantiation template file”，生产例化文件，点击 “Finish” 按钮，如下图所示：



点击 “Yes” 将 FIFO 添加到工程中，如下图所示：



11.4. FIFO 例化

在设置 FIFO 生成目录下，找到刚才生产的 FIFO 的例化文件，如下图所示：

此电脑 > Data (D:) > ETree_Board > FCore_1S > Project > 06_MegaWizard > rtl

名称	修改日期	类型	大小
greybox_tmp	2022/4/4 14:36	文件夹	
my_fifo.qip	2022/4/4 14:43	QIP 文件	1 KB
my_fifo.v	2022/4/4 14:43	Verilog File	7 KB
my_fifo_bb.v	2022/4/4 14:43	Verilog File	6 KB
my_fifo_inst.v	2022/4/4 14:43	Verilog File	1 KB
my_pll.ppf	2022/4/4 14:25	PPF 文件	1 KB
my_pll.qip	2022/4/4 14:25	QIP 文件	1 KB
my_pll.v	2022/4/4 14:25	Verilog File	15 KB
my_pll_bb.v	2022/4/4 14:25	Verilog File	11 KB
my_pll_inst.v	2022/4/4 14:25	Verilog File	1 KB
top_fpga.v	2022/4/4 14:09	Verilog File	1 KB
top_fpga.v.bak	2017/5/4 20:20	BAK 文件	1 KB

具体文件内容如下：

```
my_fifo my_fifo_inst (
    .clock ( clock_sig ),
    .data ( data_sig ),
    .rdreq ( rdreq_sig ),
    .wrreq ( wrreq_sig ),
    .empty ( empty_sig ),
    .full ( full_sig ),
    .q ( q_sig ),
    .usedw ( usedw_sig )
);
```

在顶层文件 top_fpga.v 中例化设置好的 FIFO，具体代码如下：

```
wire [7:0] fifo_data; //FIFO 数据输入
wire fifo_rdreq;      //FIFO 读使能
wire fifo_wrreq;      //FIFO 写使能
wire full;            //FIFO 满
wire empty;           //FIFO 空
```

```
wire [7:0] fifo_q;    //FIFO 数据输出
wire [7:0] fifo_usedw;//FIFO 中的数据个数

assign fifo_data = data_in;
assign data_out  = fifo_q;
assign fifo_rdreq = rdreq;
assign fifo_wrreq = wrreq;

my_fifo my_fifo_inst (
    .clock ( clk_10m ),
    .data ( fifo_data ),
    .rdreq ( fifo_rdreq ),
    .wrreq ( fifo_wrreq ),
    .empty ( empty ),
    .full ( full ),
    .q ( fifo_q ),
    .usedw ( fifo_usedw )
);

assign led = {empty,full,2'b00};
```

11.5. 测试激励

测试激励代码需要生成时钟、复位、FIFO 的读写控制、FIFO 的数据输入，下面是写好的测试激励代码：

```
`timescale 1 ns/ 1 ps
module top_fpga_vlg_tst();
// constants
// general purpose registers
reg [9:0] cnt=0;
// test vector input registers
reg clk;
reg rst_n;
reg [7:0] data_in=0;
reg rdreq=0;
reg wrreq=0;

// wires
wire [3:0] led;
wire [7:0] data_out;
wire clk_out;

// assign statements (if any)
top_fpga i1 (
```

```
// port map - connection between master ports and signals/registers
.clk(clk),
.led(led),
.rst_n(rst_n),
.data_in(data_in),
.data_out(data_out),
.rdreq(rdreq),
.wrreq(wrreq),
.clk_out(clk_out)
);

//时钟生成
initial
begin
    clk = 0;
    forever clk = #(10)~clk;
    $display("Running testbench");
end

//复位生成
initial
begin
    rst_n = 0;
    #50;
    rst_n = 1;
    #1000;
    rst_n = 1;
end

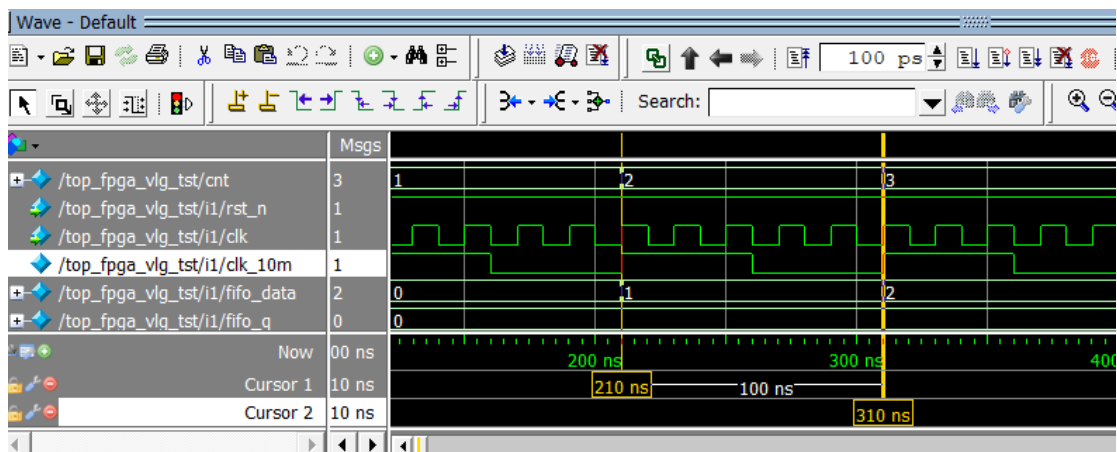
always @(posedge clk_out)
    if(~rst_n)
        cnt <= 0;
    else
        cnt <= cnt + 1'b1;

always @(posedge clk_out)
    if(~rst_n)
    begin
        data_in <= 0;
        rdreq <= 0;
        wrreq <= 0;
    end
end
```

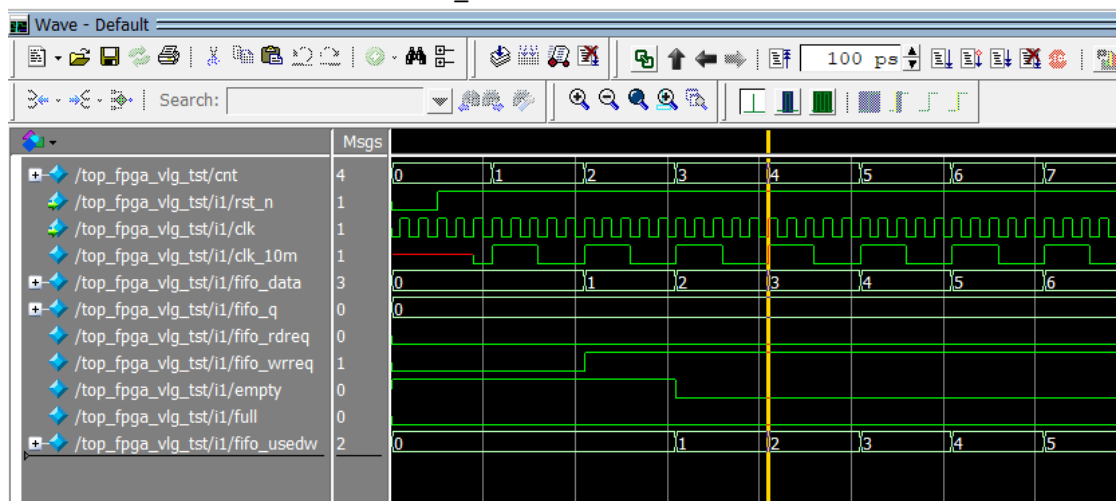
```
else if(cnt>=1 && cnt<=32)
begin
    data_in <= cnt[7:0];
    wrreq <= 1'b1;
    rdreq <= 0;
end
else if(cnt>=38 && cnt<=69)
begin
    data_in <= 0;
    wrreq <= 0;
    rdreq <= 1'b1;
end
else if(cnt>=76 && cnt<=(76+257))
begin
    data_in <= cnt[7:0];
    wrreq <= 1'b1;
    rdreq <= 0;
end
else if(cnt>=(76+257+6) && cnt<=(76+257+6+16))
begin
    data_in <= 0;
    wrreq <= 0;
    rdreq <= 1'b1;
end
else
begin
    data_in <= 0;
    wrreq <= 0;
    rdreq <= 0;
end
endmodule
```

11.6. 仿真结果

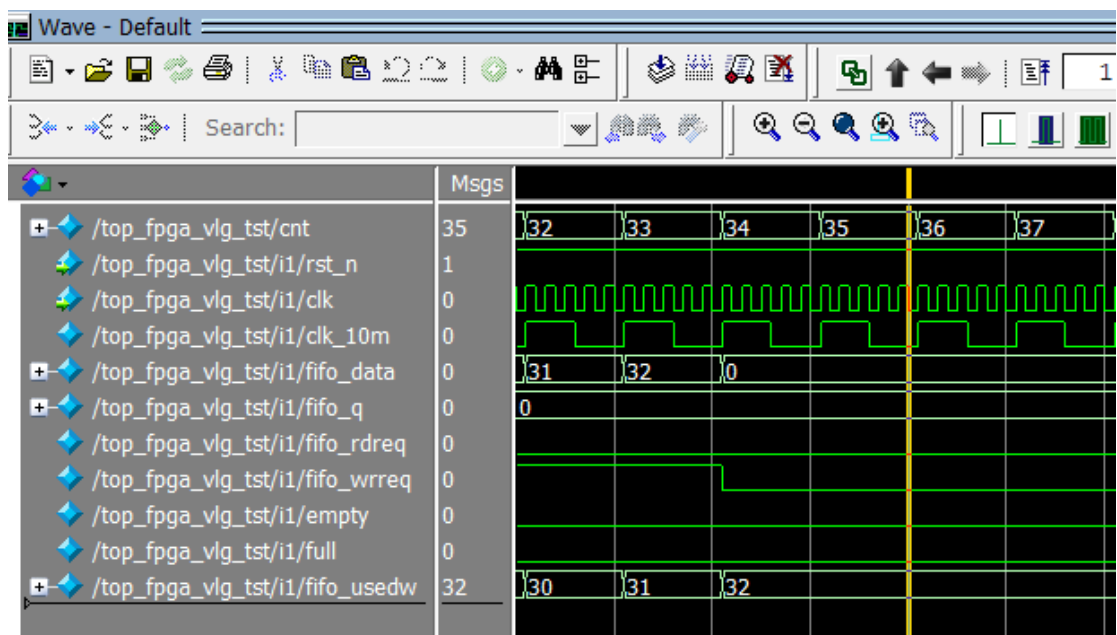
先看一下 clk_10m 是否正确, 下图为 clk_10m 的仿真波形, 可以看出时钟周期为 100ns, 正好是 clk 的 5 分频, 符合 PLL 设置要求。



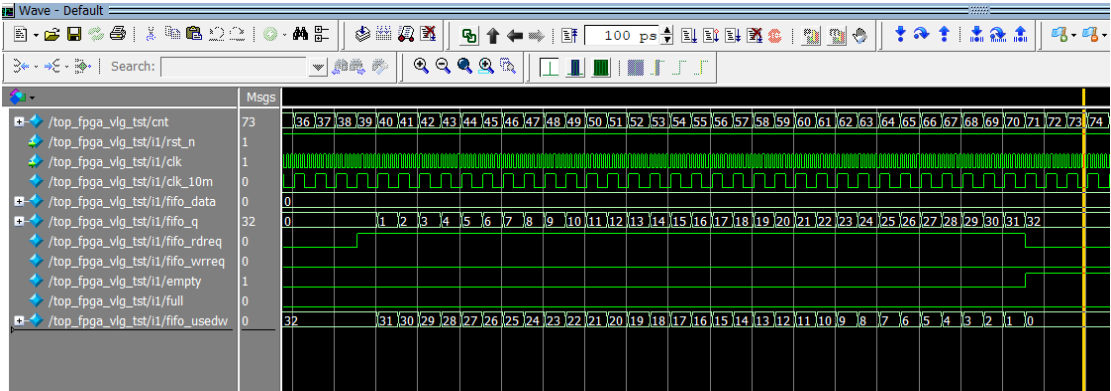
再来看一下 FIFO 的写时序，从下图可以看出，当 `fifo_wrreq` 无效时，FIFO 的 `empty` 为高，表示 FIFO 中没有数据，同时 `fifo_usedw` 也为 0。



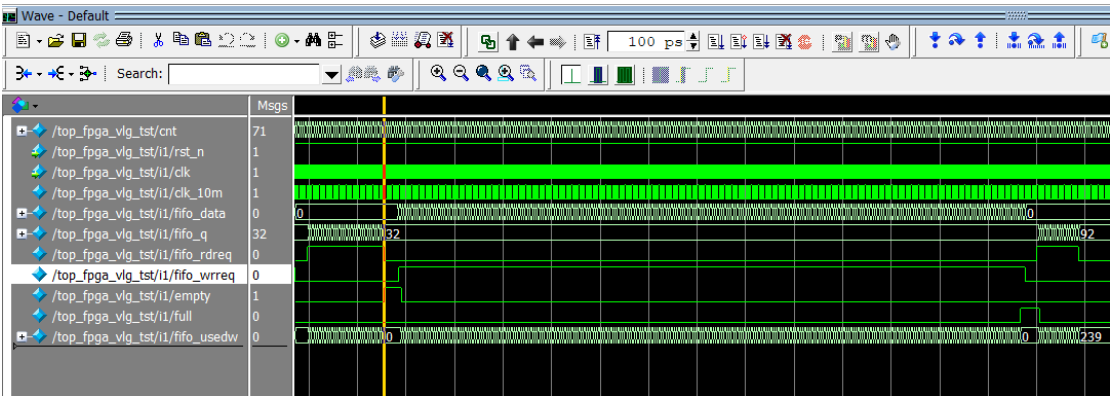
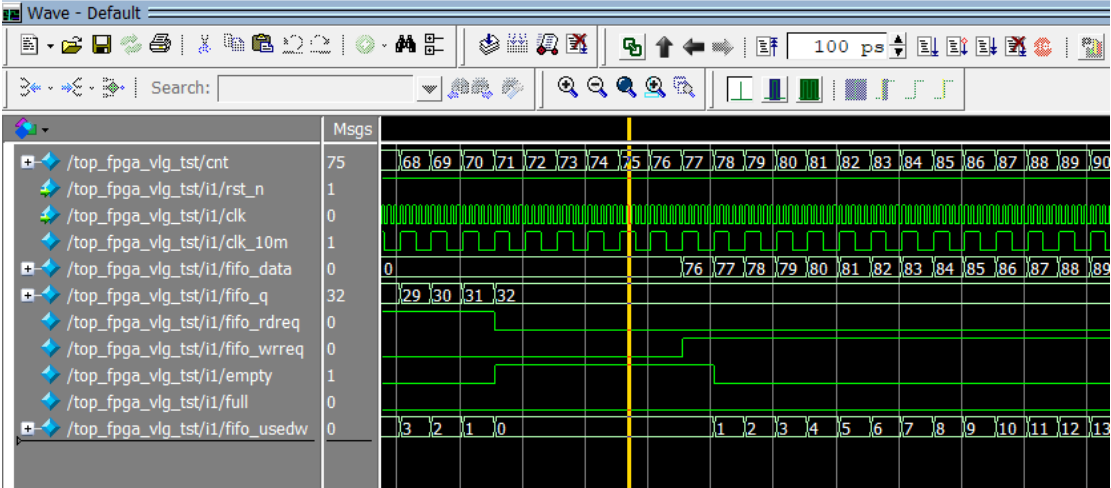
下图为 `fifo_wrrreq` 写完 32 个数据后，FIFO 中存储了 32 个连续的渐进数 1~32，同时 `fifo_usedw` 为 32，表示 FIFO 中有 32 个数据，`empty` 也变低。



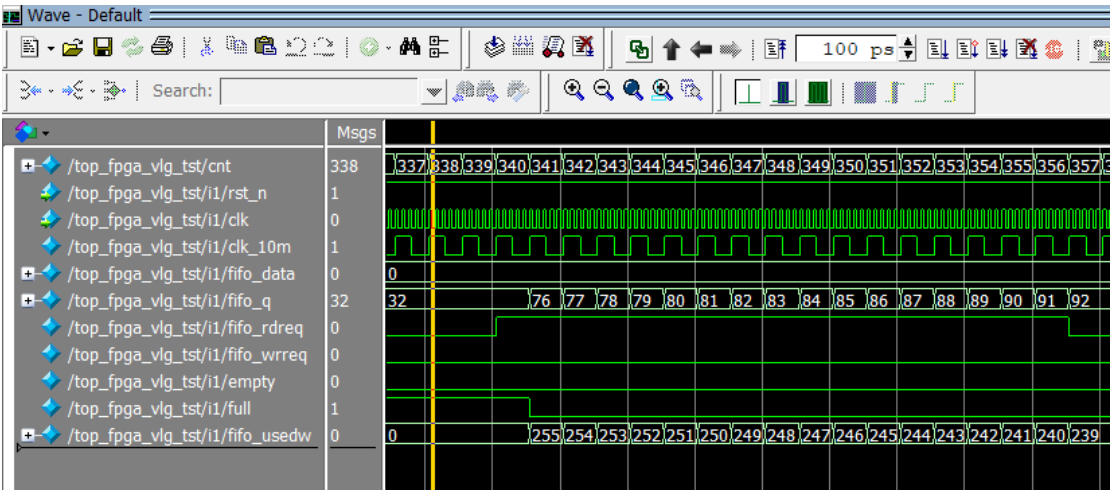
下图为 fifo_rdreq 读取 32 个数据, fifo_q 连续输出 32 个 1~32 渐近数据, 和之前 fifo_wrreq 写入的数据一致。当读取完后, empty 被拉低, 表示 FIFO 被读空, 同时 fifo_usedw 也变成了 0。



下图为 fifo_wrreq 往 FIFO 中写入了 258 个数据, full 被拉高, 表示 FIFO 被写满了。



下图为接着 fifo_rdreq 再读取部分数据, 可以看出数据从 76 开始, 和上次 fifo_wrreq 写入的第一个数据一样。

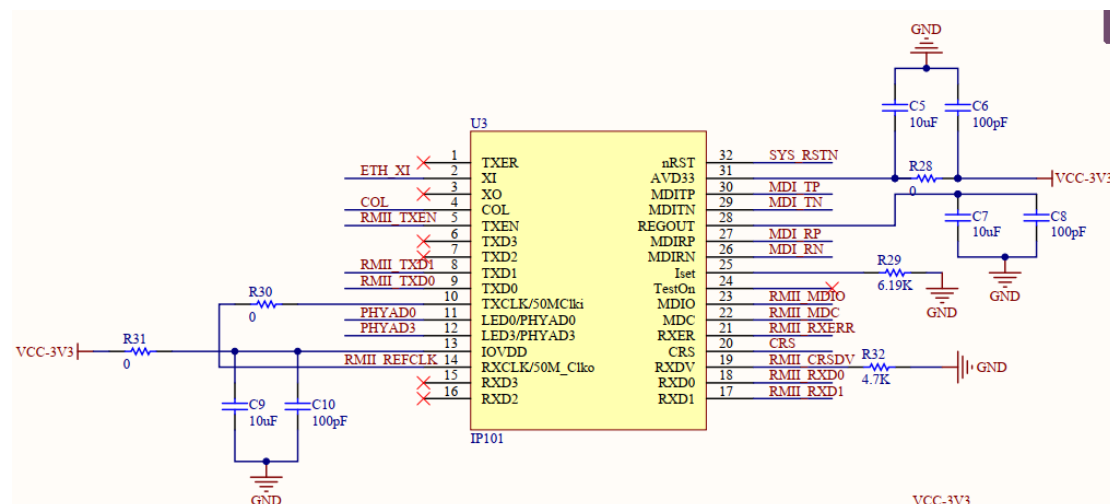


第十二章 以太网环回测试

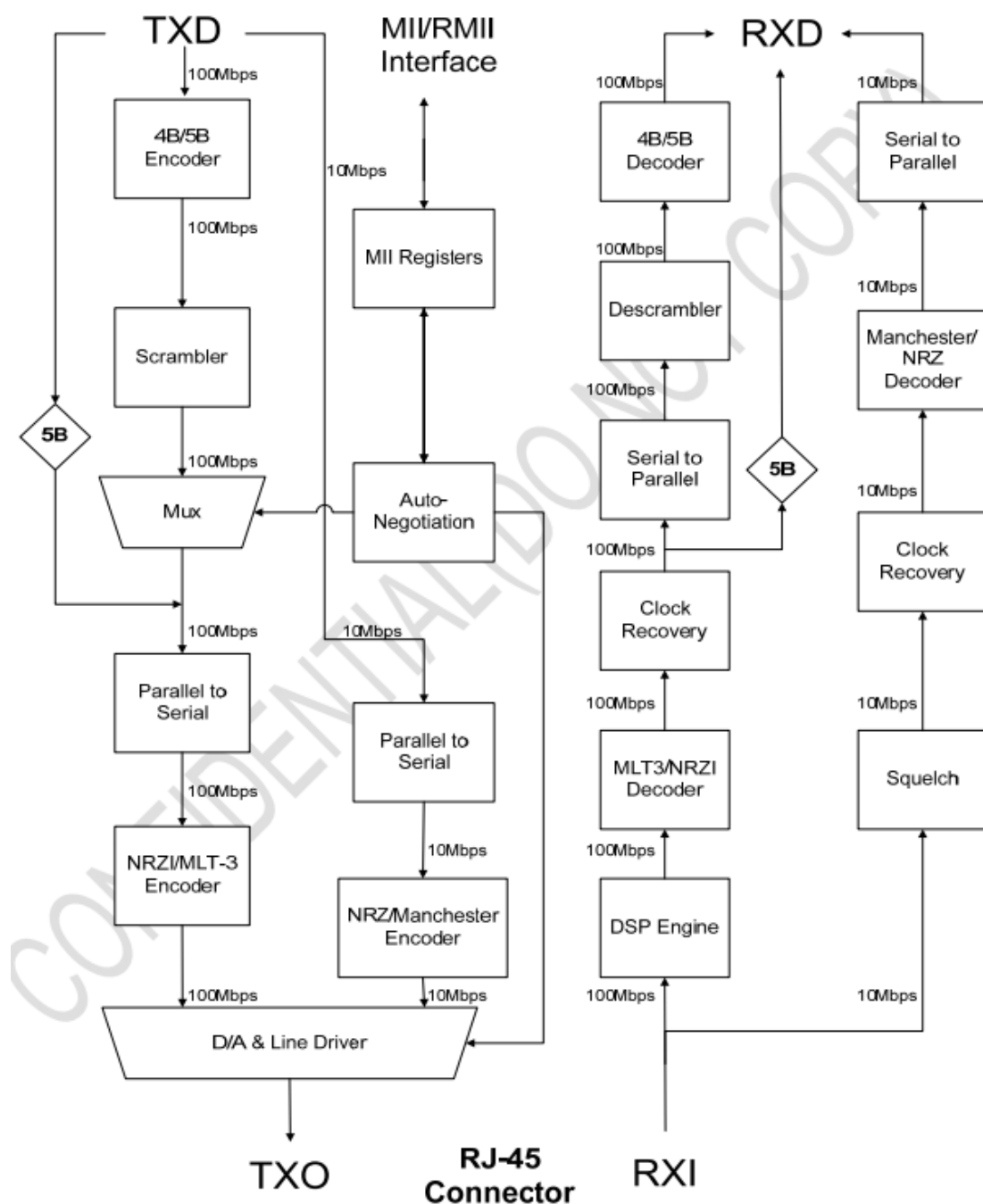
本章基于 IP101GR 百兆网卡，实现以太网 UDP 包回环测试。PC 通过网络调试助手发送给 FPGA 开发板，FPGA 再将接受到数据发给 PC，通过网络调试助手观察结果。

12.1. 硬件介绍

硬件电路如下图所示，采用 IP101GR 作为百兆网卡通信芯片。



IP101GR 是一个单端口的物理层收发器，它只有一个 RMII/MII 接口。它通过网络变压器接 RJ45 来实现全部的 10/100M 以太网物理层功能，包括物理层编码子层(PCS)，物理层介质连接设备(PMA)，双绞线物理媒介相关子层(TP-PMD)，10Base-Tx 编解码和双绞线媒介访问单元(TPMAU)。本设计采用 RMII 接口与 FPGA 通信。下图为 IP101GR 芯片的结构框图。

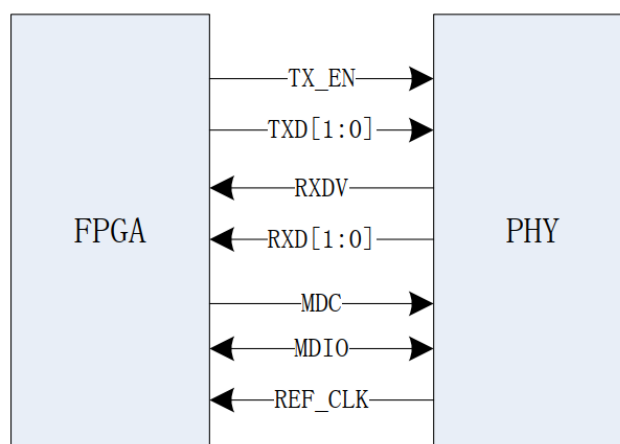


12.2. RMII

RMII(Reduced Media Independent Interface)介质无关接口是 IEEE-802.3 定义的以太网行业标准。它包括一个数据接口，以及一个 MAC 和 PHY 之间的管理接口。数据接口包括分别用于发送器和接收器的两条独立信道。每条信道都有自己的数据、时钟和控制信号。

RMII 数据接口总共需要 10 个信号，包括 TX_ER, TXD, TX_EN, TX_CLK, COL, RXD, RX_EX, RX_CLK, CRS, RX_DV。RMII 以 2 位半字节方式传送数据双向传

输，时钟速率 50MHz。其工作速率可达 100Mb/s。下图为具体信号连接示意图。



12.3. TCP/IP 协议栈简介

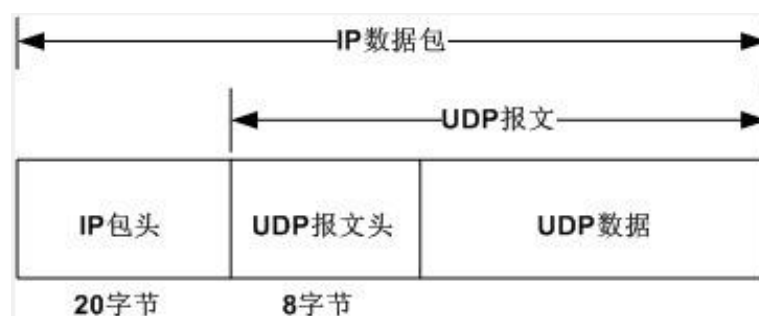
TCP/IP(Transmission Control Protocol/Internet Protocol)是供已连接因特网的计算机进行通信的通信协议。下图为 TCP/IP 的层次分布：



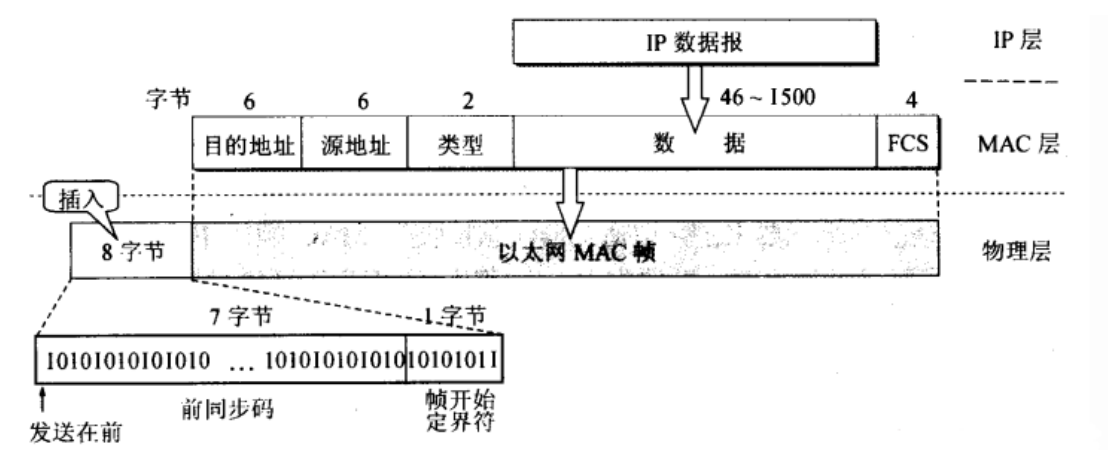
- ◆ 链路层：负责将数据合成数据块，此数据块称为帧(frame)，帧是数据链路层的传送单位；负责控制帧在物理信道上的传输，调节发送的速率以匹配接收方；负责在两个网络实体之间提供数据链路通路的建立、维持和释放管理。
- ◆ 网络层：处理分组在网络中的活动，例如分组的路由选择。在 TCP/IP 协议组件中，网络层协议包括 IP 协议(网际协议)，ICMP 协议(Internet 互连网控制报文协议)，以及 IGMP 协议(Internet 组管理协议)。
- ◆ 传输层：主要为两台主机上的应用程序提供端到端的通信。在 TCP/IP 协议组件中，有两个互不相同的传输协议：TCP(传输控制协议)和 UDP(用户数据报协议)。
- ◆ 应用层：负责处理特定的应用程序细节。

本设计使用的是 UDP/IP 协议，即使用了以太网的传输层、网络层、链路层，而物理层则由外接的 RTL8201CP 网卡完成。

下图为 UDP 报文与 IP 包之间的关系。



下图为物理层、MAC 层、IP 层成帧的示意图。



12.4. UDP 协议

UDP(User Datagram Protocol)是 OSI 参考模型中一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务。每一个数据包的前 8 个字节用来包含报头信息，剩余字节则用来包含具体的传输数据。UDP 协议经常用于图像传输、网络监控数据交换等数据传输速度要求比较高的场合。

UDP 报头由 4 个域组成，其中每个域各占用 2 个字节，具体如下：

UDP报文格式			
0	15	16	31
源端口号		目的端口号	
报文长度		校验和	
数据			

UDP 协议使用端口号为不同的应用程序保留各自的数据传输通道。数据发送一方将 UDP 数据报通过源端口发送出去，而数据接收一方则通过目标端口接收数据。

报文长度是指包括报头和数据部分在内的总字节数。以太网的数据帧的传输过程中，包长一般在 46~1500 字节。

12.5. IP 协议

IP(Internet Protoco)包位于网络层，功能是把分组发往目标网络或主机。所有的 TCP,UDP,IMCP, IGCP 的数据都以 IP 数据格式传输。

IP 数据包头前 20 个字节是固定的，后面是可变的。

版本 4 bit	首部长度 4 bit	服务类型（TOS） 8 bit	总长度（字节数） 16 bit	
标识 16 bit			标志 3 bit	片偏移 13 bit
生成时间（TTL） 8 bit	协议 8 bit		首部检验和 16 bit	
源IP地址 32 bit				
目的IP地址 32 bit				
选项（可选）				
数据				

版本

指 IP 协议的版本目前的 IP 协议版本号为 4(即 IPv4)

首部长度

可表示的最大数值是 15 个单位(一个单位为 4 字节)因此 IP 的首部长度的最大值是 60 字节

服务类型

用来获得更好的服务,在旧标准中叫做服务类型,但实际上一直未被使用过

总长度

指首部和数据之和的长度,单位为字,因此数据报的最大长度为 65535 字节,一般总长度必须不超过最大传送单元 MTU

标识

计数器,用来产生数据报的标识

标志

占 3 位,标志字段的最低位是 MF(More Fragment),MF=1 表示后面“还有分片”,MF=0 表示最后一个分片;标志字段中间的一位是 DF(Don't Fragment),只有当 DF=0 时才允许分片

片偏移

指较长的分组在分片后某片在原分组中的相对位置,片偏移以 8 个字节为偏移单位

生存时间

TTL (Time To Live) 数据报在网络中可通过的路由器数的最大值,TTL 字段是由发送端初始设置一个 8 bit 字段。推荐的初始值由分配数字 RFC 指定,当前值为 64。发送 ICMP 回显应答时经常把 TTL 设为最大值 255

协议

指出此数据报携带的数据使用何种协议以便目的主机的 IP 层将数据部分上交给哪个处理过程。1 表示为 ICMP,2 表示为 IGMP,6 表示为 TCP,17 表示为 UDP

首部校验和

只检验数据报的首部不检验数据部分，方法是计算 16 位的二进制和，首先将校验和字段(16 位)置 0，将每个 16 位的反码相加，若是奇数字节即最后未到 16 字节，则以 0 补充，结果存在校验和字段中

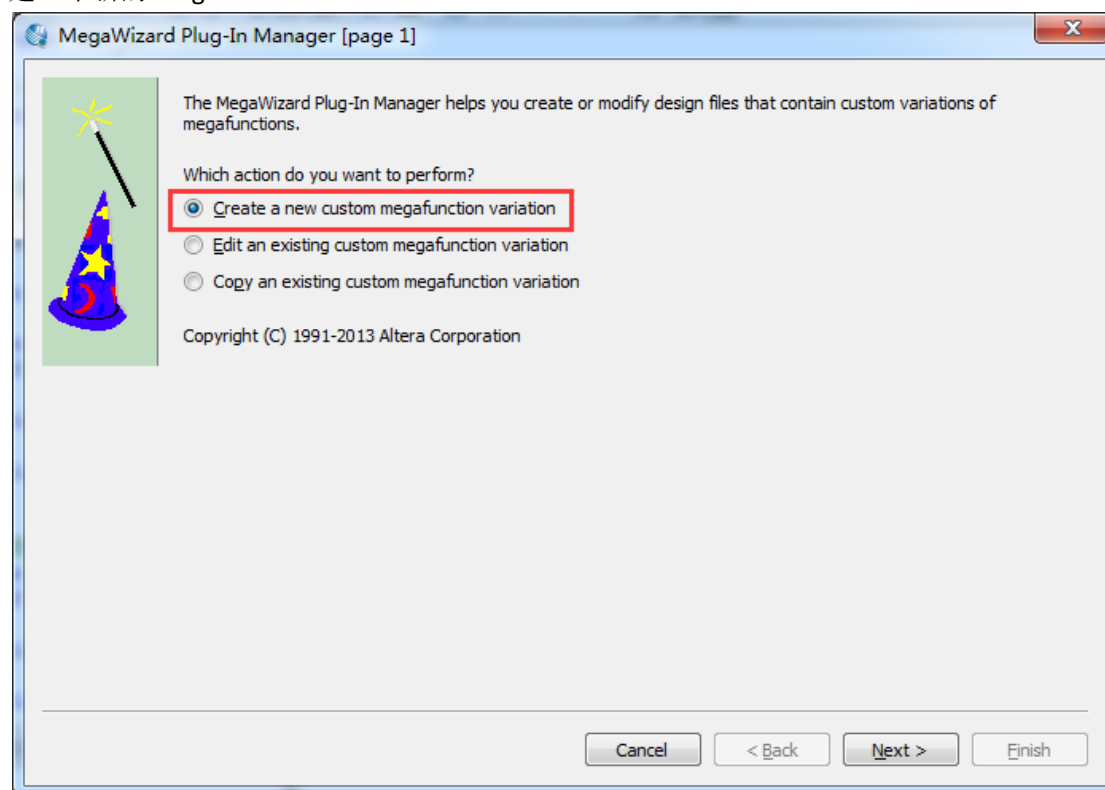
源地址和目的地址

使用 4 个数字来为计算机编址。每个计算机必须有一个唯一的 4 个数字的地址。数字在 0 到 255 之间，并由点号隔开，比如 192.168.1.60。

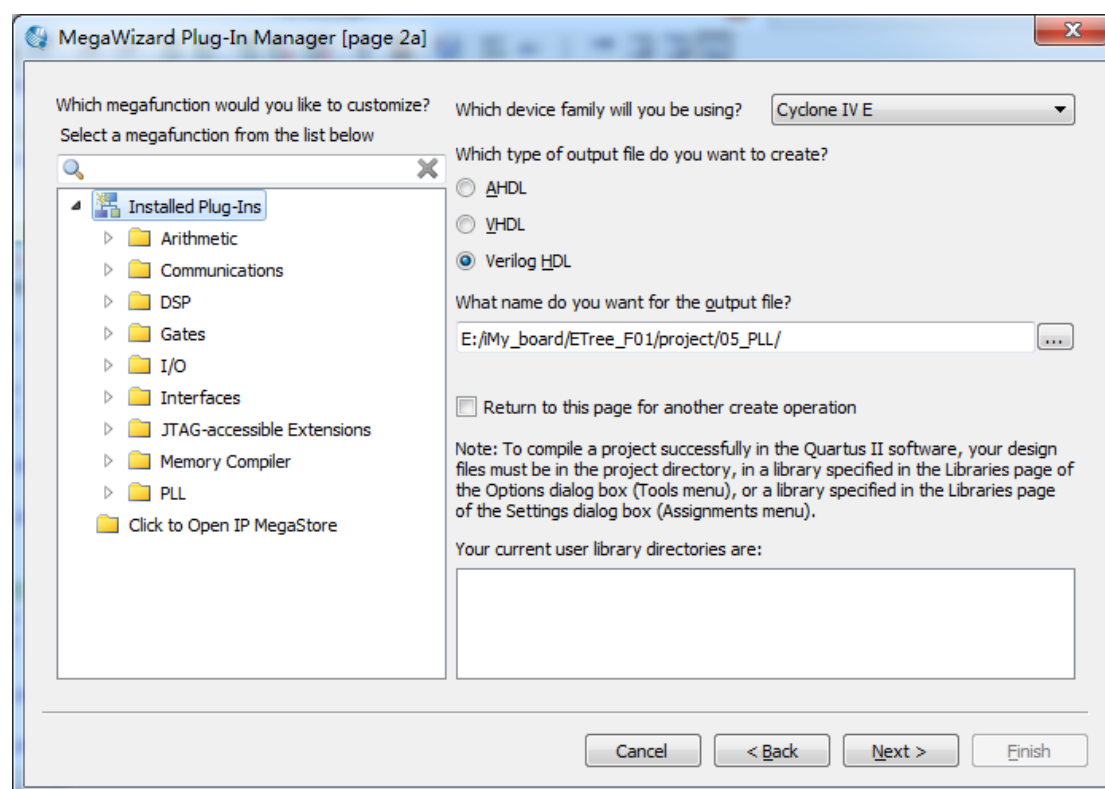
12.6. PLL 时钟生成

由于 IP101GR 网卡需要一个 25MHz 时钟作为参考，这里我利用 Quartus II 自带的 MegaWizard 里的 PLL 生成需要的时钟。

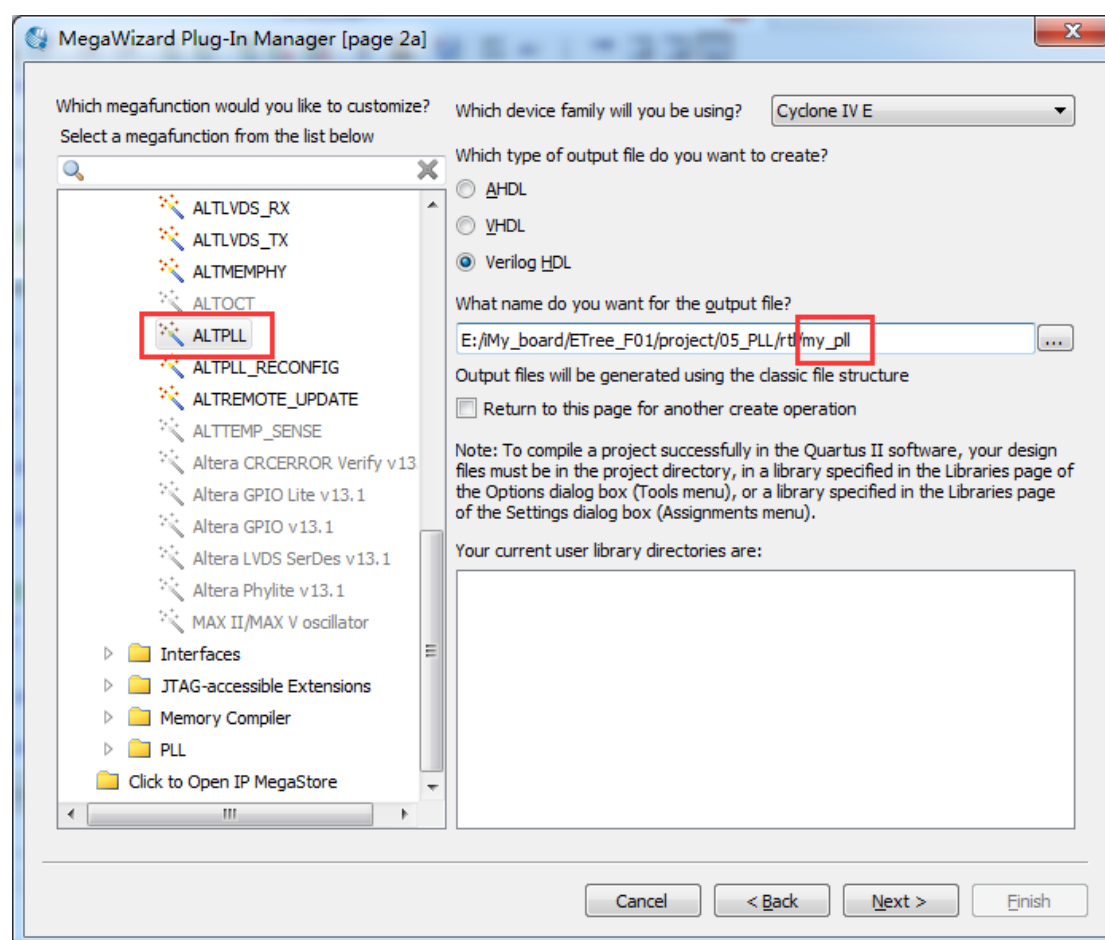
打开 Quartus II，点击 Tools | MegaWizard Plug-In Manager，弹出下面的对话框，选择创建一个新的 megafunction variation。



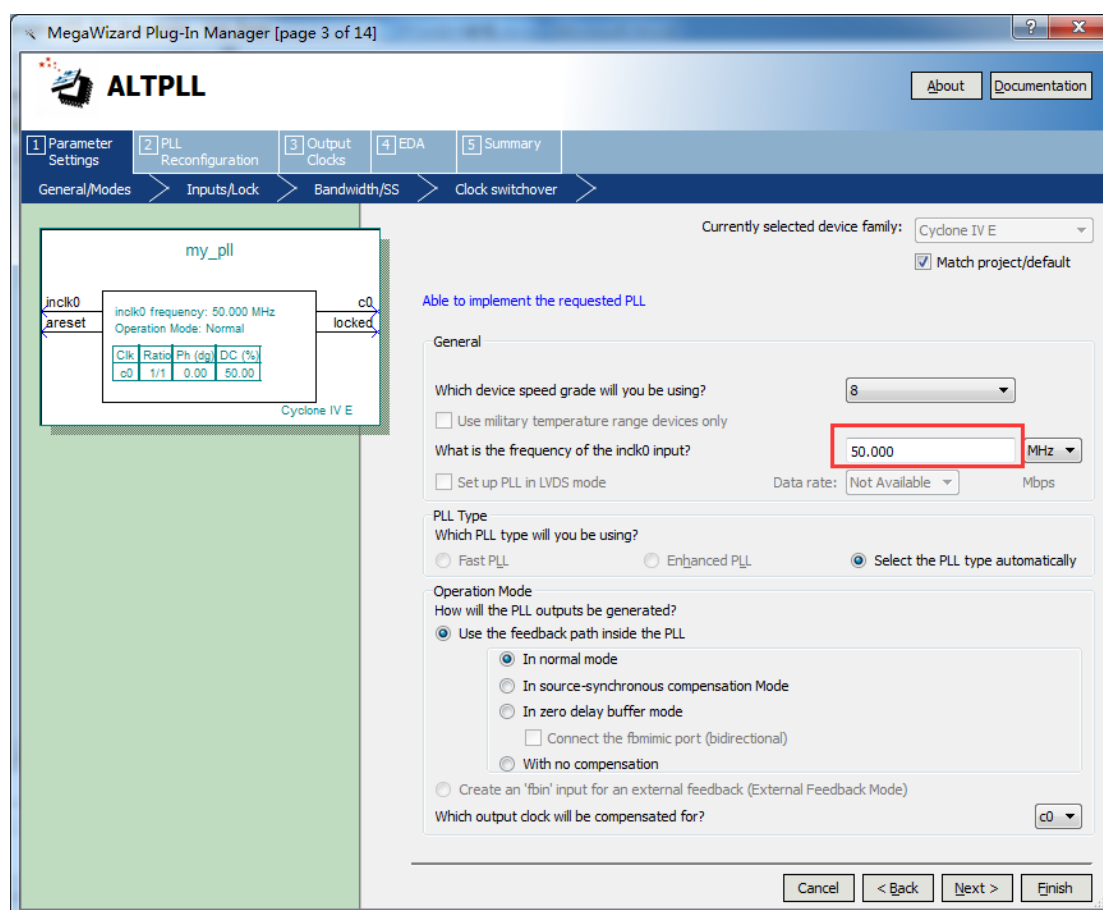
点击“Next”，弹出下面的对话框。在左侧栏中，可以看到有 9 类 IP 组件。Arithmetic 包含基本加、减、乘、除、开方等数学运算；DSP 包含各种 FIR、FFT、数字图像处理等算法；I/O 包含 DDIO、LVDS、ALTPLL 等；Interface 包含 DDR、DDR2、PCI 等各类接口；Memory Compiler 包含 FIFO、双口 RAM 等存储结构单元。



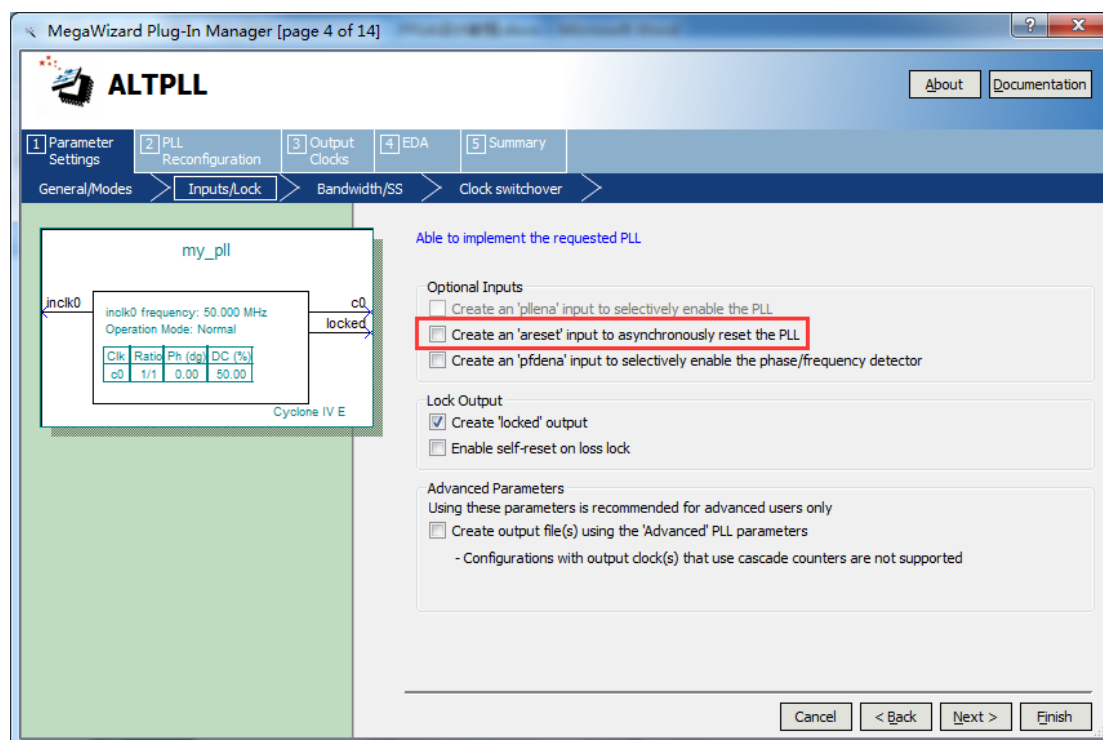
这里我们使用 I/O 中的 ALTPLL 实现 PLL 锁相环，如下图所示，命名为“my_pll”。



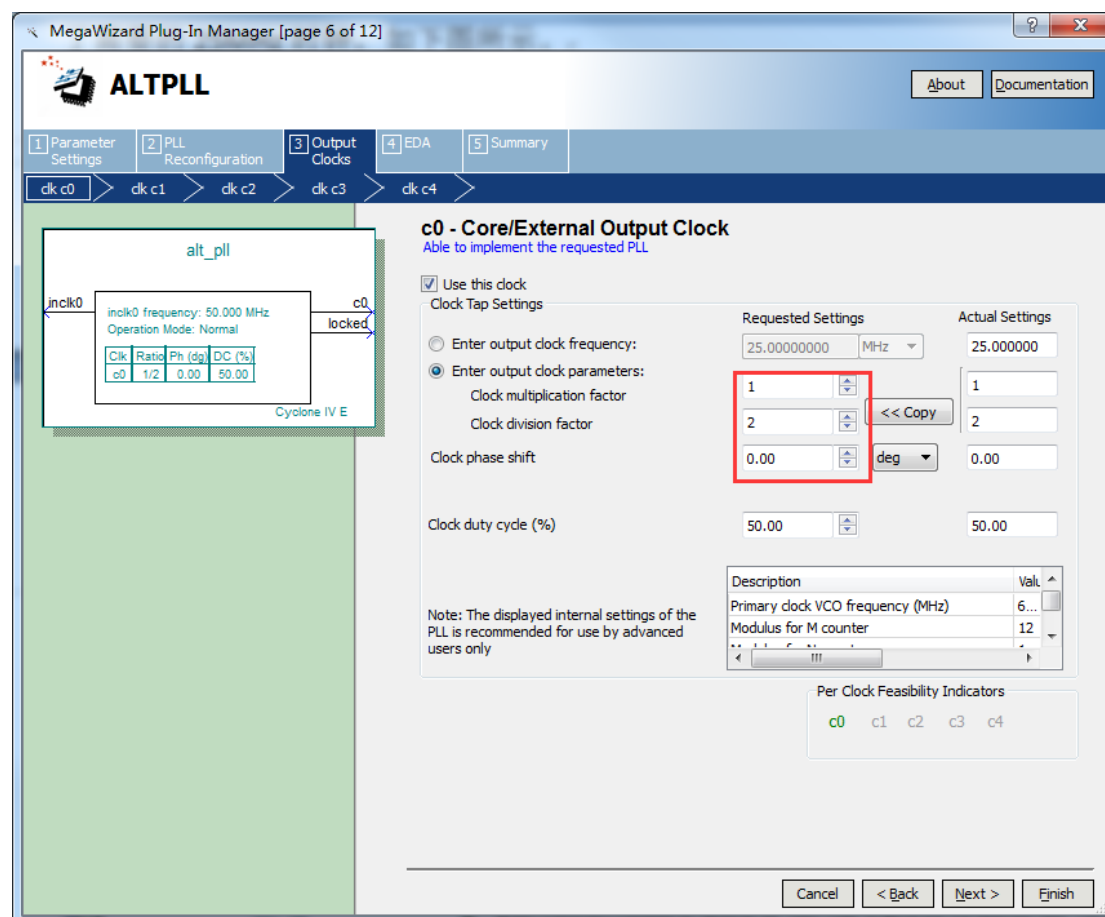
点击“Next”，弹出下面对话框，修改输入时钟为 50MHz。



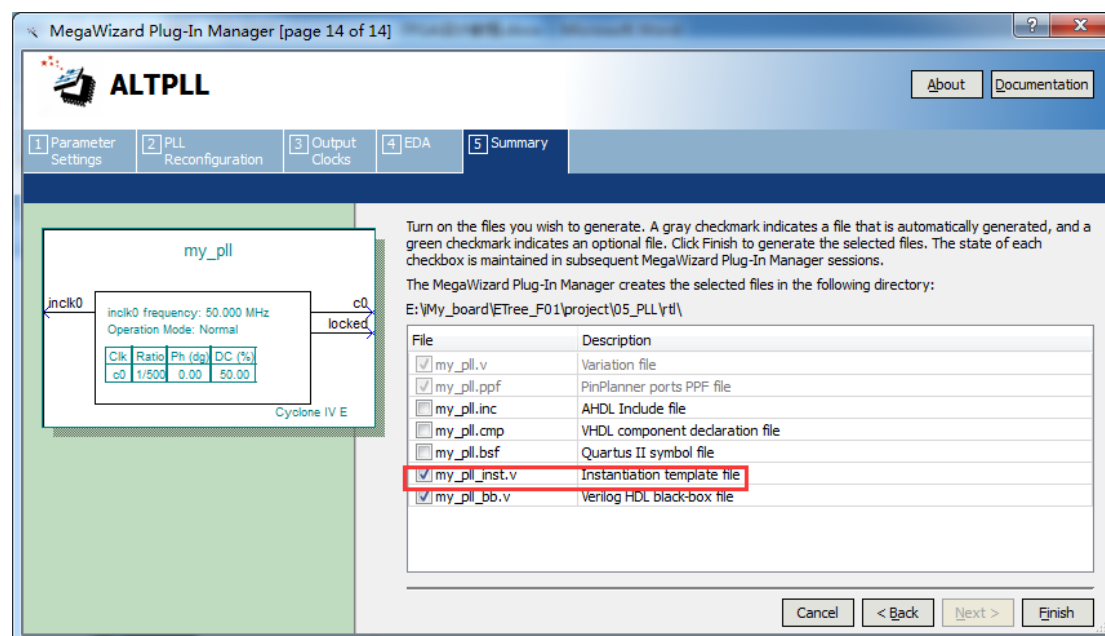
点击“Next”，取消勾选生成'areset'，如下图所示。



点击“Output Clocks”，设置时钟分频系数为2，相位为0度，占空比为50%，这样生成了需要的25MHz时钟，如下图所示。



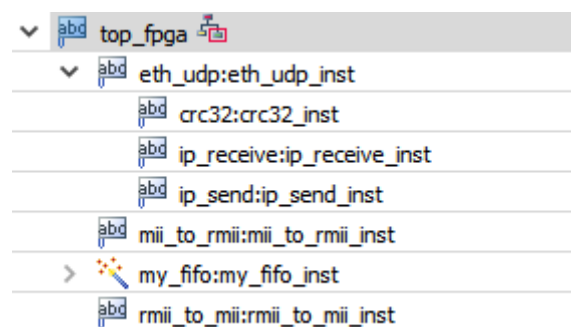
点击“Summary”，勾选“my_pll_inst.v”生成例化文件，如下图所示。



点击“Finish”完成。

12.7. HDL 设计

➤ 工程目录



Eth_udp 负责 UDP 包的接受与发送，其中 ip_send 为发送模块，ip_receive 为接受模块，crc32 为校验模块；

Mii_to_rmii 将 mii 接口转换成 rmii 接口，发送数据给网卡芯片；

Rmii_to_mii 将 rmii 接口转换成 mii 接口，然后发送给 eth_udp；

My_fifo 用于存储接受的数据帧，以便 eth_udp 获取接受的数据再发送给 PC。

➤ UDP 发送与接受模块设计

此部分可以查看具体工程代码。

➤ CRC 发送模块设计

IP 数据包的 CRC32 校验从目标 MAC 地址开始，直到包的最后一个数据。具体的 CRC32 的 Verilog HDL 代码可以在网站上直接生成，具体网址 <http://www.easics.com/webtools/crctool>。

在网页中选好设置，点击 DOWNLOAD 生成所需代码：

Polynomial

Specify the generator polynomial by clicking on the polynomial terms, or select a predefined value from the list below.

1	x	x ²	x ³	x ⁴	x ⁵	x ⁶	x ⁷	x ⁸	x ⁹	x ¹⁰	x ¹¹	x ¹²	x ¹³	x ¹⁴	x ¹⁵	x ¹⁶
x ¹⁷	x ¹⁸	x ¹⁹	x ²⁰	x ²¹	x ²²	x ²³	x ²⁴	x ²⁵	x ²⁶	x ²⁷	x ²⁸	x ²⁹	x ³⁰	x ³¹	x ³²	x ³³
x ³⁴	x ³⁵	x ³⁶	x ³⁷	x ³⁸	x ³⁹	x ⁴⁰	x ⁴¹	x ⁴²	x ⁴³	x ⁴⁴	x ⁴⁵	x ⁴⁶	x ⁴⁷	x ⁴⁸	x ⁴⁹	x ⁵⁰
x ⁵¹	x ⁵²	x ⁵³	x ⁵⁴	x ⁵⁵	x ⁵⁶	x ⁵⁷	x ⁵⁸	x ⁵⁹	x ⁶⁰	x ⁶¹	x ⁶²	x ⁶³	x ⁶⁴	x ⁶⁵	x ⁶⁶	x ⁶⁷

$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$

Select a predefined value: CRC32 Ethernet/AAL5

Data Width

Select the number of data bits to be processed in one step: 4

Output Language

☐ VHDL ☒ Verilog

[Download](#)

设计完的代码如下：

```

module crc32
(
    input wire      clk      ,    //时钟信号
    input wire      rst_n    ,    //复位信号,低电平有效
    input wire [3:0] data     ,    //待校验数据
    input wire      crc_en   ,    //crc 使能,校验开始标志
    input wire      crc_clr  ,    //crc 数据复位信号

    output reg [31:0] crc_data ,    //CRC 校验数据
    output reg [31:0] crc_next    //CRC 下次校验完成数据
);

// wire      define
wire [3:0] data_sw;    //待校验数据高低位互换

//data_sw:待校验数据高低位互换
assign data_sw = {data[0],data[1],data[2],data[3]};

//crc_next:CRC 下次校验完成数据
//CRC32 的生成多项式为:  $G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$ 
always@(*)
begin
    crc_next    <= 32'b0;
    if(crc_en == 1'b1)
    begin
        crc_next[0] <= (data_sw[0] ^ crc_data[28]);
        crc_next[1] <= (data_sw[1] ^ data_sw[0] ^ crc_data[28]
            ^ crc_data[29]);
        crc_next[2] <= (data_sw[2] ^ data_sw[1] ^ data_sw[0]
            ^ crc_data[28] ^ crc_data[29] ^ crc_data[30]);
        crc_next[3] <= (data_sw[3] ^ data_sw[2] ^ data_sw[1]
            ^ crc_data[29] ^ crc_data[30] ^ crc_data[31]);
        crc_next[4] <= (data_sw[3] ^ data_sw[2] ^ data_sw[0] ^
crc_data[28]
            ^ crc_data[30] ^ crc_data[31]) ^ crc_data[0];
        crc_next[5] <= (data_sw[3] ^ data_sw[1] ^ data_sw[0] ^
crc_data[28]
            ^ crc_data[29] ^ crc_data[31]) ^ crc_data[1];
        crc_next[6] <= (data_sw[2] ^ data_sw[1] ^ crc_data[29]
            ^ crc_data[30]) ^ crc_data[2];
        crc_next[7] <= (data_sw[3] ^ data_sw[2] ^ data_sw[0] ^
crc_data[28]
            ^ crc_data[30] ^ crc_data[31]) ^ crc_data[3];
    end
end

```

```

        crc_next[8] <= (data_sw[3] ^ data_sw[1] ^ data_sw[0] ^
crc_data[28]
            ^ crc_data[29] ^ crc_data[31]) ^ crc_data[4];
        crc_next[9] <= (data_sw[2] ^ data_sw[1] ^ crc_data[29]
            ^ crc_data[30]) ^ crc_data[5];
        crc_next[10] <= (data_sw[3] ^ data_sw[2] ^ data_sw[0] ^
crc_data[28]
            ^ crc_data[30] ^ crc_data[31]) ^ crc_data[6];
        crc_next[11] <= (data_sw[3] ^ data_sw[1] ^ data_sw[0] ^
crc_data[28]
            ^ crc_data[29] ^ crc_data[31]) ^ crc_data[7];
        crc_next[12] <= (data_sw[2] ^ data_sw[1] ^ data_sw[0] ^
crc_data[28]
            ^ crc_data[29] ^ crc_data[30]) ^ crc_data[8];
        crc_next[13] <= (data_sw[3] ^ data_sw[2] ^ data_sw[1] ^
crc_data[29]
            ^ crc_data[30] ^ crc_data[31]) ^ crc_data[9];
        crc_next[14] <= (data_sw[3] ^ data_sw[2] ^ crc_data[30]
            ^ crc_data[31]) ^ crc_data[10];
        crc_next[15] <= (data_sw[3] ^ crc_data[31]) ^ crc_data[11];
        crc_next[16] <= (data_sw[0] ^ crc_data[28]) ^ crc_data[12];
        crc_next[17] <= (data_sw[1] ^ crc_data[29]) ^ crc_data[13];
        crc_next[18] <= (data_sw[2] ^ crc_data[30]) ^ crc_data[14];
        crc_next[19] <= (data_sw[3] ^ crc_data[31]) ^ crc_data[15];
        crc_next[20] <= crc_data[16];
        crc_next[21] <= crc_data[17];
        crc_next[22] <= (data_sw[0] ^ crc_data[28]) ^ crc_data[18];
        crc_next[23] <= (data_sw[1] ^ data_sw[0] ^ crc_data[29]
            ^ crc_data[28]) ^ crc_data[19];
        crc_next[24] <= (data_sw[2] ^ data_sw[1] ^ crc_data[30]
            ^ crc_data[29]) ^ crc_data[20];
        crc_next[25] <= (data_sw[3] ^ data_sw[2] ^ crc_data[31]
            ^ crc_data[30]) ^ crc_data[21];
        crc_next[26] <= (data_sw[3] ^ data_sw[0] ^ crc_data[31]
            ^ crc_data[28]) ^ crc_data[22];
        crc_next[27] <= (data_sw[1] ^ crc_data[29]) ^ crc_data[23];
        crc_next[28] <= (data_sw[2] ^ crc_data[30]) ^ crc_data[24];
        crc_next[29] <= (data_sw[3] ^ crc_data[31]) ^ crc_data[25];
        crc_next[30] <= crc_data[26];
        crc_next[31] <= crc_data[27];

    end

end

//crc_data:CRC 校验数据

```

```

always @(posedge clk or negedge rst_n)
    if(~rst_n)
        crc_data <= 32'hff_ff_ff_ff;
    else if(crc_clr == 1'b1)
        crc_data <= 32'hff_ff_ff_ff;
    else if(crc_en == 1'b1)
        crc_data <= crc_next;

endmodule

```

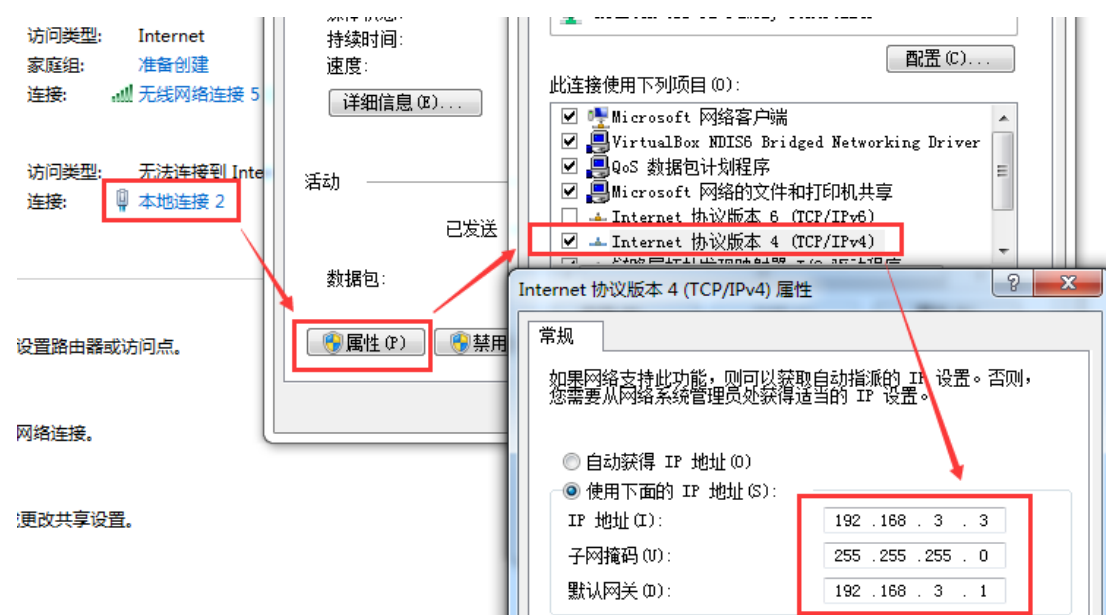
12.8. 管脚约束

选择菜单 Assignment -> Pin Planner，添加管脚约束。

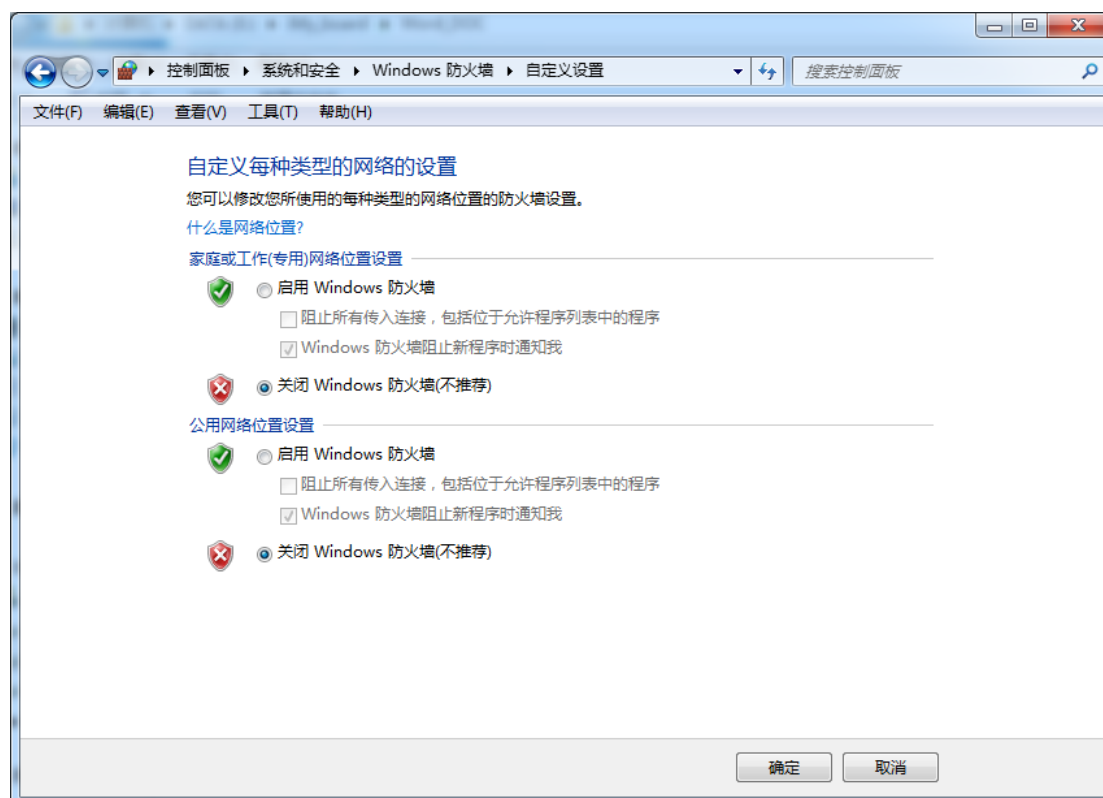
in	clk	Input	PIN_M16	5
in	e_rx[1]	Input	PIN_A7	8
in	e_rx[0]	Input	PIN_B8	8
in	e_rxd	Input	PIN_E1	1
in	e_rxdv	Input	PIN_A8	8
in	e_xer	Input	PIN_A9	7
out	e_tx[1]	Output	PIN_A6	8
out	e_tx[0]	Output	PIN_B7	8
out	e_txen	Output	PIN_B6	8
out	e_xi	Output	PIN_A5	8
out	led	Output	PIN_F6	8
in	rst_n	Input	PIN_L10	4

12.9. 实验操作

插上网线，会看到绿灯闪烁，表示网络识别了。再将本地连接的 IP 地址设置成 192.168.3.3，子网掩码设置成 255.255.255.0，网关设置成 192.168.3.1，具体如下图所示：



关闭防火墙，否则电脑无法接受视频数据。



下面通过 arp 和 netsh 命令查看和绑定板卡 MAC 和 IP 地址:

- 1) 以管理员身份打开 DOS 终端(在 Windows 开始栏中输入 cmd 命令);
- 2) 使用 arp -a 命令查看计算机网卡的 MAC 物理地址和 IDX 编号, 如下图所示:

```
管理员: 命令提示符
Microsoft Windows [版本 10.0.19044.3086]
(c) Microsoft Corporation. 保留所有权利。

C:\windows\system32>arp -a

接口: 192.168.3.3 --- 0x17
Internet 地址      物理地址      类型
192.168.3.255      ff-ff-ff-ff-ff-ff 静态
224.0.0.22         01-00-5e-00-00-16 静态
224.0.0.251        01-00-5e-00-00-fb 静态
224.0.0.252        01-00-5e-00-00-fc 静态
239.255.255.250    01-00-5e-7f-ff-fa 静态
```

- 3) 使用 netsh -c "i" add neighbors 23 "192.168.3.2" "00-0a-35-01-fe-c0" 命令绑定, 其中 **23** 即(0x17)是上面得到的 IDX 号, "192.168.3.2"为板卡 IP 地址, "00-0a-35-01-fe-c0"为板卡 MAC 地址:

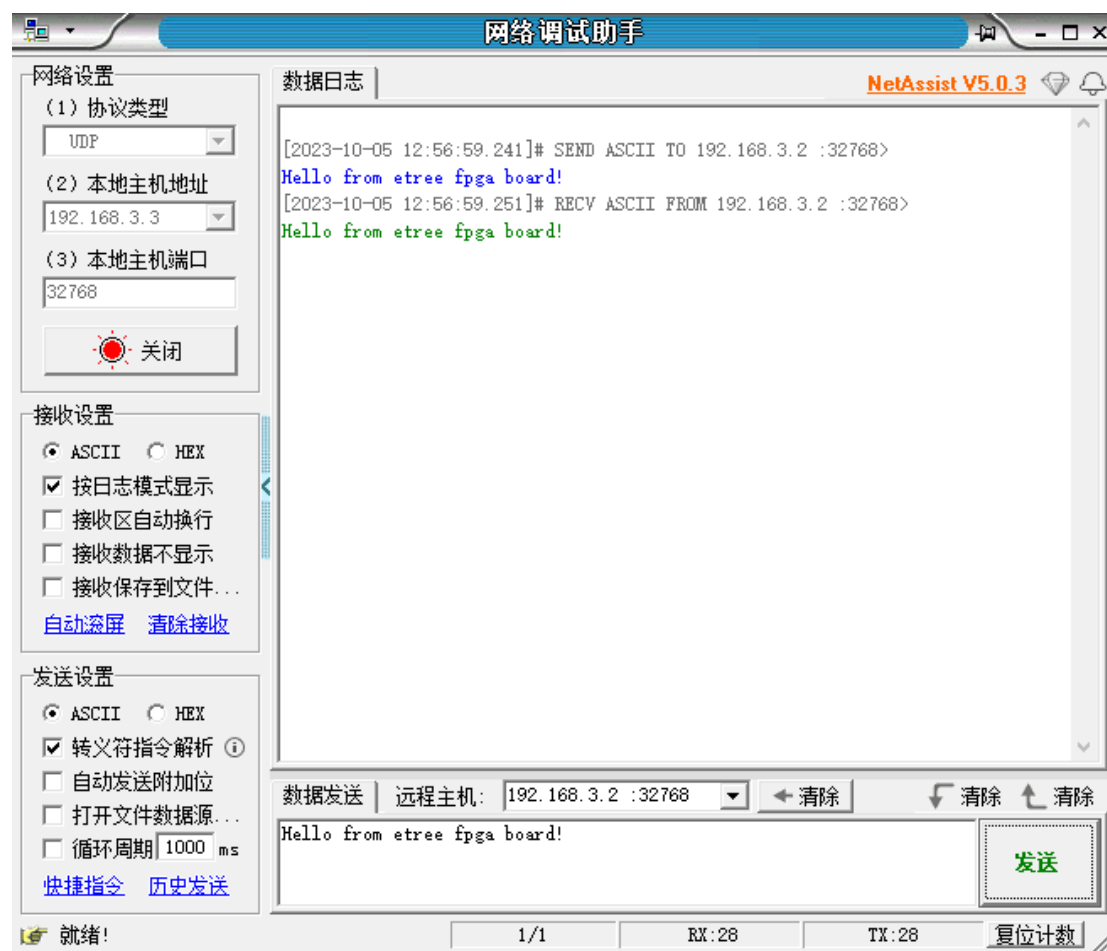
```
C:\windows\system32>netsh -c "i i" add neighbors 23 "192.168.3.2" "00-0a-35-01-fe-c0"
```

- 4) 使用 arp -a 查看板卡 IP 和 MAC 绑定结果, 192.168.3.3 有对应的 192.168.3.2 和对应的 MAC 地址, 如下图所示:

```
C:\windows\system32>arp -a

接口: 192.168.3.3 --- 0x17
Internet 地址      物理地址      类型
192.168.3.2        00-0a-35-01-fe-c0    静态
192.168.3.255      ff-ff-ff-ff-ff-ff    静态
224.0.0.22         01-00-5e-00-00-16    静态
224.0.0.251        01-00-5e-00-00-fb    静态
224.0.0.252        01-00-5e-00-00-fc    静态
239.255.255.250    01-00-5e-7f-ff-fa    静态
```

打开 Tools 目录下的‘网络调试助手’，网络设置成 UDP、192.168.3.3、32768，远程主机设置成 FPGA 开发板的 IP 地址和端口(192.168.3.2:32768)；点击‘打开’按钮，发送和接受区设置成‘ASCII’。在发送区输入“Hello from etree fpga board!”点击发送后，在‘网络数据接收’区域看到同样的字符串信息，如下图所示：



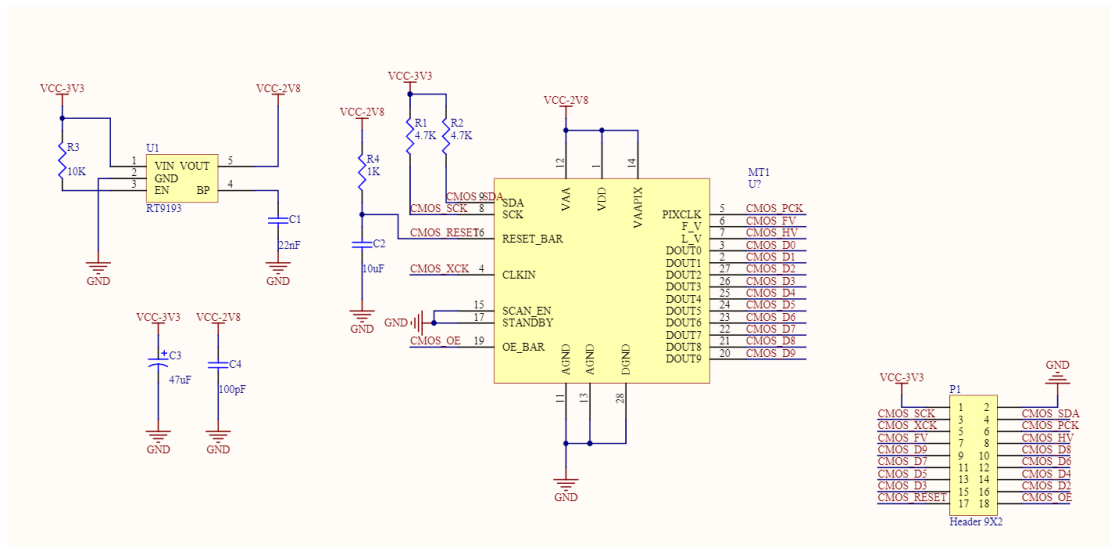
第十三章 MT9V011 摄像头以太网显示

本章基于百兆以太网卡 IP101GR 和 30 万像素摄像头模块 MT9V011，将摄像头的 30 帧/秒的视频输出通过以太网传送到 PC，然后上位机实时显示视频数据。

11.1. 硬件介绍

百兆网卡的硬件电路上一章一节介绍过了，这里不再重复。

下图为 MT9V011 硬件原理图，MT9V011 的输出接口为 DVP 并行信号。

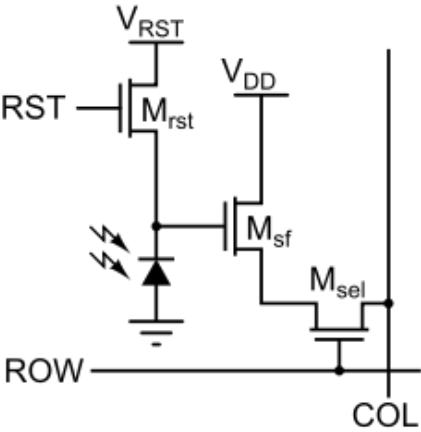


MT9V011 为 30 万像素摄像头芯片，比一般的 30 万像素摄像头芯片效果好很多，比如 OV7670，具体的摄像头参数如下表所示。

PARAMETER		TYPICAL VALUE
Optical Format		1/4-inch (4:3)
Active Imager Size		3.58mm(H) x 2.688mm (V), 8.4mm Diagonal
Active Pixels		640H x 480V
Pixel Size		5.6um x 5.6um
Color Filter Array		RGB Bayer Pattern
Shutter Type		Electronic Rolling Shutter (ERS)
Max. Data Rate/Master Clock		13.5 MPS/27 MHz
Frame Rate	VGA (640x480)	30 fps at 27 MHz
	CIF (352x288)	Programmable up to 60 fps
	QVGA (320x240)	Programmable up to 90 fps
ADC Resolution		10-bit, on-chip
Responsivity		1.9 V/lux-sec (550nm)
Dynamic Range		60dB
SNR _{MAX}		45dB
Supply Voltage		2.8V ±0.25V
Power Consumption		70mW at 2.8V, 20pF load, 27 MHz, 30 fps
Operating Temperature		-20°C to +60°C
Packaging		28-Pin PLCC

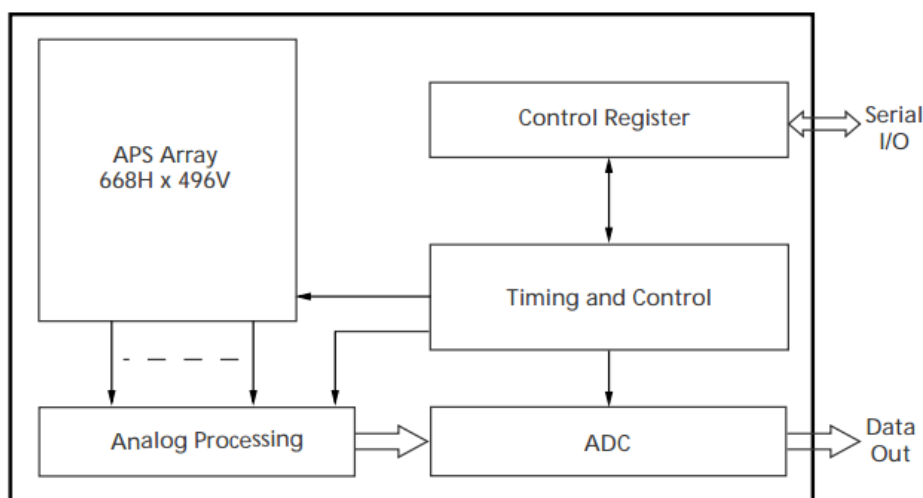
11.2. CMOS 工作原理

APS(Active Pixel Sensor)是最新的 CMOS 图像传感器的像素结构设计，下图为 3T-APS 的结构电路：

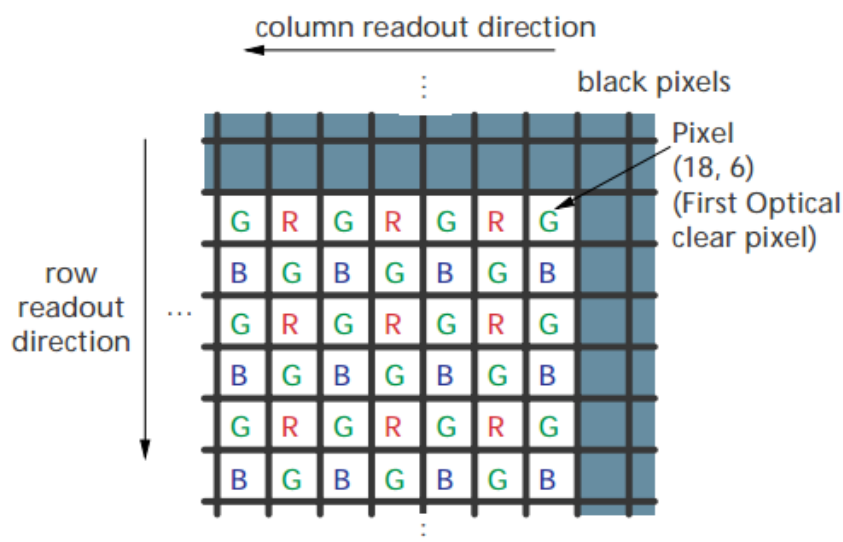


- APS 的工作过程：
- ◆ 复位晶体管 M_{rst} 开通
 - ◆ PD 复位到 $V_{rst}-V_{th}(M_{rs})$ 的阈值电压)
 - ◆ 关闭 M_{rst} ，PD 进入电气浮动状态
 - ◆ 光线入射，PD 结电容聚集光生载流子
 - ◆ 在一个积累时间后，选通晶体管 M_{sf}

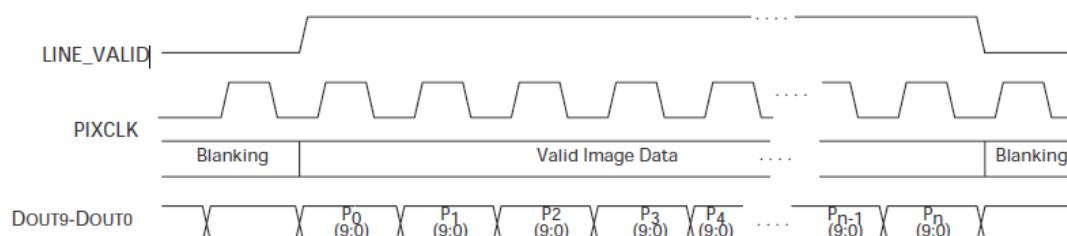
- ◆ 打开晶体管 Msel, 像素中的输出信号在垂直输出线上被读出
 - ◆ 关闭 Msel, Mrst 再次接通, 重复上述过程。
- MT9V011 为 30 万像素, 最高 30 帧, 感光阵列面积为 1/3", 内部结构如下图所示:



APS Array 和 Analog Processing 为模拟电路部分, 其中感光阵列排序为 GRBG;



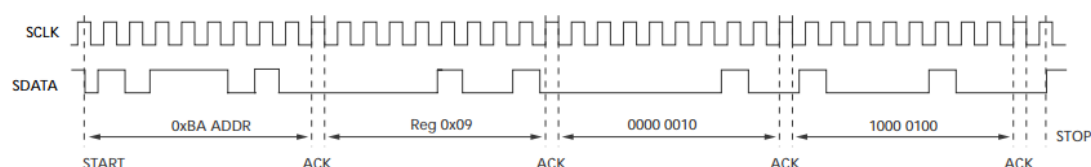
Control Register 和 Timing and Control 为数字逻辑部分, 其中 Serial IO 为 I2C 接口, Data Out 为并行 DVP 视频信号接口。具体 DVP 波形如下图所示, LINE_VALID(h_ref)为行有效信号, PIXCLK 为像素时钟信号, FRAME_VALID 为帧同步信号, Dout 为像素数据信号。



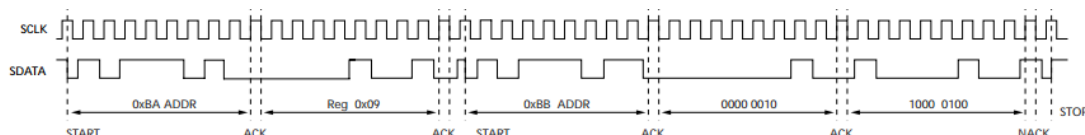


11.3. CMOS 配置接口

MT9V011 配置接口为通用的 I2C 接口, 其中写时序如下图(往地址 0xBA 寄存器中写入 16 位数据), 设备写地址为 0xBA。

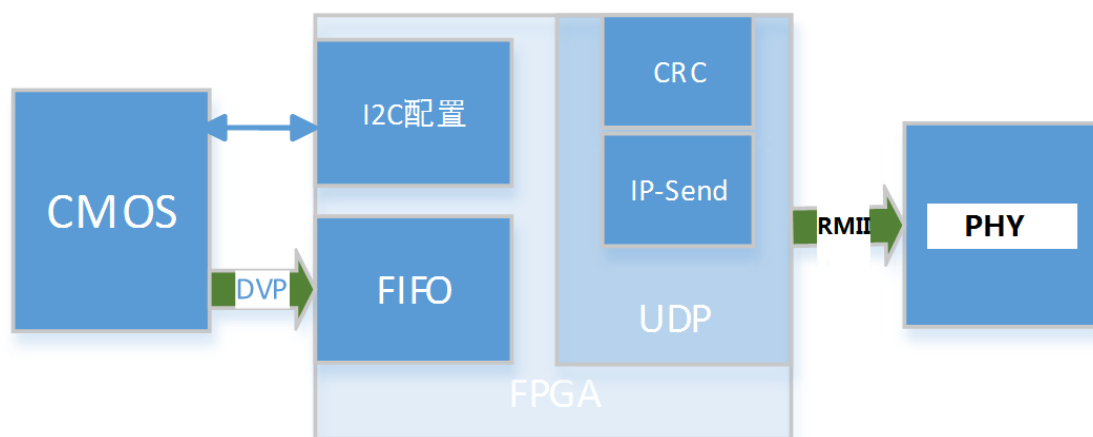


与写时序不同, 读时序稍微复杂点, 需要多发一次设备读地址 0xBB, 具体时序如下图所示。



11.4. HDL 设计

➤ 系统框架



- ✧ I2C 配置模块负责初始化 CMOS 图像传感器, 硬件配置接口为 I2C;
- ✧ FIFO 模块负责将 CMOS 的 DVP 信号缓存起来, 并准备发给 UDP 发送模块;
- ✧ IPSEND 模块负责将视频数据打包成 UDP 协议流发送给 IP101GR 网卡芯片;
- ✧ CRC 模块负责生产 CRC 校验码。

➤ I2C 配置模块

MT9V011 大概有 60 个寄存器, 这里只配置常用的几个寄存器。

寄存器 1: 0x000b, 读取的第一行

0x01	0-8	First row to be read out—default = 0x000A (10). Minimum recommended value = 0x0006.
------	-----	--

寄存器 2: 0x0012, 读取的第一列

0x02	0-9	First column to be read out—default = 0x0016 (22). Minimum recommended value = 0x0012 (18).
------	-----	---

寄存器 5: 0x0083, 行消隐区大小

寄存器 6: 0x001c, 场消隐区大小

0x05	0-9	Horizontal Blanking (number of columns)—default = 0x0083 (131 pixel clocks). Minimum value for 0x05 = 0x0009. Minimum recommended value for 0x05 = 0x007B (123 pixel clocks).
0x06	0-11	Vertical Blanking (number of rows -1)—default = 0x001C (28 rows). Minimum recommended value = 0x0003.

寄存器 0x35: 0x0313, 增益设置

0x35		GlobalGain—default = 0x0020 (32) = 1x gain. This register can be used to set all four gains at once. When read, it will return the value stored in Reg0x2B.
	0-6	Initial Gain = bits (6-0) x 0.03125.
	7, 8	Analog Gain = (Bit 8 + 1) x (Bit 7 + 1) x Initial Gain (each bit gives 2x gain).
	9,10	9, 10: Total Gain = (Bit 9 + 1) x (Bit 10 + 1) x Analog Gain (each bit gives 2x gain).

寄存器 0x20: 0x1000, 上下颠倒和左右镜像, 注意设置颠倒或镜像方式会改变 RAW 数据中 RG/GB 四通道的分布顺序。

0x20	0	Show bad frames: 1 = output all frames (including bad frames). 0 = only output good frames. A bad frame is defined as the first frame following a change to: window size or position, horizontal blanking, pixel clock speed, zoom, row or column skip, or mirroring.
	3	Column skip: 1 = read out two columns, and then skip two columns (as with rows). 0 = normal readout.
	4	Row skip: 1 = read out two rows, and then skip two rows (i.e. row 8, row 9, row 12, row 13...). 0 = normal readout.
	9	"Continuous" Line Valid (continue producing line valid during vertical blanking). 0 = Normal Line Valid (default, no line valid during vertical blanking).
	10	Line valid = "Continuous" Line Valid XOR Frame Valid. 0 = Normal Line Valid. Ineffective if Continuous Line Valid is set.
	11	The four dark rows 0 to 3 are read out in addition to the valid data. 0 = normal readout.
	To preserve a right-reading image and the correct color order, all four of these bits should be set to "1" to invert the image.	
	5	1 = readout starting 1 column later. 0 = normal readout.
	7	1 = readout starting 1 row later. 0 = normal readout.
	14	1 = read out from right to left (mirrored). 0 = normal readout.
	15	1 = read out from bottom to top (upside down). 0 = normal readout.

寄存器 0x09: 0x01cf, 曝光时间设置, 曝光时间越长图像亮度越高, 但当曝光时间超过最大曝光时间(最大曝光时间和当前最大行频有关)时, 视频帧率会变慢。

0x09	0-11	Number of rows of integration, default = 0x01FC (508).
------	------	--

//以下代码来自于 camera_if-->iic_com.v 文件

```
always @(*)
```

```

begin
  case(lut_index)
    0 : lut_data <= 24'h01_000b;
    //First row to read
    1 : lut_data <= 24'h02_0012;
    //Firts column to read
    2 : lut_data <= 24'h05_0083;
    //bit 0-9 h-blank
    3 : lut_data <= 24'h06_001c;
    //bit 0-11 v-blank
    //global gain: 0-6(initial gain) 7-8(analog gain) 9-10(total gain)
    4 : lut_data <= 24'h35_0313;
    //bit 15: upside down, bit 14: mirror(right to left)
    5 : lut_data <= 24'h20_1000;
    // 6 : lut_data <= 24'h07_3302;
    //bit 2: chip enable, bit 15: pclk invert, 8-11
    7 : lut_data <= 24'h09_01cf;
    //bit 0-11, shutter width
    default:lut_data <= 24'h000000;
  endcase
end

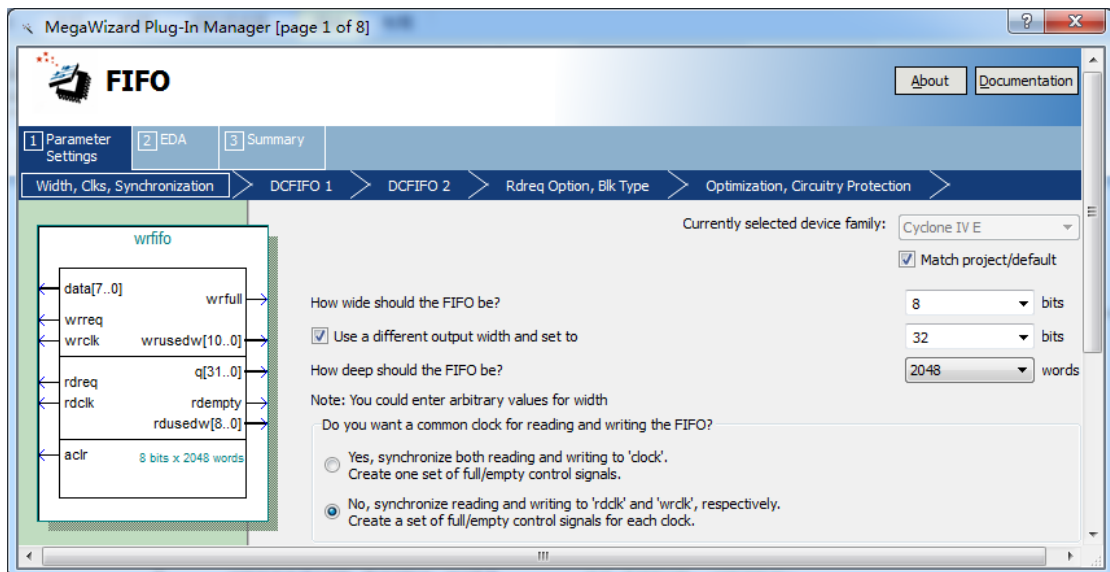
```

➤ UDP 发送模块

UDP 发送模块中的 IPSEND 和 CRC 模块可参考“错误！未找到引用源。 错误！未找到源。”，这里不再做详细讲解。

➤ FIFO 模块

FIFO 模块是一个 8 位输入、32 位输出的模块，实现 CMOS 时钟域和以太网时钟域的转换，深度为 2048×8 bit，在 MegaWizard Plug-In Manager 中设置如下图所示。



具体的顶层例化代码如下：

```
wrfifo wrfifo_inst
(
    .aclr(~rstn | cam_vsync_out) ,// input aclr
    .data(cam_data_out) ,          // input [15:0] data
    .rdclk(~E_TXC) ,              // input rdclk
    .rdreq(fifo_rd_en) ,          // input rdreq
    .wrclk(cam_pclk) ,            // input wrclk
    .wrreq(cam_hsync_out) ,       // input wrreq
    .q(fifo_q) ,                  // output [31:0] q
    .rdempty() ,                  // output rdempty
    .rdusedw(fifo_data_count) ,    // output [6:0] rdusedw
    .wrfull() ,                   // output wrfull
    .wrusedw()                     // output [8:0] wrusedw
);
```

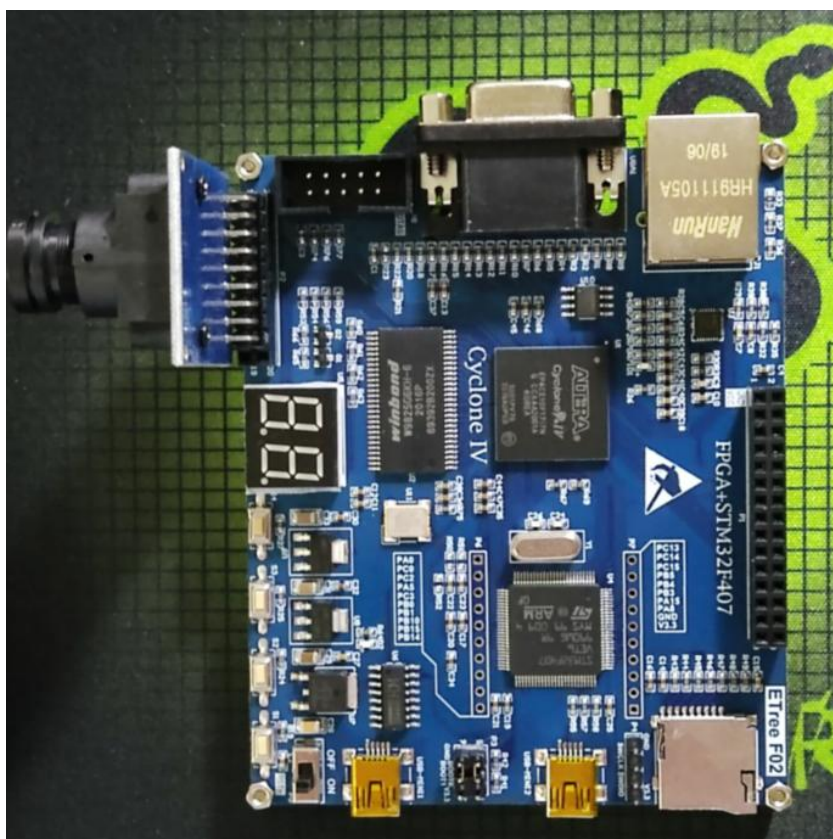
11.5. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

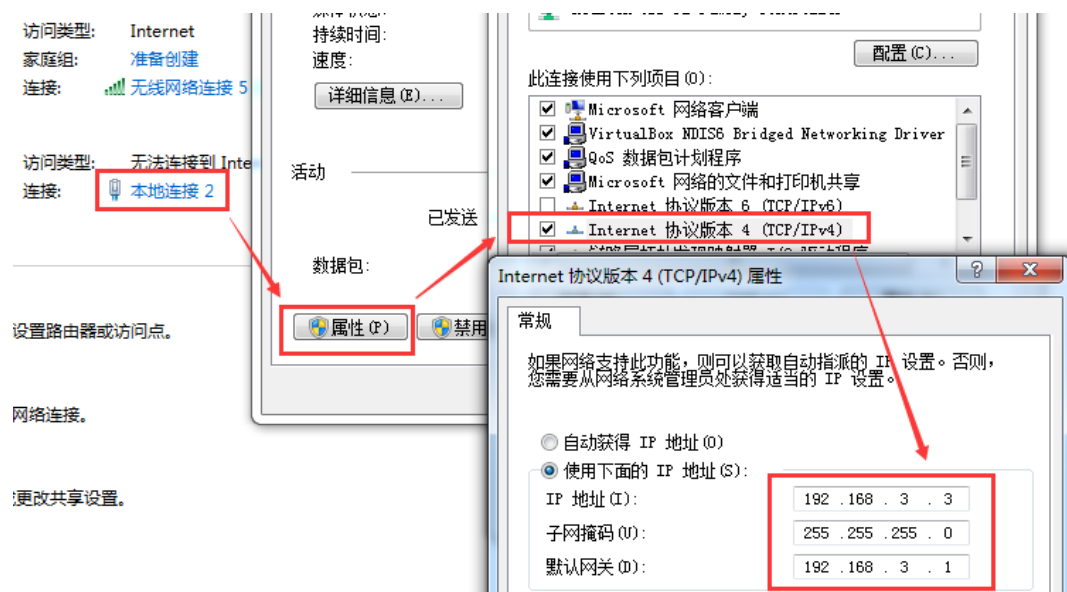
Node Name	Direction	Location	I/O Bank
in cam_data[7]	Input	PIN_D5	8
in cam_data[6]	Input	PIN_P1	2
in cam_data[5]	Input	PIN_E5	1
in cam_data[4]	Input	PIN_P2	2
in cam_data[3]	Input	PIN_F3	1
in cam_data[2]	Input	PIN_R1	2
in cam_data[1]	Input	PIN_F5	1
in cam_data[0]	Input	PIN_G5	1
in cam_hsync	Input	PIN_N2	2
in cam_pclk	Input	PIN_N1	2
out cam_pdown	Output	PIN_J6	2
out cam_reset	Output	PIN_L6	2
out cam_scl	Output	PIN_D4	1
io cam_sda	Bidir	PIN_L2	2
in cam_vsync	Input	PIN_D3	8
out cam_xclk	Output	PIN_C3	8
in clk	Input	PIN_M16	5
in e_rx[1]	Input	PIN_A7	8
in e_rx[0]	Input	PIN_B8	8
in e_rxdclk	Input	PIN_E1	1
in e_rxdv	Input	PIN_A8	8
in e_rxer	Input	PIN_A9	7
out e_tx[1]	Output	PIN_A6	8
out e_tx[0]	Output	PIN_B7	8
out e_txen	Output	PIN_B6	8
out e_xi	Output	PIN_A5	8
out led	Output	PIN_F6	8

11.6. 实验操作

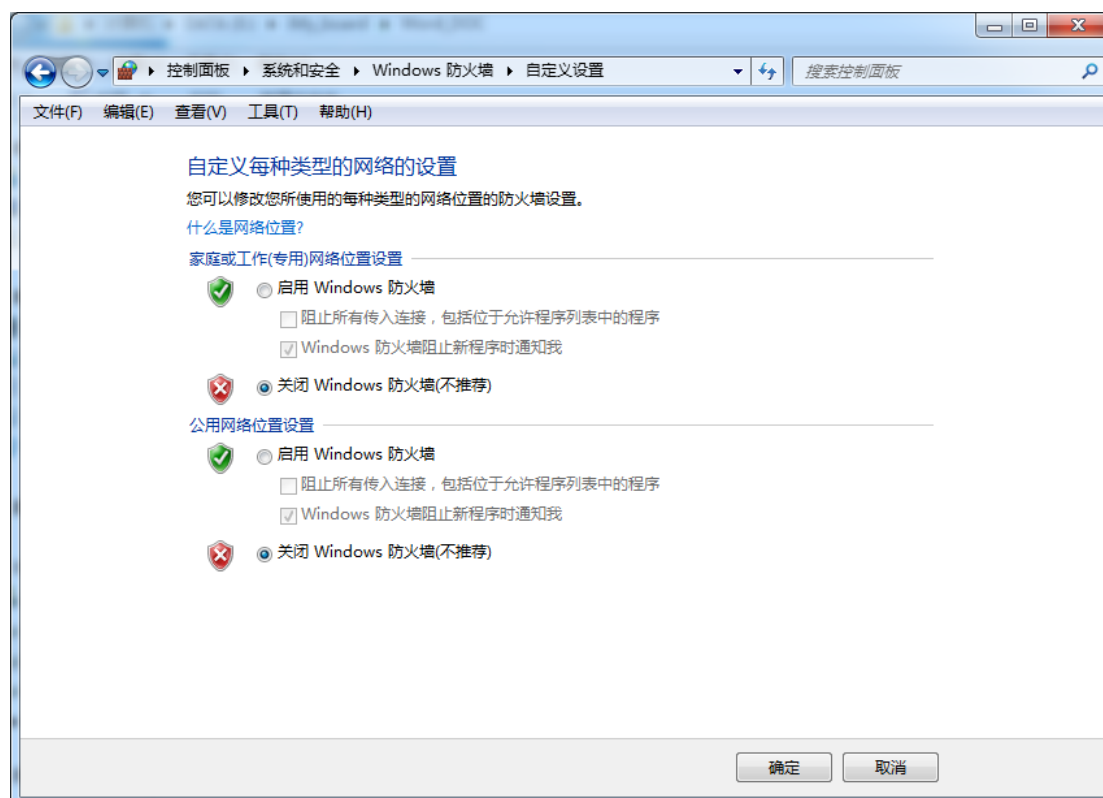
如下图所示插上 MT9V011 摄像头模块(注意 1 脚要对应, 否则不能正常工作)和网线。



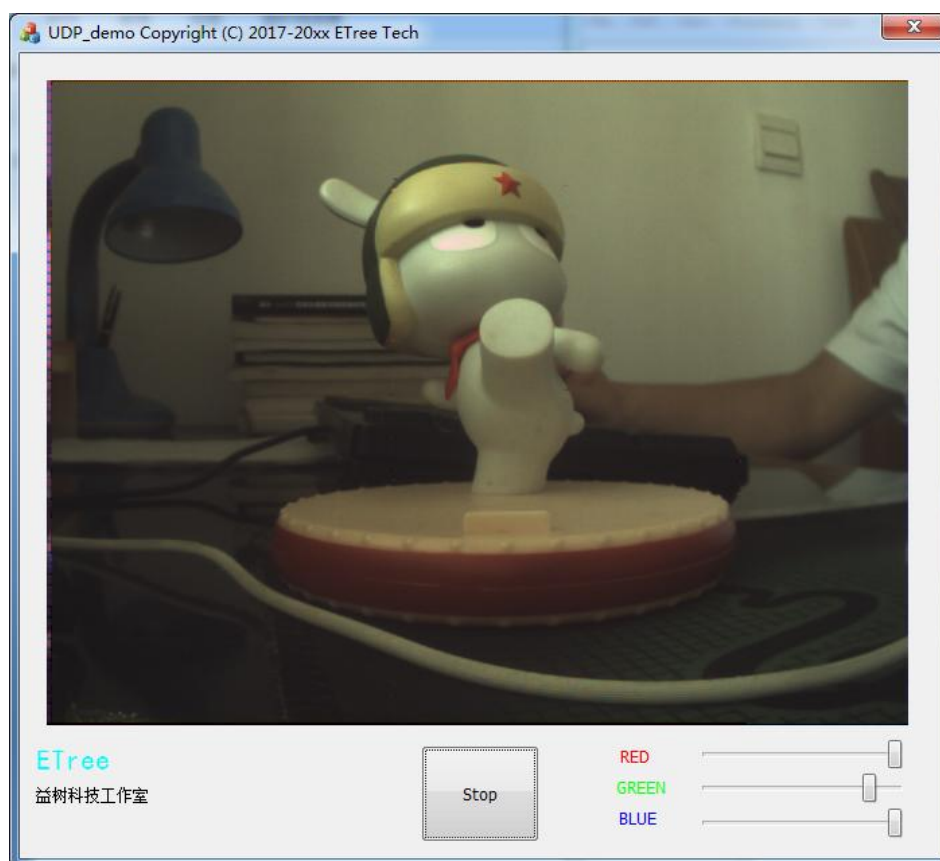
打开电源后, 会看到绿灯闪烁, 表示网络识别了。再将本地连接的 IP 地址设置成 192.168.3.3, 子网掩码设置成 255.255.255.0, 网关设置成 192.168.3.1, 具体如下图所示:



关闭防火墙, 否则电脑无法接受视频数据。



下载配置文件, 网卡的绿灯加快闪烁, 打开 Tools 目录的 UDP_demo_VGA.exe 应用程序, 点击 'Start' 按钮, 上位机实时显示 CMOS 的视频数据, 如果需要手动调整 R/G/B 各通道的分量, 可拖动界面右下方的三个进度条, 这样间接的实现了手动白平衡。具体显示结果如下图所示, 效果秒杀其他低端 30 万像素摄像头(OV7670 等)。



第十四章 数字频率计

本章设计一款数字频率计，计数范围为 0~99Hz。数字频率计用来测量摄像头 MT9V011 默认的输出帧率，并在数码管上观察结果。

14.1. 硬件介绍

本章用到摄像头模块和数码管，之前已经详细介绍过了，这里不再重复描述。

14.2. HDL 设计

数字频率计以板卡上的 50MHz 晶振作为时钟参考，定义一个秒计数器和频率值计数器，然后通过捕捉 1 秒时间内 CMOS_FV 帧同步信号的上升沿的个数来计算 CMOS 图像传感器的输出帧率。

数字频率计代码设计如下。

```
//file name      : frame_counter.v
//data          : 2017.10.1
//author        : ETree
//function      : frame counter
//log           :

module frame_counter(
    input clk, //50MHz
    input rst_n,

    input cmos_vsync,

    //ledsm
    output [7:0] sm_dat,
    output [3:0] sm_sel
);

//-----frame count-----
reg [25:0] cnt_1sec;
reg [7:0] cnt_freq;
reg [1:0] cmos_vsync_r;
reg [15:0] frame_out;

always @(posedge clk)
    cmos_vsync_r <= {cmos_vsync_r[0],cmos_vsync};

wire cmos_vsync_pos = (cmos_vsync_r==2'b01);
```

```
always @(posedge clk)
    if(~rst_n)
        cnt_1sec <= 0;
    else if(cnt_1sec==49_999_999)
        cnt_1sec <= 0;
    else
        cnt_1sec <= cnt_1sec + 1'b1;

always @(posedge clk)
    if(~rst_n)
        begin
            cnt_freq <= 0;
        end
    else if(cnt_1sec<49_999_999)
        begin
            if(cmos_vsync_pos)
                if(cnt_freq[3:0]>=4'd9)
                    begin
                        cnt_freq[3:0] <= 0;
                        cnt_freq[7:4] <= cnt_freq[7:4] + 1'b1;
                    end
                else
                    begin
                        cnt_freq[3:0] <= cnt_freq[3:0] + 1'b1;
                        cnt_freq[7:4] <= cnt_freq[7:4];
                    end
            else
                cnt_freq <= cnt_freq;
        end
    else
        cnt_freq <= 0;

always @(posedge clk)
    if(~rst_n)
        frame_out <= 0;
    else if(cnt_1sec==49_999_998)
        frame_out <= cnt_freq;
    else
        frame_out <= frame_out;

//-----数码管显示-----
parameter ZERO    = 8'b0000_0011, //8'b1100_0000,
              ONE   = 8'b1001_1111, //8'b1111_1001,
```

```

TWO      = 8'b0010_0101, //8'b1010_0100,
THREE    = 8'b0000_1101, //8'b1011_0000,
FOUR     = 8'b1001_1001, //8'b1001_1001,
FIVE     = 8'b0100_1001, //8'b1001_0010,
SIX      = 8'b0100_0001, //8'b1000_0010,
SEVEN    = 8'b0001_1111, //8'b1111_1000,
EIGHT    = 8'b0000_0001, //8'b1000_0000,
NINE     = 8'b0000_1001, //8'b1001_0000;
AA       = 8'b0001_0001,
BB       = 8'b1100_0001,
CC       = 8'b0110_0011,
DD       = 8'b1000_0101,
EE       = 8'b0110_0001,
FF       = 8'b0111_0001;

reg [7:0] sm_dat_r; //数码管段选
reg [3:0] sm_sel_r; //数码管位选
reg [7:0] sm_dat_r0; //数码管段选寄存器 0
reg [7:0] sm_dat_r1; //数码管段选寄存器 1
reg [7:0] sm_dat_r2=0; //数码管段选寄存器 0
reg [7:0] sm_dat_r3=0; //数码管段选寄存器 1
reg [1:0] sm_cnt; //数码管片选计数器
reg [19:0] timer1; //1 毫秒计数器

//-----数码管片选计数器-----
always @(posedge clk or negedge rst_n)
    if (~rst_n)
        timer1 <= 0; //计数器清零
    else if (timer1 == 32'd199_999)
        timer1 <= 0; //计数器清零
    else
        timer1 <= timer1 + 1'b1; //计数器加 1

always @(posedge clk or negedge rst_n)
    if (~rst_n)
        sm_cnt <= 0;
    else if (timer1==0)
        sm_cnt <= sm_cnt + 1'b1;
    else
        sm_cnt <= sm_cnt;

//-----数码管译码-----
always @ (*)
    case( frame_out[3:0] )

```

```
4'd0 : sm_dat_r0 <= ZERO;
4'd1 : sm_dat_r0 <= ONE;
4'd2 : sm_dat_r0 <= TWO;
4'd3 : sm_dat_r0 <= THREE;
4'd4 : sm_dat_r0 <= FOUR;
4'd5 : sm_dat_r0 <= FIVE;
4'd6 : sm_dat_r0 <= SIX;
4'd7 : sm_dat_r0 <= SEVEN;
4'd8 : sm_dat_r0 <= EIGHT;
4'd9 : sm_dat_r0 <= NINE;
4'd10: sm_dat_r0 <= AA ;
4'd11: sm_dat_r0 <= BB ;
4'd12: sm_dat_r0 <= CC ;
4'd13: sm_dat_r0 <= DD ;
4'd14: sm_dat_r0 <= EE ;
4'd15: sm_dat_r0 <= FF ;
default:sm_dat_r0 <= 8'hFF;
endcase

always @ (*)
case( frame_out[7:4] )
4'd0 : sm_dat_r1 <= ZERO;
4'd1 : sm_dat_r1 <= ONE;
4'd2 : sm_dat_r1 <= TWO;
4'd3 : sm_dat_r1 <= THREE;
4'd4 : sm_dat_r1 <= FOUR;
4'd5 : sm_dat_r1 <= FIVE;
4'd6 : sm_dat_r1 <= SIX;
4'd7 : sm_dat_r1 <= SEVEN;
4'd8 : sm_dat_r1 <= EIGHT;
4'd9 : sm_dat_r1 <= NINE;
4'd10: sm_dat_r1 <= AA ;
4'd11: sm_dat_r1 <= BB ;
4'd12: sm_dat_r1 <= CC ;
4'd13: sm_dat_r1 <= DD ;
4'd14: sm_dat_r1 <= EE ;
4'd15: sm_dat_r1 <= FF ;
default:sm_dat_r1 <= 8'hFF;
endcase

//-----数码管显示-----
always @(sm_cnt)
case(sm_cnt)
```

```
'd0:
begin
    sm_sel_r <= 4'b1101;
    sm_dat_r <= sm_dat_r0;
end
'd1:
begin
    sm_sel_r <= 4'b1110;
    sm_dat_r <= sm_dat_r1;
end
'd2:
begin
    sm_sel_r <= 4'b0111;
    sm_dat_r <= ZERO;
end
'd3:
begin
    sm_sel_r <= 4'b1011;
    sm_dat_r <= ZERO;
end
default:
begin
    sm_sel_r <= 4'b1111;
    sm_dat_r <= ZERO;
end
endcase

assign sm_sel = sm_sel_r;
assign sm_dat = sm_dat_r;

endmodule
```

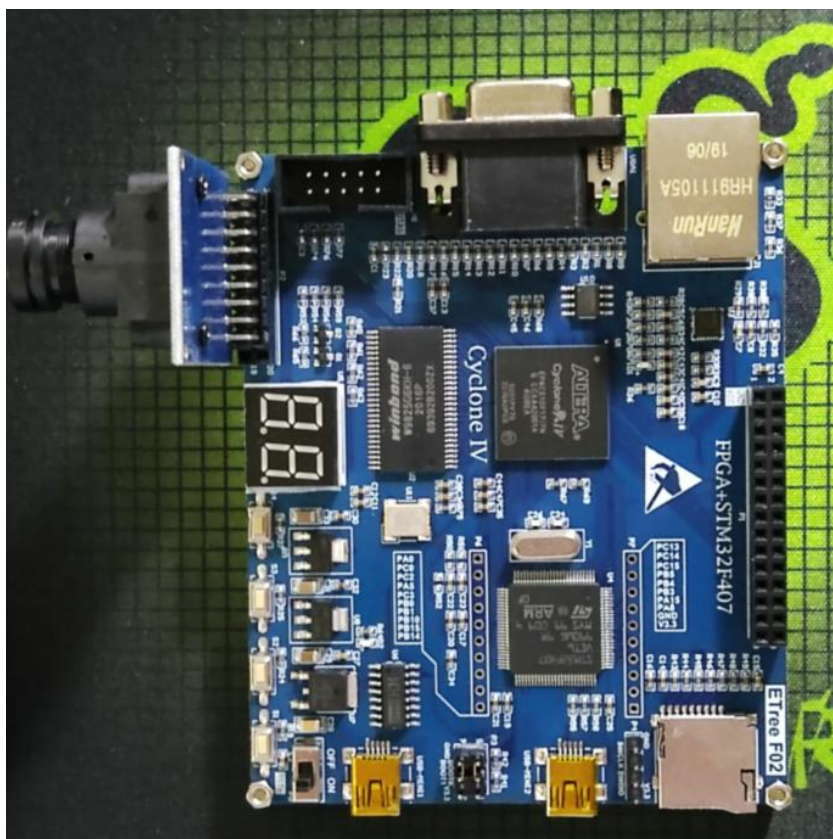
14.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

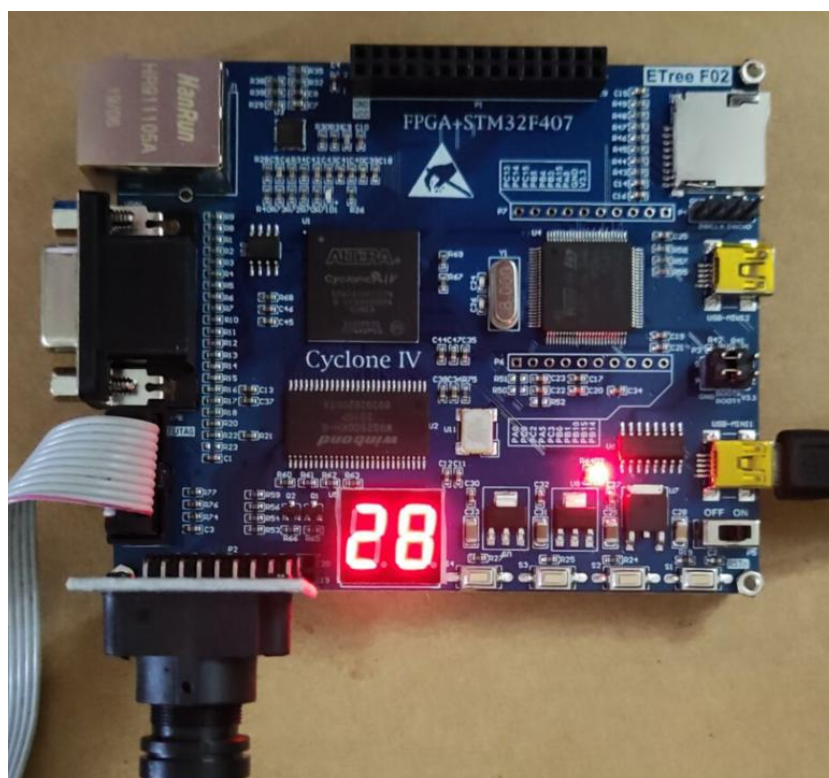
Node Name	Direction	Location	I/O Bank
in cam_data[7]	Input	PIN_D5	8
in cam_data[6]	Input	PIN_P1	2
in cam_data[5]	Input	PIN_E5	1
in cam_data[4]	Input	PIN_P2	2
in cam_data[3]	Input	PIN_F3	1
in cam_data[2]	Input	PIN_R1	2
in cam_data[1]	Input	PIN_F5	1
in cam_data[0]	Input	PIN_G5	1
in cam_hsync	Input	PIN_N2	2
in cam_pclk	Input	PIN_N1	2
out cam_pdown	Output	PIN_J6	2
out cam_reset	Output	PIN_L6	2
out cam_scl	Output	PIN_D4	1
io cam_sda	Bidir	PIN_L2	2
in cam_vsync	Input	PIN_D3	8
out cam_xclk	Output	PIN_C3	8
in clk	Input	PIN_M16	5
in e_rx[1]	Input	PIN_A7	8
in e_rx[0]	Input	PIN_B8	8
in e_rxd	Input	PIN_E1	1
in e_rxdv	Input	PIN_A8	8
in e_rxd	Input	PIN_A9	7
out e_tx[1]	Output	PIN_A6	8
out e_tx[0]	Output	PIN_B7	8
out e_txen	Output	PIN_B6	8
out e_xi	Output	PIN_A5	8
out led	Output	PIN_F6	8
in rst_n	Input	PIN_L10	4
out sm_dat[7]	Output	PIN_R4	3
out sm_dat[6]	Output	PIN_T3	3
out sm_dat[5]	Output	PIN_R3	3
out sm_dat[4]	Output	PIN_T2	3
out sm_dat[3]	Output	PIN_L7	3
out sm_dat[2]	Output	PIN_N3	3
out sm_dat[1]	Output	PIN_P3	3
out sm_dat[0]	Output	PIN_M6	3
out sm_sel[1]	Output	PIN_L4	2
out sm_sel[0]	Output	PIN_L3	2

14.4. 实验操作

如下图所示插上 MT9V011 摄像头模块(注意 1 脚要对应，否则不能正常工作)。



打开电源后，下载 FPGA 配置文件，可以看到数码管实时显示摄像头模块的视频数据帧率。



第十五章 上位机调试 CMOS 摄像头参数

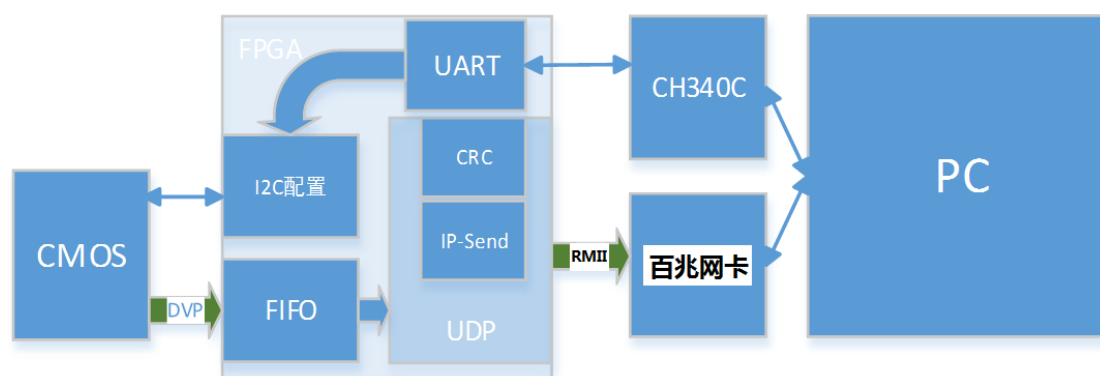
本章通过上位机软件实时修改摄像头 MT9V011 的寄存器，实现调试 CMOS 摄像头的参数功能。

15.1. 硬件介绍

本章应用到 IP101GR、MT9V011 摄像头模块和 UART，这些硬件电路前面已经做过详细介绍，故不再重复。

15.2. HDL 设计

➤ 系统框架



➤ 串口接收模块设计

串口接收上位机发送过来的连续 3 字节数据，分别为 1 字节地址和 2 字节数据。

```
//file name:      myuart_rx.v
//author:         ETree
//date:           2017.11.6
//function:       数据接收
//log:

module myuart_rx (
    input clk,
    input rst_n,
    output reg bps_start,
    input clk_bps,
    input rxd,
    output reg rx_done,
    output reg [23:0] rx_data // {1 字节地址, 2 字节数据}
);

//串口接收 buffer
```

```
reg rxd0;
reg rxd1;
reg rxd2;
reg rxd3;

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        rxd0 <= 1'b1;
        rxd1 <= 1'b1;
        rxd2 <= 1'b1;
        rxd3 <= 1'b1;
    end
    else
    begin
        rxd0 <= rxd;
        rxd1 <= rxd0;
        rxd2 <= rxd1;
        rxd3 <= rxd2;
    end
end

wire rxd_pos;    //串口接收数据的上升沿
wire rxd_neg;    //串口接收数据的下降沿

assign rxd_pos = ~rxd3 & ~rxd2 & rxd1 & rxd0;
assign rxd_neg = rxd3 & ~rxd2 & ~rxd1 & ~rxd0;

//-----接收使能信号-----
reg rx_en;    //串口接收使能信号

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        bps_start <= 1'bz;
        rx_en <= 1'b0;
    end
    else
    if(rxd_neg)
    begin
        bps_start <= 1'b1;
        rx_en <= 1'b1;
    end
end
```

```
        end
    else
        if(num==4'd9)
            begin
                bps_start <= 1'b0;
                rx_en <= 1'b0;
            end
        else
            begin
                bps_start <= bps_start;
                rx_en <= rx_en;
            end
        end
    end

end

//-----串转并-----
reg [3:0] num;           //串口接收 Bit 计数器
reg [7:0] rx_data_r;     //串转并数据 buffer

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        num <= 0;
        rx_data_r <= 0;
    end
    else
    if(rx_en)
    begin
        if(clk_bps)
        begin
            num <= num + 1'b1;
            case(num)
                4'd1: rx_data_r[0] <= rxd;
                4'd2: rx_data_r[1] <= rxd;
                4'd3: rx_data_r[2] <= rxd;
                4'd4: rx_data_r[3] <= rxd;
                4'd5: rx_data_r[4] <= rxd;
                4'd6: rx_data_r[5] <= rxd;
                4'd7: rx_data_r[6] <= rxd;
                4'd8: rx_data_r[7] <= rxd;
            //
                4'd9: rx_data_r[8] <= rxd;
            default: rx_data_r <= rx_data_r;
            endcase
        end
    end
end
```

```
        end
        else
            if(num == 4'd9)
                begin
                    num <= 4'd0;
                    rx_data_r <= rx_data_r;
                end
            else
                begin
                    num <= num;
                    rx_data_r <= rx_data_r;
                end
            end
        end
    end
    else
        begin
            num <= num;
            rx_data_r <= rx_data_r;
        end
    end
end

//-----字节拼接-----
reg [1:0] cnt_rx;

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        rx_data <= 0;
        cnt_rx <= 0;
    end
    else if(num == 4'd9)
    begin
        rx_data <= {rx_data[15:0],rx_data_r};
        if(cnt_rx>=2 || rx_data==24'H555555)
            cnt_rx <= 0;
        else
            cnt_rx <= cnt_rx + 1'b1;
    end
    else
    begin
        cnt_rx <= cnt_rx;
        rx_data <= rx_data;
    end
end
end
```

```
reg [3:0] rx_done_cnt;

always @(posedge clk or negedge rst_n)
    if(~rst_n)
        rx_done_cnt <= 4'hf;
    else if(cnt_rx==1)
        rx_done_cnt <= 0;
    else if(rx_done_cnt>=4'he)
        rx_done_cnt <= rx_done_cnt;
    else
        rx_done_cnt <= rx_done_cnt + 1'b1;

always @(posedge clk or negedge rst_n)
    if(~rst_n)
        rx_done <= 0;
    else if(cnt_rx==2)
        rx_done <= 1'b1;
    else if(rx_done_cnt<4'he)
        rx_done <= 1'b1;
    else
        rx_done <= 0;

endmodule
```

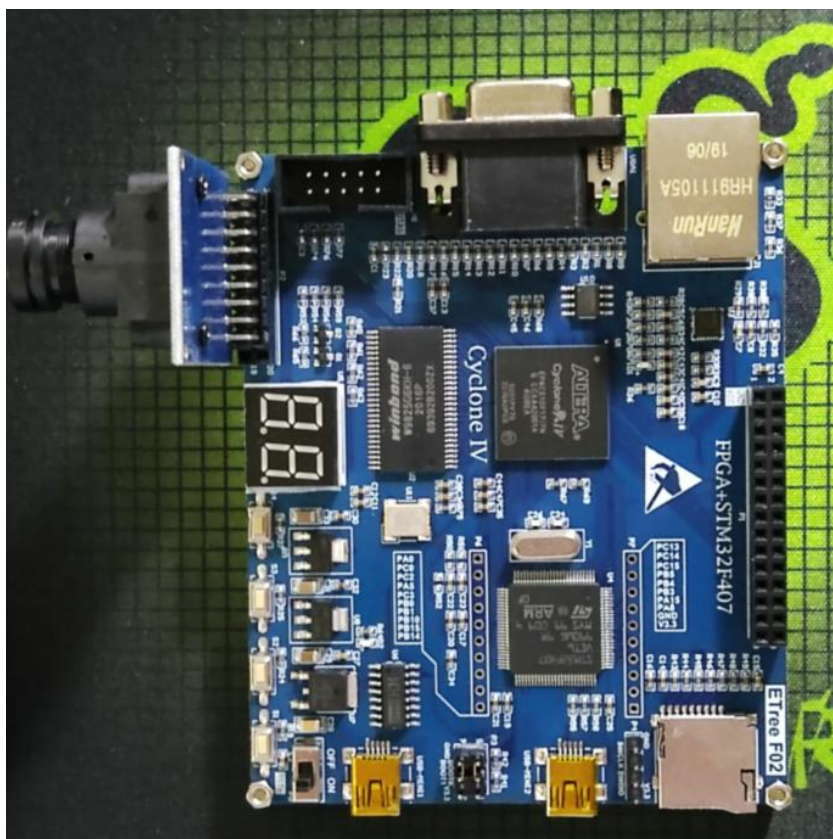
15.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

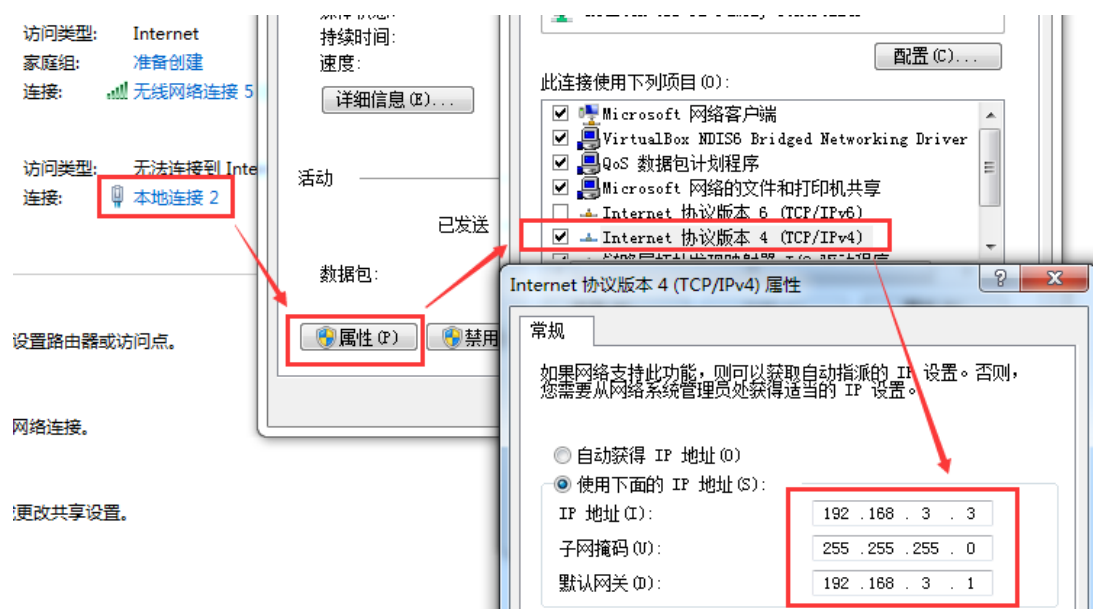
Node Name	Direction	Location	I/O Bank
in cam_data[7]	Input	PIN_D5	8
in cam_data[6]	Input	PIN_P1	2
in cam_data[5]	Input	PIN_E5	1
in cam_data[4]	Input	PIN_P2	2
in cam_data[3]	Input	PIN_F3	1
in cam_data[2]	Input	PIN_R1	2
in cam_data[1]	Input	PIN_F5	1
in cam_data[0]	Input	PIN_G5	1
in cam_hsync	Input	PIN_N2	2
in cam_pclk	Input	PIN_N1	2
out cam_pdown	Output	PIN_J6	2
out cam_reset	Output	PIN_L6	2
out cam_scl	Output	PIN_D4	1
io cam_sda	Bidir	PIN_L2	2
in e_vsync	Input	PIN_D3	8
out cam_xclk	Output	PIN_C3	8
in clk	Input	PIN_M16	5
in e_rx[1]	Input	PIN_A7	8
in e_rx[0]	Input	PIN_B8	8
in e_rxd	Input	PIN_E1	1
in e_rxdv	Input	PIN_A8	8
in e_rxr	Input	PIN_A9	7
out e_tx[1]	Output	PIN_A6	8
out e_tx[0]	Output	PIN_B7	8
out e_txen	Output	PIN_B6	8
out e_xi	Output	PIN_A5	8
out led	Output	PIN_F6	8
in rst_n	Input	PIN_L10	4
in uart_rx	Input	PIN_T13	4
out uart_tx	Output	PIN_R13	4

15.4. 实验操作

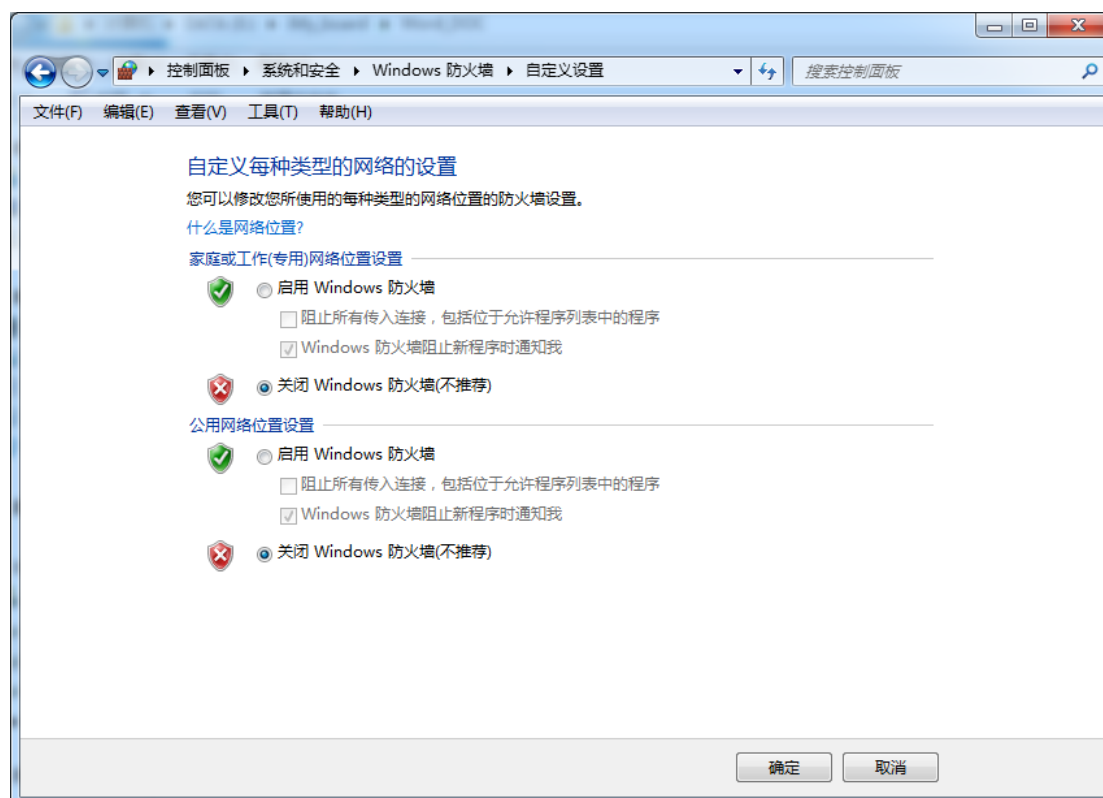
如下图所示插上 MT9V011 摄像头模块(注意 1 脚要对应, 否则不能正常工作)和网线。



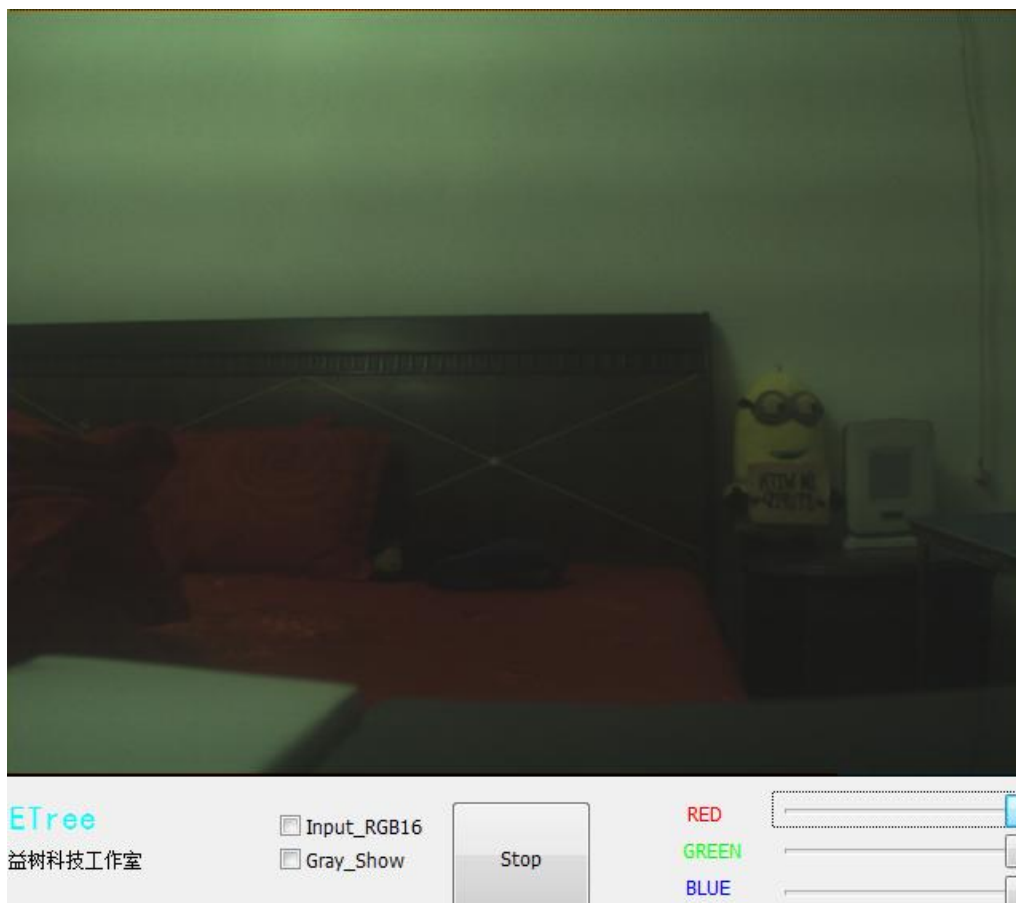
打开电源后，会看到绿灯闪烁，表示网络识别了。再将本地连接的 IP 地址设置成 192.168.3.3，子网掩码设置成 255.255.255.0，网关设置成 192.168.3.1，具体如下图所示：



关闭防火墙，否则电脑无法接受视频数据。



下载配置文件, 网卡的绿灯加快闪烁, 打开 Tools 目录的 UDP_demo_VGA.exe 应用程序, 点击 'Start' 按钮, 上位机实时显示 CMOS 的视频数据, 如下图所示, 图像亮度偏暗, 且由于环境色温(LED 日光灯)导致颜色偏绿红。



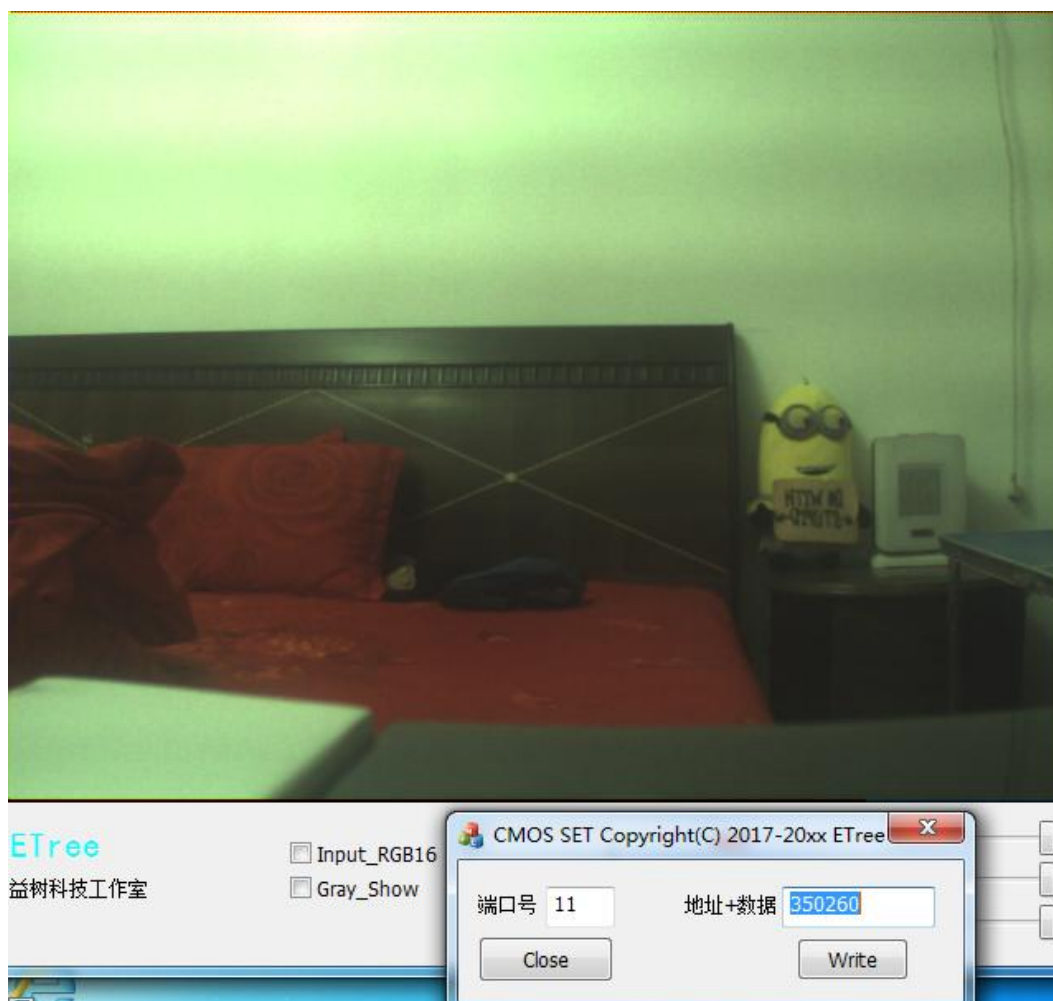
打开 Tools 目录的 CMOS_SET.exe 软件，设置端口号为 11(在设备管理器中查询端口号)，点击 ‘Open’ 按钮，在 ‘地址+数据’ 栏默认弹出 350235(十六进制，35 为地址，0235 为数据)。



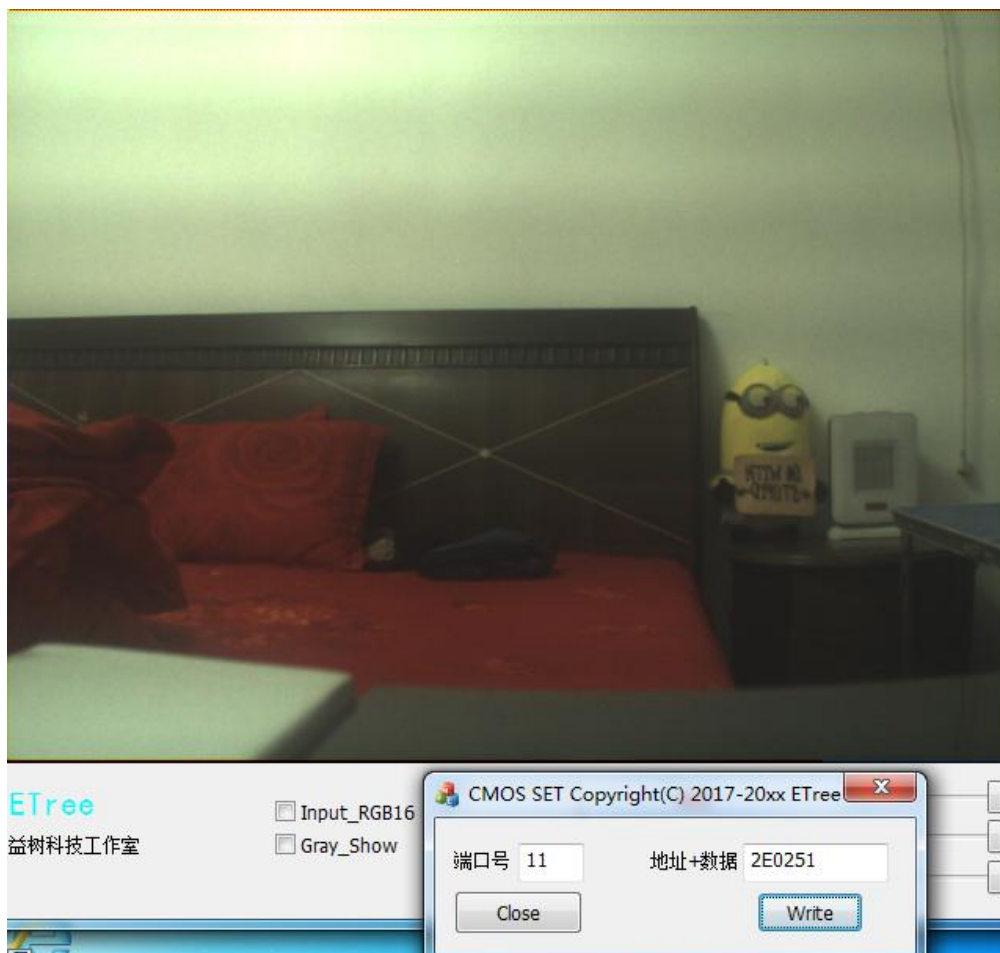
查询 MT9V011 手册知，全局增益地址为 0x35，G1 通道增益地址为 0x2B，B 通道增益地址为 0x2C，R 通道增益地址为 0x2D，G2 通道增益为 0x2E。

现在要通过在线调试这几个寄存器值，来优化图像效果。

首选，调整全局亮度，在 ‘地址+数据’ 栏输入 ‘350260’，点击 ‘Write’ 按钮，图像亮度得到增强，如下图所示。



然后，降低 G1 和 G2 通道的增益，在‘地址+数据’栏输入‘2B0251’，点击按钮‘Write’写入；再在‘地址+数据’栏输入‘2E0251’，点击按钮‘Write’写入；图像效果如下，绿色分量明显减弱。



虽然图像的绿色分量减弱了，但感觉图像偏黄，说明蓝色分量偏弱，下面通过软件增强蓝色分量，在编辑栏里输入“2C026F”，点击‘Write’按钮。最终效果图如下所示，达到了人眼的视觉效果。

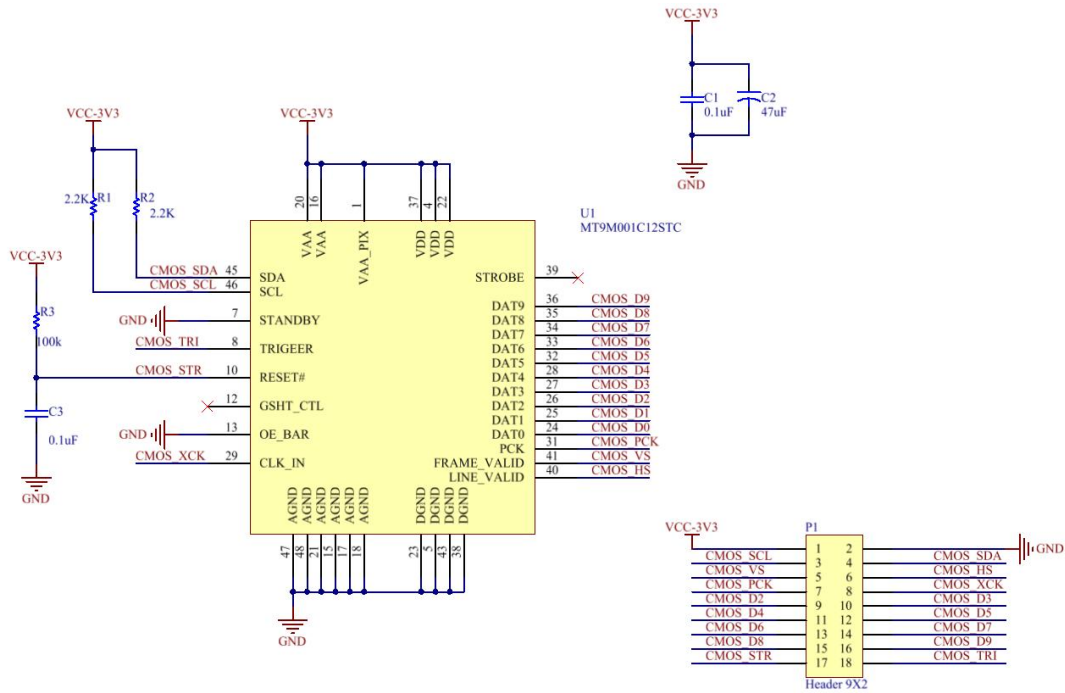


第十六章 MT9M001 摄像头以太网显示

本章基于百兆以太网卡 IP101GR 和 130 万像素摄像头模块 MT9M001，将摄像头的 8 帧/秒的视频输出通过以太网传送到 PC，然后上位机实时显示视频数据。

16.1. 硬件介绍

MT9M001 摄像头硬件原理图如下图所示。

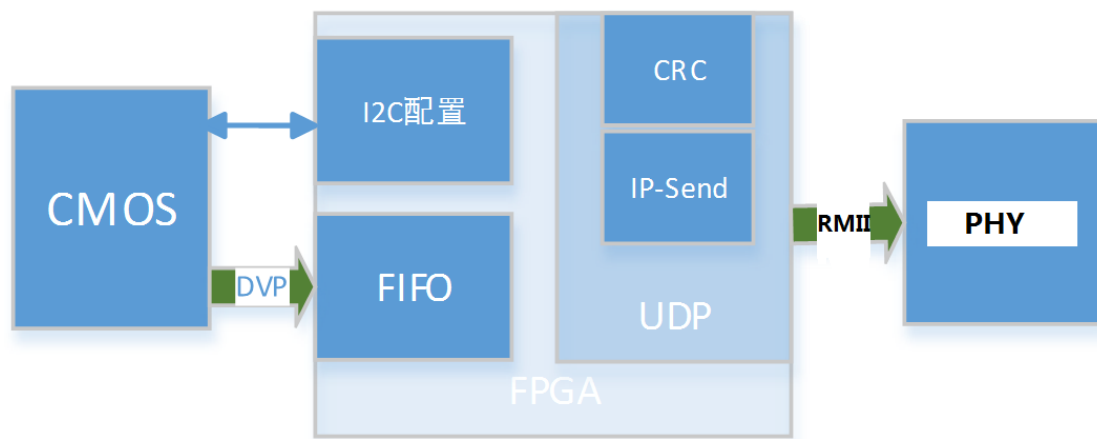


MT9M001 为 130 万像素摄像头芯片，工业级摄像头芯片，具有超大 1/2" 的感光靶面，广泛应用于工业相机中。具体的摄像头参数如下表所示。

Parameter		Typical Value
Optical format		1/2-inch (5:4)
Active imager size		6.66mm(H) x 5.32mm(V)
Active pixels		1,280H x 1,024V
Pixel size		5.2μm x 5.2μm
Shutter type		Electronic rolling shutter (ERS)
Maximum data rate/ master clock		48 MPS/48 MHz
Frame rate	SXGA (1280 x 1024)	30 fps progressive scan; programmable
ADC resolution		10-bit, on-chip
Responsivity		2.1 V/lux-sec
Dynamic range		68.2dB
SNR _{MAX}		45dB
Supply voltage		3.0V—3.6V, 3.3V nominal
Power consumption		325mW at 3.3V; Standby 275μW
Operating temperature		0°C to +70°C
Packaging		48-pin CLCC

16.2. HDL 设计

➤ 系统框架



- ✧ I2C 配置模块负责初始化 CMOS 图像传感器，硬件配置接口为 I2C；
- ✧ FIFO 模块负责将 CMOS 的 DVP 信号缓存起来，并准备发给 UDP 发送模块；
- ✧ IPSEND 模块负责将视频数据打包成 UDP 协议流发送给 IP101GR 网卡芯片；
- ✧ CRC 模块负责生产 CRC 校验码。

➤ 摄像头芯片配置(I2C)

MT9M001 的配置接口也为 I2C，具体的时序和 MT9V011 一样，这里就不详细介绍了。MT9M001 寄存器列表如下表所示，这里只配置常用的几个寄存器。

Register # (Hex)	Description	Data Format (Binary)	Default Value (Hex)
0x00	Chip Version	1000 0100 0001 0001	0x8431
0x01	Row Start	0000 0ddd dddd dddd	0x000C
0x02	Column Start	0000 0ddd dddd dddd	0x0014
0x03	Row Size (Window Height)	0000 0ddd dddd dddd	0x03FF
0x04	Col Size (Window Width)	0000 0ddd dddd dddd	0x04FF
0x05	Horizontal Blanking	0000 0ddd dddd dddd	0x0009
0x06	Vertical Blanking	0000 0ddd dddd dddd	0x0019
0x07	Output Control	0000 0000 0d00 00dd	0x0002
0x09	Shutter Width	00dd dddd dddd dddd	0x0419
0x0B	Restart	0000 0000 0000 000d	0x0000
0x0C	Shutter Delay	0000 0ddd dddd dddd	0x0000
0x0D	Reset	0000 0000 0000 000d	0x0000
0x1E	Read Options 1	1000 dddd 00dd dd00	0x8000
0x20	Read Options 2	dd01 0dd1 d00d d10d	0x1104
0x2B	Even Row, Even Column	0000 0000 0ddd dddd	0x0008
0x2C	Odd Row, Even Column	0000 0000 0ddd dddd	0x0008
0x2D	Even Row, Odd Column	0000 0000 0ddd dddd	0x0008
0x2E	Odd Row, Odd Column	0000 0000 0ddd dddd	0x0008
0x35	Global Gain	0000 0000 0ddd dddd	0x0008
0x5F	Cal Threshold	dddd dddd d0dd dddd	0x0904
0x60	Even Row, Even Column	0000 000d dddd dddd	0x0000
0x61	Odd Row, Odd Column	0000 000d dddd dddd	0x0000
0x62	Cal Ctrl	d00d d100 1001 1ddd	0x0498
0x63	Even Row, Odd Column	0000 000d dddd dddd	0x0000
0x64	Odd Row, Even Column	0000 000d dddd dddd	0x0000
0xF1	Chip Enable	0000 0000 0000 00dd	0x0001

//以下代码来自于 camera_if-->iic_com.v 文件

```

always @(*)
begin
    case(lut_index)
        0 : lut_data    <= 24'h09_0419;
        //曝光时间 默认值 419
        1 : lut_data    <= {16'h3500,8'h08};
        //增益 0x08-0x20:1-4 倍;    0x51-0x60 :4.25-8 倍;    0x61-0x67: 9-15 倍
        2 : lut_data    <= 24'h20_1180;
        3 : lut_data    <= 24'h0100ae;
        //row start
        4 : lut_data    <= 24'h020011;
        //column start
        5 : lut_data    <= 24'h0302cf;
        //number of rows -1:720
        6 : lut_data    <= 24'h0404ff;
        //number of columns - 1:1280
        7 : lut_data    <= {8'h20,16'h9104};
        //SKIP=0
        default:lut_data    <= 24'h000000;
    endcase
end

```

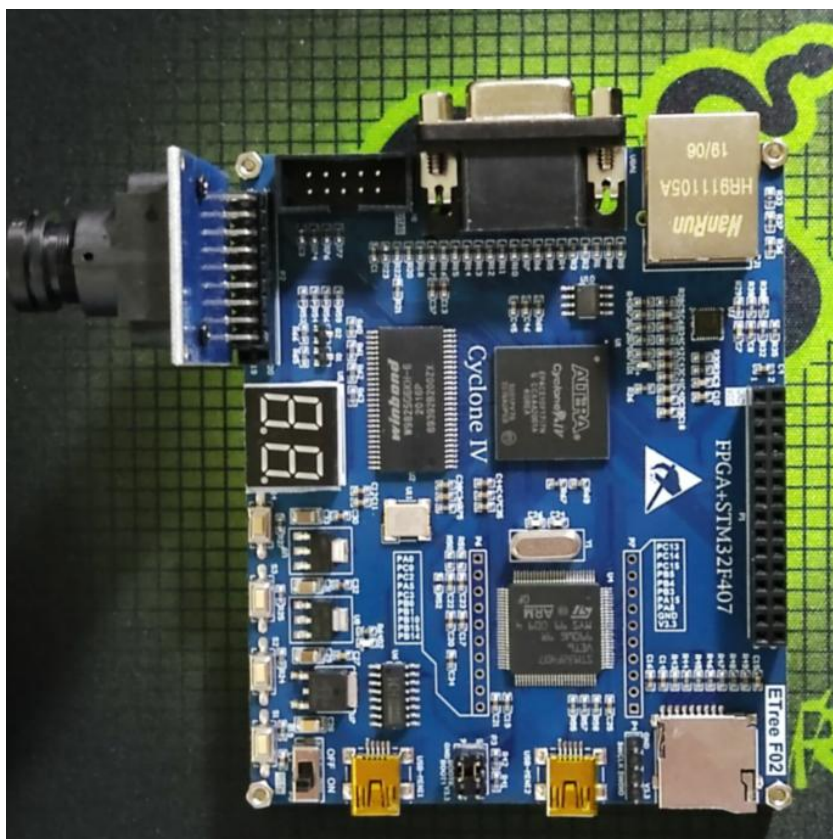

16.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

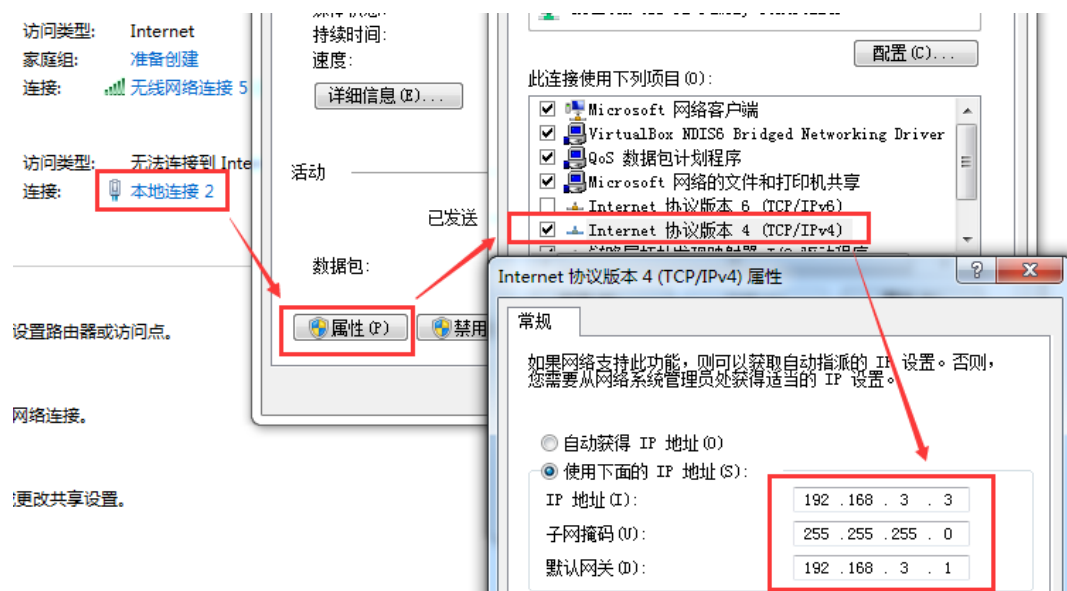
Node Name	Direction	Location	I/O Bank
 cam_data[7]	Input	PIN_G5	1
 cam_data[6]	Input	PIN_F5	1
 cam_data[5]	Input	PIN_R1	2
 cam_data[4]	Input	PIN_F3	1
 cam_data[3]	Input	PIN_P2	2
 cam_data[2]	Input	PIN_E5	1
 cam_data[1]	Input	PIN_P1	2
 cam_data[0]	Input	PIN_D5	8
 cam_hsync	Input	PIN_N1	2
 cam_pclk	Input	PIN_D3	8
 cam_pdown	Output	PIN_J6	2
 cam_reset	Output	PIN_L6	2
 cam_scl	Output	PIN_D4	1
 cam_sda	Bidir	PIN_L2	2
 cam_vsync	Input	PIN_C3	8
 cam_xclk	Output	PIN_N2	2
 clk	Input	PIN_M16	5
 e_rx[1]	Input	PIN_A7	8
 e_rx[0]	Input	PIN_B8	8
 e_rxclk	Input	PIN_E1	1
 e_rxdv	Input	PIN_A8	8
 e_rxer	Input	PIN_A9	7
 e_tx[1]	Output	PIN_A6	8
 e_tx[0]	Output	PIN_B7	8
 e_txen	Output	PIN_B6	8
 e_xi	Output	PIN_A5	8
 led	Output	PIN_F6	8
 rst_n	Input	PIN_L10	4

16.4. 实验操作

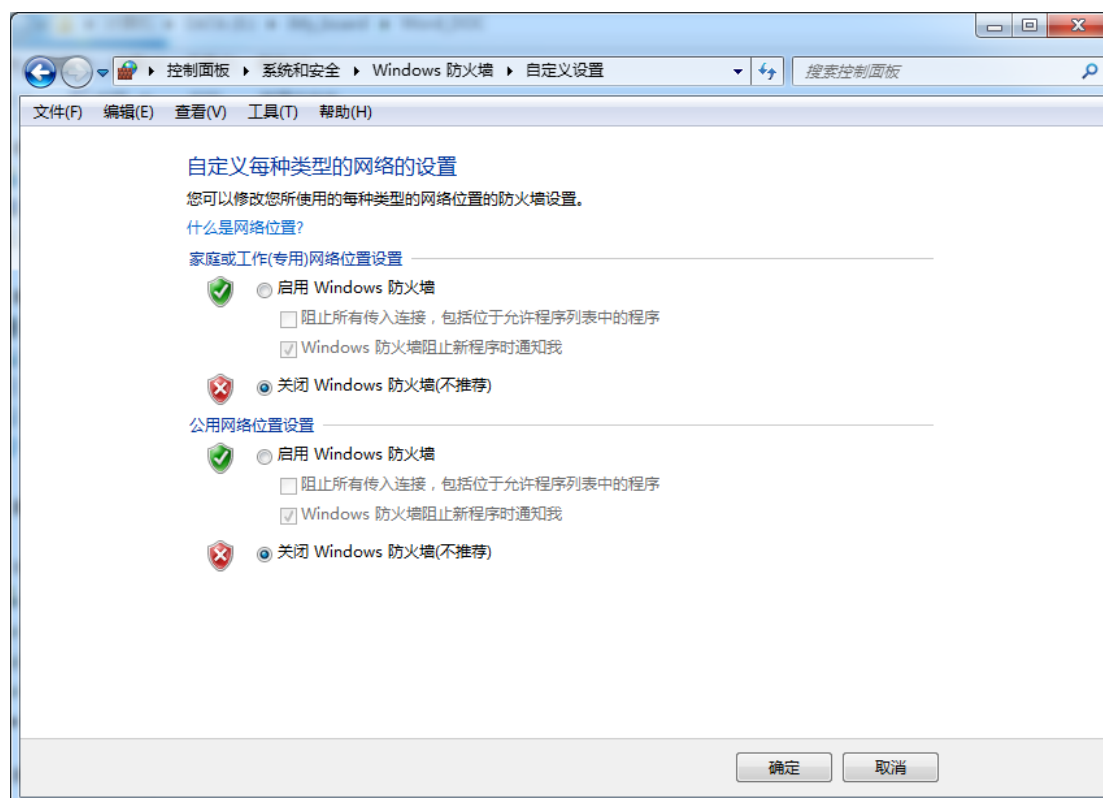
如下图所示插上 MT9M001 摄像头模块(注意 1 脚要对应，否则不能正常工作)和网线。



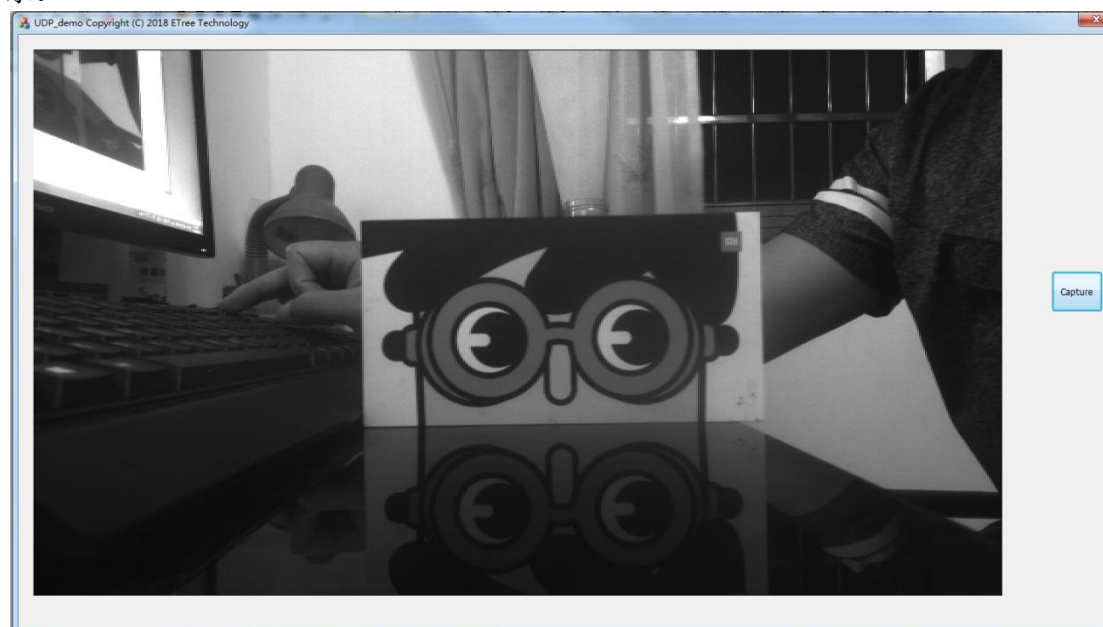
打开电源后，会看到绿灯闪烁，表示网络识别了。再将本地连接的 IP 地址设置成 192.168.3.3，子网掩码设置成 255.255.255.0，网关设置成 192.168.3.1，具体如下图所示：



关闭防火墙，否则电脑无法接受视频数据。



下载配置文件, 网卡的绿灯加快闪烁, 打开 Tools 目录的 “UDP_demo_720p_Gray.exe” 应用程序, 点击 ‘Start’ 按钮, 上位机实时显示 CMOS 的视频数据。具体显示结果如下图所示。

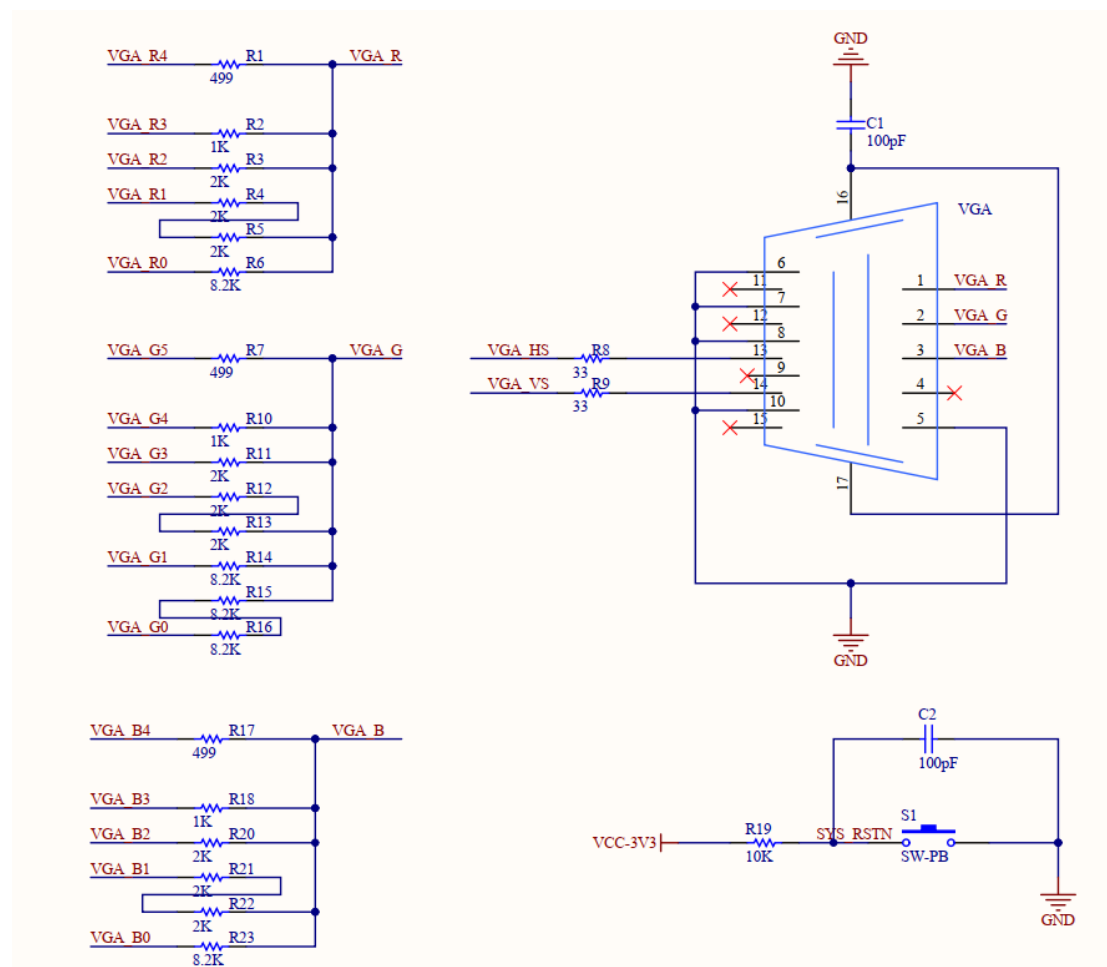


第十七章 VGA 彩条显示

本章基于 VGA 接口，实现显示器彩条显示。

17.1. 硬件介绍

FPGA 板卡采用电阻并联分压的方式实现了 VGA565(RED->5bit, GREEN->6bit, BLUE->5bit) 接口，可以实现 65536 种颜色的显示，具体原理图如下所示。



考虑到底成本方案,我们采用了电阻分压网络模拟 DAC 来替换 ADV7123 视频转换芯片。下面简单描述一下设计原理。

三基色源端和终端匹配电阻都为 75 欧姆,如下图所示:

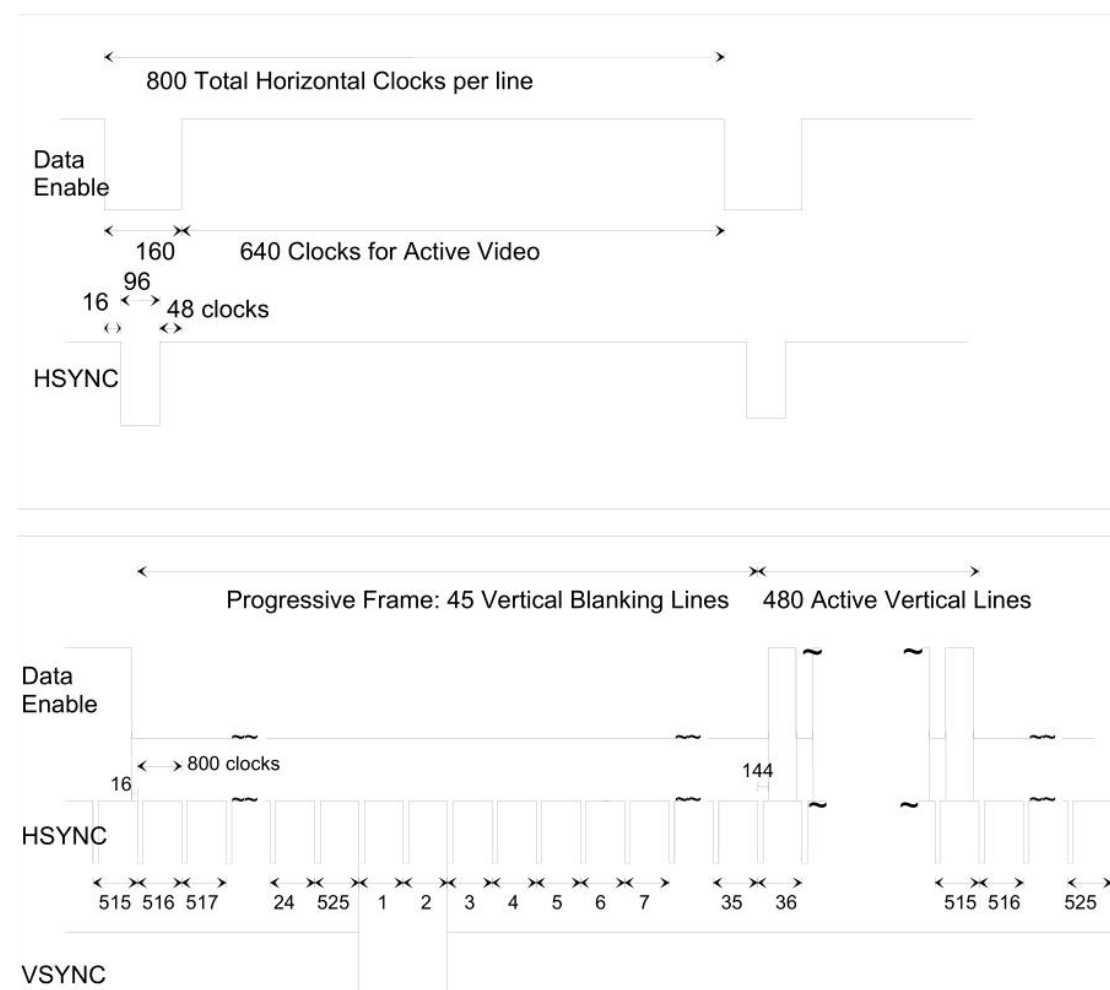


VGA 接口规定三基色分别采用模拟电压 0~0.714V 来表示, 0~0.714V 可以通过不同的电阻网络分压得出。

17.2. VGA 时序

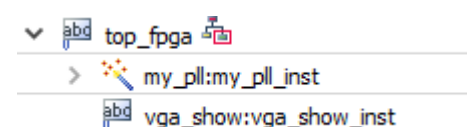
VGA(Video Graphics Array)是 IBM 在 1987 年推出的一种视频传输, 具有分辨率高、显示速率快、颜色丰富等优点, 在彩色显示器领域得到了广泛的应用。不支持热插拔, 不支持音频传输。

本设计使用的是 640×480@60Hz 的 VGA 时序接口, 计算像素时钟 $pclk = 800 \times 525 \times 60 = 25200000$, 注意这里的 HSYNC(行同步信号)和 VSYNC(场同步信号)都是低电平有效, 且 HSYNC 在数据行无效区域也要提供。



17.3. HDL 设计

➤ 工程目录



- ✧ my_pll 模块：生成需要的时钟。
- ✧ vga_show：生成需要的 VGA 时序接口。
- VGA 模块

```
//file name:    vga_show.v
//author:      ETree
//date:        2021.10.2
//function:    VGA 显示器驱动
//log:

module vga_show(
    input        vga_clk,
    input        rst_n,                //VGA 复位，低电平有效

    output       vga_hsync,
    output       vga_vsync,
    output [4:0] vga_r,
    output [5:0] vga_g,
    output [4:0] vga_b
);

parameter RED      = 16'b11111_000000_00000;
parameter GREEN    = 16'b00000_111111_00000;
parameter BLUE     = 16'b00000_000000_11111;
parameter YELLOW   = 16'b11111_111111_00000;

// 水平扫描参数的设定 640*480 VGA 60FPS_25MHz
parameter LinePeriod    = 800;           //行周期数
parameter H_SyncPulse   = 96;           //行同步脉冲 (Sync a)
parameter H_BackPorch   = 48;           //显示后沿 (Back porch b)
parameter H_ActivePix    = 640;         //显示时序段 (Display interval c)
parameter H_FrontPorch  = 16;           //显示前沿 (Front porch d)
parameter Hde_start     = 144;
parameter Hde_end       = 784;

// 垂直扫描参数的设定 640*480 VGA
parameter FramePeriod   = 525;         //列周期数
parameter V_SyncPulse   = 2;           //列同步脉冲 (Sync o)
parameter V_BackPorch   = 33;          //显示后沿 (Back porch p)
parameter V_ActivePix    = 480;        //显示时序段 (Display interval q)
parameter V_FrontPorch  = 10;          //显示前沿 (Front porch r)
parameter Vde_start     = 35;
parameter Vde_end       = 515;
```

```
reg [10 : 0] h_cnt;
reg [9 : 0] v_cnt;
reg [4 : 0] vga_r_reg;
reg [5 : 0] vga_g_reg;
reg [4 : 0] vga_b_reg;

reg hsync_r;
reg vsync_r;
reg vsync_de;
reg hsync_de;

//水平扫描计数
always @(posedge vga_clk)
    if(~rst_n)
        h_cnt <= 1;
    else if(h_cnt==LinePeriod)
        h_cnt <= 1;
    else
        h_cnt <= h_cnt+ 1'b1;

//水平扫描信号 hsync,hsync_de 产生
always @(posedge vga_clk)
begin
    if(~rst_n)
        hsync_r <= 1'b1;
    else if(h_cnt==1)
        hsync_r <= 1'b0;    //产生 hsync 信号
    else if(h_cnt==H_SyncPulse)
        hsync_r <= 1'b1;

    if(~rst_n)
        hsync_de <= 1'b0;
    else if(h_cnt==Hde_start)
        hsync_de <= 1'b1;    //产生 hsync_de 信号
    else if(h_cnt==Hde_end)
        hsync_de <= 1'b0;

end

//垂直扫描计数
always @(posedge vga_clk)
    if(~rst_n)
```

```
v_cnt <= 1;
else if(v_cnt==FramePeriod)
    v_cnt <= 1;
else if(h_cnt==LinePeriod)
    v_cnt <= v_cnt + 1'b1;

// 垂直扫描信号 vsync, vsync_de 产生
always @(posedge vga_clk)
begin
    if(~rst_n)
        vsync_r <= 1'b1;
    else if(v_cnt==1)
        vsync_r <= 1'b0;    //产生 vsync 信号
    else if(v_cnt==V_SyncPulse) vsync_r <= 1'b1;

    if(~rst_n)
        vsync_de <= 1'b0;
    else if(v_cnt==Vde_start)
        vsync_de <= 1'b1;    //产生 vsync_de 信号
    else if(v_cnt==Vde_end)
        vsync_de <= 1'b0;
end

//生成彩条测试数据
always @(posedge vga_clk)
begin
    if(~rst_n)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= 0;
    else if(h_cnt>=Hde_start && h_cnt<(Hde_start+160) && vsync_de)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= RED;
    else if(h_cnt>=(Hde_start+160) && h_cnt<(Hde_start+320) && vsync_de)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= GREEN;
    else if(h_cnt>=(Hde_start+320) && h_cnt<(Hde_start+480) && vsync_de)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= BLUE;
    else if(h_cnt>=(Hde_start+480) && h_cnt<(Hde_start+640) && vsync_de)
        {vga_r_reg,vga_g_reg,vga_b_reg} <= YELLOW;
    else
        {vga_r_reg,vga_g_reg,vga_b_reg} <= 0;
end

assign vga_hsync = hsync_r;
assign vga_vsync = vsync_r;
```



```

assign vga_r = vga_r_reg;
assign vga_g = vga_g_reg;
assign vga_b = vga_b_reg;

endmodule

```

17.4. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

	Node Name	Direction	Location	I/O Bank
out	sm_sel[0]	Output	PIN_L3	2
out	vga_blue[4]	Output	PIN_J1	2
out	vga_blue[3]	Output	PIN_J2	2
out	vga_blue[2]	Output	PIN_K1	2
out	vga_blue[1]	Output	PIN_K2	2
out	vga_blue[0]	Output	PIN_L1	2
out	vga_green[5]	Output	PIN_C2	1
out	vga_green[4]	Output	PIN_D1	1
out	vga_green[3]	Output	PIN_F2	1
out	vga_green[2]	Output	PIN_F1	1
out	vga_green[1]	Output	PIN_G2	1
out	vga_green[0]	Output	PIN_G1	1
out	vga_hsync	Output	PIN_A4	8
out	vga_red[4]	Output	PIN_B4	8
out	vga_red[3]	Output	PIN_A3	8
out	vga_red[2]	Output	PIN_B3	8
out	vga_red[1]	Output	PIN_A2	8
out	vga_red[0]	Output	PIN_B1	1
out	vga_vsync	Output	PIN_B5	8

17.5. 实验操作

插上 VGA 接口，连接 FPGA 板卡和 VGA 显示器。打开电源后，下载 FPGA 配置文件，可以看到显示器上实时显示彩条 R/G/B/Y，如下图所示。



第十八章 MT9V011 摄像头 VGA 显示

本章基于 MT9V011 摄像头和 VGA 接口实现摄像头数据的 VGA 实时显示功能。

18.1. 硬件介绍

由于摄像头的视频流格式和 VGA 显示器的视频流格式不匹配，所以需要大容量存储空间芯片 SDRAM 作为帧缓存来实现显示转换，SDRAM 采用华邦的 W9825G6KH-6 芯片，具有如下特点：

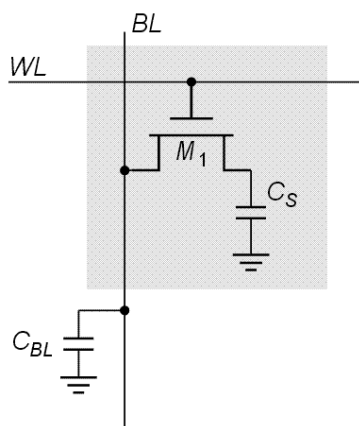
- ◆ 工作电压：VDD、VDDQ 为 3.3V。
- ◆ 所有的 PIN 和 LVTTTL 电平兼容。
- ◆ 54PIN TSOPII 封装。
- ◆ 所有的输入输出同步于时钟的上升沿。
- ◆ 数据掩码：UDQM,LDQM。
- ◆ 内部四 BANK 操作。
- ◆ 自动刷新功能。
- ◆ 8192 个刷新周期(64ms)。
- ◆ 可编程的突发长度和突发类型
 - 1, 2, 4, 8 或全页突发模式
 - 1, 2, 4 或 8 交错突发模式
- ◆ 可编程的 CAS#延时(2,3 个时钟)。

下表为 SDRAM 的引脚说明。

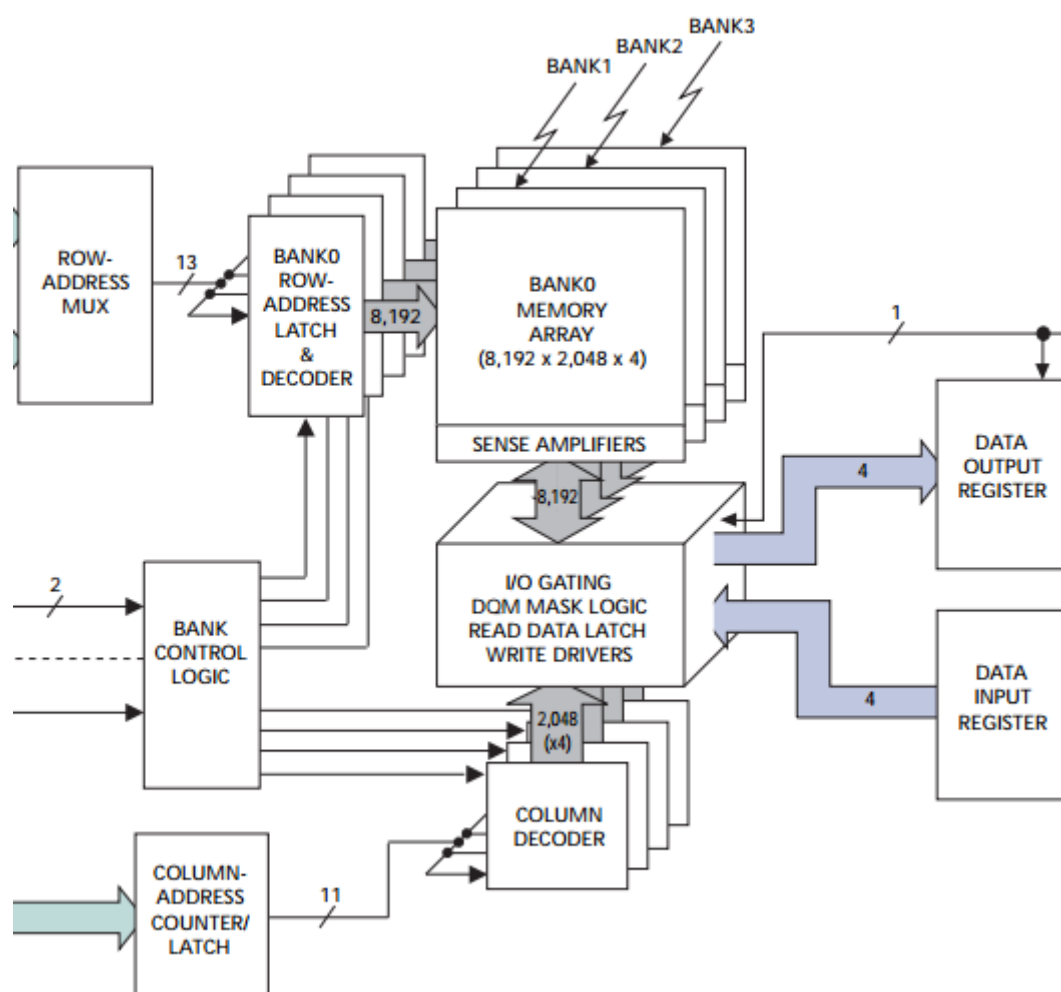
SYMBOL	TYPE	DESCRIPTION
CLK	INPUT	Clock : The system clock input. All other inputs are registered to the SDRAM on the rising edge of CLK
CKE	INPUT	Clock Enable: Controls internal clock signal and when deactivated, the SDRAM will be one of the states among power down, suspend or self refresh
$\overline{CS}$	INPUT	Chip Select: Enables or disables all inputs except CLK, CKE and DQM
BA0, BA1	INPUT	Bank Address: Selects bank to be activated during $\overline{RAS}$ activity Selects bank to be read/written during CAS activity
A0 ~ A12	INPUT	Row Address: RA0 ~ RA12, Column Address: CA0 ~ CA8 Auto-precharge flag: A10
$\overline{RAS}$, $\overline{CAS}$, $\overline{WE}$	INPUT	Command Inputs: RAS, CAS and WE define the operation Refer function truth table for details
LDQM, UDQM	I/O	Data Mask: Controls output buffers in read mode and masks input data in write mode
DQ0 ~ DQ15	I/O	Data Input / Output: Multiplexed data input / output pin
VDD / VSS	SUPPLY	Power supply for internal circuits and input buffers
VDDQ / VSSQ	SUPPLY	Power supply for output buffers
NC	-	No connection : These pads should be left unconnected

18.2. SDRAM 结构

SDRAM 芯片由若干个 cell 组成, 每个 cell 保存一个 bit 的信息。电容 C_s 的电荷状态可以表示 0 或 1 的信息。电路通过 WL(Word Line)和 BL(Bit Line)对 M_1 进行控制实现数据的读写。



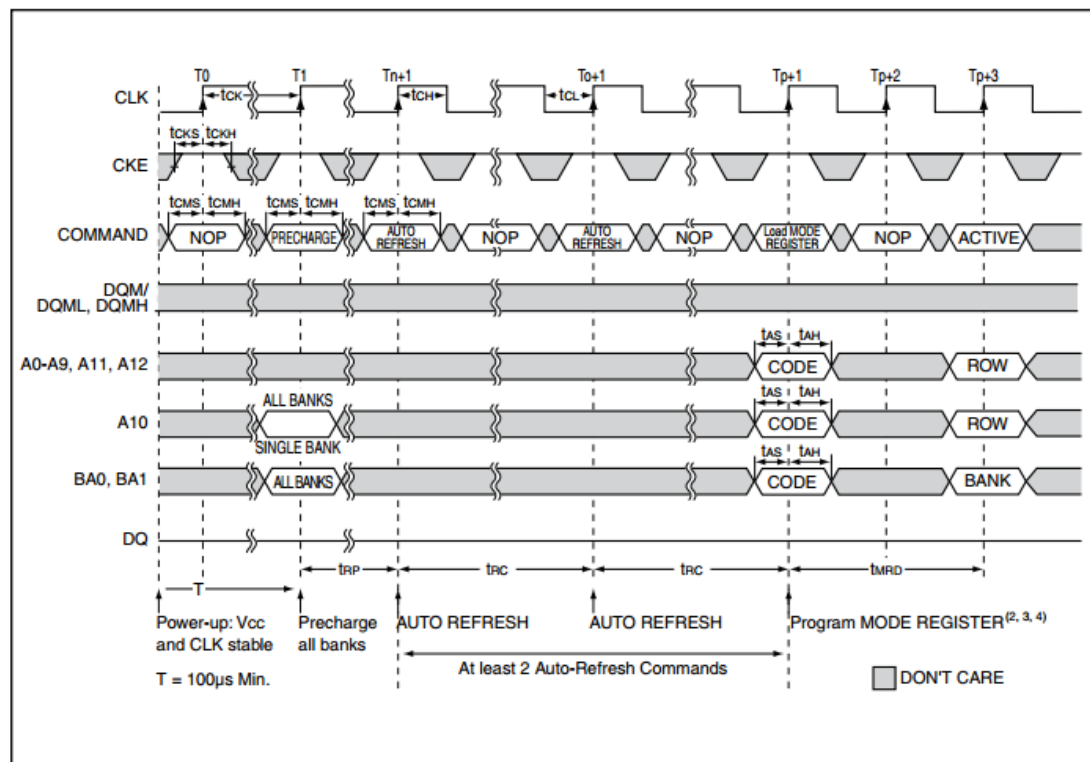
SDRAM 的管脚分为: 控制信号、地址信号和数据信号。通常一个 SDRAM 中包含多个 BANK, 每个 BANK 的存储单元是按行和列寻址的。



18.3. SDRAM 操作

➤ SDRAM 初始化

INITIALIZE AND LOAD MODE REGISTER⁽¹⁾



Notes:

1. If CS is High at clock High time, all commands applied are NOP.
2. The Mode register may be loaded prior to the Auto-Refresh cycles if desired.
3. JEDEC and PC100 specify three clocks.
4. Outputs are guaranteed High-Z after the command is issued.

从上图可知初始化的流程：100us~200us 上电延时→给所有 BANK 预充电(Precharge)→自动刷新(至少 2 次 Auto Refresh 命令)→编程模式寄存器(Program MODE register)。

这些过程中还需要间隔的发送 NOP 指令，在实际编程中要注意。模式寄存器详细定义如下图所示：

Mode Register

BA1	BA0	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
0	0	0	0	OP Code	0	0	CAS Latency			BT	Burst Length		

OP Code

A9	Write Mode
0	Burst Read and Burst Write
1	Burst Read and Single Write

Burst Type

A3	Burst Type
0	Sequential
1	Interleave

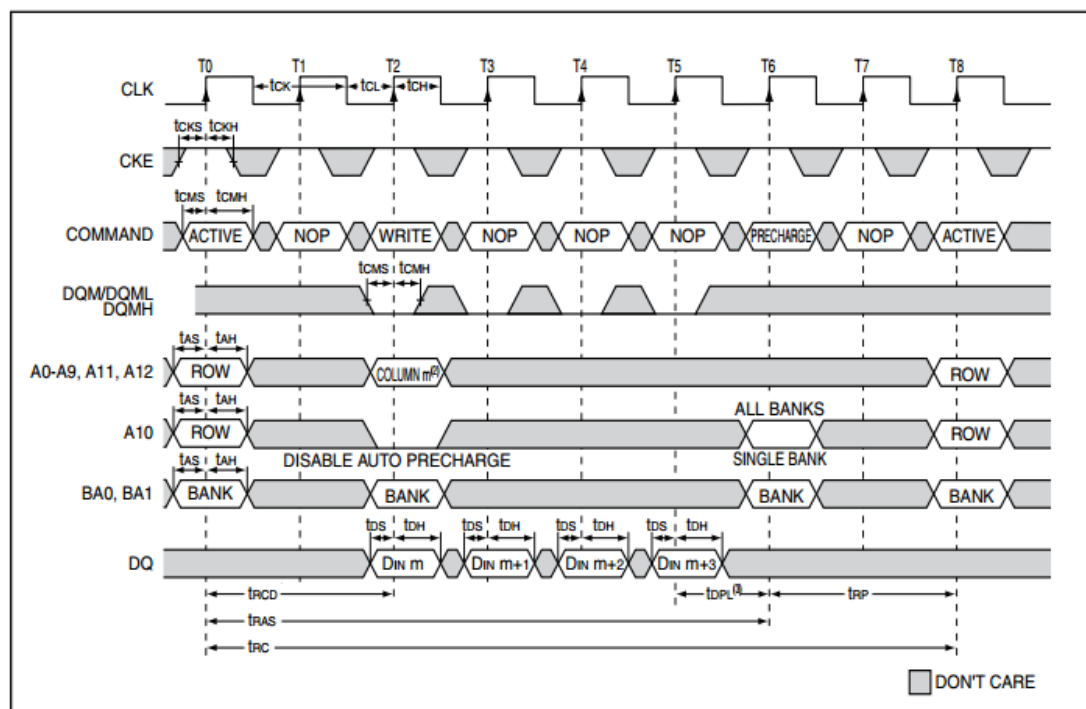
CAS Latency

A6	A5	A4	CAS Latency
0	0	0	Reserved
0	0	1	Reserved
0	1	0	2
0	1	1	3
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Reserved
1	1	1	Reserved

Burst Length

A2	A1	A0	Burst Length	
			A3 = 0	A3 = 1
0	0	0	1	1
0	0	1	2	2
0	1	0	4	4
0	1	1	8	8
1	0	0	Reserved	Reserved
1	0	1	Reserved	Reserved
1	1	0	Reserved	Reserved
1	1	1	Full Page	Reserved

➤ SDRAM 写操作

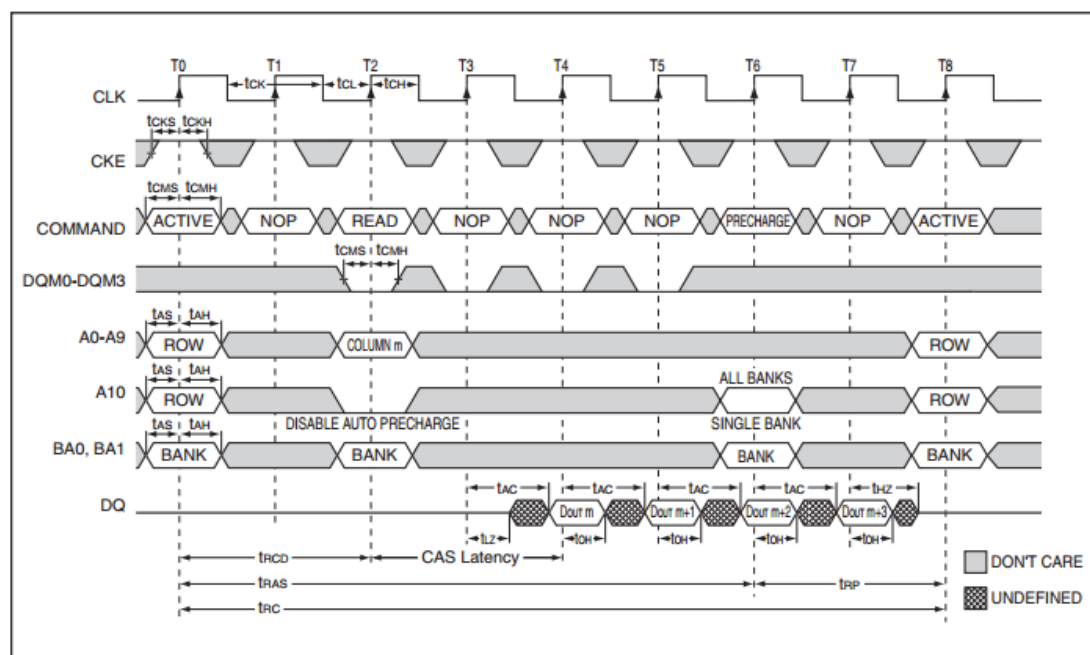
WRITE - WITHOUT AUTO PRECHARGE**Notes:**

- 1) Burst Length = 4
- 2) x16: A9, A11, and A12 = "Don't Care"
x8: A11 and A12 = "Don't Care"
- 3) tRAS must not be violated.

从上图可简化写操作流程: ROW ACTIVE 命令(需指定 ROW 地址和 BANK 地址)→tRCD 延时→WRITE 指令(需指定 BANK, 拉低 A10)。

➤ SDRAM 读操作

READ WITHOUT AUTO PRECHARGE

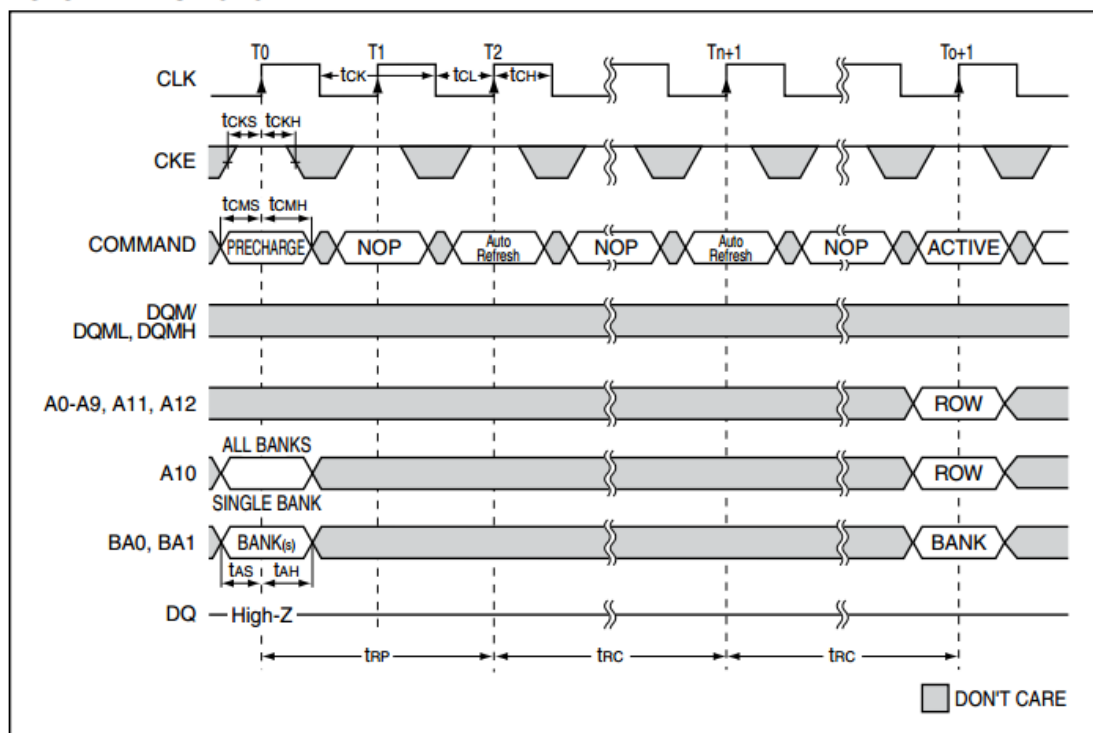


CAS latency = 2, Burst Length = 4

从上图可简化读操作流程：ROW ACTIVE 命令(需指定 ROW 地址和 BANK 地址) → tRCD 延时 → READ 指令(需指定 BANK，拉低 A10) → CAS Latency 延时 >> 读数据。

► SDRAM 自刷新操作

AUTO-REFRESH CYCLE



Notes:
1. CAS latency = 2, 3

从上图可简化自刷新操作流程：给所有 BANK 预充电(Precharge) → tRP 延时 → Auto-Refresh 命令。

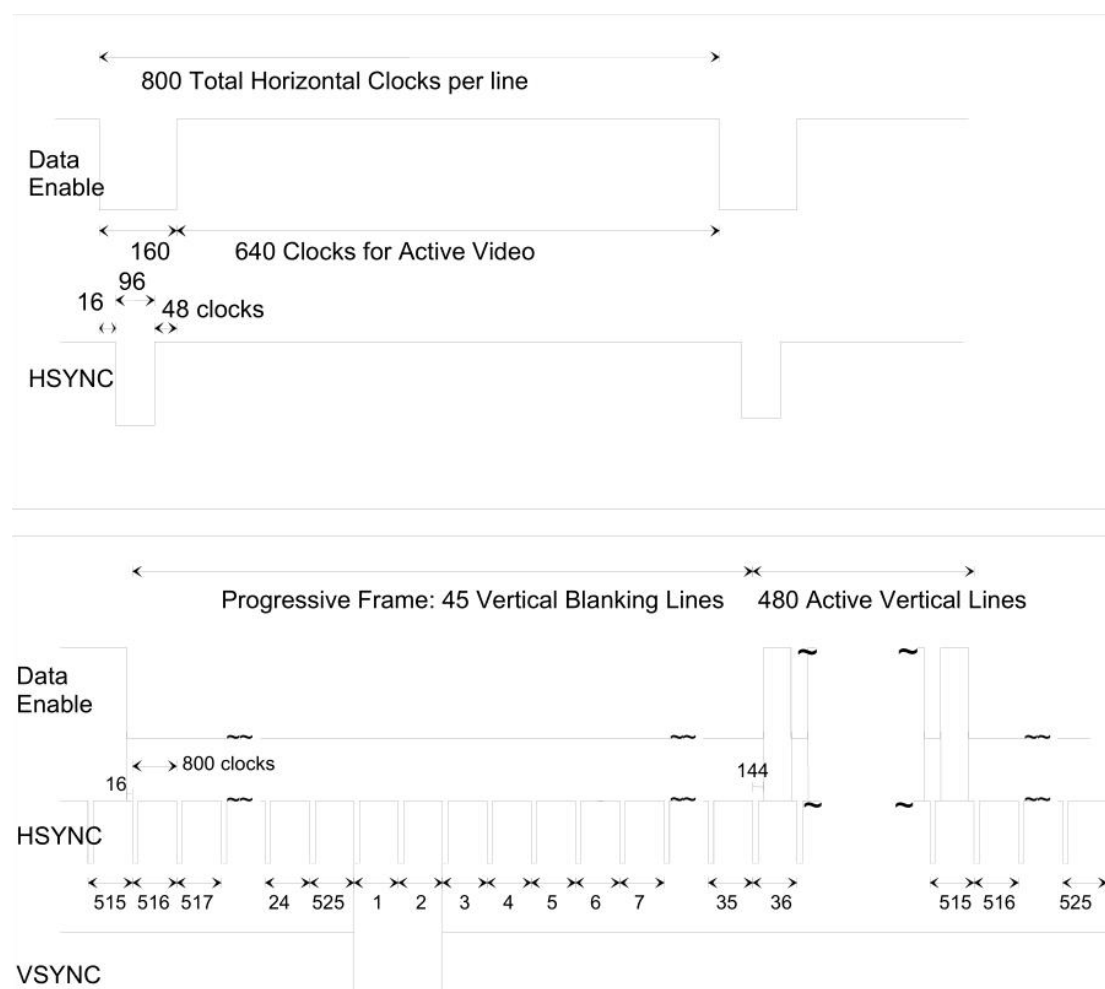
SDRAM 内部电容保存数据的最长时间是 64ms，一个 BANK 有 8192 行，则每行大约 7.8us(64ms/8192)。这里设置刷新周期为 7.8us。

具体操作指令可以参考管方提供的 datasheet。

18.4. VGA 时序

VGA(Video Graphics Array)是 IBM 在 1987 年推出的一种视频传输，具有分辨率高、显示速率快、颜色丰富等优点，在彩色显示器领域得到了广泛的应用。不支持热插拔，不支持音频传输。

本设计使用的是 640×480@60Hz 的 VGA 时序接口，计算像素时钟 $pclk = 800 \times 525 \times 60 = 25200000$ ，注意这里的 HSYNC(行同步信号)和 VSYNC(场同步信号)都是低电平有效，且 HSYNC 在数据行无效区域也要提供。

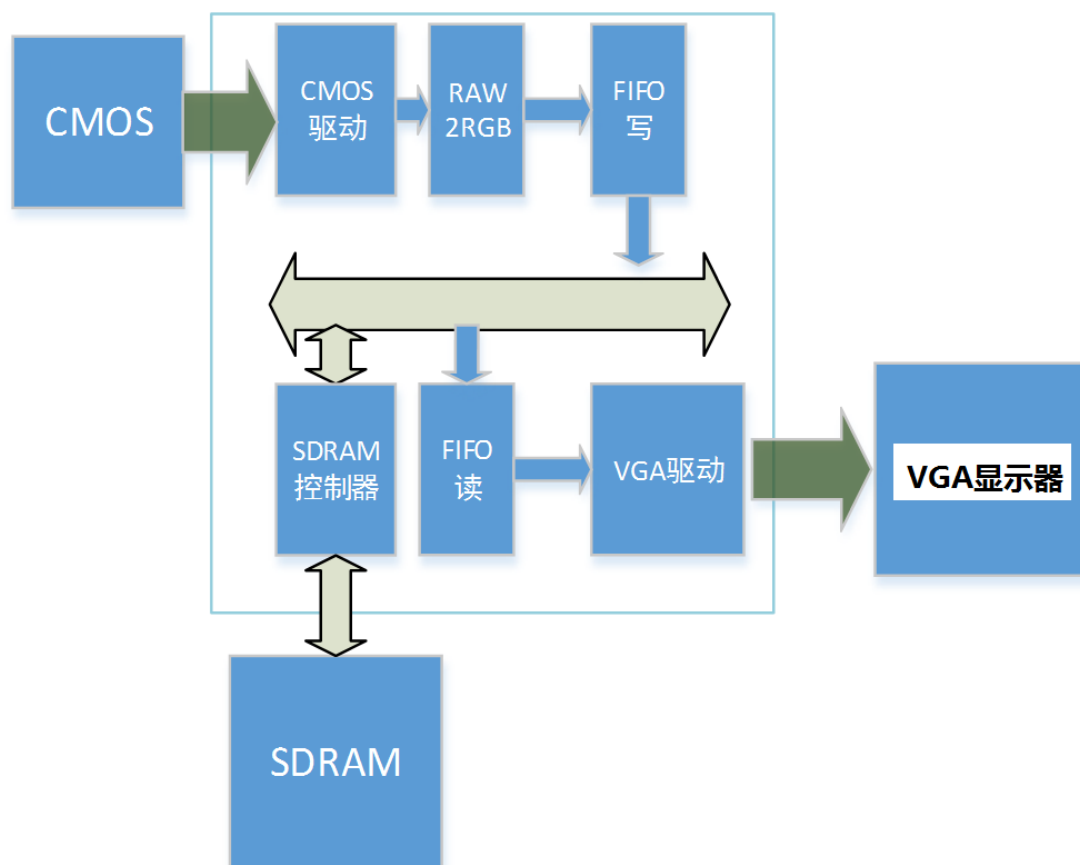


18.5. HDL 设计

➤ 系统框架

◆ CMOS 驱动：负责 CMOS 图像采集与裁剪；

- ◆ RAW2RGB: 负责将原始的 RAW 数据转换成 RG565 数据（具体原理可参考 Verilog 图像处理章节）;
- ◆ FIFO 写负责将 RAW2RGB 转换的 RGB 数据发送给 SDRAM 控制器;
- ◆ FIFO 读负责将 SDRAM 中的视频帧数据读出给 VGA 驱动模块;
- ◆ VGA 模块负责将视频信号发送给 VGA 显示器显示。



➤ VGA 驱动

```

//file name:   vga_show.v
//author:      ETree
//date:        2021.10.2
//function:    VGA 显示器驱动
//log:

module vga_show(
    input    vga_clk,
    input    rst_n,           //VGA 复位，低电平有效

    output    vga_hsync,
    output    vga_vsync,
    output [4:0] vga_r,
    output [5:0] vga_g,
    output [4:0] vga_b,

```

```

    input [15:0] sdr_data,           //SDR 中的图像数据
    output reg   sdr_rden
);

// 水平扫描参数的设定 640*480 VGA 60FPS_25MHz
parameter LinePeriod   = 800;      //行周期数
parameter H_SyncPulse  = 96;      //行同步脉冲 (Sync a)
parameter H_BackPorch  = 48 ;     //显示后沿 (Back porch b)
parameter H_ActivePix   = 640;    //显示时序段 (Display interval c)
parameter H_FrontPorch = 16;      //显示前沿 (Front porch d)
parameter Hde_start    = 144;
parameter Hde_end      = 784;

// 垂直扫描参数的设定 640*480 VGA
parameter FramePeriod  = 525;     //列周期数
parameter V_SyncPulse  = 2;       //列同步脉冲 (Sync o)
parameter V_BackPorch  = 33 ;     //显示后沿 (Back porch p)
parameter V_ActivePix   = 480;    //显示时序段 (Display interval q)
parameter V_FrontPorch = 10;      //显示前沿 (Front porch r)
parameter Vde_start    = 35;
parameter Vde_end      = 515;

reg [10 : 0] h_cnt;
reg [9 : 0] v_cnt;
reg [4 : 0] vga_r_reg;
reg [5 : 0] vga_g_reg;
reg [4 : 0] vga_b_reg;

reg hsync_r;
reg vsync_r;
reg vsync_de;
reg hsync_de;

//水平扫描计数
always @(posedge vga_clk)
    if(~rst_n)
        h_cnt <= 1;
    else if(h_cnt == LinePeriod)
        h_cnt <= 1;
    else
        h_cnt <= h_cnt+ 1'b1;

```

```
//水平扫描信号 hsync, hsync_de 产生
always @(posedge vga_clk)
begin
    if(~rst_n)
        hsync_r <= 1'b1;
    else if(h_cnt == 1)
        hsync_r <= 1'b0;    //产生 hsync 信号
    else if(h_cnt == H_SyncPulse)
        hsync_r <= 1'b1;

    if(~rst_n)
        hsync_de <= 1'b0;
    else if(h_cnt == Hde_start)
        hsync_de <= 1'b1;    //产生 hsync_de 信号
    else if(h_cnt == Hde_end)
        hsync_de <= 1'b0;

end

//垂直扫描计数
always @(posedge vga_clk)
begin
    if(~rst_n)
        v_cnt <= 1;
    else if(v_cnt == FramePeriod)
        v_cnt <= 1;
    else if(h_cnt == LinePeriod)
        v_cnt <= v_cnt + 1'b1;

// 垂直扫描信号 vsync, vsync_de 产生
always @(posedge vga_clk)
begin
    if(~rst_n)
        vsync_r <= 1'b1;
    else if(v_cnt == 1)
        vsync_r <= 1'b0;    //产生 vsync 信号
    else if(v_cnt == V_SyncPulse) vsync_r <= 1'b1;

    if(~rst_n)
        vsync_de <= 1'b0;
    else if(v_cnt == Vde_start)
        vsync_de <= 1'b1;    //产生 vsync_de 信号
    else if(v_cnt == Vde_end)
```

```
        vsync_de <= 1'b0;
end

always @(*)
begin
    if(hsync_de && vsync_de )
        sdr_rden <= 1'b1;
    else
        sdr_rden <= 0;
    end
end

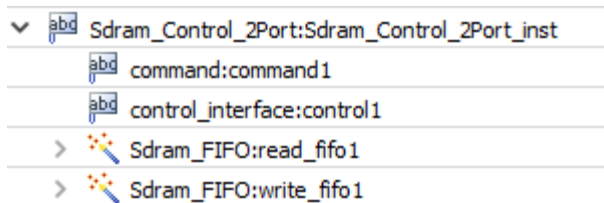
//sdr 读请求信号产生程序
always @(posedge vga_clk)
begin
    if (~rst_n)
    begin
        vga_r_reg <= 0;
        vga_g_reg <= 0;
        vga_b_reg <= 0;
    end
    else
    begin
        if(hsync_de && vsync_de )
        begin
            vga_r_reg <= sdr_data[15:11];
            vga_g_reg <= sdr_data[10:5];
            vga_b_reg <= sdr_data[4:0];
        end
        else
        begin
            vga_r_reg <= 0;
            vga_g_reg <= 0;
            vga_b_reg <= 0;
        end
    end
end

assign vga_hsync = hsync_r;
assign vga_vsync = vsync_r;
assign vga_r = (hsync_de & vsync_de)?vga_r_reg:5'b0_0000;
assign vga_g = (hsync_de & vsync_de)?vga_g_reg:6'b00_0000;
assign vga_b = (hsync_de & vsync_de)?vga_b_reg:5'b0_0000;
```

endmodule

➤ SDRAM 驱动

SDRAM 全称“Synchronous Dynamic Random Access Memory”，是动态随机存储器。本章的 SDRAM 驱动修改于“友晶”开源的 SDRAM 控制器，实现 SDRAM 读写的控制。有兴趣的读者可以研究一下，这里只做简单说明。



Command 和 Control_interface 负责 SDRAM 的命令和数据控制，实现初始化、写操作、读操作和自刷新操作；

Read_fifo 负责将 SDRAM 命令数据控制模块读出的数据给 HDMI 显示模块；

Write_fifo 负责将摄像头的视频数据写如 SDRAM 的命令控制模块。

18.6. 管脚分配

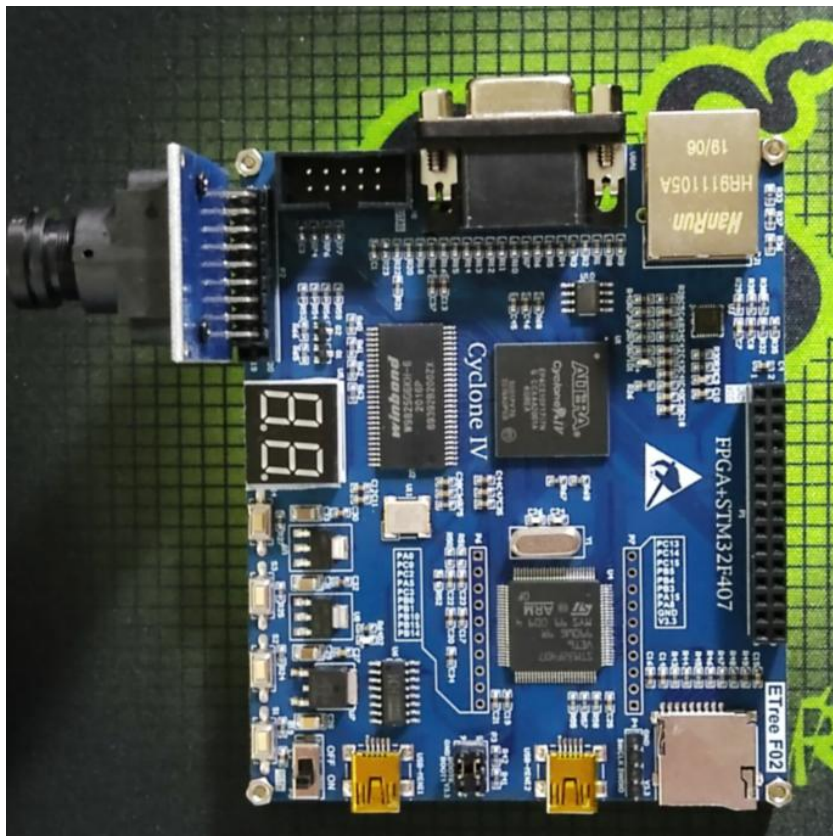
选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
out sm_sel[0]	Output	PIN_L3	2
out vga_blue[4]	Output	PIN_J1	2
out vga_blue[3]	Output	PIN_J2	2
out vga_blue[2]	Output	PIN_K1	2
out vga_blue[1]	Output	PIN_K2	2
out vga_blue[0]	Output	PIN_L1	2
out vga_green[5]	Output	PIN_C2	1
out vga_green[4]	Output	PIN_D1	1
out vga_green[3]	Output	PIN_F2	1
out vga_green[2]	Output	PIN_F1	1
out vga_green[1]	Output	PIN_G2	1
out vga_green[0]	Output	PIN_G1	1
out vga_hsync	Output	PIN_A4	8
out vga_red[4]	Output	PIN_B4	8
out vga_red[3]	Output	PIN_A3	8
out vga_red[2]	Output	PIN_B3	8
out vga_red[1]	Output	PIN_A2	8
out vga_red[0]	Output	PIN_B1	1
out vga_vsync	Output	PIN_B5	8

Node Name	Direction	Location	I/O Bank
out cam_xclk	Output	PIN_N2	2
in clk	Input	PIN_M16	5
out led	Output	PIN_F6	8
in rst_n	Input	PIN_L10	4
out sdram_addr[12]	Output	PIN_M7	3
out sdram_addr[11]	Output	PIN_P6	3
out sdram_addr[10]	Output	PIN_T4	3
out sdram_addr[9]	Output	PIN_N6	3
out sdram_addr[8]	Output	PIN_N5	3
out sdram_addr[7]	Output	PIN_M6	3
out sdram_addr[6]	Output	PIN_P3	3
out sdram_addr[5]	Output	PIN_N3	3
out sdram_addr[4]	Output	PIN_L7	3
out sdram_addr[3]	Output	PIN_T2	3
out sdram_addr[2]	Output	PIN_R3	3
out sdram_addr[1]	Output	PIN_T3	3
out sdram_addr[0]	Output	PIN_R4	3
out sdram_ba[1]	Output	PIN_R5	3
out sdram_ba[0]	Output	PIN_T5	3
out sdram_cas_n	Output	PIN_R7	3
out sdram_cke	Output	PIN_N8	3
out sdram_clk	Output	PIN_P8	3
out sdram_cs_n	Output	PIN_R6	3
io sdram_dq[15]	Bidir	PIN_R12	4
io sdram_dq[14]	Bidir	PIN_P11	4
io sdram_dq[13]	Bidir	PIN_M10	4
io sdram_dq[12]	Bidir	PIN_N9	4
io sdram_dq[11]	Bidir	PIN_P9	4
io sdram_dq[10]	Bidir	PIN_M9	4
io sdram_dq[9]	Bidir	PIN_K8	3
io sdram_dq[8]	Bidir	PIN_L8	3
io sdram_dq[7]	Bidir	PIN_T8	3
io sdram_dq[6]	Bidir	PIN_T9	4
io sdram_dq[5]	Bidir	PIN_R9	4
io sdram_dq[4]	Bidir	PIN_T10	4
io sdram_dq[3]	Bidir	PIN_R10	4
io sdram_dq[2]	Bidir	PIN_T11	4
io sdram_dq[1]	Bidir	PIN_R11	4
io sdram_dq[0]	Bidir	PIN_T12	4
out sdram_dqm[1]	Output	PIN_M8	3
out sdram_dqm[0]	Output	PIN_R8	3
out sdram_ras_n	Output	PIN_T6	3
out sdram_we_n	Output	PIN_T7	3

18.7. 实验操作

将 CMOS 图像传感器模块插入 P2 接口(注意 1 脚对应关系)、VGA 线缆插入 VGA 母座、MINI USB 线缆插入 MINI USB 接口，如下图所示。



打开电源，下载 FPGA 配置文件，VGA 显示器实时显示 CMOS 图像传感器的视频数据，如下图所示。



第十九章 液晶屏彩条显示(2.8 寸)

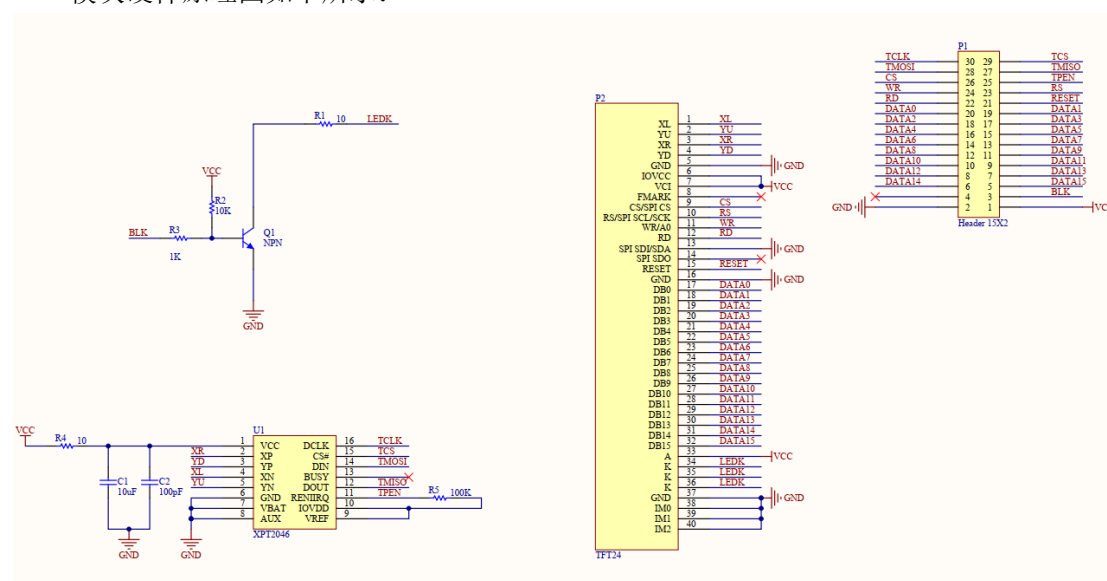
本章基于 2.8 寸液晶屏模块，实现 2.8 寸液晶屏彩条显示。

19.1. 硬件介绍

TFT 液晶屏模块分辨率为 320*240，驱动芯片为 ILI9341，可以显示 16 位色的真彩图，接口为 MCU 并口，该模块外观图如下所示：



模块硬件原理图如下所示：



ILI9341 液晶控制器自带显存，大小为 172800(240*320*18bit)。在 16 位模式下，ILI9341 采用 RGB565 格式存储像素数据，ILI9341 的 18 位数据线与 LCD GRAM 的对应关系如下图所示。

9341	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
总线																		
GRAM (16bit)	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC

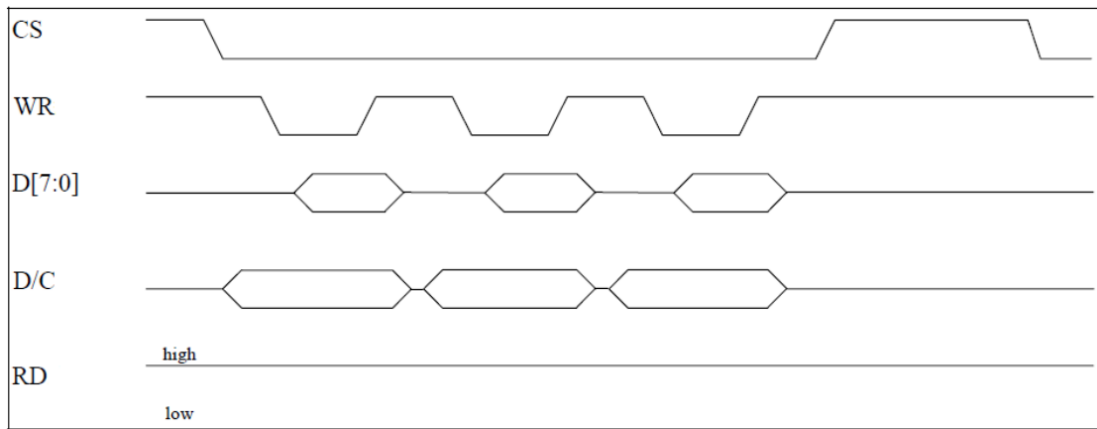
19.2. TFT-LCD 简介

TFT-LCD(Thin Film Transistor-Liquid Crystal Display)通过 MCU 并口(8080)来和其他主控通信。MCU 并口总线广泛的应用于各类液晶屏显示器上，主要包含一下信号线：

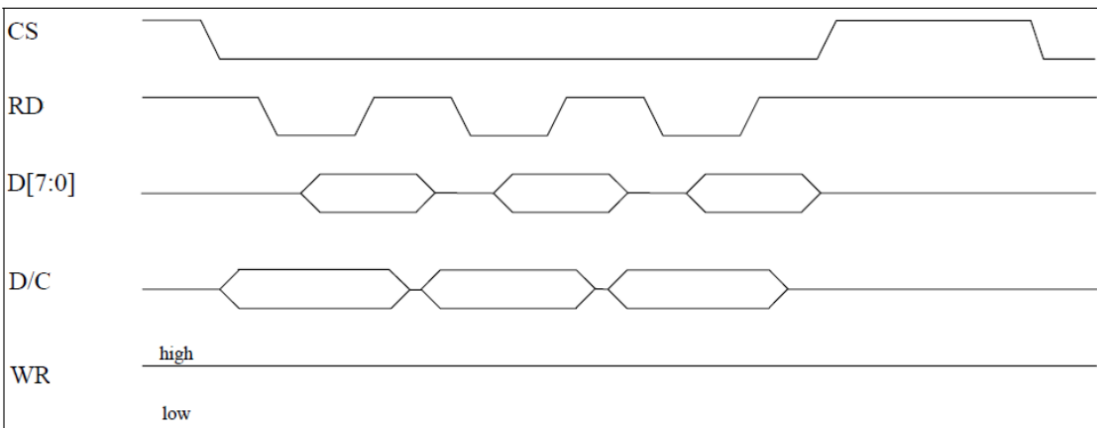
- CS: 片选信号；
- WR: 写入信号；
- RD: 读取信号；
- DATA[15:0]: 双向数据信号；
- RST: 复位信号；
- DC: 命令/数据标志(0 读，1 写)。

MCU 并口读写流程：先根据读写的数据类型，设置 DC，然后拉低 CS，接着设置 RD/WR 为低，在 RD 上升沿读取 DATA 数据，在 WR 上升沿写入 DATA 数据。

下面为写时序图：



下面为读时序图：



本章我们通过 STM32 的普通 IO 模拟 MCU 总线来驱动液晶屏显示, 其中 PB0~PB15 用于 16 位数据通道, 提高速率。

液晶屏的驱动芯片 ILI9341 自带显存, 大小为 172800(240*320*18bit)。在 16 位模式下, ILI9341 采用 RGB565 格式存储像素数据, ILI9341 的 18 位数据线与 LCD GRAM 的对应关系如下图所示。

9341 总线	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
GRAM (16bit)	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC

下面介绍 ILI9341 的几个重要命令, 关于完整的命令可以参考 ILI9341 的数据手册 datasheet。

0x36 指令: 存储访问控制指令, 用于控制读写的方向。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	1	0	1	1	0	36H
参数	1	1	↑	XX	MY	MX	MV	ML	BGR	MH	0	0	0

0x36 指令后面跟一个参数, 其中 MY/MX/MV 可以控制 ILI9341 的扫描方向。

控制位			效果
MY	MX	MV	
0	0	0	从左到右, 从上到下
1	0	0	从左到右, 从下到上
0	1	0	从右到左, 从上到下
1	1	0	从右到左, 从下到上
0	0	1	从上到下, 从左到右
0	1	1	从上到下, 从右到左
1	0	1	从下到上, 从左到右
1	1	1	从下到上, 从右到左

0x2A 列地址设置指令, 从上到下扫描(默认)下, 用于设置横坐标 x, 其中 SC 表示起始值, EC 表示结束值。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2AH
参数 1	1	1	↑	XX	SC15	SC14	SC13	SC12	SC11	SC10	SC9	SC8	SC
参数 2	1	1	↑	XX	SC7	SC6	SC5	SC4	SC3	SC2	SC1	SC0	
参数 3	1	1	↑	XX	EC15	EC14	EC13	EC12	EC11	EC10	EC9	EC8	EC
参数 4	1	1	↑	XX	EC7	EC6	EC5	EC4	EC3	EC2	EC1	EC0	

0x2B 页地址设置指令, 从左到右, 从上到下扫描(默认)下, 用于设置纵坐标 y, 其中 SP 表示开始值, EP 表示结束值。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2BH
参数 1	1	1	↑	XX	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SP
参数 2	1	1	↑	XX	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	
参数 3	1	1	↑	XX	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8	EP
参数 4	1	1	↑	XX	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0	

0x2C 写 GRAM 指令，在发完该指令后，便可以往 LCD 的 GRAM 中写入颜色数据，支持连续写。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	1	0	0	2CH
参数 1	1	1	↑	D1[15: 0]									XX
.....	1	1	↑	D2[15: 0]									XX
参数 n	1	1	↑	Dn[15: 0]									XX

0x2E 读 GRAM 指令，用于读取 ILI9341 的显存。

顺序	控制			各位描述												HEX	
	RS	RD	WR	D15~D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0		
指令	0	1	↑	XX				0	0	1	0	1	1	1	0	2EH	
参数 1	1	↑	1	XX												dummy	
参数 2	1	↑	1	R1[4:0]	XX			G1[5:0]					XX			R1G1	
参数 3	1	↑	1	B1[4:0]	XX			R2[4:0]					XX			B1R2	
参数 4	1	↑	1	G2[5:0]			XX		B2[4:0]					XX			G2B2
参数 5	1	↑	1	R3[4:0]	XX			G3[5:0]					XX			R3G3	
参数 N	1	↑	1	按以上规律输出													

19.3. HDL 设计

这里只需要实现 2.8 寸液晶屏的并口写，实现“红、绿、蓝、黄”彩条显示，由于液晶屏是 16 位 RGB565，颜色定义如下：

```
parameter RED      = 16'hf800;
parameter GREEN    = 16'h07e0;
parameter BLUE     = 16'h001f;
parameter YELLOW   = 16'hffe0;
```

关于液晶屏的具体寄存器配置这里不做介绍，可以参看芯片手册，配置代码如下：

```
always @(*)
begin
    case(lut_index)
        0 : lut_data    <= 8'hCF;    //
        1 : lut_data    <= 8'h00;
        2 : lut_data    <= 8'hD9;
        3 : lut_data    <= 8'h30;

        4 : lut_data    <= 8'hED;    //
```

```
5 : lut_data <= 8'h64;
6 : lut_data <= 8'h03;
7 : lut_data <= 8'h12;
8 : lut_data <= 8'h81;

9 : lut_data <= 8'hE8; //
10 : lut_data <= 8'h85;
11 : lut_data <= 8'h10;
12 : lut_data <= 8'h78;

13 : lut_data <= 8'hCB; //
14 : lut_data <= 8'h39;
15 : lut_data <= 8'h2C;
16 : lut_data <= 8'h00;
17 : lut_data <= 8'h34;
18 : lut_data <= 8'h02;

19 : lut_data <= 8'hF7; //
20 : lut_data <= 8'h20;

21 : lut_data <= 8'hEA; //
22 : lut_data <= 8'h00;
23 : lut_data <= 8'h00;

24 : lut_data <= 8'hC0; //Power control
25 : lut_data <= 8'h21;

26 : lut_data <= 8'hC1; //Power control
27 : lut_data <= 8'h12;

28 : lut_data <= 8'hC5; //VCM control
29 : lut_data <= 8'h32;
30 : lut_data <= 8'h3C;

31 : lut_data <= 8'hC7; //VCM control2
32 : lut_data <= 8'hC1;

33 : lut_data <= 8'h36; //Memory Access Control
34 : lut_data <= 8'h08;

35 : lut_data <= 8'h3A; //
36 : lut_data <= 8'h55;

37 : lut_data <= 8'hB1; //
```

```
38 : lut_data <= 8'h00;
39 : lut_data <= 8'h18;

40 : lut_data <= 8'hB6; //
41 : lut_data <= 8'h0A;
42 : lut_data <= 8'hA2;

43 : lut_data <= 8'hF2; //
44 : lut_data <= 8'h00;

45 : lut_data <= 8'h26; //
46 : lut_data <= 8'h01;

47 : lut_data <= 8'hE0; //SET Gamma
48 : lut_data <= 8'h0F;
49 : lut_data <= 8'h20;
50 : lut_data <= 8'h1E;
51 : lut_data <= 8'h09;
52 : lut_data <= 8'h12;
53 : lut_data <= 8'h0B;
54 : lut_data <= 8'h50;
55 : lut_data <= 8'hBA;
56 : lut_data <= 8'h44;
57 : lut_data <= 8'h09;
58 : lut_data <= 8'h14;
59 : lut_data <= 8'h05;
60 : lut_data <= 8'h23;
61 : lut_data <= 8'h21;
62 : lut_data <= 8'h00;

63 : lut_data <= 8'hE1; //Set Gamma
64 : lut_data <= 8'h00;
65 : lut_data <= 8'h19;
66 : lut_data <= 8'h19;
67 : lut_data <= 8'h00;
68 : lut_data <= 8'h12;
69 : lut_data <= 8'h07;
70 : lut_data <= 8'h2D;
71 : lut_data <= 8'h28;
72 : lut_data <= 8'h3F;
73 : lut_data <= 8'h02;
74 : lut_data <= 8'h0A;
75 : lut_data <= 8'h08;
76 : lut_data <= 8'h25;
```

```
77 : lut_data <= 8'h2D;
78 : lut_data <= 8'h0F;

79 : lut_data <= 8'h11; //EXIT SLEEP
// 80 : lut_data <= 8'h29; //Display on
// 81 : lut_data <= 8'h29; //Display on
82 : lut_data <= 8'h29; //Display on

83 : lut_data <= 8'h2A; //X region
84 : lut_data <= 8'h00;
85 : lut_data <= 8'h00;
86 : lut_data <= 8'h00; //239-->0x00EF
87 : lut_data <= 8'hEF;

88 : lut_data <= 8'h2B; //Y region
89 : lut_data <= 8'h00;
90 : lut_data <= 8'h00;
91 : lut_data <= 8'h01; //319-->0x013F
92 : lut_data <= 8'h3F;

93 : lut_data <= 8'h2C; //
default: lut_data <= 0;
endcase;
end
```

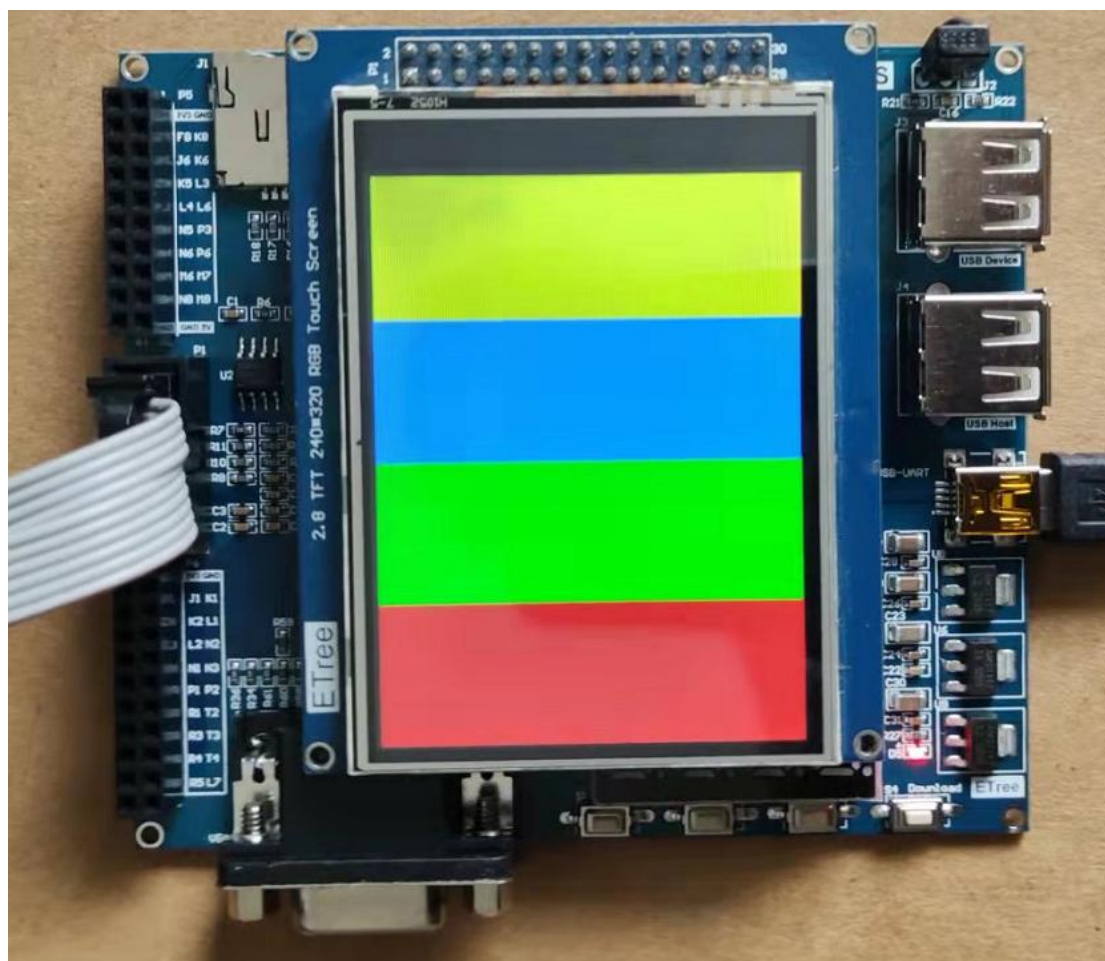
19.4. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

out	tft_blk	Output	PIN_D6	8
out	tft_cs	Output	PIN_D9	7
out	tft_data[15]	Output	PIN_F8	8
out	tft_data[14]	Output	PIN_F9	7
out	tft_data[13]	Output	PIN_F10	7
out	tft_data[12]	Output	PIN_C6	8
out	tft_data[11]	Output	PIN_F11	7
out	tft_data[10]	Output	PIN_E6	8
out	tft_data[9]	Output	PIN_B9	7
out	tft_data[8]	Output	PIN_D8	8
out	tft_data[7]	Output	PIN_A10	7
out	tft_data[6]	Output	PIN_C8	8
out	tft_data[5]	Output	PIN_B10	7
out	tft_data[4]	Output	PIN_E7	8
out	tft_data[3]	Output	PIN_A11	7
out	tft_data[2]	Output	PIN_E8	8
out	tft_data[1]	Output	PIN_B11	7
out	tft_data[0]	Output	PIN_E9	7
out	tft_rd	Output	PIN_E10	7
out	tft_reset	Output	PIN_A12	7
out	tft_rs	Output	PIN_B12	7
out	tft_tclk	Output	PIN_C11	7
out	tft_tcs	Output	PIN_B14	7
in	tft_tmis0	Input	PIN_A14	7
out	tft_tmosi	Output	PIN_E11	7
in	tft_tpen	Input	PIN_A13	7
out	tft_wr	Output	PIN_C9	7

19.5. 实验操作

插上 2.8 寸液晶屏模块(注意 1 脚对应关系), 插入 USB-Blaster 下载器, 再插入 USB 线缆给板卡供电后, 下载 FPGA 配置文件, 实际效果图如下所示:



第二十章 MT9V011 液晶屏显示[2.8 寸]

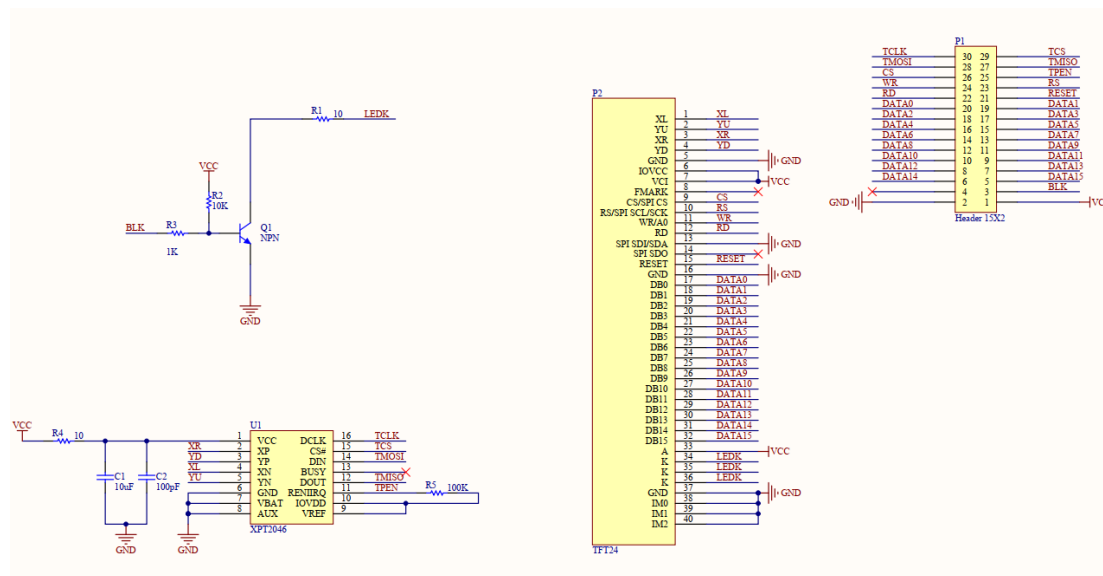
本章基于 MT9V011 摄像头和 TFT2.8 寸液晶屏，实现摄像头视频数据的实时显示。

20.1. 硬件介绍

TFT 液晶屏模块分辨率为 320*240，驱动芯片为 ILI9341，可以显示 16 位色的真彩图，接口为 MCU 并口，该模块外观图如下所示：



模块硬件原理图如下所示:



ILI9341 液晶控制器自带显存，大小为 172800(240*320*18bit)。在 16 位模式下，ILI9341 采用 RGB565 格式存储像素数据，ILI9341 的 18 位数据线与 LCD GRAM 的对应关系如下图所示。

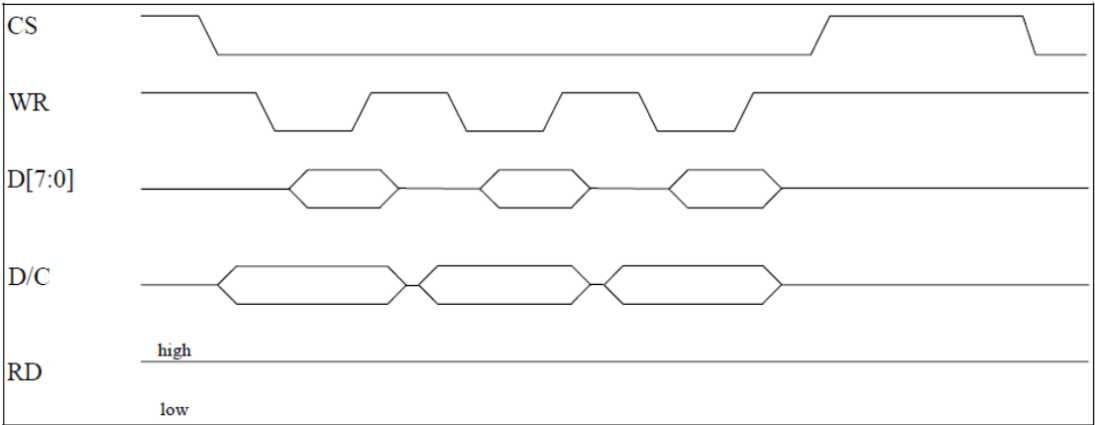
9341	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
总线																		
GRAM (16bit)	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC

20.2. MCU 并口

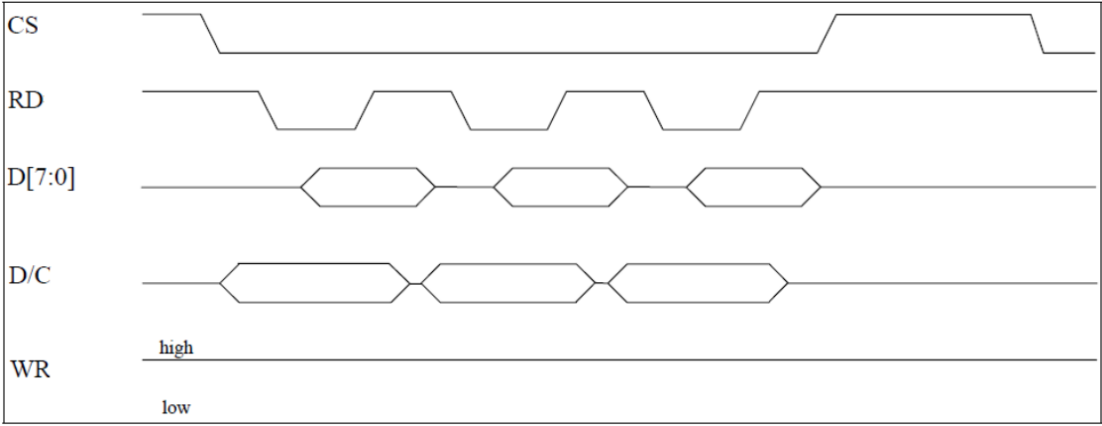
- MCU 并口总线广泛的应用于各类液晶屏显示器上，主要包含一下信号线：
- CS: 片选信号；
 - WR: 写入信号；
 - RD: 读取信号；
 - DATA[15:0]: 双向数据信号；
 - RST: 复位信号；
 - DC: 命令/数据标志(0 读，1 写)。

MCU 并口读写流程：先根据读写的数据类型，设置 DC，然后拉低 CS，接着设置 RD/WR 为低，在 RD 上升沿读取 DATA 数据，在 WR 上升沿写入 DATA 数据。

下面为写时序图：



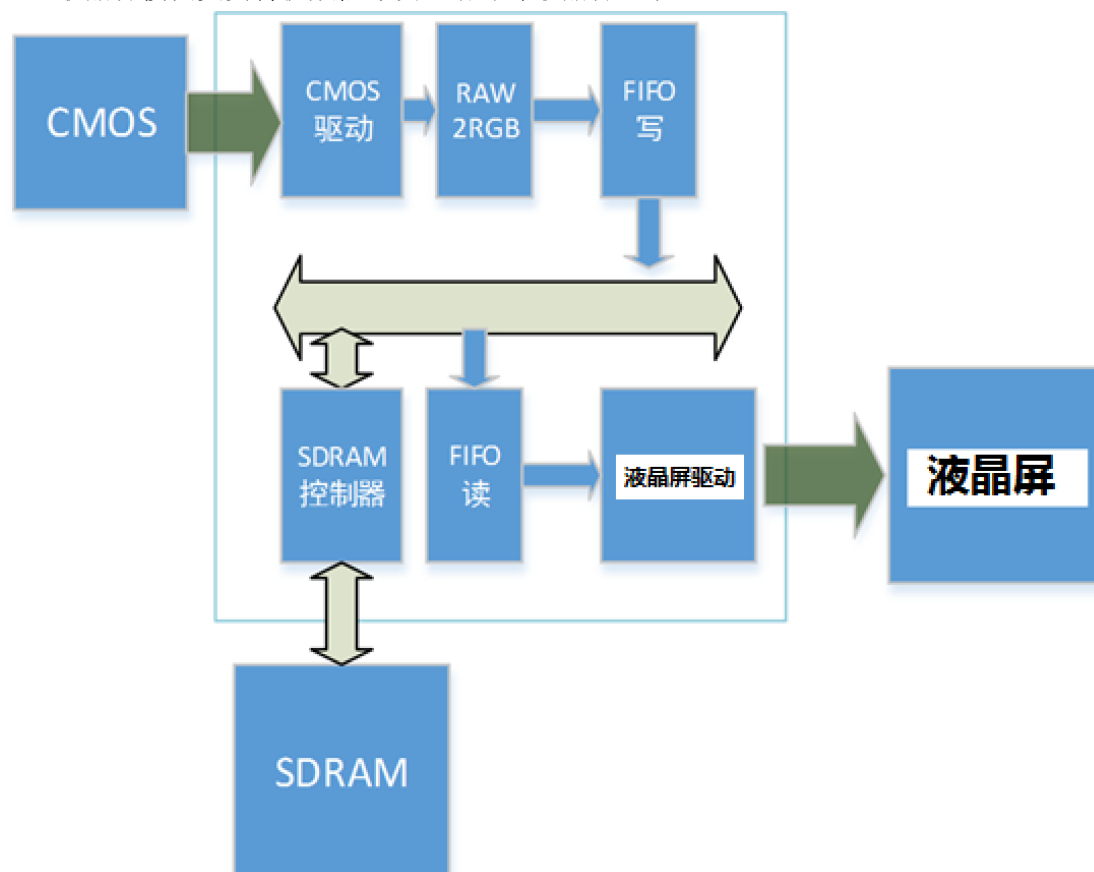
下面为读时序图：



20.3. HDL 设计

➤ 系统框架

- ◆ CMOS 驱动：负责 CMOS 图像采集与裁剪；
- ◆ RAW2RGB：负责将原始的 RAW 数据转换成 RGB656 数据；
- ◆ FIFO 写负责将 RAW2RGB 转换的的 RGB 数据发送给 SDRAM 控制器；
- ◆ FIFO 读负责将 SDRAM 中的视频帧数据读出给液晶屏驱动模块；
- ◆ 液晶屏模块负责将视频信号发送给外部液晶屏显示。



➤ 顶层模块

根据上一节的时序图，设计液晶屏接口驱动，具体液晶屏配置，这里不做介绍，可以参考 ILI9134 芯片手册。下面为顶层模块代码：

```
//file name:    top_fpga.v
//author:      ETree
//date:        2017.12.9
//function:     top file of project

module top_fpga(
    //global signal
    input  clk,
    input  rst_n,
```

```
//CMOS Port
output cam_scl,
inout cam_sda,
input [7:0] cam_data,

input cam_vsync,
input cam_hsync,
input cam_pclk,
output cam_xclk,

output cam_pdown,
output cam_reset,

//sdrn port
output [12:0] sdrn_addr,
output [1:0] sdrn_ba,
output sdrn_cas_n,
output sdrn_cke ,
output sdrn_cs_n ,
inout [15:0] sdrn_dq,
output [1:0] sdrn_dqm ,
output sdrn_ras_n,
output sdrn_we_n,
output sdrn_clk,

//TFT2.4
output tft_cs, //液晶屏片选, 低电平有效
output tft_wr,
output tft_rs,
output tft_rd,
output tft_reset, //液晶屏复位, 低电平有效
output [15:0] tft_data,
output tft_blk, //液晶屏背光, 高电平有效
//TFT Touch
output tft_tclk,
output tft_tcs,
output tft_tmosi,
input tft_tmiso,
input tft_tpen,

//VGA
output vga_hsync,
output vga_vsync,
output [4:0] vga_red,
```

```
output [5:0] vga_green,
output [4:0] vga_blue,

//ledsm
// output [7:0] sm_dat,
output [1:0] sm_sel,

//led
output led

);

//-----PLL-----
wire clk_25m;
wire clk_100m;
wire clk_100m_p;
wire locked;

my_pll my_pll_inst(
    .areset ( 0 ),
    .inclk0 ( clk ),
    .c0 ( clk_100m ),
    .c1 ( clk_100m_p ),
    .c2 ( clk_25m ),
    .locked ( locked )
);

assign sm_sel = 2'b11;

//-----Soft Reset-----
reg [15:0] rst_cnt;
reg rstn;

always @(posedge clk_25m)
    if(~rst_n)
        rst_cnt <= 0;
    else if(rst_cnt>='hfffa)
        rst_cnt <= rst_cnt;
    else
        rst_cnt <= rst_cnt + 1'b1;

always @(posedge clk_25m)
    if(rst_cnt>='hfff0)
        rstn <= 1'b1;
```

```

else
    rstn <= 0;

//-----CMOS-----
wire [7:0] cam_data_out;
wire cam_vsync_out;
wire cam_hsync_out;
wire [15:0] cam_data_test;
wire cam_vsync_test;
wire cam_hsync_test;

camera_if camera_if_inst
(
    .clk(clk_25m) , // input clk_sig
    .rst_n(rstn) , // input rst_n_sig
    .cam_scl(cam_scl) , // output cam_scl_sig
    .cam_sda(cam_sda) , // inout cam_sda_sig

    .cam_data(cam_data) , // input [7:0] cam_data_sig

    .cam_vsync(cam_vsync) , // input cam_vsync_sig
    .cam_hsync(cam_hsync) , // input cam_hsync_sig
    .cam_pclk(cam_pclk) , // input cam_pclk_sig
    .cam_xclk(cam_xclk) , // output cam_xclk_sig

    .cam_pdown(cam_pdown) , // output cam_pdown_sig
    .cam_reset(cam_reset) , // output cam_reset_sig

    .cam_data_out(cam_data_out),
    .cam_vsync_out(cam_vsync_out),
    .cam_hsync_out(cam_hsync_out),

    .cam_data_test(cam_data_test),
    .cam_vsync_test(cam_vsync_test),
    .cam_hsync_test(cam_hsync_test),

    .line_count(line_count),
    .cam_en(1'b1) // input cam_en_sig
);

defparam camera_if_inst.IMG_H=640;

//-----Video Image Process-----

```



```

wire [15:0] dvp_data_o;
wire dvp_href_o;
wire dvp_vsync_o;

vip_top vip_top_inst
(
    .cmos_pclk(cam_pclk) , // input  cmos_pclk_sig
    .cmos_href(cam_hsync_out) , // input  cmos_href_sig
    .cmos_vsync(cam_vsync_out) , // input  cmos_vsync_sig
    .cmos_data(cam_data_out) , // input [7:0] cmos_data_sig
    .cmos_href_o(dvp_href_o) , // output  cmos_href_o_sig
    .cmos_vsync_o(dvp_vsync_o) , // output  cmos_vsync_o_sig
    .cmos_data_o(dvp_data_o) // output [7:0] cmos_data_o_sig
);

//-----SDRAM-----

Sdram_Control_2Port Sdram_Control_2Port_inst
(
    //HOST
    .REF_CLK          (clk_100m) , //sdram module clock
    .OUT_CLK          (clk_100m_p) , //sdram clock input
    .RESET_N          (rstn) , //sdram module reset

    //SDRAM
    .SDR_CLK          (sdram_clk) , //sdram clock
    .CKE              (sdram_cke) , //sdram clock enable
    .CS_N             (sdram_cs_n) , //sdram chip select
    .WE_N             (sdram_we_n) , //sdram write enable
    .CAS_N            (sdram_cas_n) , //sdram column address strobe
    .RAS_N            (sdram_ras_n) , //sdram row address strobe
    .DQM              () , // (sdram_dqm) , //sdram data output
enable
    .BA              (sdram_ba) , //sdram bank address
    .SA              (sdram_addr) , //sdram address
    .DQ              (sdram_dq) , //sdram data

    //FIFO Write
    .WR_CLK          (cam_pclk) , //write fifo clock
    .WR_LOAD          (1'b0) , //write register load & fifo
clear
    .WR_DATA          (dvp_data_o) , //write data input
    .WR              (dvp_href_o) , //write data request

```

```

.WR_MIN_ADDR      (0),           //write start address
.WR_MAX_ADDR      (307200-1), //write max address
.WR_LENGTH        (9'd128),      //write burst length

//FIFO Read
.RD_CLK           (clk),          //read fifo clock
.RD_LOAD          (1'b0),         //read register load & fifo
clear
.RD_DATA          (fifo_q),       //read data output
.RD               (fifo_rd),      //read request
.RD_MIN_ADDR      (0),           //read start address
.RD_MAX_ADDR      (76800-1),     //read max address
.RD_LENGTH        (9'd128),      //read length
);

assign sdram_dqm = 2'b00;

//-----TFT-----

wire config_done;
wire [15:0] fifo_q;

tft24 tft24_inst
(
    .clk(clk) , // input clk_sig
    .rst_n(rstn) , // input rst_n_sig
    .tft_cs(tft_cs) , // output tft_cs_sig
    .tft_rs(tft_rs) , // output tft_rs_sig
    .tft_rd(tft_rd) , // output tft_rd_sig
    .tft_wr(tft_wr) , // output tft_wr_sig
    .tft_dat(tft_data) , // output [15:0] tft_dat_sig
    .tft_res(tft_reset) , // output tft_res_sig
    .data_in(fifo_q),
    .data_in_req(fifo_rd),
    .fifo_empty(0),
    .config_done(config_done) // output config_done_sig
);

assign tft_blk = 1'b1;

endmodule

```

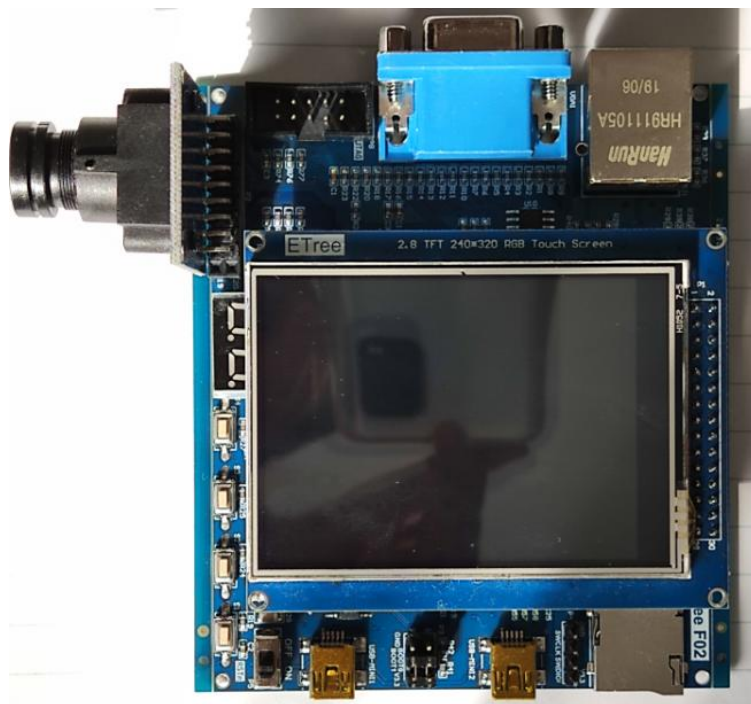
20.4. 管脚定义

选择菜单 Assignment -> Pin Planner，添加管脚约束。

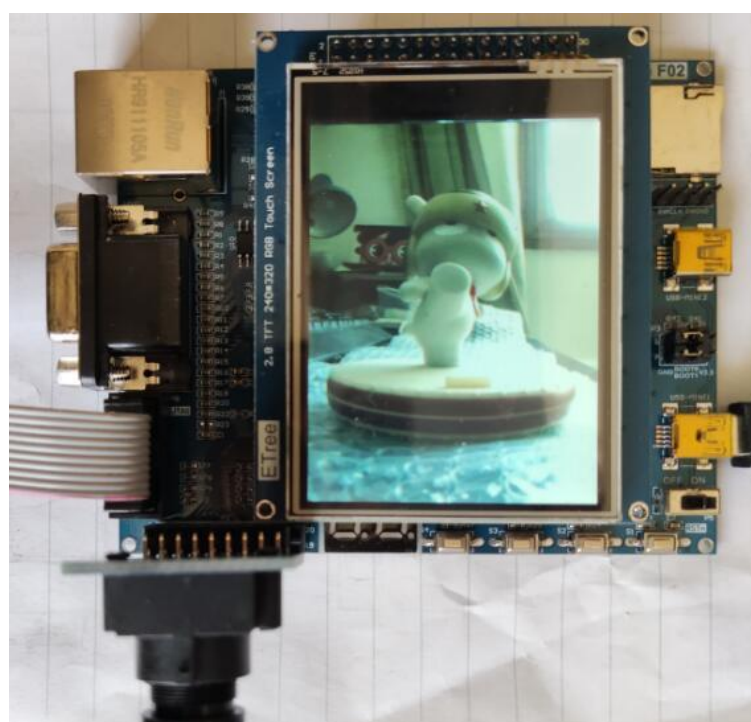
out	tft_blk	Output	PIN_D6	8
out	tft_cs	Output	PIN_D9	7
out	tft_data[15]	Output	PIN_F8	8
out	tft_data[14]	Output	PIN_F9	7
out	tft_data[13]	Output	PIN_F10	7
out	tft_data[12]	Output	PIN_C6	8
out	tft_data[11]	Output	PIN_F11	7
out	tft_data[10]	Output	PIN_E6	8
out	tft_data[9]	Output	PIN_B9	7
out	tft_data[8]	Output	PIN_D8	8
out	tft_data[7]	Output	PIN_A10	7
out	tft_data[6]	Output	PIN_C8	8
out	tft_data[5]	Output	PIN_B10	7
out	tft_data[4]	Output	PIN_E7	8
out	tft_data[3]	Output	PIN_A11	7
out	tft_data[2]	Output	PIN_E8	8
out	tft_data[1]	Output	PIN_B11	7
out	tft_data[0]	Output	PIN_E9	7
out	tft_rd	Output	PIN_E10	7
out	tft_reset	Output	PIN_A12	7
out	tft_rs	Output	PIN_B12	7
out	tft_tclk	Output	PIN_C11	7
out	tft_tcs	Output	PIN_B14	7
in	tft_tniso	Input	PIN_A14	7
out	tft_tmosi	Output	PIN_E11	7
in	tft_tpen	Input	PIN_A13	7
out	tft_wr	Output	PIN_C9	7

20.5. 实验操作

将 CMOS 图像传感器模块插入 P2 接口(注意 1 脚对应关系)，将 2.8 寸液晶屏插到 P1 接口(注意 1 脚对应关系)，将 MINI USB 线缆插入 MINI USB 接口，如下图所示：



打开电源，下载 FPGA 配置文件，可以看到液晶屏上实时显示摄像头视频数据，如下图所示：



第二十一章 HDMI 彩条显示

前面章节利用 VGA 接口实现了显示器彩条显示，本章基于 HDMI_TF 模块，实现 HDMI 显示器彩条显示。

21.1. 硬件设计

HDMI(High Definition Multimedia Interface)由日立、松下、飞利浦、Sony、东芝等电子公司共同组建的高清多媒体接口，符合高清时代标准的全新数字化视频/音频接口技术。

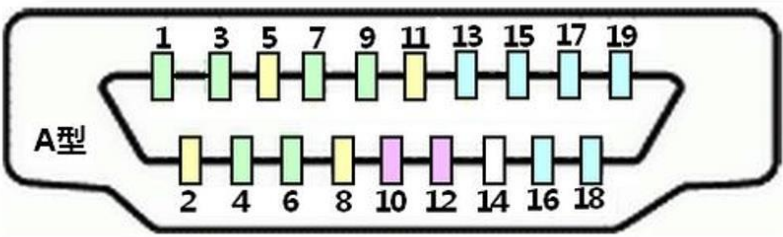
HDMI 标准的制定延续了 DVI 标准中相对成熟且易于实现的部分技术标准，整个传输原理还是基于 TMDS(Transition Minimized Differential Signaling)编码技术。针对 DVI，HDMI 做了大幅改进。

HDMI 接口体积小巧，可轻松安装于机顶盒、DVD 播放机、个人计算机、电视、游戏主机等设备。抗干扰能力更强，能实现最长 20 米的无增益传输。支持 24bit 色深处理(RGB、YCbCr4-4-4、YCbCr4-2-2)。

下图为常用的 3 种 HDMI 接口。



下图为 HDMI-A 接口的引脚定义



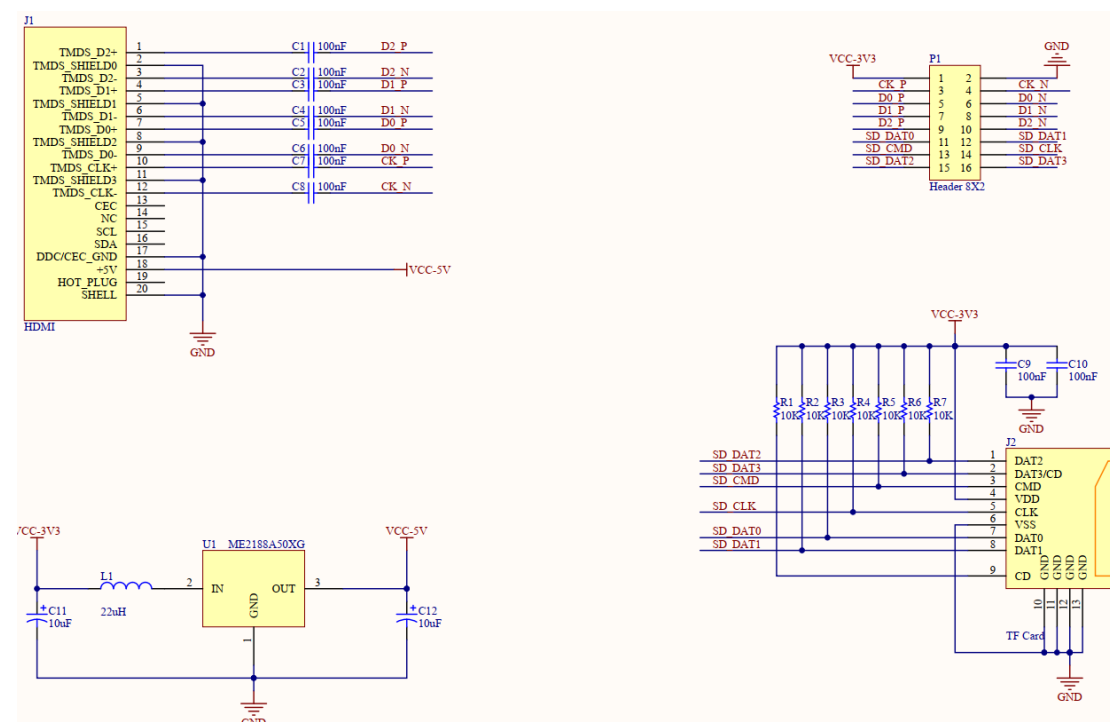
引脚	定义	引脚	定义
1	数据 2+	10	时钟+
2	数据 2 屏蔽	11	时钟屏蔽
3	数据 2-	12	时钟-
4	数据 1+	13	CEC
5	数据 1 屏蔽	14	保留
6	数据 1-	15	DDC 时钟线
7	数据 0+	16	DDC 数据线
8	数据 0 屏蔽	17	DDC/CEC 地
9	数据 0-	18	5V 电源
		19	热插拔检测

这里我们只用到了其中的 TMDs 时钟和 TMDs 数据信号。

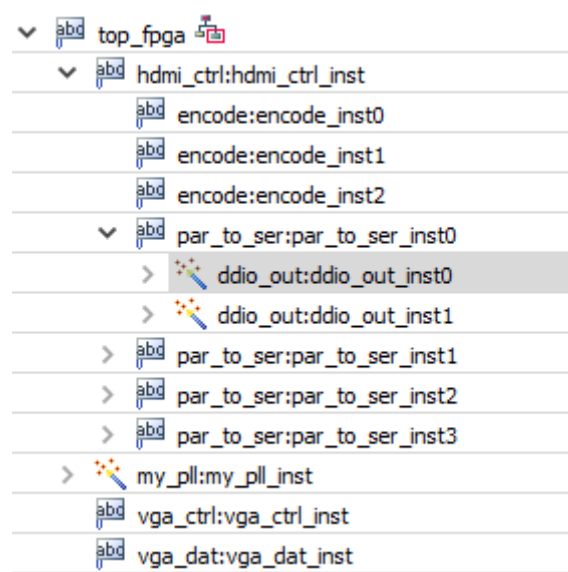
TMDs 通道包括 3 个 RGB 数据传输通道和 1 个时钟传输通道。每个通道都通过编码算法，将 8 为视频数据转成 10bit 数据。8 位数据经过编码和直流平衡得到 10 位数据，虽然增加了冗余位，但增加了在更长的同轴电缆中传输的可靠性。前 8 位数据由 8 位原始数据经过逻辑运算得到，第 9 位指示运算的方式，第 10 位用来对应直流平衡。直流平衡就是指在编码过程中保证信道中的直流偏移位零，即信道中的 0 与 1 的个数相同。

直流平衡后的 10 位数据经过单端转差分处理。TMDs 差分传输技术是利用 2 个引脚间电压差来传输信号的。于是可以采用 2 根信号线传输，一根传输原始信号，另一根传输原始信号的相反信号。

下图为 HDMI_TF 模块原理图，HDMI 的时钟和数据信号经过 100nF 电容耦合到接插件上，HDMI 的 18 脚 5V 电源用 DCDC 升压得到。

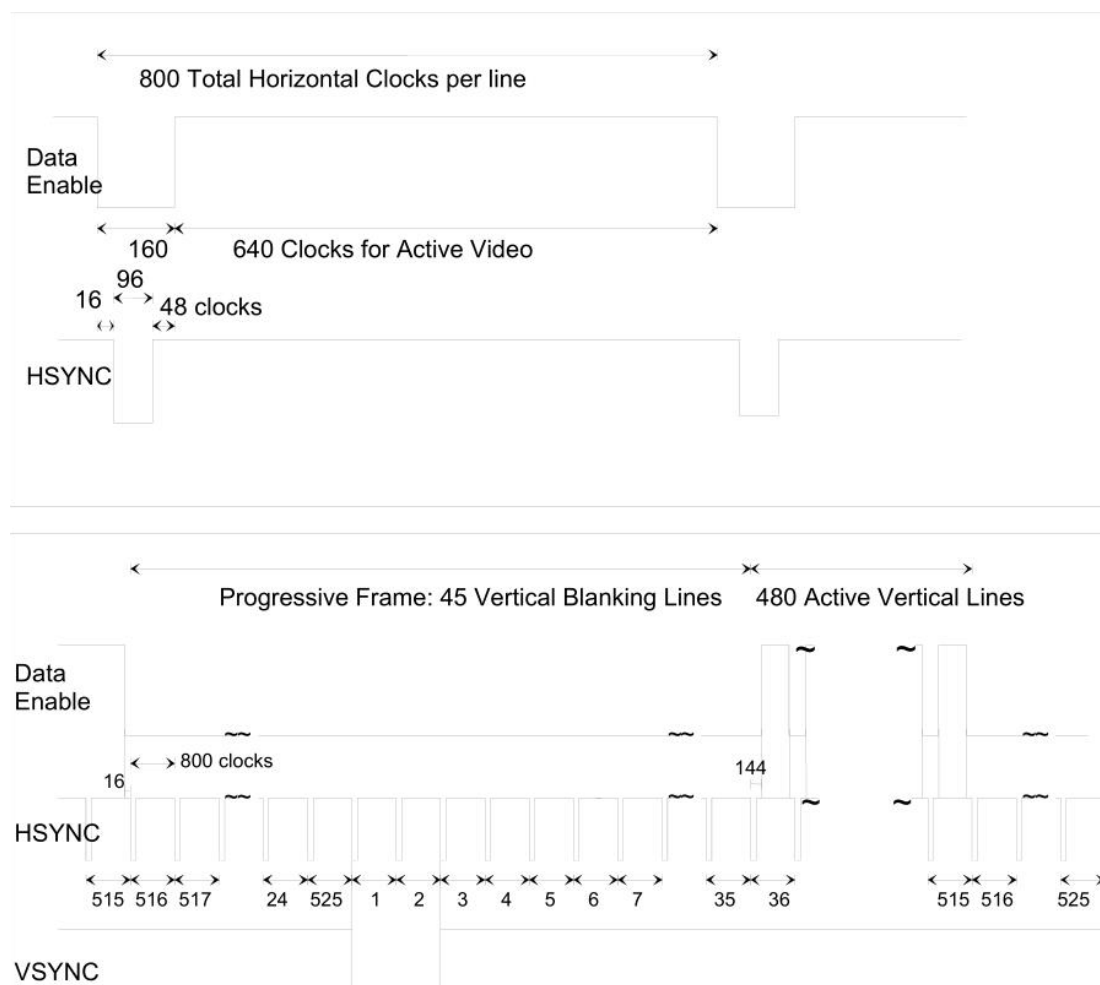


21.2. HDL 设计



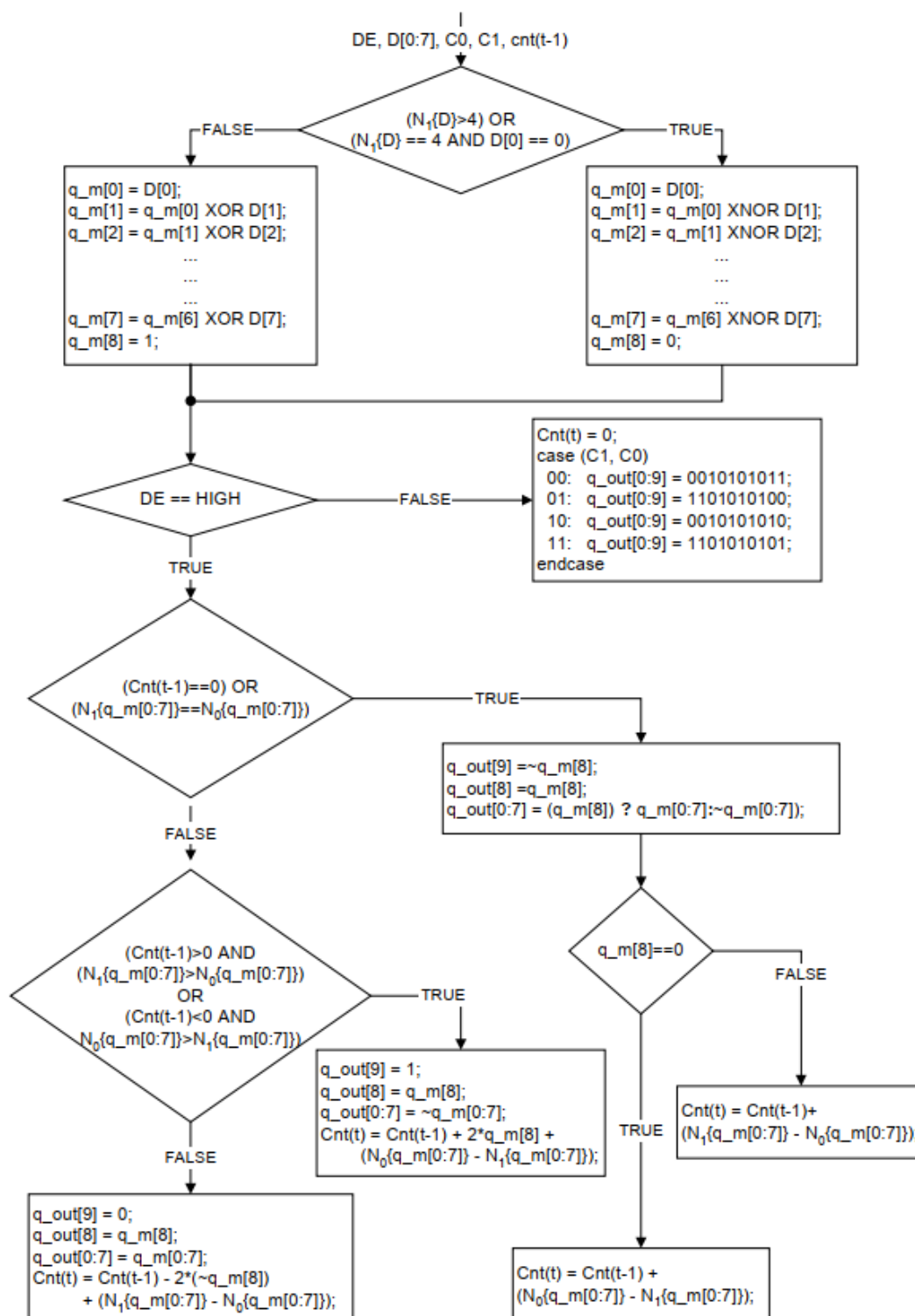
上图为工程文件的结构目录：`vga_dat`和`vga_ctrl`生成需要的时序格式的彩条数据，`my_pll`生成需要的时钟信号，`hdmi_ctrl`负责驱动 HDMI 模块。

本设计使用的是 $640 \times 480 @ 60\text{Hz}$ 的 VGA 时序接口，计算像素时钟 $pclk = 800 \times 525 \times 60 = 25200000$ ，具体的时序图如下所示。

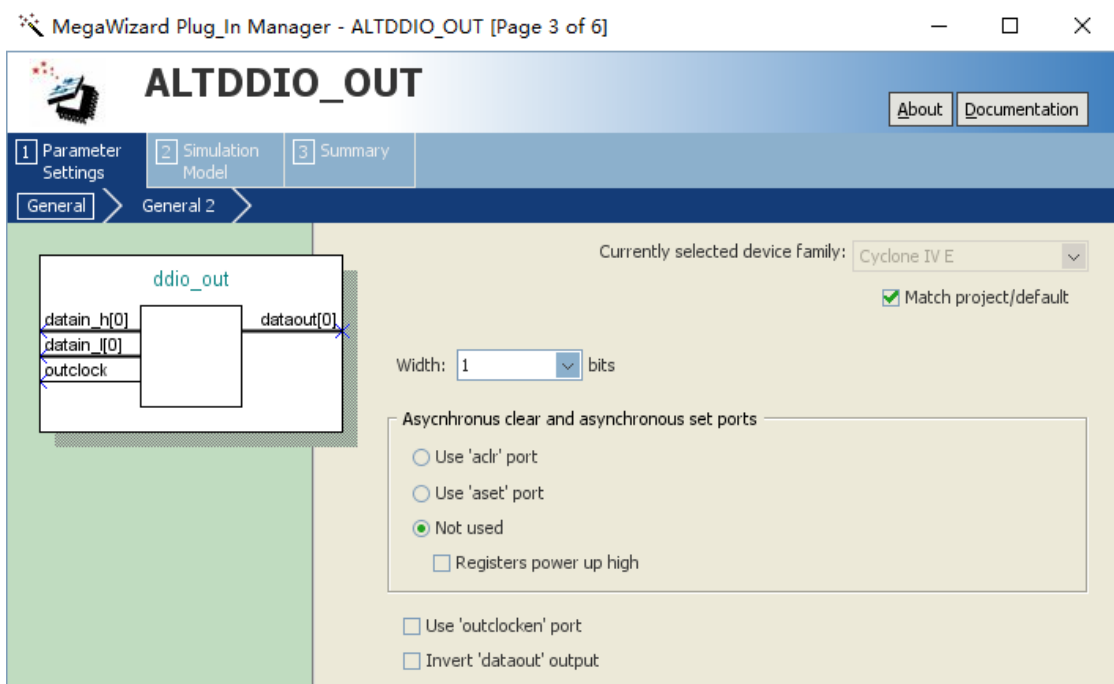


Hdmi_ctrl 中包含 encode 和 par_to_ser。

encode 负责将 8bit 数据编码成 10bit 数据，具体编码流程如下图所示：



par_to_ser 将编码后的数据串行化，ddio_out 实现 DDR 输出功能，即将两路单边沿信号转换成双边沿信号，具体的使用方法可以查阅 IP 核文档和代码。



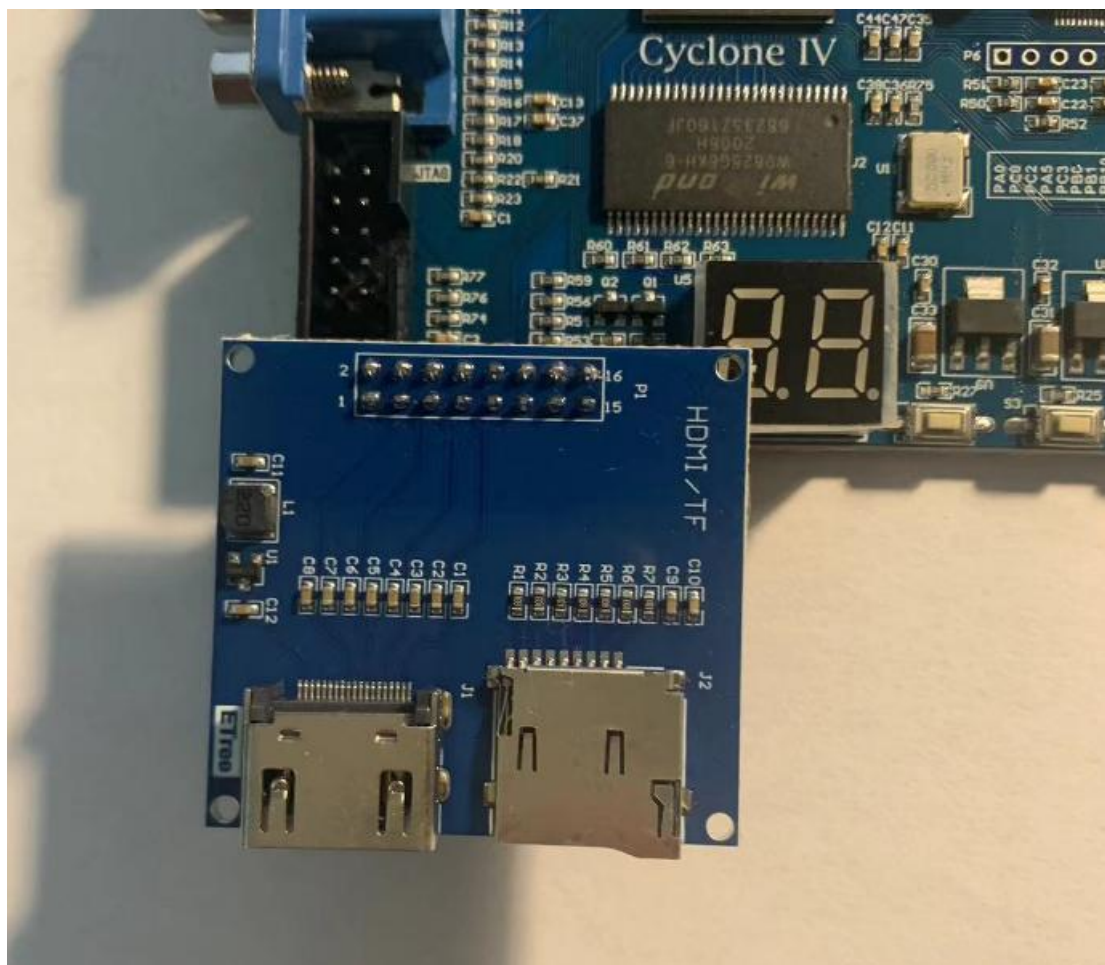
21.3. 管脚分配

选择菜单 Assignment -> Pin Planner，添加管脚约束。

Node Name	Direction	Location	I/O Bank
in clk	Input	PIN_M16	5
out hdmi_clk_n	Output	PIN_L2	2
out hdmi_clk_p	Output	PIN_D4	1
out hdmi_data_n[2]	Output	PIN_P1	2
out hdmi_data_n[1]	Output	PIN_N2	2
out hdmi_data_n[0]	Output	PIN_N1	2
out hdmi_data_p[2]	Output	PIN_D5	8
out hdmi_data_p[1]	Output	PIN_D3	8
out hdmi_data_p[0]	Output	PIN_C3	8
out led	Output	PIN_F6	8
in rst_n	Input	PIN_L10	4

21.4. 实验操作

将 HDMI_TF 模块插到 FPGA 开发板的 P2 接插件，注意 HDMI_TF 卡的 1 脚对应 P2 接插件的 1 脚，如下图所示：



用 HDMI 线连接 HDMI 显示器，插入 USB 电源，下载 FPGA 配置文件，可以看到彩条实时显示再显示器上，如下图所示：

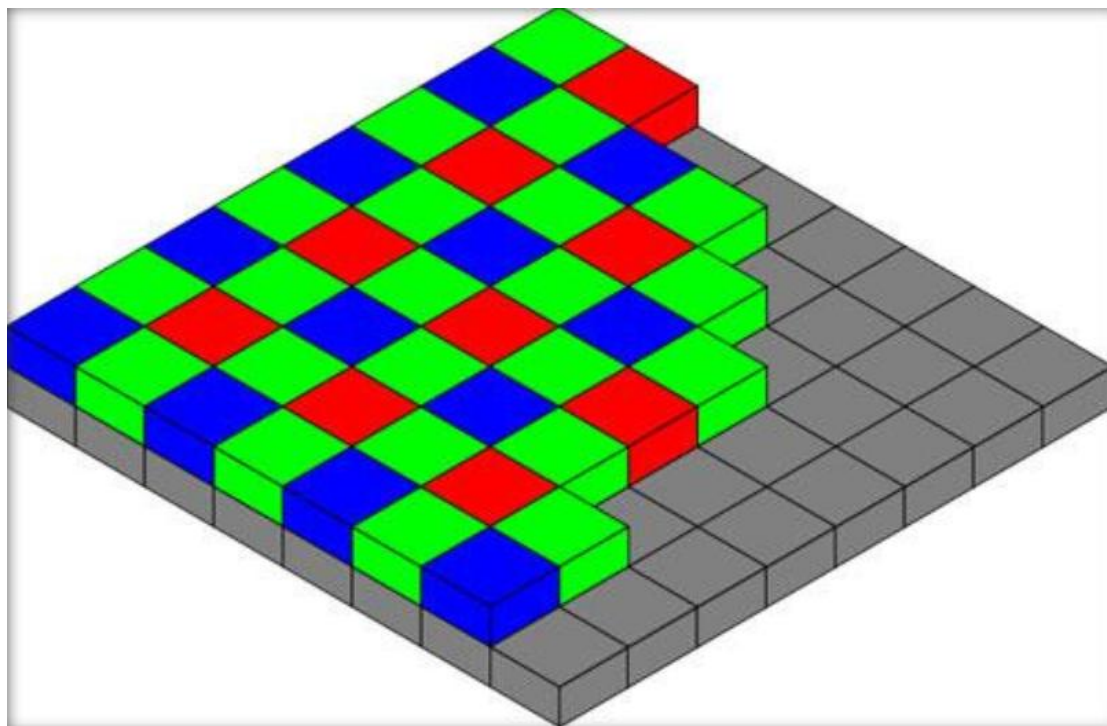


Verilog 图像处理篇

第一章 单色通道提取

本章将 CMOS 的 RG/GB 四个通道里面的一个通道提取出来,实现全分辨率的灰度显示,摄像头为 MT9V011,分辨率为 640×480 , 视频图像通过上位机软件显示,显示分辨率为 640×480 。

1.1. Bayer 阵列介绍



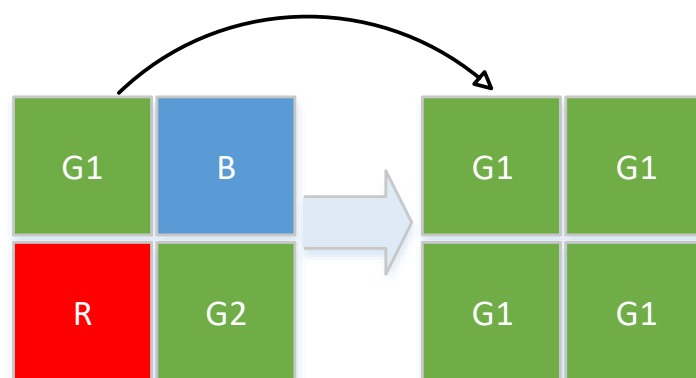
Bayer 阵列是实现 CMOS 传感器拍摄彩色图像的主要技术之一,专利注册于 1976 年,发明人为 Bryce Bayer。

Bayer 阵列模拟人眼对色彩的敏感程度,采用 1 红 2 绿 1 蓝排列的方式实现彩色信息的采集。在实际彩色显示中,需要利用反马赛克算法进行插值计算来获得一张全分辨率的图像。

Bayer 阵列又称拜耳阵列,广泛运用于现代数码相机、摄像机和手机摄像头中。

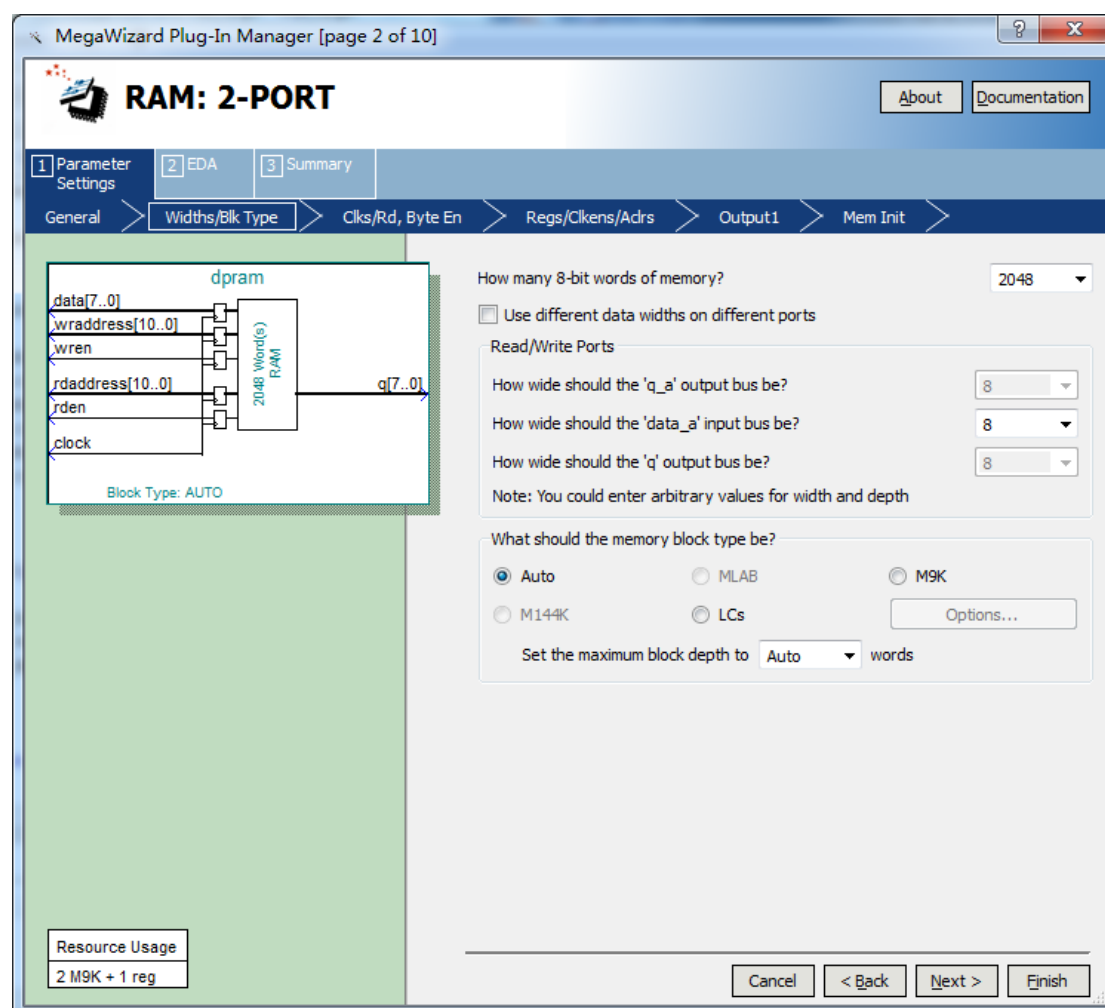
1.2. 算法原理

在 2×2 的 GBRG 阵列中,提取其中一个通道(G1 通道或 B 通道或 R 通道或 G2 通道),即 2×2 阵列中固定的一个位置,来替换其他 3 个通道的数值,这样图像中只含有一种颜色的灰度信息,最终实现了图像的灰度显示。



1.3. 算法实现

利用 DRPAM 存储 2 行(640×2)的视频数据, 然后读出偶/奇数行的偶/奇数列(其中一个像素值读取 2 次), 最后得到的 2×2 矩阵为一个通道的数值, 实现灰度显示。



顶层例化代码如下:

```
dpram dpram_inst
(
```

```

.clock(cmos_pclk) ,           // input clock
.data(cmos_data) ,           // input [7:0] data_
.rdaddress({rdaddress[10:1],1'b0}) , // input [10:0] rdaddress
.rden(rden) ,                // input rden
.wraddress(wraddress) ,      // input [10:0] wraddress
.wren(cmos_href_r) ,         // input wren
.q(q)                        // output [7:0] q
);

```

1.4. 实验效果

下图为没有实现通道提取直接送给 PC 显示的效果，从图中可以看出由于各个通道感光值不一样图片有明显的 discontinuous 现象，注意勾选 ‘Gray’ 选项。



放大细节如下图所示。



下图为经过单通道提取后的效果，图像像素点连续正常显示。



放大细节如下图所示。



第二章 亮度增强

本章实现线性亮度增强算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

2.1. 算法原理

将图像的每个像素点加上一个偏移值可以实现亮度的调节，如下面公式所示。

$$Q = a \times I + b$$

其中 I 为输入像素值， Q 为输出像素值， a 调节对比度， b 调节亮度。

2.2. 算法实现

在第一章的基础上，将灰度值加上一个偏移值，注意溢出封顶即可。

```
always @(posedge cmos_pclk)
begin
    cmos_href_o <= rden;
    cmos_vsync_o <= cmos_vsync_r;
    cmos_data_o <= (q>=8'hDF)?8'hff:(q+8'h20); //亮度增强
end
```

2.3. 实验效果

下图为原始图像。



下图为亮度增强后的图像，亮度明显变大。



第三章 对比度增强

本章实现线性对比度增强，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

3.1. 算法原理

将图像的每个像素点加上一个偏移值可以实现亮度的调节，如下面公式所示。

$$Q = a \times I + b$$

其中 I 为输入像素值， Q 为输出像素值， a 调节对比度， b 调节亮度。

3.2. 算法实现

将灰度值乘以一个大于 1 的系数(这里设置为 2)实现对比度增强，注意溢出封顶即可。

```
always @(posedge cmos_pclk)
begin
    cmos_href_o <= rden;
    cmos_vsync_o <= cmos_vsync_r;
    cmos_data_o <= (q>=8'h7F)?(8'hFF):(q*2); //对比度增强
end
```

3.3. 实验效果

下图为原始图像。



下图为对比度增强后的图片，对比度明显变大。



第四章 图像反转

本章介绍图像反转算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

4.1. 算法原理

将图像中的最大值变成最小值，最小值变成最大值，即可实现图像反正，下面为图像反转的线性公式。

$$Q = 255 - I$$

其中 I 为输入像素值，Q 为输出像素值。

4.2. 算法实现

将 255 减去输入像素值，再将得到的值输出便实现了图像翻转。

```
always @(posedge cmos_pclk)
begin
    cmos_href_o <= rden;
    cmos_vsync_o <= cmos_vsync_r;
    cmos_data_o <= 8'hff - q;    //图像反转
end
```

4.3. 实验效果

下图为实现图像反转的效果图。

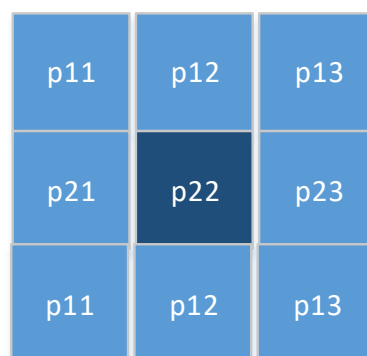


第五章 均值滤波

本章实现均值滤波算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

5.1. 算法原理

均值滤波算法采用 3×3 矩阵累加求平均，其中中间元素(p22)为原始处理像素，如下图所示，这里不考虑边界的特殊情况。

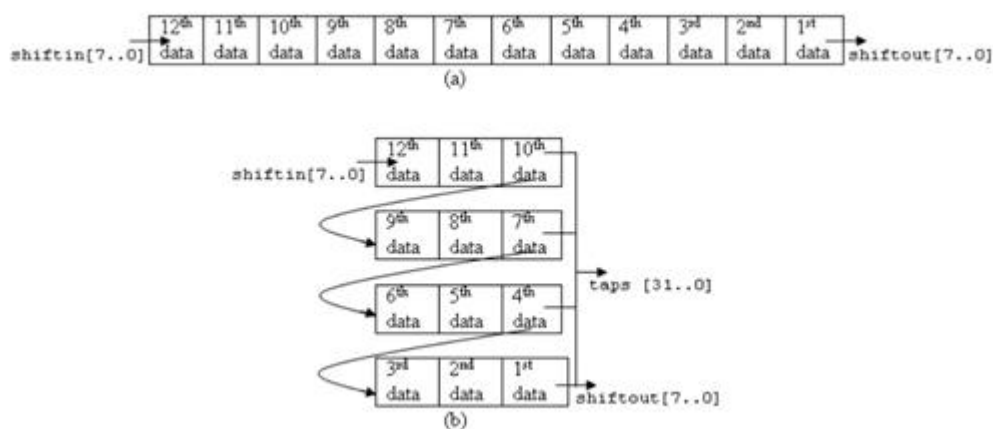


计算公式如下：

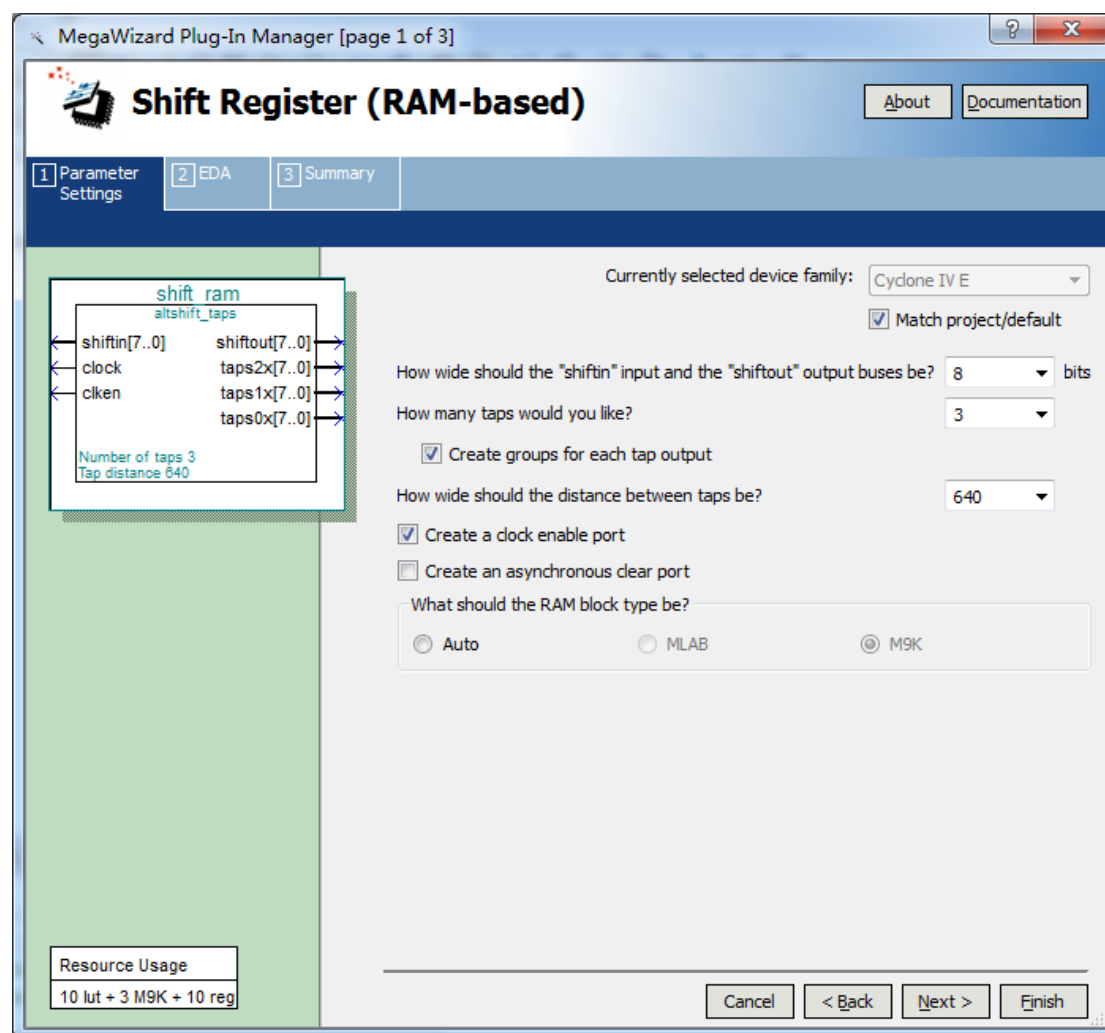
$$P_{\text{mean}} = (\sum_{i=0}^8 P_i) / 9$$

5.2. 算法实现

均值算法需要缓存 2 行或 3 行数据，然后利用寄存器打拍延时实现 3×3 的矩阵窗。数据缓存可采用 dpram 实现，不过 Quartus II 为我们提供了 shift_ram 组件，方便我们实现均值算法，时序如下图所示，具体的 shift_ram 工作原理可参考官方手册，这里不做详细介绍。



shift_ram 参数设置如下图所示。



```

wire [7:0] shiftout;
wire [7:0] taps0x;
wire [7:0] taps1x;
wire [7:0] taps2x;

shift_ram shift_ram_inst
(
    .clock(cmos_pclk) ,    // input clock_sig
    .clken(cmos_href) ,
    .shiftin(cmos_data) ,  // input [7:0] shiftin_sig
    .shiftout(shiftout) ,  // output [7:0] shiftout_sig
    .taps0x(taps0x) ,      // output [7:0] taps0x_sig
    .taps1x(taps1x) ,      // output [7:0] taps1x_sig
    .taps2x(taps2x)
);

reg [7:0] cmos_data_r0;
reg [7:0] cmos_data_r1;
reg [11:0] cmos_data_result;

```

```
reg [7:0] taps0x_r0;
reg [7:0] taps0x_r1;
reg [7:0] taps1x_r0;
reg [7:0] taps1x_r1;
reg [7:0] taps2x_r0;
reg [7:0] taps2x_r1;

always @(posedge cmos_pclk)
    if(cmos_vsync)
    begin
        taps0x_r0 <= 0;
        taps0x_r1 <= 0;
    end
    else
    begin
        taps0x_r0 <= taps0x;
        taps0x_r1 <= taps0x_r0;
    end

always @(posedge cmos_pclk)
    if(cmos_vsync)
    begin
        taps1x_r0 <= 0;
        taps1x_r1 <= 0;
    end
    else
    begin
        taps1x_r0 <= taps1x;
        taps1x_r1 <= taps1x_r0;
    end

always @(posedge cmos_pclk)
    if(cmos_vsync)
    begin
        taps2x_r0 <= 0;
        taps2x_r1 <= 0;
    end
    else
    begin
        taps2x_r0 <= taps2x;
        taps2x_r1 <= taps2x_r0;
    end

always @(posedge cmos_pclk)
    if(cmos_vsync)
```

```

    cmos_data_result <= 0;
else
    cmos_data_result <= (((taps0x + taps0x_r0) + taps0x_r1) +
                        ((taps1x + taps1x_r0) + taps1x_r1)) +
                        ((taps2x + taps2x_r0) + taps2x_r1);

```

上面代码中的‘cmos_data_result’只是累加的结果，还要求均值。在 FPGA 中实现移位操作实现除法比较好实现，所以需要将除数做一下处理，具体处理方法：将累加的和乘以 1024 再除以 1024，这样可以先人为求的 $\frac{1024}{9}$ 的值，然后和累加和相乘，最后除以 1024 得到

最终的值。 $\frac{1024}{9}=113.777$ ，取 $113 = 64 + 32 + 16 + 1$ ，具体计算公式如下：

$$\text{均值} = \text{累加和} \times (64 + 32 + 16 + 1) \div 1024$$

5.3. 实验效果

下图为原始图片效果。



原图放大细节图，数字边界上有轻微锯齿：



下图为经过均值滤波后的图像效果。



均值滤波放大后的效果图如下，数字边缘的锯齿被平滑。



第六章 高斯滤波

本章实现高斯滤波算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

6.1. 算法原理

上一章实现的均值滤波用的 3×3 矩阵加权系数为全 1，如下图所示。

1	1	1
1	1	1
1	1	1

而高斯滤波用的 3×3 矩阵加权系数是中间大两边小，如下图所示。

0.1021	0.7544	0.1021
0.7544	5.5741	0.7544
0.1021	0.7544	0.1021

计算公式如下：其中 C 为高斯加权系数。

$$P_{\text{gauss}} = (\sum_{i=0}^8 C_i \times P_i) / 9$$

6.2. 算法实现

高斯加权系数为定点小数，这里需要乘以 1024 实现整数运算，最后得到加权累加和后再除以 1024 即可，这样高斯加权系数如下图所示。

104	772	104
772	5707	772
104	772	104

```

always @(posedge cmos_pclk)
    if(cmos_vsync)
        cmos_data_result <= 0;
    else
        cmos_data_result <=
        (((taps0x*104+taps0x_r0*772)+taps0x_r1*104)+
        ((taps1x*772 + taps1x_r0*5707) +taps1x_r1*772))+
        ((taps2x*104 + taps2x_r0*772) + taps2x_r1*104);

```

上面代码中的‘cmos_data_result’只是累加的结果，还要求均值。在 FPGA 中实现移位操作实现除法比较好实现，所以需要将除数做一下处理，具体处理方法：将累加的和乘以 1024 再除以 1024，这样可以先人为求的 $\frac{1024}{9}$ 的值，然后和累加和相乘，最后除以 1024 得到

最终的值。 $\frac{1024}{9}=113.777$ ，取 $113 = 64 + 32 + 16 + 1$ ，具体计算公式如下：

$$\text{结果} = \text{累加和} \times (64 + 32 + 16 + 1) \div 1024$$

6.3. 实验效果

下图为原始图片效果。



高斯滤波后的效果图如下所示，



高斯滤波放大后的图像如下所示，数字的边缘的锯齿被平滑了一些，但整体图像的对比度比平滑滤波的效果好一些，说明高斯滤波完成了图像细节与噪声滤除之间的平衡。



下图为平滑滤波放大后的图像，好做对比。



第七章 图像锐化

本章实现图像锐化算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

7.1. 算法原理

图像锐化的主要目的是突出灰度的过渡部分，使用的算法也多种多样，这里选用二阶差分拉普拉斯算子(具体原理可参考《数字图像处理——冈萨雷斯》图像锐化章节)。常用的拉普拉斯算子如下，本设计采用第二个作为拉普拉斯算子。

0	1	0	0	-1	0	1	1	1	-1	-1	-1
1	-4	1	-1	4	-1	1	-8	1	-1	8	-1
0	1	0	0	-1	0	1	1	1	-1	-1	-1

计算公式如下：其中 R 为锐化增强后的像素， $P(4)$ 原始像素， C_i 为拉普拉斯算子。

$$R = P(4) + \sum_{i=0}^7 C_i \times P(i)$$

7.2. 算法实现

使用拉普拉斯时需要注意有负数，这里为简单处理，使用数值比较的方式避开负数表示。同时原始像素与计算好的拉普拉斯求和时要注意不要溢出，具体实现代码如下所示。

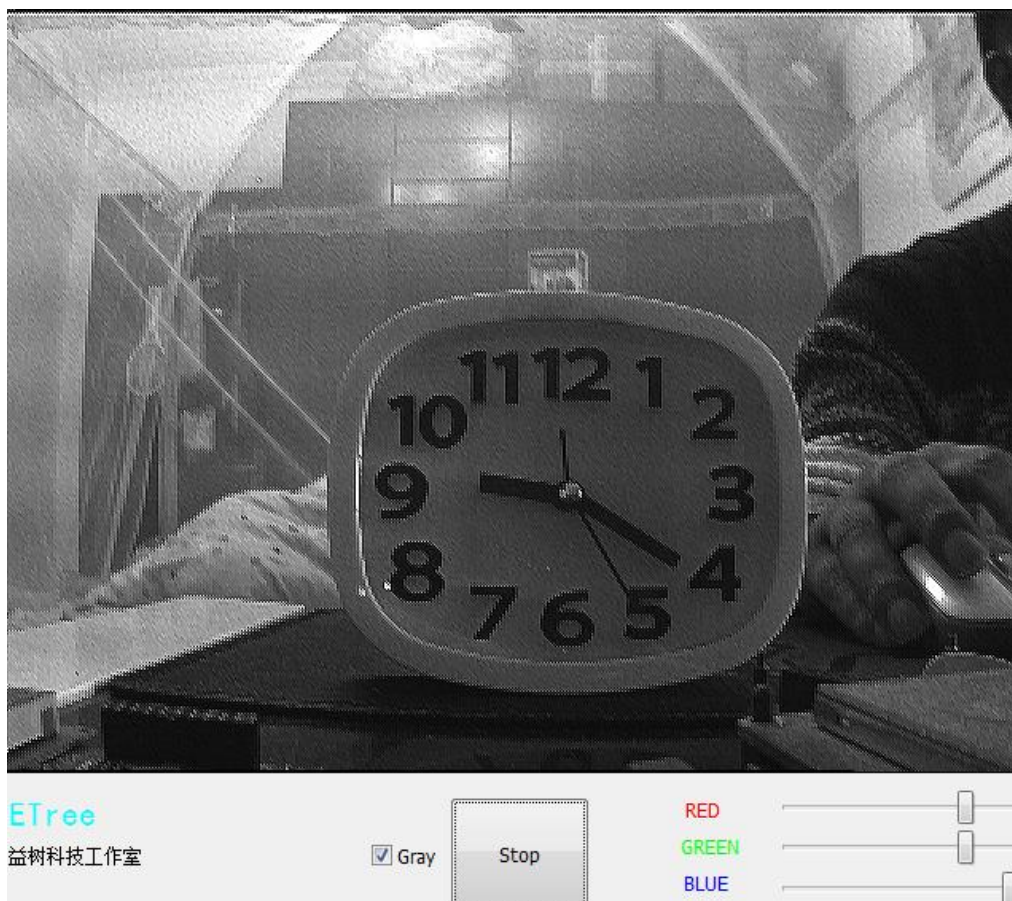
```
always @(posedge cmos_pclk)
    if(cmos_vsync)
        begin
            cmos_href_o <= 0;
            cmos_vsync_o <= 0;
            cmos_data_o <= 0;
        end
    else
        begin
            cmos_href_o <= cmos_href_r;
            cmos_vsync_o <= cmos_vsync_r;
            cmos_data_o <=
(cmos_data_result_flag==0)?(((cmos_data_result+taps1x_r1)>='hff)?
8'hff:(cmos_data_result+taps1x_r1)):
( (taps1x_r1>cmos_data_result)?(taps1x_r1-cmos_data_result):8'h01 );
        end
```

7.3. 实验效果

下图为原始图片效果。



图像锐化增强后的效果图如下所示，图像变得更加清晰。

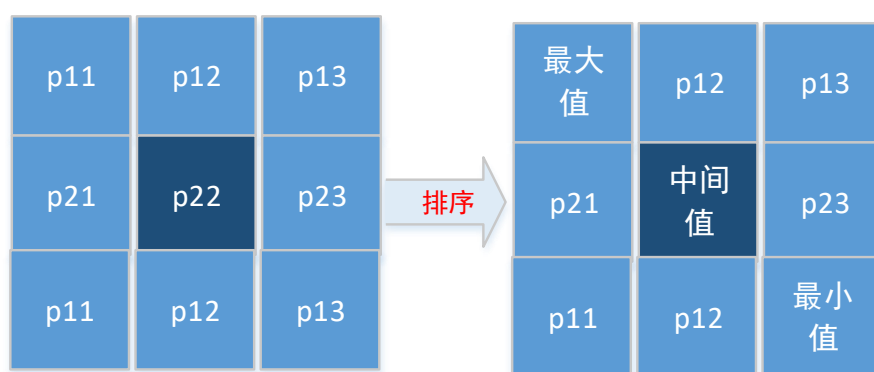


第八章 中值滤波

本章实现中值滤波算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

8.1. 算法原理

中值滤波较之前实现的算法要复杂一些，因为需要对 3×3 矩阵窗里的像素进行排序，然后找到中间值替换原始像素。



中值滤波对消除椒盐噪声非常有效，在图像处理中常用于保护边缘信息。

8.2. 算法实现

实现中值滤波关键是实现 3×3 矩阵窗内的实时排序，这里可以使用全比较法，为窗内的每个像素定义一个计数器 $cntn(n=0,1,2,3,4,5,6,7)$ 来记录比较结果，最后 $cntn$ 为 4 的值便是中间值，将对应的像素替换成原始像素即可。

```
always @(*)
    if(cmos_vsync)
        cnt0 <= 0;
    else
        cnt0 <= (taps0x>taps0x) + (taps0x_r0>taps0x) + (taps0x_r1>taps0x)
+
        (taps1x>taps0x) + (taps1x_r0>taps0x) +
(taps1x_r1>taps0x) +
        (taps2x>taps0x) + (taps2x_r0>taps0x) +
(taps2x_r1>taps0x);
... ..
always @(posedge cmos_pclk)
    if(cmos_vsync)
        begin
            cmos_href_o <= 0;
            cmos_vsync_o <= 0;
            cmos_data_o <= 0;
```



```

end
else
begin
    cmos_href_o <= cmos_href_r1;
    cmos_vsync_o <= cmos_vsync;
    cmos_data_r0 <= (cnt0==4)?taps0x:
                    (cnt1==4)?taps0x_r0:
                    (cnt2==4)?taps0x_r1:
                    (cnt3==4)?taps1x:
                    (cnt4==4)?taps1x_r0:
                    (cnt5==4)?taps1x_r1:
                    (cnt6==4)?taps2x:
                    (cnt7==4)?taps2x_r0:
                    (cnt8==4)?taps2x_r1:8'h01;
    cmos_data_o <= cmos_data_r0;
end

```

8.3. 实验效果

下图为原始图片效果。



原图放大细节图，数字边界上有轻微锯齿：



经过中值滤波后的效果图如下，



中值滤波放大后的图像如下图所示，数字的边沿变平滑的同时，数字的宽度也变窄了一点。



第九章 阈值分割

本章实现图像阈值分割算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

9.1. 算法原理

图像阈值分割有多种算法，这里只介绍简单的双阈值分割法，在高阈值和低阈值范围内的定义为 0，其他区域定义为 255。

具体计算公式如下：

$$P_{th} = \begin{cases} 0, & \text{低阈值} \leq P \leq \text{高阈值} \\ 255, & \text{其他} \end{cases}$$

9.2. 算法实现

阈值分割算法比较简单，关键是判断两个阈值的大小，这里根据具体的环境设置了高阈值为 8'h3F，低阈值为 8'h0F，如果需要自动更新两个阈值可以实时计算当前画面的平均值和均方差，然后根据它们的和确定低阈值，2 倍和确定高阈值，这里就不做详细设计，有兴趣的读者可以尝试实现。

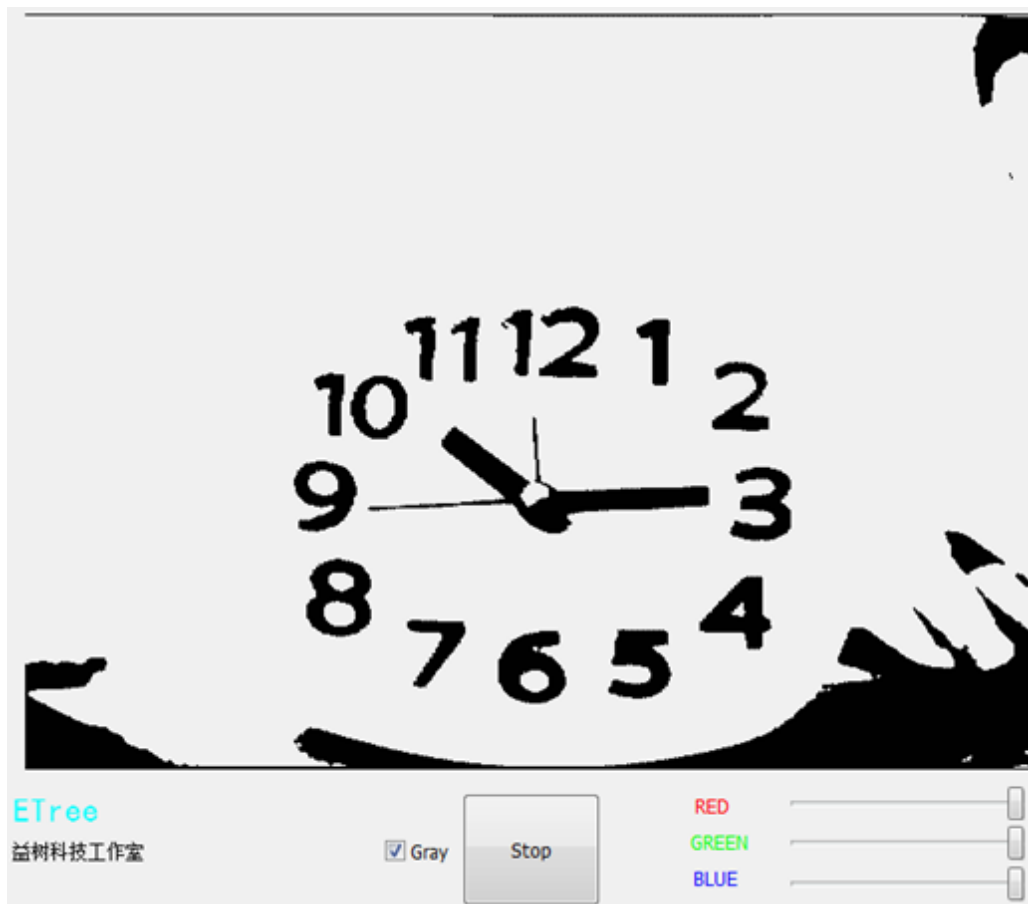
```
always @(posedge cmos_pclk)
begin
    cmos_href_r <= cmos_href;
    cmos_vsync_r <= cmos_vsync;
    cmos_data_r <= (cmos_data <= 8'h3F && cmos_data >= 8'h0F) ? 8'h01 : 8'hf0;
end
```

9.3. 实验效果

下图为原始图片效果。



阈值分割后的效果图如下，时钟的数字被分割出来。



第十章 边缘检测(Sobel)

本章实现 Sobel 边缘检测算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

10.1. 算法原理

索贝尔算子(Sobel Operator)主要用作边缘检测，它是离散差分算子。在图像上的任何一点使用此算子，将会产生对应的灰度矢量或是法矢量。

Sobel 提供了水平方向和垂直方向两个方向的滤波模板 G_X 和 G_Y ，分别如下图所示。

滤波模板 G_X			滤波模板 G_Y		
-1	0	1	1	2	1
-2	0	2	0	0	0
-1	0	1	-1	-2	-1

设 $g_x = \sum_{i=0}^7 G_X(i) \times P(i)$ ， $g_y = \sum_{i=0}^7 G_Y(i) \times P(i)$ ，则有

$$g_{xy} = \sqrt{(g_x)^2 + (g_y)^2}$$

Sobel 算子根据像素点上下、左右邻点灰度加权差，在边缘处达到极值来检查边缘。Sobel 算子优点是计算简单、速度快，但由于只采用了 2 个方向的模板，因此对纹理较为复杂的图像效果不是很好。当对精度要求不高时，Sobel 算子是一种较为常用的边缘检测算法。

在计算得出 g_{xy} 后，对其进行阈值操作，认为大于阈值的像素点为边缘点。

10.2. 算法实现

在 Sobel 算子中有负系数，这里可以参考“第七章 图像锐化”的处理方法，使用数值比较的方式避开负数表示，具体代码如下。

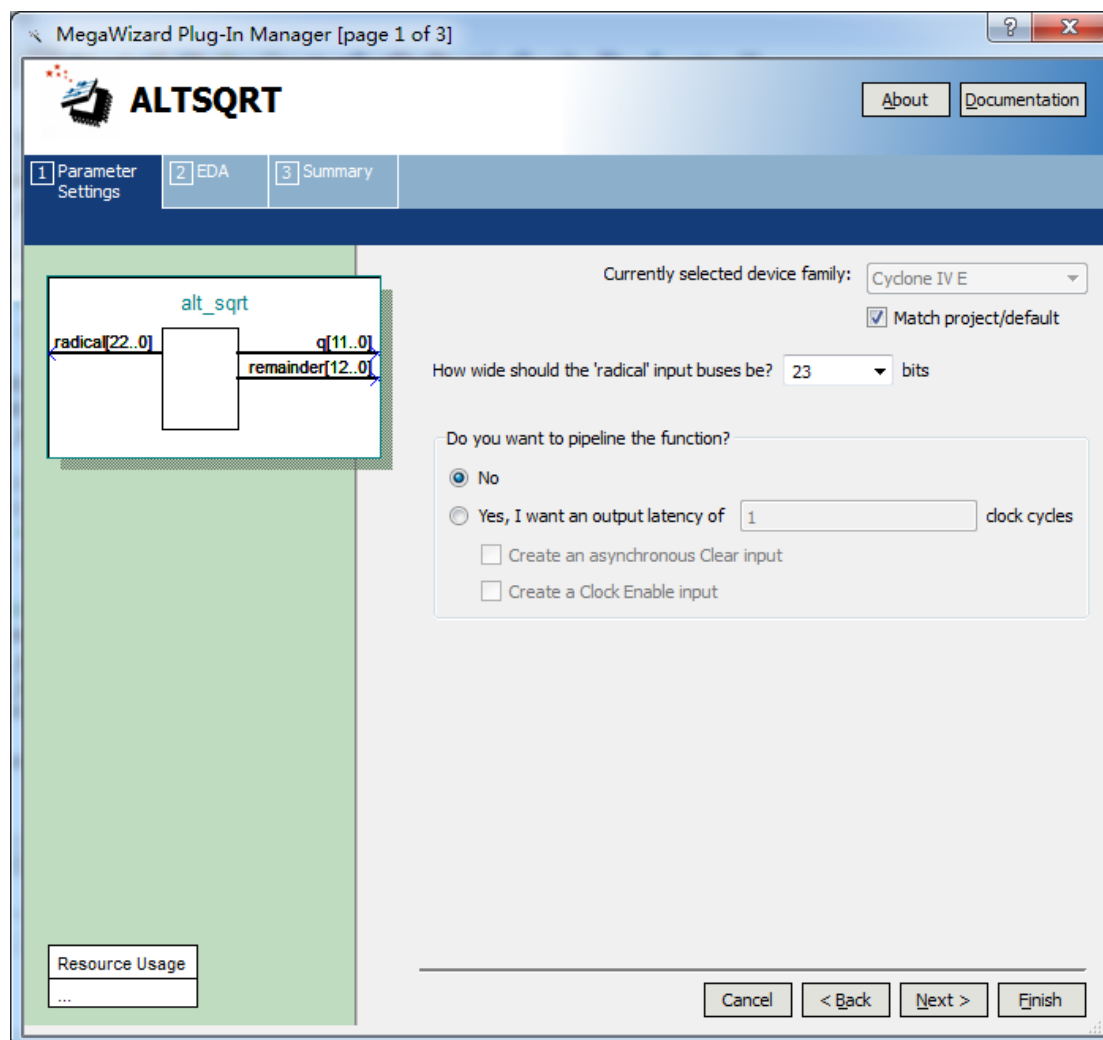
```
assign sum0_gx = ({3'b000,taps0x_r1} + {2'b00,taps1x_r1,1'b0}) +
{3'b000,taps2x_r1};
assign sum1_gx = ({3'b000,taps0x} + {2'b00,taps1x,1'b0}) +
{3'b000,taps2x};
assign sobel_gx =
(sum0_gx>=sum1_gx)?(sum0_gx-sum1_gx):(sum1_gx-sum0_gx);
assign sum0_gy = ({3'b000,taps0x} + {2'b00,taps0x_r0,1'b0}) +
{3'b000,taps0x_r1};
assign sum1_gy = ({3'b000,taps2x} + {2'b00,taps2x_r0,1'b0}) +
```

```

{3'b000,taps2x_r1};
assign sobel_gy =
(sum0_gy>=sum1_gy)?(sum0_gy-sum1_gy):(sum1_gy-sum0_gy);

```

关于开方运算，Quartus II 为我们提供了现有的 IP 可以使用，参数配置如下图所示。



```

wire [11:0] sqrt_q;

alt_sqrt    alt_sqrt_inst (
    .radical ( sobel_gxgy ),
    .q ( sqrt_q ),
    .remainder ( )
);

always @(posedge cmos_pclk)
    if(sqrt_q>='d30)
        cmos_data_o <= 8'h01;
    else
        cmos_data_o <= 8'hff;

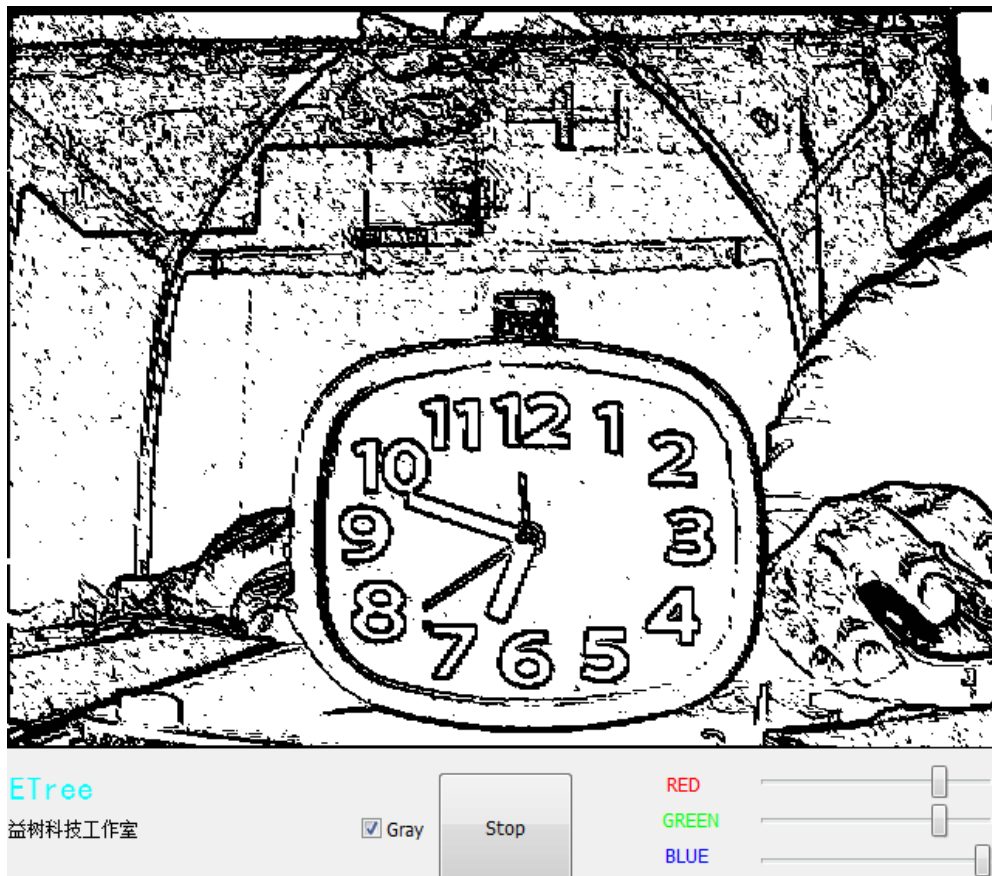
```

10.3. 实验效果

原始效果图如下所示。



经过 Sobel 边缘检测后效果图如下所示。



第十一章 图像膨胀

本章实现图像膨胀算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

11.1. 算法原理

膨胀是形态学滤波的基本运算。膨胀(dilate)就是求局部最大值，这样会使图像中的高亮区域逐渐增大，效果图将会拥有比原图更大的高亮区域，整体亮度会增加，同时可以连通相邻的高亮区域。通过膨胀可以将图像中的裂缝填补。

具体的计算公式如下：

$$g_{xy} = \max(P(i)), i = 0, 1, 2, 3, 4, 5, 6, 7, 8$$

11.2. 算法实现

在中值滤波章节，在 3×3 矩形窗内找的是中间值，这里可以用同样的原理找到最大值，具体代码如下。

```
always @(*)
    if(cmos_vsync)
        cnt0 <= 0;
    else
        cnt0 <= (taps0x>taps0x) + (taps0x_r0>taps0x) + (taps0x_r1>taps0x)
+
                (taps1x>taps0x) + (taps1x_r0>taps0x) +
(taps1x_r1>taps0x) +
                (taps2x>taps0x) + (taps2x_r0>taps0x) +
(taps2x_r1>taps0x);
... ..
always @(posedge cmos_pclk)
    if(cmos_vsync)
        begin
            cmos_href_o <= 0;
            cmos_vsync_o <= 0;
            cmos_data_o <= 0;
        end
    else
        begin
            cmos_href_o <= cmos_href_r1;
            cmos_vsync_o <= cmos_vsync;
            cmos_data_r0 <= (cnt0==0)?taps0x:
                            (cnt1==0)?taps0x_r0:
                            (cnt2==0)?taps0x_r1:
                            (cnt3==0)?taps1x:
```

```
(cnt4==0)?taps1x_r0:
(cnt5==0)?taps1x_r1:
(cnt6==0)?taps2x:
(cnt7==0)?taps2x_r0:
(cnt8==0)?taps2x_r1:8'h01;

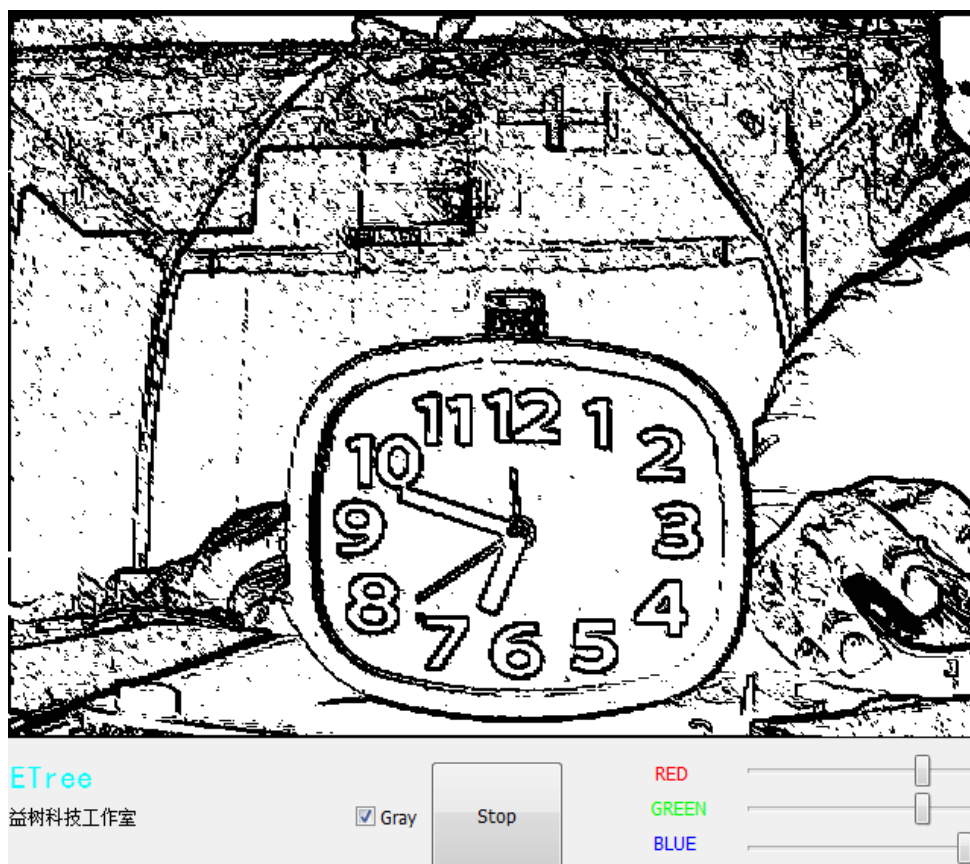
cmos_data_o <= cmos_data_r0;

end
```

11.3. 实验效果

在边缘检测(sobel)基础上进行膨胀操作，观察算法效果。

原始 sobel 效果图如下所示。



膨胀后的效果图如下所示，图像变亮了，且轮廓便细了。



第十二章 图像腐蚀

本章实现图像腐蚀算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

12.1. 算法原理

腐蚀(erode)是形态学滤波的基本运算，与膨胀是一对对立操作，求局部最小值。通过腐蚀操作后，高亮区域被腐蚀掉，会比原图更暗些。腐蚀操作会连通相邻比较暗的区域。

具体计算公式如下。

$$g_{xy} = \min(P(i)), i = 0, 1, 2, 3, 4, 5, 6, 7, 8$$

12.2. 算法实现

在中值滤波章节，在 3×3 矩形窗内找的是中间值，这里可以用同样的原理找到最小值，具体代码如下。

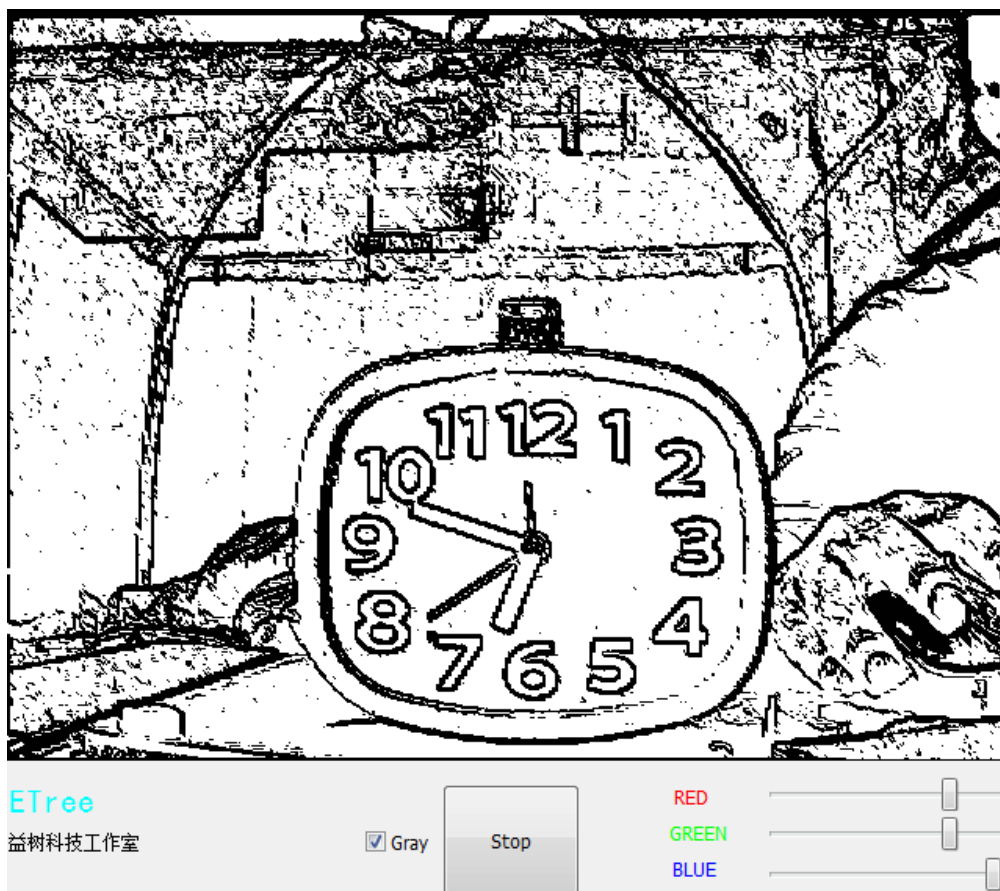
```
always @(*)
    if(cmos_vsync)
        cnt0 <= 0;
    else
        cnt0 <= (taps0x>taps0x) + (taps0x_r0>taps0x) + (taps0x_r1>taps0x)
+
        (taps1x>taps0x) + (taps1x_r0>taps0x) +
(taps1x_r1>taps0x) +
        (taps2x>taps0x) + (taps2x_r0>taps0x) +
(taps2x_r1>taps0x);
... ..
always @(posedge cmos_pclk)
    if(cmos_vsync)
        begin
            cmos_href_o <= 0;
            cmos_vsync_o <= 0;
            cmos_data_o <= 0;
        end
    else
        begin
            cmos_href_o <= cmos_href_r1;
            cmos_vsync_o <= cmos_vsync;
            cmos_data_r0 <= (cnt0==8)?taps0x:
                            (cnt1==8)?taps0x_r0:
                            (cnt2==8)?taps0x_r1:
                            (cnt3==8)?taps1x:
                            (cnt4==8)?taps1x_r0:
```

```
(cnt5==8)?taps1x_r1:  
(cnt6==8)?taps2x:  
(cnt7==8)?taps2x_r0:  
(cnt8==8)?taps2x_r1:8'h01;  
cmos_data_o <= cmos_data_r0;  
end
```

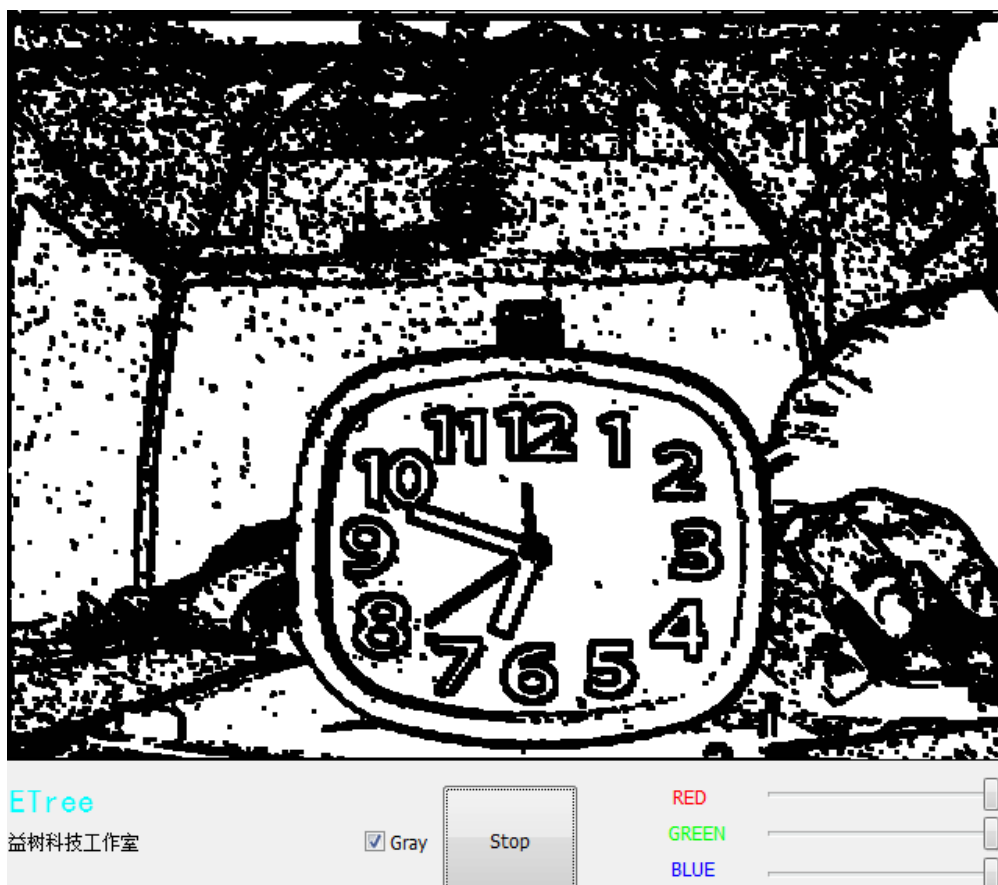
12.3. 实验效果

在边缘检测(sobel)基础上进行腐蚀操作，观察算法效果。

原始 sobel 效果图如下所示。



腐蚀后的效果图如下所示，图像变暗了，且轮廓便粗了。



第十三章 开运算

本章实现开运算算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

13.1. 算法原理

开运算(Opening)是先腐蚀后膨胀的过程。开运算一般是对象的轮廓变得更加光滑，断开狭窄的间断和消除细小的凸起物。因此，开运算可用于消除小物体。

具体计算公式如下。

$$g_{xy} = \max(\min(P(i)), i = 0, 1, 2, 3, 4, 5, 6, 7, 8)$$

13.2. 算法实现

开运算只要将“图像膨胀”和“图像腐蚀”串联起来便可实现。

```
//file name: filter_open.v
//author: ETree
//date: 2017.10.27
//function: 开运算
//log:

module filter_open(
    input cmos_pclk,
    input cmos_href,
    input cmos_vsync,
    input [7:0] cmos_data,

    output cmos_href_o,
    output cmos_vsync_o,
    output [7:0] cmos_data_o
);

wire cmos_href_dilate;
wire cmos_vsync_dilate;
wire [7:0] cmos_data_dilate;

filter_erode filter_erode_inst
(
    .cmos_pclk(cmos_pclk) , // input cmos_pclk_sig
    .cmos_href(cmos_href) , // input cmos_href_sig
    .cmos_vsync(cmos_vsync) , // input cmos_vsync_sig
    .cmos_data(cmos_data) , // input [7:0] cmos_data_sig
    .cmos_href_o(cmos_href_dilate) , // output cmos_href_o_sig
```



```

.cmos_vsync_o(cmos_vsync_dilate) , // output cmos_vsync_o_sig
.cmos_data_o(cmos_data_dilate) // output [7:0] cmos_data_o_sig
);

filter_dilate filter_dilate_inst
(
.cmos_pclk(cmos_pclk) , // input cmos_pclk_sig
.cmos_href(cmos_href_dilate) , // input cmos_href_sig
.cmos_vsync(cmos_vsync_dilate) , // input cmos_vsync_sig
.cmos_data(cmos_data_dilate) , // input [7:0] cmos_data_sig
.cmos_href_o(cmos_href_o) , // output cmos_href_o_sig
.cmos_vsync_o(cmos_vsync_o) , // output cmos_vsync_o_sig
.cmos_data_o(cmos_data_o) // output [7:0] cmos_data_o_sig
);

endmodule

```

13.3. 实验效果

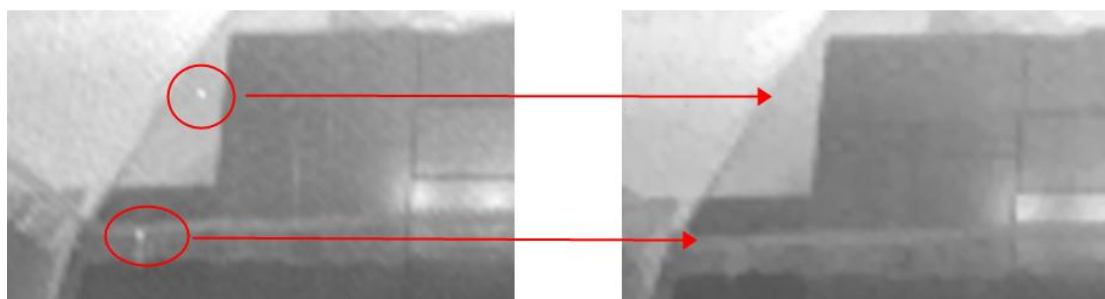
原始效果图如下所示。



开运算后的效果图如下所示，图像变的光滑了。



开运算放大后的图片，细小的凸起物被消除了。



第十四章 闭运算

本章实现闭运算算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

14.1. 算法原理

闭运算(Closing)是先膨胀后腐蚀的过程。闭运算一般是对对象的轮廓变得更加光滑，消除细小的孔洞。因此，闭运算在排除小型黑洞时经常被用到。

具体计算公式如下。

$$g_{xy} = \min(\max(P(i)), i = 0, 1, 2, 3, 4, 5, 6, 7, 8)$$

14.2. 算法实现

闭运算只要将“图像膨胀”和“图像腐蚀”串联起来便可实现。

```
//file name:    filter_close.v
//author:      ETree
//date:        2017.10.27
//function:    闭运算
//log:

module filter_close(
    input  cmos_pclk,
    input  cmos_href,
    input  cmos_vsync,
    input  [7:0] cmos_data,

    output cmos_href_o,
    output cmos_vsync_o,
    output [7:0] cmos_data_o
);

wire cmos_href_dilate;
wire cmos_vsync_dilate;
wire [7:0] cmos_data_dilate;

filter_dilate filter_dilate_inst
(
    .cmos_pclk(cmos_pclk) , // input  cmos_pclk_sig
    .cmos_href(cmos_href) , // input  cmos_href_sig
    .cmos_vsync(cmos_vsync) , // input  cmos_vsync_sig
    .cmos_data(cmos_data) , // input [7:0] cmos_data_sig
    .cmos_href_o(cmos_href_dilate) , // output cmos_href_o_sig
```

```
.cmos_vsync_o(cmos_vsync_dilate) , // output cmos_vsync_o_sig
.cmos_data_o(cmos_data_dilate) // output [7:0] cmos_data_o_sig
);

filter_erode filter_erode_inst
(
    .cmos_pclk(cmos_pclk) , // input cmos_pclk_sig
    .cmos_href(cmos_href_dilate) , // input cmos_href_sig
    .cmos_vsync(cmos_vsync_dilate) , // input cmos_vsync_sig
    .cmos_data(cmos_data_dilate) , // input [7:0] cmos_data_sig
    .cmos_href_o(cmos_href_o) , // output cmos_href_o_sig
    .cmos_vsync_o(cmos_vsync_o) , // output cmos_vsync_o_sig
    .cmos_data_o(cmos_data_o) // output [7:0] cmos_data_o_sig
);

endmodule
```

14.3. 实验效果

原始效果图如下所示。



闭运算后的效果图如下所示，图像变的光滑了。



第十五章 Tophat 形态学滤波

本章实现 Tophat 形态学滤波算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

15.1. 算法原理

Tophat 变换是原图像与开运算结果之差，突显出图像轮廓周围更亮的区域。
具体的计算公式如下：

$$g_{xy} = \begin{cases} P(i) - \text{open}(P(i)), & P(i) \geq \text{open}(P(i)) \\ 0, & \text{其他} \end{cases}$$

15.2. 算法实现

Tophat 滤波只需要在“开运算”章节的基础上，将原始像素信号同步到‘开运算’最终结果输出，这里只要在‘开运算’模块中的膨胀模块中添加一个 shift_ram，实现时钟延时的同步，代码如下。

```
wire [7:0] taps_1x_old;

shift_ram shift_ram_inst1
(
    .clock(cmos_pclk) ,      // input clock_sig
    .clken(cmos_href) ,
    .shiftin(cmos_data_old_in) , // input [7:0] shiftin_sig
    .shiftout() ,           // output [7:0] shiftout_sig
    .taps0x() ,             // output [7:0] taps0x_sig
    .taps1x(taps_1x_old) ,   // output [7:0] taps1x_sig
    .taps2x()
);

reg [7:0] taps_1x_old_r;

always @(posedge cmos_pclk)
    taps_1x_old_r <= taps_1x_old;
```

在“开运算”输出端用原图像减去“开运算”的结果，代码如下。

```
//file name:    filter_tophat.v
//author:      ETree
//date:        2017.10.27
//function:    开运算
//log:
```

```

module filter_tophat(
    input cmos_pclk,
    input cmos_href,
    input cmos_vsync,
    input [7:0] cmos_data,

    output cmos_href_o,
    output cmos_vsync_o,
    output [7:0] cmos_data_o
);

wire [7:0] cmos_data_open;
wire [7:0] cmos_data_old;

filter_open filter_open_inst
(
    .cmos_pclk(cmos_pclk) , // input cmos_pclk_sig
    .cmos_href(cmos_href) , // input cmos_href_sig
    .cmos_vsync(cmos_vsync) , // input cmos_vsync_sig
    .cmos_data_old(cmos_data_old),
    .cmos_data(cmos_data) , // input [7:0] cmos_data_sig
    .cmos_href_o(cmos_href_o) , // output cmos_href_o_sig
    .cmos_vsync_o(cmos_vsync_o) , // output cmos_vsync_o_sig
    .cmos_data_o(cmos_data_open) // output [7:0] cmos_data_o_sig
);

assign cmos_data_o =
(cmos_data_old>=cmos_data_open)?(cmos_data_old-cmos_data_open):8'h01;

endmodule

```

与 Tophat 对应的还有一个 Blackhat 形态滤波器，用“闭运算”减去原来图像，突显出原图像轮廓周围更暗的区域。“Blackhat 形态学滤波器”留给有兴趣的读者完成，算法实现方法和本章类似。

15.3. 实验效果

原始效果图如下所示。



Tophat 滤波后的效果图如下所示，图像轮廓周围更亮的区域被突显出来。



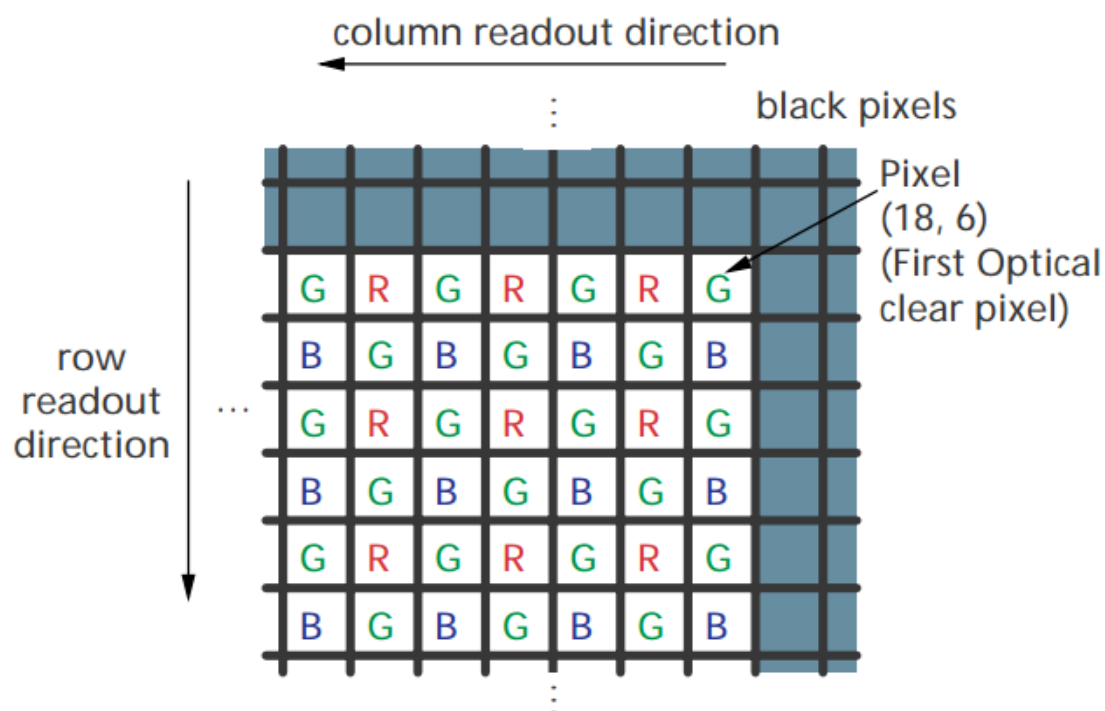
第十六章 RAW8 转 RGB888

本章实现 RAW8 转 RGB888，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

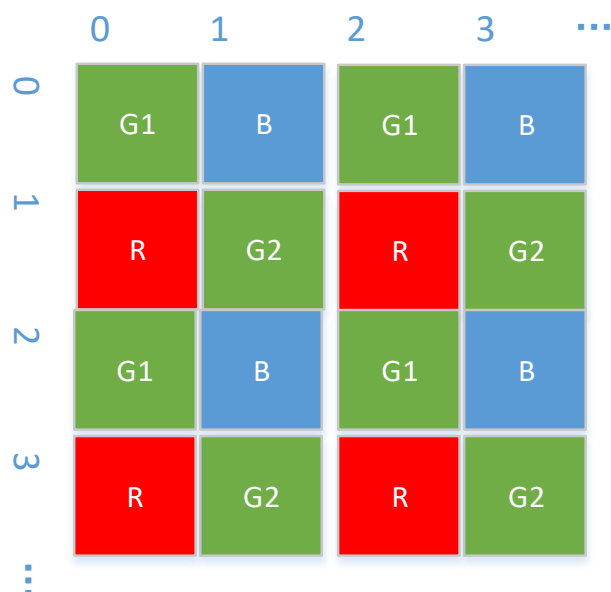
16.1. 算法原理

在“单色通道提取”章节以及介绍了 Bayer 阵列原理，这里以 2×2 窗口实现 RAW8 转 RGB888 算法。

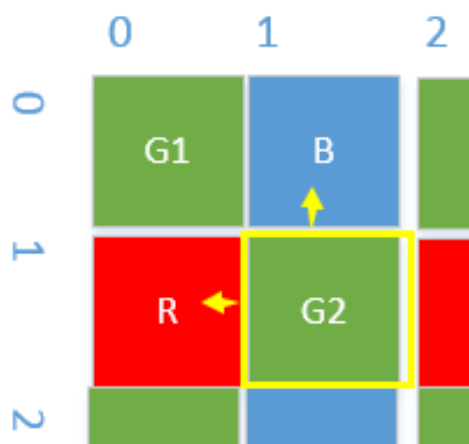
在 CMOS 初始化模块中，起始读取行为 $0x0b$ (奇数)，起始读取列为 $0x11$ (奇数)，而 MT9V011 的 Bayer 阵列如下图所示。



由上图可重新得到 640×480 的 Bayer 阵列结构，如下图所示。

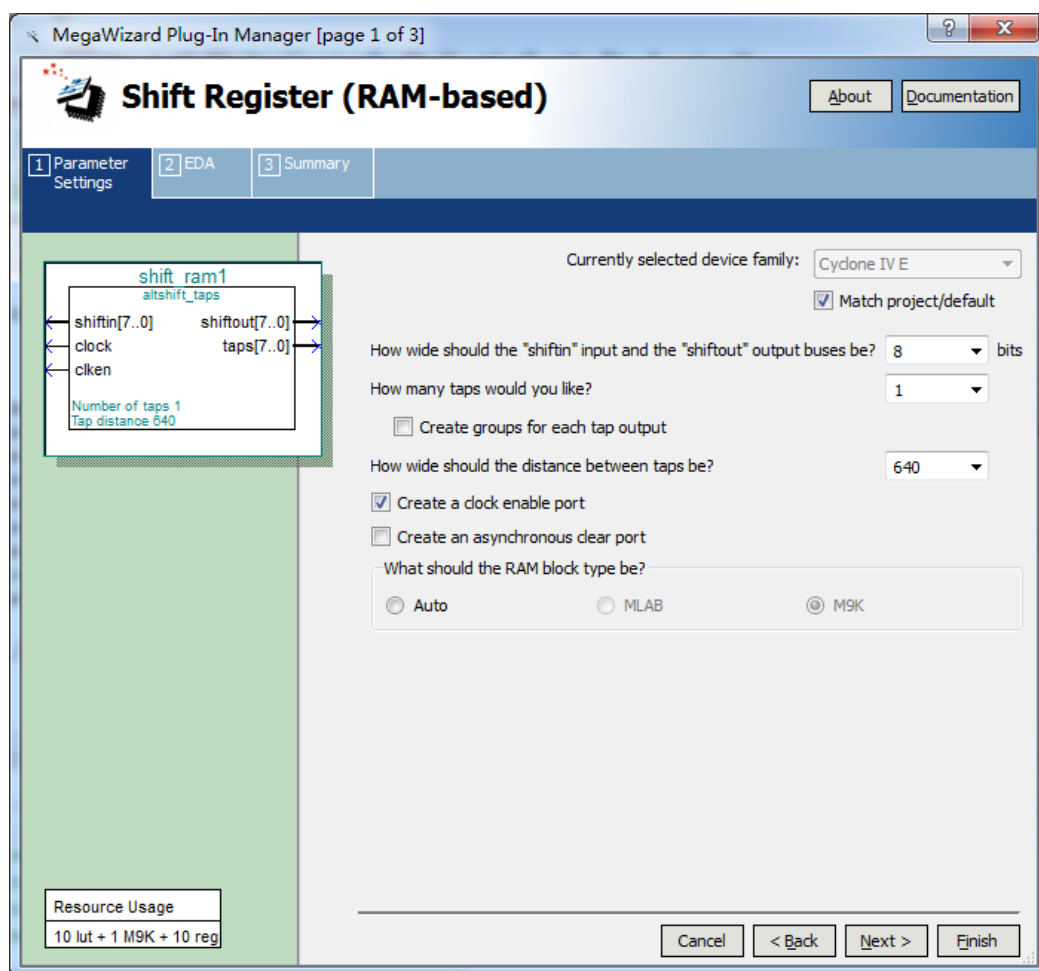


基于上图,设计一个在 2×2 的窗口,以右下角的像素为基准点,分别计算出 RED、GREEN、BLUE 分量。以『奇数行/奇数列』为例, G2 为基准点,可得
$$\begin{cases} \text{RED} = R \\ \text{GREEN} = (G1 + G2)/2 \\ \text{BLUE} = B \end{cases}$$
 以此类推可以得到『奇数行/偶数列』、『偶数行/奇数列』、『偶数行/偶数列』的情况。



16.2. 算法实现

使用 shift register 实现 2×2 的窗口设计,根据行计数器 v_cnt 和列计数器 h_cnt 分别计算四周不同情况的 RED、GREEN、BLUE 分量。



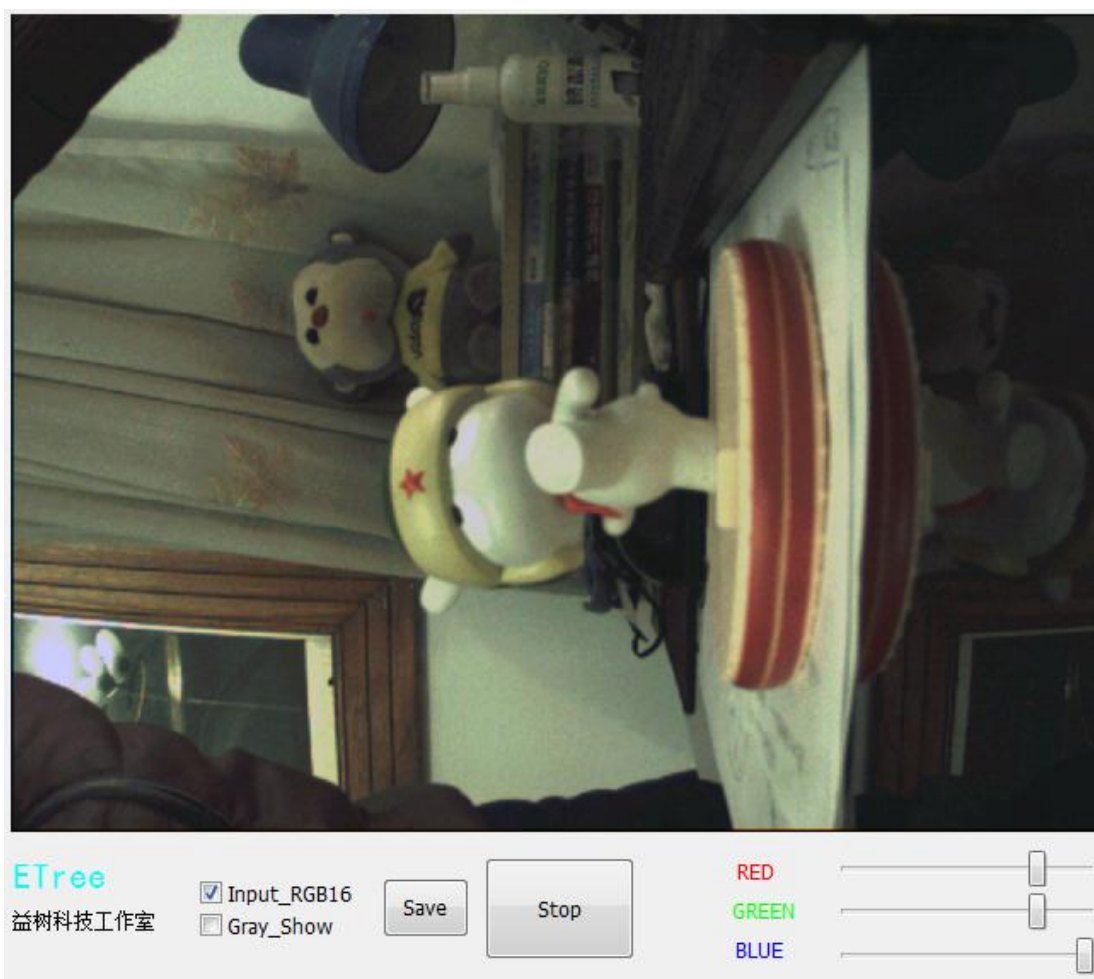
具体代码如下：

```
always @(posedge cmos_pclk)
    case ({h_cnt[0], v_cnt[0]})
        2'b00: //偶数行/偶数列
        begin
            red <= shiftout;
            green <= (cmos_data_r0 + shiftout_r0)/2;
            blue <= cmos_data_r1;
        end
        2'b10: //偶数行/奇数列
        begin
            red <= shiftout_r0;
            green <= (cmos_data_r1 + shiftout)/2;
            blue <= cmos_data_r0;
        end
        2'b01: //奇数行/偶数列
        begin
            red <= cmos_data_r0;
            green <= (cmos_data_r1 + shiftout)/2;
            blue <= shiftout_r0;
        end
    end
```

```
end
2'b11: //奇数行/奇数列
begin
    red <= cmos_data_r1;
    green <= (cmos_data_r0 + shiftout_r0)/2;
    blue <= shiftout;
end
default:
begin
    red <= 8'hff;
    green <= 8'hff;
    blue <= 8'hff;
end
endcase
```

16.3. 实验效果

RAW8 转 RGB888 的效果图如下所示，注意勾选“Input_RGB16”表示输入信号为 RGB565 信号。



第十七章 Alpha 背景叠加

本章实现 Alpha 背景叠加算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

17.1. 算法原理

Alpha 叠加算法实现背景色半透明的效果，假设颜色分量(这里以单色为例)的取值范围为 $0 \sim 1$ ，背景图片分配 0.25，原始图像分配 0.75，则表示背景的透明度为 75%，原始图像的透明度为 25%。将背景图片和原始图像叠加，则实现了背景叠加的效果。

具体计算公式如下。

$$\text{Result} = \alpha \times I + (1 - \alpha) \times J, \text{ 其中 } I \text{ 为原始图像, } J \text{ 为背景图像}$$

17.2. 算法实现

取 α 为 0.75，则 $\text{Result} = 0.75 \times I + 0.25 \times J$;

等式变换为 $4 \times \text{Result} = 3 \times I + J = I + 2 \times I + J$

```
wire [9:0] temp;

assign temp = data + (cmos_data*2+cmos_data); //TEMP = data + cmos_data*3

always @(posedge cmos_pclk)
    cmos_data_o <= temp[9:2];
```

17.3. 实现效果

下图为渐进数背景叠加在原始图像上的效果。



第十八章 直方图灰度拉伸算法

本章实现图像直方图灰度拉伸算法，摄像头为 MT9V011，分辨率为 640×480 ，视频图像通过上位机软件显示，显示分辨率为 640×480 。

18.1. 算法原理

直方图灰度拉伸属于线性图像算法，拓展图像的直方图，使其充满整个灰度级范围，这样可以提高图像对比度。

设 $f(x,y)$ 为输入图像，其最小灰度 A 和最大灰度 B 定义如下：

$$A = \min[f(x,y)], \quad B = \max[f(x,y)]$$

将 A 和 B 线性映射到 $0 \sim 255$ ，定义如下：

$$g(x,y) = \left(\frac{255}{B - A} \times [f(x,y) - A] \right)$$

18.2. 算法实现

这里需要计算当前图像中像素的最大值和最小值，由于视频数据前后的变化不是很大，为了不使用帧缓冲，我们采用上一帧计算的像素最大值和最小值来计算当前帧的图像算法。

计算图像中的最大值和最小值算法实现如下所示。

```
//caculate the first min
always @(posedge cmos_pclk)
    if(field==1'b0)
        if(cmos_vsync_r1)
            min0 <= 8'hff;
        else if(cmos_href)
            if(min0 >= cmos_data)
                min0 <= cmos_data;
            else
                min0 <= min0;

//caculate the second min
always @(posedge cmos_pclk)
    if(field==1'b1)
        if(cmos_vsync_r1)
            min1 <= 8'hff;
        else if(cmos_href)
            if(min1 >= cmos_data)
                min1 <= cmos_data;
            else
                min1 <= min1;

//caculate the first max
```



```

always @(posedge cmos_pclk)
    if(field==1'b0)
        if(cmos_vsync_r1)
            max0 <= 8'h00;
        else if(cmos_href)
            if(max0<=cmos_data)
                max0 <= cmos_data;
            else
                max0 <= max0;

//caculate the second max
always @(posedge cmos_pclk)
    if(field==1'b1)
        if(cmos_vsync_r1)
            max1 <= 8'h00;
        else if(cmos_href)
            if(max1<=cmos_data)
                max1 <= cmos_data;
            else
                max1 <= max1;

```

由上一节的计算公式可知，需要计算 $\frac{255}{B-A}$ ，在 FPGA 里面可以调用专用的 IP 或自定义一个查找表用来逼近计算结果，这里我们使用查找表方式实现。

在不考虑 $B=A$ 的情况下， $\frac{255}{B-A}$ 的取值范围为 $1 \sim 255$ ，为了提高计算精度，我们将查找表的数据表示为 $\frac{255}{B-A} \times 1024$ ，具体算法实现如下。

```

//file name:    hist_para.v
//author:      shugen.yin
//date:        2018.10.9
//function:
//log:

module hist_para(
    input clk,
    input rst_n,

    input [7:0] data_in,
    output reg [17:0] data_out
);

always @(*)
    case(data_in)
        0: data_out <= 261120; //视 B=A 等效于 A= (B+1)

```

```

1: data_out <= 261120; //255*1024 = 261120
2: data_out <= 130560;
3: data_out <= 87040;
4: data_out <= 65280;
5: data_out <= 52224;
6: data_out <= 43520;
7: data_out <= 37302;
8: data_out <= 32640;
... ..
250: data_out <= 1044;
251: data_out <= 1040;
252: data_out <= 1036;
253: data_out <= 1032;
254: data_out <= 1028;
255: data_out <= 1024;
default: ;
endcase

endmodule

```

计算完图像中像素的最大值和最小值，得到查找表系数后，我们可以根据上一节的公式计算直方图拉伸算法了，具体实现如下。

```

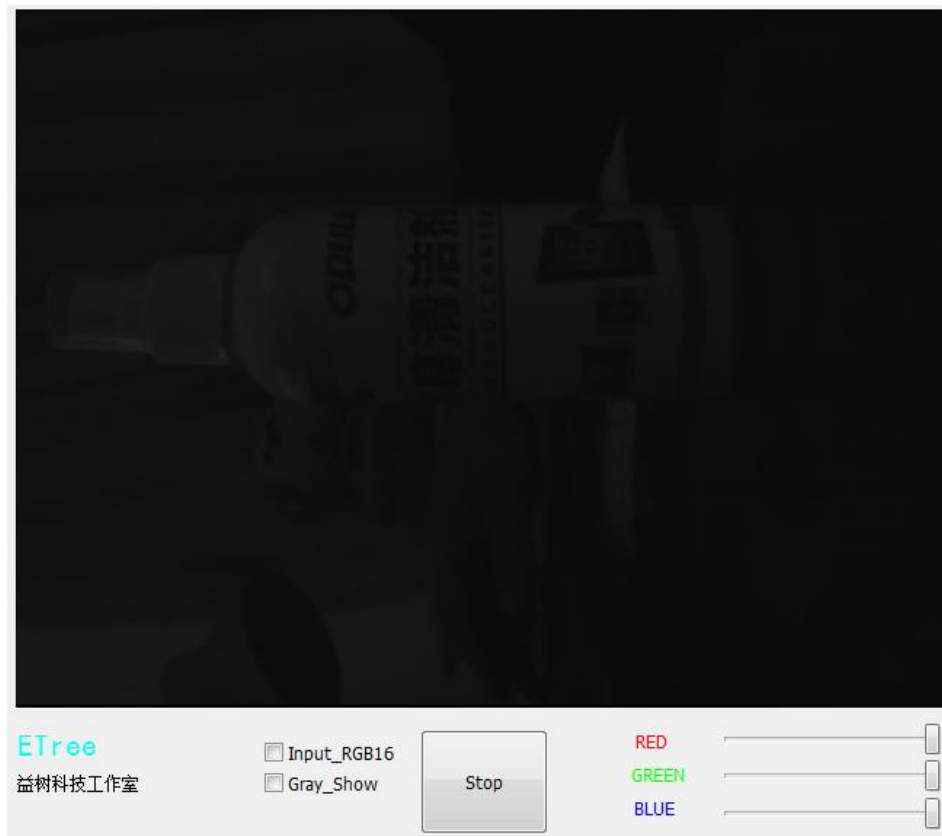
always @(posedge cmos_pclk)
    if(field==1'b0)
        cmos_data_o <= (differ_out*(cmos_data - min1))/1024;
    else
        cmos_data_o <= (differ_out*(cmos_data - min0))/1024;

```

此算法实现对光线比较昏暗的情况下效果比较好，但对椒盐噪声比较敏感。如果昏暗的图像中有比较亮的点或区域，算法效果比较差。这里可以通过计算图像的统计特性来优化算法，这里留给有兴趣的读者实现。

18.3. 实验效果

下图为没有开启直方图灰度拉伸的效果，可以看出，图像灰暗，内容不太好辨认。



下图为经过直方图灰度拉伸的效果图，可以看出图像对比度明显提高，瓶子上的文字也容易辨认。



第十九章 自动白平衡算法

本章实现 Sensor 的自动白平衡(AWB)算法,摄像头为 MT9V011,分辨率为 640×480 ,视频图像通过上位机软件显示,显示分辨率为 640×480 。

19.1. 算法原理

白平衡是随着电子影像再现真实色彩而产生的,在现在的消费类电子产品中得到广泛的应用。实现白平衡的算法有“灰度世界法”、“完美反射法”、“动态阈值法”等。这里我们介绍基于“灰度世界法”实现白平衡算法。

灰度世界法(Gray World)是以灰度世界的假设为基础的,假设一张图片具有大量的色彩变化, $R \setminus G \setminus B$ 三分量的平均值趋于同一个灰度值 K 。下面给出 K 的计算方法。

假设 R_{aver} 、 G_{aver} 、 B_{aver} 分别为红、绿、蓝三个通道的平均值,则

$$K = \frac{R_{aver} + G_{aver} + B_{aver}}{3}。$$

再计算各个通道的增益,计算方法如下。

假设 K_r 、 K_g 、 K_b 分别为红、绿、蓝三个通道的增益,则

$$K_r = \frac{K}{R_{aver}}, K_g = \frac{K}{G_{aver}}, K_b = \frac{K}{B_{aver}}。$$

最后计算每个像素中需要的红值、绿值、蓝值,计算方法如下。

假设 R 、 G 、 B 分别为原来图像中每个像素的红、绿、蓝三通道的值,假设 R_{new} 、 G_{new} 、 B_{new} 分别为需要的像素的红、绿、蓝三个通道的值,则

$$R_{new} = R \times K_r, G_{new} = G \times K_g, B_{new} = B \times K_b。$$

19.2. 算法实现

MT9V011 有 4 个色彩通道,在“RAW 转 RGB888”章节已经介绍过。我们需要计算这四个通道的平均值。由于视频数据前后的变化不是很大,为了不使用帧缓冲,我们采用上一帧计算的平均值来计算当前帧的图像算法。具体代码如下。

```
always @(posedge cmos_pclk)
    if(cmos_vsync)
        begin
            sum0 <= 0;
            sum1 <= 0;
            sum2 <= 0;
            sum3 <= 0;
        end
    else if(cmos_vsync==1'b0)
        begin
            if(cmos_href)
                case({hcnt[0],vcnt[0]})
                    2'b00:
                        begin
```

```

        sum0 <= sum0 + cmos_data;
    end
    2'b01:
    begin
        sum1 <= sum1 + cmos_data;
    end
    2'b10:
    begin
        sum2 <= sum2 + cmos_data;
    end
    2'b11:
    begin
        sum3 <= sum3 + cmos_data;
    end
    default;;
    endcase
end
else
begin
    sum0 <= sum0;
    sum1 <= sum1;
    sum2 <= sum2;
    sum3 <= sum3;
end

always @(posedge cmos_pclk)
    if((hcnt==(IMG_H-2)) && (vcnt==(IMG_V-1)))
    begin
        average0 <= sum0/76800;//76800=640*480/4
        average1 <= sum1/76800;//76800=640*480/4
        average2 <= sum2/76800;//76800=640*480/4
        average3 <= sum3/76800;//76800=640*480/4
    end
    else
    begin
        average0 <= average0;
        average1 <= average1;
        average2 <= average2;
        average3 <= average3;
    end
end

```

这里需要计算四个通道的平均值的倒数，这里我们利用查找表的方式实现。在实际查找表中存储着均值倒数的 1024 倍。

最后根据上一节的公式，可以得到白平衡后的各通道的像素值。具体代码如下。

```
wire [8:0] average;//四通道平均值
```

```

assign average = (average0+average1+average2+average3)/4;

wire [10:0] average0_reciprocal;//通道 1 倒数×1024
wire [10:0] average1_reciprocal;//通道 2 倒数×1024
wire [10:0] average2_reciprocal;//通道 3 倒数×1024
wire [10:0] average3_reciprocal;//通道 4 倒数×1024

//计算最终结果
always @(posedge cmos_pclk)
begin
    cmos_vsync_awb <= cmos_vsync;
    cmos_href_awb <= cmos_href;
    if(cmos_href)
        case({hcnt[0],vcnt[0]})
            2'b00: cmos_data_awb <=
                (((cmos_data*average*average0_reciprocal)/1024)>=255)?255:(cmos_data*
                average*average0_reciprocal)/1024; //防止溢出 255
            2'b01: cmos_data_awb <=
                (((cmos_data*average*average1_reciprocal)/1024)>=255)?255:(cmos_data*
                average*average1_reciprocal)/1024; //防止溢出 255
            2'b10: cmos_data_awb <=
                (((cmos_data*average*average2_reciprocal)/1024)>=255)?255:(cmos_data*
                average*average2_reciprocal)/1024; //防止溢出 255
            2'b11: cmos_data_awb <=
                (((cmos_data*average*average3_reciprocal)/1024)>=255)?255:(cmos_data*
                average*average3_reciprocal)/1024; //防止溢出 255
            default;;
            endcase
        else
            cmos_data_awb <= 0;
    end
end

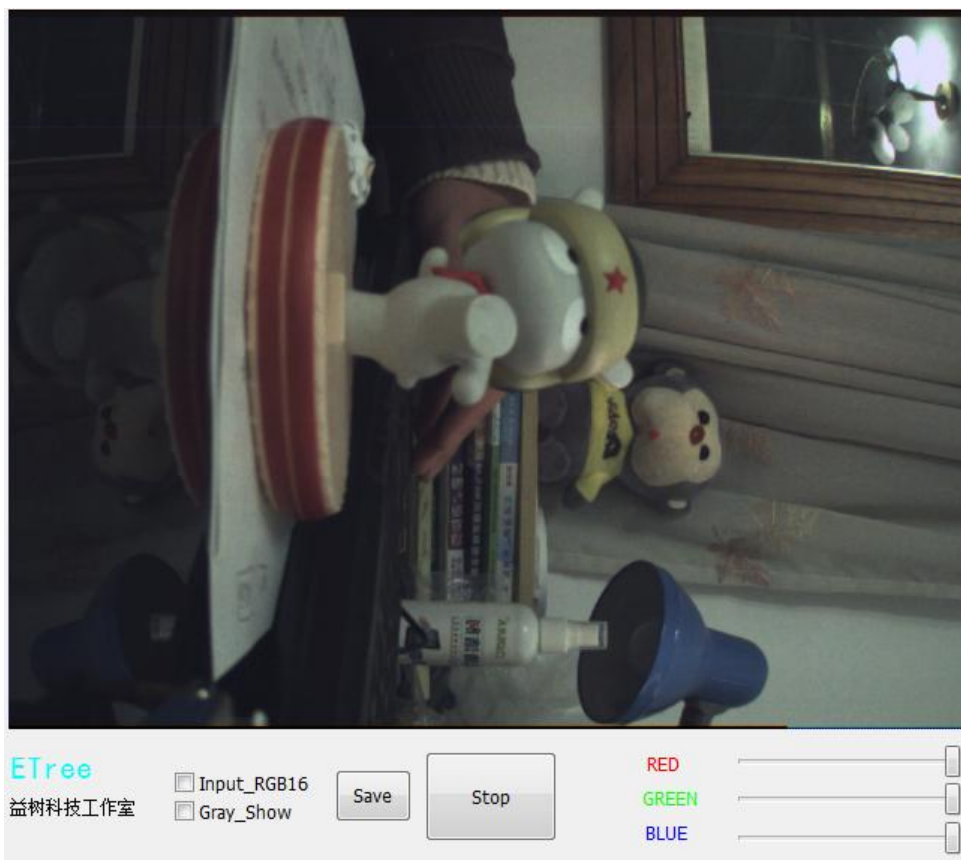
```

19.3. 实验效果

下图为白平衡前的图像效果，可以看出图片颜色偏色明显。



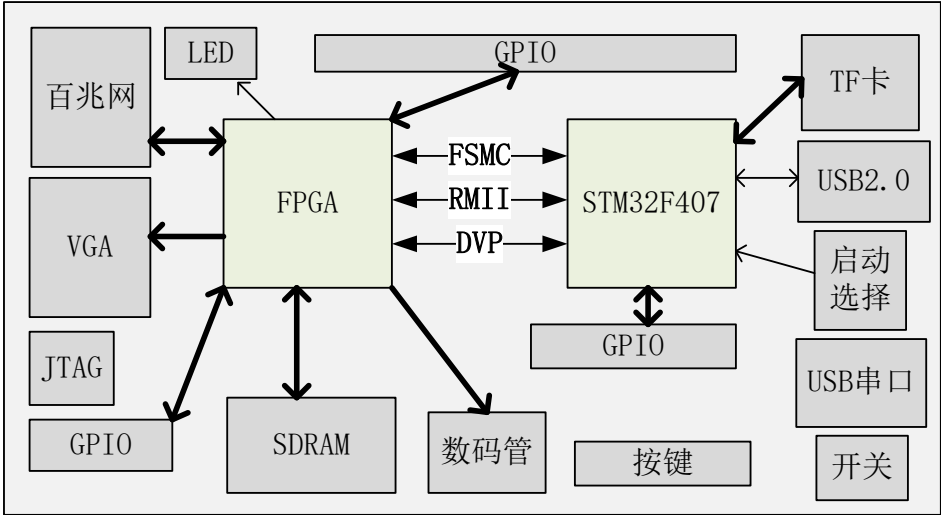
下图为白平衡后的图像效果，可以看出算法效果明显，偏色被明显消除。



STM32F4 设计篇

第一章 STM32F4 硬件平台介绍

1.1. 硬件原理框图

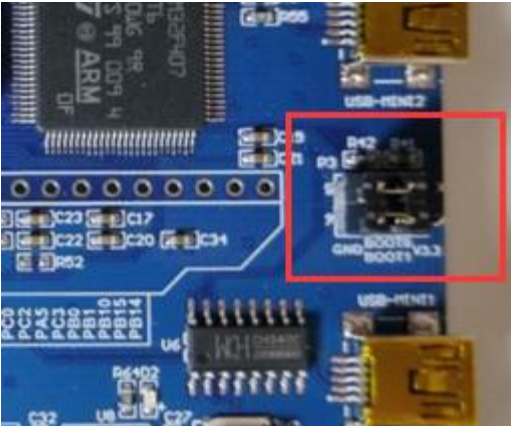


STM32 和 FPGA 之间采用并行总线 FSMC、以太网接口 RMII 和摄像头接口 DVP 连接，这样可以访问 FPGA 侧的硬件资源，包含百兆网、VGA、SDRAM、数码管、按键、LED 等。STM32 侧有 TF 卡接口、USB2.0 接口、USB 串口等。

1.2. STM32F4 启动模式

STM32 有 BOOT0 和 BOOT1 两个启动选项，在板卡的右下角的 P3 可以设置。

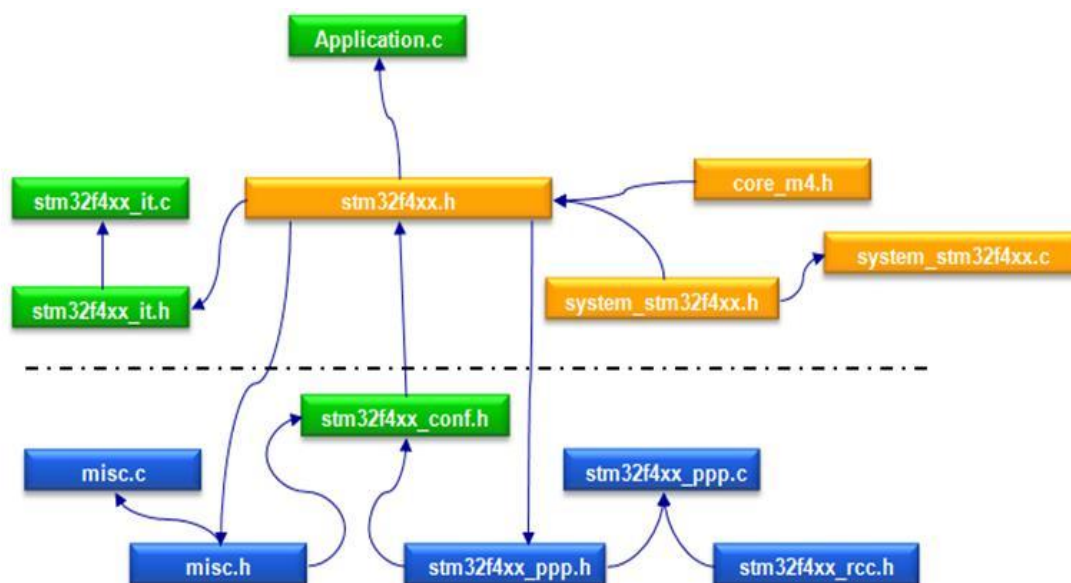
BOOT0	BOOT1	启动模式	说明
0	x	用户闪存	FLASH 启动
1	0	系统存储器	用于串口下载
1	1	SRAM 启动	用于 SRAM 中调试代码



第二章 MDK5 软件

2.1. STM32F4 官方固件库

意法半导体为了方便开发人员，提供了一套丰富的 STM32F4 固件库。ST 官方提供固件库的完整包，同时固件库也在不断完善升级中，这里我们使用 V1.4 版本的固件库，在“STM32 参考资料\STM32F4XX 固件库”目录下。下图为 STM32F4 标准外设固件库的文件关系图。

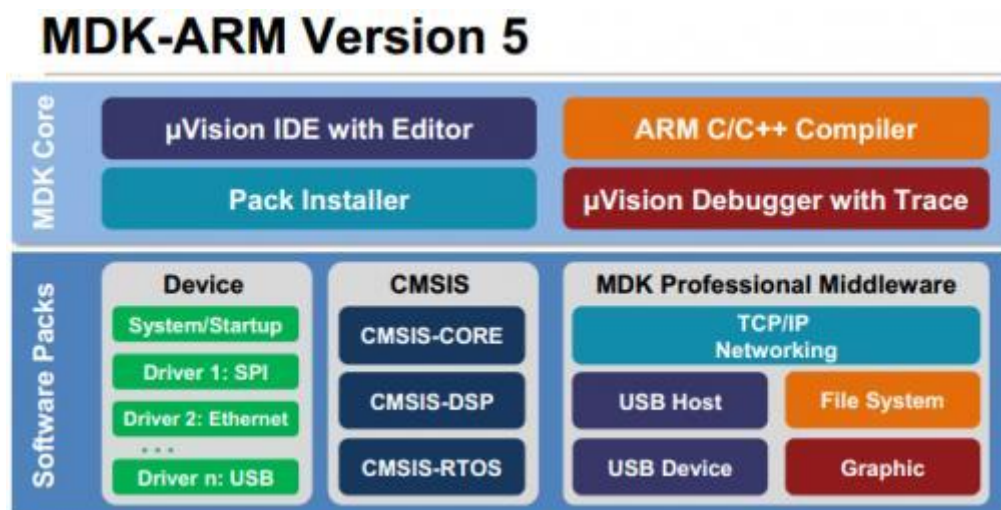


- ✧ core_cm4.h: CMSIS 核心文件，提供 M4 内核接口。
- ✧ system_stm32f4xx.h: 主要声明设置系统及总线时钟相关的函数。
- ✧ stm32f4xx.h: 主要是系统寄存器的定义声明，以及包装内存操作。
- ✧ stm32f4xx_conf.h: 外设驱动配置文件。
- ✧ misc.h: 定义中断优先级分组以及 SysTick 定时器相关的函数。
- ✧ stm32f4xx_rcc.h: 主要是一些时钟的配置和使能。
- ✧ stm32f4xx_ppp.h: 标准外设固件库，包括常用的 GPIO、ADC、USART 等。

2.2. MDK5 简介

MDK 是 RealView MDK 的简称，被全球超过 10 万的嵌入式工程师使用。MDK 是目前针对 ARM Cortex M 内核处理器的最佳开发工具。

MDK5 相对于 MDK4 和 MDK3 加强了针对 Cortex-M 控制器的支持，并且对传统的开发模式和界面做了升级，MDK5 由 MDK Core 和 Software Packs 两部分组成。



MDK5 安装包可以在：<http://www.keil.com/demo/eval/arm.htm> 下载得到。而器件支持、设备驱动、CMSIS 等组件，可以点击 MDK5 的 Build Toolbar 的最后一个图标调出 Pack Installer，来进行各种组件的安装。也可以在 <http://www.keil.com/dd2/pack> 地址下载安装。在 MDK5 安装完成后，还需要安装 STM32F4 的器件支持包：Keil.STM32F4xx_DFP.1.0.8.pack，安装文件在资料目录“tools\MDK5”下。

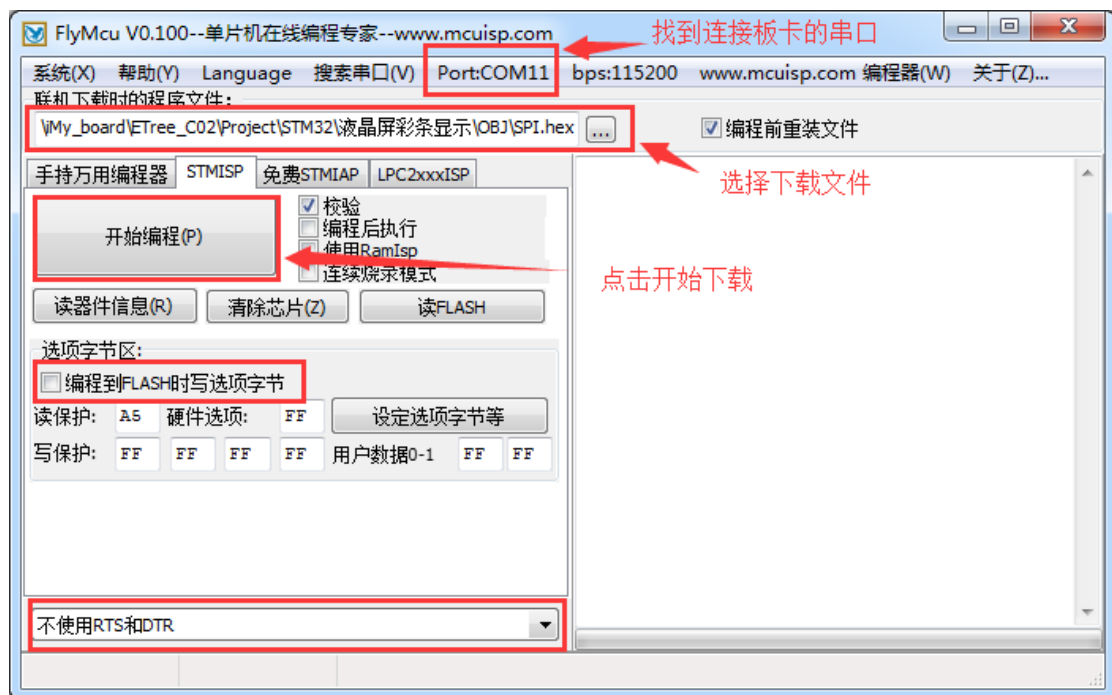
2.3. 程序下载

1) STM32 串口程序下载

STM32 一般使用串口 1 进行下载，开发板自带 USB 转串口，可以省去串口线方便程序下载。在插上串口后，电脑是无法识别这个串口芯片的，需要安装串口驱动程序，驱动安装文件在目录下。

在用串口下载之前，先要设置一下板卡上的 BOOT 启动选项，BOOT0 设置成 1，BOOT1 设置成 0。

串口下载工具在“Software\串口下载目录下”目录下，双击 FlyMcu.exe 文件。

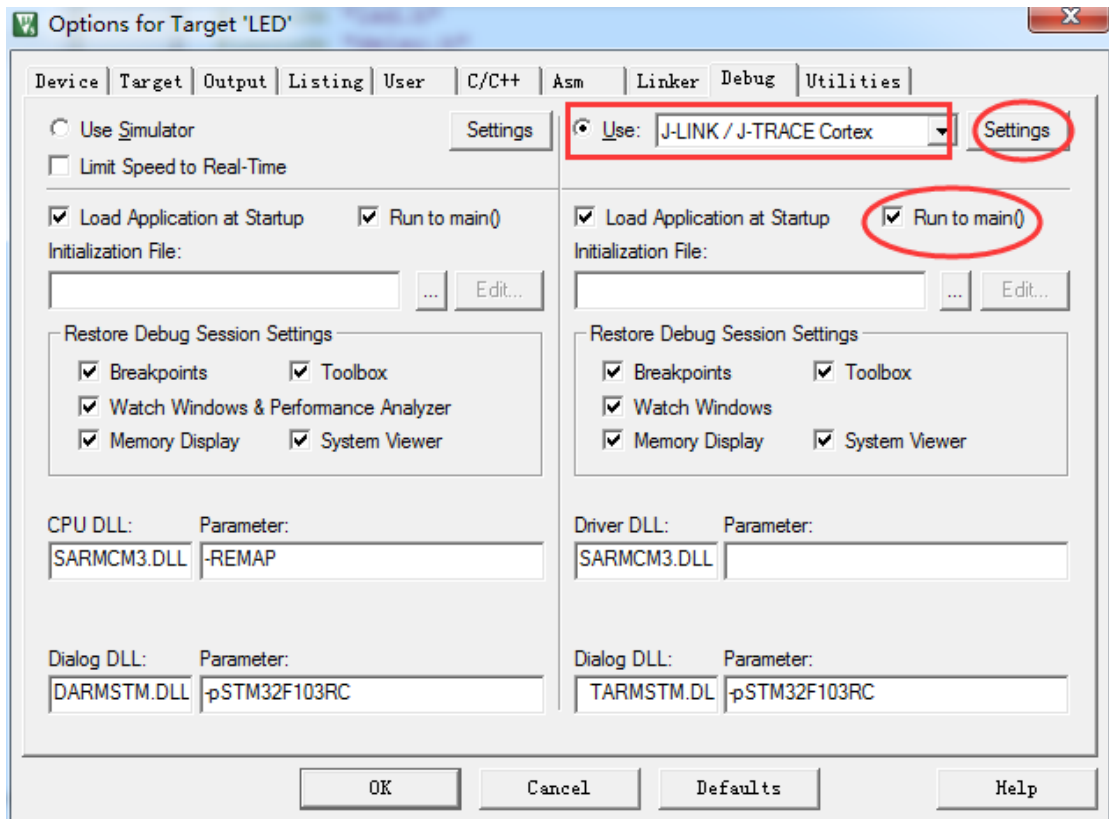


下载完成后，再将 BOOT0 设置成 0，按下复位按键或断电重启后，代码开始执行。这里推荐下面的 SWD 程序下载方式。

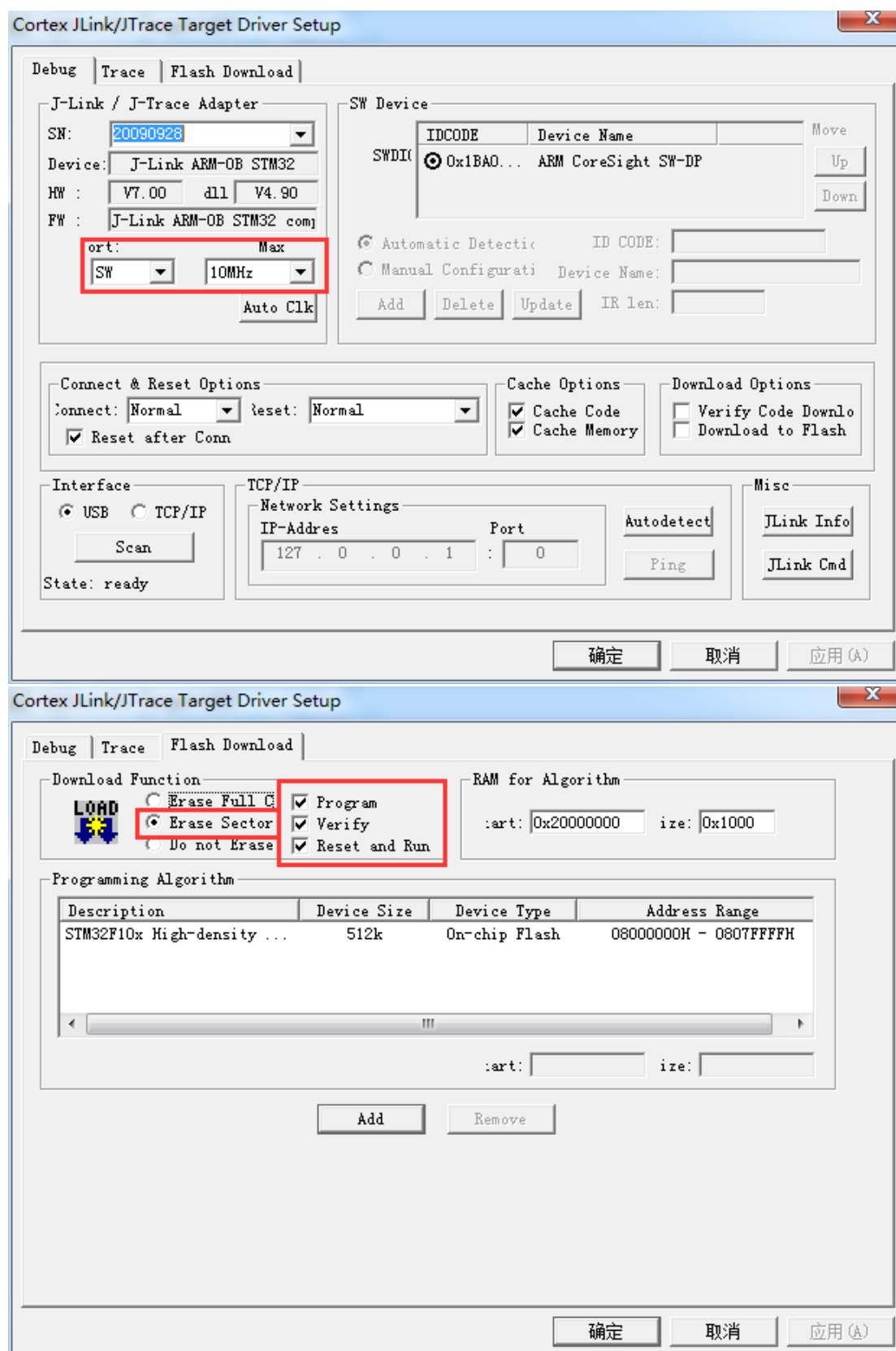
2) SWD 程序下载

在使用 SWD 下载程序前，先要安装好 Jlink 驱动。

安装好驱动后，右击工程，选择 Options for Target，在 Debug 栏选择仿真工具为 J-LINK/J-TRACE Cortex，勾选 Run to main()。



点击 Settings，设置 J-LINK 参数，如下图所示，

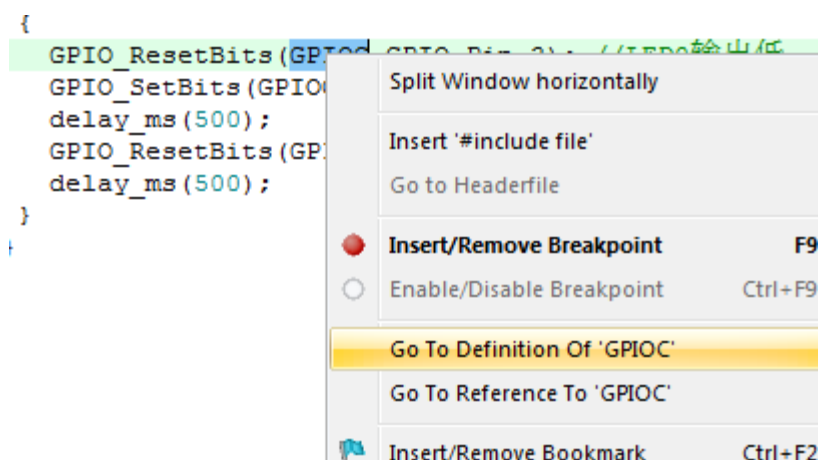


设置好 MDK5 软件后，需要把板卡右下角的 BOOT0 设置成 GND(0)。
最后点击"load"按钮便开始程序下载了。



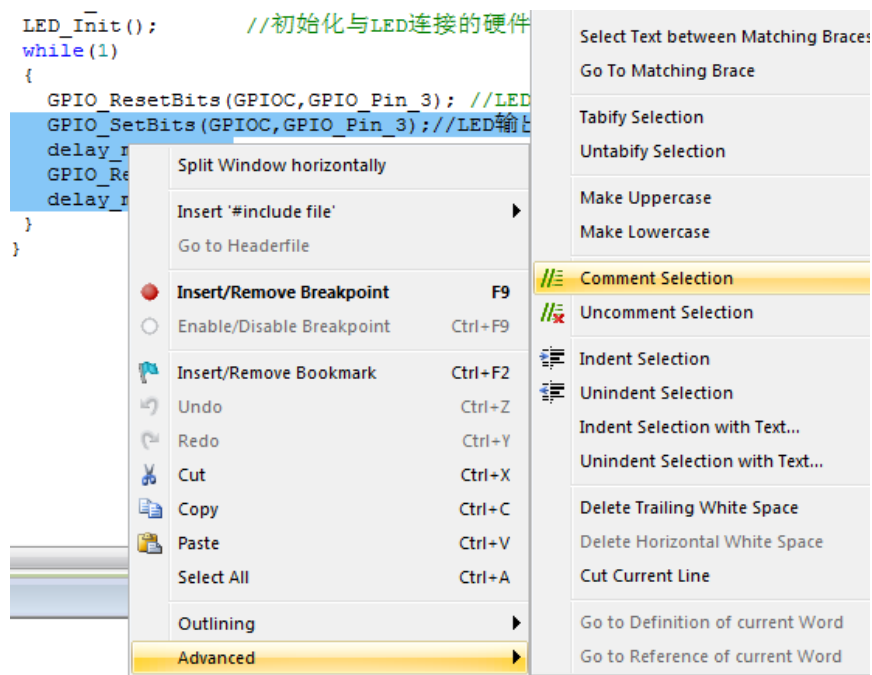
2.4. 定位函数/变量被定义的地方

大家在调试代码的时候，经常会需要查找某个函数或变量的定义，MDK 编辑器为我们提供了快速查找的方法，只要选中要查看的函数或变量，然后右击选择 Go To Definition Of 'xxxx'即可。



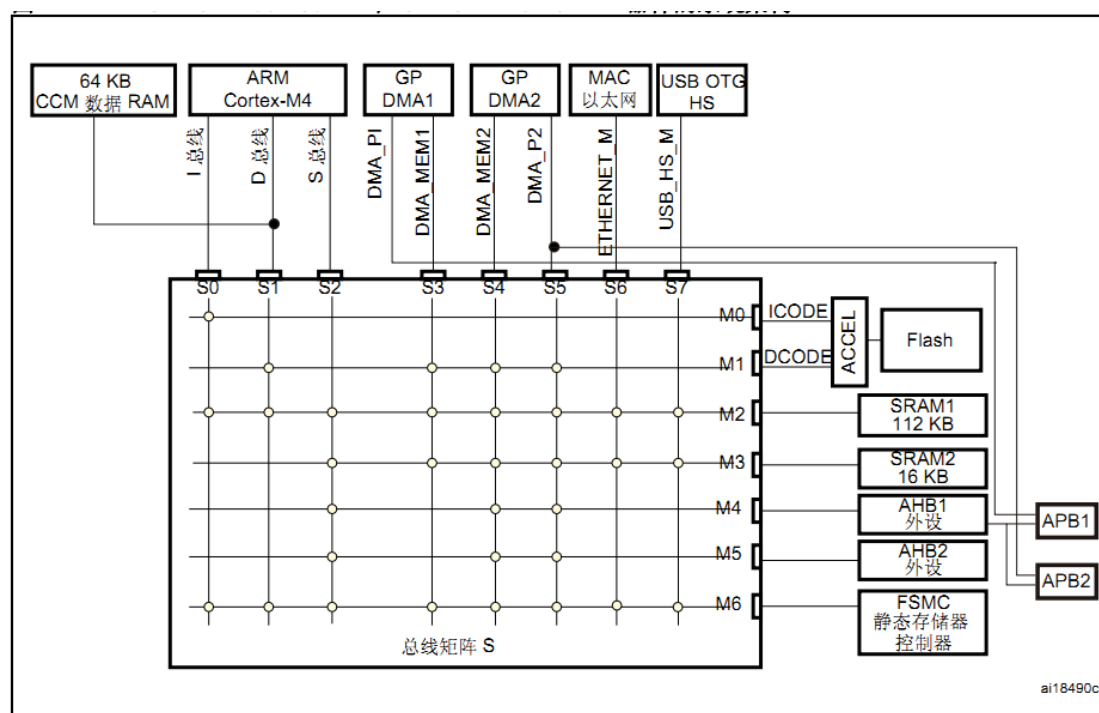
2.5. 快速注释与取消注释

我们在调试代码的时候，会需要注释一段代码或取消注释一点代码，MDK 编辑器也为我们提供了快捷的方法，选中你要注释或取消注释的代码区域，右击选择 Advanced-->Comment Selection/Uncomment Selection 即可。



第三章 STM32F4 基础知识介绍

3.1. STM32F4 总线架构



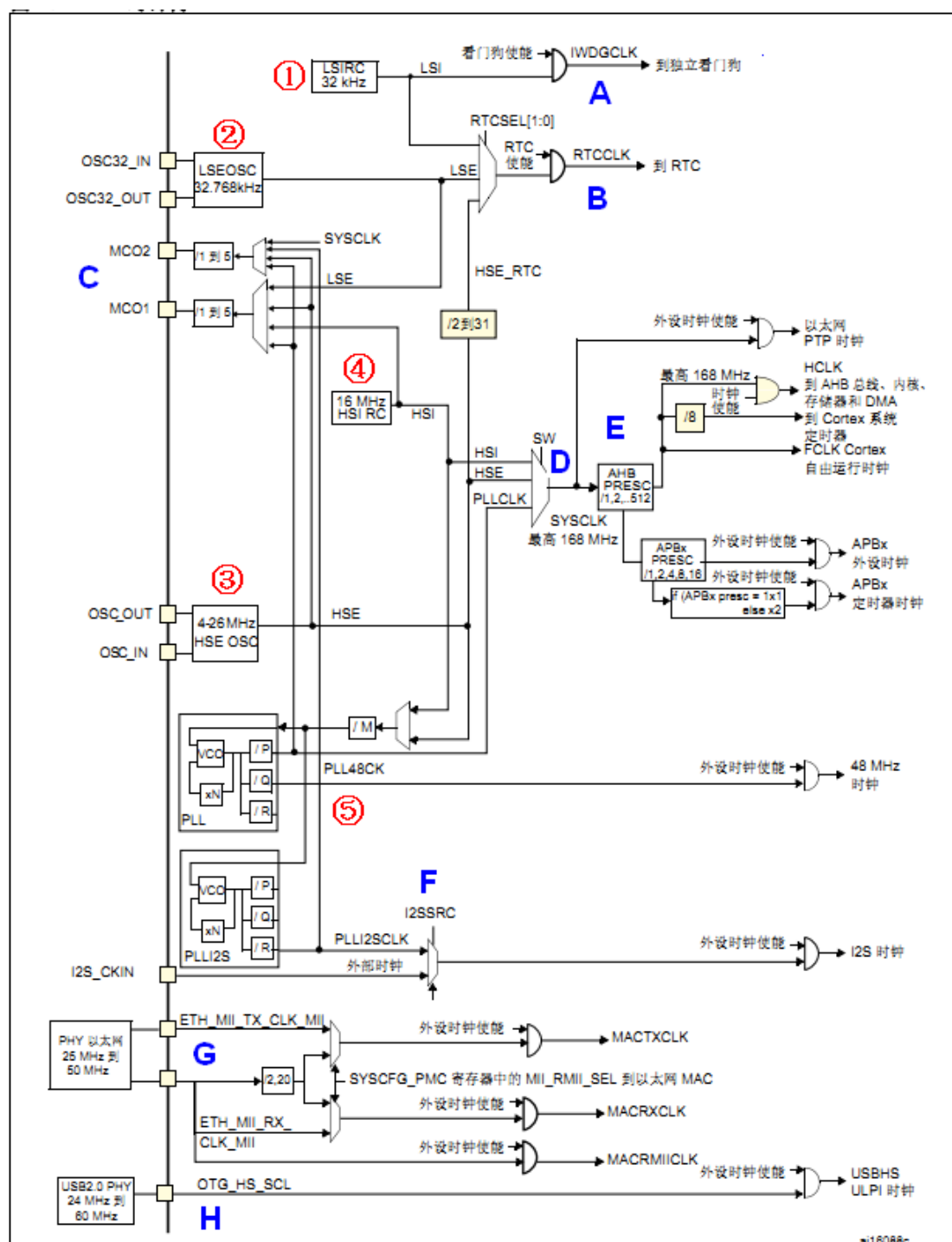
八条主控总线

- ✧ Cortex-M4 内核 I 总线, D 总线和 S 总线;
- ✧ DMA1 存储器总线, DMA2 存储器总线;
- ✧ DMA2 外设总线;
- ✧ 以太网 DMA 总线;
- ✧ USB OTG HS DMA 总线;

七条被控总线:

- ✧ 内部 FLASH ICode 总线;
- ✧ 内部 FLASH DCode 总线;
- ✧ 主要内部 SRAM1(112KB)
- ✧ 辅助内部 SRAM2(16KB);
- ✧ 辅助内部 SRAM3(64KB) (仅适用 STM32F42xx 和 STM32F43xx 系列器件);
- ✧ AHB1 外设 和 AHB2 外设;
- ✧ FSMC

3.2. STM32F4 时钟系统



在 STM32F4 中有 5 个最重要的时钟源：HSI(高速)、HSE(高速)、LSI(低速)、LSE(低速)、PLL。

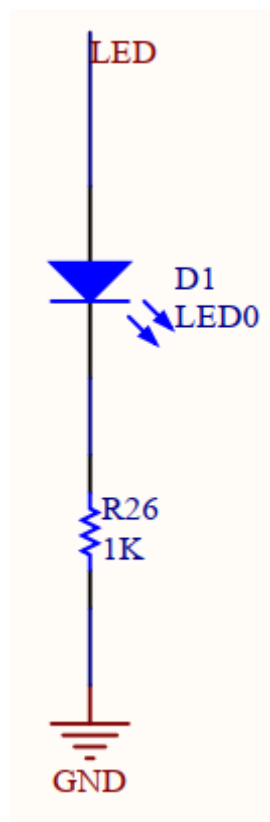
更多 STM32F4 的内容请参考《STM32F4 中文参考手册》。

第四章 LED 闪烁

本章利用 STM32F4 的 IO 控制 FPGA 端的 LED 闪烁。

4.1. 硬件设计

STM32F4 通过 PB.6 引脚连接到 FPGA 的 J15 引脚，FPGA 再将 A15 连接到 LED 引脚，这样 STM32F4 就可以控制 LED 的亮和灭。



4.2. HDL 设计

在 FPGA 中只需要一个 assign 赋值即可完成不同引脚的映射，如下所示。

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.9.9
//function:       top file of project
//log:            led light

module top_fpga(
    //global signal
    input clk, //50MHz
    input rst_n, //low active
```

```

//STM32
input stm_pb6,

//led
output led //high active
);

assign led = stm_pb6;

endmodule

```

4.3. STM32F4 IO 简介

STM32F4 的 IO 口可以由软件配置成下面 8 中模式：

- ◆ 输入悬空
- ◆ 输入上拉
- ◆ 输入下拉
- ◆ 模拟输入
- ◆ 开漏输出
- ◆ 推挽输出
- ◆ 推挽复用功能
- ◆ 开漏复用功能

在库函数中定义如下：

```

typedef enum
{
    GPIO_Mode_IN  = 0x00, /*!< GPIO Input Mode */
    GPIO_Mode_OUT = 0x01, /*!< GPIO Output Mode */
    GPIO_Mode_AF  = 0x02, /*!< GPIO Alternate function Mode */
    GPIO_Mode_AN  = 0x03 /*!< GPIO Analog Mode */
}GPIOMode_TypeDef;

```

STM32F4 有 9 组 GPIO，分别为 GPIOA~GPIOG，每个 IO 口寄存器必须按照 32 位字被访问。

4.4. 软件设计

STM32F4 的 GPIO 操作对应的库函数机器相关定义在 stm32f4xx_gpio.h 和 stm32f4xx_gpio.c 中。在使用 GPIO 前，先需要开启对应的时钟，GPIO 挂在 AHB1 总线上，下面是库函数使能对应时钟的方法：

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB, ENABLE);
```

GPIO 初始化函数：

```

/**
 * @brief  GPIO Init structure definition
 */
typedef struct

```

```

{
    uint32_t GPIO_Pin;           /*!< Specifies the GPIO pins to be
configured.

                                This parameter can be any value of @ref
GPIO_pins_define */

    GPIOMode_TypeDef GPIO_Mode;  /*!< Specifies the operating mode for
the selected pins.

                                This parameter can be a value of @ref
GPIOMode_TypeDef */

    GPIOSpeed_TypeDef GPIO_Speed; /*!< Specifies the speed for the
selected pins.

                                This parameter can be a value of @ref
GPIOSpeed_TypeDef */

    GPIOOType_TypeDef GPIO_OType; /*!< Specifies the operating output
type for the selected pins.

                                This parameter can be a value of @ref
GPIOOType_TypeDef */

    GPIOPuPd_TypeDef GPIO_PuPd;  /*!< Specifies the operating
Pull-up/Pull down for the selected pins.

                                This parameter can be a value of @ref
GPIOPuPd_TypeDef */
}GPIO_InitTypeDef;

```

下面是初始化控制 LED 的 GPIO 的代码:

```

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6; //LED
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; //100MHz
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
GPIO_Init(GPIOB, &GPIO_InitStructure);

```

在固件库中我们通常使用 `GPIO_SetBits()` 和 `GPIO_ResetBits()` 完成 GPIO 输出的高低状态。下面是控制 LED 高低的代码:

```

GPIO_SetBits(GPIOB, GPIO_Pin_6); //LED 亮
GPIO_ResetBits (GPIOB, GPIO_Pin_6); //LED 灭

```

主函数设计如下:

```

#include "sys.h"
#include "delay.h"
#include "usart.h"
#include "led.h"

int main(void)

```

```
{
    delay_init();           //延时函数初始化
    LED_Init();            //初始化 LED 硬件
    while(1)
    {
        GPIO_SetBits(GPIOB,GPIO_Pin_6); //LED 亮
        delay_ms(500);
        GPIO_ResetBits(GPIOB,GPIO_Pin_6); //LED 灭
        delay_ms(500);
    }
}
```

4.5. 实验操作

设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。实现现象为 LED 不停的闪烁。

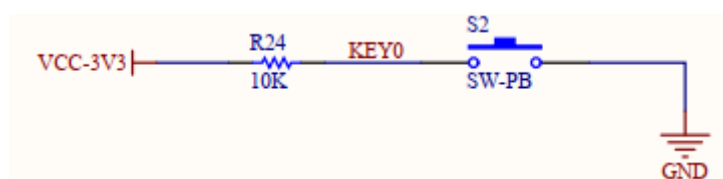
第五章 按键检测

上一章，我们是学习了如何将 STM32F4 的 IO 作为输出使用。这一章，我们学习如何将 STM32F4 的 IO 作为输入，捕获 FPGA 端的其中一个按键，每按下一次，LED 变化一次。

5.1. 硬件设计

STM32F4 通过 PB.6 引脚连接到 FPGA 的 A15 引脚，FPGA 再将 A15 连接到 LED 引脚，这样 STM32F4 就可以控制 LED 的亮和灭。

STM32F4 通过 PB.7 引脚连接到 FPGA 的 B16 引脚，FPGA 再将 B16 连接到 KEY0 引脚，这样 STM32F4 就可以捕获 KEY0 的状态。



5.2. HDL 设计

在 FPGA 中只需要一个 assign 赋值即可完成不同引脚的映射，如下所示。

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.9.9
//function:       top file of project
//log:            led light

module top_fpga(
    //global signal
    input clk, //50MHz
    input rst_n, //low active

    //key
    input key0,
    input key1,
    input key2,

    //STM32
    input stm_pa1,
    input stm_pb6,
    output stm_pb7,

    //led
    output led //high active
```

```
);

assign led = stm_pb6;
assign stm_pb7 = key0;

endmodule
```

5.3. 软件设计

在库函数中，STM32F4 通过 GPIO_ReadInputDataBit()来读取 IO 口的状态，在 key.h 中实现。

```
#define KEY0      GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7) //PB7
```

按键初始化:

```
void KEY_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA|RCC_AHB1Periph_GPIOB,
    ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7; //KEY0
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;//100M
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; GPIO_Init(GPIOB,
    &GPIO_InitStructure);
}
```

按键处理函数:

```
u8 KEY_Scan(u8 mode)
{
    static u8 key_up=1;
    if(mode)
        key_up=1;
    if(key_up&&(KEY0==0))
    {
        delay_ms(10);
        key_up=0;
        if(KEY0==0)
            return 1;
    }
    else if(KEY0==1)
        key_up=1;
    return 0;
}
```

主函数设计如下：当检测到有按键 LED 翻转。

```
#include "led.h"
#include "key.h"
```



```
int main(void)
{
    delay_init();           //延时初始化
    LED_Init();             //LED 初始化
    KEY_Init();             //按键初始化
    LED0=0;
    while(1)
    {
        if(1==KEY_Scan(0))
        {
            LED0 = !LED0;
            delay_ms(500);
        }
    }
}
```

5.4. 实验操作

设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。实现现象为：按下一次按键 LED 变亮，再按下一次 LED 变灭。

第六章 外部中断

本章通过外部中断方式实现上一章的内容。

6.1. 硬件设计

这一章用到硬件和上一章一样，这里不再介绍。

6.2. STM32F4 外部中断简介

STM32F407 有 22 个外部中断：

- ✧ EXTI 线 0~15：外部 IO 口的输入中断。
- ✧ EXTI 线 16：连接到 PVD 输出。
- ✧ EXTI 线 17：连接到 RTC 闹钟事件。
- ✧ EXTI 线 18：连接到 USB OTG FS 唤醒事件。
- ✧ EXTI 线 19：连接到以太网唤醒事件。
- ✧ EXTI 线 20：连接到 USB OTG HS(在 FS 中配置)唤醒事件。
- ✧ EXTI 线 21：连接到 RTC 入侵和时间戳事件。
- ✧ EXTI 线 22：连接到 RTC 唤醒事件。

GPIOx.0~GPIOx.15 分别对应中断线 0~15。

6.3. HDL 设计

在 FPGA 中只需要一个 assign 赋值即可完成不同引脚的映射，如下所示。

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.9.9
//function:       top file of project
//log:            led light

module top_fpga(
    //global signal
    input clk, //50MHz
    input rst_n, //low active

    //key
    input key0,
    input key1,
    input key2,

    //STM32
    input stm_pa1,
    input stm_pb6,
```

```

    output stm_pb7,

    //led
    output led //high active
);

assign led = stm_pb6;
assign stm_pb7 = key0;

endmodule

```

6.4. 软件设计

在库函数中, 通过 `SYSCFG_EXTILineConfig ()` 来配置 GPIO 与中断线之间的关系

```
SYSCFG_EXTILineConfig (EXTI_PortSourceGPIOB, EXTI_PinSource7);
```

中断初始化通过 `EXTI_Init ()` 实现:

```

NVIC_InitTypeDef  NVIC_InitStructure;
EXTI_InitTypeDef  EXTI_InitStructure;
KEY_Init ();
RCC_APB2PeriphClockCmd (RCC_APB2Periph_SYSCFG, ENABLE);

SYSCFG_EXTILineConfig (EXTI_PortSourceGPIOB, EXTI_PinSource7);

EXTI_InitStructure.EXTI_Line = EXTI_Line7;
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
EXTI_InitStructure.EXTI_LineCmd = ENABLE;
EXTI_Init (&EXTI_InitStructure);

```

设置好中断线与 GPIO 的关系后, 我们需要设置 NVIC 中断优先级

```

NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x01;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0x02;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init (&NVIC_InitStructure);

```

设置好中断优先级后, 我们需要编写中断服务函数。在 MDK 中有预定好了中断服务函数名, 关于 IO 口外部中断函数有 6 个, 分别为:

```

EXPORT EXTI0_IRQHandler
EXPORT EXTI1_IRQHandler
EXPORT EXTI2_IRQHandler
EXPORT EXTI3_IRQHandler
EXPORT EXTI4_IRQHandler
EXPORT EXTI9_5_IRQHandler
EXPORT EXTI15_10_IRQHandler

```

注意线 5~9 和线 10~15 分别共用一个中断函数。

常用的中断服务函数格式如下：

```
void EXTI1_IRQHandler(void)
{
    if(EXTI_GetITStatus(EXTI_Line2) != RESET) //判断某个线上的中断是否发生
    {
        ... ..
        EXTI_ClearITPendingBit(EXTI_Line2); //清除 LINE 上的中断标志位
    }
}
```

下面是编写好中断服务函数：

```
void EXTI9_5_IRQHandler(void)
{
    delay_ms(10); //消抖
    if(KEY0==1)
    {
        LED=!LED;
    }
    EXTI_ClearITPendingBit(EXTI_Line7); //清除 EXTI7 线路挂起位
}
```

主函数设计如下：

```
int main(void)
{
    delay_init(); //延时函数初始化
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //设置中断分组
    LED_Init(); //初始化与 LED 连接的硬件接口
    EXTIX_Init(); //外部中断初始化
    LED=1;
    while(1);
}
```

6.5. 实验操作

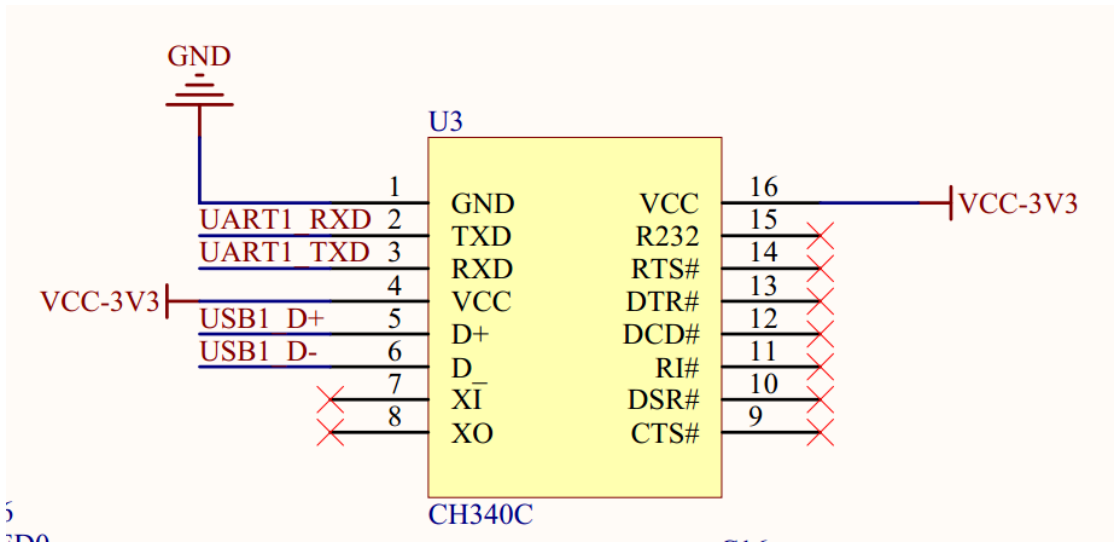
设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。实现现象为：按下一次按键 LED 变亮，再按下一次 LED 变灭。

第七章 串口通信

前面章节介绍了 IO 的使用，这一章我们开始学习 STM32F4 的串口外设，实现 STM32F4 与 PC 机之间的数据发送与接受。

7.1. 硬件设计

本设计采用 CH340C 芯片来完成 USB 转串口的工作，CH340C 具有高度集成的优点，无需外挂晶振，可节省电路板面积。



STM32 通过 PA9 和 PA10 来控制串口外设。

UART TXD	ST UART1 TX	PA8	67
UART RXD	ST UART1 RX	PA9	68
		PA10	69
		USB D-	70

7.2. STM32 串口简介

串口通信的概念非常简单，串口按位（bit）发送和接收字节，物理层只有一根通信线。串口通信最重要的参数是波特率、数据位、停止位和奇偶校验。对于两个进行通信的端口，这些参数必须匹配。

a，波特率：这是一个衡量通信速度的参数。它表示每秒钟传送的 bit 的个数。例如 300 波特表示每秒钟发送 300 个 bit。

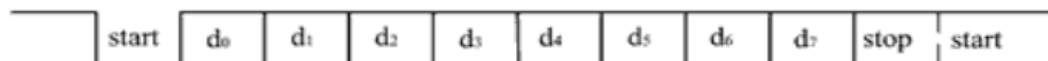
b，数据位：这是衡量通信中实际数据位的参数。典型的值为 8 位。

c，停止位：用于表示单个包的最后一位。典型的值为 1，1.5 和 2 位。由于数据是在传输线上定时的，并且每一个设备有其自己的时钟，很可能在通信中两台设备间出现了小小的不同步。因此停止位不仅仅是表示传输的结束，并且提供计算机校正时钟同步的机会。适用于停止位的位数越多，不同时钟同步的容忍程度越大，但是数据传输率同时也越慢。

d，奇偶校验位：在串口通信中一种简单的检错方式。对于偶和奇校验的情况，串口会设置校验位(数据位后面的一位)，用一个值确保传输的数据有偶个或者奇个逻辑高位。例如，如果数据是 011，那么对于偶校验，校验位为 0，保证逻辑高的位数是偶数个。如果是奇校

验，校验位为 1，这样就有 3 个逻辑高位。高位和低位不真正的检查数据，简单置位逻辑高或者逻辑低校验。这样使得接收设备能够知道一个位的状态，有机会判断是否有噪声干扰了通信或者是否传输和接收数据是否不同步。

设计采用的数据帧格式为 10 位，即 1 位起始位+8 位数据位+1 位停止位，波特率为 9600bps。注意在传输数据位时，是从低位向高位传输。



STM32F407RCT6 可提供多路串口，有分数波特率发生器、支持同步单线通信和半双工单线通讯、支持 LIN、支持调制解调器操作、智能卡协议和 IrDA SIR ENDEC 规范、具有 DMA 等。

这里我们使用串口 1 实现与 PC 的通信。

7.3. HDL 设计

由于 USB 串口同时连接到 FPGA 和 STM32F4 端，所以需要在 FPGA 端将两个串口信号都设置成输入状态，不影响 STM32F4 使用 USB 串口，具体如下所示。

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.9.9
//function:       top file of project
//log:            led light

module top_fpga(
    //global signal
    input clk, //50MHz
    input rst_n, //low active

    //key
    input key0,
    input key1,
    input key2,

    //UART
    input uart_rx,
    input uart_tx,

    //STM32
    input stm_pa1,
    input stm_pb6,
    output stm_pb7,

    //led
    output led //high active
```

```
);

assign led = stm_pb6;
assign stm_pb7 = key0;

endmodule
```

7.4. 软件设计

我们在使用 UART 前，先要使能串口时钟：

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1);
```

因为串口 1 对应引脚 PA9 和 PA10，所以我们需要使能 GPIOA 的时钟：

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA,ENABLE);
```

将串口 1 对应引脚复用映射：

```
GPIO_PinAFConfig(GPIOA,GPIO_PinSource9,GPIO_AF_USART1);
GPIO_PinAFConfig(GPIOA,GPIO_PinSource10,GPIO_AF_USART1);

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9 | GPIO_Pin_10;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF; GPIO_InitStructure.GPIO_Speed =
GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
GPIO_Init(GPIOA,&GPIO_InitStructure);
```

下面进行串口初始化：

```
USART_InitTypeDef* USART_InitStruct
USART_InitStructure.USART_BaudRate = bound; //波特率;
USART_InitStructure.USART_WordLength = USART_WordLength_8b; //字长为 8 位
数据格式
USART_InitStructure.USART_StopBits = USART_StopBits_1; //一个停止位
USART_InitStructure.USART_Parity = USART_Parity_No; //无奇偶校验位
USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None; //无硬件数据流控制
USART_InitStructure.USART_Mode = USART_Mode_Rx|USART_Mode_Tx; //收发模式
USART_Init(USART1, &USART_InitStructure); //初始化串口
```

开启串口接收中断：

```
USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
```

串口使能操作：

```
USART_Cmd(USART1, ENABLE); //使能串口
```

中断函数编写：

```
void USART1_IRQHandler(void)
{
    u8 Res;
    if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET)
    {
```

```

Res =USART_ReceiveData(USART1);

if((USART_RX_STA&0x8000)==0)
{
    if(USART_RX_STA&0x4000)
    {
        if(Res!=0x0a) USART_RX_STA=0; //0x0a 在 ASCII 码中表示“开始新一
行”

        else USART_RX_STA|=0x8000;
    }
    else
    {
        if(Res==0x0d) USART_RX_STA|=0x4000; //0x0d 在 ASCII 码中表示
“回车返回”

        else
        {
            USART_RX_BUF[USART_RX_STA&0X3FFF]=Res ;
            USART_RX_STA++;
            if(USART_RX_STA>(USART_REC_LEN-1)) USART_RX_STA=0;
        }
    }
}
}

```

主函数设计如下:

```

#include "sys.h"
#include "delay.h"
#include "usart.h"
#include "led.h"
#include "key.h"

int main(void)
{
    u8 t;
    u8 len;
    u16 times=0;
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    delay_init(168);
    uart_init(115200);
    LED_Init();
    while(1)
    {
        if(USART_RX_STA&0x8000)

```



```

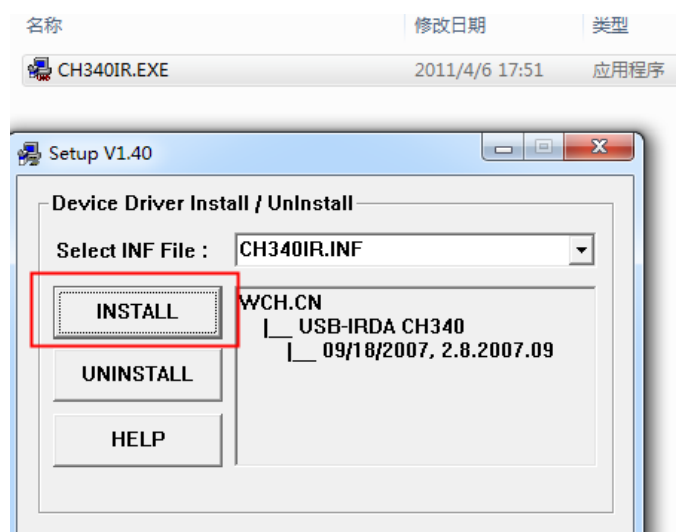
{
    len=USART_RX_STA&0x3fff;
    for(t=0;t<len;t++)
    {
        USART_SendData(USART1, USART_RX_BUF[t]);
        while(USART_GetFlagStatus(USART1,USART_FLAG_TC)!=SET);
    }
    printf("\r\n\r\n");
    USART_RX_STA=0;
}else
{
    times++;
    if(times%5000==0)
    {
        printf("\r\nETree F02\r\n");
        printf("益树科技\r\n\r\n\r\n\r\n");
    }
    if(times%200==0)
        printf("请输入数据，以回车键结束\r\n");
    if(times%30==0)
        LED=!LED;
    delay_ms(10);
}
}
}

```

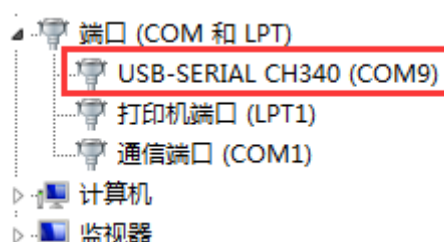
7.5. 实验操作

➤ 安装 USB 转串口驱动

双击“Tools”目录下的 CH340IR.EXE，安装设备驱动。



安装好驱动后，再插上 USB 接口，在设备管理器中可以查看 USB-SERIAL 设备。

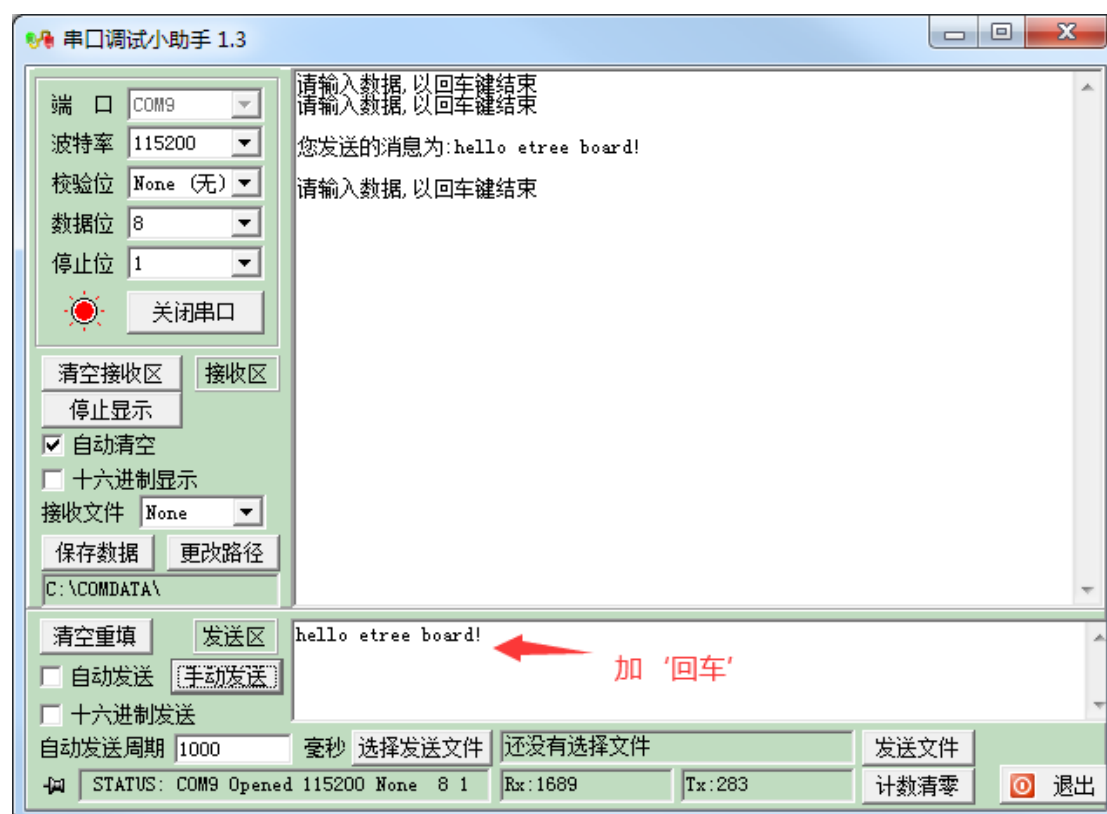


➤ 数据发送与接收

设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。

打开 Software 目录下的‘串口调试助手’，选择正确的‘端口’ (端口号可在设备管理器的端口中找到 USB-BERIAL CH340 (COMX))、波特率(这里设置成 115200)、校验位(None)、数据位(8)、停止位(1)。

点击‘打开串口’，旁边的黑点会变红。在发送栏输入“hello etree board!”并以回车键(Enter 键)结束，点击“手动发送”按钮，在接受显示区会显示 STM32F4 发送来的数据，和我们发送给 STM32F4 的一致。



第八章 定时器

本章通过通用定时器实现对 FPGA 端 LED 的控制，每 1 秒闪烁一次。

8.1. 硬件设计

这一章用到硬件和前面章一样，这里不再介绍。

8.2. STM32F4 通用定时器

STM32F4 通用定时器包含一个 16 位或 32 位自动重载计数器 CNT，该计数器由可编程预分频器 PSC 驱动。使用定时器预分频器和 RCC 时钟控制器预分频器，脉冲长度和波形周期可以精确到几个微秒到几个毫秒。STM32F4 每个通用寄存器都是独立的。

STM32F4 的通用 TIMx (TIM2~TIM5 和 TIM9~TIM14)定时器功能如下：

- ✧ 16 /32 位(仅 TIM2 和 TIM5)向上、向下、向上/向下自动装载计数器(TIMx_CNT)，注意：TIM9~TIM14 只支持向上(递增)计数方式。
- ✧ 16 位可编程(可以实时修改)预分频器(TIMx_PSC)，计数器时钟频率的分频系数为 1~65535 之间的任意数值。
- ✧ 4 个独立通道 (TIMx_CH1~4，TIM9~TIM14 最多 2 个通道)，这些通道可以用来作为：A. 输入捕获 B. 输出比较 C. PWM 生成(边缘或中间对齐模式)，注意：TIM9~TIM14 不支持中间对齐模式 D. 单脉冲模式输出
- ✧ 可使用外部信号(TIMx_ETR)控制定时器和定时器互连(可以用 1 个定时器控制另外一个定时器)的同步电路。
- ✧ 如下事件发生时产生中断/DMA (TIM9~TIM14 不支持 DMA)：
 - A. 更新：计数器向上溢出/向下溢出，计数器初始化(通过软件或者内部/外部触发)
 - B. 触发事件(计数器启动、停止、初始化或者由内部/外部触发计数)
 - C. 输入捕获
 - D. 输出比较
 - E. 支持针对定位的增量(正交)编码器和霍尔传感器电路 (TIM9~TIM14 不支持)
 - F. 触发输入作为外部时钟或者按周期的电流管理 (TIM9~TIM14 不支持)

8.3. HDL 设计

在 FPGA 中只需要一个 assign 赋值即可完成不同引脚的映射，如下所示。

```
//file name:      top_fpga.v
//author:         ETree
//date:           2017.9.9
//function:       top file of project
//log:            led light

module top_fpga(
    //global signal
```

```

input clk, //50MHz
input rst_n, //low active

//key
input key0,
input key1,
input key2,

//STM32
input stm_pa1,
input stm_pb6,
output stm_pb7,

//led
output led //high active
);

assign led = stm_pb6;
assign stm_pb7 = key0;

endmodule

```

8.4. 软件设计

这里我们使用通用定时器 3 实现定时功能。

1) 使能时钟

```
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE); //时钟使能
```

2) 初始化时钟设置

```

void TIM_TimeBaseInit(TIM_TypeDef*TIMx, TIM_TimeBaseInitTypeDef*
TIM_TimeBaseInitStruct);
//第一个参数确定使用哪个定时器，第二个参数是定时器初始化的结构体指针。
typedef struct
{
uint16_t TIM_Prescaler; //分频系数
uint16_t TIM_CounterMode; //技术方式，向上、向下或中央对齐计数
uint16_t TIM_Period; //自动装载计数周期
uint16_t TIM_ClockDivision; //时钟分频因子
uint8_t TIM_RepetitionCounter;
}TIM_TimeBaseInitTypeDef

```

下面是具体初始化代码：

```

//中断时间计算：
//Tout = (TIM_Period+1)*(TIM_Prescaler+1)/TCLK
//其中 TCLK 为 APB1 的时钟频率，这里设置为 TIM_CKD_DIV1，即 72MHz
//计算结果：

```

```
//Tout = (4999+1)*(7199+1)/72 = 500,000 us = 0.5 s
TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
TIM_TimeBaseStructure.TIM_Period = 4999;    //
TIM_TimeBaseStructure.TIM_Prescaler = 7199; //系统时钟为 72MHz
TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
TIM_TimeBaseInit(TIM3, &TIM_TimeBaseStructure);
```

3) 允许更新中断

```
void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT, FunctionalState
NewState);
//第一个参数确定使用哪个定时器, 第二个参数确定使能的定时器中断类型, 第三个参数是使
能或关闭使能;
```

下面为具体代码:

```
TIM_ITConfig(TIM3, TIM_IT_Update, ENABLE );
```

4) 设置中断优先级

中断优先级设置在前面的章节已近讲解过, 使用 `NVIC_Init` 函数实现。

5) 允许定时器工作

```
void TIM_Cmd(TIM_TypeDef* TIMx, FunctionalState NewState);
//第一个参数确定使用哪个定时器, 第二个参数是使能或关闭使能;
```

下面为具体代码:

```
TIM_Cmd(TIM3, ENABLE); //使能 TIM3 外设
```

6) 编写中断服务函数

在库函数里, 我们通过 `ITStatus TIM_GetITStatus(TIM_TypeDef* TIMx, uint16_t)` 函数读取中断状态寄存器来判断中断类型 `TIM_IT` 是否发生中断, 下面是判断定时器 3 是否发生更新(溢出)中断的代码:

```
if (TIM_GetITStatus(TIM3, TIM_IT_Update) != RESET){} //判断玩中断是否溢出
后, 还需要通过函数
TIM_ClearITPendingBit(TIM3, TIM_IT_Update); //清除中断标志位
```

完整的中断服务程序如下:

```
void TIM3_IRQHandler(void) //TIM3 中断
{
    if (TIM_GetITStatus(TIM3, TIM_IT_Update) != RESET) //检查 TIM3 更新
中断发生与否
    {
        TIM_ClearITPendingBit(TIM3, TIM_IT_Update ); //清除 TIM3 更新中断标志
        LED=!LED;
    }
}
```

8.5. 实验操作

设置好启动方式, 先下载 FPGA 的代码, 再下载 STM32 的代码。实现现象为: LED 每隔 1 秒闪烁一次。

第九章 FSMC 通信

本章通过通用 STM32F4 的 FSMC 实现与 FPGA 的读写通信,STM32F4 通过 FSMC 每隔 0.5 秒写一次循环渐进数,同时通过 FSMC 读取写下的值,并通过串口发送给 PC 显示。

9.1. 硬件设计

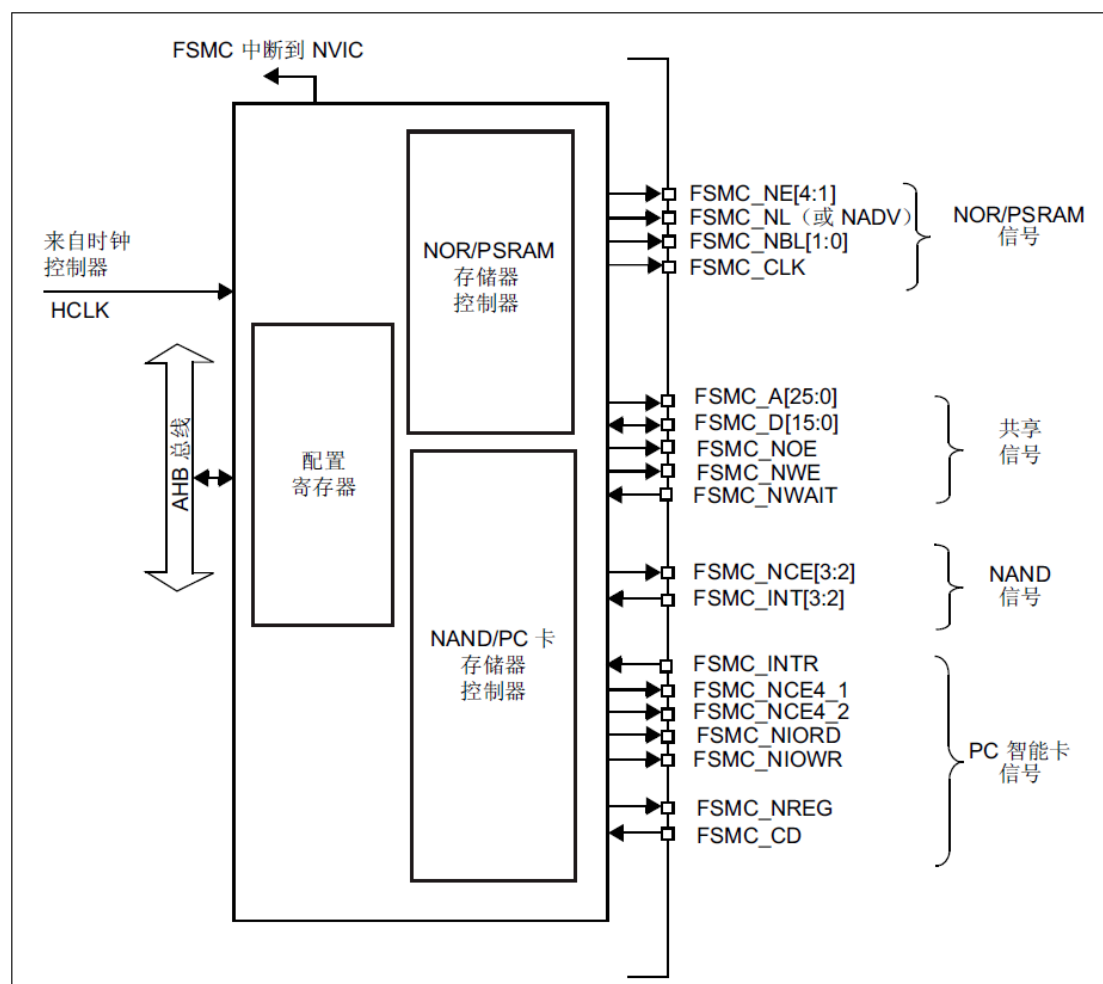
STM32F407VET6 的 FSMC 总线接口占用 PD 和 PE 大部分引脚,这些引脚都连接到 FPGA 的 IO 端,蓝色字符是 FPGA 的 IO 的引脚号,如下图所示:

L11	FSMC D2	81	PD0/FSMC_D2(CAN
D12	FSMC D3	82	PD1/FSMC_D3/(CAN
	PD2	83	PD2/UART5_RX/TIM
D14	FSMC CLK	84	PD3/FSMC_CLK/(US
G11	FSMC NOE	85	PD4/FSMC_NOE(US
C14	FSMC NWE	86	PD5/FSMC_NWE/(U
F14	FSMC NWAIT	87	PD6/FSMC_NWAIT/
D11	FSMC NE1	88	PD7/FSMC_NE1/FSM
N13	FSMC D13	55	PD8/FSMC_D13/(US
N14	FSMC D14	56	PD9/FSMC_D14/(US
M12	FSMC D15	57	PD10/FSMC_D15/(U
L13	FSMC A16	58	PD11/FSMC_A16/(U
L14	FSMC A17	59	PD12/FSMC_A17/(U
K12	FSMC A18	60	PD13/FSMC_A18/(T
J13	FSMC D0	61	PD14/FSMC_D0/(TI
J14	FSMC D1	62	PD15/FSMC_D1/(TI

R/FSMC_NBL0/PE0	97	FSMC NBL0	D15
FSMC_NBL1/PE1	98	FSMC NBL1	D16
LK/FSMC_A23/PE2	1	FSMC A23	F15
D0/FSMC_A19/PE3	2	FSMC A19	F16
D1/FSMC_A20/PE4	3	FSMC A20	G15
D2/FSMC_A21/PE5	4	FSMC A21	G16
D3/FSMC_A22/PE6	5	FSMC A22	J16
ETR/FSMC_D4/PE7	38	FSMC D4	N15
H1N/FSMC_D5/PE8	39	FSMC D5	P16
CH1/FSMC_D6/PE9	40	FSMC D6	P15
I2N/FSMC_D7/PE10	41	FSMC D7	R16
H2/FSMC_D8/PE11	42	FSMC D8	T15
I3N/FSMC_D9/PE12	43	FSMC D9	R14
I3/FSMC_D10/PE13	44	FSMC D10	T14
I4/FSMC_D11/PE14	45	FSMC D11	M11
N/FSMC_D12/PE15	46	FSMC D12	B13

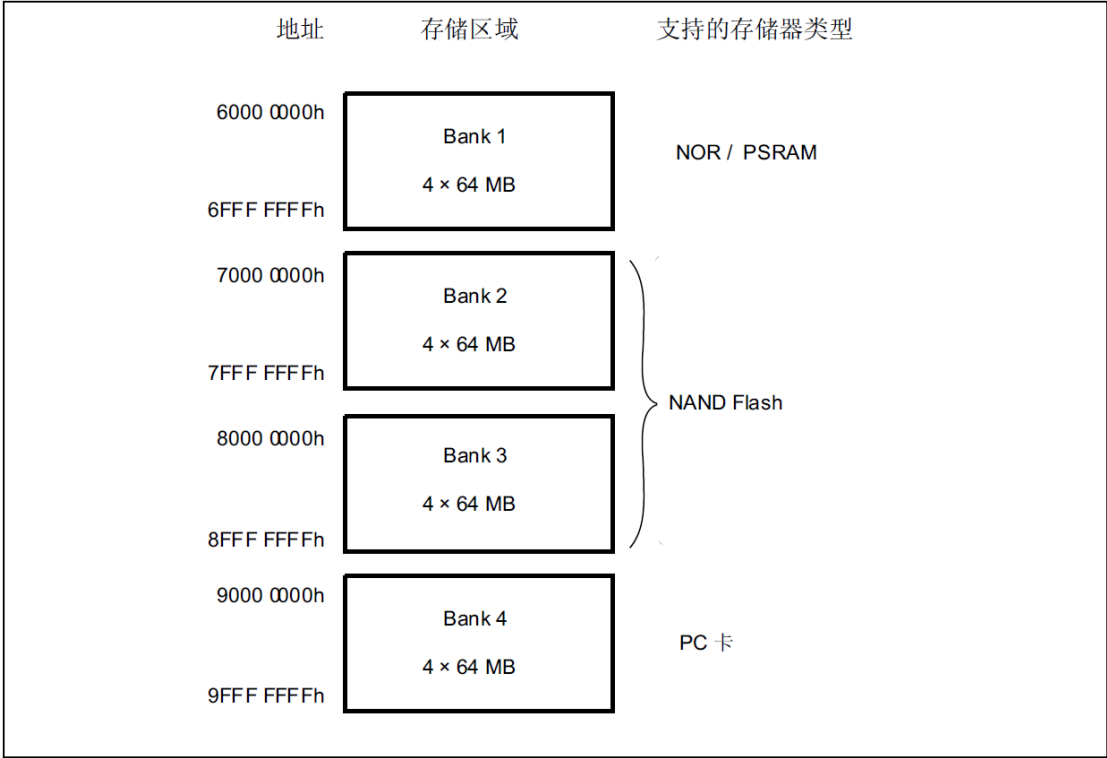
9.2. FSMC

STM32F407 系列芯片都带有 FSMC 接口，FSMC(灵活的静态存储控制器)能够与同步或异步存储器等连接，STM32F4 的 FSMC 接口支持 SRAM、NAND FLASH、NOR FLASH、PSRAM 等存储器，FSMC 的框图如下图所示：



STM32F4 的 FSMC 将外部设备分为两类：NOR/PSRAM 设备和 NAND/PC 设备，它们公用地址数据等信号，用不同的 CS 信号区分不同的设备。

FSMC 将外部存储器划分为固定的 256M 字节的四个存储区，如下图所示：



STM32F4 的 FSMC 的存储区 1(Bank 1)被分为 4 个部分，如下表所示：

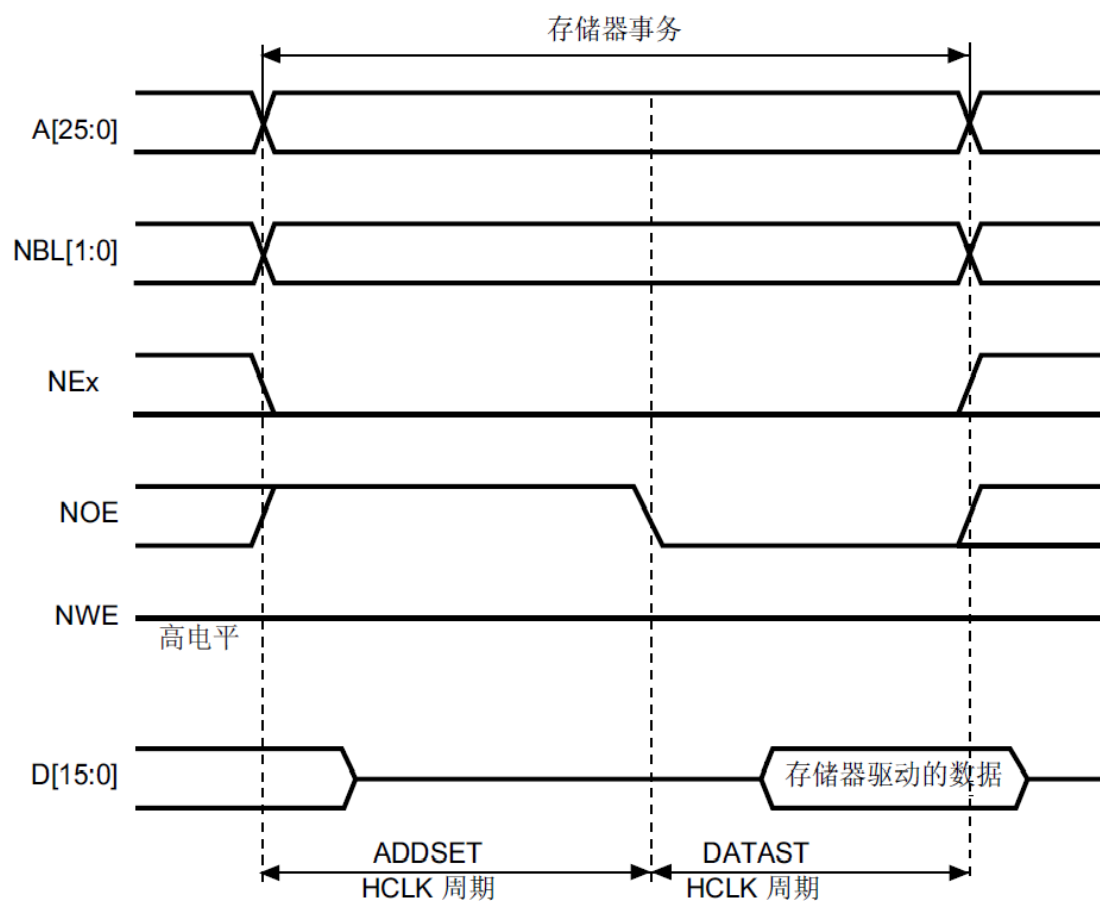
Bank1 所选区	片选信号	地址范围	HADDR	
			[27:26]	[25:0]
第 1 区	FSMC_NE1	0X6000, 0000~63FF, FFFF	00	FSMC_A[25:0]
第 2 区	FSMC_NE2	0X6400, 0000~67FF, FFFF	01	
第 3 区	FSMC_NE3	0X6800, 0000~6BFF, FFFF	10	
第 4 区	FSMC_NE4	0X6C00, 0000~6FFF, FFFF	11	

这里我们将 FSMC 作为 SRAM 接口使用，即使用存储区 1(Bank 1)。SRAM 接口包含地址信号 ADDR、数据信号 DATA、写信号 WR、读信号 RD 和片选信号 CS。

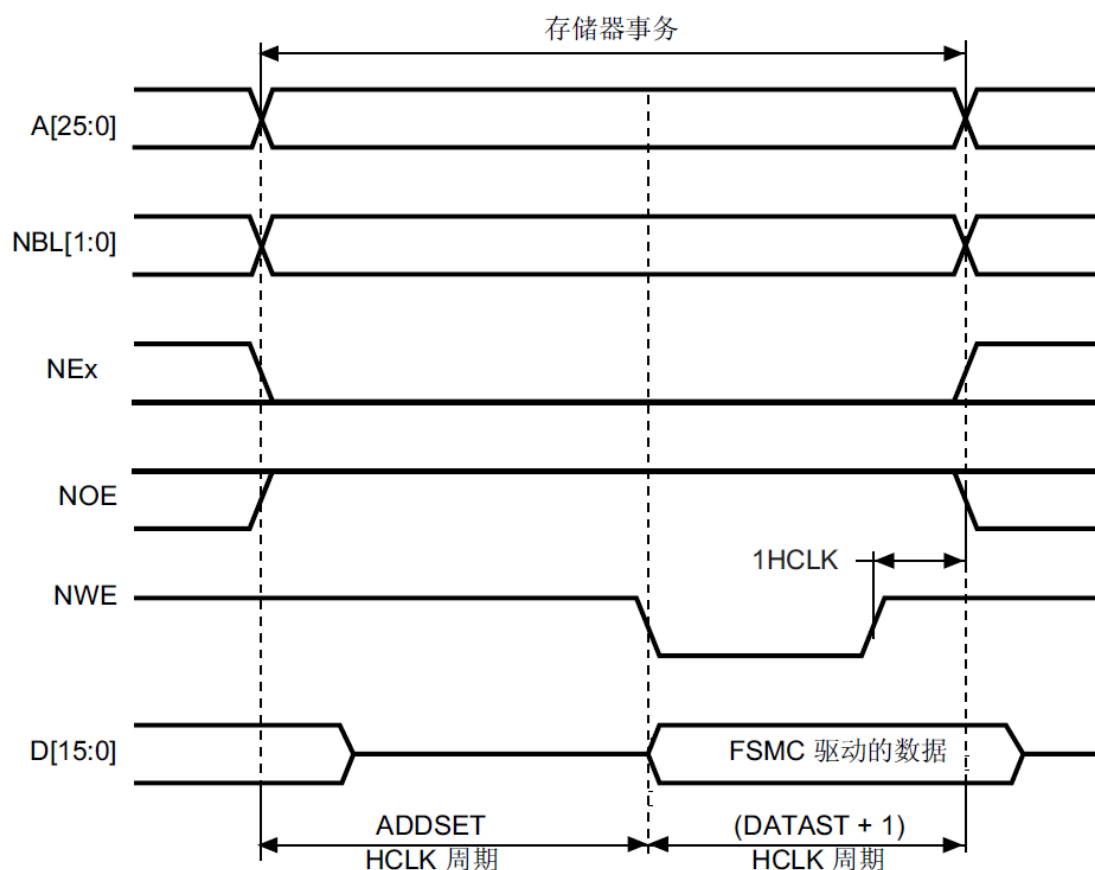
SRAM 接口为异步模式，FSMC 主要有 3 个时间参数：地址建立时间 ADDSET，数据建立时间 DATAST 和地址保持时间 ADDHLD，FSMC 有 5 种不同的异步时序模型，需要设置不同的时序参数，如下表所示：

时序模型		简单描述	时间参数
异步	Mode1	SRAM/CRAM 时序	DATAST、ADDSET
	ModeA	SRAM/CRAM OE 选通型时序	DATAST、ADDSET
	Mode2/B	NOR FLASH 时序	DATAST、ADDSET
	ModeC	NOR FLASH OE 选通型时序	DATAST、ADDSET
	ModeD	延长地址保持时间的异步时序	DATAST、ADDSET、ADDHLK

这里我们使用异步模式 A(Mode A)方式，模式 A 读时序如下图所示：



模式 A 支持独立的读写时序控制，可根据需求做实时调整。模式 A 的写时序如下图所示：



9.3. HDL 设计

我们根据上一节的 FSMC 时序图设计一款 FSMC 从设备模块，用于接收 STM32F4 发送过来的数据，同时可以根据 STM32F4 需求将数据发送回去。

具体代码设计如下：

```
//file name:      fsmc_reg.v
//author:         ETree
//date:           2020.10.4
//function:       fsmc register
//log:

module fsmc_reg(
    input clk,
    input rst_n,

    //FSMC Bus
    inout [15:0] fsmc_data,
    input fsmc_noe,
    input fsmc_nwe,
    input fsmc_ne1,
    input fsmc_a23,
    input fsmc_a22,
```

```
input fsmc_a21,
input fsmc_a20,
input fsmc_a19,
input fsmc_a18,
input fsmc_a17,
input fsmc_a16,
input fsmc_nwait,

output reg [15:0] fsmc_register0
);

reg nwe_r0;
reg nwe_r1;
reg nel_r0;
reg noe_r0;
reg noe_r1;

always @(posedge clk)
    if(~rst_n)
        begin
            nwe_r0 <= 0;
            nwe_r1 <= 0;
            nel_r0 <= 0;
            noe_r0 <= 0;
            noe_r1 <= 0;
        end
    else
        begin
            nwe_r0 <= fsmc_nwe;
            nwe_r1 <= nwe_r0;
            nel_r0 <= fsmc_nel;
            noe_r0 <= fsmc_noe;
            noe_r1 <= noe_r0;
        end
    end

wire nwe_neg;
assign nwe_neg = (nwe_r0==1'b0 && nwe_r1==1'b1);

wire noe_neg;
assign noe_neg = (noe_r0==1'b0 && noe_r1==1'b1);

parameter IDLE      = 0;
parameter FSMC_WR    = 1;
parameter FSMC_RD    = 2;
```

```
parameter END      = 3;

reg [1:0] fsmc_state;
//reg [15:0] fsmc_register0;

always @(posedge clk)
  if(~rst_n)
    begin
      fsmc_register0 <= 0;
      fsmc_state <= IDLE;
    end
  else
    begin
      case(fsmc_state)
        IDLE:
          begin
            if(nel_r0==1'b0 && nwe_neg==1'b1)
              fsmc_state <= FSMC_WR;
            else if(nel_r0==1'b0 && noe_neg==1'b1)
              fsmc_state <= FSMC_RD;
            else
              fsmc_state <= IDLE;
          end
        FSMC_WR:
          begin
            fsmc_register0 <= fsmc_data;
            fsmc_state <= END;
          end
        FSMC_RD:
          begin
            if(noe_r1==1'b1)
              fsmc_state <= END;
            else
              fsmc_state <= FSMC_RD;
          end
        END:
          begin
            fsmc_state <= IDLE;
          end
        default:
          begin
            end
          endcase
    end
```

```

end

assign fsmc_data = (fsmc_state==FSMC_RD)?fsmc_register0:'hzzzz;

endmodule

```

9.4. 软件设计

固件库提供了 3 个 FSMC 初始化函数：这里我们使用第一个函数。

```

FSMC_NORSRAMInit();
FSMC_NANDInit();
FSMC_PCCARDInit();

```

固件库对不同的存储器类型提供了不同的使能函数：这里我们使用第一个函数。

```

void FSMC_NORSRAMCmd(uint32_t FSMC_Bank, FunctionalState NewState);
void FSMC_NANDCmd(uint32_t FSMC_Bank, FunctionalState NewState);
void FSMC_PCCARDCmd(FunctionalState NewState);

```

完整的初始化代码如下，需要使能 PD、PE 时钟，使能 FSMC 时钟，声明 FSMC 接口，定义 FSMC 读写时序。

```

void FSMC_Init(void)
{
    GPIO_InitTypeDef  GPIO_InitStructure;
    FSMC_NORSRAMInitTypeDef  FSMC_NORSRAMInitStructure;
    FSMC_NORSRAMTimingInitTypeDef  readWriteTiming;
    FSMC_NORSRAMTimingInitTypeDef  writeTiming;

    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOD|RCC_AHB1Periph_GPIOE,
ENABLE);
    RCC_AHB3PeriphClockCmd(RCC_AHB3Periph_FSMC,ENABLE);

    GPIO_InitStructure.GPIO_Pin =
GPIO_Pin_0|GPIO_Pin_1|GPIO_Pin_4|GPIO_Pin_5|GPIO_Pin_7|GPIO_Pin_8
|GPIO_Pin_9|GPIO_Pin_10|GPIO_Pin_13|GPIO_Pin_14|GPIO_Pin_15;//PD0,1,4
,5,8,9,10,14,15 AF OUT
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;//100MHz
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(GPIOD, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin =
GPIO_Pin_7|GPIO_Pin_8|GPIO_Pin_9|GPIO_Pin_10|GPIO_Pin_11|GPIO_Pin_12
|GPIO_Pin_13|GPIO_Pin_14|GPIO_Pin_15;//PE7~15,AF OUT

```

```

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;//100MHz
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
GPIO_Init(GPIOE, &GPIO_InitStructure);

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11 | GPIO_Pin_12 |
GPIO_Pin_13;
GPIO_Init(GPIOD, &GPIO_InitStructure);
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2 | GPIO_Pin_3 | GPIO_Pin_4 |
GPIO_Pin_5 | GPIO_Pin_6;
GPIO_Init(GPIOE, &GPIO_InitStructure);

GPIO_PinAFConfig(GPIOD,GPIO_PinSource0,GPIO_AF_FSMC);//
GPIO_PinAFConfig(GPIOD,GPIO_PinSource1,GPIO_AF_FSMC);//
GPIO_PinAFConfig(GPIOD,GPIO_PinSource4,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource5,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource7,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource8,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource9,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource10,GPIO_AF_FSMC);

GPIO_PinAFConfig(GPIOD,GPIO_PinSource11,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource12,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource13,GPIO_AF_FSMC);

GPIO_PinAFConfig(GPIOD,GPIO_PinSource13,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource14,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOD,GPIO_PinSource15,GPIO_AF_FSMC);

GPIO_PinAFConfig(GPIOE,GPIO_PinSource7,GPIO_AF_FSMC);//PE7,AF12
GPIO_PinAFConfig(GPIOE,GPIO_PinSource8,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOE,GPIO_PinSource9,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOE,GPIO_PinSource10,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOE,GPIO_PinSource11,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOE,GPIO_PinSource12,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOE,GPIO_PinSource13,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOE,GPIO_PinSource14,GPIO_AF_FSMC);
GPIO_PinAFConfig(GPIOE,GPIO_PinSource15,GPIO_AF_FSMC);//PE15,AF12

readWriteTiming.FSMC_AddressSetupTime = 0xF; //16*HCLK
1/168M=6ns*16=96ns
readWriteTiming.FSMC_AddressHoldTime = 0x00;
readWriteTiming.FSMC_DataSetupTime = 60; //60*HCLK

```

```

=6*60=360ns

readWriteTiming.FSMC_BusTurnAroundDuration = 0x00;
readWriteTiming.FSMC_CLKDivision = 0x00;
readWriteTiming.FSMC_DataLatency = 0x00;
readWriteTiming.FSMC_AccessMode = FSMC_AccessMode_A;

writeTiming.FSMC_AddressSetupTime = 9;
writeTiming.FSMC_AddressHoldTime = 0x00;
writeTiming.FSMC_DataSetupTime = 9;
writeTiming.FSMC_BusTurnAroundDuration = 0x00;
writeTiming.FSMC_CLKDivision = 0x00;
writeTiming.FSMC_DataLatency = 0x00;
writeTiming.FSMC_AccessMode = FSMC_AccessMode_A;

FSMC_NORSRAMInitStructure.FSMC_Bank = FSMC_Bank1_NORSRAM1;
FSMC_NORSRAMInitStructure.FSMC_DataAddressMux =
FSMC_DataAddressMux_Disable;
FSMC_NORSRAMInitStructure.FSMC_MemoryType =FSMC_MemoryType_SRAM;//
FSMC_MemoryType_SRAM; //SRAM
FSMC_NORSRAMInitStructure.FSMC_MemoryDataWidth =
FSMC_MemoryDataWidth_16b;//16bit
FSMC_NORSRAMInitStructure.FSMC_BurstAccessMode =
FSMC_BurstAccessMode_Disable;// FSMC_BurstAccessMode_Disable;
FSMC_NORSRAMInitStructure.FSMC_WaitSignalPolarity =
FSMC_WaitSignalPolarity_Low;
FSMC_NORSRAMInitStructure.FSMC_AsynchronousWait =
FSMC_AsynchronousWait_Disable;
FSMC_NORSRAMInitStructure.FSMC_WrapMode = FSMC_WrapMode_Disable;
FSMC_NORSRAMInitStructure.FSMC_WaitSignalActive =
FSMC_WaitSignalActive_BeforeWaitState;
FSMC_NORSRAMInitStructure.FSMC_WriteOperation =
FSMC_WriteOperation_Enable;
FSMC_NORSRAMInitStructure.FSMC_WaitSignal = FSMC_WaitSignal_Disable;
FSMC_NORSRAMInitStructure.FSMC_ExtendedMode =
FSMC_ExtendedMode_Enable;
FSMC_NORSRAMInitStructure.FSMC_WriteBurst = FSMC_WriteBurst_Disable;
FSMC_NORSRAMInitStructure.FSMC_ReadWriteTimingStruct =
&readWriteTiming;
FSMC_NORSRAMInitStructure.FSMC_WriteTimingStruct = &writeTiming;

FSMC_NORSRAMInit(&FSMC_NORSRAMInitStructure);

FSMC_NORSRAMCmd(FSMC_Bank1_NORSRAM1, ENABLE);

```

```
delay_ms(50); // delay 50 ms
}
```

FPGA 寄存器定义，具体代码如下：在 fsmc.h 头文件中。

```
#define FPGA_REG0_BASE      ((u32) (0x60000000))
#define FPGA_REG1_BASE      ((u32) (0x60020000))
#define FPGA_REG2_BASE      ((u32) (0x60040000))
#define FPGA_REG3_BASE      ((u32) (0x60060000))
#define FPGA_REG4_BASE      ((u32) (0x60080000))
#define FPGA_REG5_BASE      ((u32) (0x600A0000))
#define FPGA_REG6_BASE      ((u32) (0x600C0000))
#define FPGA_REG7_BASE      ((u32) (0x600E0000))

#define FPGA_REG0            *(vu16 *)FPGA_REG0_BASE
#define FPGA_REG1            *(vu16 *)FPGA_REG1_BASE
```

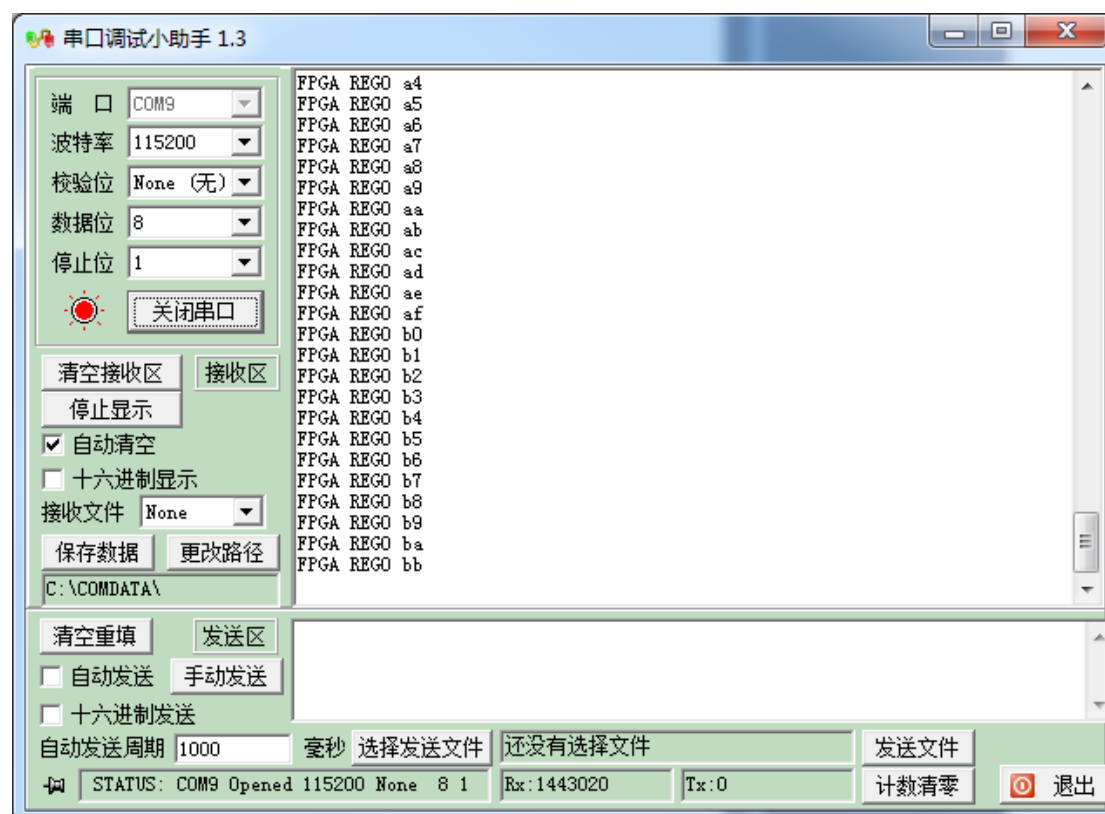
FPGA 寄存器读写函数定义，具体代码如下：在 fsmc.c 文件中。

```
u16 FPGA_RD_REG0(void)
{
    vu16 ram;
    ram = FPGA_REG0;
    return ram;
}

void FPGA_WR_REG0(vu16 data)
{
    data=data;
    FPGA_REG0 = data;
}
```

9.5. 实验操作

设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。可以看到串口调试助手不停的接受到 STM32F4 通过 FSMC 读取的寄存器值，如下图所示：



同时在 FPGA 板卡端也可以看到数码管不停的显示和串口调试助手显示的一样的值。

第十章 VGA 字符显示

在之前的章节有通过 FPGA 直接实现 VGA 彩条显示，本章通过通用 STM32F4 的 FSMC 接口控制 FPGA 端的 VGA 接口实现 VGA 字符显示功能，可用于信息的打印显示或界面设计。

10.1. 硬件设计

STM32F407VET6 的 FSMC 总线接口占用 PD 和 PE 大部分引脚，这些引脚都连接到 FPGA 的 IO 端，蓝色字符是 FPGA 的 IO 的引脚号，如下图所示：

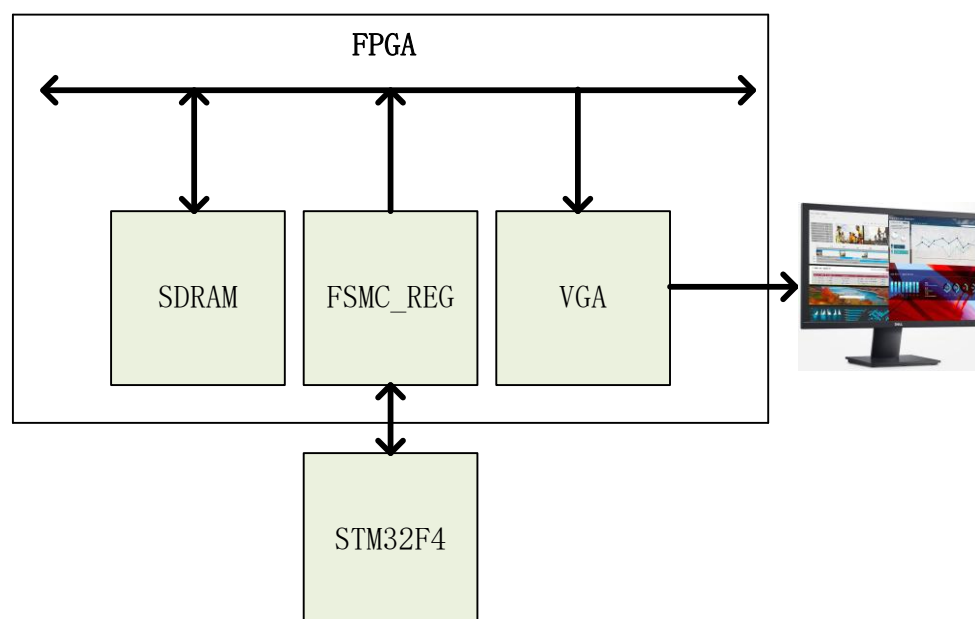
L11	FSMC D2	81	PD0/FSMC_D2(CAN
D12	FSMC D3	82	PD1/FSMC_D3/(CAN
	PD2	83	PD2/UART5_RX/TIM
D14	FSMC CLK	84	PD3/FSMC_CLK/(US
G11	FSMC NOE	85	PD4/FSMC_NOE(US
C14	FSMC NWE	86	PD5/FSMC_NWE/(U
F14	FSMC NWAIT	87	PD6/FSMC_NWAIT/
D11	FSMC NE1	88	PD7/FSMC_NE1/FSM
N13	FSMC D13	55	PD8/FSMC_D13/(US
N14	FSMC D14	56	PD9/FSMC_D14/(US
M12	FSMC D15	57	PD10/FSMC_D15/(U
L13	FSMC A16	58	PD11/FSMC_A16/(U
L14	FSMC A17	59	PD12/FSMC_A17/(U
K12	FSMC A18	60	PD13/FSMC_A18/(T
J13	FSMC D0	61	PD14/FSMC_D0/(TI
J14	FSMC D1	62	PD15/FSMC_D1/(TI

R/FSMC_NBL0/PE0	97	FSMC NBL0	D15
FSMC_NBL1/PE1	98	FSMC NBL1	D16
LK/FSMC_A23/PE2	1	FSMC A23	F15
D0/FSMC_A19/PE3	2	FSMC A19	F16
D1/FSMC_A20/PE4	3	FSMC A20	G15
D2/FSMC_A21/PE5	4	FSMC A21	G16
D3/FSMC_A22/PE6	5	FSMC A22	J16
ETR/FSMC_D4/PE7	38	FSMC D4	N15
H1N/FSMC_D5/PE8	39	FSMC D5	P16
CH1/FSMC_D6/PE9	40	FSMC D6	P15
J2N/FSMC_D7/PE10	41	FSMC D7	R16
H2/FSMC_D8/PE11	42	FSMC D8	T15
J3N/FSMC_D9/PE12	43	FSMC D9	R14
H3/FSMC_D10/PE13	44	FSMC D10	T14
H4/FSMC_D11/PE14	45	FSMC D11	M11
N/FSMC_D12/PE15	46	FSMC D12	B13

FPGA 部分需要用到 SDRAM 和 VGA 接口，在前面章节已经介绍过，这里不再重复。

10.2. HDL 设计

代码部分可以参考“MT9V011 摄像头显示”章节，这里不再介绍。



如上图所示：STM32F4 通过 FSMC 接口发送视频数据给 FPGA 的 fsmc_reg 模块，然后 fsmc_reg 将数据发送给 sdram 模块，最后 VGA 模块将视频数据从 SDRAM 读出到 VGA 显示器显示，具体代码这里不再贴出，可以查看工程源码。

10.3. 软件设计

STM32F4 通过 FSMC 将数据写入 SDRAM，需要先写入地址，然后写入数据，具体代码如下，opt_delay() 用于适当延时：

```
void SDRAM_WriteWord(u16 Buffer, u32 WriteAddr)
{
    int i=0;
    FPGA_WR_REG1(WriteAddr & 0x0000ffff);
    FPGA_WR_REG2((WriteAddr & 0xffff0000)>>16);
    FPGA_WR_REG3(Buffer);
    opt_delay(36);
}
```

默认 VGA 的分辨率为 640*480，我们根据这个分辨率设计了如下的函数，如下所示：

```
void VGA_Clear(u16 Color); //背景刷新
void VGA_ColorBar(void); //彩条显示
void VGA_DrawPoint(u16 x,u16 y, u16 color); //画点
void VGA_DrawLine(u16 x1, u16 y1, u16 x2, u16 y2); //画线
void VGA_DrawRectangle(u16 x1, u16 y1, u16 x2, u16 y2); //画框
void VGA_Draw_Circle(u16 x0,u16 y0,u8 r); //画圆
void VGA_ShowChar(u16 x,u16 y,u8 num,u8 size,u8 mode); //显示字符
```

```
void VGA_ShowString(u16 x,u16 y,u16 width,u16 height,u8 size,u8 *p); //
显示字符串
void VGA_ShowNum(u16 x,u16 y,u32 num,u8 len,u8 size); //显示数字
```

软件的主函数如下所示:

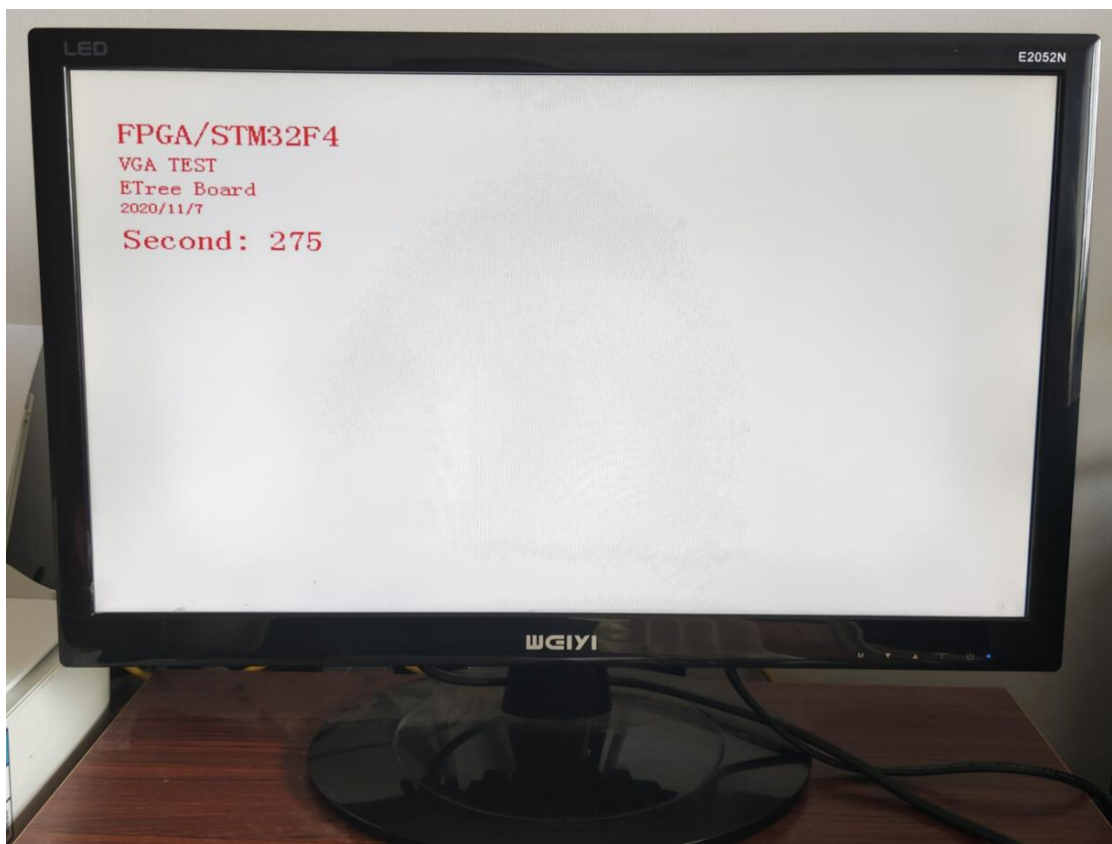
```
#include "sys.h"
#include "delay.h"
#include "usart.h"
#include "led.h"
#include "fsmc.h"
#include "vga.h"

int main(void)
{
    u32 num=0;
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);/
    delay_init(168);
    uart_init(115200);
    LED_Init();
    FSMC_Init();
    VGA_ColorBar();
    delay_ms(2000);
    VGA_Clear(WHITE);

    VGA_ShowString(30,40,200,24,24,"FPGA/STM32F4");
    VGA_ShowString(30,70,200,16,16,"VGA TEST");
    VGA_ShowString(30,90,200,16,16,"ETree Board");
    VGA_ShowString(30,110,200,12,12,"2020/11/7");
    VGA_ShowString(30,130,200,24,24,"Second:");
    while(1)
    {
        VGA_ShowNum(114,130,num++,4,24);
        delay_ms(1000);
    }
    return 0;
}
```

10.4. 实验操作

设置好启动方式,先下载 FPGA 代码,然后下载 STM32F4 的代码。可以看到在 VGA 显示器上,先有一秒的彩条显示,分别为红、绿、蓝和黄,然后显示板卡信息和秒计数器,如下图所示。

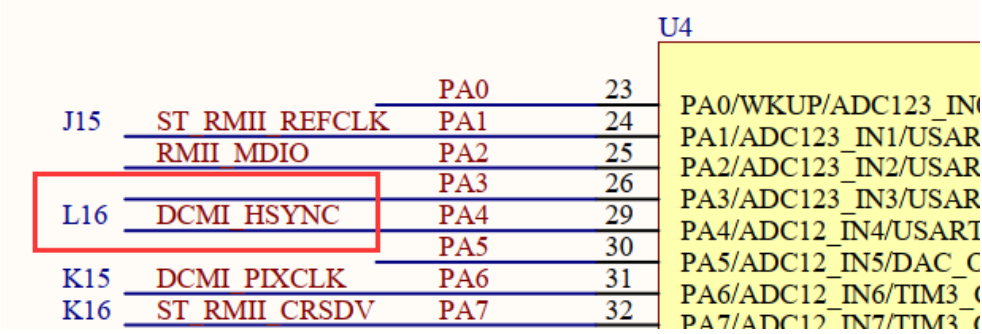


第十一章 ADC

上一章实现了 VGA 字符显示，本章通过通用 STM32F4 的 ADC 测量 FPGA 的 IO 电压，并将信息显示在 VGA 显示器上。

11.1. 硬件设计

STM32F4 的 PA4 连接到了 FPGA 的 L16 引脚，如下图所示：



我们通过 STM32F4 的 PA4 引脚来测量 FPGA 的 IO 电压值。

11.2. STM32F4 ADC

STM32F4 系列一般有 3 个 ADC，可以独立使用，也可以使用双重/三重模式。STM32F4 的 ADC 是 12 位逐次逼近型的模拟数字转换器，有 19 个通道，可测量 16 个外部源、2 个内部源和 VBAT 通道的信号。这些 ADC 可以单次、连续、扫描或间断式执行。ADC 的结果可以左对齐或右对齐方式存储。ADC 的最大转换速率为 2.4MHz，在 ADCCLK 等于 36M，采样周期为 3 个 ADC 时钟下得到。

STM32F4 的 ADC 转换分为 2 个通道组：规则通道组(正常运行)和注入通道组(中断方式)。STM32F4 的 ADC 可以进行很多种转换模式，在《STM32F4xx 中文参考手册》第 11 章有详细介绍。

下面我们来介绍使用库函数设置 ADC1 的通道 4 来进行 ADC 转换的步骤，使用到的库函数定义在 stm32f4xx_adc.c 和 stm32f4xx_adc.h 文件中。

1) 开启 PA 端口时钟和 ADC1 时钟，设置 PA4 为模拟输入。

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA, ENABLE); //使能 GPIOA 时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE); //使能 ADC1 时钟
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4; //PA4 通道 4
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AN; //模拟输入
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL; //不带上下拉
GPIO_Init(GPIOA, &GPIO_InitStructure);
```

下表为 ADC1~ADC3 的引脚与通道的对应关系：

通道号	ADC1	ADC2	ADC3
通道 0	PA0	PA0	PA0
通道 1	PA1	PA1	PA1
通道 2	PA2	PA2	PA2
通道 3	PA3	PA3	PA3
通道 4	PA4	PA4	PF6
通道 5	PA5	PA5	PF7
通道 6	PA6	PA6	PF8
通道 7	PA7	PA7	PF9
通道 8	PB0	PB0	PF10
通道 9	PB1	PB1	PF3
通道 10	PC0	PC0	PC0
通道 11	PC1	PC1	PC1
通道 12	PC2	PC2	PC2
通道 13	PC3	PC3	PC3
通道 14	PC4	PC4	PF4

2) 设置 ADC 通用控制寄存器 CCR、输入时钟分频、模式等。

在库函数中，初始化 CCR 寄存器是通过调用 ADC_CommonInit 来实现的：

```
ADC_CommonInitStructure.ADC_Mode = ADC_Mode_Independent;//独立模式
ADC_CommonInitStructure.ADC_TwoSamplingDelay=ADC_TwoSamplingDelay_5Cycles;
//两个采样之间的延迟周期数
ADC_CommonInitStructure.ADC_DMAAccessMode=ADC_DMAAccessMode_Disabled;
//禁用 DMA
ADC_CommonInitStructure.ADC_Prescaler=ADC_Prescaler_Div4;//4 分频，保证 ADC1 时钟不超过 36MHz
ADC_CommonInit(&ADC_CommonInitStructure);//初始化
```

3) 初始化 ADC1 参数，设置转换分辨率、转换方式、对齐方式、规则序列等信息。

```
ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b;//12 位模式
ADC_InitStructure.ADC_ScanConvMode = DISABLE;//非扫描模式
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;//关闭连续转换
ADC_InitStructure.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;
//禁止触发检测，使用软件触发
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;//右对齐
ADC_InitStructure.ADC_NbrOfConversion = 1;//1 个转换在规则序列中
ADC_Init(ADC1, &ADC_InitStructure);//ADC 初始化
```

4) 开启 ADC。

```
ADC_Cmd(ADC1, ENABLE);//开启 AD 转换器
```

5) 读取 ADC 值

```
ADC-RegularChannelConfig(ADC1, ADC_Channel_4, 1, ADC_SampleTime_480Cycles);
ADC_SoftwareStartConvCmd(ADC1);//使能指定的 ADC1 的软件转换启动功能
while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));//等待转换结束
ADC_GetConversionValue(ADC1);//最近一次 ADC1 规则组的转换结果
```


通过以上几个步骤就可以正常使用 STM32F4 的 ADC1 来测量电压信号了。

11.3. HDL 设计

本章的 HDL 设计和上一章一样这里不再介绍。

11.4. 软件设计

STM32F4 初始化 ADC1 的通道 4，实时测试 FPGA 的 IO 引脚状态，主函数如下所示：

```
#include "sys.h"
#include "delay.h"
#include "usart.h"
#include "led.h"
#include "vga.h"
#include "adc.h"

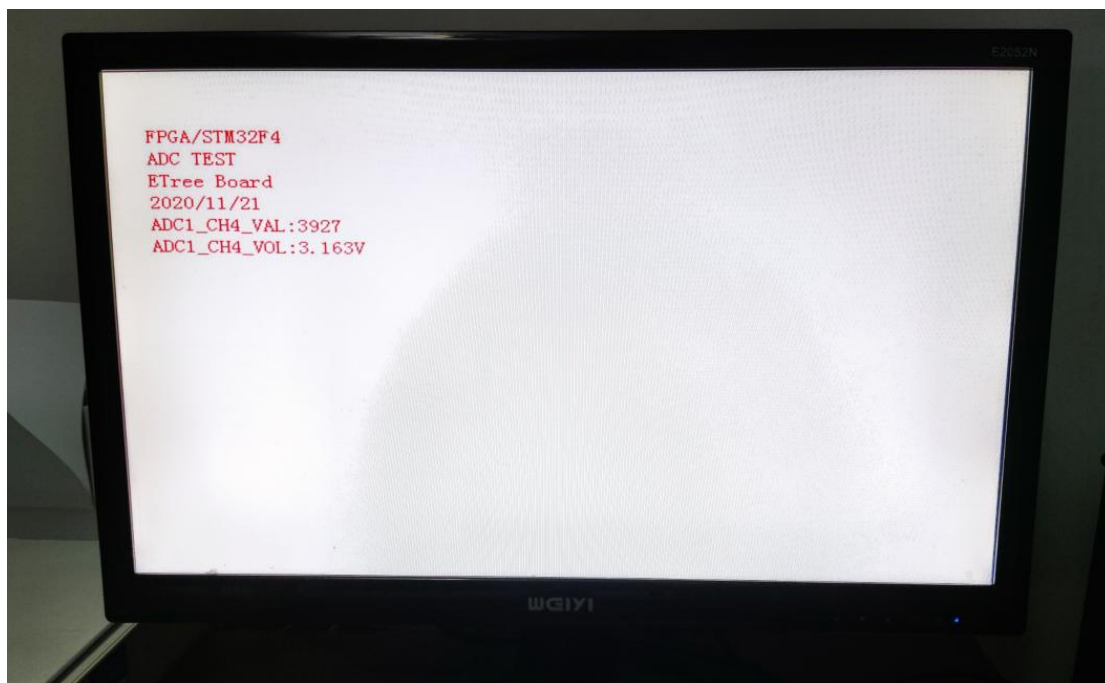
int main(void)
{
    u16 adcx;
    float temp;
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    delay_init(168);
    uart_init(115200);
    FSMC_Init();
    Adc_Init();
    VGA_Clear(WHITE);
    VGA_ShowString(30,50,200,16,16,"FPGA/STM32F4");
    VGA_ShowString(30,70,200,16,16,"ADC TEST");
    VGA_ShowString(30,90,200,16,16,"ETree Board");
    VGA_ShowString(30,110,200,16,16,"2020/11/21");
    VGA_ShowString(30,130,200,16,16,"ADC1_CH4_VAL:");
    VGA_ShowString(30,150,200,16,16,"ADC1_CH4_VOL:0.000V");
    while(1)
    {
        adcx=Get_Adc_Average(ADC_Channel_4,20);
        VGA_ShowNum(134,130,adcx,4,16);
        temp=(float)adcx*(3.3/4096);
        adcx=temp;
        VGA_ShowNum(134,150,adcx,1,16);
        temp-=adcx;
        temp*=1000;
        VGA_ShowNum(150,150,temp,3,16);
        delay_ms(250);
    }
}
```



```
}
```

11.5. 实验操作

设置好启动方式，先下载 FPGA 代码，然后下载 STM32F4 的代码。可以看到在 VGA 显示器上看到测得的 FPGA 的 IO 电压值为 3.163V，如下图所示：



第十二章 内部温度传感器

本章介绍 STM32F4 内部温度传感器的使用，并将温度值实时显示在 VGA 显示器上。

12.1. 硬件设计

本章使用的硬件和上一章一样，这个不再介绍。

12.2. STM32F4 内部温度传感器

STM32F4 内部有一个温度传感器可以用来测量 CPU 及周围的温度。这个温度传感器和 ADC1_IN16 输入通道相连，此通道将传感器的输出电压转换成数字值，其范围为-40~125 摄氏度，精度为± 1.5 摄氏度左右。

1) 设置 ADC1，开启内部温度传感器。

```
ADC_TempSensorVrefintCmd(ENABLE); //使能内部温度传感器
```

2) 读取通道 16 的 ADC 值，计算结果。

$$T(^{\circ}\text{C}) = \{ (V_{\text{sense}} - V_{25}) / \text{Avg_Slope} \} + 25$$

其中 $V_{25} = V_{\text{sense}}$ 在 25 摄氏度时的数值，典型值为 0.76；

Avg_Slope = 温度与 V_{sense} 曲线的平均斜率，典型值为 2.5mV/°C

12.3. HDL 设计

本章的 HDL 设计和上一章一样这里不再介绍。

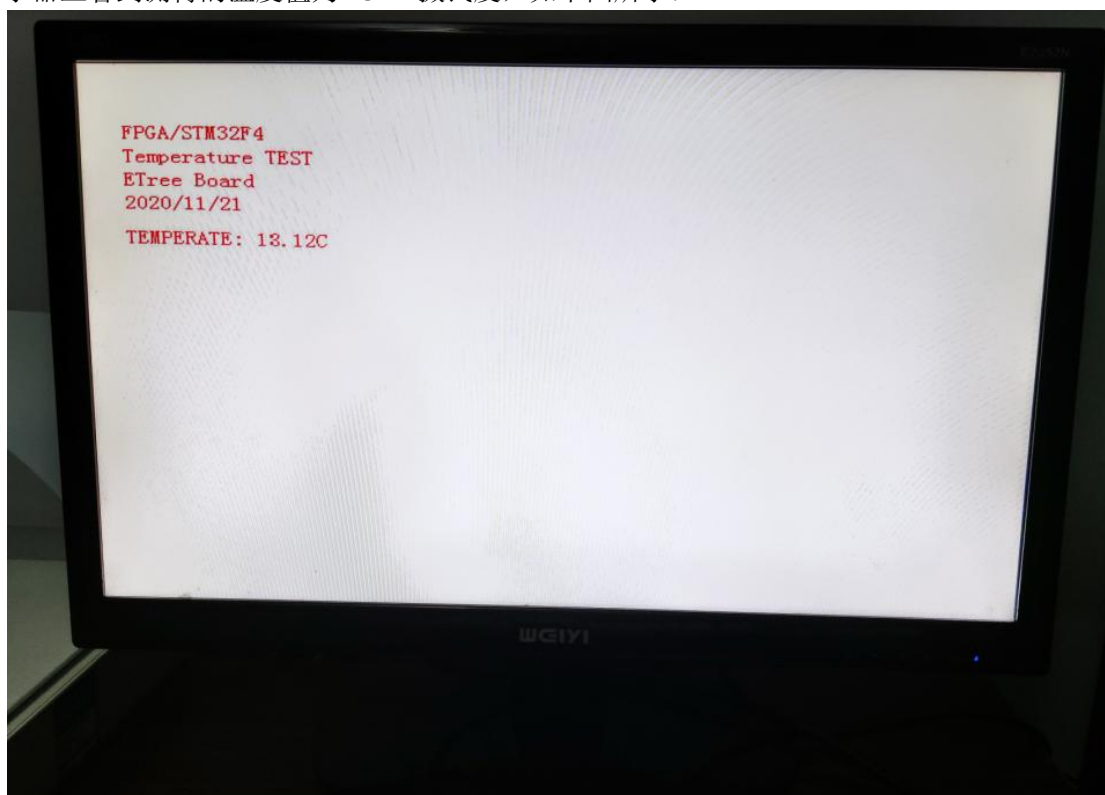
12.4. 软件设计

库函数通过 ADC_TempSensorVrefintCmd(ENABLE) 打开内部温度传感器，计算温度的函数如下所示：

```
//得到温度值
//返回值:温度值(扩大了 100 倍,单位:°C.)
short Get_Temprate(void)
{
    u32 adcx; short result;
    double temperate;
    adcx=Get_Adc_Average(ADC_Channel_16,20); //读取通道 16,20 次取平均
    temperate=(float)adcx*(3.3/4096); //电压值
    temperate=(temperate-0.76)/0.0025+25; //转换为温度值
    result=temperate*100; //扩大 100 倍.
    return result;
}
```

12.5. 实验操作

设置好启动方式，先下载 FPGA 代码，然后下载 STM32F4 的代码。可以看到在 VGA 显示器上看到测得的温度值为 18.12 摄氏度，如下图所示：



第十三章 DMA

本章介绍 STM32F4 的 DMA，利用 DMA 来实现串口数据发送，并在 VGA 显示器上显示当前进度。

13.1. 硬件设计

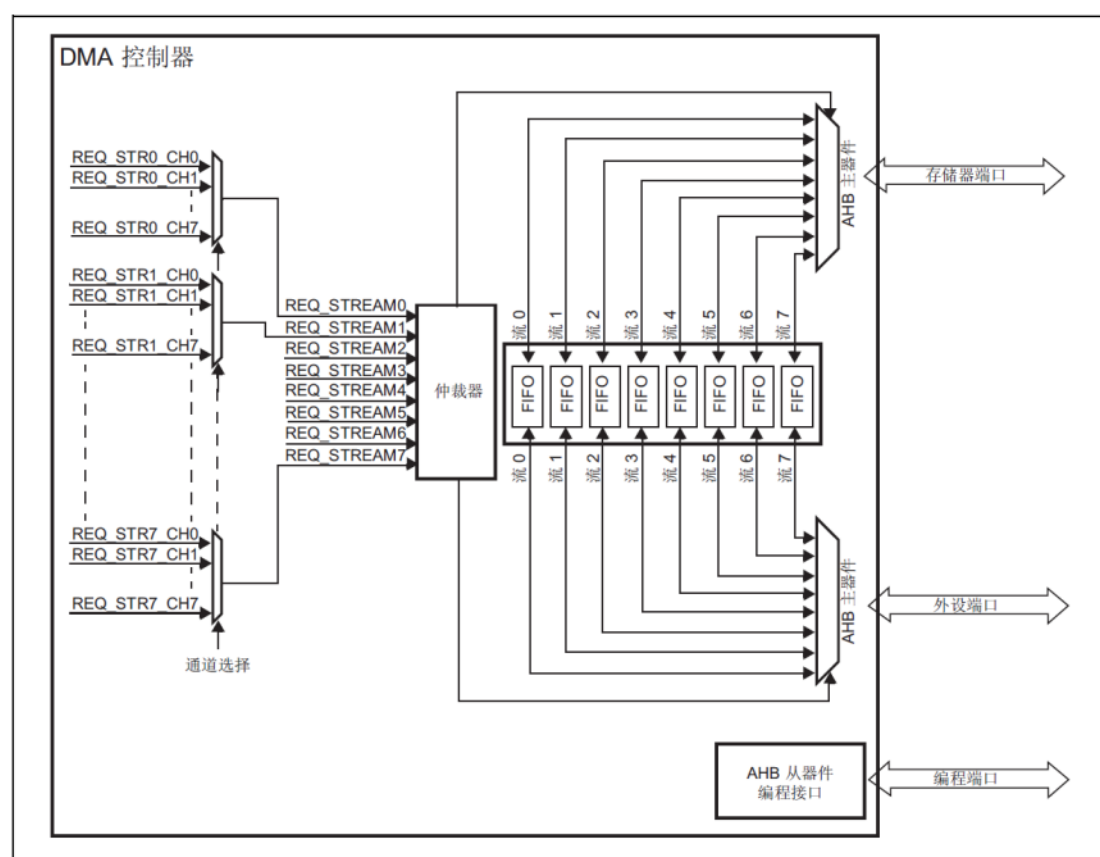
本章使用的硬件和上一章一样，这个不再介绍。

13.2. STM32F4 DMA

DMA(Direct Memory Access)直接存储方式，无需 CPU 直接控制传输，为硬件 RAM 与 I/O 设备开辟了一条专用通道，使 CPU 的效率大大提高。

STM32F4 最多有 2 个 DMA 控制器(DMA1/DMA2)，共 16 个数据流，每个数据流多大 8 个通道。其中每个数据流通道都有一个仲裁器，用于 DMA 请求间的优先级判断。

下图为 DMA 控制器框图：



DMA 控制器可以执行下列事务：

- 外设到存储器传输
- 存储器到外设传输
- 存储器到存储器传输

注意只有 DMA2 控制器支持存储器到存储器的传输，DMA1 不支持。

关于 DMA 相关寄存器描述请参考《STM32F4xx 中文参考手册》，本章的串口 1 的发送属于 DMA2 的数据流 7，通道 4，下面介绍使用库函数的配置步骤和方法。DMA 相关的库函数支持在文件 stm32f4xx_dma.c 以及对应的头文件 stm32f4xx_dac.h 中。

1) 使能 DMA2 时钟，等待数据流可配置

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2,ENABLE); //DMA2 时钟使能
while (DMA_GetCmdStatus(DMA_Streamx) != DISABLE){} //等待 DMA 可配置
```

2) 初始化 DMA2 数据流 7，包含配置通道、外设地址、存储器地址、传输数据量等

```
/* 配置 DMA Stream */
DMA_InitStructure.DMA_Channel = chx; //通道选择
DMA_InitStructure.DMA_PeripheralBaseAddr = par; //DMA 外设地址
DMA_InitStructure.DMA_Memory0BaseAddr = mar; //DMA 存储器 0 地址
DMA_InitStructure.DMA_DIR = DMA_DIR_MemoryToPeripheral; //存储器到外设模式
DMA_InitStructure.DMA_BufferSize = ndtr; //数据传输量
DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
//外设非增量模式
DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable; //存储器增量模式
DMA_InitStructure.DMA_PeripheralDataSize =
DMA_PeripheralDataSize_Byte;
//外设数据长度:8 位
DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Byte;
//存储器数据长度:8 位
DMA_InitStructure.DMA_Mode = DMA_Mode_Normal; //使用普通模式
DMA_InitStructure.DMA_Priority = DMA_Priority_Medium; //中等优先级
DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Disable;
DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full;
DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single; //单次传输
DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single;
//外设突发单次传输
DMA_Init(DMA_Streamx, &DMA_InitStructure); //初始化 DMA Stream
```

3) 使能串口 1 的 DMA 发送

```
USART_DMACmd(USART1,USART_DMAREq_Tx,ENABLE); //使能串口 1 的 DMA 发送
```

4) 使能 DMA2 数据流 7，启动传输

```
DMA_Cmd(DMA2_Stream7, ENABLE);
```

5) 查询 DMA 传输状态

在 DMA 传输过程中，使用下面函数来查询 DMA 通道的传输状态：

```
DMA_GetFlagStatus(DMA2_Stream7,DMA_FLAG_TCIF7);
```

下面函数用于获取当前剩余的数据量大小：

```
DMA_GetCurrDataCounter(DMA1_Channel4);
```

13.3. HDL 设计

本章的 HDL 设计和上一章一样这里不再介绍。

13.4. 软件设计

具体代码这里不再贴出，可以查看 HARDWARE 分组下的 dma.c 和 dma.h 文件。主控函数 main.c 如下所示：

```
#include "sys.h"
#include "delay.h"
#include "usart.h"
#include "led.h"
#include "vga.h"
#include "dma.h"

#define SEND_BUF_SIZE 8000
u8 SendBuff[SEND_BUF_SIZE];
const u8 TEXT_TO_SEND[]={"ETree FPGA/STM32F4 DMA 串口实验"};

int main(void)
{
    u32 i=0;
    float pro=0;
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    delay_init(168);
    uart_init(115200);
    LED_Init();
    FSMC_Init();

    MYDMA_Config(DMA2_Stream7,DMA_Channel_4,(u32)&USART1->DR,(u32)SendBuff,SEND_BUF_SIZE); //DMA2,STEAM7,CH4
    VGA_Clear(WHITE);
    VGA_ShowString(30,50,200,16,16,"FPGA/STM32F4");
    VGA_ShowString(30,70,200,16,16,"DMA TEST");
    VGA_ShowString(30,90,200,16,16,"ETree Board");
    VGA_ShowString(30,110,200,16,16,"2020/11/24");
    VGA_ShowString(30,130,200,16,16,"DMA2 Start");

    for(i=0;i<sizeof(TEXT_TO_SEND);i++)
    {
        SendBuff[i] = TEXT_TO_SEND[i];
    }

    printf("\r\nDMA DATA:\r\n");
    VGA_ShowString(30,150,200,16,16,"Start Transimit....");
    VGA_ShowString(30,170,200,16,16,"  %" );
    USART_DMACmd(USART1,USART_DMAREq_Tx,ENABLE);
```

```

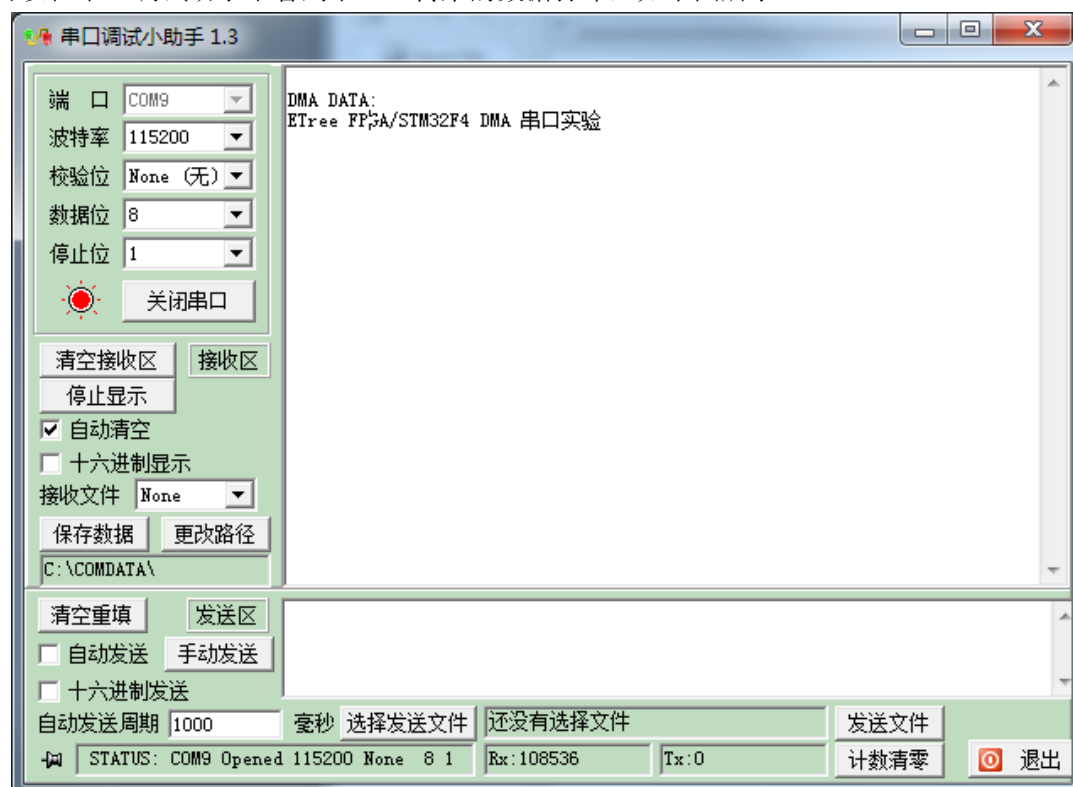
MYDMA_Enable(DMA2_Stream7,SEND_BUF_SIZE);

while(1)
{
if(DMA_GetFlagStatus(DMA2_Stream7,DMA_FLAG_TCIF7)!=RESET)
{
DMA_ClearFlag(DMA2_Stream7,DMA_FLAG_TCIF7);
break;
}
pro=DMA_GetCurrDataCounter(DMA2_Stream7);
pro=1-pro/SEND_BUF_SIZE;
pro*=100;
VGA_ShowNum(30,170,pro,3,16);
}
VGA_ShowNum(30,170,100,3,16);
VGA_ShowString(30,150,200,16,16,"Transimit Finished!");
while(1);
return 0;
}

```

13.5. 实验操作

插好 USB 串口线，设置好启动方式，先下载 FPGA 代码，然后下载 STM32F4 的代码。可以在串口调试助手看到串口 1 传来的数据打印，如下图所示：



同时在 VGA 显示上可以看到传输的状态，如下图所示：



第十四章 液晶屏显示(2.8 寸)

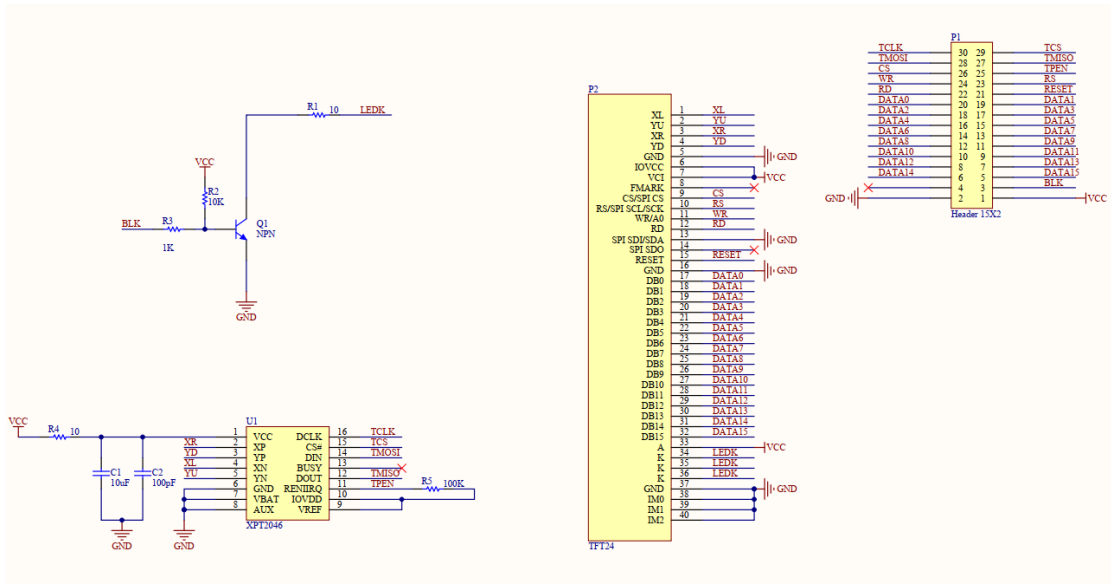
本章基于 FSMC 总线驱动 FPGA 侧的 2.8 寸液晶屏。

14.1. 硬件设计

TFT 液晶屏模块分辨率为 320*240，驱动芯片为 ILI9341，可以显示 16 位色的真彩图，接口为 MCU 并口，该模块外观图如下所示：



模块硬件原理图如下所示：



ILI9341 液晶控制器自带显存，大小为 172800(240*320*18bit)。在 16 位模式下，ILI9341 采用 RGB565 格式存储像素数据，ILI9341 的 18 位数据线与 LCD GRAM 的对应关系如下图所示。

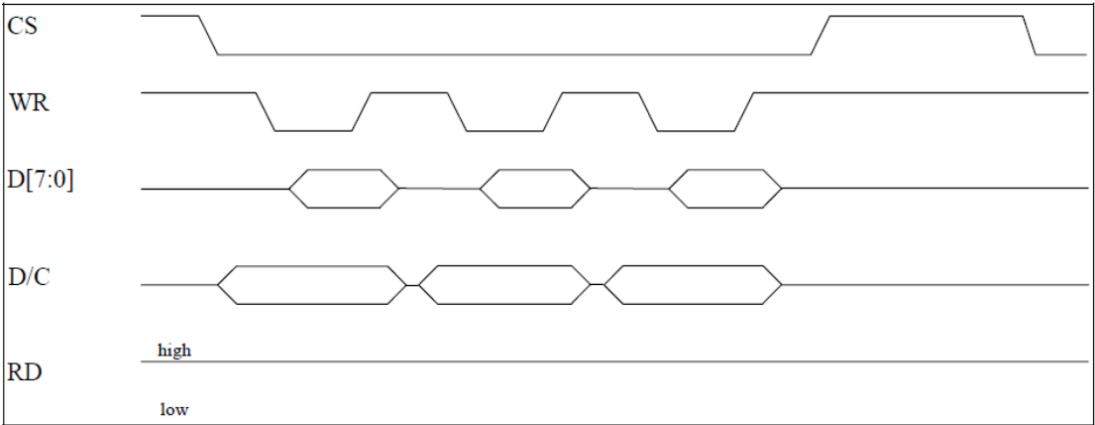
9341	D17	D16	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
总线																		
GRAM (16bit)	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC

14.2. MCU 并口

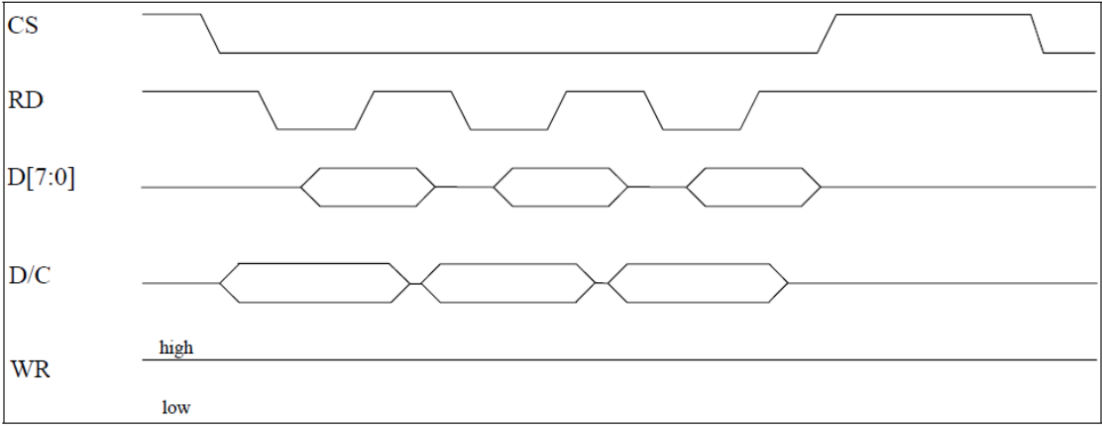
- MCU 并口总线广泛的应用于各类液晶屏显示器上，主要包含一下信号线：
- CS: 片选信号；
 - WR: 写入信号；
 - RD: 读取信号；
 - DATA[15:0]: 双向数据信号；
 - RST: 复位信号；
 - DC: 命令/数据标志(0 读，1 写)。

MCU 并口读写流程：先根据读写的数据类型，设置 DC，然后拉低 CS，接着设置 RD/WR 为低，在 RD 上升沿读取 DATA 数据，在 WR 上升沿写入 DATA 数据。

下面为写时序图：



下面为读时序图：



14.3. HDL 设计

2.8 寸液晶屏有下面一组信号，这里我们只用液晶屏做为显示使用，故只需通过 FSMC 写完成液晶屏驱动。下面是液晶屏和 FSMC 的信号连接，

- CS: 片选信号，连接 “fsmc_ne1”;
- WR: 写入信号，连接 “fsmc_nwe”;
- RD: 读取信号，置 1;
- DATA[15:0]: 双向数据信号，连接 “fsmc_data”;
- RST: 复位信号，置 1;
- DC: 命令/数据标志(0 读, 1 写), 连接 “fsmc_a16”。

具体代码如下所示:

```
assign tft_cs = fsmc_ne1;
assign tft_wr = fsmc_nwe;
assign tft_rs = fsmc_a16;
assign tft_data = fsmc_data;
assign tft_rd = 1'b1;
assign tft_reset = 1'b1;
assign tft_blk = 1'b1;
```

14.4. 软件设计

这里介绍液晶屏写数据和命令函数的实现。

液晶屏写指令时 RS 为 0，同时 RS 连接在 FSMC 的 A16 地址线上，只需要使 A16 为 0 即可，通过调用 FPGA_WR_REG0 函数即可，具体如下所示:

```
void LCD_WR_REG(vu16 regval)
{
    FPGA_WR_REG0(regval);
}
```

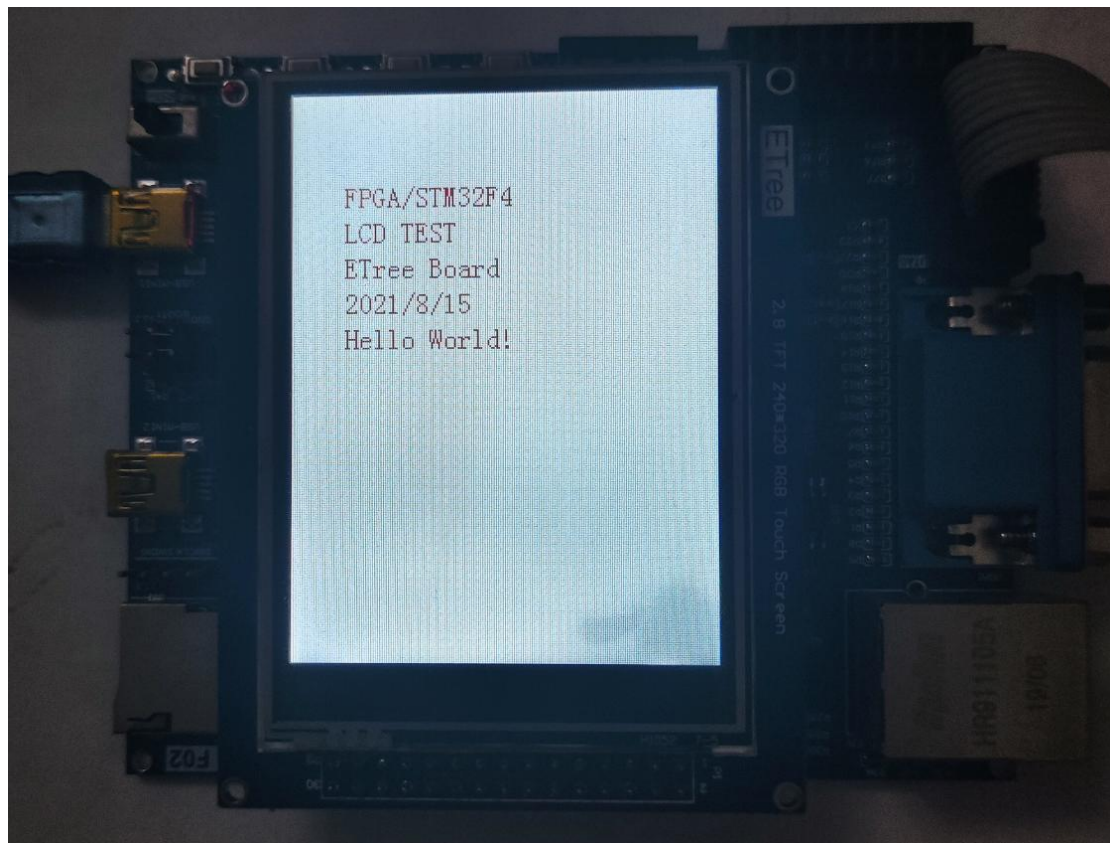
液晶屏写数据时 RS 为 1，同时 RS 连接在 FSMC 的 A16 地址线上，只需要使 A16 为 1 即可，通过调用 FPGA_WR_REG1 函数即可，具体如下所示:

```
void LCD_WR_REG(vu16 regval)
{
    FPGA_WR_REG1(regval);
}
```

关于其他函数可以参看具体工程源码，这里不做介绍了。

14.5. 实验操作

插上 2.8 寸液晶屏，设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。我们可以看到液晶屏显示板卡信息，如下图所示:



第十五章 SDRAM 读写

之前章节 SDRAM 作为显存使用, STM32F4 只能写入 SDRAM。本章利用官方的 SDRAM IP 核实现 STM32F4 通过 FSMC 读写 SDRAM, 同时通过 FSMC 总线实现外部液晶屏显示。

15.1. 硬件设计

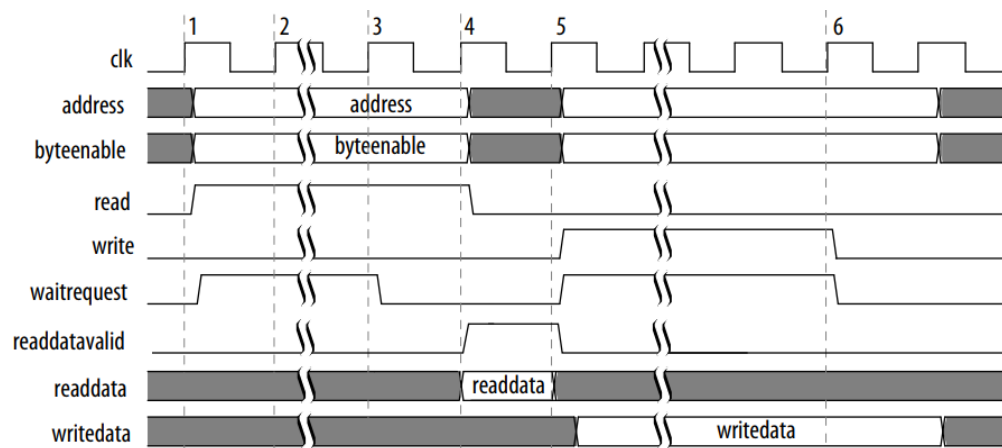
本章使用的硬件和上一章一样, 这个不再介绍。

15.2. Avalon MM 总线

Avalon MM 是 ALTERA 推出的一种片上处理器和外设连接的一种总线结构, 一般包含下面信号:

- address, Master->Slave, 地址信号, 默认 Master 以字节为单位, Slave 以字为单位。
- byteenable, Master->Slave, 字节使能信号。
- read, Master->Slave, 读信号请求。
- readdata, Slave -> Master, 读取的数据。
- write, Master->Slave, 写信号请求。
- writedata, Master->Slave, 写数据。
- waitrequest, Slave -> Master, 当 Master 检测到此信号有效时, 保持当前操作不变。
- readdatavalid, Slave -> Master, 读取数据有效信号。

下图为 Avalon MM 的读写时序:



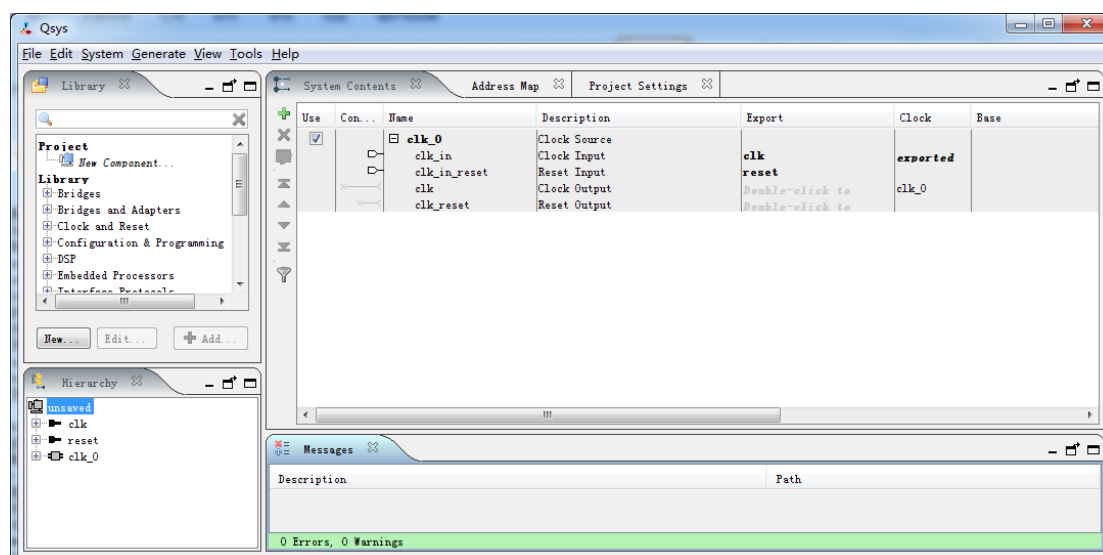
时钟上升沿给出 address/byteenable/writedata/read/write/信号, 若 waitrequest 为高则保持信号有效, 直到 waitrequest 信号为低。当 readdatavalid 信号为高, 表示读取数据有效, 此时读取 readdata 数据。

15.3. Qsys 搭建

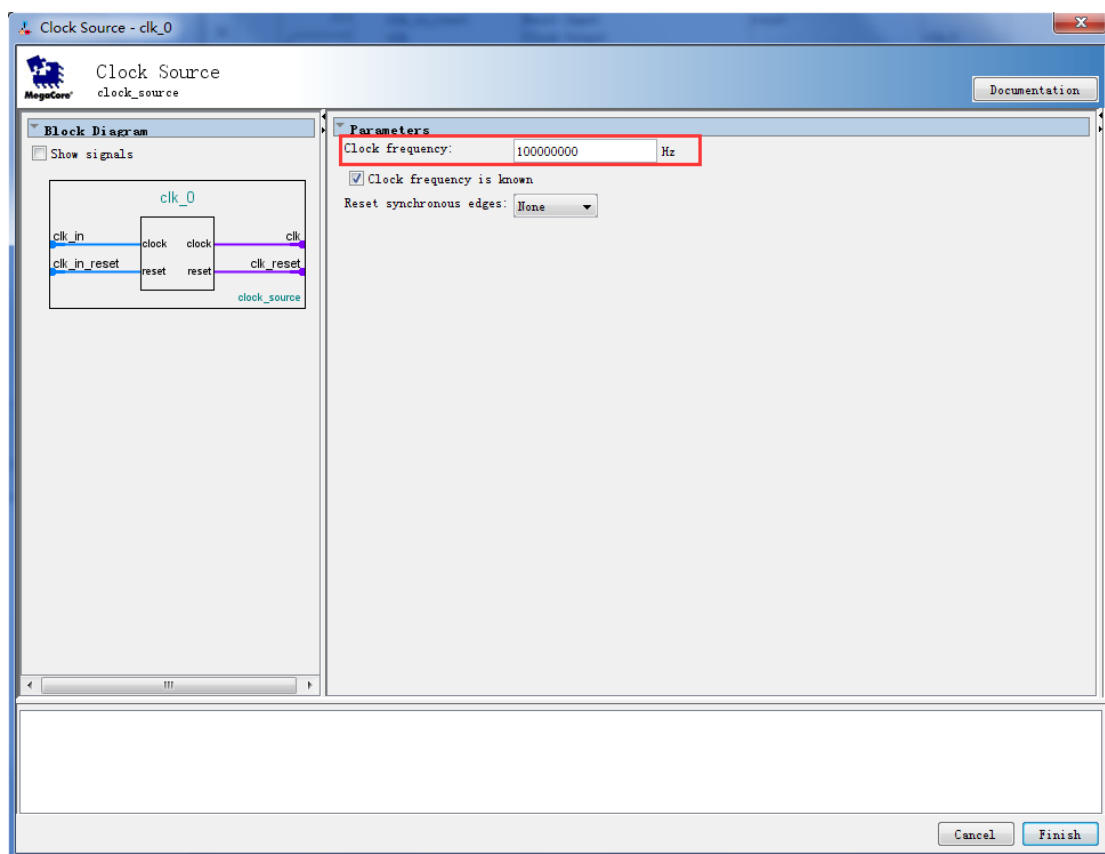
Qsys 是 ALTERA 公司(现已被 Intel 收购)提供的系统集成工具, 用于搭建片上可编程系统 (SOPC), 用户可以通过 Qsys 调用官方的 IP 核, 也可以自定义 IP 核添加到系统中。



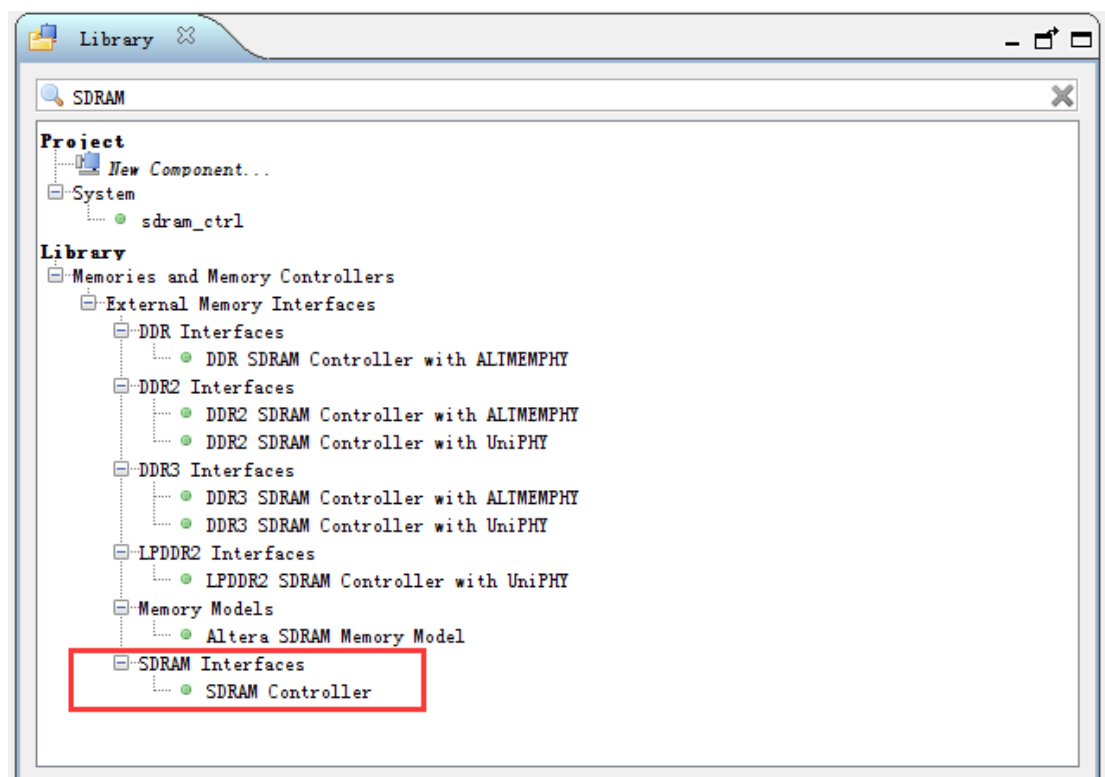
建立新的 Quartus II 工程(top_fpga)，顶层文件为 top_fpga.v。
点击"Tools"->"Qsys"，弹出 Qsys 设计平台：



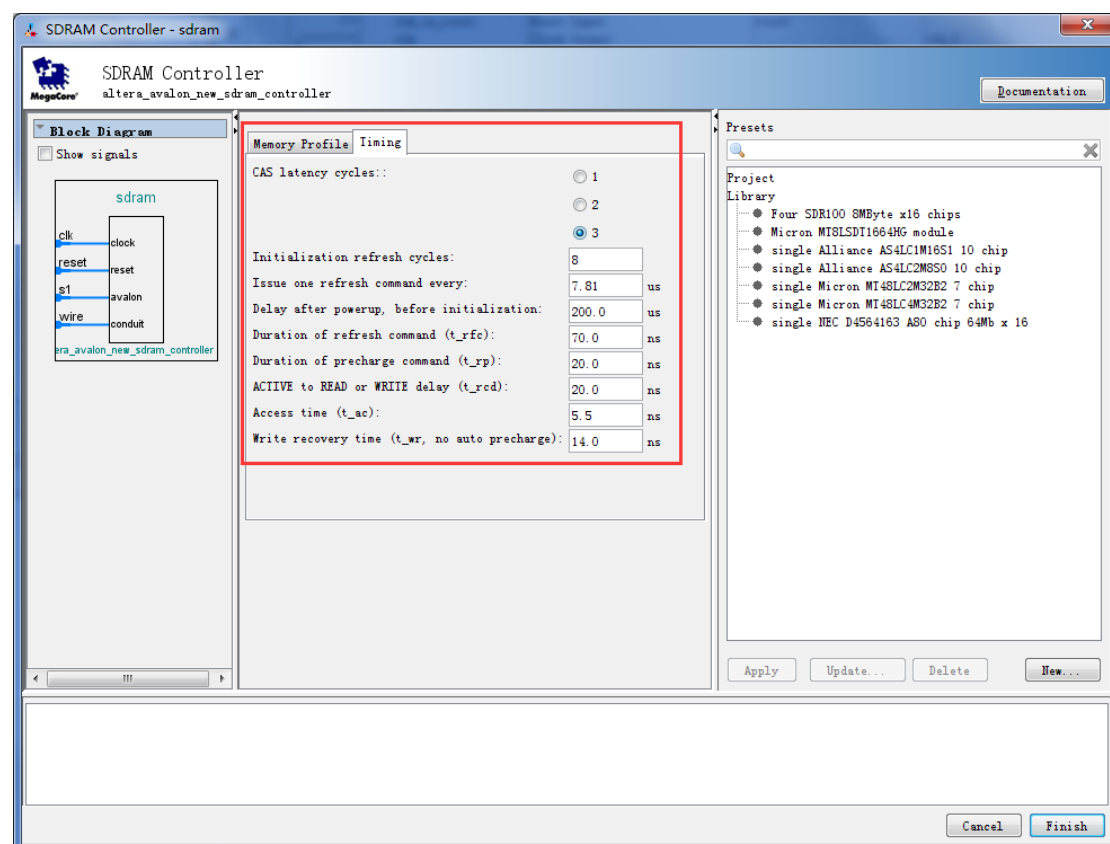
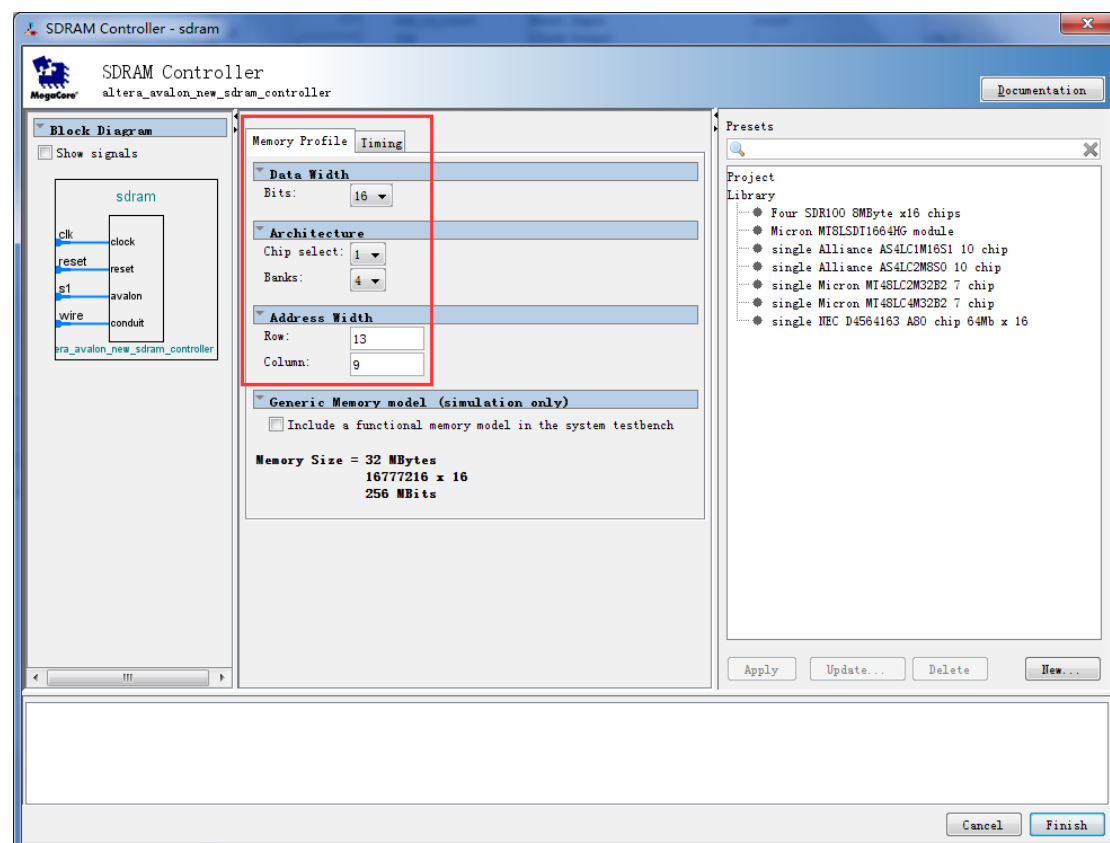
设置 clk_0 时钟为 100M，如下图所示：



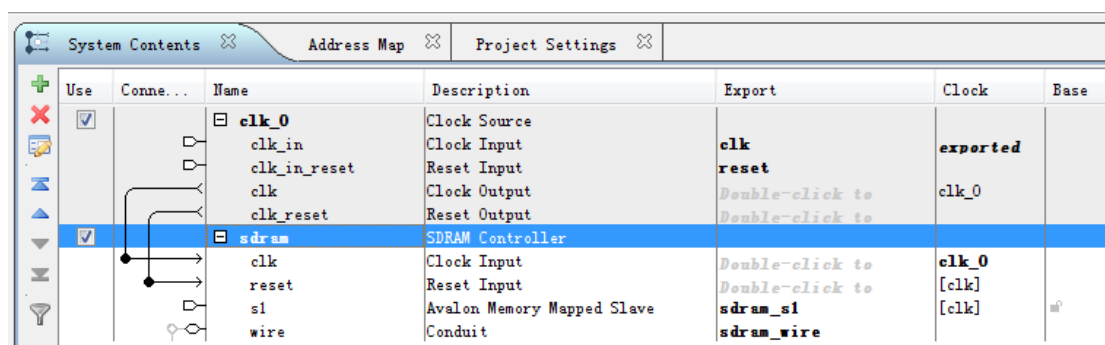
在 Library 栏输入"SDRAM", 找到 SDRAM Controller, 双击添加到系统中, 如下图所示:



双击 SDRAM 组件, 设置 SDRAM 参数, 如下图所示:



将 clk_0 的 clk 输出连接到 sdram 的 clk 输入, clk_0 的 clk_reset 连接到 sdram 的 reset, 双击 SDRAM 的 “Export” 部分, 重命名为 “sdram_s1” 和 “sdram_wire”。搭建好的 Qsys 系统如下图所示:



点击菜单 Generate | Generate, 弹出系统保存界面, 输入文件名为 “sdram_ctrl”, 如下图所示。

15.4. HDL 设计

上一节搭建好了 Qsys 系统用于调用 SDRAM IP 核, 顶层例化代码如下: 例化名为 sdram_ctrl_inst。

```
//-----SDRAM-----
wire [23:0] av_address;
wire      av_chipselect;
wire [15:0] av_writedata;
wire      av_write_n;
wire      av_read_n;
wire [15:0] av_readdata;
wire av_readdatavalid;
wire av_waitrequest;

sdram_ctrl sdram_ctrl_inst
(
    .clk_clk(clk_100m) ,    // input clk_clk_sig
    .reset_reset_n(reset) ,    // input reset_reset_n_sig

    .sdram_wire_addr(sdram_addr) ,    // output [12:0] sdram_wire_addr_sig
    .sdram_wire_ba(sdram_ba) ,    // output [1:0] sdram_wire_ba_sig
    .sdram_wire_cas_n(sdram_cas_n) ,    // output sdram_wire_cas_n_sig
    .sdram_wire_cke(sdram_cke) ,    // output sdram_wire_cke_sig
    .sdram_wire_cs_n(sdram_cs_n) ,    // output sdram_wire_cs_n_sig
    .sdram_wire_dq(sdram_dq) ,    // inout [15:0] sdram_wire_dq_sig
    .sdram_wire_dqm(sdram_dqm) ,    // output [3:0] sdram_wire_dqm_sig
    .sdram_wire_ras_n(sdram_ras_n) ,    // output sdram_wire_ras_n_sig
    .sdram_wire_we_n(sdram_we_n) ,    // output sdram_wire_we_n_sig

```

```

        .sdram_sl_address(av_address) , // input [23:0] sdram_sl_address_sig
        .sdram_sl_byteenable_n(2'b00) , // input [1:0]
sdram_sl_byteenable_n_sig
        .sdram_sl_chipselect(av_chipselect) , // input
sdram_sl_chipselect_sig
        .sdram_sl_writedata(av_writedata) , // input [15:0]
sdram_sl_writedata_sig
        .sdram_sl_read_n(av_read_n) , // input sdram_sl_read_n_sig
        .sdram_sl_write_n(av_write_n) , // input sdram_sl_write_n_sig
        .sdram_sl_readdata(av_readdata) , // output [15:0]
sdram_sl_readdata_sig
        .sdram_sl_readdatavalid(av_readdatavalid) , // output
sdram_sl_readdatavalid_sig
        .sdram_sl_waitrequest(av_waitrequest) // output
sdram_sl_waitrequest_sig
    );

```

例化好 sdram_ctrl 后, 还需要编写 AVALON MM 读写模块, 下面 Avalon MM 常用的信号。

- address, Master->Slave, 地址信号, 默认 Master 以字节为单位, Slave 以字为单位。
- byteenable, Master->Slave, 字节使能信号。
- read, Master->Slave, 读信号请求。
- readdata, Slave -> Master, 读取的数据。
- write, Master->Slave, 写信号请求。
- writedata, Master->Slave, 写数据。
- waitrequest, Slave -> Master, 当 Master 检测到此信号有效时, 保持当前操作不变。
- readdatavalid, Slave -> Master, 读取数据有效信号。

这里我们用到的信号有: address, byteenable, read, readdata, write, writedata, waitrequest, readdatavalid。具体的代码可以参看工程源码, 顶层例化代码如下:

```

sdram_ops sdram_ops_inst
(
    .clk(clk_100m) , // input clk_sig
    .rst_n(rstn) , // input rst_n_sig
    .wr_en(fsmc_register3_valid) , // input wr_en_sig
    .rd_en(fsmc_load) , // input rd_en_sig
    .wrddata_i(fsmc_register3) , // input [15:0] wrdata_i_sig
    .address_i(fsmc_addr_o) , // input [23:0] address_i_sig

    .av_address(av_address) , // output [23:0] av_address_sig
    .av_chipselect(av_chipselect) , // output av_chipselect_sig
    .av_writedata(av_writedata) , // output [15:0] av_writedata_sig
    .av_read_n(av_read_n) , // output av_read_n_sig
    .av_write_n(av_write_n) , // output av_write_n_sig
    .av_readdata(av_readdata) , // input [15:0] av_readdata_sig
    .av_readdatavalid(av_readdatavalid) , // input
av_readdatavalid_sig

```

```

.av_waitrequest(av_waitrequest) ,    // input  av_waitrequest_sig

.rddatavalid() ,    // output  rddatavalid_sig
.readdata(fsmc_rddata) // output [15:0] readdata_sig
);

```

STM32F4 通过 FSMC 总线访问 SDRAM, 同时 STM32 还需要通过 FSMC 访问 2.8 寸液晶屏, 这里我们使用 “fsmc_a19” 来切换 SDRAM 和 2.8 寸液晶屏设备。当 “fsmc_a19” 为 0 时, 选择通过 FSMC 访问 SDRAM, 当 “fsmc_a19” 为 1 时, 通过 FSMC 访问 2.8 寸液晶屏, 核心代码如下所示:

```

case(fsmc_state)
IDLE:
begin
    fsmc_register0_valid <= 0;
    fsmc_register1_valid <= 0;
    fsmc_register2_valid <= 0;
    fsmc_register3_valid <= 0;
    fsmc_register4_valid <= 0;
    fsmc_register5_valid <= 0;
    fsmc_register6_valid <= 0;
    fsmc_register7_valid <= 0;
    cnt_read              <= 0;
    if(fsmc_a19==0)
    begin
        if(ne1_r0==1'b0 && nwe_neg==1'b1)
        begin
            fsmc_state <= FSMC_WR;
            fsmc_rd_valid <= 1'b0;
            fsmc_load <= 1'b0;
        end
        else if(ne1_r0==1'b0 && noe_neg==1'b1)
        begin
            fsmc_state <= FSMC_RD;
            fsmc_rd_valid <= 1'b0;
            fsmc_load <= 1'b1;
        end
        else
        begin
            fsmc_state <= IDLE;
            fsmc_rd_valid <= 1'b0;
            fsmc_load <= 1'b0;
        end
    end
end
else
begin

```

```

tft_cs <= fsmc_ne1;
tft_wr <= fsmc_nwe;
tft_rs <= fsmc_a16;
tft_data <= fsmc_data;
fsmc_state <= IDLE;

end
end

```

15.5. 软件设计

由于液晶屏和 SDRAM 公共用了—个 FSMC 端口,STM32F4 通过 FSMC 总线访问 SDRAM,同时 STM32 还需要通过 FSMC 访问 2.8 寸液晶屏,这里我们使用“fsmc_a19”来切换 SDRAM 和 2.8 寸液晶屏设备。当“fsmc_a19”为 0 时,选择通过 FSMC 访问 SDRAM,当“fsmc_a19”为 1 时,通过 FSMC 访问 2.8 寸液晶屏。其中“fsmc_a19”对应着 STM32F4 的 PE3 引脚,这里我们通过 LCD_SET 宏定义来设置,具体代码如下:

```
#define LCD_SEL PEout (3)
```

在“VGA 字符显示”章节,已经定义了 SDRAM_WriteWord 函数,用来写入 SDRAM 一个字,具体代码如下,opt_delay()用于适当延时:

```

void SDRAM_WriteWord(u16 Buffer, u32 WriteAddr)
{
    int i=0;
    FPGA_WR_REG1(WriteAddr & 0x0000ffff);
    FPGA_WR_REG2((WriteAddr & 0xffff0000)>>16);
    FPGA_WR_REG3(Buffer);
    opt_delay(36);
}

```

由于 VGA 显示会大量占用 SDRAM 的带宽资源(即长期读取 SDRAM 用于显示),所以需要加入 opt_delay()用于适当延时,而本章 SDRAM 仅仅用于 STM32F4 的读写,没有用于 VGA 显示,所以不需要加 opt_delay(),这样提高了 STM32F4 读写 SDRAM 的速度和效率,具体代码如下:

```

void SDRAM_WriteWord(u16 Buffer, u32 WriteAddr)
{
    int i=0;
    FPGA_WR_REG1(WriteAddr & 0x0000ffff);
    FPGA_WR_REG2((WriteAddr & 0xffff0000)>>16);
    FPGA_WR_REG3(Buffer);
}

u16 SDRAM_ReadWord(u32 ReadAddr)
{
    int i=0;
    u16 Buffer;
    FPGA_WR_REG1(ReadAddr & 0x0000ffff);
    FPGA_WR_REG2((ReadAddr & 0xffff0000)>>16);
}

```

```
Buffer = FPGA_RD_REG3();  
return Buffer;  
}
```

本章还定义了 SDRAM_WriteBuffer 和 SDRAM_ReadBuffer 函数用于连续数据的读写，具体代码如下：

```
void SDRAM_WriteBuffer(u16* pBuffer,u32 WriteAddr,u32 n)  
{  
    for(;n!=0;n--)  
    {  
        SDRAM_WriteWord(*pBuffer,WriteAddr);  
        WriteAddr++;  
        pBuffer++;  
    }  
}  
  
void SDRAM_ReadBuffer(u16* pBuffer,u32 ReadAddr,u32 n)  
{  
    u32 i=0;  
    for(i=ReadAddr;i<ReadAddr+n;i++)  
    {  
        *pBuffer=SDRAM_ReadWord(i);  
        pBuffer++;  
    }  
}
```

在 main.c 函数中，定义一个 testbuf[8]数组用写入 SDRAM 数据，又定义一个 mybuffer[8] 用于读取 SDRAM 中的数据，具体代码如下：

```
int main(void)  
{  
    u32 num=0;  
    u16 testbuf[8]={11,22,33,44,55,66,77,88};  
    u16 mybuffer[8];  
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);  
    delay_init(168);  
    uart_init(115200);  
    LED_Init();  
    FSMC_Init();  
    LCD_SEL = 1;           //选中 LCD  
    LCD_Init();  
    delay_ms(1000);  
    //LCD_Clear(WHITE);  
    POINT_COLOR=RED;  
    LCD_ShowString(30,50,200,16,16,"FPGA/STM32F4");  
    LCD_ShowString(30,70,200,16,16,"SDRAM TEST");  
}
```

```
LCD_ShowString(30,90,200,16,16,"ETree Board");
LCD_ShowString(30,110,200,16,16,"2021/9/11");
LCD_ShowString(30,130,200,16,16,"Hello World!");

LCD_SEL = 0;           //选中 SDRAM

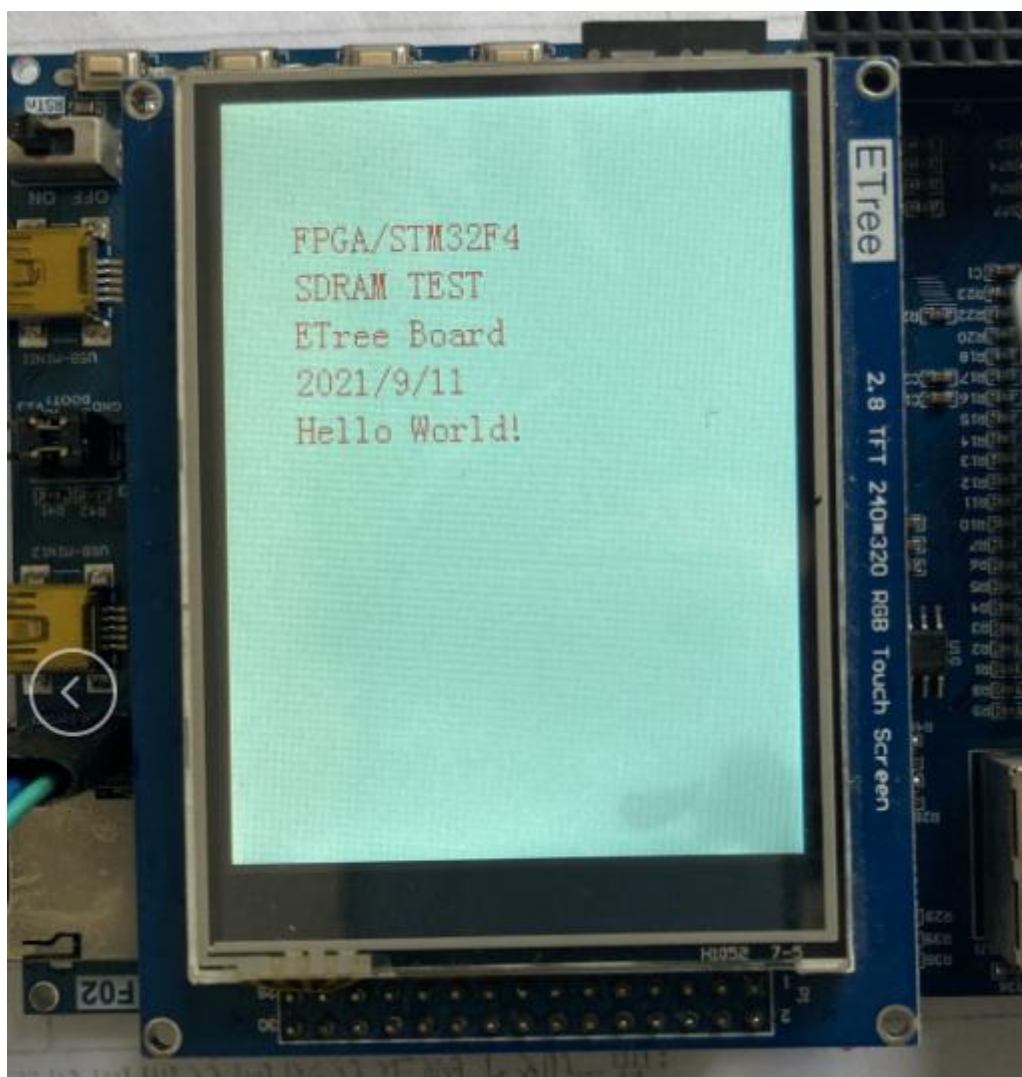
SDRAM_WriteBuffer(testbuf,0,8);
SDRAM_ReadBuffer(mybuffer,0,8);

printf("MyBuffer is ");
for(num=0;num<8;num++)
    printf("%d ",mybuffer[num]);

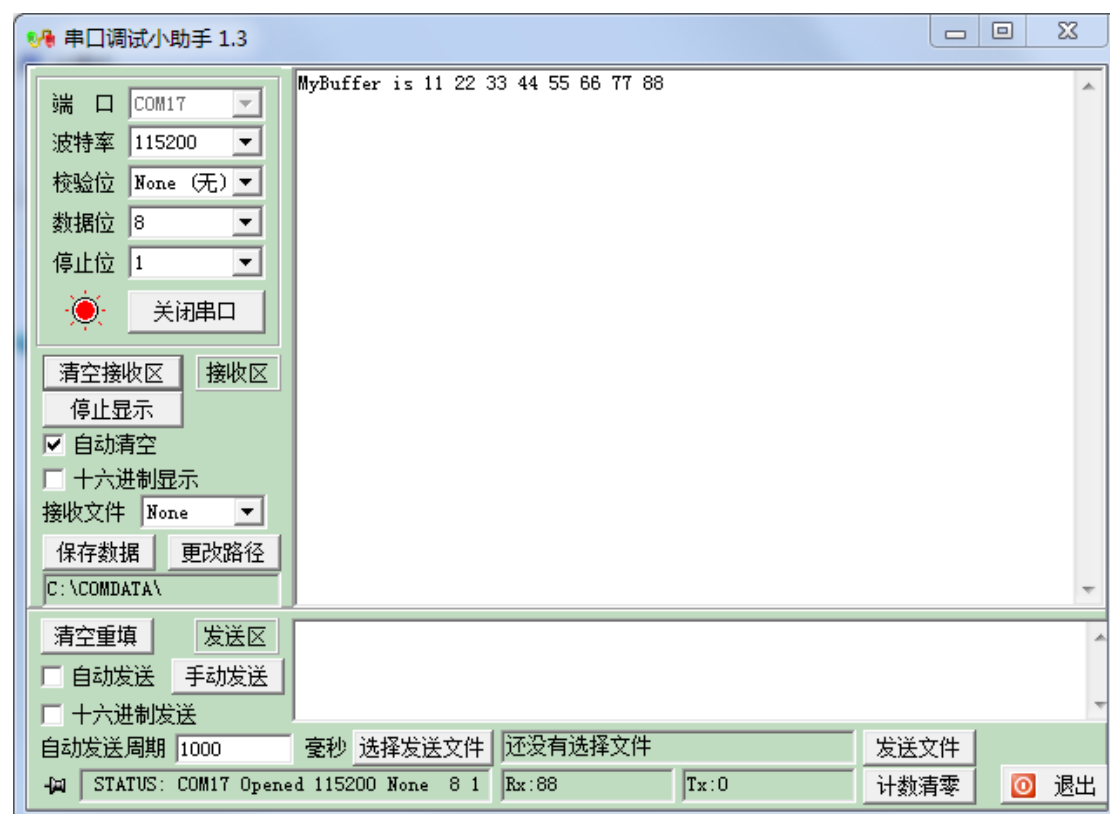
while(1);
return 0;
}
```

15.6. 实验操作

插上 2.8 寸液晶屏，设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。我们可以看到液晶屏显示板卡信息，如下图所示：



串口打印数据如下图所示，和预先定义的 `testbuf` 数据一样。



第十六章 内存管理

本章介绍如何申请和释放内存，即内存管理。

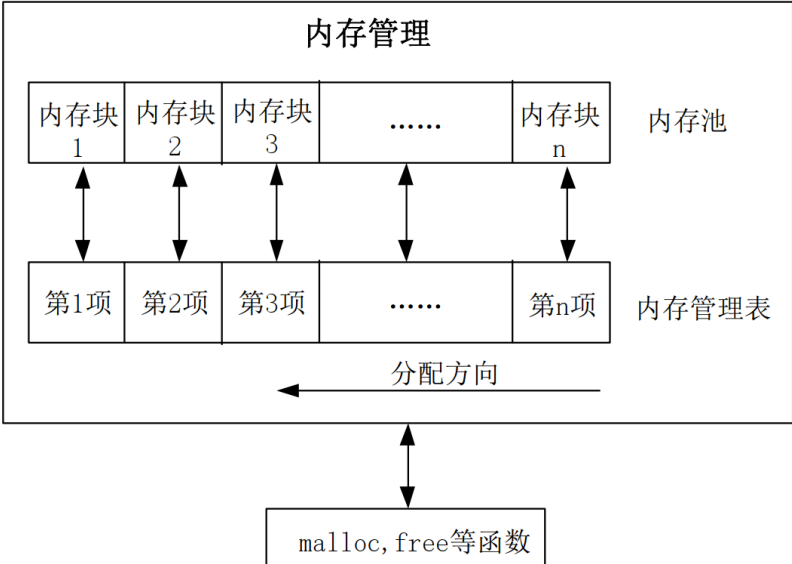
15.1. 硬件设计

本章使用的硬件和上一章一样，这个不再介绍，2.8 寸液晶屏用来显示程序运行的信息和结果。

15.2. 内存管理简介

内存管理主要目的是高效、快速的分配和释放内存。内存管理的方法比较多，最终通过 `malloc` 和 `free` 两个函数实现。

本章介绍一种比较简单的实现方法：分块式内存管理，具体原理如下图所示：



从上图我们可以看出，分块式内存管理由内存池和内存管理表组成。内存池被等分为 n 块，对应的内存管理表，大小也为 n ，内存管理表的每一项对应内存池的一块内存。

内存管理表的值代表意义为：当该项值为 0 时，代表对应的内存块没有被占用，当该项值为非 0 时，代表该项对应的内存块已经被占用，其数值代表被连续占用的内存块数。

内存分配的方向为从顶到底，即从内存的最末端开始寻找空内存块。在内存管理初始化后，内存表全部清零，表示没有占用任何内存块。

下面具体介绍分配和释放原理。

分配原理：当指针 p 调用 `malloc` 申请内存时，首先判断 p 要分配的内存块数量 m ，然后从第 n 项开始向下查找，直到找到 m 块连续的空内存块，找到后将这 m 个内存块管理项都设置成 m 表示被占用，最后把这个空内存块的地址返回给指针 p ，完成一次分配。如果内存不够则返回 `NULL` 给 p 表示分配失败。

释放原理：当 p 申请的内存用完后，调用 `free` 释放。首先 `free` 函数判断 p 指针的内存地址对应的内存块，然后找到对应的内存管理表项目，得到占用的内存块数目 m ，最后将这 m 个内存管理表的项目值都清零，完成一次内存释放。

15.3. HDL 设计

本章的 HDL 设计和上一章一样这里不再介绍。

15.4. 软件设计

在工程中，我们添加了 MALLOC 分组，包含了 malloc.c 和 malloc.h 文件。

我们定义 malloc_dev 结构体来管理内存控制器，具体定义如下：

```
struct _m_malloc_dev
{
    void (*init) (u8);           //初始化
    u8 (*perused) (u8);         //内存使用率
    u8 *membase[SRAMBANK];      //内存池
    u16 *memmap[SRAMBANK];      //内存管理状态表
    u8 memrdy[SRAMBANK];        //内存管理是否就绪
};
```

内存池定义如下：__align(4) 定义内存池以 32 字节对齐

```
__align(32) u8 mem1base[MEM1_MAX_SIZE]; //内部 SRAM 内存池
__align(32) u8 mem2base[MEM2_MAX_SIZE] __attribute__((at(0x68000000)));
//外部 SRAM 内存池
__align(32) u8 mem3base[MEM3_MAX_SIZE] __attribute__((at(0x10000000)));
//内部 CCM 内存池
```

STM32F4 内部内存分为两块：普通内存(地址 0x2000 0000 开始，共 128KB，任何外设都可以访问)，CCM 内存(地址 0x1000 0000 开始，共 64KB，仅 CPU 可以访问)。

而外部 SRAM 地址从 0x6800 0000 开始，共 1024KB，F02 开发板没有用到外部 SRAM。

下面是用户可以调用的函数，用于内存分配、释放和重分配。

```
void myfree(void *ptr);           //内存释放(外部调用)
void *mymalloc(u32 size);         //内存分配(外部调用)
void *myrealloc(void *ptr,u32 size); //重新分配内存(外部调用)
```

其他子函数可以在源码工程中查看，这里就不做说明了。

在 main.c 函数，通过 myalloc 函数调用，myfree 函数释放，具体如下所示：

```
int main(void)
{
    u32 num=0;
    u8 *p=0;
    u8 sramx=0; //默认内部 SRAM

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    delay_init(168);
    uart_init(115200);
    LED_Init();
    FSMC_Init();
```

```
my_mem_init(SRAMIN);
my_mem_init(SRAMCCM);

LCD_SEL = 1;
LCD_Init();
delay_ms(1000);
POINT_COLOR=RED;
LCD_ShowString(30,50,200,16,16,"FPGA/STM32F4");
LCD_ShowString(30,70,200,16,16,"MALLOC TEST");
LCD_ShowString(30,90,200,16,16,"ETree Board");
LCD_ShowString(30,110,200,16,16,"2021/9/11");

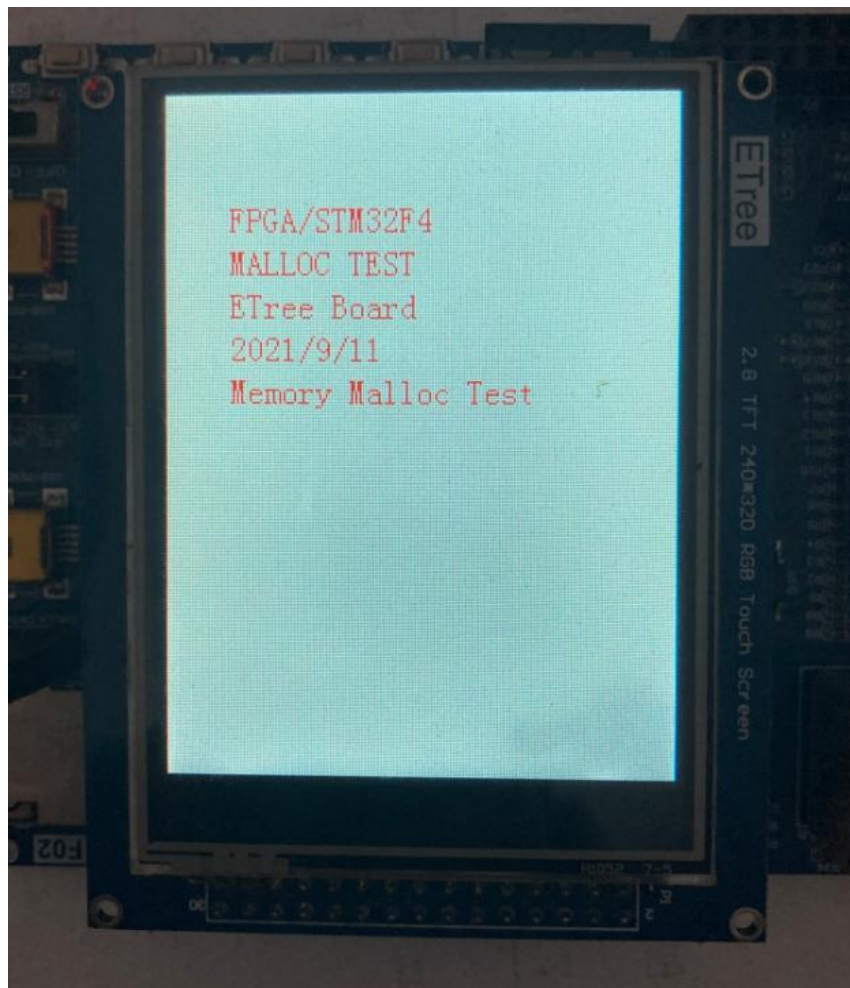
p = mymalloc(sramx,2048); //申请内部 2K 内存

if(p!=NULL)
    sprintf((char*)p,"Memory Malloc Test",18);
LCD_ShowString(30,130,200,16,16,p);
myfree(sramx,p); //释放内存

while(1);
return 0;
}
```

15.5. 实验操作

插上 2.8 寸液晶屏，设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。我们可以看到液晶屏显示板卡信息，如下图所示：



第十七章 TF-SD 卡

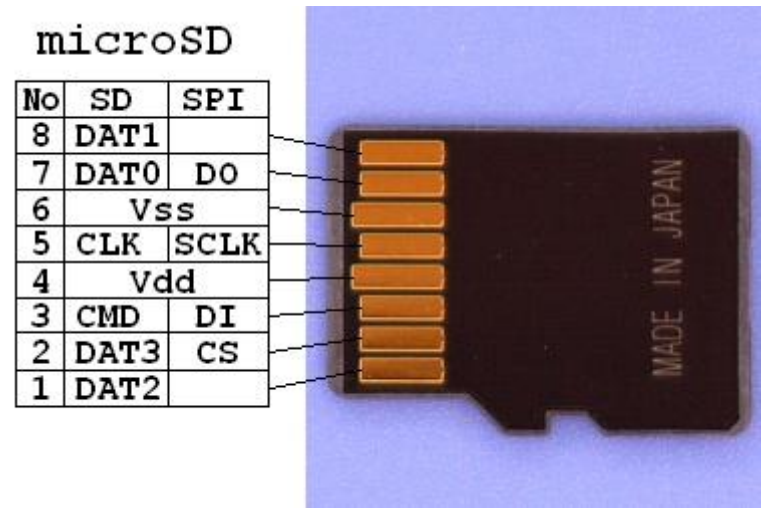
本章通过 STM32F4 的 SDIO 接口驱动 TF-SD 卡，实现卡信息获取与数据读取并读取，读者需要准备一片 2.0 标准的 micro SD 卡。注意在实验之前备份好重要的数据资料！

16.1. 硬件设计

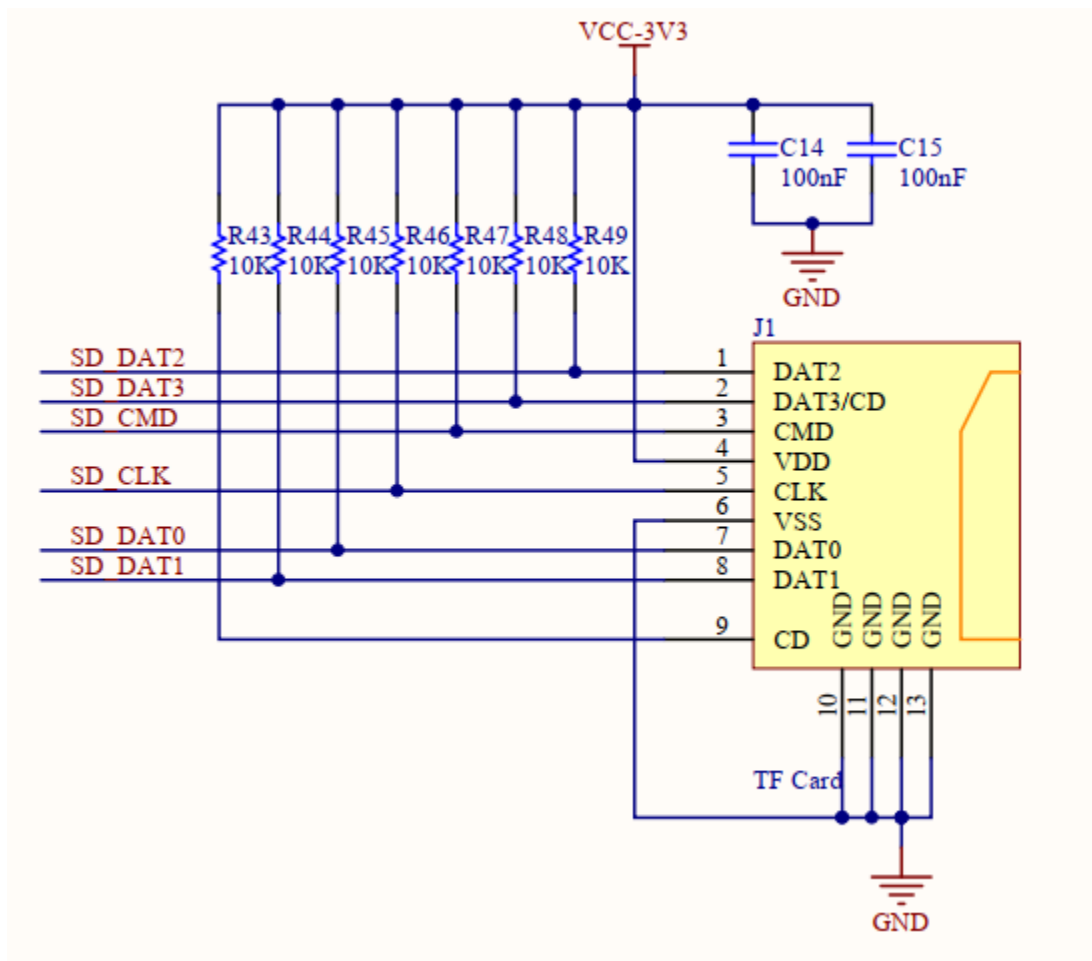
SD 卡(Secure Digital Memory Card)，基于 MMC 发展而来，被广泛应用于数码产品中。SD 卡可分为 3 类：SD 卡(0~2G)、SDHC 卡(2~32G)、SDXC 卡(32G~2T)。SD 卡有 9 个 PIN，支持 SPI 和 SDIO 模式：

PIN	1	2	3	4	5	6	7	8	9
SDIO	CD/DAT3	CMD	VSS	VCC	CLK	VSS	DAT0	DAT1	DAT2
SPI	CS	MOSI	VSS	VCC	CLK	VSS	MISO	NC	NC

而我们板卡上位 micro SD 卡接口，接口定义稍微有点区别，如下图所示。



下图为 micro SD 卡插槽原理图：



TF-SD 卡连接到了 STM32F4 的 SDIO 引脚，如下图所示：

J11	SD DAT0	DCMI D2	PC8	65	PC7/TIM8_CH2/I2S5_MCK/SDIO
K11	SD DAT1	DCMI D3	PC9	66	PC8/TIM8_CH3/SDIO_D0/(TIM3
	SD DAT2	DCMI D4	PC10	78	PC9/TIM8_CH4/SIO_D1(TIM3
F13	SD DAT3		PC11	79	PC10/USART4_TX/SDIO_D2/(US
	SD CLK		PC12	80	PC11/USART4_RX/SDIO_D3/(US
			PC13	7	PC12/UART5_TX/SDIO_CK/(US

16.2. HDL 设计

本章 FPGA 代码和之前章节一样，具体如下所示：

```
//file name:      top_fpga.v
//author:        ETree
//date:          2017.9.9
//function:      top file of project
//log:           led light

module top_fpga(
    //global signal
    input clk, //50MHz
    input rst_n, //low active

    //key
```

```
input key0,
input key1,
input key2,

//UART
input uart_rx,
input uart_tx,

//STM32
input stm_pa1,
input stm_pb6,
output stm_pb7,

//led
output led //high active
);

assign led = stm_pb6;
assign stm_pb7 = key0;

endmodule
```

16.3. 软件设计

关于 SD 卡的协议和 SDIO 的相关内容可以参考“学习资料”目录下的《SD 卡 2.0 协议.pdf》和《STM32F4xx 中文参考手册》第 28 章，这里不做讲解。

STM32F4 软件流程：初始化 SDIO 接口-->初始化 SD 卡-->配置 SDIO DMA-->打印 SD 卡信息-->等待按键按下-->读取 SD 卡扇区 0 数据并打印在串口终端。

主函数如下所示：

```
#include "sys.h"
#include "delay.h"
#include "usart.h"
#include "led.h"
#include "key.h"
#include "malloc.h"
#include "sdio_sdcard.h"

void show_sdcard_info(void)
{
    switch(SDCardInfo.CardType)
    {
        case SDIO_STD_CAPACITY_SD_CARD_V1_1:printf("Card Type:SDSC V1.1\r\n");break;
        case SDIO_STD_CAPACITY_SD_CARD_V2_0:printf("Card Type:SDSC
```

```
V2.0\r\n");break;
    case SDIO_HIGH_CAPACITY_SD_CARD:printf("Card Type:SDHC
V2.0\r\n");break;
    case SDIO_MULTIMEDIA_CARD:printf("Card Type:MMC
Card\r\n");break;
}
printf("Card
ManufacturerID:%d\r\n",SDCardInfo.SD_cid.ManufacturerID);
printf("Card RCA:%d\r\n",SDCardInfo.RCA);
printf("Card Capacity:%d
MB\r\n",(u32)(SDCardInfo.CardCapacity>>20));
printf("Card BlockSize:%d\r\n\r\n",SDCardInfo.CardBlockSize);
}

int main(void)
{
    u8 key;
    u32 sd_size;
    u8 t=0;
    u8 *buf;

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    delay_init(168);
    uart_init(115200);
    LED_Init();
    KEY_Init();

    my_mem_init(SRAMIN);
    my_mem_init(SRAMCCM);

    printf("KEY0:Read Sector 0\r\n");
    while(SD_Init())
    {
        printf("SD Card Error!\r\n");
        delay_ms(500);
        printf("Please Check!\r\n");
        delay_ms(500);
        LED=!LED;
    }
    printf("SD Card OK\r\n");
    show_sdcard_info();

    while(1)
```



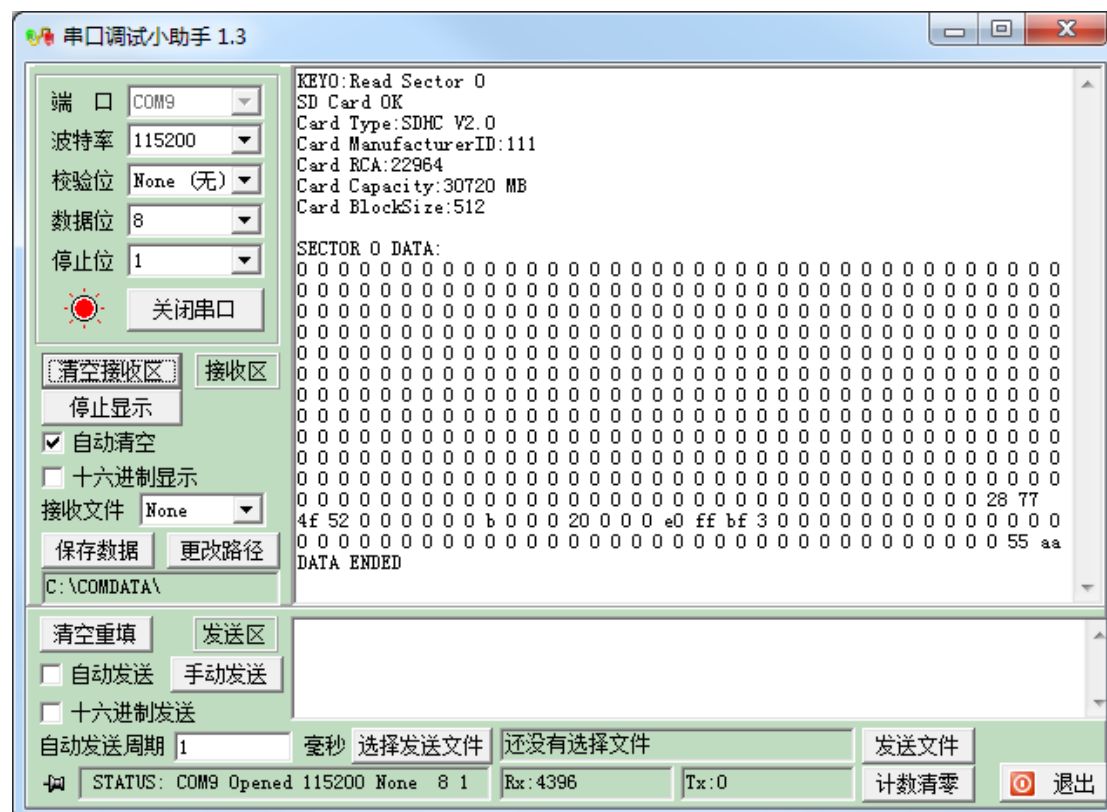
```
{
    key=KEY_Scan(0);
    if(key==1)//KEY0
    {
        buf=mymalloc(0,512);
        if(SD_ReadDisk(buf,0,1)==0)
        {
            printf("SECTOR 0 DATA:\r\n");
            for(sd_size=0;sd_size<512;sd_size++)printf("%x
",buf[sd_size]);
            printf("\r\nDATA ENDED\r\n");
        }
        myfree(0,buf);
    }
    t++;
    delay_ms(10);
    if(t==20)
    {
        LED=!LED;
        t=0;
    }
} }
```

16.4. 实验操作

插入 TF-SD(Micro SD)卡, 设置好启动方式, 先下载 FPGA 的代码, 再下载 STM32 的代码。我们可以看到串口调试助手打印信息, 如下图所示:



按下 FPGA 板卡上的 S2 按键，读出 SD 卡 Sector 0 的数据，打印信息如下图所示：



第十八章 FATFS 文件系统

上一章节我们学习了如何使用 SD/TF 卡，不过要真正用好 SD 卡，还需要使用文件系统。本章我们使用 FATFS 来管理 SD 卡。

18.1. 硬件设计

本章使用的硬件和上一章一样，这个不再介绍。

18.2. FATFS 简介

FATFS 是免费开源的 FAT 文件系统，专门为小型嵌入式系统设计，完全用 C 语言编写，具有很好的移植性。

FATFS 有如下特点：

- ◆ Windows 兼容的 FAT 文件系统(FAT12/FAT16/FAT32)
- ◆ 平台无关，移植简单
- ◆ 代码少、效率高
- ◆ 多种配置选项
 - ◇ 支持多卷(物理驱动器或分区，最多 10 卷)
 - ◇ 多个 ANSI/OEM 代码页包括 DBCS
 - ◇ 支持长文件名、ANSI/OEM 或 Unicode
 - ◇ 支持 RTOS
 - ◇ 支持多种扇区大小
 - ◇ 只读、最小化的 API 和 I/O 缓冲区



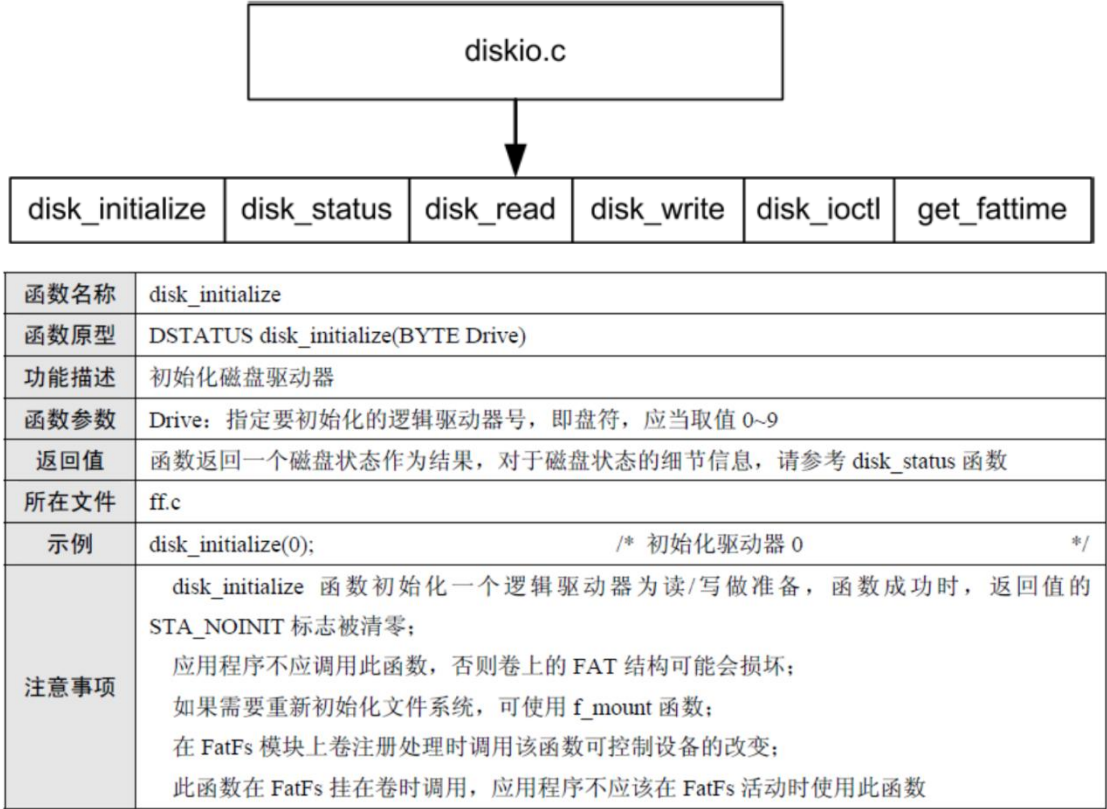
FATFS 源码可以在 http://elm-chan.org/fsw/ff/00index_e.html 网站下载，解压后可得到两个文件夹：doc 和 src。在 src 里是我们需要的源码文件，其中与平台无关的是：

- ◆ ffconf.h: FATFS 配置文件
- ◆ ff.h: FATFS 和应用模块公用的包含文件
- ◆ ff.c: FATFS 模块

- ◆ diskio.h: FATFS 和 disk I/O 模块公用的包含文件
- ◆ interger.h: 数据类型定义
- ◆ option: 可选外部功能

与平台相关的代码:

- ◆ diskio.c: FATFS 和 disk I/O 模块接口层文件。
FATFS 在移植时, 一般只需要修改 2 个文件: ffconf.h 和 diskio.c。
在 ffconf.h 中需要修改一些配置选项来满足我们的需求, 下面介绍几个重要的配置选项:
 - ◆ _USE_STRFUNC: 用来设置对字符串类的操作, 比如 f_putc、f_putc 等, 这里设置成 1
 - ◆ _USE_MKFS: 用来是使能格式化, 这里设置成 1
 - ◆ _USE_FASTSEEK: 用来快速定位, 这里设置为 1
 - ◆ _USE_LABEL: 用来设置是否支持磁盘盘符读取与设置, 这里设置为 1
 - ◆ _CODE_PAGE: 用于设置语言类型, 这里设置成 936
 - ◆ _USE_LFN: 用于设置是否支持长文件名, 0 表示不支持, 1~3 表示支持, 这里设置成 3, 通过 ff_memalloc 函数来动态分配长文件名的存储位置
- diskio.c 下底层驱动编写, 一般有 6 个接口函数:



函数名称	disk_status
函数原型	DSTATUS disk_status(BYTE Drive)
功能描述	返回当前磁盘驱动器的状态
函数参数	Drive: 指定要确认的逻辑驱动器号, 即盘符, 应当取值 0~9
返回值	磁盘状态返回下列标志的组合, FatFs 只使用 STA_NOINIT 和 STA_PROTECTED STA_NOINIT: 表明磁盘驱动未初始化, 下面列出了产生该标志置位或清零的原因: 置位: 系统复位, 磁盘被移除和磁盘初始化函数失败; 清零: 磁盘初始化函数成功 STA_NODISK: 表明驱动器中没有设备, 安装磁盘驱动器后总为 0 STA_PROTECTED: 表明设备被写保护, 不支持写保护的总为 0, 当 STA_NODISK 置位时非法
所在文件	ff.c
示例	disk_status(0); /* 获取驱动器 0 的状态 */
函数名称	disk_read
函数原型	DRESULT disk_read(BYTE Drive, BYTE* Buffer, DWORD SectorNumber, BYTE SectorCount)
功能描述	从磁盘驱动器上读取扇区
函数参数	Drive: 指定逻辑驱动器号, 即盘符, 应当取值 0~9 Buffer: 指向存储读取数据字节数组的指针, 需要为所读取字节数的大小, 扇区统计的扇区大小是需要的 注: FatFs 指定的内存地址并不总是字对齐的, 如果硬件不支持不对齐的数据传输, 函数里需要进行处理 SectorNumber: 指定起始扇区的逻辑块 (LBA) 上的地址 SectorCount: 指定要读取的扇区数, 取值 1~128
返回值	RES_OK(0): 函数成功 RES_ERROR: 读操作期间产生了任何错误且不能恢复它 RES_PARERR: 非法参数 RES_NOTRDY: 磁盘驱动器没有初始化
所在文件	ff.c
函数名称	disk_write
函数原型	DRESULT disk_write(BYTE Drive, const BYTE* Buffer, DWORD SectorNumber, BYTE SectorCount)
功能描述	向磁盘写入一个或多个扇区
函数参数	Drive: 指定逻辑驱动器号, 即盘符, 应当取值 0~9 Buffer: 指向要写入字节数组的指针, 注: FatFs 指定的内存地址并不总是字对齐的, 如果硬件不支持不对齐的数据传输, 函数里需要进行处理 SectorNumber: 指定起始扇区的逻辑块 (LBA) 上的地址 SectorCount: 指定要写入的扇区数, 取值 1~128
返回值	RES_OK(0): 函数成功 RES_ERROR: 读操作期间产生了任何错误且不能恢复它 RES_WRPRT: 媒体被写保护 RES_PARERR: 非法参数 RES_NOTRDY: 磁盘驱动器没有初始化
所在文件	ff.c
注意事项	只读配置中不需要此函数

函数名称	disk_ioctl
函数原型	DRESULT disk_ioctl(BYTE Drive, BYTE Command, void* Buffer)
功能描述	控制设备指定特性和除了读/写外的杂项功能
函数参数	Drive: 指定逻辑驱动器号, 即盘符, 应当取值 0~9 Command: 指定命令代码 Buffer: 指向参数缓冲区的指针, 取决于命令代码, 不使用时, 指定一个 NULL 指针
返回值	RES_OK(0): 函数成功 RES_ERROR: 读操作期间产生了任何错误且不能恢复它 RES_PARERR: 非法参数 RES_NOTRDY: 磁盘驱动器没有初始化
所在文件	ff.c
注意事项	CTRL_SYNC: 确保磁盘驱动器已经完成了写处理, 当磁盘 I/O 有一个写回缓存, 立即刷新原扇区, 只读配置下不适用此命令 GET_SECTOR_SIZE: 返回磁盘的扇区大小, 只用于 f_mkfs() GET_SECTOR_COUNT: 返回可利用的扇区数, _MAX_SS >= 1024 时可用 GET_BLOCK_SIZE: 获取擦除块大小, 只用于 f_mkfs() CTRL_ERASE_SECTOR: 强制擦除一块的扇区, _USE_ERASE > 0 时可用

函数名称	get_fattime
函数原型	DWORD get_fattime()
功能描述	获取当前时间
函数参数	无
返回值	当前时间以双字值封装返回, 位域如下: bit31:25 年 (0~127) (从 1980 开始) bit24:21 月 (1~12) bit20:16 日 (1~31) bit15:11 小时 (0~23) bit10:5 分钟 (0~59) bit4:0 秒 (0~29)
所在文件	ff.c
注意事项	get_fattime 函数必须返回一个合法的时间即使系统不支持实时时钟, 如果返回 0, 文件没有一个合法的时间; 只读配置下无需此函数

FATFS 提供了很多 API 函数, 这些函数 FATFS 有详细介绍, 这里不多介绍了。在使用 FATFS 时, 需要通过 f_mount 函数注册一个工作区, 才能使用 API 函数。

18.3. HDL 设计

本章 FPGA 代码和之前章节一样, 这里不做介绍。

18.4. 软件设计

diskio.c 和 ffconf.h 修改好后, 还定义了 exfuncs.c、exfuncs.h、fattester.c、fattester.h 文件, 为了方便 FATFS 的使用, 具体可以查看工程源码, 下面为 main 函数:

```
int main(void)
```



```
{
    u32 total, free;
    delay_init(); //延时函数初始化
    uart_init(9600); //串口初始化为 9600
    exfuns_init(); //为 fatfs 相关变量申请内存
    LCD_Init(); //初始化液晶
    LED_Init(); //LED 初始化
    usmart_dev.init(72);
    mem_init(); //初始化内存池
    POINT_COLOR=RED; //设置字体为红色
    LCD_ShowString(60, 50, 200, 16, 16, "CPLD&STM32");
    LCD_ShowString(60, 70, 200, 16, 16, "FATFS TEST");
    LCD_ShowString(60, 90, 200, 16, 16, "ETree Board");
    LCD_ShowString(60, 110, 200, 16, 16, "2021/2/24");
    while(SD_Initialize()) //检测 SD 卡
    {
        LCD_ShowString(60, 150, 200, 16, 16, "SD Card Error!");
        delay_ms(200);
        LCD_Fill(60, 150, 240, 150+16, WHITE); delay_ms(200); //清除显示
    }
    LCD_ShowString(60, 130, 200, 16, 16, "=====");
    f_mount(fs[0], "0:", 1); //挂载 SD 卡

    while(exf_getfree("0", &total, &free)) //得到 SD 卡的总容量和剩余容量
    {
        LCD_ShowString(60, 150, 200, 16, 16, "Fatfs Error!"); delay_ms(200);
        LCD_Fill(60, 150, 240, 150+16, WHITE);
        delay_ms(200); //清除显示
    }
    POINT_COLOR=BLUE; //设置字体为蓝色
    LCD_ShowString(60, 150, 200, 16, 16, "FATFS OK!");
    LCD_ShowString(60, 170, 200, 16, 16, "SD Total Size: MB");
    LCD_ShowString(60, 190, 200, 16, 16, "SD Free Size: MB");
    LCD_ShowNum(172, 170, total>>10, 5, 16); //显示 SD 卡总容量 MB
    LCD_ShowNum(172, 190, free>>10, 5, 16); //显示 SD 卡剩余容量 MB
    while(1)
    {
        delay_ms(200);
        LED0=!LED0;
    }
}
```

18.5. 实验操作

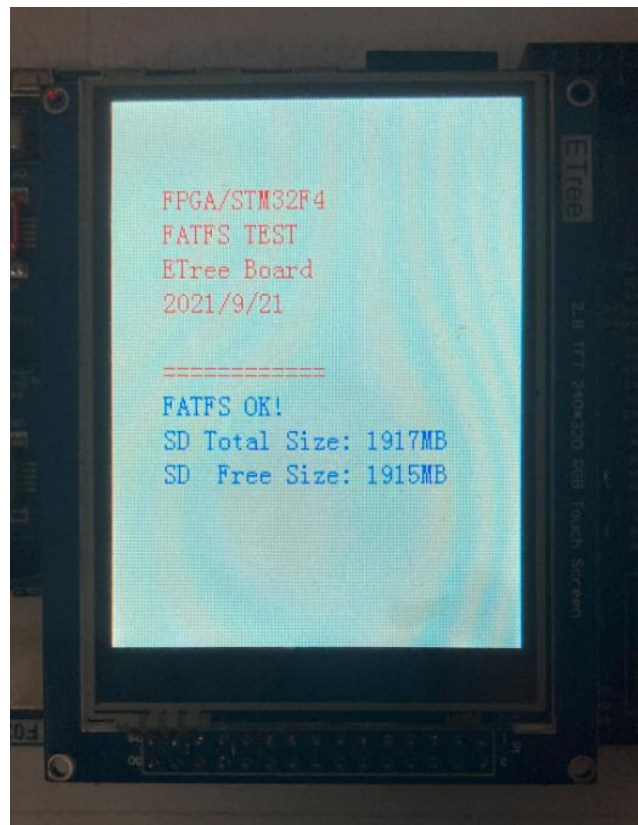
实验前, 用户需要自行准备一片 SD2.0 标准的格式化为 FAT32 文件系统的 Micro SD 卡, 本设计使用了一片 Kingston 的 2GB 的 Micro SD 卡, 如下图所示。



插入 TF-SD(Micro SD)卡, 设置好启动方式, 先下载 FPGA 的代码, 再下载 STM32 的代码。我们可以看到串口调试助手打印信息, 如下图所示:



同时液晶屏显示如下图所示:



第十九章 TF 卡图片显示

上一章节我们学习了如何使用 FATFS 文件系统，但没有涉及到文件的操作。本章我们基于 FAT32 文件系统读取 TF 卡中的图片显示在 2.8 寸液晶屏上。

19.1. 图片格式简介

本章介绍 BMP 和 JPEG 两种图片的格式。

BMP 是 Windows 下的标准图片格式，使用非常广泛。它采用位映射存储格式，除了图像深度不同，不采用其他任何压缩方式。BMP 图像按照从左到右、从上到下的顺序扫描。

典型的 BMP 图片由四部分组成：

- ◆ 位图头文件数据结构，包含 BMP 图像文件的类型、显示内容等
- ◆ 位图信息数据结构，包含 BMP 图像的宽、颜色定义等信息
- ◆ 调色板，可选项，一般真彩色(24bit)不需要调色板
- ◆ 位图数据，根据 BMP 位图的位数和大小而不同

JPEG 是 Joint Photographic Experts Group 的缩写，文件后缀名为.jpg 或.jpeg。与 BMP 图像不同的是 JPEG 图片是有损压缩格式，压缩比率一般在 10:1 到 40:1 之间，所以图像的大小一般比较小。

JPEG 的解码过程可以简述为下面几个部分：

- ◆ 从文件头读出文件的相关信息
- ◆ 从图像数据流读取一个最小编码单元 MCU，并提取出各个颜色分量单元
- ◆ 将颜色分量单元从数据流中恢复成矩阵数据
- ◆ 将 8×8 的数据矩阵进一步解码
- ◆ 色域空间 YCbCr 转到 RGB
- ◆ 排列整合各个 MCU 的解码数据

JPEG 的解码是比较复杂的，这里 FATFS 的作者提供了一个轻量级的 JPEG 解码库：TjpegDec，最少需要 3KB 的 RAM 和 3.5KB 的 ROM 即可实现 JPEG 解码，本章采用了这个解码库。

19.2. 硬件设计

本章使用的硬件和上一章一样，这个不再介绍。

19.3. HDL 设计

在前面“液晶屏显示”章节是单向操作液晶屏(即只往液晶屏中写指令或数据)，本章需要对液晶屏双向操作(即往液晶屏中写指令或读数据)。这里需要对“fsmc_reg”文件做一下改动，添加液晶屏接口和 FSMC 读取状态，具体设计代码如下：

```
//file name:      fsmc_reg.v
//author:         ETree
//date:           2021.9.1
//function:       fsmc register
//log:
```

```
module fsmc_reg(  
    input clk,  
    input rst_n,  
  
    //TFT2.8  
    output reg tft_cs,           //液晶屏片选，低电平有效  
    output reg tft_wr,  
    output reg tft_rs,  
    output reg tft_rd,  
    output reg tft_reset,       //液晶屏复位，低电平有效  
    inout  [15:0] tft_data,  
    output reg tft_blk,         //液晶屏背光，高电平有效  
    //TFT Touch  
    output tft_tclk,  
    output tft_tcs,  
    output tft_tmosi,  
    input  tft_tmiso,  
    input  tft_tpen,  
  
    //FSMC Bus  
    inout  [15:0] fsmc_data,  
    input  fsmc_noe,  
    input  fsmc_nwe,  
    input  fsmc_ne1,  
    input  fsmc_a23,  
    input  fsmc_a22,  
    input  fsmc_a21,  
    input  fsmc_a20,  
    input  fsmc_a19,  
    input  fsmc_a18,  
    input  fsmc_a17,  
    input  fsmc_a16,  
    input  fsmc_nwait,  
  
    output reg [15:0] fsmc_register0,  
    output reg fsmc_register0_valid,  
    output reg [15:0] fsmc_register1,  
    output reg fsmc_register1_valid,  
    output reg [15:0] fsmc_register2,  
    output reg fsmc_register2_valid,  
    output reg [15:0] fsmc_register3,  
    output reg fsmc_register3_valid,  
    output reg [15:0] fsmc_register4,
```

```
output reg fsmc_register4_valid,
    output reg [15:0] fsmc_register5,
output reg fsmc_register5_valid,
    output reg [15:0] fsmc_register6,
output reg fsmc_register6_valid,
    output reg [15:0] fsmc_register7,
output reg fsmc_register7_valid,

output reg fsmc_rd_valid,
input [15:0] fsmc_rddata,

output reg [31:0] fsmc_addr_o,
output reg [31:0] fsmc_len_o,
output reg      fsmc_load
);

reg nwe_r0;
reg nwe_r1;
reg nel_r0;
reg noe_r0;
reg noe_r1;

always @(posedge clk)
    if(~rst_n)
        begin
            nwe_r0 <= 0;
            nwe_r1 <= 0;
            nel_r0 <= 0;
            noe_r0 <= 0;
            noe_r1 <= 0;
        end
    else
        begin
            nwe_r0 <= fsmc_nwe;
            nwe_r1 <= nwe_r0;
            nel_r0 <= fsmc_nel;
            noe_r0 <= fsmc_noe;
            noe_r1 <= noe_r0;
        end
    end

wire nwe_neg;
assign nwe_neg = (nwe_r0==1'b0 && nwe_r1==1'b1);

wire noe_neg;
```

```
assign noe_neg = (noe_r0==1'b0 && noe_r1==1'b1);

parameter IDLE      = 0;
parameter FSMC_WR   = 1;
parameter FSMC_RD   = 2;
parameter F_END     = 3;

reg [1:0] fsmc_state;
reg [7:0] cnt_read;

always @(posedge clk)
    if(~rst_n)
        begin
            fsmc_state <= IDLE;
            fsmc_register0 <= 0;
            fsmc_register0_valid <= 0;
            fsmc_register1 <= 0;
            fsmc_register1_valid <= 0;
            fsmc_register2 <= 0;
            fsmc_register2_valid <= 0;
            fsmc_register3 <= 0;
            fsmc_register3_valid <= 0;
            fsmc_register4 <= 0;
            fsmc_register4_valid <= 0;
            fsmc_register5 <= 0;
            fsmc_register5_valid <= 0;
            fsmc_register6 <= 0;
            fsmc_register6_valid <= 0;
            fsmc_register7 <= 0;
            fsmc_register7_valid <= 0;
            fsmc_load <= 0;
            cnt_read <= 0;
        end
    else
        begin
            case(fsmc_state)
            IDLE:
                begin
                    fsmc_register0_valid <= 0;
                    fsmc_register1_valid <= 0;
                    fsmc_register2_valid <= 0;
                    fsmc_register3_valid <= 0;
                    fsmc_register4_valid <= 0;
                    fsmc_register5_valid <= 0;
```

```
fsmc_register6_valid <= 0;
fsmc_register7_valid <= 0;
cnt_read             <= 0;
if(fsmc_a19==0)
begin
    if(ne1_r0==1'b0 && nwe_neg==1'b1)
    begin
        fsmc_state <= FSMC_WR;
        fsmc_rd_valid <= 1'b0;
        fsmc_load <= 1'b0;
    end
    else if(ne1_r0==1'b0 && noe_neg==1'b1)
    begin
        fsmc_state <= FSMC_RD;
        fsmc_rd_valid <= 1'b0;
        fsmc_load <= 1'b1;
    end
    else
    begin
        fsmc_state <= IDLE;
        fsmc_rd_valid <= 1'b0;
        fsmc_load <= 1'b0;
    end
end
end
else
begin
    fsmc_state <= IDLE;
end
end
FSMC_WR:
begin
    case({fsmc_a18,fsmc_a17,fsmc_a16})
    3'b000:
    begin
        fsmc_register0 <= fsmc_data;
        fsmc_register0_valid <= 1'b1;
    end
    3'b001:
    begin
        fsmc_register1 <= fsmc_data;
        fsmc_register1_valid <= 1'b1;
        fsmc_addr_o[15:0] <= fsmc_data;
    end
end
```

```
3'b010:
begin
    fsmc_register2 <= fsmc_data;
    fsmc_register2_valid <= 1'b1;
    fsmc_addr_o[31:16] <= fsmc_data;
    fsmc_load <= 1'b0;
end
3'b011:
begin
    fsmc_register3 <= fsmc_data;
    fsmc_register3_valid <= 1'b1;
    fsmc_load <= 1'b0;
end
3'b100:
begin
    fsmc_register4 <= fsmc_data;
    fsmc_register4_valid <= 1'b1;
end
3'b101:
begin
    fsmc_register5 <= fsmc_data;
    fsmc_register5_valid <= 1'b1;
    fsmc_load <= 0;
end
3'b110:
begin
    fsmc_register6 <= fsmc_data;
    fsmc_register6_valid <= 1'b1;
    fsmc_load <= 0;
end
3'h111:
begin
    fsmc_register7 <= fsmc_data;
    fsmc_register7_valid <= 1'b1;
    fsmc_load <= 0;
end
default:;
endcase
fsmc_state <= F_END;
end
FSMC_RD:
begin
    if(cnt_read>='hff)
        cnt_read <= cnt_read;
```

```

        else
            cnt_read <= cnt_read + 1'b1;

            if(noe_r1==1'b1)
            begin
                fsmc_state <= F_END;
                fsmc_rd_valid <= 1'b0;
                fsmc_load <= 1'b0;
            end
        else
        begin
            fsmc_state <= FSMC_RD;
            if(cnt_read=='h13)
                fsmc_rd_valid <= 1'b1;
            else
                fsmc_rd_valid <= 1'b0;
            fsmc_load <= 1'b0;
        end
    end
F_END:
begin
    fsmc_state <= IDLE;
    fsmc_register0_valid <= 0;
    fsmc_register1_valid <= 0;
    fsmc_register2_valid <= 0;
    fsmc_register3_valid <= 0;
    fsmc_register4_valid <= 0;
    fsmc_register5_valid <= 0;
    fsmc_register6_valid <= 0;
    fsmc_register7_valid <= 0;
    fsmc_load          <= 0;
    fsmc_rd_valid      <= 0;
    cnt_read           <= 0;

end
default:
begin

end
endcase
end

assign fsmc_data =
(fsmc_a19==1)?((fsmc_noe==1'b0)?tft_data:'hzzzz):((fsmc_state==FSMC_R
D)?fsmc_rddata:'hzzzz);

```



```

assign tft_data =
(fsmc_a19==1)?((fsmc_noe==1'b0)?'hzzzz:fsmc_data):('hzzzz);

always @(*)
begin
    tft_cs <= fsmc_nel;
    tft_wr <= fsmc_nwe;
    tft_rs <= fsmc_a16;
    tft_rd <= fsmc_noe;
    tft_reset <= rst_n;
    tft_blk <= 1'b1;
end

endmodule

```

19.4. 软件设计

bmp.c 和 bmp.h 实现对 BMP 图片的解码, tjpgd.c 和 tjpgd.h 实现对 JPEG 图片的解码, 具体代码可以查看工程源码。

下面为 piclib.c 的源码:

```

#include "piclib.h"
#include "lcd.h"

_pic_info picinfo; //图片信息
_pic_phy pic_phy; //图片显示物理接口
//LCD 驱动部分, 没有提供划横线函数, 需要自己实现
void piclib_draw_hline(u16 x0,u16 y0,u16 len,u16 color)
{
    if((len==0)|| (x0>lcddev.width)|| (y0>lcddev.height))return;
    LCD_Fill(x0,y0,x0+len-1,y0,color);
}
//填充颜色
//x,y:起始坐标
//width, height: 宽度和高度。
//*color: 颜色数组
void piclib_fill_color(u16 x,u16 y,u16 width,u16 height,u16 *color)
{
    LCD_Color_Fill(x,y,x+width-1,y+height-1,color);
}
//画图初始化, 在画图之前, 必须先调用此函数
//指定画点/读点
void piclib_init(void)
{
    pic_phy.read_point=LCD_ReadPoint; //读点函数实现

```

```

pic_phy.draw_point=LCD_Fast_DrawPoint; //画点函数实现
pic_phy.fill=LCD_Fill; //填充函数实现
pic_phy.draw_hline=piclib_draw_hline; //画线函数实现
pic_phy.fillcolor=piclib_fill_color; //颜色填充函数实现
picinfo.lcdwidth=lcddev.width; //得到 LCD 的宽度像素
picinfo.lcdheight=lcddev.height; //得到 LCD 的高度像素
picinfo.ImgWidth=0; //初始化宽度为 0
picinfo.ImgHeight=0; //初始化高度为 0
picinfo.Div_Fac=0; //初始化缩放系数为 0
picinfo.S_Height=0; //初始化设定的高度为 0
picinfo.S_Width=0; //初始化设定的宽度为 0
picinfo.S_XOFF=0; //初始化 x 轴的偏移量为 0
picinfo.S_YOFF=0; //初始化 y 轴的偏移量为 0
picinfo.staticx=0; //初始化当前显示到的 x 坐标为 0
picinfo.staticy=0; //初始化当前显示到的 y 坐标为 0
}
//快速 ALPHA BLENDING 算法.
//src:源颜色
//dst:目标颜色
//alpha:透明程度(0~32)
//返回值:混合后的颜色.
u16 piclib_alpha_blend(u16 src,u16 dst,u8 alpha)
{
    u32 src2; u32 dst2;
    //Convert to 32bit |-----GGGGGG-----RRRRR-----BBBBB|
    src2=((src<<16)|src)&0x07E0F81F;
    dst2=((dst<<16)|dst)&0x07E0F81F;
    //Perform blending R:G:B with alpha in range 0..32
    //Note that the reason that alpha may not exceed 32 is that there are
only
    //5bits of space between each R:G:B value, any higher value will
overflow
    //into the next component and deliver ugly result.
    dst2=((((dst2-src2)*alpha)>>5)+src2)&0x07E0F81F;
    return (dst2>>16)|dst2;
}
//初始化智能画点
//内部调用
void ai_draw_init(void)
{
    float temp,temp1;
    temp=(float)picinfo.S_Width/picinfo.ImgWidth;
    temp1=(float)picinfo.S_Height/picinfo.ImgHeight;
    if(temp<temp1) temp1=temp; //取较小的那个

```

```

    if(temp1>1) temp1=1;
    //使图片处于所给区域的中间
    picinfo.S_XOFF+=(picinfo.S_Width-temp1*picinfo.ImgWidth)/2;
    picinfo.S_YOFF+=(picinfo.S_Height-temp1*picinfo.ImgHeight)/2;
    temp1*=8192;//扩大 8192 倍
    picinfo.Div_Fac=temp1;
    picinfo.staticx=0xffff;
    picinfo.staticy=0xffff;//放到一个不可能的值上面
}
//判断这个像素是否可以显示
// (x,y) :像素原始坐标
//chg :功能变量.
//返回值:0,不需要显示.1,需要显示
u8 is_element_ok(u16 x,u16 y,u8 chg)
{
    if(x!=picinfo.staticx||y!=picinfo.staticy)
    {
        if(chg==1)
        {
            picinfo.staticx=x;
            picinfo.staticy=y;
        }
        return 1;
    }else return 0;
}
//智能画图
//FileName:要显示的图片文件 BMP/JPG/JPEG/GIF
//x,y,width,height:坐标及显示区域尺寸
//fast:使能 jpeg/jpg 小图片(图片尺寸小于等于液晶分辨率)快速解码,0,不使能;1,使能.
//图片在开始和结束的坐标点范围内显示
u8 ai_load_picfile(const u8 *filename,u16 x,u16 y,u16 width,u16 height,u8 fast)
{
    u8 res;//返回值
    u8 temp;
    if((x+width)>picinfo.lcdwidth)return PIC_WINDOW_ERR; //x 坐标超范围了.
    if((y+height)>picinfo.lcdheight)return PIC_WINDOW_ERR; //y 坐标超范围了.
    //得到显示方框大小
    if(width==0||height==0)return PIC_WINDOW_ERR; //窗口设定错误
    picinfo.S_Height=height;
    picinfo.S_Width=width;

```

```

//显示区域无效
if(picinfo.S_Height==0||picinfo.S_Width==0)
{
    picinfo.S_Height=lcddev.height;
    picinfo.S_Width=lcddev.width;
    return FALSE;
}
if(pic_phy.fillcolor==NULL) fast=0;//颜色填充函数未实现,不能快速显示
//显示的开始坐标点
picinfo.S_YOFF=y;
picinfo.S_XOFF=x;
//文件名传递
temp=f_typedell((u8*)filename); //得到文件的类型
switch(temp)
{
    case T_BMP:
        res=stdbmp_decode(filename); //解码 bmp
        break;
    case T_JPG:
    case T_JPEG:
        res=jpg_decode(filename,fast); //解码 JPG/JPEG
        break;
    case T_GIF:
        res=gif_decode(filename,x,y,width,height); //解码 gif
        break;
    default:
        res=PIC_FORMAT_ERR; //非图片格式!!!
        break;
}
return res;
}

```

这段代码有 7 个函数，其中 `piclib_draw_hline` 和 `piclib_fill_color` 函数用于驱动 LCD 液晶屏。

`piclib_init` 函数用于图片解码的初始化，其中 `_pic_phy` 结构体用于管理底层 LCD 接口，`_pic_info` 结构体用于图片缩放处理。

`piclib_alpha_blend` 函数用于实现半透明效果，在 `bmp` 解码中可能被用到。

`ai_draw_init` 函数用于实现图片在显示区域的初始化，根据图片大小选择缩放比例和坐标偏移。

`ls_element_ok` 函数用于判断一个点是不是该显示，用在图片缩放。

`Ai_load_picfile` 函数是整个图片显示的对外接口，外部程序通过调用该函数实现图片的显示，用到 `f_typedell` 函数来判断文件的后缀名。

具体 `main` 函数，可查看工程代码，这里不再贴出。

19.5. 实验操作

实验前，用户需要自行准备一片 SD2.0 标准的格式化为 FAT32 文件系统的 Micro SD 卡，本设计使用了一片 Kingston 的 2GB 的 Micro SD 卡，如下图所示。



用读卡器将“测试图片”目录下的 PICTURE 文件夹拷贝到 SD/TF 卡的根目录下，如下所示：



插入 TF-SD(Micro SD)卡，设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。同时液晶屏显示如下图所示：



第二十章 摄像头液晶显示

基于 FAT32 文件系统读取 TF 卡中的图片显示在 2.8 寸液晶屏上, 这章我们基于 STM32F4 的专用摄像头接口 DCMI, 来实现 MT9V011 摄像头视频数据实时显示在 2.8 寸液晶屏上。

20.1. DCMI 简介

STM32F4 自带一个 DCMI(数字摄像头)接口, 该接口为同步并行接口, 具有如下特点:

- 8 位、10 位、12 位或 14 位并行接口
- 内嵌码/外部行同步和帧同步
- 连续模式或快照模式
- 裁剪功能
- 支持以下数据格式:

8/10/12/14 位逐行视频: 单色或原始拜尔 (Bayer) 格式

YCbCr 4:2:2 逐行视频

RGB 565 逐行视频

压缩数据: JPEG

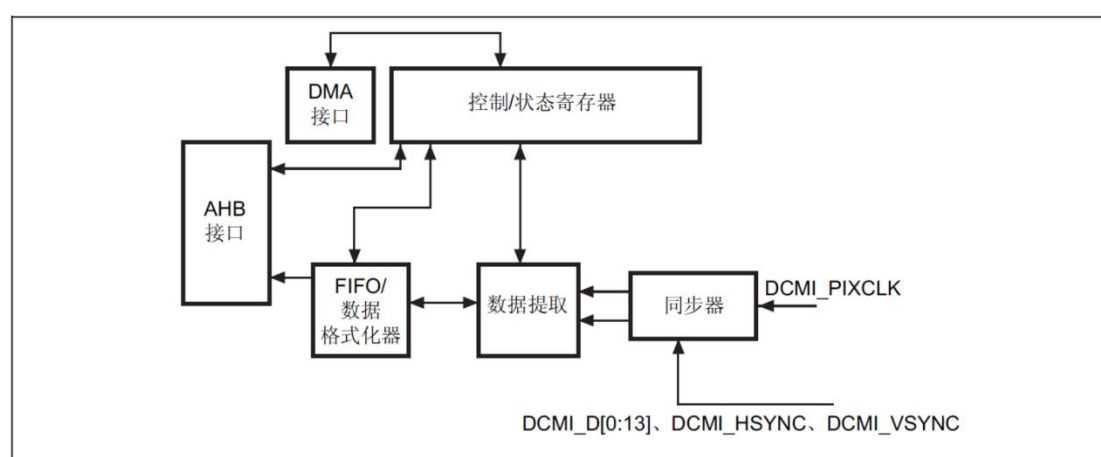
DCMI 接口包括下面这些信号:

- 数据输入 (D[0:13]), 用于接摄像头的的数据输出, 接 MT9V011 只用了 8 位数据。
- 水平同步 (行同步) 输入 (HSYNC), 用于接摄像头的 HSYNC/HREF 信号。
- 垂直同步 (场同步) 输入 (VSYNC), 用于接摄像头的 VSYNC 信号。
- 像素时钟输入 (PIXCLK), 用于接摄像头的 PCLK 信号, 最高可达 54MB/s。

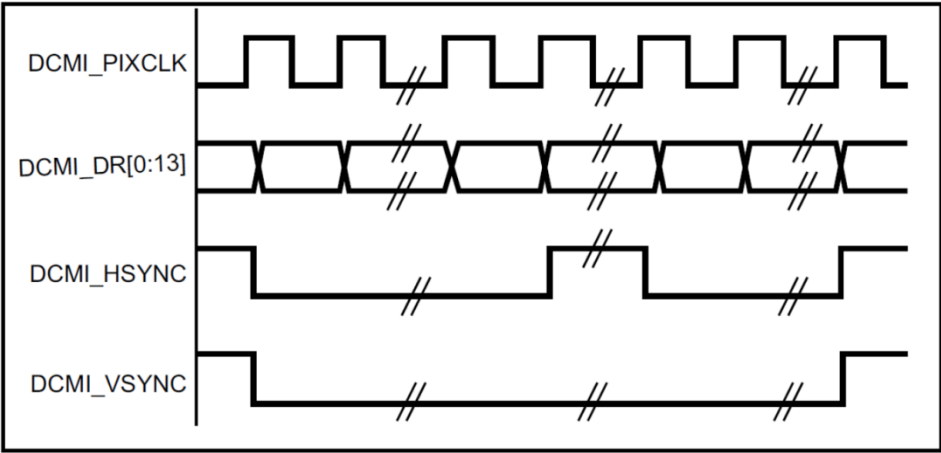
DCMI 接收到摄像头数据被放到 32 位数据寄存器(DCMI_DR)中, 然后通过通用 DMA 进行传输。图像缓冲区由 DMA 管理, 而不是由摄像头接口管理。

数据流可由可选的 HSYNC (水平同步) 信号和 VSYNC (垂直同步) 信号同步, 或通过数据流中嵌入的同步码同步。

DCMI 接口框图下图所示:



DCMI 接口数据与 PIXCLK 保持同步, HSYNC 信号表示行开始/结束, VSYNC 信号表示帧开始/结束, 具体时序如下图所示:



MT9V011 摄像头模块数据位宽为 8 位，需要通过设置 DCMI_CR 中的 EDM[1:0]=00，此时 DCMI_D0~D7 有效。捕获数据的第一个字节放在 32 数据的低字节，如下表所示：

字节地址	31:24	23:16	15:8	7:0
0	D _{n+3} [7:0]	D _{n+2} [7:0]	D _{n+1} [7:0]	D _n [7:0]
4	D _{n+7} [7:0]	D _{n+6} [7:0]	D _{n+5} [7:0]	D _{n+4} [7:0]

DCMI 接口支持 DMA 传输，当 DCMI_CR 寄存器中的 CAPTURE 位设置为 1 时激活 DMA 接口。DCMI 接口每次接受到一个完整的 32 位数据都会触发一次 DMA 请求。

数据传输与 VSYNC 信号同步，没帧结束时都将激活 VSIF(垂直同步中断标志)，可以通过这个帧中断来判断一帧数据是否采集完成。

DCMI 接口支持快照模式和连续采集模式，一般采用连续采集模式。由于 2.8 寸液晶屏不需要地址信号，只要往其写入数据地址自动增 1，可通过 DMCI 的 DMA 连续采集到一个地址，不用地址自动递增。

关于更多的 DCMI 资料请参考《STM32F4xx 中文参考手册》。
下面我们来用库函数实现 DCMI 驱动摄像头的步骤：

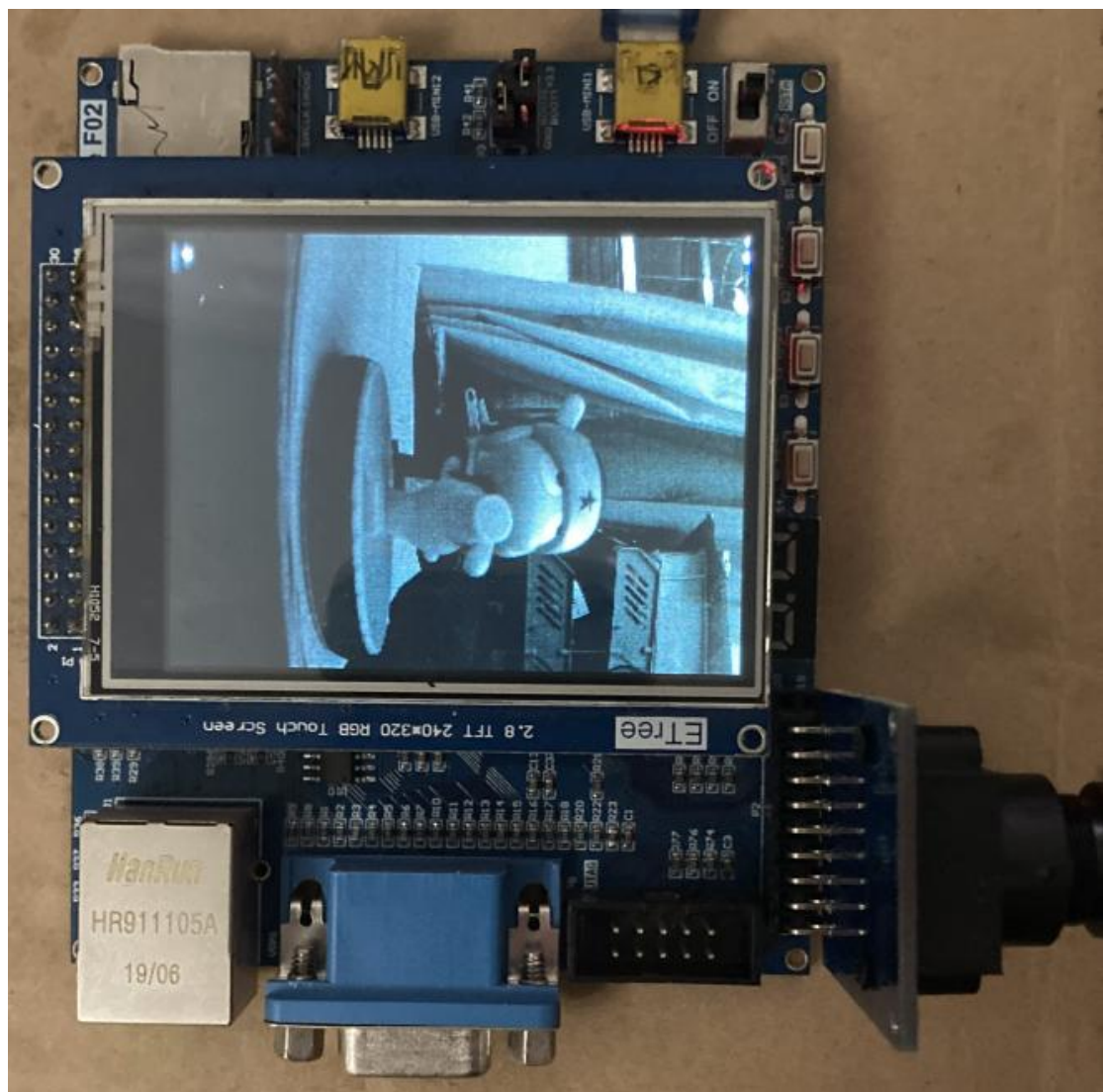
- 1) 配置相关引脚模式和复用功能(AF13)，使能时钟。
- 2) 配置 DCMI 相关设置
- 3) 配置 DMA
- 4) 使能 DCMI 捕获。

20.2. 硬件设计

20.3. HDL 设计

20.4. 软件设计

20.5. 实验操作

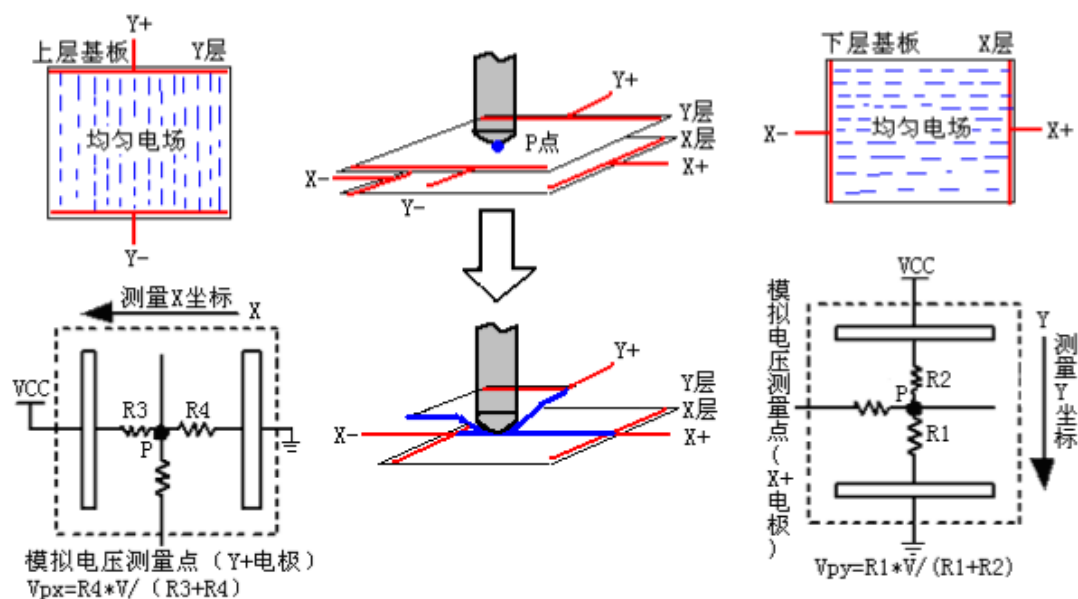


第二十一章 触控屏

前面章节的 2.8 寸液晶屏都是显示作用，本章介绍如何使用 2.8 寸液晶屏的触摸功能。

21.1. 电阻屏简介

电阻触摸屏上有一块与显示器表面配合的电阻薄膜层，当手指触摸屏幕时，触摸点位置电阻发生变化，在 X 和 Y 轴方向产生电压信号，然后送给触摸屏控制器，控制器计算出(X,Y)位置，再根据这个位置模拟鼠标的方式工作。

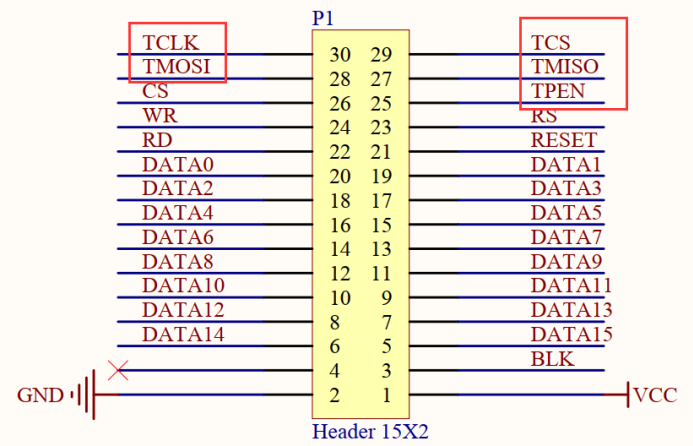
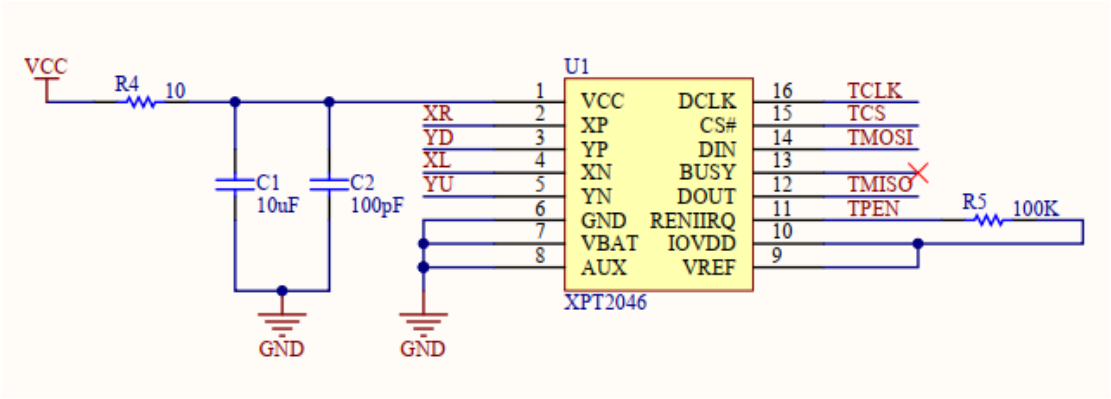


电阻触摸屏的优点是精度高、价格便宜、抗干扰能力强、稳定性，缺点是容易被划伤、透光不好、不支持多点触摸。

触摸屏控制芯片为 XPT2046，是一款 4 导线制触摸屏控制器，内含 12 位分辨率 125KHz 转换速率逐步逼近型 A/D 转换器。XPT2046 支持从 1.5V 到 5.25V 的低电压的 I/O 接口，通过执行两次 A/D 转换检测出被按的屏幕位置，还可以测量加在触摸屏上的压力。关于芯片的详细使用可参考芯片手册。

21.2. 硬件设计

下图为触摸屏的电路原理图：



2.8 寸液晶屏插在 FPGA 端的 P1 接插件上，由于 FSMC 的高地址位 A20/A21/A22/A23 和 NBL0 和 NBL1 没有使用，这里我们把液晶屏的 5 个触摸引脚映射到这些引脚即可。

0/PE0	97	FSMC NBL0	D15
1/PE1	98	FSMC NBL1	D16
3/PE2	1	FSMC A23	F15
9/PE3	2	FSMC A19	F16
0/PE4	3	FSMC A20	G15
1/PE5	4	FSMC A21	G16
2/PE6	5	FSMC A22	J16

21.3. HDL 设计

具体映射代码如下：

```
//-----Touch-----
assign tft_tclk = fsmc_a21;
assign tft_tcs = fsmc_a22;
assign tft_tmosi = fsmc_a20;
assign fsmc_nbl0 = tft_tpen;
assign fsmc_nbl1 = tft_tmiso;
```

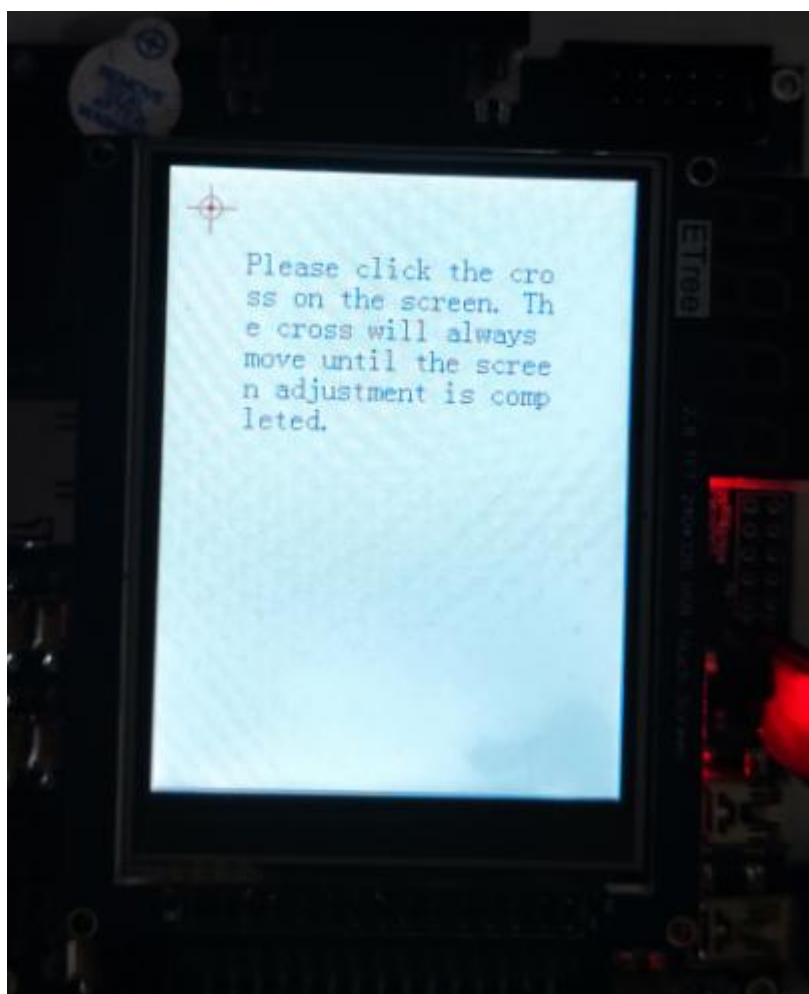
21.4. 软件设计

触控芯片的对应引脚如下所示：

```
#define PEN PEin(0) //PE0 INT
#define DOUT PEin(1) //PE1 MISO
#define TDIN POut(4) //PE4 MOSI
#define TCLK POut(5) //PE5 SCLK
#define TCS POut(6) //PE6 CS
```

21.5. 实验操作

插上 2.8 寸触控屏，设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。液晶屏弹出下面提示，依次点击(最好用触控笔)四个角上的十字坐标。



完成校准后，可以在屏幕上画图或者写字了，按下右上角的 RST 后会清除屏幕，如下图所示：

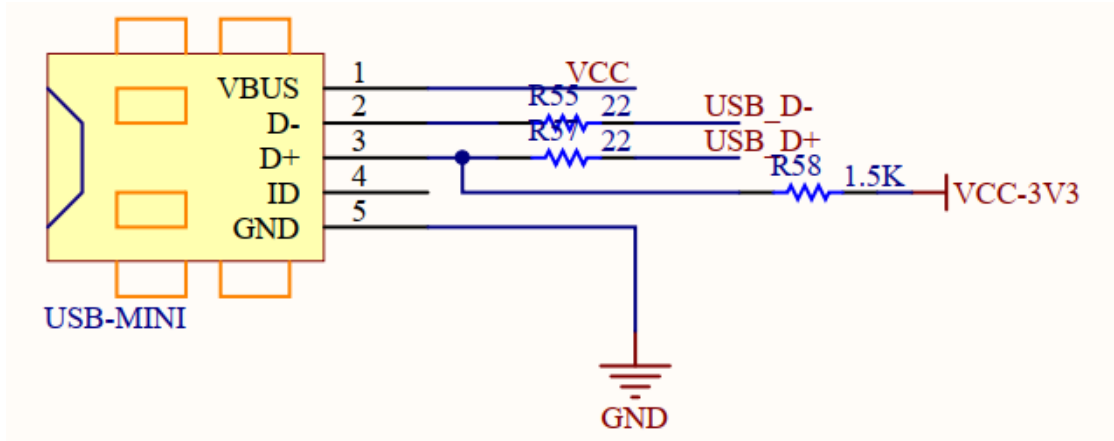


第二十二章 USB 读卡器

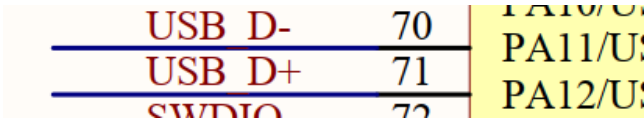
本章通过通用 STM32F4 的 USB2.0 接口实现一个 USB 读卡器。

19.1. 硬件设计

本章用到 SD 卡和 USB2.0 接口，USB2.0 接口电路如下图所示：



USB2.0 接口连接到 STM32F4 的 PA11 和 PA12 引脚，如下图所示：



19.2. USB

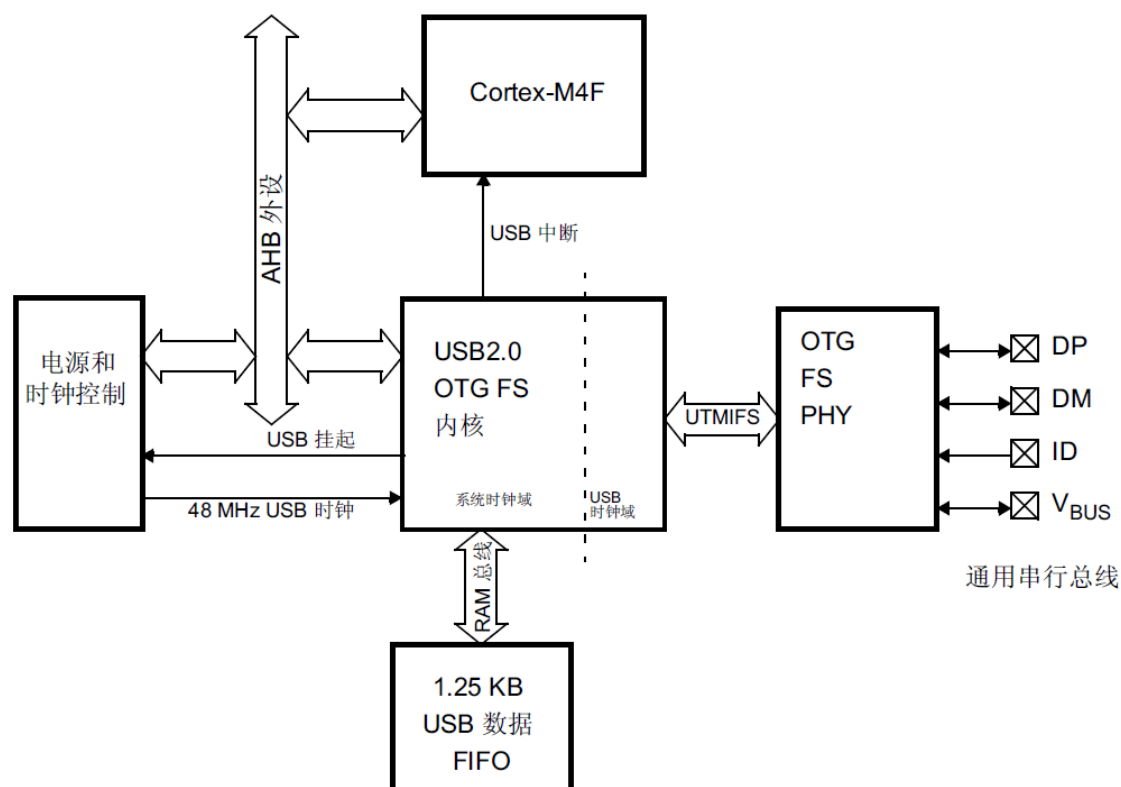
USB(Universal Serial Bus)通用串行总线，广泛应用于电脑及其他电子设备中。USB 发展到现在有 USB1.0/1.1/2.0/3.0/3.1 多个版本，STM32F4 自带的 USB 是 USB1.1 标准。USB1.1 标准共有四根线：VCC、GND、D+和 D-。在 USB 主机上，D-和 D+都是接了 15K 的下拉电阻，在没有设备插入时，D+和 D-都是低电平。在 USB 设备中，如果是高速设备，则在 D+上接一个 10K 的电阻到 VCC，如果是低速设备，则在 D-上接一个 10K 电阻到 VCC。

STM32F407 系列芯片自带 USB OTG FS（全速）和 USB OTG HS（高速），其中 HS 需外扩高速 PHY 芯片实现，这里不做介绍。

STM32F407 的 USB OTG FS 主要特性可分为三类：

- ✧ 通用特性：经 USB-IF 认证，符合通用串行总线规范第 2.0 版，集成全速 PHY，且完全支持标准规范的 OTG 补充第 1.3 版中的 OTG 协议等功能。
- ✧ 主机 HOST 模式特性：通过外部电荷泵生成 VBUS 电压，多大 8 个主机通道，管理共享 RX FIFO、一个周期性 TX FIFO 和一个非周期性 TX FIFO 等功能。
- ✧ 从机 SLAVE 模式特性：1 个双向控制端点 0、3 个 IN 端点 EP，3 个 OUT 端点 EP，管理一个共享 Rx FIFO 和一个 Tx-OUT FIFO，以高效使用 USB 数据 RAM 等功能。

STM32F407 USB OTG FS 框图如下图所示：



ST 提供的 USB OTG 库，可以在：

<http://www.stmcu.org/download/index.php?act=ziliao&id=150> 下载。资料“学习资料/STM32 USB 学习资料”目录下有，文件名：stm32_f105-07_f2_f4_usb-host-device_lib.zip。该库包含了 STM32F4 USB 主机（Host）和从机（Device）驱动库，并提供了 10 个例程供我们参考，

19.3. HDL 设计

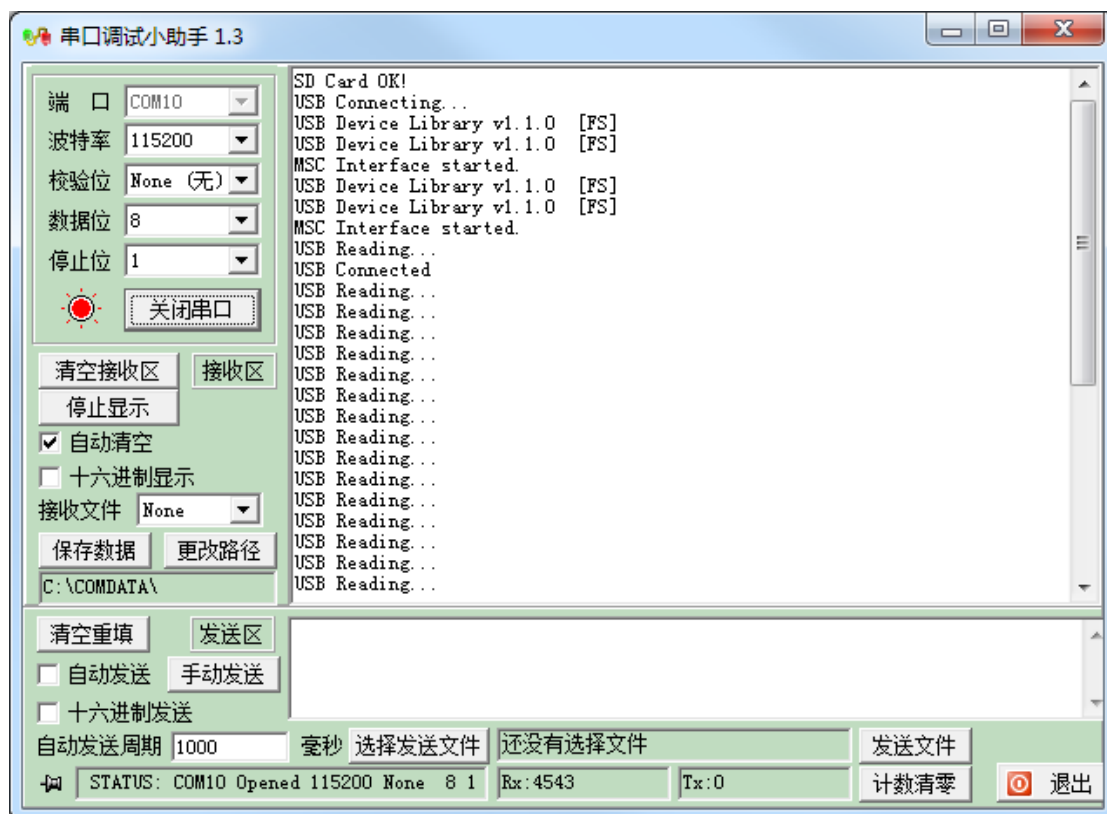
本章 FPGA 代码和上一章一样，这里不再贴出。

19.4. 软件设计

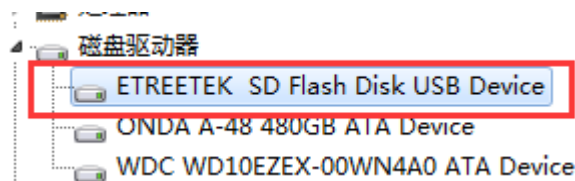
本章在上一章基础上进行修改，代码移植于 ST 官方例程：STM32_USBHost-Device_Lib_V2.1.0\Project\USB_Device_Examples\MSC。

19.5. 实验操作

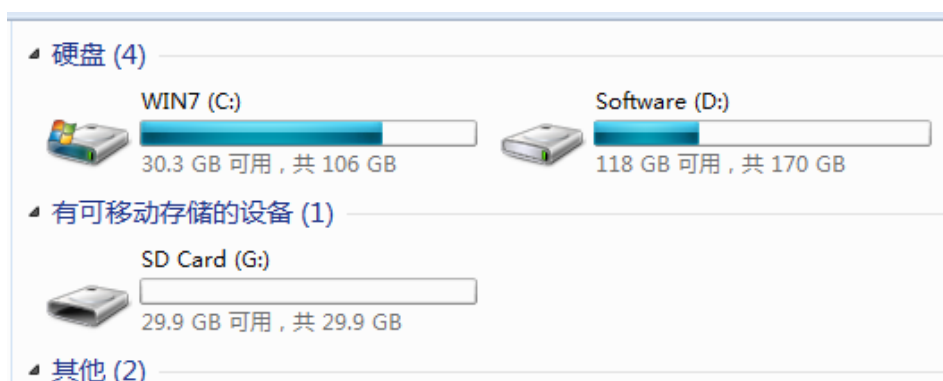
插入 TF-SD(Micro SD)卡、USB 接口和 USB 串口，设置好启动方式，先下载 FPGA 的代码，再下载 STM32 的代码。我们可以看到串口调试助手打印信息，如下图所示：



在设备管理中可以看到我们的 USB 读卡器设备，如下图所示：



同时在“计算机”喜爱可以看到“有可移动存储设备”，这样我们可以正常使用 USB 读卡器了，如下图所示：



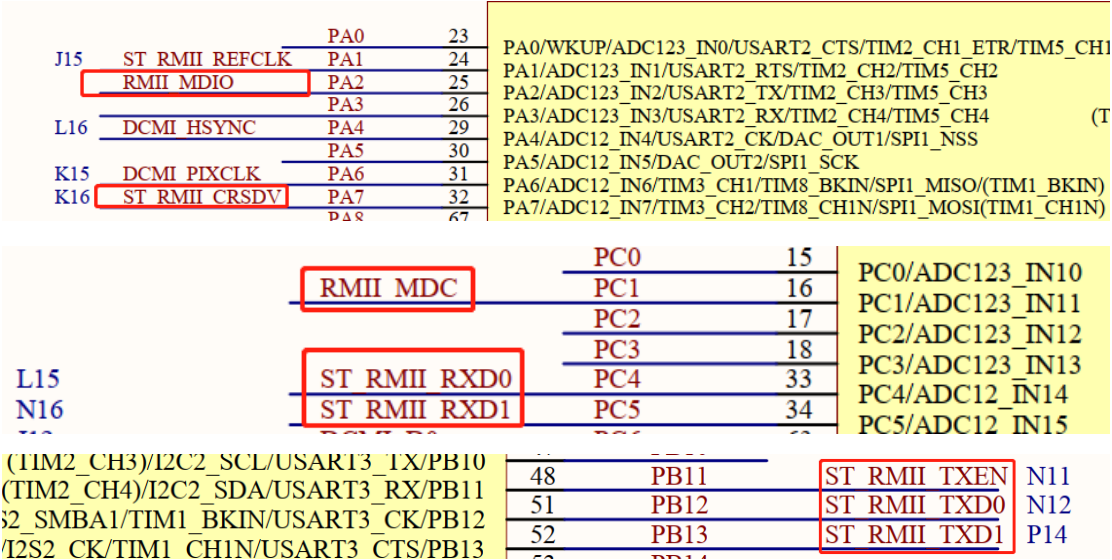
第二十三章 LWIP 移植

本章通过通用 STM32F4 的以太网接口实现 LWIP 的移植，最后通过 PING 命令验证移植的结果。

23.1. STM32F4 以太网简介

23.2. 硬件设计

STM32F4 的以太网接口(RMII)直接连接到了 FPGA 的 IO 口,MDIO 和 MDC 直接连接在以太网 PHY 的 MDIO 和 MDC 引脚上，如下图所示：旁边蓝色的字符(如 J15/L16...表示和 FPGA 的哪个 IO 相连)。



23.3. HDL 设计

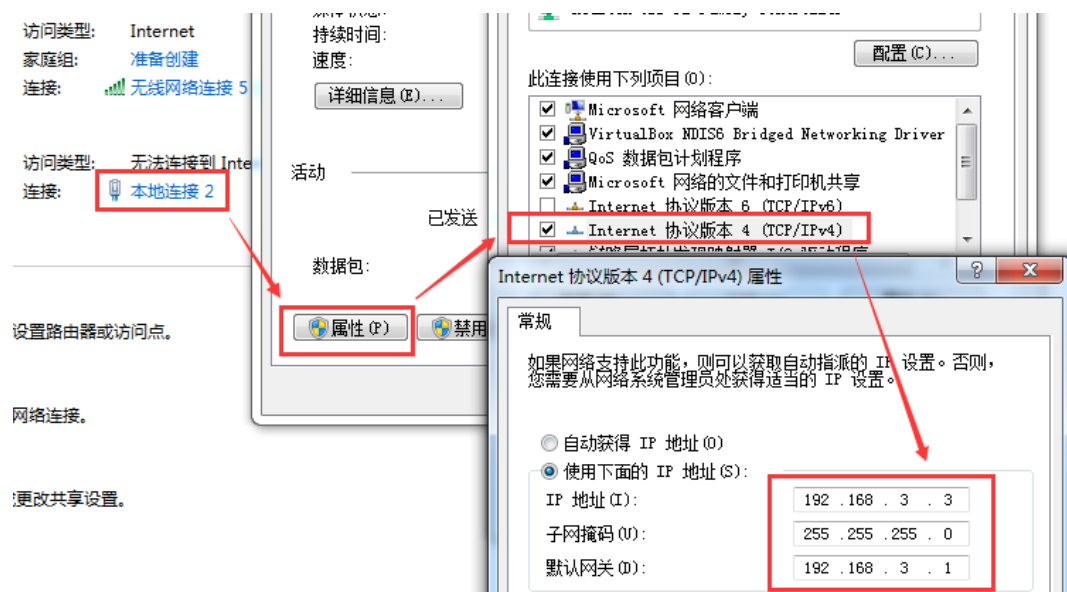
这里只需要将 FPGA 端对应的以太网 PHY 的 MII 接口映射到 STM32F4 的 MII 端口即可，具体代码如下：

```
assign e_xi = clk_25m;
assign e_txen = stm_rmii_txen;
assign e_tx[0] = stm_rmii_txd0;
assign e_tx[1] = stm_rmii_txd1;
assign stm_rmii_crsdv = e_rxdv;
assign stm_rmii_rxd0 = e_rx[0];
assign stm_rmii_rxd1 = e_rx[1];
assign stm_rmii_refclk = e_rxclk;
```

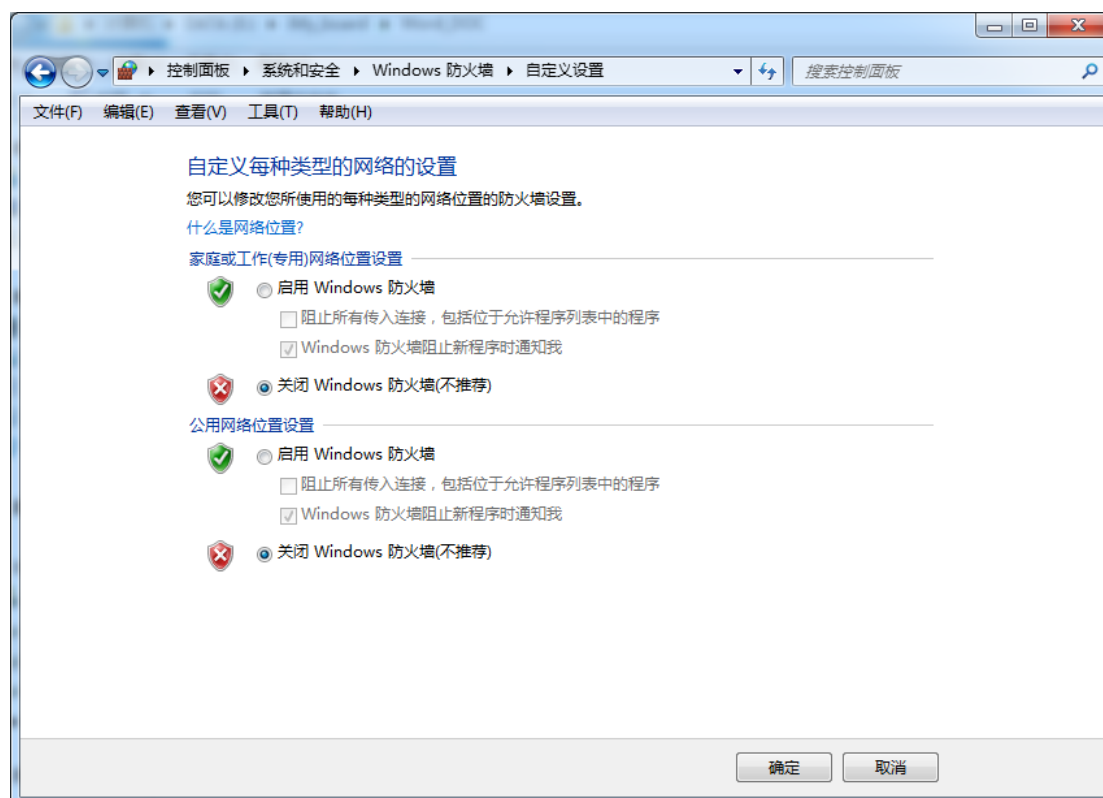
23.4. 软件设计

23.5. 实验操作

将本地连接的 IP 地址设置成 192.168.3.3，子网掩码设置成 255.255.255.0，网关设置成 192.168.3.1，具体如下图所示：



关闭防火墙，否则电脑无法接受视频数据。



插上网线, 设置好启动方式, 先下载 FPGA 的代码, 再下载 STM32 的代码可以看到串口调试助手打印信息, 如下图所示。



打开 WINDOWS 的 DOS 界面, 输入 “ping 192.168.3.30” 命令, 通信正常, 如下图所示:



```
管理员: C:\Windows\system32\cmd.exe
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>ping 192.168.3.30

正在 Ping 192.168.3.30 具有 32 字节的数据:
来自 192.168.3.30 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.3.30 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.3.30 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.3.30 的回复: 字节=32 时间<1ms TTL=255

192.168.3.30 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Users\Administrator>
```

第二十四章 UDP 通信

本章通过通用 STM32F4 的以太网接口实现 LWIP 的 UDP。

24.1. 硬件设计

本章使用的硬件和上一章一样，这个不再介绍。

24.2. HDL 设计

本章 FPGA 代码和之前章节一样，这里不做介绍。

24.3. 软件设计

24.4. 实验操作

END

